
Python Data Science Program — currículo completo

232 clases · 9 partes · v3.7.0

Python Data Science Program — currículo completo

232 clases · 9 partes · bundle unificado del currículo v3.

Índice general

- Parte 0 — Parte 0 — Prerrequisitos: Python + NumPy + pandas + visualización + SQL + APIs (49 clases)
- Parte 1 — Parte 1 — Machine Learning Clásico (50 clases)
- Parte 2 — Parte 2 — Deep Learning — Keras, TensorFlow, Transformers, RL y Despliegue (75 clases)
- Parte 3 — Parte 3 — Estadística Inferencial y Causal (19 clases)
- Parte 4 — Parte 4 — MLOps — Modelos en Producción (14 clases)
- Parte 5 — Parte 5 — Ingeniería de Datos (8 clases)
- Parte 6 — Parte 6 — Sistemas de Recomendación (7 clases)
- Parte 7 — Parte 7 — Ética, Fairness y Privacidad (6 clases)
- Parte 8 — Parte 8 — Capstones — Proyectos Integradores (4 clases)

Parte 0 — Parte 0 — Prerrequisitos: Python + NumPy + pandas + visualización + SQL + APIs

49 clases en esta parte.

Clase 001 — Clase 001 — Instalación de Python 3.12+ y entornos virtuales (venv, uv, conda)

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, Python Data Science Handbook, prefacio "Installation Considerations".

Duración estimada: 90 min (45 lectura + 45 práctica).

Objetivo

Que el alumno deje su máquina lista para trabajar en data science: con Python 3.12+ instalado correctamente, con al menos dos gestores de entornos virtuales funcionando (venv nativo + uv o conda), y con la disciplina de nunca instalar paquetes en el Python del sistema. Al final de la clase, deberá poder crear un entorno limpio, activarlo, instalar dependencias declaradas, y reproducirlo exactamente en otra máquina.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Verificar qué versión de Python tiene su máquina y desde qué ruta se ejecuta (python -V, which python / where python, sys.executable).
2. Crear, activar y destruir entornos virtuales con venv, uv venv y conda create — y explicar cuándo conviene cada uno.
3. Instalar dependencias desde requirements.txt y desde pyproject.toml, y congelar versiones

reproducibles (pip freeze, uv pip compile, conda env export).

4. Diagnosticar el error más frecuente del principiante: "instalé un paquete pero el import falla" (causa: pip instaló en un Python distinto del que ejecuta el notebook).
5. Justificar por qué un entorno virtual por proyecto es no-negociable en data science (reproducibilidad, conflictos de versiones, aislamiento de experimentos).

Prerrequisitos

- Acceso a una terminal (PowerShell en Windows, Terminal en macOS/Linux).
- Permisos para instalar software en la máquina propia.
- Conocimiento previo: ninguno. Esta es la primera clase del programa.

Temas

#	Tema	Por qué importa
1	Qué es Python "del sistema" y por qué no t	Romperlo deja tu OS inestable (macOS/Linux
2	Instaladores oficiales vs. pyenv / mise	Cómo tener varias versiones de Python conv
3	venv (stdlib)	El más simple, viene incluido — el "default
4	uv (Astral)	Rápido (10–100× pip), reemplazo moderno de
5	conda / mamba	Necesario cuando hay dependencias C/CUDA (
6	requirements.txt vs pyproject.toml vs envi	Tres formatos, tres ecosistemas — cuándo u
7	El bug más común: "pip install funciona pe	Diagnóstico con sys.executable y pip -V.

Dataset / recursos

No requiere dataset. Es una clase de setup. Los únicos archivos generados son los propios entornos virtuales (que se ignoran en git vía .gitignore).

Recursos externos:

- Python.org downloads — instalador oficial.
- uv docs — gestor moderno de Astral.
- Miniconda — instalación mínima de conda (evita Anaconda completo, pesa 3 GB).

Ejercicios

1. Diagnóstico inicial. Abre una terminal y reporta: versión de Python (python -V), ruta absoluta (where python en Windows, which python en Unix) y el contenido de sys.path ejecutando un script. Anótalo — lo usarás de baseline.
2. Crea un entorno venv llamado .venv en un directorio nuevo, actívalo, instala numpy==2.1.0 y pandas==2.2.3, y verifica con pip list. Luego desactívalo y comprueba que numpy ya no se importa desde el Python global.
3. Replica el mismo entorno con uv. Instala uv (pipx install uv o el instalador oficial), corre uv venv, uv pip install numpy pandas, y compara la velocidad contra el paso anterior.
4. Genera requirements.txt congelando versiones exactas con pip freeze > requirements.txt. Borra el entorno, créalo desde cero y reinstala con pip install -r requirements.txt. Verifica que las versiones coinciden.
5. Provoca y resuelve el bug clásico. Desde Jupyter (ejecutando con un Python distinto al del venv activo), corre !pip install seaborn y luego import seaborn. Diagnostica por qué falla (o por qué se instaló en el lugar equivocado) usando import sys; sys.executable en una celda.

Homework verificable

Entrega un repo (o carpeta zip) con:

- [] README.md indicando tu OS, versión de Python instalada y gestor elegido (venv / uv / conda).
- [] .gitignore con .venv/ y __pycache__/ (mínimo).
- [] requirements.txt con exactamente: numpy>=2.0, pandas>=2.2, matplotlib>=3.8, jupyter>=1.0.
- [] Un script verify.py que imprima:
 - sys.version
 - sys.executable
 - numpy.__version__, pandas.__version__
- [] Output de python verify.py pegado al final del README.md.

Criterio de aceptación: otra persona debe poder clonar tu repo, ejecutar:

```
python -m venv .venv
source .venv/bin/activate # o .venv\Scripts\activate en Windows
pip install -r requirements.txt
python verify.py
```

...y obtener un output similar al que tú reportaste (mismo Python mayor.menor, mismos paquetes importables).

Definiciones y características

Python "del sistema"

: Intérprete instalado por el OS (macOS y Linux lo traen por default; Windows no). Lo usan herramientas internas del sistema — modificarlo puede romper el OS. Regla: nunca instales paquetes ahí.

Entorno virtual (venv)

: Carpeta autocontenida con su propio Python ejecutable y sus propios paquetes. Aislada del Python global y de otros venvs. Crear/destruir es barato — uno por proyecto es la norma.

pip

: Gestor de paquetes oficial de Python. Instala desde PyPI (Python Package Index). Característica clave: usa el Python con el que se invoca — siempre prefiere python -m pip install sobre pip install solo.

uv (Astral)

: Reemplazo moderno (2024+) escrito en Rust. Drop-in compatible con pip+venv pero 10–100× más rápido. Maneja también versiones de Python (reemplazo de pyenv). API: uv venv, uv pip install, uv python install 3.12.

conda / mamba

: Gestor multilenguaje (no solo Python). Maneja también dependencias binarias C/C++/CUDA (PyTorch+GPU, geopandas, rdkit). Más pesado pero imprescindible para esos casos. mamba es conda en C++ (más rápido).

requirements.txt vs pyproject.toml vs environment.yml

: Tres formatos, tres ecosistemas. requirements.txt (pip, lockfile). pyproject.toml (estándar moderno PEP 621, declarativo). environment.yml (conda).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ModuleNotFoundError aunque acabo de instal	pip y python apuntan a Pythons distintos.

Permission denied al hacer pip install	Estás instalando en el Python del sistema
.venv\Scripts\Activate.ps1 cannot be loaded	Execution policy bloquea scripts. Fix: Set
Pip muy lento descargando	Red lenta o servidor PyPI lejano. Fix: pr
Borré la carpeta .venv/ y pip list sigue m	Tu shell aún tiene el PATH del venv viejo.
conda activate no funciona en script no-in	conda init requiere shell interactivo. Fix

Preguntas frecuentes

¿venv, uv o conda — cuál uso?

venv si quieres simplicidad y solo tocas Python puro (stdlib + pandas + numpy + sklearn). uv si haces lo mismo pero quieres velocidad — recomendado en 2026. conda solo si necesitas dependencias C/CUDA (PyTorch con GPU, geopandas, química).

¿Puedo tener venv y conda en la misma máquina?

Sí — coexisten. Lo único: no actives ambos a la vez. Tu PATH se confunde y python apunta a uno u otro impredeciblemente.

¿Por qué .venv/ en el repo está en .gitignore?

Porque pesa cientos de MB y es reconstruible desde requirements.txt/pyproject.toml. Versionar el venv duplica el repo y crea conflicto si cambias de OS.

¿Debo actualizar pip al crear un venv?

Sí, primer paso: python -m pip install --upgrade pip. El pip dentro de venvs viejos tiene bugs conocidos y es lento.

¿Cómo sé en qué venv estoy?

python -c "import sys; print(sys.executable)" o which python (Unix) / where python (Windows). El prompt suele cambiar ((.venv) C:\...>) pero no siempre — confirma con el comando.

¿Necesito Python 3.12 específicamente?

Para este curso, sí (varias features usadas dependen). En general, usa la última estable. Python 3.10 ya pierde soporte en oct/2026.

Referencias

- VanderPlas, Python Data Science Handbook, Preface § "Installation Considerations".
- PEP 405 — venv (especificación oficial).
- Astral uv — Getting started.
- Conda vs pip vs venv — comparación oficial.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 002 — Clase 002 — Jupyter y JupyterLab — kernels, magics, debugging, profiling

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, Python Data Science Handbook, cap. 1 — IPython: Beyond Normal Python.

Duración estimada: 90 min.

Objetivo

Que el alumno deje de usar Jupyter como un editor de texto con botón "play" y empiece a usarlo como un entorno exploratorio profesional: con magics que ahorran horas, debugger interactivo (%debug), y profiling real (%timeit, %prun). Al final debe poder diagnosticar por qué un notebook es lento sin adivinar.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar kernel, frontend (Notebook vs JupyterLab vs VS Code) y servidor — y saber qué pasa cuando uno se cuelga.
2. Usar magics esenciales: %timeit, %%time, %run, %load, %matplotlib inline, %debug, %who, %xmode.
3. Conectar un kernel específico a un notebook (ipykernel install --user --name <env>) sin pelearse con el venv equivocado.
4. Debuggear una excepción con %debug y pdb (n, s, c, q, p, l).
5. Profilar código lento con %timeit (microbenchmark) y %prun (line profiler) para decidir dónde optimizar.

Temas

#	Tema	Por qué importa
1	Kernel ↔ frontend ↔ servidor	Saber cuál murió cuando el notebook se cue
2	Modo comando vs modo edición + atajos	Velocidad real: A/B/X/M/Y/Esc/Enter.
3	Magics line (%) vs cell (%%)	El 80% del valor de Jupyter está en las ma
4	%timeit y %%time	Microbenchmark riguroso (varias corridas,
5	%debug + pdb	Inspección post-mortem sin re-correr todo
6	%prun y %lprun	Saber qué función pesa antes de optimizar.
7	Registro de kernels por venv	Cada proyecto, su propio kernel — evita el

Definiciones y características

Kernel

: Proceso Python (u otro lenguaje) que ejecuta el código de las celdas. Vive separado del frontend; si lo matas, pierdes el estado en memoria pero los archivos siguen intactos. Cada notebook se asocia a UN kernel, normalmente el del venv del proyecto.

Frontend

: La interfaz visual (Notebook clásico, JupyterLab, VS Code, Cursor, Colab). Todas hablan el mismo protocolo con el kernel — puedes cambiar de frontend sin perder datos si guardas el .ipynb.

Magic

: Comando especial de IPython, no de Python. Empieza con % (afecta una línea) o %% (afecta la celda entera). Ejemplos: %timeit, %matplotlib inline, %%time, %debug. No funcionan fuera de IPython/Jupyter.

%timeit vs %%time

: %timeit corre la expresión muchas veces, descarta outliers y reporta el mejor → microbenchmark estadísticamente serio. %%time mide una sola corrida del bloque → bueno para operaciones largas donde

repetir cuesta. Característica clave: usa `%timeit` para algo en milisegundos, `%%time` para algo en segundos.

`pdb / %debug`

: Debugger interactivo de Python. `%debug` lo lanza en modo post-mortem después de una excepción — entras al stack en el punto del error sin re-correr nada. Comandos: `n` siguiente línea, `s` entra a función, `c` continúa, `p` var imprime, `u/d` sube/baja en stack, `q` salir.

Dataset / recursos

No requiere dataset externo. Usamos arreglos sintéticos con `numpy.random` para benchmarks. Para el ejercicio de debug, generamos un `ValueError` intencional.

Ejercicios

1. Atajos sin mouse. Crea 5 celdas, navega solo con teclado: convierte 2 a markdown, ejecuta todo en orden, borra una, deshaz. Cronométrate.
2. Registra tu kernel. Desde un venv recién creado: `python -m ipykernel install --user --name ds-lab-001 --display-name 'DS Lab 001'`. Abre Jupyter, selecciona ese kernel, verifica con `import sys; sys.executable`.
3. Benchmark vectorización. Con `%timeit`, compara sumar `range(10_000)` con un `for` vs `np.arange(10_000).sum()`. Anota cuántas veces más rápido es NumPy.
4. Post-mortem. Provoca un `ZeroDivisionError`, luego ejecuta `%debug` en la siguiente celda y navega el stack con `u/d`, inspecciona variables con `p`.
5. Profila una función. Escribe una función que ordene una lista 1000 veces con sort burbuja. Ejecuta `%prun -s cumulative tu_func()`. Identifica la línea más cara.

Homework verificable

Entrega un notebook `homework.ipynb` con: (a) celda que muestra `sys.executable` confirmando que usas un kernel registrado por ti; (b) benchmark `%timeit` comparando `sum(range(N))` vs `np.arange(N).sum()` para `N=10k, 100k, 1M`; (c) tabla markdown con los resultados; (d) gráfico simple del speedup.

Criterio de aceptación: El notebook abre con kernel propio (no el global), las 3 mediciones corren sin errores, y la conclusión incluye un número concreto ("NumPy es ~50× más rápido para `N=1M`").

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>ModuleNotFoundError</code> aunque acabo de instal	El kernel activo NO es el venv donde corri
El notebook está "congelado" / la barra di	Una celda quedó atrapada en bucle infinito
Cambié código de un módulo importado y el	Python cachea módulos importados. Fix: <code>%lo</code>
<code>%timeit</code> en una celda con asignación da err	Las variables creadas dentro de <code>%timeit</code> no
Outputs gigantes hacen el .ipynb pesado y	Cada output (imagen, tabla) queda guardado

Preguntas frecuentes

¿Notebook clásico o JupyterLab o VS Code?

Para aprender, VS Code (mismo backend, mejor UX: autocomplete con type hints, debug gráfico, git inline). Para reuniones colaborativas en navegador, JupyterLab. El Notebook clásico es legacy — sigue funcionando pero ya no recibe features.

¿Debo crear un kernel por proyecto o usar uno global?

Uno por proyecto. Cada proyecto tiene dependencias distintas que entran en conflicto: el kernel global tarde o temprano se rompe. Comando: `python -m ipykernel install --user --name <proyecto>`.

¿Cuándo `%timeit` no es confiable?

Cuando lo que mides toca disco/red/GPU — la varianza es enorme y el min no representa típico. Usa `%%time` con varios runs manuales y reporta mediana. Tampoco confiable si la primera corrida hace JIT (numba) — calienta con un run previo.

`%debug` no funciona, no muestra prompt

Necesita haber ocurrido una excepción en el kernel justo antes. Si la celda falló pero el kernel se reinició, perdiste el stack. También: en VS Code Jupyter, usa el panel de debug en su lugar (más cómodo).

¿Por qué mi notebook tarda 30 segundos en abrir si pesa solo 200 KB?

Probablemente trae outputs binarios grandes (imágenes inline en base64). El JSON parece chico pero al renderizar el navegador procesa MB. Limpia outputs y guarda.

Referencias

- VanderPlas, cap. 1 — IPython: Beyond Normal Python.
- IPython magics reference
- JupyterLab user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 003 — Clase 003 — Git y GitHub para data scientists

Parte: 0 — Prerrequisitos · Fuente: *Pro Git* (Chacon & Straub) — caps. 2 y 3 · *GitHub docs*.

Duración estimada: 120 min.

Objetivo

Que el alumno use git no como "botón save" sino como un sistema serio de versionado: commits atómicos con mensajes útiles, branches por feature, PRs con review, y resolución de conflictos sin pánico. Adicionalmente: ignorar correctamente los archivos típicos de DS (datos pesados, notebooks con output, secrets).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Inicializar un repo, hacer commits atómicos con mensajes en formato convencional.
2. Trabajar con branches: crear, cambiar, merge y resolver un conflicto sin perder código.
3. Configurar `.gitignore` para un proyecto de DS (datos, `.env`, secrets, outputs de notebooks).
4. Abrir y revisar un PR en GitHub desde la línea de comandos con gh.
5. Recuperar trabajo perdido con git reflog (la red de seguridad invisible).

Temas

#	Tema	Por qué importa
1	Modelo de git: working tree → staging → re	Sin este modelo mental, todo parece magia.
2	Commits atómicos + mensajes convencionales	Un commit = un cambio lógico reversible.
3	Branches y merge vs rebase	Cuándo usar cada uno; por qué no rebasear
4	.gitignore para data science	Datos, modelos, notebooks con output, .env
5	Conflictos: anatomía y resolución	<<<<<<, =====, >>>>>> y cómo no entrar
6	Pull Requests + review en GitHub	El review es donde se transfiere conocimiento
7	git reflog — la red de seguridad	Aunque borres una rama, los commits viven

Definiciones y características

Repositorio (repo)

: Carpeta con un subdirectorio `.git/` que guarda toda la historia. Características: contenido inmutable identificado por SHA-1, ramas son punteros móviles, todo cambio publicado es eterno (aunque borres el commit, vive en reflog 90 días).

Commit

: Snapshot inmutable del estado del repo en un momento. Tiene SHA-1, padre(s), autor, fecha, mensaje. Característica: atómico — debería poder revertirse solo sin romper nada.

Branch (rama)

: Puntero móvil a un commit. Mover el puntero es barato. HEAD apunta a la rama actual. La rama main no es especial; solo es la rama por defecto del proyecto.

Working tree / Staging / Repo / Remote

: Las 4 zonas: working tree (lo que editas) → staging area (lo preparado con git add) → repo local (lo commitado) → remote (GitHub/GitLab). Cada git mueve cosas entre estas 4 zonas.

Merge vs Rebase

: Merge crea un commit nuevo que junta dos historias (preserva ambas). Rebase reescribe los commits de tu rama encima de otra (historia lineal pero modificada). Característica clave: nunca rebases ramas compartidas — reescribir SHAs rompe a tus compañeros.

Conventional Commits

: Convención que prescribe tipo(scope): descripción. Tipos: feat, fix, docs, refactor, test, chore, perf, style. Beneficio: changelogs y semver automáticos.

Dataset / recursos

No requiere dataset. El "dataset" son los propios cambios que el alumno hace en archivos de prueba. Para el ejercicio del `.gitignore`, simulamos archivos típicos de DS (csv pesado, `.env`, `.ipynb_checkpoints/`).

Ejercicios

1. Repo desde cero. git init, crea 3 archivos (README.md, data.csv, notebook.ipynb), haz 3 commits con mensajes en formato tipo: descripción (feat/fix/docs/chore).
2. Branch + conflicto. Crea rama feature/x, modifica una línea en README.md. Vuelve a main, modifica la misma línea distinto. Mergea, resuelve el conflicto a mano.
3. `.gitignore` profesional. Genera uno que ignore: `.env/`, `__pycache__`, `.ipynb_checkpoints/`, `.csv` en `data/raw/`, `.env`, `models/.pkl`. Verifica con git status que no aparecen.

4. PR desde la CLI. Crea repo en GitHub (con `gh repo create`), push, crea PR con `gh pr create` y descripción no trivial.

5. Recuperación. Borra una rama con commits. Recupera el HEAD con `git reflog + git checkout <sha> + git switch -c rescate`.

Homework verificable

Repo público en GitHub con: 5+ commits en formato convencional, al menos 1 branch mergeada, un `.gitignore` de DS completo, README con badges (build status si aplica) y un PR cerrado.

Criterio de aceptación: El historial (`git log --oneline`) se lee como cambios atómicos coherentes. `git status` limpio después de un experimento. PR mergeado con descripción legible.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
error: failed to push some refs to 'origin'	El remote tiene commits que no tienes loca
fatal: refusing to merge unrelated histori	Estás juntando dos repos sin ancestro común
Hice <code>git reset --hard</code> y perdí mi trabajo	Si fue local y no había commit: perdido. S
Please tell me who you are al hacer commit	Falta config global. Fix: <code>git config --glo</code>
Commit con archivo enorme; ahora <code>git push</code>	GitHub bloquea blobs >100 MB. Fix: NO bast
Mergeé un PR pero ahora hay conflictos en	Alguien mergeó algo antes y tu base local
<code>.gitignore</code> no funciona — el archivo sigue	Si el archivo ya estaba trackeado antes de

Preguntas frecuentes

¿Merge o rebase?

Regla simple: merge para todo lo público, rebase solo localmente antes de PR para limpiar tus propios commits. Nunca rebases una rama que alguien más usa.

¿Force push (`git push -f`) es siempre malo?

En main o ramas compartidas: catástrofe. En tu propia rama de feature después de rebase: aceptable. Mejor usar `--force-with-lease` que falla si alguien más empujó mientras.

¿Cómo deshago el último commit?

Si NO empujaste: `git reset --soft HEAD~1` (mantiene cambios staged) o `--hard` (los borra). Si YA empujaste y quieres revertirlo sin reescribir historia: `git revert HEAD` (crea commit nuevo que deshace).

¿Squash o no squash al mergear?

Squash = un solo commit final con todo el PR. Bueno para mantener historia limpia en main. Pierdes el detalle de pasos intermedios. Política común: squash en PRs pequeños, merge commit en grandes.

¿Está bien commitear el `.venv/` o el `data/raw/customers.csv`?

NO. `.venv/` se reconstruye con `requirements.txt`. Datos grandes/sensibles van fuera del repo (DVC, S3, etc.) — ver clase 159 Parte 4.

Tengo 30 commits "wip" en mi rama, ¿qué hago antes del PR?

`git rebase -i main` para entrar al rebase interactivo. Cambia pick por squash (o fixup) en los commits intermedios; quedará un historial limpio.

Referencias

- Pro Git book — cap. 2 Git Basics, cap. 3 Branching.
- Conventional Commits
- GitHub CLI manual

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 004 — Clase 004 — Estructura reproducible de proyecto (cookiecutter-data-science)

Parte: 0 — Prerrequisitos · Fuente: cookiecutter-data-science v2 · Hidden Technical Debt in ML Systems (Sculley et al., 2015).

Duración estimada: 60 min.

Objetivo

Que el alumno deje de crear proyectos como "una carpeta con notebooks" y empiece a usar una estructura estándar que separa código, datos, modelos, notebooks de exploración y documentación. Esto no es estética — es lo que permite que un compañero entienda el proyecto en 5 minutos y que el código viva más allá del notebook donde nació.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Generar un proyecto con la plantilla cookiecutter-data-science (CCDS v2).
2. Justificar la separación data/raw (inmutable) ↔ data/interim ↔ data/processed.
3. Mover código de un notebook a src/ cuando deja de ser exploratorio.
4. Documentar dependencias en pyproject.toml (no en requirements.txt suelto).
5. Reconocer los olores de un proyecto mal estructurado (notebooks con números 01/02/03, código duplicado, datos en git).

Temas

#	Tema	Por qué importa
1	Estructura CCDS v2	Convención > improvisación.
2	data/raw es sagrado	Nunca se modifica; siempre se puede regene
3	notebooks/ vs src/	Exploración vs producción.
4	pyproject.toml como fuente de verdad de de	Reemplaza requirements.txt suelto.
5	Makefile como interfaz	make data, make train, make test — lo lee
6	Olores típicos	Untitled27.ipynb, datos en git, final_FINA

Complemento: Testing con pytest

El folder tests/ que genera CCDS suele quedar vacío — y es un error. Un data scientist necesita tests por tres razones concretas: las funciones de feature engineering se reusan en producción y un bug silencioso te corrompe el dataset; los pipelines reproducibles se rompen sin avisar cuando cambias una dependencia; y al refactorizar código de notebook a src/ querés saber que el comportamiento sigue siendo el mismo. Tests bien

escritos son la red que te deja moverte rápido sin romper lo que ya funciona.

Estructura mínima: archivos `test_.py` dentro de `tests/`, funciones `test_()` adentro, y `assert` para verificar. `pytest` descubre todo automáticamente.

Concepto	Para qué sirve
<code>assert</code>	Verificación básica: <code>assert resultado == e</code>
<code>pytest.fixture</code>	Datos o objetos reutilizables entre tests
<code>pytest.mark.parametrize</code>	Corre el mismo test con varios inputs dist
<code>pytest.raises</code>	Verifica que una función levante la excepc
<code>conftest.py</code>	Fixtures compartidas entre varios archivos
<code>--cov</code>	Reporta cobertura de código (requiere pyte

Mini-ejemplo. Función en `src/mi_proyecto/features.py`:

```
def normalize(x):
    return (x - x.mean()) / x.std()
```

Test en `tests/test_features.py`:

```
import numpy as np
import pandas as pd
from mi_proyecto.features import normalize

def test_normalize_media_cero():
    s = pd.Series([1.0, 2.0, 3.0, 4.0, 5.0])
    result = normalize(s)
    assert np.isclose(result.mean(), 0.0)
    assert np.isclose(result.std(), 1.0)
```

Correr los tests: `pytest -v` para ver todo lo que corre, `pytest --cov=src tests/` para incluir cobertura.

¿Qué testear en DS? Funciones puras de transformación (limpieza, encoding, feature engineering) sí — son determinísticas y rápidas. Modelos entrenados con aleatoriedad no se testean directamente sobre métricas exactas; o fijás `random_state/seeds` y verificás contra un valor congelado, o usás snapshots tolerantes (`pytest.approx`). Lo que importa es testear la lógica que escribiste, no reinventar scikit-learn.

Definiciones y características

Cookiecutter

: Generador de proyectos a partir de plantillas. Tomas una plantilla (URL de un repo), respondes 3-5 preguntas y obtienes un proyecto con estructura pre-armada. Característica: idempotente — la plantilla no sabe ni le importa el contenido futuro del proyecto.

CCDS (cookiecutter-data-science)

: Plantilla específica para proyectos de DS, v2 (2023+). Separa `data/raw`, `data/interim`, `data/processed`, `src/`, `notebooks/`, `reports/`, `docs/`. Es convención, no dogma.

Editable install (`pip install -e .`)

: Instala el paquete pero apuntando al código fuente — los cambios se reflejan sin reinstalar. Habilita `from mi_proyecto.features import x` desde notebooks dentro del repo.

`pyproject.toml`

: Estándar moderno (PEP 621) para metadata + dependencias + config de tools (ruff, mypy, pytest). Reemplaza setup.py + setup.cfg + requirements.txt suelto + configs sueltas.

Makefile

: Archivo con "recetas" nombradas (make data, make train). Escrito en tabs (no espacios), las dependencias se declaran arriba (target: dep1 dep2). En proyectos DS funciona como interfaz humana a comandos típicos.

Dataset / recursos

Para el ejercicio principal, descarga el dataset de Palmer Penguins (<https://github.com/allisonhorst/palmerpenguins>) — pequeño (~13 KB), público, ideal para que data/raw/penguins.csv ocupe poco y se pueda commitear sin mala práctica.

Ejercicios

1. Genera un proyecto CCDS. pipx run cookiecutter <https://github.com/drivendataorg/cookiecutter-data-science> con nombre ds-lab-004. Explora la estructura.
2. Mueve código a src/. Toma una función de un notebook tuyo previo y muévela a src/<proyecto>/features.py. Importa desde el notebook con from <proyecto>.features import
3. Convierte requirements.txt a pyproject.toml (sección [project.dependencies]).
4. Refactoriza un notebook caótico. Toma uno con bloques copy-paste y extrae 2 funciones a src/.
5. Lista 5 olores en un repo público que conozcas y propón cómo arreglarlos.

Homework verificable

Un repo público con estructura CCDS, data/raw/ con un dataset pequeño, al menos 1 notebook que importa funciones desde src/, pyproject.toml con deps, Makefile con make setup y make data.

Criterio de aceptación: Un compañero clona, corre make setup && make data y obtiene la misma estructura de datos procesados que tú reportaste. README explica el flujo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ModuleNotFoundError: No module named 'mi_p	El paquete no está instalado en el venv de
CSV en data/raw/ apareció en git status y	El .gitignore no cubre data/raw/ o no se a
Tengo 3 notebooks con la misma función lim	Copy-paste. Fix: extrae a src/mi_proyecto/
El compañero clonó el repo y make data fal	Make no está instalado en Windows por defa
Editó pyproject.toml y los imports siguen	Tras cambios en metadata o entry-points de
pytest corre pero dice collected 0 items	Los archivos o funciones no respetan la co

Preguntas frecuentes

¿Realmente necesito una plantilla? ¿No puedo improvisar?

Puedes, pero perderás el 80% de la ganancia: el compañero que ya conoce CCDS sabe dónde buscar; sin plantilla, cada proyecto es una caja de sorpresas.

¿data/raw/ o data/01_raw/?

CCDS v2 usa data/raw/, data/interim/, data/processed/, data/external/. La numeración 01_/02_/03_ la verás en Kedro y en algunas variantes — ambas son válidas, sigue una sola convención por proyecto.

¿requirements.txt o pyproject.toml?

pyproject.toml para declarar las deps de tu paquete. requirements.txt (generado con pip freeze o uv pip compile) es el lockfile con versiones exactas para reproducir. No están en conflicto; conviven.

¿Y si el proyecto es solo un notebook exploratorio?

No fuerces CCDS. Carpeta con notebook.ipynb, data.csv y README.md está bien. La estructura completa aporta cuando el proyecto vive >3 meses o tiene >1 persona.

¿Por qué los notebooks tienen prefijo numérico como 0.01-jvp-eda.ipynb?

Convención CCDS: <fase>.<orden>-<iniciales>-<tema>. Fase 0=exploración, 1=features, 2=modelos, 3=reportes. Iniciales del autor evitan conflictos cuando varias personas crean notebooks.

¿Hace falta testear notebooks?

Los notebooks no se testean directamente — son scratchpads exploratorios y cambian todo el tiempo. Lo que sí se testea es el código que migrás del notebook a src/: en el momento que una función deja de ser exploración y pasa a src/mi_proyecto/, le escribís su test_* correspondiente. Regla práctica: si la función vive en src/, tiene test; si vive en una celda, no.

Referencias

- cookiecutter-data-science v2 docs
- Sculley et al., Hidden Technical Debt in ML Systems (NeurIPS 2015).
- Palmer Penguins dataset
- pytest docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 005 — Clase 005 — VS Code / Cursor para Python y Jupyter

Parte: 0 — Prerrequisitos · Fuente: VS Code Python docs · Cursor docs · The Pragmatic Programmer cap. "Power Editing".

Duración estimada: 60 min.

Objetivo

Que el alumno deje de usar VS Code como Notepad y lo configure como un IDE serio para Python + Jupyter: selector de intérprete, debugger gráfico, linter (ruff), formatter (ruff format), tests integrados y notebooks editables. Bonus: cuándo conviene Cursor (VS Code + IA integrada).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Configurar VS Code con la extensión Python + Jupyter, seleccionando el intérprete del venv del proyecto.
2. Debuggear un script Python paso a paso desde el panel gráfico (breakpoints, watch, call stack).
3. Editar y ejecutar notebooks sin Jupyter web — con autocompletado, type hints y debug de celda.

4. Configurar ruff como linter + formatter (reemplaza black + isort + flake8 en un solo tool).
5. Decidir cuándo usar Cursor (idéntico a VS Code + IA integrada con autorización por chat).

Temas

#	Tema	Por qué importa
1	Selección de intérprete por workspace	El bug "funciona en terminal pero no en VS
2	Debugger gráfico vs print	Breakpoints, watch, evaluación expresiones
3	Notebooks nativos en VS Code	Mejor UX que Jupyter web para edición; mis
4	ruff = linter + formatter en uno	Reemplaza black/isort/flake8/pylint. Más r
5	Tests integrados (pytest)	Run/debug tests con un click; coverage inl
6	Extensiones esenciales	Python, Jupyter, GitLens, ruff, Even Bette
7	Cursor: cuándo sí	Cuando quieres pair-programming con IA sin

Definiciones y características

Workspace

: Concepto de VS Code = una carpeta (o conjunto de carpetas) con configuración asociada en `.vscode/settings.json`. La configuración del workspace override a la del usuario. Característica: pones `.vscode/` en git para que todos los colaboradores hereden la misma config.

Intérprete Python

: Ejecutable concreto (`/path/to/.venv/bin/python`). VS Code recuerda uno por workspace. Es el origen del 90% de los "funciona en mi máquina" entre IDE y terminal.

ruff

: Linter + formatter en un solo binario, escrito en Rust. Reemplaza black + isort + flake8 + (parte de) pylint con un único tool 10–100× más rápido. Config en `[tool.ruff]` de `pyproject.toml`.

Breakpoint

: Marca en una línea (F9) que pausa la ejecución cuando llega ahí. Permite inspeccionar variables, paso a paso, evaluar expresiones — mil veces más eficiente que print.

launch.json

: Config de debug de VS Code. Define perfiles: "debug archivo actual", "debug tests", "debug Django", etc. Cada perfil tiene su program, args, env, justMyCode.

Dataset / recursos

Sin dataset externo. Usamos un script Python con un bug intencional para practicar debug gráfico, y un notebook trivial para verificar autocompletado/type hints.

Ejercicios

1. Selecciona intérprete. En VS Code: `Ctrl+Shift+P` → "Python: Select Interpreter" → elige el del `.venv` del proyecto. Verifica con `print(sys.executable)` en una celda.
2. Debug paso a paso. Toma un script con un bug, pon breakpoint (F9), ejecuta con F5, navega con F10 (next), F11 (step in), `Shift+F11` (step out). Inspecciona variables en panel.
3. Configura ruff. En `pyproject.toml`: `[tool.ruff]` con `line-length = 100`, `[tool.ruff.lint]` con `select = ["E", "F", "I", "UP"]`. Habilita format on save.

4. Edita un notebook. Abre notebook.ipynb en VS Code, ejecuta una celda, comprueba que el autocompletado funciona con type hints de pandas.

5. Tests con un click. Instala pytest, crea tests/test_simple.py con 2 tests (uno OK, uno FAIL). Usa el panel "Testing" para correr/debuggear.

Homework verificable

Repo con pyproject.toml que incluye [tool.ruff], .vscode/settings.json con python.defaultInterpreterPath y format-on-save habilitado, screenshot del debugger gráfico mostrando un breakpoint activo y variables inspeccionadas.

Criterio de aceptación: Otro alumno clona el repo, abre en VS Code, y al guardar un .py se aplica ruff format automáticamente. El screenshot muestra al menos 1 breakpoint y la variable inspeccionada.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Python interpreter is not selected" al ab	Workspace nuevo, VS Code no eligió uno. Fi
Format-on-save no aplica ruff aunque está	Falta declarar ruff como formatter por def
Debugger arranca pero se salta mis breakpo	Estás corriendo el archivo (Ctrl+F5 = sin
Tests no aparecen en el panel "Testing"	VS Code no detectó pytest. Fix: Ctrl+Shift
Cambié interpreter y los imports siguen ro	VS Code cachea symbols del intérprete viej

Preguntas frecuentes

¿VS Code o Cursor?

Cursor = VS Code + IA integrada (chat con contexto del repo, edición multi-archivo). Si pagas Copilot o no te interesa IA, quédate en VS Code. Si quieres pair-programming con IA sin saltar a otra app, Cursor. Las extensiones son las mismas.

¿Debo commitear .vscode/?

Sí la parte compartida: settings.json (interpreter path relativo, formatter, etc.), extensions.json (recomendaciones). No lo personal: .vscode/launch.json con paths absolutos del tester.

¿Notebook en VS Code o en JupyterLab?

VS Code para escribir/refactorizar (autocomplete con type hints, debug por celda, git inline). JupyterLab cuando alguien necesita un navegador y no quiere instalar VS Code (alumno, demo en proyector).

¿Para qué justMyCode: false?

Por default, el debugger se salta código de librerías de terceros (numpy, pandas) — útil para no perderte. Pero a veces el bug viene desde dentro de pandas (datos malformados); con false puedes entrar a ver.

¿Ruff reemplaza todo el stack? ¿No necesito black?

Sí — ruff format es drop-in replacement de black (mismo output prácticamente). Mismo con isort (ruff check --select I --fix) y flake8 (ruff check). Único caso donde aún conviene black: si tu org ya tiene CI con black configurado y no quieres tocar.

Referencias

- VS Code Python docs

- VS Code Jupyter docs
- ruff docs
- Cursor docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 006 — Clase 006 — Python: tipos, estructuras, control de flujo

Parte: 0 — Prerrequisitos · Fuente: Python Tutorial oficial (caps. 3-5) · Fluent Python (Ramalho, 2ª ed.) cap. 1.

Duración estimada: 120 min.

Objetivo

Refrescar (o instalar) los cimientos de Python que el resto del programa asume: tipos primitivos, las 4 estructuras built-in (list, tuple, set, dict), control de flujo (if/for/while), unpacking, truthiness y la diferencia entre mutables e inmutables — la fuente del 90% de bugs sutiles.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar tipos mutables (list, dict, set) vs inmutables (tuple, str, int, frozenset) y predecir el efecto en asignaciones.
2. Usar las 4 estructuras eligiendo bien: list (orden + duplicados), tuple (inmutable, rápida), set (unicidad), dict (lookup $O(1)$).
3. Aplicar unpacking en for, returns múltiples y args/*kwargs.
4. Evaluar truthiness correctamente ([], {}, 0, "", None son falsy; el resto es truthy).
5. Identificar el bug del default mutable en funciones (def f(x, lst=[])) y por qué es trampa.

Temas

#	Tema	Por qué importa
1	Mutable vs inmutable	Define qué pasa con <code>a = b</code> .
2	list, tuple, set, dict — cuándo cada uno	Complejidad y semántica distintas.
3	Iteración: for, enumerate, zip	Idiomático > C-style.
4	Unpacking y starred expressions	<code>a, *b, c = [1,2,3,4,5]</code> .
5	Truthiness y operadores and/or	Evalúan al objeto, no al booleano.
6	Default mutables: el clásico	<code>def f(x, lst=[])</code> comparte la lista entre l

Definiciones y características

Mutable

: Objeto cuyo estado puede modificarse después de crearlo (list, dict, set, bytearray). Consecuencia: si dos variables apuntan al mismo objeto mutable, cambiar una cambia la otra (alias).

Inmutable

: Objeto que NO se puede modificar; cualquier 'modificación' produce un objeto nuevo (int, float, str, tuple, frozenset). Característica clave: son hashables y pueden ser claves de dict o miembros de set.

Hashable

: Objeto con `__hash__` definido y constante durante su vida. Los inmutables built-in son hashables; las listas y dicts NO lo son. Es lo que permite el $O(1)$ lookup de set/dict.

Unpacking

: Sintaxis para asignar múltiples variables desde un iterable (a, b = (1, 2) o a, resto = [1, 2, 3, 4]). El `*` captura el resto en una lista. Funciona en for, return, llamadas a funciones (f(lista), g(*dict)).

Truthy / Falsy

: Cómo evalúa Python un objeto en contexto booleano (if x:). Falsy: False, None, 0, 0.0, "", [], {}, set(). Truthy: todo lo demás (incluso ' ' con espacio, [0], {None: None}).

Dataset / recursos

Datos sintéticos pequeños generados en el notebook (lista de diccionarios simulando estudiantes). No requiere descarga.

Ejercicios

1. Cuenta palabras. Dado un texto, devuelve un dict[str, int] con frecuencias. Sin usar Counter.
2. Unique con orden. Recibe list[int], devuelve la lista de únicos manteniendo el orden de primera aparición.
3. Reproduce el bug del default mutable. Escribe def add(item, target=[]), llámala 3 veces con add('x'). Observa. Explica por qué y arregla.
4. Top-K palabras. Mismo texto del ejercicio 1, devuelve las 5 más frecuentes ordenadas por frecuencia descendente.
5. Grupos por inicial. Dado list[str], devuelve dict[str, list[str]] agrupando por primera letra (case-insensitive).

Homework verificable

Notebook homework.ipynb con las 5 funciones de los ejercicios, cada una con: (a) implementación, (b) 3 casos de prueba (incluyendo edge cases — lista vacía, string vacío), (c) docstring corto explicando complejidad.

Criterio de aceptación: Las 5 funciones pasan sus casos de prueba; los edge cases manejados sin excepción.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
TypeError: unhashable type: 'list'	Intentaste usar una lista como clave de dict
Modifiqué una lista y otra variable también	Las dos apuntan al mismo objeto (alias). F
Función con default =[] comparte la lista	El default se evalúa una vez al definir la función
KeyError al acceder a dict que no tiene la clave	d['x'] lanza KeyError si no existe. Fix: u.get('x', default)
if items != None: en vez de if items is not None	Para singletons (None, True, False) usa si

Preguntas frecuentes

¿Cuándo tupla y cuándo lista?

Tupla para records heterogéneos de tamaño fijo (punto = (3.5, 7.1), (nombre, edad)). Lista para colecciones homogéneas que crecen (temperaturas = [22.1, 22.5, ...]). Tupla además sirve como clave de dict; lista no.

¿Set o dict para chequear pertenencia?

Ambos son $O(1)$. Set si solo necesitas saber si está. Dict si además necesitas asociar valor. Una list para esto es $O(n)$ — usa set/dict si haces `if x in coleccion` con N grande.

¿`copy()` me da una copia completa? ¿Y con listas anidadas?

`.copy()` es shallow: copia el contenedor pero comparte las referencias internas. Para anidamiento, `import copy; copy.deepcopy(x)` — más lento pero independiente.

¿Por qué $0.1 + 0.2 \neq 0.3$?

Floats son IEEE 754 binarios; 0.1 y 0.2 no tienen representación exacta. Fix: para igualdad usa `math.isclose(a, b)`. Para precisión decimal financiera, `from decimal import Decimal`.

¿Qué pasa con dict si insertas la misma clave dos veces?

El segundo valor sobrescribe al primero. El orden de inserción se preserva (Python 3.7+), así que `list(d)` da las keys en el orden en que se insertaron por primera vez.

Referencias

- Python Tutorial — Data Structures
- Ramalho, *Fluent Python 2e* — cap. 1 The Python Data Model.
- Python Tutorial — Control flow

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 007 — Clase 007 — Comprehensions y generadores

*Parte: 0 — Prerrequisitos · Fuente: Ramalho, *Fluent Python 2e* — caps. 2 (Sequences) y 17 (Iterators, Generators, Coroutines).*

Duración estimada: 90 min.

Objetivo

Que el alumno escriba código Python idiomático: list/dict/set comprehensions en vez de `for+append`, generadores cuando el dataset no cabe en memoria, y entienda la diferencia fundamental entre construir una lista y producir un iterable perezoso.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Convertir loops `for+append` a list/dict/set comprehensions sin perder legibilidad.
2. Usar generadores (`yield` y generator expressions) para procesar datos que no caben en RAM.
3. Distinguir `[x for x in xs]` (lista) vs `(x for x in xs)` (generador): memoria y consumo.
4. Encadenar generadores con `itertools` (`chain`, `islice`, `takewhile`, `groupby`).

5. Identificar cuándo NO usar comprehension (lógica compleja, side effects, debug difícil).

Temas

#	Tema	Por qué importa
1	List comprehension: [expr for x in xs if c]	Idiomático, eficiente, legible si es simpl
2	Dict/set comprehensions	Mismo patrón, otra estructura.
3	Generator expressions: (expr for x in xs)	Perezoso, memoria O(1).
4	Funciones generadoras con yield	Reescribe procesos como streams.
5	itertools — la caja de herramientas	chain, islice, groupby, accumulate, combin
6	Comprehension vs loop: cuándo NO	Lógica >2 líneas, side effects, debug.

Definiciones y características

List comprehension

: Expresión [expr for x in iterable if cond] que construye una lista completa en memoria. Equivalente idiomático a for + append + if. Más rápida y legible si la expresión es simple.

Generator expression

: Lo mismo pero con paréntesis (expr for x in iterable if cond). Devuelve un generador perezoso: produce cada valor on-demand, memoria O(1). Solo puedes recorrerlo una vez.

Iterador

: Objeto con método `__next__()` que devuelve el siguiente valor o lanza `StopIteration`. Es lo que está detrás de un for. Características: lazy, single-pass, memoria O(1).

Generador

: Función con yield (o generator expression) que produce un iterador. Cada yield pausa la función y emite un valor; al siguiente next() retoma desde ahí. Mantiene el estado local entre llamadas.

itertools

: Librería stdlib con bloques perezosos componibles: chain (concatena), islice (slicing perezoso), groupby (agrupa consecutivos), accumulate (sum/prod corridos), takewhile, combinations, product.

Dataset / recursos

Datos sintéticos: rango grande de números (1M elementos) para mostrar diferencia memoria lista vs generador. Sin descarga.

Ejercicios

1. De for a comprehension. Toma 3 loops for+append (cuadrados, filtra pares, mapea a strings) y conviértelos.
2. Generador de Fibonacci infinito. Función con yield que produce Fibonacci. Úsala con itertools.islice para tomar los primeros 20.
3. Memoria: lista vs generador. Mide RAM (con tracemalloc) de `sum([i for i in range(10_000_000)])` vs `sum(i for i in range(10_000_000))`. Reporta la diferencia.
4. Procesa CSV línea por línea. Lee un archivo grande con yield línea por línea, filtra por una condición, cuenta sin cargar todo en memoria.
5. Pivot con dict comprehension. Dada `list[tuple[str, int]]` (nombre, puntaje), construye `dict[str, list[int]]`

agrupando puntajes por nombre.

Homework verificable

Notebook que: (1) reescribe 3 loops como comprehensions, (2) implementa generador Fibonacci con islice, (3) comparativa RAM lista vs generador con tracemalloc y tabla de resultados, (4) lee un CSV $\geq 10k$ filas con generador y filtra sin cargar entero.

Criterio de aceptación: La medición de RAM muestra $>100\times$ menos memoria con generador. CSV se procesa sin OOM.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Iteré un generador y la segunda vez está v	Los generadores son single-pass. Tras cons
MemoryError al hacer [expensive(x) for x i	Construyes lista completa en RAM. Fix: usa
itertools.groupby me agrupa raro	Solo agrupa elementos consecutivos con mis
Generator expression dentro de función fal	Estás haciendo gen[0] — generadores no son
List comprehension con if-else no funciona	if al final filtra; if-else va al principi

Preguntas frecuentes

¿Cuándo comprehension y cuándo for clásico?

Comprehension cuando es una expresión clara en 1 línea. For clásico cuando hay >2 statements, side effects (print, mutación), o la lógica es más legible explícita.

¿Generator expression o list?

Generator si el resultado solo se consume una vez y N es grande (RAM importa). List si necesitas recorrer 2+ veces, indexar, o hacer len(). Truco: `sum(xx for x in xs)` evita lista temporal vs `sum([xx for x in xs])`.

¿Cuánto más rápido es vs un for tradicional?

~10-30%, no mil veces más. La ganancia real es legibilidad. Si necesitas mil veces, no es comprehension lo que buscas — es numpy/vectorización (clase 015).

¿Generador infinito (while True: yield ...) es buena idea?

Sí, pero ojo: nunca lo conviertas a lista directo (`list(gen)`) — bucle infinito hasta OOM. Acopla con `itertools.islice(gen, N)` para truncar.

yield from vs yield?

yield from sub_iterable delega: yieldea todos los valores del sub-iterable. Equivale a `for x in sub: yield x` pero más rápido y propaga `send()/throw()` correctamente. Útil para componer generadores.

Referencias

- Ramalho, Fluent Python 2e — caps. 2 y 17.
- PEP 202 — List Comprehensions
- PEP 255 — Simple Generators
- itertools docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.

- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 008 — Clase 008 — Funciones: args, kwargs, lambdas, closures

Parte: 0 — Prerrequisitos · Fuente: Ramalho, Fluent Python 2e — cap. 7 (Functions as First-Class Objects), cap. 9 (Decorators and Closures).

Duración estimada: 90 min.

Objetivo

Que el alumno use funciones como ciudadanos de primera clase: pasarlas como argumento, retornarlas, escribir lambdas cuando aportan, y entender closures — la base de los decoradores que verán más adelante. Sin esto, el código pandas/sklearn parece magia.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Definir funciones con argumentos posicionales, keyword-only, args y *kwargs.
2. Pasar funciones como argumento (callbacks: `sorted(xs, key=fn)`, `df.apply(fn)`).
3. Usar lambdas donde son legibles (callbacks cortos) y evitarlas donde no (lógica).
4. Explicar y escribir closures (función que captura variables del scope exterior).
5. Anticipar la diferencia entre args y *, args (keyword-only marker).

Temas

#	Tema	Por qué importa
1	Argumentos: posicional, keyword, default	Cuatro modos, una sintaxis.
2	args y *kwargs	Funciones que aceptan número variable.
3	Keyword-only con * separador	<code>def f(a, *, b) → b solo nombrado.</code>
4	Funciones como objetos	Asignables, pasables, retornables.
5	Lambdas: dónde sí y dónde no	Callbacks cortos sí; lógica compleja no.
6	Closures: capturando scope	Base mental de los decoradores.

Definiciones y características

First-class object

: En Python, funciones son ciudadanos de primera clase: se asignan a variables (`f = saludar`), se pasan como argumento (`sorted(xs, key=f)`), se retornan de otras funciones. Esto habilita callbacks, decoradores y closures.

args / kwargs*

: args captura argumentos posicionales sobrantes en una tupla. kwargs captura argumentos nombrados sobrantes en un dict. Convención: solo el `*` y `**` importan; los nombres args/kwargs son convención.

Keyword-only argument

: Argumento que solo puede pasarse nombrado: declarado después de `*` o args en la signatura. `def f(a, *, b)` obliga a `f(1, b=2)`. Mejora legibilidad en APIs con muchos params.

Lambda

: Función anónima de UNA expresión: `lambda x: x*2`. Sin nombre, sin docstring, sin múltiples statements. Útil para callbacks cortos (`sorted(xs, key=lambda p: p['edad'])`). Si necesitas más, usa `def`.

Closure

: Función que captura variables del scope donde fue definida y las mantiene vivas aunque ese scope termine. Base mental de los decoradores. Para modificar la variable capturada, usa `nonlocal`.

Decorador

: Función que recibe función y retorna función (típicamente envuelta). Sintaxis: `@dec` antes de `def`. Implementado típicamente con closure + `@functools.wraps` para preservar metadata original.

Dataset / recursos

Datos sintéticos pequeños (lista de dicts simulando ventas). Sin descarga.

Ejercicios

1. Función con todo. Define `f(a, b=10, args, c, *kwargs)`. Llámala de 3 formas distintas que sean válidas. Identifica qué llamadas son inválidas y por qué.
2. `sorted` con `key`. Dada `list[dict]` de personas, ordena por edad (`asc`) y por nombre alfabético. Usa `lambda` primero, luego `operator.itemgetter`.
3. Closure contador. Escribe `make_counter()` que retorna una función que cada vez que se llama incrementa y retorna un contador interno. ¿Por qué funciona?
4. Memoización manual. Implementa un decorador `@memoize` usando closure + dict. Aplícalo a Fibonacci recursivo y mide el speedup con `%timeit`.
5. Compose. Escribe `compose(f, g, h)` que retorna una función equivalente a `lambda x: f(g(h(x)))`.

Homework verificable

Notebook con: (a) implementación y demo de `make_counter` explicando con comentario por qué el contador persiste; (b) `@memoize` aplicado a Fibonacci recursivo con benchmark (`N=35`) antes/después; (c) ordenamiento de `list[dict]` por 2 criterios usando `itemgetter`.

Criterio de aceptación: `memoize` reduce `Fibonacci(35)` de segundos a milisegundos. Counter independiente entre instancias.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>UnboundLocalError: local variable 'x' refe</code>	Asignaste a <code>x</code> dentro de la función → Pytho
<code>SyntaxError: positional argument follows k</code>	Llamaste <code>f(a=1, 2)</code> — posicionales primero.
<code>TypeError: f() got multiple values for arg</code>	Pasaste <code>x</code> posicional Y nombrado: <code>f(5, x=10</code>
Mi decorador rompe <code>help(funcion)</code>	Sin <code>@functools.wraps(fn)</code> , el wrapper pierd
Lambda en loop captura el último valor	<code>[lambda: i for i in range(3)]</code> — todas las

Preguntas frecuentes

¿Cuándo `args`, `kwargs` y cuándo argumentos explícitos?*

Argumentos explícitos siempre que conozcas la signatura — el IDE te ayuda y el lector entiende. `args`, `*kwargs` solo en wrappers genéricos (decoradores, factories) que deben aceptar cualquier llamada.

¿Lambda o def?

Lambda solo si: (a) cabe en una expresión, (b) la usas inmediatamente (callback), (c) un nombre no aportaría. En todos los demás casos, def con nombre — más debuggeable, soporta docstring y type hints.

¿Closure es lo mismo que decorador?

Decorador suele estar implementado con closure, pero closure $\neq$ decorador. Closure es cualquier función que captura su entorno; decorador es un patrón específico (función $\rightarrow$ función).

¿Por qué necesito nonlocal en make_counter?

Sin nonlocal, count += 1 dentro de inner se interpretaría como variable local nueva y daría UnboundLocalError. nonlocal le dice: 'esa variable vive en el scope inmediato exterior, modifícala'.

¿Cuál es el costo de pasar funciones como argumento?

Mínimo (es solo una referencia). Lo costoso es la invocación repetida en bucles tight (cada llamada Python tiene overhead). Para esto, NumPy/Cython/Numba.

Referencias

- Ramalho, Fluent Python 2e — caps. 7 y 9.
- Python docs — More on Defining Functions
- PEP 3102 — Keyword-Only Arguments

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 009 — Clase 009 — Manejo de excepciones y context managers

Parte: 0 — Prerrequisitos · Fuente: Python Tutorial cap. 8 (Errors and Exceptions) · Ramalho, Fluent Python 2e — cap. 18 (Context Managers).

Duración estimada: 75 min.

Objetivo

Que el alumno maneje excepciones con criterio (sin except: pass), construya jerarquías de excepciones propias cuando aporta, y use context managers (with) — tanto los built-in como propios con @contextmanager — para garantizar limpieza de recursos. Sin esto, el código de carga de datos es una bomba de relojería.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar los 3 tipos de errores (Syntax, runtime exceptions, logical) y dónde se manejan.
2. Capturar excepciones específicas (except ValueError, no except:) y propagar las que no sabes manejar.
3. Crear una excepción propia heredando de la jerarquía estándar (class DatasetCorruptoError(Exception)).
4. Usar with para archivos, sesiones HTTP, transacciones DB.

5. Escribir un context manager propio con @contextmanager (timer, supress, change_dir).

Temas

#	Tema	Por qué importa
1	Jerarquía de excepciones built-in	BaseException → Exception → ValueError/Key
2	try/except/else/finally	Cada bloque tiene un rol específico.
3	Capturar específico, no genérico	except: esconde bugs.
4	Excepciones propias	Comunican intención en vez de cargar mensa
5	Context managers: protocolo __enter__/___ex	Garantiza cleanup.
6	@contextmanager de contextlib	Crear cms con función + yield.

Definiciones y características

Excepción

: Objeto que se 'lanza' (raise) cuando algo anómalo ocurre. Sube por el stack hasta que un except lo captura, o termina el programa. Todas heredan de BaseException; las que debes capturar heredan de Exception.

try/except/else/finally

: try: código riesgoso. except: maneja una excepción específica. else: corre si try no lanzó (raro pero útil). finally: cleanup garantizado, lanzó o no.

Jerarquía de excepciones

: BaseException → SystemExit / KeyboardInterrupt / Exception → ValueError / TypeError / LookupError (→ KeyError, IndexError) / OSError / ArithmeticError (→ ZeroDivisionError). Captura siempre la más específica que sepas manejar.

Excepción propia (custom)

: Subclase de Exception (o subclase específica). Permite tipificar errores de tu dominio (class DatasetCorruptoError(Exception): ...) en vez de strings. El caller puede except DatasetCorruptoError con precisión.

Context manager

: Objeto con __enter__ y __exit__ que se usa con with. Garantiza setup/teardown (abrir/cerrar archivo, conectar/desconectar BD, lock/unlock). El __exit__ corre incluso si hay excepción.

@contextmanager

: Decorador de contextlib que convierte una función con yield en context manager. Pre-yield = __enter__, post-yield = __exit__. Mucho más corto que escribir la clase completa.

Dataset / recursos

Archivo temporal generado en el notebook. Sin descarga.

Ejercicios

1. Captura específica. Escribe una función parse_int_safe(s, default=0) que use try/except solo para ValueError. Demuestra que no esconde otros errores (ej. TypeError si pasas un dict).
2. Excepción propia. Define class DatasetCorruptoError(Exception) con un atributo linea. Lanzala desde una función cargar_csv cuando una línea no tenga el número correcto de columnas.
3. with para archivo. Lee un archivo línea por línea contando palabras. Compara con la versión sin with

(manual open/close) y muestra qué pasa si hay excepción a mitad.

4. Context manager propio: timer. Con `@contextmanager`, escribe `with timer("carga")`: que imprima cuánto duró el bloque.

5. Context manager: `change_dir`. `with cd("/tmp")`: cambia de directorio al entrar y vuelve al salir — incluso si hay excepción.

Homework verificable

Notebook con: (a) `parse_int_safe` con tests de los 3 casos (válido, inválido, otro tipo); (b) `DatasetCorruptoError` usada en una función `cargar_csv` que valida `#columnas`; (c) decorador-context manager timer aplicado a 2 operaciones; (d) `cd` context manager.

Criterio de aceptación: Excepciones se capturan solo donde sabes manejarlas. timer reporta segundos correctamente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>except:</code> o <code>except Exception:</code> ciego esconde	Cualquier error queda silenciado (incluso
<code>finally</code> con <code>return</code> traga la excepción	Si <code>finally</code> ejecuta <code>return</code> , la excepción de
<code>with open(f)</code> as <code>f</code> , <code>open(g)</code> as <code>g</code> : falla por	Esa sintaxis requiere Python 3.10+. Fix: <code>e</code>
Capturé <code>KeyError</code> pero el código sigue romp	Tu <code>except</code> está más arriba del <code>try</code> , o el <code>er</code>
Excepción dentro de generator no se captur	Las excepciones en generators son tricky;

Preguntas frecuentes

¿`except Exception` o `except`?

Casi siempre `except Exception` — más explícito. `except:` desnudo captura también `KeyboardInterrupt` y `SystemExit`, lo cual rompe `Ctrl+C` y `sys.exit()`.

¿Cuándo creo una excepción propia?

Cuando el caller necesita diferenciar este error de otros. `raise ValueError('CSV corrupto')` obliga al caller a parsear strings; `raise DatasetCorruptoError(linea=42)` le da un tipo y un atributo concreto.

¿`with` solo para archivos?

No — para CUALQUIER recurso que necesite cleanup. Files (`open`), conexiones DB (`engine.connect()`), locks (`threading.Lock`), sesiones HTTP (`requests.Session`), transacciones (`db.atomic()`), timing (`with timer(...)`).

¿Debo capturar y re-lanzar para añadir contexto?

Sí, con `raise ExceptionNueva(...) from e` — preserva el `stacktrace` original como `'__cause__'`. El log muestra ambos errores en cadena.

¿`pass` en `except` es siempre malo?

Casi siempre sí. Si tienes que silenciar, al menos `log.warning(...)`. Solo OK cuando la excepción es esperada (`except FileNotFoundError: pass` al limpiar archivos opcionales).

Referencias

- Python Tutorial — Errors and Exceptions
- Ramalho, Fluent Python 2e — cap. 18 Context Managers and else Blocks.

- contextlib docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábralo desde el laboratorio del programa o desde Jupyter.

Clase 010 — Clase 010 — OOP básico, dataclasses, herencia

Parte: 0 — Prerrequisitos · Fuente: Ramalho, Fluent Python 2e — caps. 5 (Data Class Builders) y 14 (Inheritance) · Python Tutorial cap. 9.

Duración estimada: 90 min.

Objetivo

Que el alumno escriba clases cuando aportan (no por hábito Java), use `@dataclass` para records sin boilerplate, entienda herencia con criterio (preferir composición), y conozca los métodos dunder más usados (`__repr__`, `__eq__`, `__lt__`, `__len__`).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Definir clases con `__init__`, atributos de instancia y métodos.
2. Usar `@dataclass` para records inmutables/mutables sin escribir `__init__`/`__repr__`/`__eq__`.
3. Heredar y sobrescribir métodos con `super()`.
4. Implementar dunders esenciales: `__repr__`, `__str__`, `__eq__`, `__lt__`, `__len__`, `__iter__`.
5. Decidir entre clase, `dataclass` o `NamedTuple` según el caso.

Temas

#	Tema	Por qué importa
1	Clase mínima: <code>__init__</code> + atributos + método	El bloque básico.
2	<code>@dataclass(frozen=True)</code>	Records inmutables sin boilerplate.
3	Herencia + <code>super()</code>	Reutilizar implementación de la clase base
4	Composición > herencia	"Has-a" generalmente mejor que "is-a".
5	Métodos dunder	Integran tu clase con <code>len()</code> , <code>==</code> , <code>repr()</code> , <code>s</code>
6	<code>dataclass</code> vs <code>NamedTuple</code> vs <code>TypedDict</code>	Elegir según necesidad de mutabilidad/comp

Definiciones y características

Clase / instancia

: Una clase es una plantilla (`class Punto:`); una instancia es un objeto concreto (`p = Punto(3, 4)`). `__init__` se llama al crear la instancia. `self` es la convención para referirse a la instancia dentro de los métodos.

Método dunder ("magic method")

: Método con doble underscore (`__init__`, `__repr__`, `__eq__`, `__lt__`, `__len__`, `__iter__`, `__add__`). Python los invoca implícitamente con sintaxis especial (`len(obj) → obj.__len__()`).

@dataclass

: Decorador que genera `__init__`, `__repr__`, `__eq__` automáticamente desde las anotaciones de tipo de la

clase. Reduce boilerplate. Con `frozen=True` la hace inmutable y hashable.

Herencia

: `class B(A)` — B hereda atributos y métodos de A; puede sobrescribirlos. `super()` invoca al método de la clase padre. Múltiple herencia existe pero se complica (MRO).

Composición

: "B tiene un A" (atributo) en vez de "B es un A" (herencia). Generalmente preferible: menos acoplamiento, sin problemas de herencia múltiple/diamante.

Polimorfismo

: Distintas clases responden al mismo método con comportamiento distinto (`Animal.hablar()` → 'guau' o 'miau' según la subclase). Permite tratar instancias heterogéneas uniformemente.

NamedTuple vs dataclass vs TypedDict

: `NamedTuple`: tupla con nombres, inmutable, hashable, sin métodos custom. `dataclass`: clase con boilerplate auto, mutable por default (mejor con métodos). `TypedDict`: dict con esquema (estructural, no nominal).

Dataset / recursos

Sintético: lista de objetos Punto y Estudiante. Sin descarga.

Ejercicios

1. Clase Punto. Define `Punto(x, y)` con `__repr__`, `__eq__`, distancia al origen y `__add__` para sumar puntos.
2. Dataclass Estudiante. `@dataclass` con nombre, notas: `list[float]`, método `promedio()`. Crea 3 instancias, ordena por promedio.
3. Frozen Vector. `@dataclass(frozen=True)` para un vector 2D inmutable. Intenta modificar un atributo y observa la excepción.
4. Herencia. `Animal` con `hablar()` → 'genérico'. `Perro(Animal)` que sobrescribe a 'guau'. `Gato(Animal)` a 'miau'.
5. Composición. Coche que tiene un Motor (composición) en vez de heredar de Motor. Justifica por qué.

Homework verificable

Notebook con: (a) Punto con 4 dunders y tests; (b) `@dataclass` Estudiante con sort por promedio; (c) `@dataclass(frozen=True)` Vector que demuestra inmutabilidad lanzando excepción; (d) jerarquía `Animal` → `Perro/Gato` con polimorfismo (lista mixta llamando `hablar()`).

Criterio de aceptación: Las 4 clases pasan tests; `frozen=True` lanza `FrozenInstanceError` al asignar.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Olvidé self en un método y el error es con	Cualquier método de instancia recibe self
<code>@dataclass</code> con field mutable default rompe	<code>@dataclass class X: items: list = []</code> lanza
<code>__eq__</code> definido pero <code>__hash__</code> rompe	Definir <code>__eq__</code> sin <code>__hash__</code> hace la clase
<code>super().__init__(...)</code> olvidado en subclase	Atributos del padre quedan sin inicializar
Modifico atributo y otra instancia también	Asignaste un mutable como class attribute,

Preguntas frecuentes

¿Cuándo necesito OOP en data science?

Menos de lo que crees. Para análisis exploratorio, funciones + dicts/dataclasses bastan. Necesitas clases cuando hay: estado mutable complejo (modelos sklearn), polimorfismo (varios algoritmos misma interfaz), o frameworks que lo exigen (PyTorch nn.Module).

¿@dataclass o namedtuple o pydantic?

namedtuple: record inmutable simple, sin validación. dataclass: record con métodos opcionales. pydantic (no en stdlib): cuando además quieres validación de tipos en runtime, parsing desde JSON, etc. (lo verás en MLOps).

¿Composición > herencia siempre?

Como regla. Usa herencia solo cuando is-a sea genuino (PerroLabrador is-a Perro is-a Animal). Para has-a (Coche tiene un Motor), composición. Para reutilizar comportamiento sin jerarquía, considera mixins o protocols.

¿property y getters/setters Java-style?

En Python no escribes getNombre()/setNombre(). Usa atributo público (self.nombre = ...). Si después necesitas lógica, conviertes a @property sin cambiar el caller. Es la magia.

¿Cuándo __slots__?

Optimización: define los atributos permitidos y ahorra memoria (~50%) al no usar __dict__ por instancia. Útil solo en clases con millones de instancias. Costo: pierde herencia múltiple y dinamismo.

Referencias

- Ramalho, Fluent Python 2e — caps. 5, 11, 14.
- dataclasses docs
- Python Tutorial — Classes

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 011 — Clase 011 — pathlib, lectura y escritura de archivos

Parte: 0 — Prerrequisitos · Fuente: Python Tutorial cap. 10 · pathlib docs · Effective Python (Slatkin) ítem 38.

Duración estimada: 60 min.

Objetivo

Que el alumno deje de usar os.path.join + strings y adopte pathlib.Path — API orientada a objetos, multiplataforma (Windows/Unix), con métodos legibles para todas las operaciones de filesystem que hace todo el tiempo en DS (leer CSV, listar archivos, crear carpetas).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Construir paths con Path(...) / 'subdir' / 'file.csv' (operador /).
2. Leer/escribir archivos texto y binarios con métodos de Path (read_text, write_bytes).
3. Listar y filtrar archivos con iterdir, glob, rglob (recursivo).
4. Crear/eliminar estructuras de directorios sin pelear con os.makedirs(exist_ok=True).
5. Manejar rutas relativas vs absolutas y entender __file__.

Temas

#	Tema	Por qué importa
1	Path vs strings	Objetos con métodos > concatenación manual
2	Operador / para componer	Legible y multiplataforma.
3	read_text / write_text / read_bytes	One-liners para operaciones simples.
4	glob y rglob	Patrones tipo shell: .csv, /.py.
5	makedirs(parents=True, exist_ok=True)	Crea árbol completo idempotente.
6	Path(__file__).parent y resolve()	Localizar recursos relativos al script.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada:

- Clase 032b — Parquet, Arrow, PyArrow, DuckDB

Definiciones y características

Path

: Objeto que representa una ruta de filesystem orientada a objetos (pathlib.Path). Sobreescribe / para componer rutas, multiplataforma (Windows usa \, Unix /, transparente). Tiene métodos para casi todo: leer, escribir, listar, mover, borrar.

Ruta absoluta vs relativa

: Absoluta: empieza desde la raíz (C:\dev\proyecto\data.csv o /home/user/data.csv). Relativa: parte del cwd (data.csv). Las rutas relativas dependen de dónde se ejecuta — fuente de bugs.

Path.cwd() vs Path(__file__).parent

: cwd() es el directorio donde se ejecutó el script (cambia según el invocante). __file__ es la ruta al archivo Python actual; .parent su carpeta. Usa __file__ para recursos que viven junto al script.

glob vs rglob

: Patrones tipo shell. glob('.csv') busca en el directorio actual (1 nivel). rglob('.csv') busca recursivo (todo el árbol). ** significa 'cualquier número de directorios'.

read_text / write_text

: One-liners para texto: Path('x.txt').write_text(contenido, encoding='utf-8'). Para binario: read_bytes / write_bytes. Para JSON/CSV usa librerías especializadas.

Dataset / recursos

Carpeta temporal creada en el notebook con archivos sintéticos. Sin descarga.

Ejercicios

1. Construye una ruta multiplataforma. Dado Path.home() / 'datos' / '2026' / 'enero.csv', imprime cómo se ve en Windows vs Unix.

2. Lista CSVs. En una carpeta con archivos mixtos (.csv, .txt, .py), lista solo los .csv ordenados por tamaño.
3. Búsqueda recursiva. En un árbol de carpetas, encuentra todos los .py que contengan la palabra TODO en su contenido.
4. Escribe + lee. Genera 3 archivos txt con write_text, léelos con read_text, concaténalos en uno solo.
5. Ruta del script. Escribe un script que cargue un dataset que vive al lado del script (no del cwd), usando Path(__file__).parent / 'data.csv'.

Homework verificable

Script inventario.py que recibe un directorio y produce un reporte CSV con: nombre, tamaño_bytes, extensión, última_modificación para cada archivo recursivamente, usando solo pathlib (no os).

Criterio de aceptación: El script corre tanto en Windows como en Linux/macOS sin cambios.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
FileNotFoundError aunque el archivo existe	Estás usando ruta relativa y el cwd no es
PermissionError al escribir	Carpeta read-only, OneDrive sincronizando,
mkdir() falla si el directorio ya existe	Default es exist_ok=False. Fix: p.mkdir(pa
Mezclo os.path y pathlib	Algunos funciones esperan strings (open(),
glob(*.CSV) no encuentra archivo.csv	Case-sensitive en Linux, insensible en Win
ImportError: Missing optional dependency '	pandas delega Parquet/Arrow a pyarrow (o f

Preguntas frecuentes

¿pathlib o os.path?

pathlib para código nuevo. os.path es la API funcional vieja (strings + funciones); pathlib es OO y mucho más legible. Solo usa os.path para compatibilidad con código viejo.

¿Cómo evito el típico 'C:/Users/...' vs '/home/...' cross-platform?

No hardcodees rutas absolutas. Usa Path.home(), Path(__file__).parent, tempfile.gettempdir(). Y siempre Path + /, nunca strings concatenados.

¿Cómo leo un CSV grande con pathlib?

Pathlib es para paths, no parsing. Combina: pd.read_csv(Path('data') / 'big.csv'). El Path se convierte a string automáticamente.

¿Path('a') / 'b/c' o Path('a') / 'b' / 'c'?

Ambas funcionan: Path parsea separadores. Pero la primera es menos explícita; prefiere la segunda.

¿shutil o pathlib para mover/copiar?

shutil para operaciones recursivas (copytree, rmtree, move) — pathlib solo cubre operaciones simples. Combinable: shutil.copy(src_path, dst_path) acepta Path directo.

¿Cuándo dejo de usar CSV?

Regla práctica: CSV solo para intercambio con no-técnicos (alguien lo va a abrir en Excel o mandarlo por mail). Para cualquier dataset >100 MB, o uno que vas a releer más de una vez en tu pipeline, pasalo a

Parquet: ahorrás disco, ganás velocidad de lectura y no perdés tipos. Si lo único que querés es cachear un DataFrame entre dos scripts de la misma máquina, usá Feather.

Referencias

- pathlib docs
- PEP 428 — Object-oriented filesystem paths
- Slatkin, Effective Python 2e — ítem 38 Use Pathlib instead of os.path.
- Apache Arrow / Parquet

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 012 — Clase 012 — Logging

Parte: 0 — Prerrequisitos · Fuente: Python Tutorial logging HOWTO · The Pragmatic Programmer — "Programming by Coincidence".

Duración estimada: 60 min.

Objetivo

Que el alumno deje de usar print para debug y aprenda el módulo logging estándar: niveles (DEBUG/INFO/WARNING/ERROR/CRITICAL), handlers (consola, archivo), formatters, y configuración por módulo. Es la diferencia entre código que se debuggea reiniciando el notebook y código que se debuggea leyendo logs.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar los 5 niveles de logging y cuándo usar cada uno.
2. Configurar un logger con logging.basicConfig y entender por qué basicConfig solo funciona una vez.
3. Crear loggers por módulo con logging.getLogger(__name__).
4. Agregar handlers: uno a consola (INFO+), otro a archivo (DEBUG+).
5. Formatear logs con timestamp, módulo y nivel.

Temas

#	Tema	Por qué importa
1	print vs logging	print() es output; logging es observabilidad
2	Niveles: DEBUG/INFO/WARNING/ERROR/CRITICAL	Filtran qué se ve sin tocar código.
3	Logger jerárquico por módulo	getLogger(__name__) para herencia natural.
4	Handlers: consola, archivo, rotating	Mismo log → múltiples destinos.
5	Formatters	Timestamp + nivel + módulo + mensaje.
6	logging.basicConfig y sus límites	Solo afecta el primero; preferir config ex

Definiciones y características

Logger

: Punto de entrada para emitir logs. Se obtiene con `logging.getLogger(__name__)` — esto crea un logger nombrado por el módulo. Característica clave: los loggers son jerárquicos (separados por `.`); la config de root propaga a hijos.

Handler

: Define a dónde van los logs (consola, archivo, syslog, sentry...). Un logger puede tener N handlers. Cada handler tiene su propio nivel y formatter.

Formatter

: Define cómo se renderiza el log: `'%(asctime)s [%(levelname)s] %(name)s: %(message)s'`. Campos comunes: `asctime`, `levelname`, `name` (logger), `message`, `funcName`, `lineno`, `pathname`.

Nivel (DEBUG/INFO/WARNING/ERROR/CRITICAL)

: Severidad ascendente. Filtran qué se emite. DEBUG: detalle interno; INFO: progreso normal; WARNING: algo raro pero no fatal; ERROR: operación falló; CRITICAL: sistema no puede continuar.

dictConfig

: Forma declarativa de configurar logging desde un dict (o YAML/JSON). Más mantenible que llamadas `basicConfig/addHandler` dispersas. Convención: una sola llamada en el `entrypoint`.

Dataset / recursos

Genera un log file de demo. Sin descarga.

Ejercicios

1. Reemplaza prints. Toma una función con 5 prints y conviértelos a logger con niveles apropiados.
2. Logger por módulo. Crea 2 archivos `.py` que cada uno usa `getLogger(__name__)`. Configura el root logger una vez; verifica que ambos heredan.
3. Handler doble. Configura: consola = INFO+, archivo `app.log` = DEBUG+. Genera 5 logs de niveles distintos y verifica qué aparece en cada destino.
4. Formato con timestamp. Cambia el formato a `'%(asctime)s [%(levelname)s] %(name)s: %(message)s'`. Inspecciona output.
5. Logger en notebook. Pelea con `basicConfig` no recordando estado entre reinicios — usa `dictConfig` o `force=True`.

Homework verificable

Notebook + 2 módulos `.py` que importan y loguean. Un `logging_config.py` con `dictConfig` que define: consola (INFO+, formato corto) y `app.log` (DEBUG+, formato verbose con timestamp). El notebook ejecuta funciones que generan logs de distintos niveles desde ambos módulos. Adjunta el `app.log` resultante.

Criterio de aceptación: `app.log` contiene timestamp y módulo correcto en cada línea; consola filtra DEBUG.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>logging.basicConfig(...)</code> no tiene efecto	<code>basicConfig</code> solo aplica si root no tiene h
Logs duplicados (cada mensaje aparece 2 ve	Configuraste el mismo handler dos veces (c
<code>log.debug(f'valor: {expensive_call()}')</code> si	El f-string se construye antes de pasar a
Mi logger emite en INFO pero quiero ver DE	El nivel está en handler o logger raíz. Fi

logging rompe en multiprocessing

Handlers no son fork-safe. Fix: en cada pr

Preguntas frecuentes

¿print o logging?

logging siempre en código que vivirá >1 día. print solo para REPL/scripts one-shot. Logging te da niveles, timestamps, módulo origen, múltiples destinos, integración con observabilidad.

¿Dónde configuro logging?

Una sola vez en el entrypoint (`__main__`, `app.py`, `cli.py`). Cada módulo solo hace `log = logging.getLogger(__name__)`; nunca llama a `basicConfig/addHandler` desde un módulo importable.

¿Por qué `getLogger(__name__)`?

Crea un logger jerárquico nombrado por el módulo. Permite silenciar uno específico (`logging.getLogger('mi_app.db').setLevel(WARNING)`) sin tocar el resto. Pattern estándar.

¿Cómo formato un dict/object en el mensaje?

`log.info('user=%s data=%s', user_id, data)` (mejor que f-string por el lazy). Para JSON estructurado, usa `python-json-logger` o `stdlib` con `custom formatter`.

¿logging propaga al root logger?

Por default, sí — cada logger propaga al padre hasta root. Si configuras handlers en root y en hijos, verás el mensaje dos veces. Fix: `logger.propagate = False` en el hijo, o solo configura root.

Referencias

- Logging HOWTO
- Logging Cookbook
- `logging.config` — `dictConfig`

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 013 — Clase 013 — Type hints y mypy

Parte: 0 — Prerrequisitos · Fuente: *Fluent Python 2e cap. 8 (Type Hints in Functions)* · *typing docs* · *mypy docs*.

Duración estimada: 75 min.

Objetivo

Que el alumno anote tipos en sus funciones y dataclasses — no por dogma, sino porque permiten que el IDE autocomplete bien, que mypy detecte bugs antes de runtime, y que el lector entienda la intención. Tipos como documentación verificable.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Anotar funciones con tipos en parámetros y retorno (`def f(x: int) -> str`).
2. Usar tipos compuestos: `list[int]`, `dict[str, float]`, `tuple[int, str]`, `Optional[X]`, `X | None`.
3. Definir tipos personalizados con `TypeAlias` y `Protocol` (structural typing).
4. Ejecutar `mypy` sobre código y interpretar sus errores.
5. Reconocer cuándo `type hints` aportan (APIs públicas, data classes) y cuándo no (notebooks exploratorios).

Temas

#	Tema	Por qué importa	
1	Sintaxis básica: <code>x: int, -> bool</code>	Solo anotaciones — no afectan ru	
2	Tipos compuestos modernos (3.9)	Sin <code>from typing import List</code> .	
3	<code>Optional[X]</code> y <code>`X`</code>	<code>None`</code> (3.10+)	Cuando algo puede ser <code>None</code> .
4	<code>Literal</code> , <code>TypedDict</code> , <code>Protocol</code>	Tipos avanzados útiles.	
5	<code>mypy</code> : instalar y correr	Static type checker.	
6	<code>reveal_type(x)</code> y <code># type: ignore</code>	Diagnóstico y escape hatch.	
7	Cuándo Sí y cuándo NO	API pública sí; notebook exploratc	

Definiciones y características

Type hint (annotation)

: Anotación de tipo en signature o variable: `def f(x: int) -> str`. Python NO verifica en runtime — son metadata leída por IDE/linter/mypy. Disponibles en `f.__annotations__`.

`Optional[X]` / `X | None`

: Indica que el valor puede ser `X` o `None`. `Optional[X] ≡ Union[X, None] ≡ X | None` (PEP 604, 3.10+). Usa la sintaxis con `|` en código nuevo.

`TypedDict`

: Esquema para diccionarios: `class PersonaDict(TypedDict): nombre: str; edad: int`. Permite tipear dicts que vienen de JSON/API sin convertirlos a `dataclass`.

`Literal`

: Restringe a un conjunto de valores: `Literal['asc', 'desc']` solo acepta esas 2 strings. Útil para flags, modos, enums simples.

`Protocol` (structural typing)

: Define interfaz por estructura (duck typing tipado): `class TienePromedio(Protocol): def promedio(self) -> float: ....` Cualquier clase con `.promedio()` la satisface, sin heredarla.

`mypy`

: Static type checker oficial. Lee tu código, sigue las anotaciones, reporta inconsistencias antes de ejecutar. Modo `--strict` lo hace exigente; recomendado para librerías.

Dataset / recursos

Funciones de ejemplo en el notebook. Sin descarga.

Ejercicios

1. Anota una función. Toma una función de los ejercicios de clase 008 (sin tipos) y anótala completa.
2. `Optional` vs `default`. Distingue `def f(x: int = 0)` (default 0) de `def f(x: int | None = None)` (puede no haber

valor).

3. TypedDict. Define class PersonaDict(TypedDict) con nombre: str, edad: int. Úsala como tipo de un parámetro.

4. Corre mypy. Instala mypy, créate un archivo con un bug de tipo intencional (def f(x: int) -> str: return x + 1) y corre mypy archivo.py. Lee y explica el error.

5. Protocol. Define class TienePromedio(Protocol) con método promedio() -> float. Acepta cualquier clase que lo implemente (duck typing tipado).

Homework verificable

Repo con un módulo analytics.py (5+ funciones completamente anotadas), pyproject.toml que incluye mypy en [tool.mypy] con strict = true, y screenshot/log de mypy analytics.py sin errores.

Criterio de aceptación: mypy --strict corre sin errores ni warnings. Tipos consistentes y precisos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
from typing import List da deprecation war	Python 3.9+ usa lowercase: list[int] en ve
Optional[int] = 0 confunde	Optional[int] admite None. Si tu default e
mypy se queja "Cannot find module 'libreria"	Lib sin type stubs publicados. Fix: pip in
Anoté pero mypy no encuentra errores	mypy no se llamó. Fix: mypy archivo.py. No
reveal_type(x) no existe en runtime	Es exclusivo de mypy: lo lees como output

Preguntas frecuentes

¿Tipo hints son obligatorios?

No — Python no los verifica. Pero son fuertemente recomendados en: APIs públicas (funciones que importan otros), librerías, código compartido. En notebooks exploratorios, casi nunca aportan.

¿Slow my code los type hints?

No — son metadata, no impactan runtime. from __future__ import annotations además las hace lazy (strings, evaluadas solo si introspeccionas).

¿mypy en CI: estricto o permisivo?

Empieza permisivo (sin --strict), arregla lo obvio, luego activa --strict gradualmente. Si arrancas estricto en un repo viejo, te ahogas en errores y termina ignorado.

¿Y si no sé qué tipo poner?

Any (de typing) es válido — desactiva el check para ese caso. Mejor que mentir con un tipo incorrecto. Convención: comentario # TODO: tipo correcto para revisión futura.

¿Pydantic vs dataclass + type hints?

dataclass + hints: tipos solo a nivel mypy, no en runtime. Pydantic: valida en runtime (parsing de JSON con coerción y errores legibles). Para input externo (API, config) → Pydantic. Para internal record → dataclass.

Referencias

- Ramalho, Fluent Python 2e — cap. 8.

- typing docs
- mypy docs
- PEP 484 — Type Hints
- PEP 604 — X | Y syntax

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 014 — Clase 014 — NumPy: tipos, creación, atributos

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, Python Data Science Handbook, cap. 2 — Introduction to NumPy, §§ 2.1–2.2.

Duración estimada: 75 min.

Objetivo

Que el alumno entienda el modelo mental de un ndarray — bloque contiguo de memoria con shape, dtype y strides — y sepa crear arrays de las 6 formas más útiles (array, zeros, arange, linspace, random, desde lista). Sin este modelo, todo el rendimiento de NumPy parece magia.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar por qué ndarray es 50–100× más rápido que list (memoria contigua + dtype fijo + sin overhead Python).
2. Crear arrays con np.array, np.zeros, np.ones, np.full, np.arange, np.linspace.
3. Inspeccionar un array con shape, dtype, ndim, size, nbytes, itemsize.
4. Cambiar dtype explícitamente con astype y entender promociones implícitas (int + float = float).
5. Generar arrays aleatorios reproducibles con np.random.default_rng(seed).

Temas

#	Tema	Por qué importa
1	ndarray: memoria contigua + dtype fijo	Lo que lo hace rápido.
2	Creación: array, zeros, arange, linspace	Las 6 formas más usadas.
3	dtype: int8/16/32/64, float32/64, bool	Memoria y precisión.
4	Atributos: shape, dtype, ndim, size, nbyte	Diagnóstico instantáneo.
5	astype y promoción de tipos	El bug clásico de overflow int8.
6	random moderno: default_rng(seed)	El API legacy np.random.seed está deprecad

Definiciones y características

ndarray

: Estructura de datos central de NumPy. Bloque contiguo de memoria con dtype fijo + metadata (shape, strides). 50–100× más rápido que list por evitar el overhead de PyObject por elemento y permitir vectorización SIMD.

dtype

: Tipo de elemento del array: int8/16/32/64, uint8 (imágenes RGB), float32/64, bool, complex64/128. Determina memoria por elemento (itemsize) y precisión/rango.

shape

: Tupla con tamaño de cada dimensión: (10,) 1D, (3, 4) matriz, (2, 3, 4) tensor 3D. len(shape) = ndim. prod(shape) = size (total elementos).

strides

: Bytes que el array salta para moverse 1 paso en cada dimensión. Permite vistas eficientes sin copiar memoria (transpose, slicing). Detalle interno pero útil para entender por qué algunas operaciones son gratis.

Generator (random)

: API moderno para aleatoriedad: rng = np.random.default_rng(seed). Reemplaza al legacy np.random.seed() + funciones globales. Características: PCG64 (mejor algoritmo), múltiples generadores independientes, API consistente.

Promoción de tipo

: Cuando operas arrays de dtypes distintos, NumPy promueve al más amplio: int + float = float, int8 + int16 = int16. La regla evita pérdida silenciosa pero puede generar overflow en otros casos (clase: overflow int8).

Dataset / recursos

Sintético: arrays generados en el notebook (escalares, matrices, aleatorios reproducibles). Sin descarga.

Ejercicios

1. Memoria. Crea list(range(1_000_000)) y np.arange(1_000_000). Compara sys.getsizeof y arr.nbytes. Calcula el ratio.
2. Las 6 formas. Crea: vector 100 ceros, matriz 5×5 unos, vector 0..1 con 50 puntos equiespaciados, matriz 3×3 de 7s, vector de 100 aleatorios uniformes [0,1).
3. Bug de dtype. Crea np.array([100, 200, 50], dtype=np.int8) y suma 200 a cada elemento. Observa el resultado y explica.
4. Diagnóstico. Dado un array, escribe una función que imprima shape, dtype, ndim, size, nbytes y memoria humana (KB/MB).
5. Random reproducible. Genera 1000 normales N(0,1) con seed=42. Calcula media y std. Repite — debe dar exactamente lo mismo.

Homework verificable

Notebook que: (a) compara memoria list vs ndarray para N=1M con tabla; (b) crea las 6 formas y reporta dtype default de cada una; (c) reproduce el bug de overflow int8 con explicación; (d) función info(arr) con diagnóstico completo.

Criterio de aceptación: El ratio memoria list/ndarray es >5×. La función info reporta todos los atributos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OverflowError silencioso con int8/uint8	NumPy no lanza excepción — hace wrap-around
np.array([1, 2, 'x']) queda como dtype='<U'	NumPy promueve TODO a string para ser homo
np.empty(3) con basura en vez de ceros	empty NO inicializa — más rápido que zeros

Reproduzco un experimento con seed y da re	Estás usando el API legacy (np.random.seed
np.arange(0.1, 1.0, 0.1) no incluye 1.0	Floats — el último step da 0.9999... por i

Preguntas frecuentes

¿np.array([1,2,3]) o np.asarray([1,2,3])?

asarray no copia si ya es ndarray (más eficiente); array siempre copia por default. Usa asarray cuando aceptas cualquier 'array-like' y no necesitas garantizar copia.

¿float32 o float64?

Default es float64 — máxima precisión. float32 cuando trabajas con redes neuronales en GPU (la mitad de memoria, suficiente para gradientes), imágenes, o necesitas duplicar la velocidad de IO.

¿Cuándo int32 y cuándo int64?

Default depende del OS (int64 Unix/macOS, int32 Windows pre-numpy 2.0). En 2026, NumPy 2+ usa int64 en todas las plataformas. Solo bajas a int32 si memoria es crítica y sabes que tus valores caben.

¿np.random.seed(42) ya no se usa?

Funciona pero está deprecated en favor del nuevo API. Razones: estado global (peligroso), Mersenne Twister (lento), no permite múltiples streams independientes. Usa default_rng(seed).

¿Por qué arr.nbytes no coincide con sys.getsizeof(arr)?

nbytes cuenta solo los datos (size * itemsize). getsizeof cuenta también el header del objeto ndarray (~100 bytes). Para arrays grandes la diferencia es despreciable.

Referencias

- VanderPlas, cap. 2, §§ 2.1–2.2 Understanding Data Types + The Basics of NumPy Arrays.
- NumPy user guide — Array creation
- NumPy dtypes
- Generator random API

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 015 — Clase 015 — NumPy: ufuncs y vectorización

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2, § 2.3 Computation on NumPy Arrays: Universal Functions.

Duración estimada: 75 min.

Objetivo

Que el alumno abandone los for loops sobre arrays NumPy y use ufuncs (universal functions) para operaciones elementwise — la fuente real del speedup. Ufuncs son C compilado vectorizado; un for Python sobre array es lo peor de ambos mundos.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Identificar una ufunc (np.add, np.multiply, np.sin, np.exp, np.log, comparadores).
2. Reemplazar un for+append por una expresión vectorizada y medir el speedup.
3. Usar el parámetro out= para escribir el resultado in-place (evita allocar memoria extra).
4. Combinar ufuncs con operadores aritméticos (+, -, /, *).
5. Reconocer las trampas de la vectorización (overflow, NaN propagación, división por cero).

Temas

#	Tema	Por qué importa
1	¿Qué es una ufunc?	Función C vectorizada elementwise.
2	Ufuncs unarias y binarias	np.exp(x) vs np.add(x, y).
3	Operadores → ufuncs	$a + b \equiv \text{np.add}(a, b)$.
4	out= para in-place	Memoria $O(1)$ extra.
5	Trampas: overflow, NaN, inf, división por	NumPy avisa pero no para.
6	np.where(cond, a, b)	Ternario vectorizado.

Definiciones y características

Ufunc (universal function)

: Función NumPy implementada en C que opera elementwise y vectorizada (SIMD cuando posible). Características: rápida (10-100× vs Python), broadcasting automático, soporta out= para in-place.

Vectorización

: Operar sobre arrays completos en vez de loops Python: `arr * 2` en vez de `[x*2 for x in arr]`. La operación corre en C compilado sobre memoria contigua, sin overhead del intérprete por elemento.

In-place (out=)

: Escribir el resultado de una ufunc en un array existente, sin allocar memoria nueva: `np.multiply(a, 2, out=a)`. Útil con arrays grandes donde la copia temporal duplicaría la memoria pico.

Propagación de NaN

: Cualquier operación que tenga NaN como input produce NaN. `np.array([1, np.nan, 3]).sum()` → nan. Para ignorar usa variantes nan*: `nansum`, `nanmean`, `nanmedian`.

`np.where(cond, a, b)`

: Ternario vectorizado: para cada elemento, si cond es True usa a, si no usa b. Equivale a `[a if c else b for c, a, b in zip(cond, a, b)]` pero ~100× más rápido.

Dataset / recursos

Sintético: arrays grandes para benchmark. Sin descarga.

Ejercicios

1. Benchmark. Calcula `[xx + 2x + 1 for x in range(1_000_000)]` vs `arr*2 + 1`. Mide con `%timeit`.
2. Logaritmo y exponencial. Con `np.exp` y `np.log`, verifica que `log(exp(x)) ≈ x` para 1000 valores. Reporta el error máximo.
3. In-place vs alloc. `arr = arr * 2 + 1` vs `np.multiply(arr, 2, out=arr); np.add(arr, 1, out=arr)`. Compara

tracemalloc.

4. np.where ternario. Dado un array de notas, crea otro array con 'aprobado' si nota ≥ 4 , 'reprobado' si no.

5. Trampa NaN. Crea np.array([1, 2, np.nan, 4]).sum() y .mean(). Compara con np.nansum y np.nanmean.

Homework verificable

Notebook: (a) reescribe 3 loops como expresiones vectorizadas + tabla con %timeit (3 N distintos); (b) demuestra out= con tracemalloc; (c) usa np.where para clasificar datos; (d) maneja NaN con nansum/nanmean y compara con propagación.

Criterio de aceptación: Speedup $>50\times$ en $N=1M$. out= muestra memoria ≈ 0 extra. NaN-handling correcto.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
for i in range(len(arr)): arr[i] = ... es	Loops Python sobre array NumPy = lo peor d
RuntimeWarning: divide by zero / invalid v	NumPy avisa pero no para: $1/0 \rightarrow \text{inf}$, $0/0 \rightarrow$
out= con dtype incompatible	np.add(int_arr, 0.5, out=int_arr) falla —
Resultado de np.where no es lo esperado	Los 3 args se evalúan completos: np.where(
arr.sum() da NaN y no sé por qué	Hay un NaN escondido en el array. Fix: pri

Preguntas frecuentes

¿Cuánto más rápido es vectorizar?

Típicamente 50-100 $\times$ para arrays de 1M elementos. Para arrays pequeños (<100), la ganancia es menor o nula (overhead constante). Mide con %timeit, no asumas.

¿arr + 1 o np.add(arr, 1)?

Equivalentes. Operadores son sintaxis dulce sobre ufuncs. Usa np.add(...) cuando necesitas out= (in-place) o where= (mask).

¿NumPy aprovecha mi GPU?

No — solo CPU. Para GPU: CuPy (drop-in replacement), PyTorch tensors, JAX. NumPy 2 está mejorando vectorización CPU (SIMD wider, BLAS) pero sigue siendo CPU.

¿Por qué arr ** 2 es más rápido que arr * arr?*

Suelen empatar (** también es ufunc). Para potencias enteras pequeñas (2, 3), NumPy a veces usa atajos. Mide con %timeit en tu caso específico.

¿np.where o boolean mask?

Mask (arr[cond] = valor) si vas a modificar in-place o filtrar (arr[arr>0]). np.where(cond, a, b) si necesitas un array nuevo con dos valores posibles según condición.

Referencias

- VanderPlas, cap. 2 § 2.3 Computation on NumPy Arrays.
- NumPy ufuncs reference

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.

- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 016 — Clase 016 — NumPy: agregaciones

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2 § 2.4 Aggregations: Min, Max, and Everything in Between.

Duración estimada: 60 min.

Objetivo

Que el alumno reduzca arrays a estadísticos (sum, mean, std, percentile, min, max) controlando el axis correcto — la fuente del 50% de los bugs de pandas/sklearn cuando alguien se confunde de eje. También: variantes nan* y reducciones acumulativas.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Calcular sum, mean, std, var, median, percentile sobre arrays.
2. Controlar el eje con axis=0 (a lo largo de filas, da resultado por columna) y axis=1 (a lo largo de columnas, da por fila).
3. Usar variantes nan* (nansum, nanmean, etc.) cuando hay datos faltantes.
4. Reducciones acumulativas con cumsum y cumprod.
5. Encontrar índice del min/max con argmin/argmax.

Temas

#	Tema	Por qué importa
1	Reducciones básicas	sum, mean, std, var, min, max, median, per
2	Eje: el bug más común	axis=0 reduce filas (resultado por columna
3	Variantes NaN-aware	nansum, nanmean, nanmedian, nanstd.
4	Acumulativas	cumsum, cumprod — útiles para series tempo
5	argmin/argmax	Posición del extremo.
6	all y any	Reducciones booleanas.

Definiciones y características

Agregación / reducción

: Operación que colapsa un array a menos dimensiones: sum, mean, std, min, max, argmax. Sin axis, reduce a un escalar; con axis=N, elimina la dimensión N.

axis=0 vs axis=1

: Regla: el axis que pasas es el que desaparece. En matriz (filas, cols): axis=0 colapsa filas → un valor por columna; axis=1 colapsa cols → un valor por fila. Mnemónico inverso al que muchos esperan.

argmin / argmax

: Devuelven el índice del extremo (no el valor). arr.argmax() = posición del máximo; arr[arr.argmax()] = valor máximo. Con axis= devuelven array de índices por fila/columna.

Variantes nan*

: Versiones que ignoran NaN en vez de propagarlo: `nansum`, `nanmean`, `nanstd`, `nanmin`, `nanmax`, `nanmedian`, `nanpercentile`, `nanargmax`. Útiles cuando los datos tienen missing.

Acumulativas (`cumsum`, `cumprod`)

: No colapsan — devuelven array de igual shape con valores acumulados hasta cada posición. Útiles para series temporales (precio acumulado, drawdown).

`percentile` / `quantile`

: Valor por debajo del cual cae el N% de los datos. `np.percentile(arr, 50)` = mediana. `np.percentile(arr, [25, 50, 75])` = cuartiles.

Dataset / recursos

Sintético (matriz aleatoria 100×10 simulando "10 features × 100 muestras") generado en el notebook. Sin descarga.

Ejercicios

1. Promedio por columna. Dada matriz 100×4 de ventas (filas=día, cols=tienda), calcula la media por tienda y por día.
2. Estadísticos completos. Para un array de 1000 normales, reporta mean, std, median, p25, p75, min, max.
3. Con NaN. Inserta 50 NaN aleatorios en el array anterior. Compara mean (propaga) vs nanmean.
4. Cumsum. Genera array de retornos diarios aleatorios. Calcula el precio acumulado con `cumprod(1+r)`.
5. Mejor tienda. Con la matriz del ejercicio 1, usa `argmax(axis=0)` para encontrar el día de mayor venta de cada tienda.

Homework verificable

Notebook con matriz simulada 365 días × 5 tiendas de ventas, reportando: media/std por tienda, mejor y peor día de cada tienda, `cumsum` total anual, % de días con NaN simulados (20 aleatorios) usando variantes `nan*`.

Criterio de aceptación: Eje correcto en todas las agregaciones; valores reproducibles con seed.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>M.sum(axis=0)</code> da resultado por columna y e	Confusión clásica. Regla: <code>axis=0</code> reduce fi
<code>arr.mean()</code> da NaN aunque solo hay un par d	Cualquier NaN propaga. Fix: <code>np.nanmean(arr</code>
<code>np.argmax(matriz)</code> devuelve un solo número	Sin axis, aplana el array primero y devuel
<code>np.percentile([1,2,3,4], 50)</code> da 2.5 no 2	Interpolación lineal por default. Si quier
<code>cumsum</code> de floats acumula error de redondeo	Sumas múltiples introducen error numérico.

Preguntas frecuentes

¿`arr.sum()` o `np.sum(arr)`?

Equivalentes. Método del array (`.sum()`) es más legible en cadenas (`arr.clip(0).sum()`). Función (`np.sum`) acepta también listas (no solo ndarray).

¿Cómo recuerdo qué axis colapsa cuál?

El axis que pasas es el que desaparece. Si shape es (3, 4) y haces `sum(axis=0)`, queda (4,) — desapareció

dim 0. Si axis=1, queda (3,).

¿std usa N o N-1?

Default ddof=0 (divide por N — desviación poblacional). Para desviación muestral (N-1), arr.std(ddof=1). Pandas y scipy.stats usan N-1 por default — cuidado al comparar.

¿np.median ignora NaN?

No — usa np.nanmedian. Mismo patrón que mean/nanmean.

¿Hay una agregación 'top-3' built-in?

No directa. Usa np.partition(arr, -3)[-3:] para los 3 mayores (más rápido que sort completo). Si necesitas ordenados, .sort() después.

Referencias

- VanderPlas, cap. 2 § 2.4.
- NumPy statistics functions

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 017 — Clase 017 — NumPy: broadcasting

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2 § 2.5 Computation on Arrays: Broadcasting.

Duración estimada: 75 min.

Objetivo

Que el alumno internalice las reglas de broadcasting — el mecanismo por el que NumPy operó arrays de shapes distintos sin copiar datos. Es lo que hace que `M - M.mean(axis=0)` centrado por columna sea una línea, no un bucle anidado.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Recitar las 3 reglas de broadcasting (alinear por la derecha, dim 1 estira, falla si no es 1 ni igual).
2. Predecir la shape del resultado de una operación entre arrays de shapes distintos.
3. Centrar y escalar matrices por fila/columna sin loops.
4. Usar `np.newaxis` (o `None`) para promover un vector a matriz fila/columna.
5. Diagnosticar un `ValueError`: operands could not be broadcast together leyendo las shapes.

Temas

#	Tema	Por qué importa
1	Las 3 reglas	Padding a la derecha, dim 1 estira, error
2	Vector + matriz	Vector como fila o como columna.
3	<code>np.newaxis</code> / <code>None</code>	Insertar eje de tamaño 1.
4	Caso canónico: centrar/escalar	$X - X.\text{mean}(\text{axis}=0)$ y $(X - \mu) / \sigma$.

5	Outer product sin loop	<code>a[:, None] * b[None, :].</code>
6	ValueError común: "operands could not be b	Cómo leerlo.

Definiciones y características

Broadcasting

: Mecanismo por el que NumPy opera arrays de shapes distintos sin copiar memoria, estirando virtualmente las dimensiones de tamaño 1. Lo que hace posible `X - X.mean(axis=0)` (centrado por columna) en una línea sin loops.

Regla 1 — padding por la izquierda

: Si los arrays tienen distinta cantidad de dimensiones, la shape del menor se rellena con 1s a la izquierda. (4,) operado con (3, 4) se trata como (1, 4) vs (3, 4).

Regla 2 — estirar dim 1

: En cada dimensión donde los tamaños difieren, si uno es 1 se estira al otro. (3, 1) y (3, 4) → ambos (3, 4) (la primera se estira en eje 1).

Regla 3 — fallo

: Si en alguna dimensión los tamaños son distintos y ninguno es 1, lanza `ValueError: operands could not be broadcast together`. No hay forma de inferir qué hacer.

`np.newaxis` (alias `None`)

: Inserta una dimensión de tamaño 1 donde lo pongas. `v[:, None]` convierte vector (3,) en columna (3, 1). Crítico para forzar broadcasting en la dirección correcta.

Outer product vía broadcasting

: `a[:, None] * b[None, :]` produce matriz (`len(a)`, `len(b)`) con todos los productos par a par — equivalente a `np.outer(a, b)` pero usando broadcasting puro.

Dataset / recursos

Sintético: matriz de features 100×5 para estandarización. Sin descarga.

Ejercicios

1. Predice antes de ejecutar. Para shapes (3,), (3,1), (1,3), (2,3,4) × (4,), predice la shape del resultado. Verifica.
2. Estandariza features. Matriz 100×5 aleatoria. Resta media por columna y divide por std por columna en una línea.
3. Outer product. Vectores `a=[1,2,3]`, `b=[10,20,30,40]`. Calcula la matriz outer (3×4) sin `np.outer`, solo broadcasting.
4. Distance matrix. Dados 5 puntos 2D, construye matriz 5×5 de distancias euclídeas entre pares — sin `cdist`, solo broadcasting.
5. Diagnostica error. Intenta `np.ones((3,4)) + np.ones((4,3))`. Lee el `ValueError` y explica.

Homework verificable

Notebook que: (a) predice shapes de 4 operaciones broadcasting y verifica; (b) estandariza una matriz feature por columna en una línea; (c) construye distance matrix de 100 puntos sin loop; (d) provoca y explica un error de broadcasting.

Criterio de aceptación: Las predicciones coinciden. Estandarización: media $\approx$ 0, std $\approx$ 1 por columna.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ValueError: operands could not be broadcas	Las shapes alineadas por la derecha no son
Resté M.mean(axis=0) y los promedios queda	mean(axis=0) devuelve shape (n_cols,) — se
a + b con shapes (3,) y (3,) da escalar (s	No — da array (3,) elementwise. Producto p
Memoria explota en una operación 'inocente	X[:, None, :] - X[None, :, :] produce arra
a[None] + b no se broadcastea como espero	a[None] añade dim al inicio. Quizás quería

Preguntas frecuentes

¿Cómo predigo la shape del resultado?

(1) Alinea las shapes por la derecha. (2) En cada columna alineada: si son iguales o uno es 1, OK; si no, error. (3) Resultado: por dimensión, toma el max de las dos.

¿Broadcasting copia memoria?

No — es virtual. NumPy itera con strides 0 en las dims estiradas. Por eso es tan eficiente: cero alloc extra (excepto el array resultado).

¿X - X.mean(0) o X - X.mean(0, keepdims=True)?

Para 2D ambos funcionan (broadcasting alinea). En 3D+, keepdims=True preserva la dimensión como 1 y evita confusiones. Recomendado en general.

¿np.newaxis o None?

Alias — arr[:, None] y arr[:, np.newaxis] son idénticos. None es más conciso; muchos prefieren np.newaxis por explicitud.

¿Qué hago si broadcasting no me sirve?

Operaciones que no se ajustan a las reglas: usa np.einsum (más expresivo), np.tensordot, o reshape explícito. Como último recurso, loop Python — pero busca librería específica antes.

Referencias

- VanderPlas, cap. 2 § 2.5 Broadcasting.
- NumPy broadcasting docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 018 — Clase 018 — NumPy: boolean masks y fancy indexing

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2, §§ 2.6–2.7 Comparisons, Masks, and Boolean Logic + Fancy Indexing.

Duración estimada: 75 min.

Objetivo

Que el alumno seleccione, filtre y modifique sub-arrays de tres formas: slicing (visto), máscaras booleanas (`arr[arr > 0]`) y fancy indexing (`arr[[0, 3, 5]]`). Saber cuál devuelve vista vs copia y cuándo cada uno es la herramienta correcta.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Filtrar elementos con máscaras booleanas: `arr[arr > 0]`, `arr[(a > 0) & (a < 10)]`.
2. Combinar máscaras con `&`, `|`, `~` — NO con `and/or` (no vectorizan).
3. Seleccionar por índices con fancy indexing: `arr[[0, 3, 5]]` o `arr[idx_array]`.
4. Modificar in-place con máscara: `arr[arr < 0] = 0` (clipping).
5. Diferenciar vista vs copia: slicing es vista; fancy indexing y máscara son copia.

Temas

#	Tema	Por qué importa
1	Comparaciones elementwise → <code>a > b</code>	<code>arr > 0</code> no devuelve un bool, devuelve un array de bools.
2	<code>np.count_nonzero</code> , <code>np.sum</code> sobre arrays de bools	Cuenta cuántos True.
3	Combinar máscaras con <code>&</code> , <code> </code> , <code>~</code>	Operadores bitwise — no <code>and/or</code> .
4	Fancy indexing con array de índices	Selección no contigua.
5	Vista vs copia	Slicing = vista; mask/fancy = copia
6	<code>np.where(cond)</code> (sin alternativas)	Devuelve índices donde se cumple la condición.

Definiciones y características

Boolean mask

: Array de bools del mismo shape que el original. `arr[mask]` extrae solo los elementos donde `mask` es True. Devuelve un nuevo array (copia, no vista).

Fancy indexing

: Indexar con array de enteros (índices arbitrarios, posiblemente no contiguos). `arr[[0, 3, 5]]` selecciona esas 3 posiciones. Devuelve copia.

Vista (view) vs copia (copy)

: Slicing (`arr[:5]`) → vista (mismo storage, mutarla muta el original). Mask / fancy → copia (storage independiente). Fuente del 70% de los bugs sutiles.

Operadores bitwise vs lógicos

: Para combinar masks: `&`, `|`, `~` (bitwise, vectorizados, elementwise). NO uses `and`, `or`, `not` — son escalares Python y dan `ValueError: truth value of an array is ambiguous`.

`np.where(cond)` 1-arg

: Sin alternativas, devuelve tupla de arrays de índices donde se cumple la condición. Diferente a `np.where(cond, a, b)` (ternario).

Dataset / recursos

Sintético: array de precipitación diaria (365 valores). Sin descarga.

Ejercicios

1. Cuenta días lluviosos. Dado array de 365 días con precipitación (mm), cuenta cuántos tuvieron >5mm.
2. Estadísticos por máscara. Calcula precipitación media solo en días lluviosos (>0mm).
3. AND/OR combinados. Días entre 1 y 10 mm. Días <1 o >50 mm.
4. Clipping. Reemplaza valores negativos por 0 in-place (`arr[arr < 0] = 0`).
5. Vista vs copia. Demuestra con un experimento que `arr[:5]` modifica el original pero `arr[arr > 0]` no.

Homework verificable

Notebook con array sintético de 365 días de precipitación generado con seed. Calcula: (a) días lluviosos y su media, (b) días extremos (>50mm), (c) demo de vista vs copia, (d) clipping in-place, (e) índices del top 10 días más lluviosos con `argsort`.

Criterio de aceptación: Resultados reproducibles. Demo vista/copia muestra comportamiento opuesto.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar	
ValueError: The truth value of an array wi	Usaste and/or/not con arrays. Fix: <code>&/'</code>	<code>/~</code> con paréntesis: <code>(a > 0) & (a < 10)</code> (lo
Modifico <code>arr[mask]</code> y el original no cambia	Mask devuelve copia. Fix: para modificar i	
<code>arr[idx1, idx2]</code> con fancy indexing me da a	Si <code>idx1</code> e <code>idx2</code> son arrays del mismo length	
Slice modifica el original sin querer	<code>subset = arr[:10]</code> ; <code>subset[0] = 99</code> modifica	
Mask con shape distinto al array	Lanza <code>IndexError: boolean index did not ma</code>	

Preguntas frecuentes

¿Mask o fancy indexing?

Mask cuando la selección viene de una condición sobre los valores (`arr[arr > 0]`). Fancy cuando ya tienes los índices específicos (de un `argsort`, por ejemplo, o de business logic).

¿Cómo modifico múltiples elementos con mask?

`arr[arr < 0] = 0` (clipping). Funciona para asignar escalar a todos los True, o array del mismo tamaño que la cantidad de Trues: `arr[arr < 0] = -arr[arr < 0]` (abs solo donde negativo).

¿`arr[idx]` con `idx` como booleano o entero?

NumPy distingue: bool array del mismo shape → mask; int array → fancy indexing. Lista Python de bools también funciona, pero ojo con `[0, 1, 0, 1]` que puede interpretarse como ints (índices 0 y 1) o bools — usa `np.array(...)` explícito si hay duda.

¿`np.where(cond)` o `np.nonzero(cond)`?

Idénticos cuando `where` se llama con un solo argumento. `nonzero` es más explícito del intent ("dónde NO es 0/False").

¿Cómo combino mask con `np.where` ternario?

`np.where(cond, valor_si_true, valor_si_false)` — ternario vectorizado. La diferencia con `arr[cond] = X`: `where` construye array nuevo; mask + asignación modifica in-place.

Referencias

- VanderPlas, cap. 2 §§ 2.6, 2.7.

- NumPy indexing user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábralo desde el laboratorio del programa o desde Jupyter.

Clase 019 — Clase 019 — NumPy: ordenamiento y búsqueda

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2 § 2.8 Sorting Arrays · NumPy sorting reference.

Duración estimada: 60 min.

Objetivo

Que el alumno ordene arrays con criterio: sort vs argsort, ordenamiento por eje, partial sort con partition, y búsqueda binaria con searchsorted. Útil para top-K, rankings, alineación de series.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Ordenar con `np.sort(arr)` (devuelve copia) y `arr.sort()` (in-place).
2. Obtener índices del orden con `argsort` — base de top-K y rankings.
3. Ordenar por eje en matrices con `axis=0` o `axis=1`.
4. Top-K eficiente con `np.partition` (no ordena completo, solo separa).
5. Búsqueda binaria con `np.searchsorted` en arrays ordenados ($O(\log n)$).

Temas

#	Tema	Por qué importa
1	<code>np.sort</code> vs <code>arr.sort()</code>	Copia vs in-place.
2	<code>argsort</code> : el truco del top-K	Índices que ordenarían el array.
3	Ordenamiento por eje	Por fila o por columna.
4	<code>np.partition</code> para top-K	Más rápido que sort completo.
5	<code>np.searchsorted</code> — binaria $O(\log n)$	Inserción en array ordenado.
6	<code>np.unique</code>	Únicos + opcionalmente cuentas.

Definiciones y características

`np.sort` vs `arr.sort()`

: `np.sort(arr)` devuelve copia ordenada (no muta). `arr.sort()` ordena in-place y retorna `None`. Mismo patrón que `sorted(list)` vs `list.sort()`.

`argsort`

: Devuelve los índices que ordenarían el array. `idx = arr.argsort()`; `ordenado = arr[idx]`. Base de top-K, rankings, alineación entre arrays correlacionados.

`np.partition`

: Quick-select $O(n)$: garantiza que los K menores quedan en las primeras K posiciones (no necesariamente ordenados entre sí), el resto después. Mucho más rápido que sort completo cuando solo necesitas top-K.

`np.searchsorted`

: Búsqueda binaria $O(\log n)$ en array ordenado. Devuelve el índice donde insertar un valor para mantener orden. Útil para calcular percentiles, bucketing.

`np.unique`

: Devuelve únicos ordenados. Con `return_counts=True` devuelve también las frecuencias — alternativa rápida a Counter para datos numéricos.

Dataset / recursos

Sintético: puntajes de 1M estudiantes. Sin descarga.

Ejercicios

1. Top-10. Dado array de 1M puntajes, obtén los 10 más altos. Compara `np.sort()[-10:]` vs `np.partition`.
2. Ranking. Con `argsort`, asigna a cada estudiante su ranking (1 = mejor).
3. Ordena matriz por columna. Matriz 10×5 ; ordena cada columna por su valor.
4. Mediana por bisect. Implementa una función que dado un valor `v` y un array ordenado, devuelve su posición percentil usando `searchsorted`.
5. `np.unique` con cuentas. Dado array de categorías, obtén valores únicos y sus frecuencias.

Homework verificable

Notebook con array de 100k puntajes: (a) top-100 con `partition` y benchmark vs sort completo; (b) ranking con `argsort.argsort()`; (c) percentil de un valor dado con `searchsorted`; (d) `unique` con `return_counts` y `barplot` top-10 categorías.

Criterio de aceptación: `partition` $>10\times$ más rápido que sort completo para $N=100k$ y $K=100$.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Ordeno con <code>arr.sort()</code> y la variable queda	<code>sort()</code> es in-place — modifica <code>arr</code> y retorn
Top-K con <code>sort()[-K:]</code> es lento para N gran	Sort completo es $O(N \log N)$. Fix: <code>np.partition</code>
<code>argsort</code> da resultado raro con matriz	Sin axis, ordena cada fila/columna indepen
<code>np.searchsorted</code> da índice fuera del array	Si el valor es mayor que todos, devuelve <code>l</code>
<code>np.unique(arr_2d)</code> aplana el array	Por default, <code>unique</code> trabaja sobre array ap

Preguntas frecuentes

¿Cuándo `argsort` y cuándo `sort`?

Si solo necesitas los valores ordenados, `sort`. Si necesitas el orden para aplicarlo a otros arrays correlacionados (ej: ordenar nombres por puntajes), `argsort` te da los índices.

¿`partition` o `heapq.nlargest`?

`partition` para arrays NumPy (vectorizado, C). `heapq.nlargest(K, lst)` para listas Python. Para K muy pequeño (3-5) sobre N grande, similares; `partition` gana en arrays grandes.

¿Cómo ordeno por múltiples claves (lexicográfico)?

`np.lexsort([clave_secundaria, clave_principal])` — devuelve índices. Atención: el orden es al revés (último

argumento = key primaria).

¿np.unique preserva el orden de primera aparición?

No — siempre ordena. Para preservar orden de aparición: `_, idx = np.unique(arr, return_index=True); arr[np.sort(idx)]`.

¿Existe equivalente a SQL ORDER BY x DESC?

`arr[arr.argsort()][:-1]` o `arr[arr.argsort()][:-1]`. NumPy no tiene flag `reverse=` como `sorted()` Python — invierte tú.

Referencias

- VanderPlas, cap. 2 § 2.8.
- NumPy sorting reference

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 020 — Clase 020 — NumPy: álgebra lineal con `numpy.linalg`

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 2 § 2.9 Structured Arrays (referencia) · Numerical Linear Algebra (Trefethen & Bau).

Duración estimada: 90 min.

Objetivo

Que el alumno opere con vectores y matrices al nivel necesario para entender ML: producto punto, multiplicación matricial, inversa, sistema de ecuaciones (solve), descomposiciones (SVD, eigen). Saber cuándo no usar la inversa (lentitud + inestabilidad numérica).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Multiplicar vectores y matrices con `@` (operador moderno) y `np.dot`.
2. Resolver sistemas $Ax = b$ con `np.linalg.solve` (NO con `inv(A) @ b`).
3. Calcular norma, determinante, rango, traza.
4. Computar SVD con `np.linalg.svd` y entender qué retorna.
5. Calcular eigenvalores/eigenvectores con `np.linalg.eig` / `eigh` (simétrica).

Temas

#	Tema	Por qué importa
1	<code>@</code> operador (PEP 465): multiplicación matricial	Reemplaza <code>np.matmul</code> .
2	Producto punto vs producto matricial	Vector-vector vs matriz-matriz.
3	Resolver sistemas: solve vs inv	Por qué NUNCA usar inv.
4	Norma, det, rank, trace	Diagnóstico estructural de matrices.
5	SVD — la factorización universal	Base de PCA, regresión lineal, recomendado

6

Eigen

Base de PCA conceptual.

Definiciones y características

Operador @

: Multiplicación matricial (PEP 465). $A @ B \equiv \text{np.matmul}(A, B) \equiv A.\text{dot}(B)$. NO confundir con $*$ (elementwise). Disponible Python 3.5+.

Producto punto vs producto matricial

: Punto (vector·vector $\rightarrow$ escalar): $a @ b = \text{sum}(a*b)$. Matricial (matriz @ matriz $\rightarrow$ matriz): regla "fila por columna". Shapes: $(m,n) @ (n,p) \rightarrow (m,p)$.

`np.linalg.solve(A, b)`

: Resuelve $Ax = b$ usando descomposición LU. Siempre preferible a $\text{inv}(A) @ b$: más rápido (no construye inversa), más estable numéricamente, menos memoria.

SVD (Singular Value Decomposition)

: Descomposición universal: $M = U \cdot \Sigma \cdot V^T$. U y V son ortogonales; Σ diagonal con valores singulares decrecientes ≥ 0 . Base de PCA, recomendadores (matrix factorization), compresión, pseudo-inversa.

Eigen (eigenvalues/eigenvectors)

: Para A cuadrada, $A v = \lambda v$. `eig` general; `eigh` para matrices simétricas (más rápido, garantiza eigenvalores reales). Base de PCA conceptual.

Número de condición (cond)

: Ratio entre el valor singular más grande y el más pequeño. Mide sensibilidad de la solución a perturbaciones. $\text{cond} > 1e10$ matriz mal condicionada, solve perderá precisión.

Dataset / recursos

Sintético: matrices y vectores para los ejercicios. Sin descarga.

Ejercicios

1. Producto punto. Dados dos vectores 100-dim aleatorios, calcula `np.dot(a, b)` y verifica que coincide con `sum(a*b)`.
2. Multiplicación matricial. $(50, 30) @ (30, 20) \rightarrow (50, 20)$. Verifica shapes y un elemento manualmente.
3. Resuelve sistema. Genera $A = (5,5)$ aleatoria, $b = (5,)$, resuelve $Ax = b$ con `solve`. Verifica $A @ x \approx b$.
4. Inv vs solve benchmark. Para A (1000,1000) y b (1000,), mide tiempo de $\text{inv}(A) @ b$ vs `solve(A, b)`. Reporta speedup.
5. SVD de matriz baja rank. Crea $M = u @ v.T$ (rank 1). Calcula SVD y observa que solo el primer valor singular es no-cero.

Homework verifiable

Notebook que: (a) compara $\text{inv}(A) @ b$ vs `solve(A, b)` en tiempo Y precisión (`np.allclose`); (b) implementa regresión lineal cerrada $\beta = (X^T X)^{-1} X^T y$ y luego con `solve`; (c) calcula SVD de una matriz y verifica $M = U @ \text{diag}(s) @ V^T$; (d) eigen de matriz de covarianza.

Criterio de aceptación: `solve` más rápido y más preciso que `inv`. SVD reconstruye la matriz dentro de tolerancia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
A @ B falla con ValueError: shapes ... not	Las dimensiones internas no coinciden: (m,
Implementé <code>inv(A) @ b</code> y los resultados son	<code>inv</code> es inestable numéricamente para <code>matric</code>
<code>np.linalg.solve</code> lanza <code>LinAlgError: Singula</code>	$\det(A) \approx 0$ — el sistema no tiene solución
A * B da resultado raro y esperaba A @ B	Operador * es elementwise (Hadamard produc
SVD devuelve <code>Vt</code> no <code>V</code>	Por convención NumPy devuelve V^T (transpu

Preguntas frecuentes

¿@, `np.matmul`, o `np.dot`?

Para `matriz×matriz` son equivalentes — @ es el más legible. `np.dot` tiene comportamiento distinto para arrays >2D (no broadcasting); @ y `matmul` sí. Usa @ siempre que puedas.

¿`eig` o `eigh`?

`eigh` si la matriz es simétrica (covarianza, kernel matrices, métrica de distancias). Más rápido, garantiza eigenvalores reales. `eig` para matrices generales (no simétricas) — puede dar valores complejos.

¿Por qué `np.linalg.det` para chequear singularidad es mala idea?

El determinante es 0 o no-0 sin gradiente útil — para matrices grandes, `det` puede ser tan chico/grande que cause underflow/overflow numérico. Mejor: `np.linalg.cond(A)` — si $> 1e10$, problemática.

¿Cuándo necesito BLAS/LAPACK?

NumPy ya los usa por debajo (vía OpenBLAS o MKL). Si tu `np.linalg.solve` parece lento, instala MKL (`pip install mkl`) o usa la build de conda con `mkl`.

¿GPU para álgebra lineal?

CuPy (drop-in replacement de NumPy con CUDA), PyTorch tensors (`.to('cuda')`), o JAX. Para matrices >1000×1000 la GPU vale la pena.

Referencias

- VanderPlas cap. 2 (overview NumPy).
- `numpy.linalg` reference
- PEP 465 — @ operator
- Trefethen & Bau, Numerical Linear Algebra (1997) — fondo matemático.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 021 — Clase 021 — NumPy: aleatoriedad y semillas

Parte: 0 — Prerrequisitos · Fuente: Numerical Recipes cap. 7 (Random Numbers) · NumPy `random`.Generator docs.

Duración estimada: 60 min.

Objetivo

Que el alumno genere números aleatorios reproduciblemente con el API moderno (`np.random.default_rng(seed)`), use las distribuciones más comunes (uniforme, normal, Bernoulli, Poisson, exponencial), y entienda por qué la reproducibilidad es no-negociable en ciencia de datos.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear un Generator con `np.random.default_rng(seed)` y usarlo para reproducibilidad.
2. Generar muestras de uniforme, normal, integers, binomial, Poisson, exponencial.
3. Permutar y muestrear sin/con reemplazo con `permutation` y `choice`.
4. Reproducir un experimento exactamente con el mismo seed.
5. Saber por qué `np.random.seed()` (API legacy) es deprecated en favor de Generator.

Temas

#	Tema	Por qué importa
1	<code>np.random.default_rng(seed)</code>	El API moderno (Generator-based, PCG64).
2	Distribuciones continuas: uniform, normal,	Las más usadas en simulación.
3	Distribuciones discretas: integers, binomi	Conteos y procesos.
4	<code>permutation</code> y <code>choice</code>	Mezclar y muestrear.
5	Reproducibilidad: por qué importa	Misma cosa con mismo seed.
6	Múltiples generadores independientes	Evita interferencia entre experimentos.

Definiciones y características

Generador pseudoaleatorio (PRNG)

: Algoritmo determinístico que produce secuencia que parece aleatoria. Con la misma semilla (seed) produce la misma secuencia → reproducible. NumPy 2026 usa PCG64 por default.

Seed (semilla)

: Estado inicial del PRNG. Mismo seed → misma secuencia, siempre, en cualquier máquina. Es la base de la reproducibilidad científica.

`np.random.default_rng(seed)`

: API moderno (NumPy 1.17+). Devuelve Generator independiente — no toca estado global, múltiples generadores no se interfieren. Reemplaza a `np.random.seed()` + funciones globales (deprecated).

Distribuciones continuas comunes

: `uniform(lo, hi)`, `normal( $\mu$ ,  $\sigma$ )`, `exponential(scale)`, `gamma(shape, scale)`, `beta(a, b)`. Cada una con parámetros que controlan ubicación y dispersión.

Distribuciones discretas

: `integers(lo, hi)` (uniforme discreto), `binomial(n, p)` (éxitos en n trials), `poisson( $\lambda$ )` (eventos en intervalo).

Bootstrap

: Técnica: resampleas con reemplazo del sample original B veces, recalculas el estadístico → obtienes distribución empírica del estimador. Sin asumir CLT ni normalidad.

Dataset / recursos

Sintético: simulación Monte Carlo. Sin descarga.

Ejercicios

1. Reproducibilidad. Crea 2 rngs con seed=42, genera 1000 normales con cada uno. Verifica que son idénticos.
2. Distribuciones. Genera 10000 muestras de: uniforme [0,1], normal(5,2), exponential($\lambda=1/3$), poisson($\lambda=4$). Calcula media y std empírica y compara con teórica.
3. Monte Carlo de π . Estima π lanzando puntos en un cuadrado 2×2 y contando cuántos caen dentro del círculo unitario. Compara con π real.
4. Bootstrap. Dado un sample de 30 valores, estima la distribución de la media por bootstrap (1000 resamples con reemplazo).
5. Permutación. Mezcla un array de 100 elementos con permutation. Verifica que es la misma cuando usas el mismo seed.

Homework verificable

Notebook con: (a) Monte Carlo de π con N=10k, 100k, 1M reportando error; (b) bootstrap de la media de un sample (95% CI vs CLT); (c) demo de reproducibilidad con dos rngs; (d) tabla comparando momento empírico vs teórico para 4 distribuciones.

Criterio de aceptación: MC converge a π . Bootstrap CI similar al CLT. Reproducibilidad exacta.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Pongo seed pero el resultado cambia entre	Probablemente otra librería (sklearn, pyto
np.random.seed(42) no funciona como antes	Está deprecated en favor de default_rng. A
rng.choice(arr, size=N, replace=False) fal	Pediste más muestras únicas que elementos
Genero millones de números 'aleatorios' y	Estás materializando lista en RAM. Fix: si
Bootstrap CI parece muy estrecho	Pocos resamples (B). Fix: usa $B \geq 1000$ par

Preguntas frecuentes

¿Por qué un generador independiente y no el global?

(1) No interfiere con libs que también usan random global. (2) Múltiples experimentos simultáneos sin conflicto. (3) API más limpia. (4) Algoritmo más rápido (PCG64). El legacy es solo por compatibilidad.

¿Mismo seed da mismos números en Linux/Windows/Mac?

Sí — PCG64 es determinístico cross-platform. La única diferencia podría venir de orden de operaciones float (paralelismo), pero default_rng es single-threaded.

¿Qué seed elegir?

Cualquier entero. 42 es convención (Hitchhiker's Guide). 0 funciona pero genera secuencias 'menos aleatorias' en los primeros bits con algunos PRNG (no PCG64). Lo importante es registrarlo para reproducir.

¿Bootstrap mejor que t-test?

Diferente caso. t-test: asume normalidad, da p-valor analítico. Bootstrap: sin asunción, da distribución empírica de cualquier estadístico (media, mediana, ratio). Para datos no-normales o estadísticos complejos,

bootstrap gana.

¿Cuántas muestras necesito para Monte Carlo?

El error decrece como $1/\sqrt{N}$. Para 1 decimal de precisión: $N \approx 100$. Para 2 decimales: $N \approx 10k$. Para 3: $N \approx 1M$. Verifica convergencia haciendo $N=100, 1k, 10k, 100k$ y mirando que el resultado se estabilice.

Referencias

- NumPy random.Generator
- NEP 19 — Random number generator policy
- Press et al., Numerical Recipes 3e — cap. 7 Random Numbers.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 022 — Clase 022 — Pandas: Series y DataFrame

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3, §§ 3.1–3.2 Introducing Pandas Objects.

Duración estimada: 90 min.

Objetivo

Que el alumno entienda qué es una Series (ndarray + index) y un DataFrame (dict de Series alineadas por index), cómo se construyen desde 5 fuentes distintas, y por qué el index es el rasgo que distingue pandas de NumPy.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear Series y DataFrames desde dict, lista de tuplas, arrays NumPy, CSV y desde otro DataFrame.
2. Inspeccionar un DataFrame con head, tail, info, describe, dtypes, shape.
3. Acceder a columnas como atributo (df.col) y como key (df['col']) — y saber cuándo cada uno falla.
4. Modificar el index con set_index, reset_index, rename.
5. Convertir Series ↔ DataFrame ↔ ndarray cuando sea necesario.

Temas

#	Tema	Por qué importa
1	Series = ndarray + index	Index permite alineación automática.
2	DataFrame = dict de Series alineadas	Por eso df['col'] devuelve Series.
3	Construcción desde 5 fuentes	dict, lista de dicts, arrays, CSV, Series.
4	.loc vs .iloc vs []	Tres formas de acceso.
5	Index labels vs posición	El bug clásico cuando el index no es 0..N.
6	info y describe como first-look	Lo primero que mira un DS.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada:

- Clase 032a — Polars: DataFrames modernos

Definiciones y características

Series

: Estructura 1D = ndarray + index labelled. `s[label]` accede por etiqueta, `s.iloc[N]` por posición. Característica clave: el index permite alineación automática entre Series.

DataFrame

: Estructura 2D = dict de Series que comparten el mismo index. Cada columna puede tener su propio dtype (a diferencia de un ndarray). `df['col']` devuelve Series.

Index

: Estructura que etiqueta cada fila (o columna). Puede ser entero default (`RangeIndex`), strings, fechas (`DatetimeIndex`), o jerárquico (`MultilIndex`). Inmutable — para cambiarlo, `set_index/reset_index`.

Alineación automática

: Cuando operas dos Series/DataFrames, pandas alinea por index, NO por posición. Posiciones sin match → NaN. Esto es lo que distingue pandas de NumPy.

dtype en pandas

: Cada columna tiene su dtype. Tipos clásicos: `int64`, `float64`, `bool`, `object` (strings o mixto). Modernos (NA-aware): `Int64`, `Float64`, `boolean`, `string`, `category`.

Dataset / recursos

Palmer Penguins (descargable con `seaborn/palmerpenguins`) — 344 filas × 7 columnas, públicas, sin issues de licencia. Reemplaza al iris dataset.

Ejercicios

1. Series desde dict. Crea Series con población de 5 ciudades. Accede por label y por posición.
2. DataFrame desde dict de listas. Construye DataFrame de 5 estudiantes (nombre, edad, nota). Inspecciona con `info()` y `describe()`.
3. Lee Palmer Penguins. `pd.read_csv` desde URL pública. Reporta shape, dtypes, % de NaN por columna.
4. Index labeled. Setea species como index. Compara `df.loc['Adelie']` vs `df.iloc[0]`.
5. Alineación automática. Crea 2 Series con index parcialmente solapado. Súmalas. Observa los NaN.

Homework verificable

Notebook que: (a) carga Palmer Penguins y reporta `info()`, `describe()`, missing por col; (b) muestra los 3 métodos de acceso a una columna (`df.col`, `df['col']`, `df.loc[:, 'col']`); (c) cambia el index a species, vuelve a default con `reset_index`; (d) demuestra alineación automática sumando dos Series.

Criterio de aceptación: Carga sin error, los 3 accesos producen la misma Series, alineación produce NaN donde corresponde.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>AttributeError</code> al hacer <code>df.column with spa</code>	Acceso como atributo solo funciona con nom
<code>KeyError: 'columna'</code> aunque está en el Data	Espacios invisibles o capitalización disti

Sumar dos Series y aparecen NaN inesperado	Index no coincide en algunos labels. Fix:
pd.read_csv infiere mal los dtypes	Pandas adivina por las primeras filas; si
DataFrame is not callable: TypeError: 'Dat	Hiciste df() por error en vez de df (parén

Preguntas frecuentes

¿df['col'] o df.col?

df['col'] siempre — funciona con cualquier nombre, no choca con métodos. df.col falla con espacios, choca con .shape, .head, etc. Solo úsalo en exploración rápida.

¿Por qué read_csv me da Unnamed: 0?

El CSV se guardó con index (df.to_csv('x.csv')). Fix: al leer, pd.read_csv('x.csv', index_col=0). O al guardar, df.to_csv('x.csv', index=False).

¿Cuándo Series y cuándo DataFrame de 1 columna?

Series es más liviana y la mayoría de operaciones la prefieren. DataFrame de 1 col cuando vas a concatenar con otros DataFrames o necesitas mantener la columna nombrada. s.to_frame() convierte.

¿copy() cuándo?

Cuando vas a modificar un subset y NO quieres afectar el original. subset = df[cond].copy(). Si solo lees, no necesitas copy — y consumes el doble de memoria.

¿Qué pasa con NaN en una columna int?

Pandas la promueve a float64 automáticamente (porque NumPy int no soporta NaN). Para mantener int: usa dtype nullable Int64 (mayúscula) que sí permite pd.NA.

¿Tengo que aprender pandas Y polars?

Por ahora, no. Pandas primero — es la base del curso, lo vas a ver en clases, libros, Stack Overflow y en cualquier código que toques. Cuando lo domines y te topes con un dataset que no entra en RAM o un pipeline lento, ahí asomate a polars (Parte 5). La transición es directa porque los conceptos (DataFrame, columnas, group_by, joins) son los mismos; lo que cambia es la sintaxis y el motor.

Referencias

- VanderPlas, cap. 3 §§ 3.1, 3.2.
- pandas user guide — DataFrame
- Palmer Penguins
- Polars user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 023 — Clase 023 — Pandas: indexación (loc, iloc, at, iat)

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.3 Data Indexing and Selection.

Duración estimada: 75 min.

Objetivo

Que el alumno domine los 4 indexers de pandas y elija el correcto según el caso. El bug "SettingWithCopyWarning" y el bug del slicing por label inclusivo nacen aquí — saber qué indexer usar evita ambos.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Usar `.loc[row_label, col_label]` para acceso por etiqueta (inclusivo en slicing).
2. Usar `.iloc[row_pos, col_pos]` para acceso por posición entera (exclusivo, como Python).
3. Usar `.at / .iat` para acceso a un único valor (más rápido que `loc/iloc`).
4. Evitar `SettingWithCopyWarning` usando `.loc` para asignar en una vista.
5. Filtrar filas con boolean mask dentro de `.loc`: `df.loc[df['edad'] > 30, 'nombre']`.

Temas

#	Tema	Por qué importa
1	<code>[]</code> directo: shortcut con quirks	Columnas → Series; filas → <code>KeyError</code> .
2	<code>.loc</code> : por label, slicing inclusivo	El indexer principal del 80% del tiempo.
3	<code>.iloc</code> : por posición, slicing exclusivo (co	Cuando no te importa el label.
4	<code>.at / .iat</code> : single value	Optimizado para 1 celda — útil en loops.
5	Mask + <code>loc</code> para filtros con asignación	<code>df.loc[mask, 'col'] = valor</code> .
6	<code>SettingWithCopyWarning</code> : qué es y cómo evit	Usar <code>.loc</code> para asignar.

Definiciones y características

`.loc[fila, col]`

: Acceso por etiqueta. Slicing es inclusivo en ambos extremos (`df.loc['a':'c']` incluye 'c'). Acepta booleano: `df.loc[df['x'] > 0, 'col']`.

`.iloc[fila, col]`

: Acceso por posición entera. Slicing es exclusivo del extremo (estilo Python). `df.iloc[0:5]` da 5 filas (índices 0..4).

`.at[fila, col] / .iat[fila, col]`

: Versiones para acceder/asignar un único valor. ~10× más rápidos que `.loc/.iloc` cuando estás en loops. Mismo patrón label vs posición.

Indexer chaining (`df[cond][col] = x`)

: Encadenar dos `[]` operaciones de acceso. Crea ambigüedad: ¿es vista o copia? Origen del `SettingWithCopyWarning`. Evítalo siempre.

`SettingWithCopyWarning`

: Aviso de pandas: "estoy haciendo algo ambiguo, puede que tu asignación se pierda". Causado por chaining o asignación sobre subset no-explicito. Solución universal: `.loc[cond, col] = x` en una sola operación.

Dataset / recursos

Palmer Penguins desde URL (mismo de clase 022) o el sintético si no hay internet.

Ejercicios

1. Acceso simple. Carga penguins. Obtén la columna species con los 3 métodos: `df.species`, `df['species']`, `df.loc[:, 'species']`.
2. loc inclusivo vs iloc exclusivo. Con index 0..N por default, compara `df.loc[0:5]` vs `df.iloc[0:5]`. ¿Cuántas filas devuelve cada uno?
3. Filtro + columnas seleccionadas. Pingüinos Adelie machos con `bill_length > 40`: `df.loc[(df.species=='Adelie') & (df.sex=='male') & (df.bill_length_mm > 40), ['species', 'island', 'bill_length_mm']]`.
4. Asignación segura. Crea una columna `is_big` que sea `True` si `body_mass_g > 4500`, usando `.loc`.
5. Provoca y arregla `SettingWithCopyWarning`. Slicea con `df[df.x > 0]` y modifica → ve warning. Hazlo con `.loc` → sin warning.

Homework verificable

Notebook que: (a) muestra los 3 métodos de acceso a columna; (b) compara loc vs iloc en slicing con tabla; (c) filtra Adelie machos con `bill_length>40` mostrando 3 columnas; (d) reproduce y arregla `SettingWithCopyWarning` con explicación.

Criterio de aceptación: Los filtros producen el subset correcto; la versión con `.loc` no emite warning.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>SettingWithCopyWarning</code> aparece y no sé por	Estás asignando sobre un resultado de slice
<code>df.loc[0:5]</code> da 6 filas no 5	loc es inclusive end. Para 5 filas: <code>df.iloc</code>
<code>KeyError</code> con <code>.loc[5]</code> cuando hice <code>set_index</code>	El index ya no es 0..N — es la columna id.
Asignación a vista no modifica el original	Pandas 3+ va a ser estricto: la vista no s
<code>df.col1 = x</code> no actualiza la columna	Como atributo, asignar no agrega columna n

Preguntas frecuentes

¿loc o iloc?

loc cuando el index es semántico (nombres, fechas, ids). iloc cuando solo importa la posición (top-K por orden, primeros 10, último). Mezclarlos en el mismo código causa confusión.

¿Por qué loc slicing es inclusivo?

Decisión histórica: para labels (strings, fechas), incluir el end es lo intuitivo ('enero':'marzo' debe incluir marzo). Para enteros default queda raro — usa iloc ahí.

¿at vale la pena vs loc para single value?

Solo en hot loops (>10k iteraciones). Para uso interactivo, loc es lo suficientemente rápido y más legible.

¿Cómo selecciono múltiples columnas?

Lista entre brackets: `df[['a', 'b', 'c']]` (devuelve DataFrame). Notar el `[[[]]`. Un solo `[]` con string devuelve Series.

¿`.loc[mask, cols]` o `.query()` + `[cols]`?

Ambos válidos. `.loc[mask, cols]` para máscaras computadas; `.query()` para filtros declarativos largos. Misma velocidad para datasets <100k.

Referencias

- VanderPlas, cap. 3 § 3.3.
- pandas Indexing user guide
- SettingWithCopyWarning explained

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 024 — Clase 024 — Pandas: operaciones y alineación

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.4 Operating on Data in Pandas.

Duración estimada: 60 min.

Objetivo

Que el alumno entienda cómo pandas alinea automáticamente por index en operaciones entre Series/DataFrames, cómo manejar NaN resultantes, y use apply/map para transformaciones custom (con consciencia de cuándo es lento).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Predecir el resultado de operar dos Series/DataFrames con indexes parcialmente distintos.
2. Usar fill_value en operaciones para no propagar NaN: `s1.add(s2, fill_value=0)`.
3. Aplicar funciones con apply (lento, flexible), map (Series), applymap / df.map (elementwise).
4. Vectorizar transformaciones cuando se puede en vez de apply (10–100× más rápido).
5. Usar ufuncs NumPy sobre Series — pandas las soporta directamente y preserva el index.

Temas

#	Tema	Por qué importa
1	Alineación automática por index	Producto, suma, todo — pandas alinea, no a
2	fill_value para operaciones	Reemplaza el NaN antes de calcular.
3	apply axis=0 vs axis=1	Por columna vs por fila — costoso en filas
4	map para Series con dict	<code>s.map({'A': 1, 'B': 2})</code> .
5	df.map (era applymap) — elementwise	Cell-by-cell, lento.
6	Vectorización > apply	Si puedes hacerlo con ufunc, hazlo.

Definiciones y características

Alineación por index

: Operación matemática entre dos pandas (Series + Series, DF + Series, etc.) alinea por etiqueta de index. Labels que no estén en ambos → NaN.

fill_value en operaciones

: Parámetro que reemplaza NaN durante la operación: `s1.add(s2, fill_value=0)` trata índices ausentes como 0 en vez de propagar NaN.

apply

: Aplica función a cada fila (axis=1) o columna (axis=0) de un DataFrame, o cada elemento de una Series. Lento porque itera en Python. Usa solo si vectorización no aplica.

map (Series)

: Aplica función o dict a cada elemento. Útil para recodificación rápida: `s.map({'A': 1, 'B': 2})`. Más rápido que apply por su API más restringida.

df.map (antes applymap)

: Aplica función a cada celda del DataFrame (elementwise). Lento; usa solo cuando vectorización no aplica. applymap está deprecated en pandas 2.1+.

Ufunc-aware

: Pandas Series respeta ufuncs NumPy: `np.log(s)`, `np.sqrt(s)` funcionan y preservan el index. Ventaja sobre convertir a array y perder labels.

Dataset / recursos

Sintético + Palmer Penguins. Sin descarga adicional.

Ejercicios

1. Suma con alineación. Dos Series con index parcialmente solapado. Súmalas (default) y con `fill_value=0`.
2. apply por fila. Define una función que reciba una fila de penguins y devuelva $BMI = \text{body_mass} / \text{bill_length}^2$. Aplica con `axis=1`.
3. Mismo cálculo vectorizado. Implementa BMI con operaciones vectorizadas. Mide ambos con `%timeit`.
4. map con dict. Mapea species a códigos: `{'Adelie': 0, 'Chinstrap': 1, 'Gentoo': 2}`.
5. ufunc NumPy preserva index. Aplica `np.log` a una columna; verifica que el index sigue intacto.

Homework verificable

Notebook con penguins: (a) BMI por fila con apply vs vectorizado (tabla `%timeit`); (b) species → código numérico con map; (c) demo de alineación con `fill_value`; (d) `np.log` sobre `body_mass` preservando index.

Criterio de aceptación: Vectorizado >50× más rápido que apply. Mapping y alineación correctos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
apply toma 10× más que vectorización equiv	Iteración Python por fila. Fix: piensa si
s1 + s2 produce NaN aunque los datos están	Index distinto (incluso por orden diferent
df.apply(func, axis=1) rompe con KeyError	Tu función accede <code>row['col_x']</code> pero esa co
df.map(lambda x: ...) falla "object has no	Es método de DataFrame solo en pandas ≥2.1
np.log(df['x']) da error con NaN	Si NaN está presente, log da NaN. Si negat

Preguntas frecuentes

¿Cuándo apply y cuándo NO?

NO: si lo puedes hacer con `df['a'] * df['b']` u operación pandas built-in (groupby, transform, etc.). Sí: lógica compleja por fila que no se descompone.

¿`apply(axis=1)` con tipos mixtos da problemas?

Sí — pandas convierte cada row a Series con dtype común. Si tienes int y str, queda object y los operadores `<`, `>` rompen. Mejor extrae columnas y opera directo.

¿Cómo paralelizo `apply`?

`pandarallel`, `swifter`, `modin` — drop-in replacements. Pero antes de paralelizar, asegúrate que tu `apply` no es vectorizable (el speedup es mayor).

¿`s.map(dict)` o `s.replace(dict)`?

`map`: mapea uno-a-uno; valores no encontrados en dict → NaN. `replace`: reemplaza solo los que aparecen; el resto queda igual. Elige según comportamiento deseado en valores faltantes.

¿Por qué pandas a veces es más lento que NumPy?

Overhead del index, manejo de NaN, dtype-aware. Para operaciones puramente numéricas sobre datos limpios y rectangulares, NumPy puede ser 2-5× más rápido. Pandas vale por la API, no por velocidad raw.

Referencias

- VanderPlas, cap. 3 § 3.4.
- pandas — `apply`, `map`

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 025 — Clase 025 — Pandas: datos faltantes

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.5 Handling Missing Data.

Duración estimada: 75 min.

Objetivo

Que el alumno detecte, cuantifique y maneje datos faltantes con criterio. Eliminar es la opción fácil pero suele ser incorrecta: cuándo eliminar, cuándo imputar (media, mediana, forward-fill), y cuándo el faltante es señal que merece su propia columna.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Detectar NaN con `isna()`, `notna()` y cuantificar por columna/fila.
2. Eliminar filas/columnas con NaN usando `dropna` con `how/thresh/subset`.
3. Imputar con `fillna`: valor escalar, media/mediana, forward/backward fill, interpolación.
4. Distinguir NaN vs None vs `pd.NA` y por qué importan los dtypes nullable (`Int64`, `boolean`).
5. Decidir entre eliminar/imputar/dejar — y crear columna `was_missing` cuando el faltante es informativo.

Temas

#	Tema	Por qué importa
---	------	-----------------

1	Tipos de missing en pandas: NaN, None, NaT	Cada uno tiene caso de uso.
2	Detección: isna, notna, isna().sum()	First-look obligatorio.
3	dropna: how='any'/'all', thresh, subset	Eliminar con precisión.
4	fillna: escalar, dict, ffill, bfill, inter	Imputar según contexto.
5	Dtypes nullable: Int64, Float64, boolean	El nuevo missing nativo.
6	was_missing como feature	Cuando el missing es señal.

Definiciones y características

NaN (Not a Number)

: Float especial IEEE 754 que representa missing en columnas numéricas. Característica: NO es igual a sí mismo (`np.nan == np.nan` → `False`). Usa `pd.isna()` para detectarlo.

None vs NaN

: None es objeto Python (en columnas object). NaN es float (en columnas numéricas). Pandas trata ambos como missing pero internamente son distintos.

pd.NA

: Sentinel "missing universal" (pandas 1.0+) compatible con dtypes nullable (Int64, boolean, string). Comportamiento más consistente que NaN en operaciones.

MCAR / MAR / MNAR

: MCAR (Missing Completely At Random): missing es aleatorio. MAR (Missing At Random): missing depende de variables observadas. MNAR (Missing Not At Random): depende del propio valor missing (peor caso).

Imputación

: Reemplazar missing con un valor estimado. Estrategias: media/mediana/moda global, por grupo (`groupby.transform`), forward-fill (series temporales), KNN, regresión, MICE.

was_missing flag

: Columna booleana adicional que registra qué filas tenían missing antes de imputar. Permite al modelo usar el hecho de que faltaba como feature (a veces es informativo).

Dataset / recursos

Palmer Penguins (tiene NaN reales en sex y mediciones). Sin descarga adicional si ya está en clases anteriores.

Ejercicios

1. Cuantifica. Carga penguins, reporta % de NaN por columna y por fila.
2. Eliminar filas con cualquier NaN. `df.dropna(how='any')`. Compara shape antes/después.
3. Eliminar solo filas con NaN en sex. `df.dropna(subset=['sex'])`. Más selectivo.
4. Imputar. Rellena `bill_length_mm` con la mediana por especie (`groupby + transform`). Justifica por qué la mediana es mejor que la media aquí.
5. Forward fill en series temporales. Crea una Series con NaN intercalados. Aplica `ffill`, `bfill`, `interpolate`. Compara.

Homework verificable

Notebook con penguins: (a) reporte completo de missing (% por col, % por fila, filas más incompletas); (b) 3

estrategias: drop all, drop subset, imputar por grupo; (c) columna bill_was_missing y demuestra que el flag puede mejorar un modelo simple; (d) demo de dtypes nullable Int64.

Criterio de aceptación: Imputación por grupo no introduce sesgo; el flag was_missing añade información.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
if df['col'] == np.nan no funciona	NaN no es igual a nada (incluso a sí mismo)
df.dropna() borra todo	Sin parámetros, borra fila con CUALQUIER N
Imputar con media global introduce sesgo	Si los grupos tienen medias muy distintas,
Una columna int se volvió float tras leer	Tiene NaN → promoción automática (NumPy in
fillna(0) mata el flag de "missing"	Pierdes la información de que faltaba. Fix

Preguntas frecuentes

¿Eliminar, imputar o flag?

Depende: <1% missing aleatorio → drop. 5-30% MAR → imputar (mediana por grupo). >50% → eliminar columna o tratar como categoría 'missing'. Si missing es informativo → flag + imputar.

¿Mediana o media para imputar?

Mediana es más robusta a outliers (recomendada por default). Media si la distribución es ~normal y sin outliers. Para categóricas: moda.

¿ffill siempre bueno para series temporales?

Bueno cuando los valores cambian poco entre observaciones (precios, temperaturas). Malo si los gaps son largos o el dato es volátil. Considera interpolate(method='time') para mejor resultado.

¿Cómo sé si la imputación afecta mi modelo?

Compara métricas: (a) baseline drop, (b) imputación X, (c) imputación + flag. Si el flag mejora el modelo, el missing era informativo (caso MNAR).

¿KNN/MICE para imputación en pandas?

No nativo. Usa sklearn.impute.KNNImputer o IterativeImputer (= MICE). Más caro pero mejor que mediana en datasets con correlaciones fuertes.

Referencias

- VanderPlas, cap. 3 § 3.5.
- pandas — Missing data user guide
- pandas nullable Integer dtypes

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 026 — Clase 026 — Pandas: MultiIndex

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.6 Hierarchical Indexing.

Duración estimada: 75 min.

Objetivo

Que el alumno use índices jerárquicos (MultiIndex) cuando hay estructura natural en los datos (país × ciudad, año × mes, sector × empresa). Saber cuándo aporta vs cuándo complica — el 80% del tiempo en data science aplanado es mejor.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear MultiIndex desde tuplas, arrays, producto cartesiano (from_product).
2. Indexar con .loc[(nivel1, nivel2)] y .loc[:, ('grupo', 'col')].
3. Aplanar y reconstruir con unstack(), stack(), reset_index().
4. Decidir cuándo MultiIndex aporta (groupby con múltiples claves devuelve uno automáticamente) y cuándo es más legible aplanar.
5. Renombrar niveles con rename(level=...) y reordenarlos con swaplevel.

Temas

#	Tema	Por qué importa
1	MultiIndex: motivación	Datos con jerarquía natural.
2	Construcción: tuples, arrays, from_product	3 formas comunes.
3	Indexación: tuple selector	.loc[('A', 2024)].
4	stack / unstack — pivot rápido	Mover niveles entre filas y columnas.
5	groupby + multiindex resultado	groupby con 2+ claves devuelve MultiIndex.
6	Cuándo aplanar	Para CSV de salida, plot, scikit-learn.

Definiciones y características

MultiIndex

: Índice jerárquico con N niveles. Cada fila identificada por tupla de N labels (('España', 2024)). Útil cuando los datos tienen estructura natural (país→ciudad, año→mes).

Nivel (level)

: Cada "capa" del MultiIndex. Se referencia por nombre (level='año') o posición (level=0). Útil en operaciones como unstack(level=...).

stack / unstack

: Mueven niveles entre filas y columnas. unstack sube un nivel del index a columnas (long→wide). stack baja un nivel de columnas al index (wide→long). Reversibles.

xs (cross-section)

: Slice por un valor en un nivel: df.xs(2024, level='año'). Más limpio que indexar con tuplas parciales.

Aplanar (flatten)

: Convertir MultiIndex a Index plano: df.reset_index() (vuelve a default 0..N) o df.index = ['_'.join(map(str, t)) for t in df.index] (concatena niveles).

Dataset / recursos

Sintético: ventas por país/año.

Ejercicios

1. Construye desde tuplas. Crea DataFrame con index [(España, 2023), (España, 2024), (Chile, 2023), (Chile, 2024)] y 2 cols ventas/clientes.
2. from_product. Mismo con pd.MultiIndex.from_product([países, años]).
3. Acceso jerárquico. df.loc['España'], df.loc[['España', 2024]]. Compara con df.xs(2024, level=1) para slice por nivel.
4. unstack y stack. Convierte tu MultiIndex en wide (años como columnas) y de vuelta.
5. groupby produce MultiIndex. Carga penguins, agrupa por (species, sex) y agrega mean(). Aplana con reset_index().

Homework verificable

Notebook con ventas trimestre×región sintéticas (4 trimestres × 3 regiones × 2 años): (a) construir con from_product; (b) acceso a un trimestre específico; (c) total por región (unstack); (d) groupby penguins por (species, sex) → MultiIndex → aplanar.

Criterio de aceptación: MultiIndex con shape correcto; unstack/stack reversibles.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
KeyError al acceder df.loc['España', 2024]	loc con MultiIndex requiere tupla: df.loc[
unstack() lanza ValueError: Index contains	Tienes filas duplicadas en (index, columns
groupby([a, b]).sum() devuelve cosa extraña	Es correcto: groupby con N keys devuelve M
Plot ignora niveles del MultiIndex	matplotlib/seaborn esperan columnas planas
sort_index() ordena raro con MultiIndex	Default ordena por todos los niveles. Para

Preguntas frecuentes

¿Cuándo MultiIndex aporta vs cuándo complica?

Aporta en análisis interactivo con slicing por nivel frecuente. Complica para export a CSV, plot, sklearn — aplanar ahí.

¿set_index([a, b]) vs groupby([a, b])?

set_index solo mueve cols al index (sin agregar). groupby colapsa filas por las cols (con sum/mean/agg). Diferentes operaciones.

¿Cómo evito MultiIndex en groupby?

groupby([a, b], as_index=False) devuelve DataFrame plano directamente. O .reset_index() después.

¿stack(future_stack=True) qué significa?

Es el comportamiento del nuevo stack (default en pandas 3+). Maneja NaN distinto al legacy. Mejor pasarlo siempre explícito para suprimir warnings.

¿Performance MultiIndex vs Index plano?

MultiIndex tiene overhead. Para datasets grandes (>1M filas) con acceso intenso, aplanar al final del pipeline.

Referencias

- VanderPlas, cap. 3 § 3.6.
- pandas MultiIndex user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 027 — Clase 027 — Pandas: concat, merge, join

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 §§ 3.7–3.8 Combining Datasets: Concat/Merge.

Duración estimada: 90 min.

Objetivo

Que el alumno junte datasets correctamente: concat (apilado simple), merge (SQL-style joins) y join (atajo por index). El error más común es usar el join equivocado y obtener duplicados o filas perdidas — saber qué tipo (inner/left/right/outer) evita semanas de bugs.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Apilar DataFrames con `pd.concat` por filas (`axis=0`) o columnas (`axis=1`).
2. Hacer joins SQL-style con `pd.merge`: inner, left, right, outer, cross.
3. Diagnosticar duplicados generados por merge con `validate='one_to_one' | 'many_to_one' | ....`
4. Joinear por index con `df1.join(df2)` (atajo para merge por index).
5. Usar `indicator=True` para saber qué filas vienen de cada lado del merge.

Temas

#	Tema	Por qué importa
1	<code>concat axis=0</code> (filas) vs <code>axis=1</code> (columnas)	Apilado simple con alineación de index.
2	<code>merge how='inner'/'left'/'right'/'outer'</code>	Los 4 tipos de join SQL.
3	<code>on</code> vs <code>left_on/right_on</code>	Cuando los nombres de columna difieren.
4	<code>validate</code> para evitar duplicación	1:1, 1:m, m:1, m:m.
5	<code>indicator=True</code> para auditar	Columna <code>_merge</code> con <code>left_only/right_only/both</code>
6	<code>df.join</code> por index	Atajo idiomático.

Definiciones y características

concat

: Apila DataFrames por filas (`axis=0`, default) o columnas (`axis=1`). Alinea por el otro eje. No requiere key; es apilamiento puro.

merge (SQL-style join)

: Combina dos DataFrames por una key común. `how='inner'/'left'/'right'/'outer'/'cross'` controla qué filas se conservan.

INNER JOIN

: Solo filas presentes en AMBOS lados de la key. Si la key no matchea, se descarta. Default de merge.

LEFT JOIN

: Todas las filas del lado izquierdo (df1). Si no hay match en el derecho, columnas derechas quedan NaN. Útil para enriquecer datos sin perder ninguno.

OUTER JOIN

: Todas las filas de ambos lados; NaN donde no hay match. Vista "unión". Útil para auditoría.

validate

: Parámetro de merge que valida la cardinalidad esperada: 'one_to_one', 'one_to_many', 'many_to_one', 'many_to_many'. Si los datos no la cumplen, lanza error → evita duplicados ocultos.

indicator=True

: Añade columna _merge con 'left_only'/'right_only'/'both'. Útil para auditar qué tipo de match tuvo cada fila.

Dataset / recursos

Sintético: tabla de clientes + tabla de órdenes (relación 1:N).

Ejercicios

1. Concat por filas. 3 DataFrames mensuales con mismas columnas → uno anual. ignore_index=True.
2. Inner join. Clientes + órdenes por cliente_id. Verifica que solo aparecen clientes con al menos 1 orden.
3. Left join. Clientes + órdenes, conservando clientes sin órdenes (NaN en cols de orden).
4. Detectar duplicados. Provoca un merge muchos-a-muchos no intencional. Usa validate='one_to_many' para que falle si hay duplicación oculta.
5. indicator=True. Auditar cuántas filas son left_only / right_only / both.

Homework verificable

Notebook con clientes (10) + órdenes (25): (a) 4 tipos de join con _merge indicator; (b) tabla con conteo de cada tipo; (c) detección de relación con validate; (d) join por index con df.join.

Criterio de aceptación: Counts de cada join coherentes (inner ≤ left ≤ outer). validate lanza excepción si la relación esperada falla.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Tras merge tengo más filas que el DataFram	Relación uno-a-muchos no esperada. Fix: va
merge produce KeyError en la key	Tipos distintos: int vs str aunque el valo
Columnas se renombran con _x/_y tras merge	Ambos DataFrames tenían cols con el mismo
pd.concat([df1, df2]) da columnas extra co	Los dos tenían columnas distintas (pandas
concat ignora mi ignore_index=True y queda	Si tus DFs tienen index distintos, sin ign

Preguntas frecuentes

¿merge o join?

merge es la API rica (por columnas, control total). df1.join(df2) es atajo cuando ambos tienen index alineado a

la key. Mismo motor por dentro.

¿Cuándo concat vs merge?

concat: apilas datos con la misma estructura (mes 1, mes 2, mes 3 → año). merge: combinas datasets diferentes que comparten una key (clientes + órdenes).

¿validate siempre?

Sí — cuesta nada y atrapa el bug "silenciosamente generé el doble de filas". Recomendado en todo merge de producción.

¿Cómo merge por múltiples columnas?

merge(df1, df2, on=['a', 'b']) o left_on=['a','b'], right_on=['x','y']. La key compuesta es lista de strings.

¿Merge es lento con datasets grandes?

Con N=1M ya empieza a notarse. Acelera: setea index a la key antes (set_index('key').join(...)) o usa DuckDB (SELECT ... JOIN) — frecuentemente más rápido.

Referencias

- VanderPlas, cap. 3 §§ 3.7-3.8.
- pandas Merge user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 028 — Clase 028 — Pandas: groupby (split-apply-combine)

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.9 Aggregation and Grouping.

Duración estimada: 90 min.

Objetivo

Que el alumno aplique el patrón split-apply-combine que es el patrón fundamental de análisis tabular: dividir por grupo, aplicar función, recombinar. Saber elegir entre agg, transform, filter y apply — cada uno tiene su rol.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Agrupar por una o más columnas con groupby y aplicar agregaciones (sum, mean, count).
2. Usar agg con dict para distintas funciones por columna: agg({'a': 'sum', 'b': 'mean'}).
3. transform para preservar la shape original (broadcasting del estadístico de grupo).
4. filter para filtrar grupos enteros según condición.
5. Diferenciar los 4 métodos del groupby y elegir el correcto.

Temas

#	Tema	Por qué importa
---	------	-----------------

1	Split-apply-combine: el patrón	El más común en análisis tabular.
2	agg (= aggregate)	Reduce a una fila por grupo.
3	transform	Misma shape — útil para imputar/normalizar
4	filter	Conserva grupos completos según condición.
5	apply: el más flexible, el más lento	Cuando los 3 anteriores no alcanzan.
6	Múltiples columnas de agrupación	groupby(['a','b']) → MultiIndex.

Definiciones y características

Split-apply-combine

: Patrón: (1) split divide datos por valor de columna(s) → grupos; (2) apply ejecuta función en cada grupo; (3) combine junta resultados. El más usado en análisis tabular.

agg (= aggregate)

: Reduce cada grupo a una fila (sum, mean, count, std). Acepta función nombrada, lista de funciones, o dict por columna: `agg({'a': 'sum', 'b': ['min','max']})`.

transform

: Aplica función por grupo PERO mantiene la shape original (broadcastea resultado a cada fila). Ideal para z-score por grupo, imputación por grupo, ratios.

filter (groupby)

: Filtra grupos completos (no filas) según una condición. `g.filter(lambda x: len(x) > 100)` mantiene solo grupos con >100 filas.

apply (groupby)

: El más flexible y el más lento. Cualquier función custom por grupo (puede devolver Series, DataFrame, escalar). Úsalo solo cuando agg/transform/filter no alcanzan.

Named aggregation

: Sintaxis pandas 0.25+: `g.agg(total=('monto', 'sum'), n=('id', 'count'))`. Más legible que el dict tradicional, permite renombrar en el mismo paso.

Dataset / recursos

Palmer Penguins (groupby por species y/o sex).

Ejercicios

1. Agg básico. Penguins agrupado por species: media de cada feature numérica.
2. Agg con dict. Por species: mean de bill_length, max de body_mass, count de filas.
3. Transform: z-score por grupo. Crea columna mass_z = z-score de body_mass dentro de su species.
4. Filter: solo grupos grandes. Conserva solo species con >100 individuos.
5. Apply custom. Por species, devuelve el pingüino con mayor body_mass (un DataFrame por grupo).

Homework verificable

Notebook con penguins: (a) agg múltiple por (species, sex); (b) transform z-score por species; (c) filter species con n>50; (d) apply que devuelva el top-3 más pesado por species; (e) tabla groupby.size() por sex x island.

Criterio de aceptación: z-score por grupo tiene media ≈ 0 y std ≈ 1 dentro de cada species.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>g.apply(...)</code> lanza <code>FutureWarning</code> sobre <code>inc</code>	Pandas 2.2+ cambia comportamiento. Fix: <code>g</code> .
<code>g.mean()</code> solo muestra cols numéricas	Comportamiento intencional (pandas 2+). Fi
<code>g['col'].transform(...)</code> da error "function"	Tu función devolvió shape distinta a la en
Resultado de <code>g.agg(...)</code> tiene <code>MultIndex</code> e	Lista de funciones por columna → <code>MultIndex</code>
<code>groupby(col).size()</code> vs <code>count()</code> dan resulta	<code>size</code> : número de filas por grupo (incluye N

Preguntas frecuentes

¿agg, transform, filter o apply?

agg: reduces a 1 fila por grupo (resumen). transform: mantienes shape original (z-score). filter: excluyes grupos enteros. apply: lo demás, asumiendo overhead.

¿Cómo agrego columnas usando agg + named?

`g.agg(total=('monto', 'sum'), avg=('monto', 'mean'), n=('id', 'count')).reset_index()`. Tres columnas nombradas en una sola operación.

¿`groupby([a, b]).agg(...).reset_index()` o `as_index=False`?

Equivalentes. `as_index=False` evita el `reset_index()` posterior. Para encadenar con `merge/concat`, `as_index=False` es más limpio.

¿Cómo agrupo por una expresión derivada?

Pasa Series directa: `df.groupby(df['fecha'].dt.year)`. O crea columna temporal: `df.groupby(df['fecha'].dt.year.rename('año'))`.

¿groupby es lento con miles de grupos?

Con $N=1M$ filas y $K=1000$ grupos, debería ser $<1s$. Si es más lento, posibles causas: agg con función custom Python (no built-in), keys con dtype object (string), o falta de sort. Usa `sort=False` si no necesitas orden.

Referencias

- VanderPlas, cap. 3 § 3.9.
- pandas groupby user guide
- Wickham, "The split-apply-combine strategy for data analysis" (J Stat Software, 2011).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 029 — Clase 029 — Pandas: pivot tables y crosstab

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.10 Pivot Tables.

Duración estimada: 60 min.

Objetivo

Que el alumno construya tablas pivot (estilo Excel) con `pivot_table` y tablas de contingencia con `crosstab`. Son atajos sobre `groupby` pensados para resumen×visualización rápida.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Usar `pivot_table` con `index`, `columns`, `values`, `aggfunc`.
2. Añadir totales con `margins=True`.
3. Construir tablas de contingencia con `pd.crosstab` y normalizar (`normalize='all'/'index'/'columns'`).
4. Diferenciar pivot (sin agregar) vs `pivot_table` (con `aggfunc`, agrega duplicados).
5. Visualizar una pivot como heatmap básico para confirmar patrones.

Temas

#	Tema	Por qué importa
1	pivot vs <code>pivot_table</code>	pivot no acepta duplicados; <code>pivot_table</code> sí
2	Parámetros: <code>index</code> , <code>columns</code> , <code>values</code> , <code>aggfun</code>	Análogos a Excel.
3	<code>margins=True</code> : totales	Útil para verificar.
4	<code>crosstab</code> : tabla de contingencia	Counts entre dos categóricas.
5	<code>normalize</code> en <code>crosstab</code>	Proporciones por fila/col/total.
6	Pivot → heatmap	Detectar patrones visualmente.

Definiciones y características

`pivot_table`

: Resumen tabular estilo Excel: defines `index`, `columns`, `values` y `aggfunc`. Acepta duplicados (agrega). El atajo más usado para reportes.

`pivot` (sin `_table`)

: Variante que NO agrega — falla si hay duplicados en (`index`, `columns`). Más estricta; úsala solo cuando garantizas unicidad.

`crosstab`

: Tabla de contingencia entre 2 categóricas: counts cruzados. Con `normalize='index'/'columns'/'all'` muestra proporciones.

`margins=True`

: Añade fila/columna "Total" al pivot. Útil para verificar manualmente y para reportes ejecutivos.

Heatmap de pivot

: Renderizar el pivot como matriz coloreada (`plt.imshow` o `seaborn.heatmap`) — patrones visuales saltan a la vista.

Dataset / recursos

Palmer Penguins. Sin descarga adicional.

Ejercicios

1. Pivot básico. Penguins: índice `species`, columnas `sex`, valores `body_mass mean`.
2. Pivot con totales. Mismo con `margins=True`.

3. Crosstab counts. Counts species × island.
4. Crosstab normalizado. Mismo con normalize='index' (% por fila).
5. Pivot → heatmap. Toma un pivot table y plotéala con matplotlib imshow.

Homework verificable

Notebook con penguins: (a) pivot_table (species × island, mean body_mass); (b) crosstab species × island, count y normalizado; (c) verificación de totales con margins; (d) heatmap simple del pivot.

Criterio de aceptación: Pivot con shape correcto; sum de normalize='index' = 1.0 por fila.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
pivot() lanza ValueError: Index contains d	Hay duplicados en (index, columns). Fix: u
pivot_table da NaN donde no hay datos	Combinaciones (index × columns) sin filas.
crosstab cuenta cosas raras con muchos NaN	Crosstab cuenta filas no-NaN por default.
Pivot con cols numéricas float queda feo	Sin aggfunc explícito, pandas usa mean. Si
Plot del pivot rompe por MultiIndex	Pivot con múltiples niveles de columnas →

Preguntas frecuentes

¿pivot_table o groupby + unstack?

Equivalentes en resultado. pivot_table es más declarativo, mejor para reportes. groupby + unstack más componible, mejor en pipelines.

¿crosstab o pivot_table con aggfunc='count'?

Equivalentes para counts. crosstab tiene API más simple para 2 categóricas. pivot_table más flexible (varias valores, varias funcs).

¿Cómo ordeno el pivot?

Por valores: pivot.sort_values('col_x', ascending=False). Por suma: pivot.loc[pivot.sum(axis=1).sort_values(ascending=False).index].

¿Reportes Excel-like exportables?

pivot.to_excel('reporte.xlsx') directo. O to_csv para CSV. Para formato fino (colores, formulas), usa openpyxl o xlsxwriter.

¿Cuándo no usar pivot?

Cuando los datos ya están en formato wide y solo necesitas plot/agregaciones — usar groupby directo. Pivot es para transformar long → wide.

Referencias

- VanderPlas, cap. 3 § 3.10.
- pandas Pivot guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.

- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 030 — Clase 030 — Pandas: operaciones vectorizadas sobre strings

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.11 Vectorized String Operations.

Duración estimada: 60 min.

Objetivo

Que el alumno limpie y transforme columnas de texto sin caer en `apply(lambda x: ...)`, usando el accessor `.str` de pandas — vectorizado, NaN-aware, con métodos análogos a los de Python (`lower`, `strip`, `replace`, `split`, `contains`, `regex`).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Usar `.str` para aplicar operaciones de string vectorizadamente a una Series.
2. Manejar NaN automáticamente (los métodos `.str` propagan NaN sin error).
3. Aplicar regex con `.str.contains(patron)`, `.str.extract(...)`, `.str.replace(...)`.
4. Dividir y unir con `.str.split(sep, expand=True)` que produce un DataFrame.
5. Trabajar con categorical cuando el cardinalidad es baja (memoria y speedup).

Temas

#	Tema	Por qué importa
1	Accessor <code>.str</code>	Métodos vectorizados que respetan NaN.
2	Casos típicos: <code>lower</code> , <code>strip</code> , <code>replace</code> , <code>cont</code>	El 80% del trabajo.
3	Regex con <code>.str.extract</code> y grupos nombrados	Extracción estructurada.
4	<code>.str.split(expand=True)</code> → DataFrame	Desnormalizar columnas combinadas.
5	<code>dtype='string'</code> (nullable) vs object	El moderno y NA-aware.
6	Categorical para baja cardinalidad	Menos memoria, groupby más rápido.

Complemento previo: Regex con el módulo `re`

Antes de meternos con `.str.extract` / `.str.contains`, conviene tener una intro mínima a expresiones regulares. Pandas usa regex por debajo en casi todos los métodos de texto, y en scraping (BeautifulSoup, parsing de HTML/logs) son prerequisite invisible. Sin entender regex, los patrones se vuelven magia negra que se copypastea de Stack Overflow.

Metacaracteres esenciales

Símbolo	Significado	Ejemplo
.	Cualquier carácter (excepto \n)	<code>a.c</code> matchea <code>abc</code> , <code>a c</code> , <code>a3c</code>
*	0 o más repeticiones del anterior	<code>ab*</code> matchea <code>a</code> , <code>ab</code> , <code>abbb</code>
+	1 o más repeticiones	<code>ab+</code> matchea <code>ab</code> , <code>abbb</code> (no <code>a</code>)
?	0 o 1 (opcional)	<code>colou?r</code> matchea <code>color</code> y <code>co</code>
\d	Dígito [0-9]	<code>\d\d\d</code> matchea <code>123</code>
\w	Word char [a-zA-Z0-9_]	<code>\w+</code> matchea <code>hola_123</code>
\s	Whitespace (espacio, tab, \n)	<code>\s+</code> matchea uno o más espacios

[]	Set de caracteres	[aeiou] matchea una vocal		
()	Grupo de captura	(\d+)(\d+) captura ambos r		
^	Inicio de string	^Hola matchea solo si arrai		
\$	Fin de string	\.com\$ matchea solo si terr		
`\`	`	OR	`gato\`	perro` matchea cualquiera
{n,m}	Entre n y m repeticiones	\d{2,4} matchea 2 a 4 dígitos		

Funciones clave de re

Función	Para qué sirve
re.search(pat, s)	Busca el primer match en cualquier parte d
re.match(pat, s)	Igual que search pero solo desde el inicio
re.findall(pat, s)	Devuelve lista con todos los matches.
re.sub(pat, repl, s)	Reemplaza todos los matches por repl. Equi
re.compile(pat)	Pre-compila el patrón. Útil si lo vas a us
re.IGNORECASE (flag)	Match case-insensitive. Se pasa como flags

Mini-ejemplo — extraer dominio de email con grupos nombrados:

```
import re

email = "vladimir.acuna@gmail.com"
pat = re.compile(r"(?P<usuario>[w.]+)@(?P<dominio>[w.]+)")
m = pat.search(email)
print(m.group("usuario")) # vladimir.acuna
print(m.group("dominio")) # gmail.com
```

Raw strings (r"...")

Siempre se usan raw strings con regex. Sin la r, Python interpreta \d, \w, \s como secuencias de escape y muchas veces te las come o te tira DeprecationWarning. Con r"d+" le decís a Python "esto es literal, no lo toques, pásaselo crudo a re". Regla: toda regex va en raw string, sin excepciones.

Herramienta recomendada para iterar patrones: regex101.com — testa en vivo, explica cada token y soporta flavor Python.

Definiciones y características

Accessor .str

: Espacio de nombres en Series con métodos string vectorizados. Análogos a los de Python (.lower(), .split(), .replace()) pero aplicados elementwise y NaN-aware (propagan NaN sin error).

.str.extract(pattern)

: Aplica regex con grupos () y devuelve DataFrame con una columna por grupo. Soporta grupos nombrados ((?P<dominio>...)).

.str.split(sep, expand=True)

: Divide cada string y opcionalmente expande a DataFrame de columnas. Útil para denormalizar 'Apellido, Nombre' → 2 cols.

dtype 'string' (nullable)

: Versión moderna del dtype para texto. Diferencias con object: NA-aware (usa pd.NA), futuras

optimizaciones. Recomendado en pandas 2+.

Categorical

: Dtype para columnas con cardinalidad baja (pocos valores únicos). Almacena cada valor como entero + diccionario. Ahorra ~10× memoria y acelera groupby/sort.

Dataset / recursos

Sintético: emails, nombres con espacios, fechas como string.

Ejercicios

1. Lower + strip. Lista de emails con mayúsculas y espacios. Normaliza con `.str.lower().str.strip()`.
2. Extract dominio. De una columna de emails, extrae el dominio con regex `(@.+)$`.
3. Split nombre completo. Columna 'Ana García' → nombre, apellido en columnas separadas.
4. Filtro por contains. Filas donde la columna descripcion contiene la palabra (case-insensitive) 'urgente'.
5. Categorical. Convierte una columna con 5 valores únicos en 100k filas a Categorical. Compara memoria.

Homework verificable

Notebook con CSV sintético de contactos (nombre, email, teléfono): (a) normalizar email (lower+strip); (b) extraer dominio; (c) separar nombre/apellido; (d) flag de email corporativo (no gmail/yahoo/hotmail); (e) convertir país a Categorical y reportar memoria.

Criterio de aceptación: Operaciones manejan NaN sin error. Categorical reduce memoria al menos 5×.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
'NoneType' has no attribute 'lower' al hac	Hay NaN/None en la Series. Fix: usa <code>.str.l</code>
Regex no captura nada con <code>.str.extract</code>	Falta <code>()</code> para definir grupo, o el pattern
<code>.str.contains('foo')</code> lanza error con NaN	Por default, na=NaN propaga. Fix: <code>s.str.co</code>
Convertí a Categorical y el sort sale alfa	Categorical por default es no-ordenado. Fi
<code>.str.split(',')</code> da listas, no columnas	Sin <code>expand=True</code> . Fix: <code>s.str.split(',', exp</code>
Regex con <code>"\d+"</code> no matchea o tira Deprecat	Olvidaste la <code>r</code> del raw string — Python int

Preguntas frecuentes

¿`.str.lower()` o `.apply(str.lower)`?

`.str.lower()` siempre — vectorizado, maneja NaN, mucho más rápido en N grande. `apply` es loop Python disfrazado.

¿Cuándo convertir a Categorical?

Cuando la cardinalidad es baja (~<5% de N filas) y vas a hacer groupby/sort. Para 100k filas de 5 países: enorme ganancia. Para 100k filas de 80k strings únicos: no ayuda.

¿'string' o object dtype?

'string' para código nuevo (NA-aware). object sigue siendo default por compat. Conviértelo explícito: `df['col'] = df['col'].astype('string')`.

¿Regex case-insensitive?

`s.str.contains('foo', case=False)` o `flags=re.IGNORECASE`. También `s.str.lower().str.contains('foo')` (más explícito).

¿Cómo elimino acentos?

Pandas no trae nativo. Usa `unidecode` o `s.str.normalize('NFKD').str.encode('ascii', 'ignore').str.decode('ascii')`.

¿Por qué `\d` a veces falla?

Porque te olvidaste de la `r` del raw string. `"\d"` en Python plano no es un escape válido y según la versión te lo come o te tira `DeprecationWarning` (y en Python 3.12+ va directo a `SyntaxWarning`). Con `r"\d"` el backslash se pasa literal a `re`, que es quien lo interpreta como "dígito". Regla mecánica: toda regex en raw string, sin pensarlo.

Referencias

- VanderPlas, cap. 3 § 3.11.
- pandas Text data user guide
- pandas Categorical
- Python re HOWTO

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 031 — Clase 031 — Pandas: series de tiempo, resampling, rolling

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.12 Working with Time Series.

Duración estimada: 90 min.

Objetivo

Que el alumno trabaje con datos temporales correctamente: parsear fechas, indexar por `DatetimeIndex`, hacer resampling (cambiar la frecuencia) y rolling (ventanas móviles para tendencias).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Parsear strings de fecha con `pd.to_datetime(..., format=..., errors=...)`.
2. Indexar por `DatetimeIndex` y slicear con strings de fecha (`df.loc['2024-01':'2024-03']`).
3. Resamplear a otra frecuencia: `df.resample('M').sum()`, 'W', 'D', 'H'.
4. Aplicar ventanas móviles con `rolling(window).mean()` para suavizar tendencias.
5. Manejar zonas horarias con `tz_localize` y `tz_convert`.

Temas

#	Tema	Por qué importa
1	<code>pd.to_datetime</code> con <code>errors='coerce'</code>	Parseo robusto.
2	<code>DatetimeIndex</code> y slicing por fecha	Sintaxis natural: <code>'2024-01':'2024-03'</code> .

3	Resampling: 'D', 'W', 'M', 'Q', 'Y', 'H'	Cambiar frecuencia + agregar.
4	Rolling windows	Suavizado, medias móviles.
5	shift y diff	Diferencias entre periodos, lag features.
6	Timezones: localize → convert	Cuando los datos tienen TZ.

Definiciones y características

Timestamp / DatetimeIndex

: Tipo nativo de pandas para fechas-horas. Vectorizado. Permite indexar con strings: `df.loc['2024-01':'2024-03']`.

`pd.to_datetime`

: Conversor robusto `string→Timestamp`. `errors='coerce'` convierte fallos a `NaT` (Not a Time) en vez de excepción.

Resampling

: Cambiar la frecuencia de una serie: `diaria→mensual`, `horaria→semanal`. Requiere agregación (`sum`, `mean`, `last`, `ohlc`). Códigos: 'D', 'W', 'ME' (month-end), 'h', 'min'.

Rolling window

: Ventana móvil: para cada punto, aplicar función a los últimos N (`.rolling(N).mean()`). Suaviza tendencias, calcula medias móviles.

`shift` / `diff` / `pct_change`

: `shift(N)`: desplaza N posiciones (`NaN` al inicio). `diff(N)`: `s - s.shift(N)` (cambio absoluto). `pct_change()`: cambio relativo.

`tz_localize` vs `tz_convert`

: `localize` asigna TZ a fecha naive (no convierte). `convert` convierte de una TZ a otra (mantiene el instante). Patrón: `localize` primero, `convert` después.

Dataset / recursos

Sintético: serie diaria de 2 años de ventas. Sin descarga.

Ejercicios

1. Parseo robusto. Lista de fechas con formatos mixtos ('2024-01-15', '15/02/2024', 'foo'). Parsea con `errors='coerce'`. Reporta `NaT`.
2. Slice por fecha. Con índice `datetime`, selecciona Q1 2024 con `df.loc['2024-01':'2024-03']`.
3. Resample diaria → mensual. Suma ventas por mes con `df.resample('M').sum()`.
4. Rolling 7-day mean. Calcula media móvil de 7 días sobre ventas diarias. Plotea junto a la serie original.
5. shift para lag feature. Crea columna `ventas_lag_1` con `shift(1)`. Útil para features de ML.

Homework verificable

Notebook con serie sintética de 2 años: (a) parseo robusto; (b) slice por trimestre; (c) resample a mensual con `sum` y `mean`; (d) rolling 7/30 días con `plot`; (e) `diff` y `pct_change` para variación.

Criterio de aceptación: Plots muestran tendencia clara. Resample correcto (#meses esperado).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
pd.to_datetime falla con ValueError	Formato no estándar. Fix: format='%d/%m/%Y'
df.loc['2024-13'] lanza KeyError	Mes inválido. Fix: verifica formato — pand
resample('M') warning sobre deprecation	Pandas 2.2+ prefiere 'ME' (month-end) o 'M'
Resta de fechas da Timedelta en lugar de n	Es correcto — usa .dt.days, .dt.seconds pa
Operaciones con TZ mezcladas: Cannot compa	Una fecha tiene timezone, la otra no. Fix:

Preguntas frecuentes

¿Cuándo DatetimeIndex?

Cuando vas a hacer resample, slicing por fechas, o cualquier operación temporal. set_index('fecha') después de parsear.

¿resample o groupby(date.dt.month)?

resample es nativo y maneja gaps + zonas horarias mejor. groupby para agrupaciones no temporales ("por mes ignorando año").

¿Rolling cuánto Nuevo iniciar?

Default: NaN en los primeros N-1 (no hay datos suficientes para la ventana). Si quieres usar lo que haya: .rolling(N, min_periods=1).

¿Cómo manejo zonas horarias en producción?

Regla: guarda todo en UTC, convierte solo para mostrar. df['fecha'] = pd.to_datetime(...).dt.tz_localize('UTC') al ingerir, tz_convert('America/Santiago') al exhibir.

¿pd.date_range o pd.timestamp_range?

date_range — timestamp_range no existe. pd.date_range('2024-01-01', '2024-12-31', freq='D') para 365 fechas diarias.

Referencias

- VanderPlas, cap. 3 § 3.12.
- pandas Time Series user guide

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 032 — Clase 032 — Pandas: eval y query

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 3 § 3.13 High-Performance Pandas: eval and query.

Duración estimada: 45 min.

Objetivo

Que el alumno conozca df.eval y df.query — herramientas para expresar operaciones y filtros con sintaxis

tipo SQL en strings. Útiles para legibilidad en cadenas largas y, en datasets muy grandes, también más rápidos (usan numexpr).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Filtrar con `df.query("col > 10 and other == 'X'")`.
2. Calcular columnas nuevas con `df.eval('z = x + y')` o `df.eval('x * 2')`.
3. Referenciar variables locales en query/eval con prefijo `@`: `df.query('x > @threshold')`.
4. Decidir cuándo usar query (legibilidad en cadenas largas) vs filtro tradicional (mejor autocompletado IDE).
5. Saber que el speedup real solo aparece con datasets >10k filas y expresiones complejas.

Temas

#	Tema	Por qué importa
1	<code>df.query</code> — sintaxis tipo SQL	Una sola string en vez de máscara compuest
2	<code>df.eval</code> — expresiones aritméticas	Calcula columnas sin temporales.
3	Variables locales con <code>@</code>	Pasar valores del scope.
4	<code>numexpr</code> para speedup	Solo en datasets grandes.
5	Trade-off: legibilidad vs introspección ID	Query strings no tienen autocompletado.

Definiciones y características

`df.query()`

: Filtro como string tipo SQL: `df.query('precio > 100 and categoria == "A"')`. Más legible que máscara booleana compuesta cuando hay >2 condiciones.

`df.eval()`

: Evalúa expresiones aritméticas: `df.eval('total = precio * cantidad')`. Con `inplace=True` añade la columna al DF.

Variables locales (`@var`)

: Dentro de query/eval, prefijo `@` referencia variables del scope Python: `df.query('x > @threshold')`.

`numexpr`

: Motor de cómputo que vectoriza expresiones en C/SIMD. Usado por eval/query bajo el capó cuando los datasets son grandes. Acelera operaciones aritméticas complejas.

Trade-off legibilidad vs IDE

: Query strings: más legibles para humanos. Filtros tradicionales: autocomplete del IDE, type checking. Elige según contexto.

Dataset / recursos

Sintético: DataFrame grande para benchmark. Sin descarga.

Ejercicios

1. Filter tradicional vs query. `df[(df.a > 10) & (df.b < 5) & (df.c == 'x')]` vs `df.query('a > 10 and b < 5 and c == "x"')`. Compara legibilidad.
2. Variable local. `threshold = 100`; filtra con `df.query('precio > @threshold')`.

3. eval para nueva columna. `df.eval('total = precio * cantidad', inplace=True)`.

4. Benchmark. Genera df 1M filas. Compara filter tradicional vs query con `%timeit`.

5. eval con `inplace=False` vs cálculo tradicional `df['total'] = df['precio'] * df['cantidad']` — verifica resultados idénticos.

Homework verificable

Notebook con df 100k filas: (a) 3 filtros equivalentes (mask, query, query con `@var`); (b) eval para crear 2 columnas derivadas; (c) benchmark tradicional vs query en N=100k y N=1M; (d) reporte: cuándo conviene cada uno.

Criterio de aceptación: Resultados idénticos entre métodos. Speedup de query aparece en $N \geq 100k$.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
UndefinedVariableError: name 'X' is not de	Variable Python no prefijada con <code>@</code> . Fix: <code>d</code>
Strings dentro de query con comillas doble	Mezcla de quotes. Fix: usa quotes opuestas
<code>df.eval('col_nueva = ...')</code> no añade la col	Sin <code>inplace=True</code> . Fix: <code>df.eval('col = xy',</code>
query/eval más lento que máscara tradicion	Para datasets pequeños (<10k filas), el ov
Funciones custom no funcionan en query	Solo aritmética + operadores. Fix: <code>df.quer</code>

Preguntas frecuentes

¿query o máscara tradicional?

Máscara para filtros simples (1-2 condiciones) y autocomplete del IDE. query para filtros largos (4+ condiciones), parametrizables (`@var`), o cuando el lector lo encuentra más claro.

¿eval y query siempre son más rápidos?

No — solo en datasets grandes (>100k) con expresiones complejas. Para pequeños son iguales o ligeramente más lentos.

¿Qué operadores acepta query?

Aritméticos (+ - / *), comparación (==, <, >, <=, >=, !=), boolean (and, or, not o &, |, ~), in, not in. No funciones.

¿query reemplaza a SQL en pandas?

Para filtros, sí (mismo nivel expresivo). Para joins y aggregations, no — usa merge y groupby clásicos. Para datasets >1GB, considera DuckDB directo.

¿Hay eval peligroso (security)?

`pd.eval` no usa `exec()` Python — es parser propio limitado. No ejecuta arbitrary code. Pero: nunca pases strings de usuario sin sanitizar a query/eval.

Referencias

- VanderPlas, cap. 3 § 3.13.
- pandas eval/query docs
- numexpr project

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 033 — Clase 033 — Polars: DataFrames modernos

Parte: 0 — Prerrequisitos · Fuente: Polars User Guide + Vink (2020+) · Duración estimada: 75 min.

Objetivo

Conocer Polars — la librería de DataFrames moderna (Rust + Arrow) que está reemplazando a pandas en proyectos donde performance importa. Aprender su API (similar a pandas pero con expresiones lazy y paralelismo automático) y entender cuándo conviene Polars sobre pandas (datasets > 1 GB, pipelines con muchas transformaciones, multi-core).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Instalar Polars (pip install polars) y leer datos con `pl.read_csv`, `pl.read_parquet`, `pl.scan_csv` (lazy).
- Aplicar la API de expresiones: `df.select(pl.col('precio').sum())`, `pl.col('x').filter(pl.col('y') > 0).mean()`.
- Diferenciar eager (DataFrame) de lazy (LazyFrame) — y por qué lazy permite optimizaciones del query planner.
- Hacer groupby, join, pivot, unpivot con sintaxis Polars y comparar con pandas.
- Reconocer el speedup típico: 5-30× sobre pandas en operaciones comunes (single-machine, multi-core).

Temas

- Polars vs pandas vs DuckDB: el panorama 2026.
- Arrow como formato columnar in-memory.
- Eager (DataFrame) vs Lazy (LazyFrame).
- Expresiones encadenables: `pl.col(...).operation()`.
- Query optimization automática: predicate pushdown, projection pushdown.
- Multi-threading automático.

Definiciones y características

- Polars: librería de DataFrames escrita en Rust, ABI estable en Python.
- `pl.DataFrame`: estructura eager — los datos están en RAM.
- `pl.LazyFrame`: descripción de pipeline; ejecución diferida hasta `.collect()`.
- Expresión (`pl.col`, `pl.lit`): describe operación sobre columna; se compone y optimiza.
- `pl.scan_csv` / `scan_parquet`: leer lazy — solo se materializa lo que necesitas.
- Query optimization: el planner reordena filtros y projections para minimizar trabajo.
- streaming: para datasets mayores que RAM (`collect(streaming=True)`).

Dataset / recursos

- NYC Taxi (~100 MB Parquet) o cualquier CSV > 100 MB.
- Librerías: polars, pandas (para comparar), pyarrow.

Ejercicios

1. Eager básico: `df = pl.read_csv('archivo.csv')`; `df.head()`; `df.describe()`. Comparar con pandas.

2. Expresiones: `df.filter(pl.col('precio') > 100).group_by('categoria').agg(pl.col('precio').mean().alias('precio_medio'))`.
3. Lazy + collect: `lf = pl.scan_csv('big.csv').filter(...).group_by(...).agg(...); result = lf.collect()`. Comparar tiempo vs eager.
4. Query plan: `lf.explain()` muestra el plan optimizado. Identificar predicate pushdown.
5. Pandas ↔ Polars: `df.to_pandas()` y `pl.from_pandas(pd_df)`. Útil para mantener compatibilidad gradual.

Homework verificable

Sobre un dataset > 500 MB (NYC Taxi recomendado):

1. Pipeline en pandas y en Polars (lazy): filtrar viajes con `tip_amount > 5`, agrupar por hora del día, calcular media y desviación.
2. Medir wall time de cada uno.
3. Reportar speedup.

Criterio de aceptación: Polars debe ser $\geq 3\times$ más rápido que pandas; los resultados numéricos deben coincidir (± 0.001).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Mezclar APIs (<code>df['col'].sum()</code> estilo panda)	Funciona pero no usa optimizaciones. Fix:
LazyFrame no devuelve datos	Es perezoso. Fix: <code>.collect()</code> al final del
Sin <code>streaming=True</code> en datasets enormes	OOM. Fix: <code>lf.collect(streaming=True)</code> .
Columnas con tipos string mezclados → <code>sche</code>	Polars es estricto con tipos. Fix: <code>pl.read</code>
Esperar índices como en pandas	Polars NO tiene índice. La columna que qui

Preguntas frecuentes

¿Polars reemplaza a pandas?

Para data engineering / pipelines: cada vez más sí. Para ad-hoc analysis en Jupyter y compatibilidad con todo el ecosistema (sklearn, statsmodels, plotting): pandas sigue siendo el default. Ambos coexisten.

¿Polars o DuckDB?

Polars: API DataFrame Python-first. DuckDB: SQL embebido, mejor para joins masivos y queries SQL. Se complementan: muchos pipelines usan DuckDB para reads grandes + Polars para transformaciones.

¿`pl.col` vs `df['col']`?

`pl.col` es una expresión (reusable, optimizable). `df['col']` materializa una Series — útil para inspección pero no para pipelines.

¿Polars en producción?

Sí. Empresas como Goldman Sachs, Citadel, Bank of America lo usan. Estable, mantenimiento activo, patrocinado por la compañía Polars Cloud.

¿Polars soporta lazy streaming desde S3?

Sí: `pl.scan_parquet('s3://bucket/*.parquet')` con credentials configuradas. Lee solo lo que necesita.

Referencias

- Polars User Guide
- Polars Cookbook
- McKinney (2022 blog), Apache Arrow and the "10 Things I Hate About pandas".
- Comparativa: <<https://h2oai.github.io/db-benchmark/>> — Polars consistentemente top.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 034 — Clase 034 — Parquet, Arrow, PyArrow, DuckDB

Parte: 0 — Prerrequisitos · Fuente: docs Apache Arrow + DuckDB + Wes McKinney blogs. Duración estimada: 75 min.

Objetivo

Conocer el stack columnar moderno que reemplaza al CSV para datos serios: Parquet (formato en disco), Arrow (formato in-memory), PyArrow (la implementación Python), y DuckDB (SQL embebido sobre Parquet/Arrow). Saber por qué el ecosistema entero (Polars, pandas 2.x, Spark, BigQuery, DataFusion) convergió a este stack.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Leer y escribir Parquet con pandas, polars y pyarrow.
- Aplicar column pruning (leer solo columnas necesarias) y predicate pushdown (leer solo filas necesarias).
- Manejar particionado por columna (year=2024/month=03/) para queries eficientes.
- Hacer queries SQL con DuckDB directamente sobre Parquet sin cargarlo a RAM.
- Reconocer ventajas de Arrow: zero-copy entre librerías (Polars ↔ pandas ↔ Spark).

Temas

- CSV: limitaciones (sin tipos, fila por fila, sin compresión).
- Parquet: columnar, comprimido (snappy/zstd), tipos preservados, metadata por chunk.
- Arrow: formato in-memory zero-copy.
- PyArrow: API Python para ambos.
- DuckDB: "SQLite para analytics", consulta Parquet directo.
- Particionado tipo Hive.

Definiciones y características

- Parquet: formato columnar en disco. Standard de facto para data engineering.
- Arrow: formato columnar in-memory estandarizado. Zero-copy entre lenguajes.
- PyArrow: implementación Python de Arrow (pa.Table, pa.parquet.read_table).
- Column pruning: leer solo las columnas requeridas → ahorra I/O y RAM.
- Predicate pushdown: aplicar filtros en el read level → no carga lo que descartas.
- DuckDB: in-process SQL OLAP DB. pip install duckdb y ya.
- Hive partitioning: directorios clave=valor/; el lector deduce particiones.

Dataset / recursos

- NYC Taxi Parquet (Cloudfront público).
- Librerías: pyarrow, duckdb, polars, pandas.

Ejercicios

1. CSV → Parquet: leer un CSV grande con pandas, escribir Parquet con `df.to_parquet('file.parquet', compression='zstd')`. Comparar tamaño en disco (típicamente 3-10× menor).
2. Column pruning: leer SOLO 2 columnas de un Parquet de 50 columnas con `pq.read_table('f.parquet', columns=['a', 'b'])`. Comparar tiempo vs leer todo.
3. DuckDB sobre Parquet: `duckdb.sql("SELECT date, AVG(amount) FROM 'taxi/*.parquet' WHERE amount > 10 GROUP BY date").df()`. Sin cargar nada explícitamente.
4. Particionado: escribir `df.to_parquet('out', partition_cols=['year', 'month'])`. Inspeccionar estructura de directorios.
5. Arrow zero-copy: `arrow_table = polars_df.to_arrow(); pandas_df = arrow_table.to_pandas()`. Verificar que es rápido (no copia memoria).

Homework verificable

Sobre NYC Taxi (varios meses Parquet, ~5 GB total):

1. Query con DuckDB: total tip_amount por mes para passenger_count >= 2. Sin descargar todos los archivos a memoria.
2. Reportar tiempo y RAM peak (psutil o resource).
3. Comparar contra pandas leyendo todo y filtrando.

Criterio de aceptación: DuckDB completa el query usando < 2 GB RAM mientras pandas explota (OOM o muy lento).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Parquet sin compresión → archivos enormes	Fix: <code>compression='zstd'</code> (default moderno)
<code>read_parquet</code> carga todo aunque solo necesi	Sin column pruning. Fix: <code>columns=['a', 'b']</code>
DuckDB no encuentra archivos	Path glob mal. Fix: <code>'data/.parquet', 'data'</code>
Particionado no detectado	Estructura no Hive. Fix: directorios <code>key=v</code>
Mixing PyArrow Table con pandas DataFrame	Confusión. Fix: decidir un formato; conver

Preguntas frecuentes

¿Parquet o CSV en 2026?

Parquet, salvo para interchange manual con herramientas no técnicas (Excel). 3-10× más chico, 10-100× más rápido de leer, tipos preservados.

¿DuckDB o Spark?

DuckDB para single-machine, datasets < TB. Spark para cluster, > TB. La frontera se mueve: DuckDB maneja hasta cientos de GB cómodamente.

¿Arrow es solo formato o también compute?

Ambos. Arrow Compute (`pyarrow.compute`) tiene operaciones vectorizadas. Polars usa el motor de Arrow + propio (rust).

¿Por qué zstd?

Compresión mejor que snappy a costo similar de CPU. Es el default en formats modernos (Polars write_parquet default).

¿pandas 2.x soporta Arrow nativo?

Sí — `pd.read_csv(..., dtype_backend='pyarrow')` usa Arrow types internamente. Más rápido, menos RAM.

Referencias

- Apache Arrow project
- Apache Parquet docs
- DuckDB docs
- McKinney (2017 blog), Apache Arrow and the "10 Things I Hate About pandas".

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 035 — Clase 035 — Matplotlib: anatomía figura/axes

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 § 4.1 Visualization with Matplotlib.

Duración estimada: 60 min.

Objetivo

Que el alumno entienda la jerarquía de objetos de matplotlib (Figure → Axes → Artist) y use la API orientada a objetos (`fig`, `ax = plt.subplots()`) en vez del interfaz pyplot estilo MATLAB. Esto es lo que separa gráficos publicables de notebooks de cualquier curso introductorio.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar la jerarquía Figure → Axes → Artist y por qué la API OO es preferible.
2. Crear una figura con `fig`, `ax = plt.subplots(figsize=(8, 4))` y configurar título, ejes, leyenda.
3. Guardar una figura a PNG/SVG/PDF con DPI controlado.
4. Cerrar figuras explícitamente para liberar memoria en notebooks que generan muchas.
5. Configurar defaults con `plt.rcParams` (font, line width, colors).

Temas

#	Tema	Por qué importa
1	Figure (canvas) → Axes (gráfico) → Artist	Modelo mental.
2	pyplot vs OO API	<code>plt.plot</code> (state-based) vs <code>ax.plot</code> (explíci
3	<code>fig</code> , <code>ax = plt.subplots()</code>	El patrón canónico.
4	<code>fig.savefig</code> y formatos	PNG raster vs SVG/PDF vector.
5	Liberar memoria: <code>plt.close(fig)</code>	Importante en loops.
6	<code>plt.rcParams</code> y stylesheets	Defaults globales.

Definiciones y características

Figure

: El canvas/lienzo completo (ventana, archivo PNG). Una Figure contiene N Axes.

Axes

: Un gráfico individual con sus ejes X/Y, ticks, labels, leyenda. No es plural de axis — es la palabra técnica de matplotlib para 'el rectángulo donde se dibuja'.

Artist

: Todo lo dibujado: lines (Line2D), puntos (PathCollection), texto, leyenda. Cada element es un Artist con propiedades modificables.

API OO vs pyplot

: OO: `fig, ax = plt.subplots(); ax.plot(...)` — explícito, escalable. pyplot: `plt.plot(...)` — estilo MATLAB, estado global, menos predecible. Usa OO siempre.

rcParams

: Dict global con defaults de matplotlib: `plt.rcParams['figure.figsize'] = (10, 5)`. Modificar afecta todos los plots subsiguientes hasta reset.

Dataset / recursos

Sintético: serie temporal corta + scatter. Sin descarga.

Ejercicios

1. Hello world. Crea figura 8×4, plot de $y = \sin(x)$ para $x \in [0, 2\pi]$. Título, xlabel, ylabel.
2. Dos líneas en un axes. Misma figura: $\sin(x)$ y $\cos(x)$ con colores distintos y leyenda.
3. Guarda 3 formatos. Mismo plot a PNG (100 DPI), PNG (300 DPI), SVG. Compara tamaños.
4. Loop sin leak. Genera 20 plots en loop. Cierra cada uno con `plt.close(fig)`. Verifica que `len(plt.get_fignums())` queda en 0.
5. rcParams. Cambia `font.size` y `lines.linewidth` para tu sesión. Verifica el efecto.

Homework verificable

Notebook: (a) figura sin/cos con todos los elementos (título, labels, leyenda, grid); (b) guardar PNG@300dpi y SVG; (c) generar 50 plots en loop sin memory leak; (d) demo de rcParams modificados.

Criterio de aceptación: Plot publicable (labels, leyenda, tamaño razonable). Loop deja 0 figuras abiertas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Mi notebook se llena de memoria al hacer <code>m</code>	Cada <code>plt.subplots()</code> deja Figure abierta. F
<code>savefig('out.png')</code> da PNG con fondo transp	Default es transparent. Fix: <code>fig.savefig('</code>
Labels cortados al guardar	Bounding box no incluye text fuera del axe
<code>plt.plot(x, y)</code> no muestra nada en notebook	Falta <code>%matplotlib inline</code> (default suele es
Texto en español sale mal	Default font no tiene tildes/ñ. Fix: <code>plt.r</code>

Preguntas frecuentes

¿`fig, ax = plt.subplots()` o `fig = plt.figure(); ax = fig.add_subplot()`?

subplots() es el shortcut más usado. Da figure + axes en una línea, devuelve grid si pasas (n, m). El segundo es más explícito y útil para layouts custom (GridSpec).

¿PNG, SVG o PDF?

PNG para web/presentaciones (raster, DPI fijo). SVG para documentos editables / publicación (vector, escala infinita). PDF para LaTeX (también vector).

¿Cuándo constrained_layout=True vs tight_layout()?

constrained_layout=True (al crear figure): más nuevo, más confiable, maneja colorbars y leyendas externas mejor. tight_layout() (después): legacy, falla con elementos no estándar.

¿Por qué mi plot se ve diferente entre script y notebook?

Backend distinto: notebook usa inline, script usa Qt5Agg/Tk. DPI y tamaño cambian. Para consistencia: plt.rcParams['figure.dpi'] = 100 explícito.

¿Está bien usar plt.plot() directo?

Para 1 plot rápido en notebook, OK. Para cualquier cosa más compleja (subplots, savefig, reutilizable), siempre API OO.

Referencias

- VanderPlas, cap. 4 § 4.1.
- matplotlib quick start
- Anatomy of a figure (matplotlib gallery)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 036 — Clase 036 — Matplotlib: line, scatter, bar, histogram, boxplot

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 §§ 4.2–4.5 Simple Line/Scatter/Bar/Histogram Plots.

Duración estimada: 75 min.

Objetivo

Que el alumno conozca los 5 plots básicos que cubren el 80% del trabajo de EDA, y sepa cuándo cada uno: line (tendencia temporal), scatter (relación dos variables), bar (categóricas), histogram (distribución), boxplot (5 estadísticos + outliers).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Elegir el plot correcto según el tipo de variables (continua/categórica) y el objetivo.
2. Ajustar marker, color, linestyle, alpha para legibilidad.
3. Construir histogramas con bins adecuados (regla de Freedman-Diaconis o 'auto').
4. Interpretar boxplot: mediana, Q1/Q3, whiskers, outliers.

5. Combinar bar + error bars para mostrar incertidumbre.

Temas

#	Tema	Por qué importa
1	Line: tendencias y series temporales	El más fácil de leer mal.
2	Scatter: relación entre dos variables	Con c= y s= para 3ª/4ª dimensión.
3	Bar y barh: categóricas	Vertical vs horizontal.
4	Histogram: distribución de una continua	Bins importan.
5	Boxplot: distribución resumida + outliers	Cuando hay muchos grupos.
6	Errorbar y fill_between	Mostrar incertidumbre.

Definiciones y características

Line plot

: Une puntos con líneas. Implica continuidad/orden en X — solo úsalo cuando X tiene orden natural (tiempo, espacio, secuencia).

Scatter

: Puntos no conectados. Muestra relación entre 2 continuas. Con c= codificas 3ª dim (color), con s= 4ª (tamaño). Más de 4 dims sobrecarga.

Bar / barh

: Barras para categóricas. Vertical (bar) si etiquetas son cortas, horizontal (barh) si son largas o muchas.

Histogram

: Distribución de UNA continua. Bins importan: pocos esconden estructura, muchos generan ruido. bins='auto' usa Freedman-Diaconis (buen default).

Boxplot

: Resumen de distribución: mediana (línea), Q1-Q3 (caja), whiskers (1.5×IQR), outliers (puntos). Útil para comparar muchos grupos rápido.

Errorbar

: Barra + línea vertical/horizontal indicando incertidumbre (std, IC95%). Sin esto, las barras mienten visualmente.

Dataset / recursos

Palmer Penguins. Sin descarga adicional.

Ejercicios

1. Line. Serie temporal de ventas mensuales (sintética). Anota máximo con flecha.
2. Scatter. body_mass vs bill_length, color por species. Adicionalmente: s= con flipper_length para tamaño.
3. Bar. Count por species, ordenado descendente. Vertical y horizontal — compara legibilidad.
4. Histogram. Distribución de body_mass con bins='auto' y bins=10. Compara.
5. Boxplot. body_mass por species: 3 cajas lado a lado. Identifica outliers.

Homework verificable

Notebook con penguins: (a) 5 plots básicos cada uno bien etiquetado; (b) scatter decorado con color y tamaño codificando 3 dimensiones; (c) bar con errorbars de std; (d) boxplot agrupado con interpretación de outliers.

Criterio de aceptación: Cada plot tiene título, labels, leyenda donde aplica. Bins justificados.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Line plot entre 2 categorías "A", "B"	Conectar categorías con línea engaña (sugi
Histograma con escala Y rara	Por default density=False (counts). Si qui
Boxplot todos iguales por outliers extremo	Outliers dominan visualmente; cajas quedan
Bar chart con colores random distrae	Sin codificación significativa, color = ru
Scatter con miles de puntos = blob negro	Overplotting. Fix: alpha=0.3, hexbin (plt.

Preguntas frecuentes

¿Pie chart cuándo?

Casi nunca. El ojo humano compara mal ángulos. Para proporciones: bar o stacked bar. Pie tolerable solo con 2-3 categorías y proporciones muy distintas.

¿Cuántos bins en un histograma?

bins='auto' (Freedman-Diaconis) es buen default. Si es estudios académicos: regla de Sturges (bins=int(np.log2(n)+1)). Experimenta con 10/30/50 si dudas.

¿Boxplot o violinplot?

Boxplot: rápido, 5 estadísticos, outliers claros. Violinplot: muestra distribución completa (multimodalidad). Para comparar 3-10 grupos, ambos OK. Para >10, boxplot gana en densidad.

¿Errorbars con std o con IC?

Std: dispersión natural de los datos. IC95% de la media: incertidumbre del estimador (más pequeño con N grande). Para inferencia, IC. Para describir, std.

¿Plot 3D buen idea?

Casi nunca. Oclusión + perspectiva engañan. 2D con color/tamaño suele comunicar mejor. Excepción: superficies analíticas $z = f(x, y)$.

Referencias

- VanderPlas, cap. 4 §§ 4.2-4.5.
- matplotlib gallery

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 037 — Clase 037 — Matplotlib: subplots y gridspec

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 § 4.6 Multiple Subplots.

Duración estimada: 60 min.

Objetivo

Que el alumno organice múltiples plots en una sola figura — con `plt.subplots(n, m)` para grillas regulares y con `GridSpec` para layouts irregulares (un plot grande + varios pequeños). Crítico para informes y dashboards.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear grillas regulares con `fig, axes = plt.subplots(2, 3, figsize=...)`.
2. Iterar sobre `axes.flat` para llenar la grilla con loops.
3. Compartir ejes con `sharex=True`, `sharey=True` para comparar.
4. Usar `GridSpec` para layouts irregulares (1 grande + 3 pequeños).
5. Usar `constrained_layout=True` en vez de `tight_layout()` (más confiable).

Temas

#	Tema	Por qué importa
1	<code>plt.subplots(nrows, ncols)</code>	Grilla regular.
2	Iterar con <code>.flat</code>	Llenar muchos plots en loop.
3	<code>sharex/sharey</code>	Comparar con misma escala.
4	<code>GridSpec</code> para layouts irregulares	1 grande + N pequeños.
5	<code>constrained_layout</code> vs <code>tight_layout</code>	El primero es mejor.
6	<code>add_subplot</code> con posiciones custom	Cuando necesitas full control.

Definiciones y características

`plt.subplots(n, m)`

: Crea figure + grilla regular de n filas × m columnas de axes. Devuelve (fig, axes) donde axes es array 2D — itera con `.flat` para llenar.

`sharex / sharey`

: Comparten escala entre axes del grid. Ideal para comparar distribuciones de la misma magnitud sin distorsión visual.

`GridSpec`

: Layout irregular avanzado: define grilla y asigna spans manualmente ("plot grande arriba + 3 pequeños abajo"). Mucho más flexible que `subplots(n, m)`.

`add_subplot(spec)`

: Añade un axes a una figure según una posición específica (índice 121, o `GridSpec`). Útil cuando `subplots` no alcanza.

`constrained_layout=True`

: Algoritmo moderno (default en pandas 3+) que ajusta spacing automáticamente. Maneja colorbars, leyendas externas, `suptitle` sin overlap. Recomendado sobre `tight_layout()`.

Dataset / recursos

Palmer Penguins. Sin descarga.

Ejercicios

1. Grilla 2×2. 4 histogramas de las 4 features numéricas de penguins en una figura.
2. Grilla con loop. Itera `axes.flat` para plot consistente.
3. `sharey=True`. 3 boxplots por species lado a lado con misma escala Y.
4. `GridSpec` irregular. Un scatter grande (2×2) + 1 hist arriba (1×2) + 1 hist a la derecha (2×1) — marginal histograms.
5. `constrained_layout`. Compara una figura compleja con `tight_layout()` vs `constrained_layout=True` — observa diferencia.

Homework verificable

Notebook con penguins: (a) grilla 2×2 hist; (b) 3 boxplots con `sharey`; (c) layout `GridSpec` con scatter central + marginales arriba/derecha; (d) misma figura comparando `tight_layout` vs `constrained_layout`.

Criterio de aceptación: Sin superposición de labels. Layouts limpios. Marginales alineadas al scatter.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>axes</code> es array 1D cuando esperaba 2D	Si pides <code>subplots(1, 3)</code> o <code>subplots(3, 1)</code> ,
<code>subplots(1, 1)</code> devuelve <code>axes</code> escalar	No es array. Fix: si quieres array siempre
Plots se superponen / labels cortados	Sin <code>constrained_layout=True</code> ni <code>tight_layout</code>
<code>sharey=True</code> pero un plot tiene escala dist	Quizás esa subplot necesita un eje secunda
<code>GridSpec</code> con índices confusos	El orden es <code>gs[filas, cols]</code> igual que NumPy.

Preguntas frecuentes

¿Cuándo `subplots(n, m)` vs `GridSpec`?

Regular: `subplots`. Irregular (un grande + varios pequeños, joint plot, dashboard): `GridSpec`.

¿`fig.suptitle` o `ax.set_title` con `subplots`?

`suptitle` = título de toda la figura (encima de todos). `ax.set_title` = título por subplot. Combinables.

¿Cómo controlo espacio entre subplots?

`subplots(..., gridspec_kw={'hspace': 0.3, 'wspace': 0.3})` (ratios relativos al tamaño del axes). Con `constrained_layout` suele ser automático.

¿`figsize` cómo elegir?

Para artículos: ancho de columna (típicamente 3.5" para 1 col, 7" para ancho página). Para presentaciones: aspect ratio 16:9 → (12, 6.75).

¿`axes.flat` o `axes.flatten()`?

`flat` es iterador (no copia). `flatten()` devuelve nuevo array. Para iterar, `flat` ahorra memoria.

Referencias

- VanderPlas, cap. 4 § 4.6.

- GridSpec tutorial

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábralo desde el laboratorio del programa o desde Jupyter.

Clase 038 — Clase 038 — Matplotlib: legends, colorbars, ticks, anotaciones

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 §§ 4.7–4.9 Customizing Legends, Colorbars, Ticks.

Duración estimada: 60 min.

Objetivo

Que el alumno controle los detalles que distinguen un plot ad-hoc de uno publicable: leyenda fuera del gráfico, colorbar discreto, ticks personalizados, y anotaciones (flechas, texto) para guiar la atención del lector.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Posicionar leyenda fuera del axes con `bbox_to_anchor`.
2. Configurar colorbar con label, ticks discretos, y categoría.
3. Personalizar ticks: rotación, formato (`FuncFormatter`, `PercentFormatter`), scale log.
4. Anotar puntos con `ax.annotate(..., xy=..., xytext=..., arrowprops=...)`.
5. Añadir líneas de referencia con `axhline/axvline` (umbrales, medias).

Temas

#	Tema	Por qué importa
1	Legend con <code>bbox_to_anchor</code>	Sacarla del axes.
2	Colorbar con label y ticks discretos	Cuando hay codificación por color.
3	Tick formatters: percent, scientific, cust	Legibilidad.
4	<code>ax.annotate</code> con flecha	Resaltar un punto específico.
5	<code>axhline</code> / <code>axvline</code> / <code>axhspan</code>	Líneas y bandas de referencia.
6	Log scale: <code>ax.set_yscale('log')</code>	Cuando hay rango grande.

Definiciones y características

Leyenda (`ax.legend`)

: Mapeo labels↔elementos del plot. Auto-detecta si pones `label=` en `plot/scatter`. Posición con `loc=` o `bbox_to_anchor=` para colocarla fuera del axes.

Colorbar

: Escala de color para plots con codificación continua (`scatter(c=valor)`, `imshow`). Indica cómo se mapean valores a colores. Siempre etiqueta con `cbar.set_label(...)`.

Tick / TickFormatter

: Marcas en los ejes y sus etiquetas. Formatter define el formato: `PercentFormatter`, `FuncFormatter(lambda x,p: f'${x:.0f}')`, `LogFormatter`.

`ax.annotate`

: Texto con flecha apuntando a un punto. Parámetros: `xy` (punto a apuntar), `xytext` (posición del texto), `arrowprops` (estilo flecha).

Líneas/bandas de referencia

: `axhline(y)` horizontal, `axvline(x)` vertical, `axhspan(y1, y2)` banda horizontal. Útiles para umbrales, medias, eventos.

Dataset / recursos

Sintético: serie con outliers, scatter con categorías.

Ejercicios

1. Leyenda fuera. Plot con 5 líneas, leyenda a la derecha fuera del axes.
2. Colorbar. Scatter con `c=` continuo (ej: density), colorbar con label.
3. PercentFormatter. Bar chart con eje Y formateado como porcentaje.
4. Anotar outlier. Scatter con un punto extremo; flecha + texto identificándolo.
5. Log scale. Plot de valores con rango grande (1, 10, 100, 1000); compara linear vs log.

Homework verificable

Notebook con: (a) plot multi-línea con leyenda externa; (b) scatter con colorbar etiquetado; (c) bar % usando PercentFormatter; (d) plot con anotación de máximo via flecha; (e) comparativa lineal vs log en datos exponenciales.

Criterio de aceptación: Cada elemento visual tiene propósito. Anotaciones legibles, no superpuestas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Leyenda tapa parte del plot	Default es 'best' que a veces no encuentra
Colorbar es enorme/desproporcionada	Default fill toda la altura. Fix: <code>fig.colorbar(..., shrink=0.5)</code>
Ticks rotados se cortan	Texto rotado no entra en el bounding box.
<code>annotate</code> con flecha que apunta mal	<code>xy</code> y <code>xytext</code> están en coordenadas de datos
<code>axhline(media)</code> invisible	Pintada gris claro encima de fondo similar

Preguntas frecuentes

¿Leyenda dentro o fuera?

Dentro si hay espacio (1-3 series, plot no saturado). Fuera (`bbox_to_anchor`) si son 5+ series o el plot está denso.

¿Colorbar discreta o continua?

Continua si los datos son continuos (densidad, precio). Discreta (con `N` boundaries) si son categóricas/cuantiles — `BoundaryNorm` + `ListedColormap`.

¿`set_xticks` o `set_xticklabels`?

`set_xticks` define dónde van las marcas. `set_xticklabels` define qué texto se muestra. Usa ambos juntos para control fino; nunca uses solo el segundo (deprecation warning).

¿Cómo añadir emojis/Unicode a texts?

Default font (DejaVu Sans) trae básicos. Para emoji color, fuente especial ('Segoe UI Emoji' en Windows). Mejor: evita emojis dentro del plot, úsalos en título/labels markdown.

¿Log scale para presentaciones?

Si los datos cubren >2 órdenes de magnitud (1, 100, 10000), sí. Pero siempre indícalo en el título o axis label ("escala log") — no es obvio.

Referencias

- VanderPlas, cap. 4 §§ 4.7-4.9.
- matplotlib text and annotations

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 039 — Clase 039 — Matplotlib: stylesheets

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 § 4.11 Customizing Matplotlib: Configurations and Stylesheets.

Duración estimada: 30 min.

Objetivo

Que el alumno aproveche stylesheets built-in y propios para mantener consistencia visual entre plots y proyectos — y deje de configurar manualmente rcParams en cada notebook.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Listar stylesheets disponibles con `plt.style.available`.
2. Aplicar un style globalmente (`plt.style.use(...)`) o solo a un bloque (with `plt.style.context(...)`).
3. Crear style propio en un archivo `.mplstyle` y usarlo.
4. Combinar styles (uno + ajustes manuales).
5. Elegir style según contexto (informe, presentación, B&N para impresión).

Temas

#	Tema	Por qué importa
1	<code>plt.style.available</code>	Catálogo built-in.
2	<code>plt.style.use(...)</code> global	Afecta todos los plots subsiguientes.
3	with <code>plt.style.context(...)</code>	Temporal, ideal para un bloque.
4	Archivo <code>.mplstyle</code> propio	Reusar entre proyectos.
5	Stylesheets comunes	default, seaborn-v0_8-whitegrid, ggplot, f
6	<code>rcParams</code> override puntual	Style + ajuste fino.

Definiciones y características

Stylesheet

: Conjunto de rcParams predefinidos en un archivo .mplstyle. Activa con `plt.style.use('nombre')`. Cambia colors, fonts, grids, spines, etc., consistentemente.

`plt.style.available`

: Lista de stylesheets built-in: default, ggplot, seaborn-v0_8-whitegrid, bmh, grayscale, dark_background, etc.

`plt.style.context(name)`

: Aplica style solo dentro de un with block; al salir, vuelve al anterior. Útil cuando quieres style distinto para 1 plot sin afectar el resto.

`.mplstyle propio`

: Archivo de texto con clave: valor (igual sintaxis que rcParams). Ubícalo donde sea y carga con `plt.style.use('/ruta/al.mplstyle')`.

rcParams override en context

: with `plt.rc_context({'figure.figsize': (12, 6)})`: aplica overrides temporales sobre el style activo.

Dataset / recursos

Sintético: mismo plot en varios styles. Sin descarga.

Ejercicios

1. Catalogo. Imprime `plt.style.available`. Identifica 5 que suenen útiles.
2. Galería visual. Mismo scatter plot bajo 4 styles distintos (default, ggplot, seaborn-whitegrid, grayscale).
3. Bloque temporal. Con `with plt.style.context('seaborn-v0_8-darkgrid')`: aplica style solo a 1 figura.
4. Style propio. Crea `mi_style.mplstyle` con tus defaults preferidos. Úsalo.
5. Style + override. Aplica ggplot y luego cambia `figure.figsize` para un plot específico.

Homework verificable

Notebook: (a) galería de 4 styles sobre un mismo dataset; (b) crear `informe.mplstyle` con paleta corporativa simulada (3 colores principales); (c) demo de uso temporal con `plt.style.context`; (d) comparativa B&N (grayscale) vs color para una figura que podría imprimirse.

Criterio de aceptación: Galería con plots reconocibles; style propio aplica colores definidos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>plt.style.use('seaborn')</code> da error	Renombraron a <code>seaborn-v0_8-whitegrid</code> (y va
Style activado pero un plot no lo respeta	Aplicado después de crear el axes. Style a
Colors definidos en .mplstyle no aparecen	Sintaxis <code>cycler</code> incorrecta. Fix: <code>axes.prop</code>
<code>dark_background</code> rompe scatter color	Background oscuro, color de marker default
Style se mantiene tras cerrar notebook	<code>style.use</code> afecta rcParams globales del ker

Preguntas frecuentes

¿Style propio o usar built-in?

Built-in para empezar. Propio cuando tu organización/proyecto tiene paleta corporativa o convenciones

específicas.

¿Style afecta seaborn?

Seaborn maneja su propio system con `sns.set_theme()`. Coordinar ambos puede colisionar. Recomendado: elige uno (style de matplotlib O `sns.set_theme`).

¿Cómo veo todos los styles?

`plt.style.available` lista todos. Para galería visual: [matplotlib style sheets reference](#).

¿bmh qué es?

Style del libro Bayesian Methods for Hackers — popular para gráficos estadísticos limpios.

¿Style para impresión B&N?

grayscale convierte automáticamente. Pero verifica: `cmaps` con poca variación de luminancia ('jet') quedan ilegibles. Mejor 'viridis' o `linestyles` distintos.

Referencias

- VanderPlas, cap. 4 § 4.11.
- matplotlib stylesheets gallery

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 040 — Clase 040 — Matplotlib: 3D plotting

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 § 4.12 Three-Dimensional Plotting in Matplotlib.

Duración estimada: 45 min.

Objetivo

Que el alumno sepa cuándo (raramente) usar 3D y cómo hacerlo bien: scatter 3D, superficies (`plot_surface`), wireframes y contornos. Spoiler: la mayoría de las veces un buen 2D + color comunica mejor.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear axes 3D con `projection='3d'`.
2. Scatter, line, surface, wireframe, contour en 3D.
3. Controlar ángulo de vista con `ax.view_init(elev, azim)`.
4. Reconocer cuándo NO usar 3D: la mayoría de las veces hay una alternativa 2D mejor.

Temas

#	Tema	Por qué importa
1	<code>projection='3d'</code>	Habilita el 3D toolkit.
2	Scatter 3D con codificación por color	3 dims + 4 (color).
3	<code>plot_surface</code> para $z = f(x, y)$	Funciones bivariadas.

4	plot_wireframe y contour3D	Alternativas más simples.
5	view_init: rotar interactivo	En notebooks con %matplotlib widget.
6	Cuándo NO usar 3D	Casi siempre.

Definiciones y características

projection='3d'

: Parámetro al crear axes (fig.add_subplot(111, projection='3d')) que habilita el toolkit 3D (mplot3d). Sin esto, las llamadas 3D fallan.

plot_surface(X, Y, Z)

: Superficie 3D para $z = f(x, y)$. Requiere mesh: $X, Y = \text{np.meshgrid}(x, y)$; $Z = f(X, Y)$. Soporta cmap para color por valor de Z.

plot_wireframe(X, Y, Z)

: Como surface pero solo líneas — más liviano, mejor para densidad alta, peor para forma.

view_init(elev, azim)

: Define ángulo de cámara. elev elevación (0=horizontal, 90=vista superior). azim rotación horizontal en grados. Animar varia azim para 360°.

Oclusión

: Limitación fundamental del 3D: objetos al frente tapan los del fondo, sin forma confiable de elegir qué ver. La razón principal por la que 3D es problemático.

Dataset / recursos

Sintético: superficie analítica + nube de puntos. Sin descarga.

Ejercicios

1. Scatter 3D. 200 puntos con coords (x, y, z) y color por una 4ª variable.
2. Superficie. $z = \sin(\sqrt{x^2 + y^2})$ en mesh 50×50. plot_surface con colormap.
3. Wireframe + contour. Misma función con plot_wireframe. Compara legibilidad con superficie llena.
4. view_init. Cambia (elev, azim) a 4 ángulos y graba una grilla 2×2.
5. Reto: 2D que vence al 3D. Para tu scatter 3D del ejercicio 1, propón un 2D + color/tamaño que comunique igual o mejor.

Homework verificable

Notebook: (a) scatter 3D con 4 dimensiones (xyz + color); (b) superficie $z=f(x,y)$; (c) wireframe del mismo z; (d) grilla 2×2 con 4 view_init distintos; (e) ejercicio de "2D vence al 3D": versión 2D del scatter.

Criterio de aceptación: Plots 3D legibles (no espagueti). Versión 2D del scatter comparable.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
projection='3d' da error "unknown projecti	No importaste el toolkit. Fix: from mpl_to
Surface plot tarda mucho con mesh grande	100×100 mesh = 10k vertices. Fix: reduce r
3D scatter con miles de puntos = mancha	Oclusión. Fix: reduce N (sampling), o usa

view_init no se aplica si llamado después	El orden importa: configura ángulo ANTES d
Labels Z se cortan o solapan	3D tiene limitaciones de layout. Fix: ajus

Preguntas frecuentes

¿Realmente necesito 3D?

Pregúntate: ¿hay una versión 2D con color/tamaño que comunique igual? Casi siempre sí. 3D vale para superficies analíticas ($z = f(x,y)$), datos físicos 3D, o cuando es interactivo (rotable).

¿%matplotlib widget para rotar interactivo?

Sí — en JupyterLab/Notebook permite rotar con el mouse. Requiere pip install ipympl. Default inline da imagen estática.

¿plotly 3D mejor?

Sí para uso interactivo en web/dashboard. No para reportes estáticos (mismo problema de oclusión, peso del HTML).

¿Animaciones 3D?

matplotlib.animation.FuncAnimation + variando view_init por frame. Bonito pero costoso de generar. Considera plotly que es interactivo nativo.

¿Axes3D deprecated?

En matplotlib moderno, basta subplot_kw={'projection': '3d'} — no necesitas importar Axes3D explícitamente.

Referencias

- VanderPlas, cap. 4 § 4.12.
- matplotlib mplot3d tutorial

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 041 — Clase 041 — Seaborn: distribuciones, relaciones, categóricas, facetas

Parte: 0 — Prerrequisitos · Fuente: VanderPlas, cap. 4 § 4.13 Visualization with Seaborn · seaborn docs.

Duración estimada: 75 min.

Objetivo

Que el alumno use seaborn cuando aporta sobre matplotlib puro: defaults estéticos, API tipada para DataFrames ($x=$, $y=$, $hue=$, $col=$), distribuciones (histplot, kdeplot, displot), relaciones (scatterplot, Implot), categóricas (boxplot, violinplot, swarmplot), y facetas (grilla automática por categoría).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Usar la API moderna (figure-level vs axes-level) y elegir la correcta.

2. Construir un pairplot para EDA rápido de un DataFrame.
3. Codificar 3 dimensiones con hue, style, size.
4. Hacer facetas con col= y row= para grillas automáticas.
5. Personalizar themes con sns.set_theme(style=..., palette=...).

Temas

#	Tema	Por qué importa
1	seaborn vs matplotlib	Seaborn es matplotlib + defaults + API tip
2	Figure-level (displot, relplot, catplot) v	Cuándo cada uno.
3	hue, style, size	Codificar dimensiones extra.
4	Facetas con col, row	Grilla automática por categoría.
5	pairplot para EDA	Matriz de scatters.
6	Themes y paletas	Defaults consistentes.

Definiciones y características

Seaborn

: Wrapper sobre matplotlib con (1) defaults estéticos mejores, (2) API tipada para DataFrames (x=, y=, hue=), (3) plots estadísticos directos (regresión, distribuciones, facetas).

Figure-level (displot, relplot, catplot)

: Funciones que crean su propia figura, soportan facetas (col=, row= para grilla automática). Más alto nivel, menos control fino.

Axes-level (histplot, scatterplot, boxplot)

: Funciones que dibujan en un ax que tú pasas. Más bajo nivel, integran con layouts custom de matplotlib.

hue, style, size

: Codifican dimensiones extra: hue (color por categoría), style (marker por categoría), size (tamaño por valor continuo).

pairplot

: Matriz de scatterplots de todas las parejas de variables numéricas, diagonal con KDE/histograma. EDA visual en una línea.

Faceta (col/row)

: Una sub-figura por cada valor de una categórica. relplot(..., col='species', row='sex') produce grilla n_species × n_sex de plots automática.

Dataset / recursos

Palmer Penguins (seaborn lo trae built-in via sns.load_dataset('penguins')).

Ejercicios

1. Pairplot. Penguins, color por species. EDA en 1 línea.
2. Scatter con hue + size. body_mass vs flipper, hue por species, size por bill_length.
3. KDE distribución. body_mass por species (3 KDE en mismo plot).
4. Boxplot + swarm. Combinar boxplot con swarm para ver puntos individuales.

5. Facetas. `sns.relplot(...col='species', row='sex')` para $3 \times 2 = 6$ subplots automáticos.

Homework verificable

Notebook con penguins: (a) pairplot completo; (b) violin + swarm de `body_mass` por (`species`, `sex`); (c) faceta 2×3 de scatter; (d) tema custom + paleta; (e) decisión documentada: cuándo usar figure-level vs axes-level.

Criterio de aceptación: Plots de EDA legibles. Decisiones de hue/style/col justificadas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Mezclo <code>sns.set_theme()</code> con <code>plt.style.use()</code> .	Compiten por los <code>rcParams</code> . Fix: elige uno.
Figure-level no me deja agregar <code>ax.set_title</code>	Devuelve <code>FacetGrid</code> , no axes. Fix: <code>g.set_title</code>
pairplot con muchas columnas tarda eternid	N^2 scatters; con 20 cols son 400 plots. Fi
hue con muchas categorías → leyenda enorme	Default muestra todas las categorías. Fix:
<code>sns.scatterplot(...)</code> no aparece en notebook	Falta capturar return o <code>plt.show()</code> . Fix: a

Preguntas frecuentes

¿seaborn o matplotlib puro?

Seaborn para plots estadísticos rápidos con `DataFrame` (hue, facetas, defaults). Matplotlib para control fino, customización extrema, o plots que no son estadísticos.

¿Cuándo figure-level vs axes-level?

Figure-level si quieres facetas o el plot es el output completo. Axes-level si necesitas integrar con layout custom (subplots manuales).

¿pairplot o `corrmatrix`?

pairplot muestra relaciones visualmente (no lineales, outliers, clusters). Heatmap de `corr()` muestra coeficiente Pearson (solo lineal). Complementarios.

¿Tema corporativo en seaborn?

`sns.set_theme(palette=['#0F766E', '#D9A441', '#7C3AED'])` o palette custom: `sns.color_palette('husl', n_colors=5)`. Combina con `style='whitegrid'`.

¿Seaborn maneja datasets grandes (>1M)?

Plots scatter/hist sí. Pairplot con muchas cols se vuelve lento. Para datasets enormes, considera `datashader` o `downsample` antes.

Referencias

- VanderPlas, cap. 4 § 4.13.
- seaborn user guide
- Waskom, seaborn paper (JOSS, 2021)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 042 — Clase 042 — Visualización geográfica (Plotly / folium)

Parte: 0 — Prerrequisitos · Fuente: *Plotly Choropleth docs* · *folium docs* · *Cartographies of the Mind (background)*.

Duración estimada: 60 min.

Objetivo

Que el alumno construya mapas básicos cuando los datos tienen componente geográfico: folium (mapas Leaflet interactivos, markers, choropleth), plotly (choropleth, scatter geo). Sin entrar a GIS profundo (eso es geopandas, fuera del scope de Parte 0).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Crear mapa folium centrado, con tile layer básico.
2. Añadir markers con popup, tooltip, color según valor.
3. Construir choropleth (mapa de calor por región) con folium o plotly.
4. Decidir entre folium y plotly geo según destino (HTML standalone vs dashboard).
5. Citar fuentes de tiles y GeoJSON públicos.

Temas

#	Tema	Por qué importa
1	Sistemas de coordenadas: lat/Ing	Convención: lat primero en folium, Ing pri
2	folium: mapa + markers + popups	Mapas Leaflet en notebook.
3	folium choropleth con GeoJSON	Mapas de calor por país/región.
4	plotly choropleth y scatter_geo	Cuando ya usas plotly.
5	Tile providers (OSM, CartoDB)	Estética y licencia.
6	Cuándo geopandas	Análisis geoespacial real.

Definiciones y características

Coordenadas geográficas (lat, Ing)

: Latitud (-90 a 90): N/S del ecuador. Longitud (-180 a 180): E/W de Greenwich. Convención y orden varía: folium [lat, Ing], GeoJSON [Ing, lat] — fuente clásica de bugs.

folium

: Wrapper Python de Leaflet.js. Mapas interactivos (zoom, pan, popups) embebidos en notebook como HTML. Ideal para exploración.

plotly geo

: Mapas en plotly (scatter_geo, choropleth). Bonitos, interactivos, integran con dashboards Plotly/Dash. Para choropleth de países usa códigos ISO-3.

Choropleth

: Mapa de calor por región: color de cada polígono representa un valor (PIB, población, casos). Requiere GeoJSON con las shapes.

Tile provider

: Servidor de "baldosas" (imágenes 256×256) que componen el mapa de fondo. OpenStreetMap (default), CartoDB (limpio), Mapbox (premium).

geopandas

: Extensión de pandas con geometrías (Shapely). Para análisis espacial (intersection, buffer, dissolve), no solo visualizar. Fuera del scope Parte 0.

Dataset / recursos

Sintético: lista de ciudades con coords + métrica simulada. GeoJSON de países público desde un CDN para choropleth.

Ejercicios

1. Mapa con markers. 5 ciudades españolas con marker y popup mostrando nombre + población.
2. Markers coloreados. Mismo, pero color verde si pop>1M, rojo si <500k.
3. Choropleth folium. Mapa mundial con un valor sintético por país (ej: PIB).
4. Choropleth plotly. Lo mismo con plotly.express.choropleth.
5. Comparar. ¿Cuándo folium (mapa físico explorable) vs plotly (integra con dashboard)?

Homework verificable

Notebook: (a) mapa folium con 10+ markers + popups + tooltips; (b) choropleth folium de un dataset por país; (c) mismo choropleth con plotly express; (d) reporte 1-párrafo comparando ambos.

Criterio de aceptación: Mapas funcionales en notebook; popups muestran info correcta; choropleth con leyenda.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Marcadores aparecen en medio del océano	Invertiste lat/Ing. Fix: folium espera [la
Mapa folium no se renderiza en VS Code	Algunas extensiones de notebook no muestra
choropleth plotly sale gris	El código ISO no matchea con el shape. Fix
Mapa pesa MB y carga lento	Demasiados markers (>1000). Fix: usa Marke
Tiles fallan a cargar	Provider con rate limit (Mapbox sin token,

Preguntas frecuentes

¿folium o plotly?

folium para exploración interactiva en notebook (mapa real con zoom/pan/popups). plotly para integrar en dashboard o cuando ya usas plotly. Ambos producen HTML standalone.

¿Cómo encuentro lat/Ing de un lugar?

Click derecho en Google Maps → "¿Qué hay aquí?". O geocodifica con geopy (from geopy.geocoders import Nominatim).

¿GeoJSON dónde lo consigo?

Para países: Natural Earth (público). Para regiones: portal gov del país. Para custom: dibuja en geojson.io.

¿Cuándo necesito geopandas?

Análisis espacial real: "¿cuántos puntos caen en este polígono?", "buffer de 1km alrededor", re-proyección entre CRS. Para solo visualizar puntos en mapa: folium basta.

¿Mapas offline?

Tiles son online por default. Para offline: descarga tiles con mbtiles, sirve local con folium.raster_layers.TileLayer(url='file:///...'). Setup complejo, considera si vale.

Referencias

- folium docs
- plotly choropleth docs
- Natural Earth GeoJSON

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 043 — Clase 043 — SQL fundamental: SELECT, WHERE, JOIN, GROUP BY, HAVING

Parte: 0 — Prerrequisitos · Fuente: SQL for Data Scientists (Tanimura) caps. 1-3 · SQLite docs · DuckDB docs.

Duración estimada: 120 min.

Objetivo

Que el alumno escriba consultas SQL no triviales — SELECT con filtros, JOINS (inner/left), agregaciones con GROUP BY y filtros sobre agregados con HAVING. Y entienda el orden de ejecución lógico (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT), que es lo que confunde a todo el mundo al principio.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Escribir SELECT con filtros WHERE y operadores (=, <>, IN, BETWEEN, LIKE, IS NULL).
2. Hacer JOIN (INNER, LEFT, RIGHT, FULL) y reconocer cuándo cada uno.
3. Agrupar y agregar con GROUP BY + COUNT, SUM, AVG, MAX, MIN.
4. Filtrar agregados con HAVING (no se puede con WHERE).
5. Recitar el orden lógico: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Temas

#	Tema	Por qué importa
1	SELECT, FROM, WHERE	Lo básico, sin trampa.
2	Operadores WHERE	=, <>, IN, BETWEEN, LIKE, IS NULL.
3	JOINS (inner/left/right/full)	Análogos a pandas merge.
4	GROUP BY + agregadas	COUNT/SUM/AVG/MAX/MIN.

5	HAVING vs WHERE	HAVING filtra después de GROUP BY.
6	ORDER BY, LIMIT, OFFSET	Final del pipeline.
7	Orden lógico ≠ orden escrito	El gran malentendido.

Definiciones y características

SQL (Structured Query Language)

: Lenguaje declarativo para bases de datos relacionales. Describes qué quieres (no cómo) y el motor lo ejecuta. Estandarizado pero con dialectos (SQLite, PostgreSQL, MySQL, BigQuery).

Orden lógico vs escrito

: Escribes: `SELECT-FROM-WHERE-GROUP-HAVING-ORDER`. Ejecuta: `FROM-WHERE-GROUP-HAVING-SELECT-ORDER-LIMIT`. Por eso `WHERE SUM(...)` falla (aún no agrupado) — usa `HAVING`.

JOIN

: Combina filas de 2+ tablas por una key. Tipos: `INNER` (intersección), `LEFT` (todo left + match right), `RIGHT` (espejo), `FULL OUTER` (todo unión), `CROSS` (producto cartesiano).

GROUP BY + HAVING

: `GROUP BY`: agrupa filas por valor(es). `HAVING`: filtra los grupos después de agregar (no se puede con `WHERE`).

Funciones de agregación

: Operan sobre grupos: `COUNT(*)`, `COUNT(DISTINCT x)`, `SUM`, `AVG`, `MIN`, `MAX`, `STDDEV`. Devuelven UN valor por grupo.

DuckDB

: Motor SQL embebido (como SQLite) pero columnar y optimizado para analytics. Lee CSV/Parquet directo (`FROM 'file.csv'`). Drop-in para queries analíticas, mucho más rápido que SQLite en agregados.

Dataset / recursos

SQLite en memoria con 2 tablas sintéticas: clientes (10 filas) y ordenes (30 filas). Generado en el notebook con `sqlite3` stdlib. Sin descarga.

Ejercicios

1. `SELECT` básico. Lista de clientes con país = 'ES'.
2. `JOIN`. Cada orden con el nombre del cliente.
3. `LEFT JOIN`. Todos los clientes, sumando órdenes (NaN si no tienen).
4. `GROUP BY + HAVING`. Clientes con más de 3 órdenes y monto total > 200.
5. Orden lógico. Explica con tus palabras por qué `WHERE total > 100` no funciona si total es `SUM(monto)` — necesitas `HAVING`.

Homework verificable

Notebook con SQLite en memoria: (a) crea 2 tablas y carga datos sintéticos; (b) 5 consultas progresivas (filter, join, group, having, top-N); (c) explica el orden lógico con un ejemplo; (d) mismo ejercicio con DuckDB (sustituye `sqlite3.connect(':memory:')`).

Criterio de aceptación: Las 5 consultas producen el resultado esperado; DuckDB devuelve igual.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
column "x" must appear in GROUP BY clause	Seleccionaste col no agregada ni en GROUP
WHERE SUM(monto) > 100 falla	WHERE corre ANTES de GROUP. Fix: usa HAVING
INNER JOIN pierde filas que esperaba ver	La key no matchea (NULL, tipos, espacios).
COUNT(col) devuelve menos que COUNT(*)	COUNT(col) ignora NULL en esa columna. Fix
SELECT * después de JOIN trae cols duplica	Ambas tablas tienen id. Fix: aliasa: SELE

Preguntas frecuentes

¿COUNT(*) o COUNT(1)?

Equivalentes en motores modernos (parser optimiza). COUNT(*) es más legible — úsalo.

¿Cuándo DISTINCT?

Cuando hay filas duplicadas que no deberían contarse. Cuidado: en SELECT con muchas cols puede ser caro. Mejor agrupa con GROUP BY si vas a agregar después.

¿UNION o UNION ALL?

UNION quita duplicados (más caro). UNION ALL mantiene todo (más rápido). Usa ALL si sabes que no hay duplicados (más común).

¿SQLite o PostgreSQL para aprender?

SQLite para arrancar (sin servidor, 1 archivo). El SQL es 90% igual. Migras a PostgreSQL cuando necesites: tipos avanzados, concurrencia, escala, JSON nativo.

¿Cómo trato fechas en SQL?

Cada motor su dialecto. SQLite: strings ISO '2024-01-15' + date(), strftime(). PostgreSQL: tipo DATE/TIMESTAMP nativo. Estándar: DATE '2024-01-15'.

Referencias

- Tanimura, SQL for Data Scientists, caps. 1-3.
- SQLite SELECT docs
- DuckDB docs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 044 — Clase 044 — SQL avanzado: CTEs, window functions, subqueries correlacionadas

Parte: 0 — Prerrequisitos · Fuente: Tanimura, SQL for Data Scientists caps. 4-5 · PostgreSQL docs (window functions).

Duración estimada: 120 min.

Objetivo

Que el alumno escriba SQL legible y potente: CTEs (WITH) para descomponer queries complejas, window functions (OVER) para rankings/totales corridos/lag/lead sin perder filas, y subqueries correlacionadas cuando aportan.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Escribir CTEs con WITH name AS (...) para mejorar legibilidad.
2. Encadenar múltiples CTEs: WITH a AS (...), b AS (...) SELECT
3. Aplicar window functions: ROW_NUMBER(), RANK(), LAG(), LEAD(), SUM() OVER (PARTITION BY ... ORDER BY ...).
4. Calcular ranking por grupo con ROW_NUMBER() OVER (PARTITION BY ...).
5. Diferenciar subquery (independiente) vs correlacionada (depende de la outer).

Temas

#	Tema	Por qué importa
1	CTEs: WITH name AS (...)	Descomponer queries largas.
2	Múltiples CTEs encadenadas	Pipeline legible.
3	Recursive CTEs	Jerarquías, grafos.
4	Window functions: OVER (PARTITION BY ...	Agregar sin colapsar filas.
5	ROW_NUMBER, RANK, DENSE_RANK	Diferencias sutiles.
6	LAG, LEAD: comparar con fila anterior/sigu	Series temporales.
7	Subqueries correlacionadas	Cuando la subquery depende de la outer.

Definiciones y características

CTE (Common Table Expression)

: Vista temporal dentro de una query con WITH nombre AS (...). Descompone queries complejas en pasos legibles. Puedes encadenar múltiples: WITH a AS (...), b AS (...) SELECT

Recursive CTE

: CTE que se referencia a sí misma. Útil para jerarquías (árbol organizacional), grafos, generar series (calendario diario). Sintaxis: WITH RECURSIVE t AS (caso_base UNION ALL caso_recursivo).

Window function

: Agregación que no colapsa filas — añade el resultado por fila. Sintaxis: FUNC() OVER (PARTITION BY col ORDER BY col2). Ejemplos: ROW_NUMBER, RANK, LAG, LEAD, SUM() OVER (...).

PARTITION BY vs GROUP BY

: PARTITION BY (en window): subgrupos para la función, pero mantiene cada fila. GROUP BY: reduce a una fila por grupo.

LAG / LEAD

: Acceden a la fila anterior/siguiente dentro de la partition. LAG(monto, 1) OVER (PARTITION BY cliente ORDER BY fecha). Útil para diffs, growth rates.

Subquery correlacionada

: Subquery que depende de la outer query (referencia sus columnas). Se ejecuta una vez por cada fila de la

outer. Más lenta que JOIN equivalente.

Dataset / recursos

SQLite con ordenes (cliente_id, fecha, monto) de clase 041 — extendido. Sin descarga.

Ejercicios

1. CTE básica. Reescribe una query con subquery anidada usando WITH.
2. ROW_NUMBER por grupo. Top-1 orden por cliente (mayor monto).
3. Total corrido. SUM(monto) OVER (PARTITION BY cliente_id ORDER BY fecha) — total acumulado por cliente.
4. LAG. Por cliente, diferencia entre el monto actual y el anterior.
5. Recursive CTE. Genera serie de fechas día a día desde 2024-01-01 a 2024-01-31.

Homework verificable

Notebook: (a) 3 versiones de la misma query (anidada → CTE → CTEs múltiples) comparando legibilidad; (b) top-3 órdenes por cliente con ROW_NUMBER; (c) total corrido y delta vs orden anterior; (d) recursive CTE para calendario diario.

Criterio de aceptación: Las 3 versiones devuelven exactamente el mismo resultado. Window functions sin error.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
syntax error at or near "OVER"	Motor sin soporte de window functions (SQL
ROW_NUMBER() da números repetidos	Olvidaste OVER (...). Sin él, no es window
CTE recursiva nunca termina	Caso base falta o caso recursivo no conver
LAG(x) OVER (ORDER BY fecha) da NULL en la	Comportamiento esperado — no hay fila ante
CTE da mismo resultado pero más lento que	Algunos motores no inlineaban CTEs (Postgr

Preguntas frecuentes

¿CTE o subquery?

CTE si el lector necesita entender qué hace cada paso (legibilidad). Subquery si es trivial y de un solo uso. Para queries >10 líneas, CTE casi siempre gana.

¿ROW_NUMBER, RANK o DENSE_RANK?

Para valores [10, 20, 20, 30]: ROW_NUMBER [1,2,3,4] (siempre único). RANK [1,2,2,4] (huecos). DENSE_RANK [1,2,2,3] (sin huecos). Elige según semántica.

¿Window function es lo mismo que groupby+merge en pandas?

Conceptualmente sí — g.transform(...) en pandas hace lo equivalente. Window functions son la versión SQL más eficiente.

¿Cuándo subquery correlacionada vs JOIN?

Casi siempre JOIN o window function es más rápido. Correlacionada solo cuando no tiene equivalente JOIN (raro) o el optimizador del motor la maneja bien (motores modernos).

¿Top-N por grupo?

Patrón estándar: WITH ranked AS (SELECT , ROW_NUMBER() OVER (PARTITION BY grupo ORDER BY metric DESC) rn FROM tabla) SELECT FROM ranked WHERE rn <= N.

Referencias

- Tanimura, SQL for Data Scientists, caps. 4-5.
- PostgreSQL window functions tutorial
- Modern SQL — CTEs

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 045 — Clase 045 — SQL desde Python: sqlite3, SQLAlchemy, DuckDB

Parte: 0 — Prerrequisitos · Fuente: Python stdlib sqlite3 · SQLAlchemy docs · DuckDB Python docs.

Duración estimada: 75 min.

Objetivo

Que el alumno conecte Python con SQL de las 3 formas que va a encontrar en producción: sqlite3 (stdlib, demo local), SQLAlchemy (ORM/engine genérico para PostgreSQL/MySQL), y DuckDB (columnar embebido para análisis sobre CSV/Parquet sin servidor).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Conectar y consultar con sqlite3 stdlib, usando placeholders ? (NUNCA concatenar SQL).
2. Usar SQLAlchemy create_engine(URL) + pd.read_sql para queries a cualquier RDBMS.
3. Usar DuckDB para hacer SQL sobre DataFrames y CSV/Parquet directamente.
4. Prevenir SQL injection con queries parametrizadas.
5. Decidir entre sqlite/SQLAlchemy/DuckDB según el caso.

Temas

#	Tema	Por qué importa
1	sqlite3 stdlib: connect, cursor, fetchall	Para demos y BDs ligeras.
2	Placeholders ? y :nombre	NUNCA concatenar strings.
3	SQLAlchemy create_engine('postgresql://...	Soporta todos los RDBMS.
4	pd.read_sql y df.to_sql	Pasarela pandas ↔ BD.
5	DuckDB: SQL sobre DataFrames y archivos	duckdb.query("SELECT ... FROM df").
6	Cuándo cada uno	Trade-offs.

Definiciones y características

sqlite3 (stdlib)

: Driver Python para SQLite. Sin dependencias externas. Patrón: connect(...) → cursor() → execute(sql,

params) → fetchall().

Parameterized query (? o :name)

: Placeholder donde el driver substituye valores escapados. Única forma segura de pasar datos de usuario — previene SQL injection.

SQLAlchemy

: Toolkit ORM + Core para Python. Backend-agnostic: cambias el URL del engine y migras entre SQLite/Postgres/MySQL sin tocar queries. create_engine('postgresql://...').

DuckDB

: OLAP DB embebida. SQL sobre DataFrames pandas (duckdb.query('SELECT ... FROM df')) y archivos CSV/Parquet directos (FROM 'data.csv'). Mucho más rápido que sqlite3 para analytics.

SQL injection

: Inyección de SQL malicioso via concatenación de strings con input de usuario. Prevención: SIEMPRE parameterized queries, nunca f-string con valores externos.

pd.read_sql / df.to_sql

: Pasarela pandas↔BD. Acepta connection o engine. read_sql_query para queries complejas; read_sql_table para tablas completas.

Dataset / recursos

Penguins descargado a CSV local para DuckDB; datos sintéticos para sqlite/SQLAlchemy.

Ejercicios

1. sqlite3 con placeholders. Crea tabla, inserta 5 filas usando executemany con tuples, consulta con ? placeholder. Demuestra el bug si concatenas.
2. df.to_sql y pd.read_sql. Carga un DataFrame a SQLite y consulta de vuelta.
3. SQLAlchemy engine. Crea engine SQLite. Usa pd.read_sql con engine.
4. DuckDB sobre DataFrame. Carga penguins en df. duckdb.query('SELECT species, AVG(body_mass_g) FROM df GROUP BY species').df().
5. DuckDB sobre CSV. Mismo query pero FROM 'penguins.csv' directo, sin cargar a pandas.

Homework verificable

Notebook con 3 backends del mismo análisis: (a) sqlite3 stdlib + cursor; (b) SQLAlchemy engine + pd.read_sql; (c) DuckDB sobre CSV. Documenta cuándo elegirías cada uno. Demuestra explícitamente el peligro de SQL injection con concatenación vs placeholders.

Criterio de aceptación: Las 3 versiones devuelven el mismo resultado. Demo de injection sin daño real.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OperationalError: no such table: X después	Falta con.commit(). sqlite3 no auto-commit
Concatené input de usuario en query y func	Hasta que el usuario malicioso prueba ' ; D
SQLAlchemy 2.0 — Engine.execute no existe	API cambió: ahora with engine.connect() as
DuckDB lee CSV pero pierde tipos	Pandas adivina mejor. Fix: duckdb.read_csv

pd.read_sql lento con N grande

Driver Python carga todo a Python. Fix: ch

Preguntas frecuentes

¿sqlite3, SQLAlchemy o DuckDB?

sqlite3 para demos/tests/scripts locales. SQLAlchemy para producción con Postgres/MySQL. DuckDB para EDA sobre CSV/Parquet sin servidor — el más rápido para analytics.

¿pd.read_sql es seguro contra injection?

Sí si pasas params: read_sql('SELECT * FROM t WHERE x=:val', con, params={'val': user_input}). No si concatenas strings.

¿ORM (SQLAlchemy declarative) o queries directas?

ORM cuando el modelo se usa en muchas partes (web app con N modelos). Queries directas para análisis ad-hoc. Pueden coexistir.

¿DuckDB sobre Parquet vs sobre pandas?

Parquet directo es más rápido (no carga a Python). Pandas cuando ya tienes el DataFrame en memoria. DuckDB es smart: optimiza ambos casos.

¿Cerrar conexión manualmente?

Usa context manager: with sqlite3.connect(...) as con: ... o con.close() en finally. Conexiones dejadas abiertas consumen handles del OS.

Referencias

- Python sqlite3 docs
- SQLAlchemy tutorial
- DuckDB Python API
- OWASP — SQL injection prevention

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 046 — Clase 046 — NoSQL: MongoDB con pymongo

Parte: 0 — Prerrequisitos · Fuente: MongoDB docs · pymongo docs · MongoDB: The Definitive Guide (Bradshaw et al.) cap. 1.

Duración estimada: 75 min.

Objetivo

Que el alumno entienda el modelo NoSQL documento (collections de JSON-like), cuándo conviene sobre SQL, y use pymongo para CRUD básico + queries con operadores típicos. Sin pretender competir con un curso entero de MongoDB.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar modelo relacional (tablas + filas) vs documento (collections + docs JSON).
2. Reconocer cuándo NoSQL aporta (schema flexible, datos jerárquicos, escala horizontal).
3. Conectar con pymongo, hacer insert/find/update/delete.
4. Filtrar con operadores: \$gt, \$lt, \$in, \$regex, \$and, \$or.
5. Hacer agregaciones con el pipeline (\$match, \$group, \$sort).

Temas

#	Tema	Por qué importa
1	SQL vs NoSQL — cuándo cada uno	No "NoSQL es mejor" — distinto.
2	Modelo documento: collections + docs JSON	Schema flexible.
3	pymongo: connect, insert_one, find, update	CRUD básico.
4	Operadores de query: \$gt/\$lt/\$in/\$regex	Equivalentes a WHERE.
5	Aggregation pipeline	\$match/\$group/\$sort — análogo a SQL.
6	Cuándo NO usar Mongo	Cuando relacional es claramente mejor.

Definiciones y características

NoSQL documento

: Familia de DBs que almacena documentos JSON-like (BSON en Mongo) en lugar de filas en tablas. Schema flexible — cada documento puede tener campos distintos.

Collection

: Equivalente a una tabla en SQL, pero sin schema fijo. Contiene documentos del mismo "tipo" lógico (productos, usuarios, eventos).

Documento (dict BSON)

: Unidad de almacenamiento. JSON con tipos extra (Date, ObjectId, Decimal128). Puede contener arrays y sub-documentos anidados.

Operador (\$gt, \$in, \$elemMatch)

: Prefijo \$ en las queries Mongo: {'precio': {'\$gt': 100}} ≈ WHERE precio > 100. La query es un dict JSON, no string.

Aggregation pipeline

: Equivalente Mongo a CTEs encadenadas: lista de etapas (\$match, \$group, \$sort, \$project, \$lookup) que procesan documentos secuencialmente.

\$elemMatch

: Operador para filtrar por condiciones sobre elementos de un array dentro del documento. Útil con arrays de sub-docs (reviews dentro de producto).

Dataset / recursos

MongoDB local (Docker o Atlas free tier) — o usar mongomock para tests. Datos sintéticos: collection de productos.

Ejercicios

1. CRUD básico. Conecta a Mongo (o mongomock), inserta 5 productos, lee todos, actualiza uno, borra uno.

2. Find con operadores. Productos con precio > 100 y categoría en ['libros', 'musica'].
3. Update con \$set y \$inc. Incrementa stock de un producto en 10 unidades.
4. Aggregation pipeline. Promedio de precio por categoría con \$group.
5. Documento jerárquico. Inserta un producto con array de reviews (sub-documentos). Consulta los que tienen alguna review con rating < 3 usando \$elemMatch.

Homework verificable

Notebook con mongomock (no requiere Mongo real): (a) collection productos con 20 docs sintéticos; (b) 5 queries demostrando operadores; (c) aggregation pipeline con \$match → \$group → \$sort; (d) reporte: 3 casos donde Mongo es mejor que SQL y 3 donde no.

Criterio de aceptación: Las queries funcionan; el reporte tiene casos justificados.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
pymongo.errors.ServerSelectionTimeoutError	No conecta al servidor. Fix: verifica que
Update sin \$set reemplaza el documento ent	update_one(filter, {'precio': 100}) reempl
Aggregation con \$group sin _id falla	\$group requiere _id (la key de agrupación,
Query con {} devuelve todo (no None)	{ } es "sin filtro" en Mongo. Si querías ma
Cuento con count_documents({}) y va lento	Sin filtro, recorre toda la collection. Fi

Preguntas frecuentes

¿SQL o NoSQL?

SQL para datos tabulares con relaciones, integridad referencial, reporting/BI. NoSQL documento para schema variable, datos jerárquicos naturales, escala write masiva. No es "mejor" — distinto.

¿find_one o find?

find_one devuelve dict (o None). find devuelve cursor iterable. Para 1 doc: find_one. Para muchos: list(coll.find(query)) o iterar el cursor.

¿pymongo vs Motor (async)?

pymongo síncrono, default. Motor asíncrono (asyncio) — para web apps con muchas concurrent connections.

¿Cómo tipo los documentos en Python?

Usa pydantic con BaseModel. Convierte dict ↔ tipo validado. Combinado con FastAPI, casi gratis (verás en MLOps).

¿Mongo para data science?

Como fuente sí (extraes datos con aggregation, los pasas a pandas). Para análisis ya no — pandas/DuckDB son mejores. Mongo brilla en operaciones (logs, eventos).

Referencias

- pymongo docs
- MongoDB query operators
- mongomock

- Bradshaw, MongoDB: The Definitive Guide 3e, cap. 1.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 047 — Clase 047 — APIs REST con requests

Parte: 0 — Prerrequisitos · Fuente: HTTP: The Definitive Guide caps. 1-2 · requests docs.

Duración estimada: 90 min.

Objetivo

Que el alumno consuma APIs REST públicas con requests: GET con parámetros, manejo de status codes, autenticación (header, bearer token), paginación, rate limiting con Retry, y carga eficiente con Session. Lo mínimo para no romper la API del proveedor ni tu pipeline.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Hacer GET/POST con requests, manejar params, headers, body JSON.
2. Verificar status code (200 vs 4xx vs 5xx) y usar `raise_for_status()`.
3. Autenticarse con header Authorization: Bearer ... o API key en header/query.
4. Pagar correctamente cuando la API devuelve resultados en páginas.
5. Rate-limiting con `urllib3.util.retry.Retry` para reintentos exponenciales.
6. Reusar conexión con `requests.Session` para múltiples requests.

Temas

#	Tema	Por qué importa
1	Métodos HTTP: GET, POST, PUT, DELETE	Verbos REST.
2	Status codes: 2xx/3xx/4xx/5xx	Cómo reaccionar a cada uno.
3	Params, headers, body	Las 3 formas de mandar datos.
4	Autenticación: Bearer token, API key	Header Authorization.
5	Paginación: offset/limit, cursor, link hea	3 patrones comunes.
6	Rate limiting + retry exponencial	No tirar la API ajena.
7	<code>requests.Session</code> para reuso	Más rápido + cookies persistentes.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada:

- Clase 046a — `async / httpx / aiohttp` para data scientists

Definiciones y características

REST (REpresentational State Transfer)

: Estilo arquitectónico para APIs web sobre HTTP. Recursos identificados por URLs, operaciones por verbos HTTP (GET=leer, POST=crear, PUT=update, DELETE=borrar).

requests

: Librería Python de facto para hacer HTTP. API simple: `requests.get(url, params=..., headers=..., timeout=...)`. Soporta auth, cookies, sessions, retry.

Status code

: Número HTTP que indica resultado: 2xx éxito, 3xx redirect, 4xx error cliente (404 no encontrado, 401 no auth, 403 prohibido, 429 rate limited), 5xx error servidor.

Paginación

: API que devuelve resultados en bloques (no todo de golpe). Patrones: offset/limit (`?page=2`), cursor (`?after=<id>`), Link header (`<url>; rel="next"`).

Rate limiting

: Política del servidor: máximo N requests/seg/usuario. Excederlo → 429. Respétalo con delays y retries exponenciales.

Session

: Reuso de conexión TCP/TLS entre requests. Mantiene cookies. ~10× más rápido para múltiples requests al mismo host vs `requests.get` repetido.

Bearer token

: Esquema de auth común: Authorization: Bearer <token> header. Token suele ser JWT o opaque string emitido por OAuth/login.

Dataset / recursos

API pública sin auth: <https://api.coingecko.com> (precios cripto). Sin API key necesaria.

Ejercicios

1. GET básico. `requests.get('https://api.github.com')`. Inspecciona `status_code`, `headers`, `.json()`.
2. Con params. GitHub search: <https://api.github.com/search/repositories?q=python+ml&sort=stars>. Imprime top 5.
3. `raise_for_status` + try. Pega a una URL que devuelve 404 (/notfound) y maneja la excepción.
4. Paginación. GitHub events API. Itera 3 páginas con `page=1,2,3`.
5. Session + Retry. Configura una Session con HTTPAdapter + Retry (3 intentos, backoff 1s). Verifica que reintenta en 5xx simulado.

Homework verificable

Notebook que: (a) consulta una API pública (CoinGecko, GitHub, JSONPlaceholder) con GET; (b) maneja status codes con try/except; (c) pagina 3+ páginas; (d) configura Session con Retry exponencial; (e) reporta cuánto se tardó vs un loop sin Session.

Criterio de aceptación: Maneja al menos un error sin crash. Pagination devuelve datos esperados.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>requests.exceptions.ConnectTimeout</code> o Timeo	API lenta o sin red. Fix: siempre pasa tim
API responde 200 pero <code>.json()</code> lanza error	Body no es JSON (HTML de error, vacío). Fi

Hardcodeé el token en el código y subí a G	Catástrofe de seguridad — el token es públ
Mi script tira la API ajena (HTTP 429)	Sin rate limiting. Fix: <code>time.sleep()</code> entre
HTTPError no se lanza con status 4xx	requests NO lanza por default. Fix: <code>r.raise</code>
RuntimeError: <code>asyncio.run()</code> cannot be call	El notebook ya tiene un event loop corrien

Preguntas frecuentes

¿requests o httpx?

requests sigue siendo la default (estable, ubicua). httpx drop-in con async support (async with `httpx.AsyncClient()` as client:). Para async/HTTP2, httpx; para todo lo demás, requests.

¿Cuándo Session?

Más de 2-3 requests al mismo host. La primera request hace handshake TCP/TLS (~100ms); Session lo reusa. Para single request, no aporta.

¿json= o data= en POST?

json=dict: serializa a JSON y setea Content-Type: application/json. data=dict: form-encoded (application/x-www-form-urlencoded). Para APIs REST modernas, casi siempre json=.

¿Cómo paginar genéricamente?

Loop hasta que la API diga "no más": `while True: r = requests.get(url, params=...); items.extend(r.json()["data"]); if not r.json().get('next'): break.`

¿Auth OAuth desde Python?

Para casos simples (Bearer fijo): pasa el header. Para OAuth flow completo: authlib, requests-oauthlib. Para producción: librería oficial del proveedor (google-auth, pyOpenSSL, etc.).

¿Cuándo paso de requests a httpx async?

Regla práctica: cuando tenés >20 requests simultáneos al mismo proveedor y el cuello de botella es latencia de red (no CPU). Si tu script está 90% del tiempo esperando respuestas HTTP, async te da un speedup de 10-100x. Si en cambio estás parseando JSONs gigantes o haciendo cálculo pesado, async no ayuda — ahí lo tuyo es multiprocessing. Pocas requests (<10): seguí con requests, no vale la complejidad.

Referencias

- requests docs
- urllib3 Retry
- GitHub REST API
- HTTP status codes (MDN)
- httpx docs
- asyncio + Jupyter (autoawait)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 048 — Clase 048 — Web scraping con BeautifulSoup

Parte: 0 — Prerrequisitos · Fuente: *Web Scraping with Python* (Mitchell, 2ª ed.) caps. 1-3 · BeautifulSoup docs.

Duración estimada: 75 min.

Objetivo

Que el alumno extraiga datos de páginas HTML cuando no hay API disponible, usando requests + BeautifulSoup. Y entienda los límites éticos y legales: robots.txt, rate limiting humano, ToS, datos personales, copyright. Lo último que debe hacer al scrapear es tirar abajo el sitio o meterse en problemas.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Parsear HTML con BeautifulSoup(html, 'html.parser').
2. Encontrar elementos con find, find_all, select (CSS selectors).
3. Extraer texto y atributos (.text, ['href']).
4. Respetar robots.txt y rate limit (delay entre requests).
5. Identificar cuándo scraping es buena idea vs cuándo buscar otra fuente (API, dataset público).

Temas

#	Tema	Por qué importa
1	HTTP → HTML → parser tree	Cómo funciona scraping.
2	BeautifulSoup: find vs select	Selectores CSS son más potentes.
3	Extracción de texto y atributos	.text, .get_text(strip=True), ['href'].
4	Páginas dinámicas (JS) — requests no las r	Para eso: Playwright/Selenium.
5	robots.txt — qué dice y por qué respetar	User-agent, Disallow, Crawl-delay.
6	Ética: ToS, rate limiting, datos personale	Lo que sí, lo que no.

Definiciones y características

Web scraping

: Extracción programática de datos de páginas HTML cuando no hay API. Pipeline típico: descargar HTML con requests, parsear con BeautifulSoup, extraer con selectores.

BeautifulSoup

: Parser HTML tolerante a errores. soup = BeautifulSoup(html, 'html.parser'). Permite navegar el árbol y buscar por tag, atributo o selector CSS.

CSS selector

: Sintaxis para localizar elementos: '.class', '#id', 'tag', 'parent > child', 'tag[attr=val]'. Usados con soup.select(...). Más expresivos que find_all.

DOM (Document Object Model)

: Representación en árbol del HTML. Cada tag es un nodo; tiene padre, hermanos, hijos. BeautifulSoup navega este árbol con .parent, .next_sibling, .find_all, etc.

robots.txt

: Archivo en raíz del dominio (/robots.txt) que declara qué paths pueden crawlear los bots. Estándar de facto; respetarlo es legalmente importante (variable por jurisdicción) y éticamente siempre.

JS rendering

: Páginas SPA (React, Vue) cargan contenido vía JavaScript tras el HTML inicial. requests solo trae el HTML inicial — JS no se ejecuta. Solución: Playwright o Selenium (navegador headless).

Dataset / recursos

Página HTML simple servida desde un string en el notebook (sin tocar internet). Ejercicios opcionales con <https://quotes.toscrape.com> (sitio diseñado para practicar).

Ejercicios

1. Parsea HTML local. Crea un HTML con 3 productos (`<div class='product'>`). Extrae nombres y precios con `find_all`.
2. Selectores CSS. Lo mismo con `soup.select('.product .price')`.
3. Tabla a DataFrame. `pd.read_html(url)` para una tabla HTML — bonus: requests + BeautifulSoup para tablas custom.
4. Scrape ético. Scrapea quotes.toscrape.com (público, diseñado para esto). Respeta Crawl-delay. 3 páginas con `time.sleep(1)` entre cada una.
5. Inspeccionar robots.txt. Lee <https://quotes.toscrape.com/robots.txt> con requests. Identifica qué paths están Disallow.

Homework verificable

Notebook: (a) HTML local con 5 productos, extrae nombre/precio/url; (b) scrape quotes.toscrape.com (3 páginas, con delay); (c) consulta robots.txt y razona; (d) listado de 3 escenarios cuando scrapear es buena idea y 3 cuando no.

Criterio de aceptación: Scraping respeta delays. Análisis de robots.txt correcto.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>soup.find('div')</code> devuelve None aunque hay	Buscas un atributo específico que no match
Scrapeo y recibo HTML distinto al que veo	El sitio renderiza con JavaScript. request
HTTP 403 Forbidden	El sitio detecta tu bot (User-Agent vacío)
Site funciona en navegador pero scraper de	Anti-bot agresivo (Cloudflare, reCAPTCHA).
Encoding raro (acentos Ã¡)	Pandas/requests dedujo encoding mal. Fix:

Preguntas frecuentes

¿Scraping es legal?

Depende: jurisdicción, ToS del sitio, naturaleza del dato. Datos personales = casi siempre regulado (GDPR/LGPD). Datos públicos sin ToS prohibitivo = generalmente OK. Consulta abogado para casos serios.

¿find_all o select?

`select` (CSS selectors) — más potente, sintaxis estándar web, más legible. `find_all` para casos simples o cuando integras con código heredado.

¿Cómo descargo imágenes?

`r = requests.get(url_imagen); open('img.jpg', 'wb').write(r.content)`. Para muchas, usa Session + thread pool.

¿Scrapy vs BeautifulSoup?

BeautifulSoup: librería de parsing. Una página, una request. Scrapy: framework completo (crawler, pipelines, throttling). Para proyectos serios (miles de páginas), Scrapy.

¿Y si el sitio me bloquea?

Respetar. Aumentar agresividad (proxies, rotating User-Agents) puede ser ilegal en algunas jurisdicciones (CFAA en USA). Considera: ¿realmente vale la pena? ¿hay otra fuente?

Referencias

- Mitchell, Web Scraping with Python 2e, caps. 1-3.
- BeautifulSoup docs
- quotes.toscrape.com (sitio para practicar)
- Google — robots.txt

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 049 — Clase 049 — async / httpx / aiohttp para data scientists

Parte: 0 — Prerrequisitos · Fuente: docs asyncio + httpx + Beazley Python Concurrency. Duración estimada: 80 min.

Objetivo

Aprender asyncio y httpx —el HTTP client moderno con soporte sync + async + HTTP/2— para hacer scraping y consumo de APIs en paralelo sin bloquear. Pasar de "1 request por segundo" con requests a "100+ concurrentes" con httpx.AsyncClient. Comparar con aiohttp (alternativa popular) y concurrent.futures (parallelism con threads).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir async def y await; ejecutar con asyncio.run.
- Usar httpx.AsyncClient para fetches concurrentes con asyncio.gather.
- Limitar concurrencia con asyncio.Semaphore para no DOS-ear a la API.
- Implementar rate limiting + retries con backoff exponencial.
- Decidir entre asyncio, threading, multiprocessing según I/O-bound vs CPU-bound.

Temas

- Event loop, coroutines, await.
- httpx: API unificada sync/async, HTTP/2, timeouts.
- asyncio.gather, asyncio.as_completed.
- Semaphore para limitar concurrencia.
- Backoff exponencial con tenacity o backoff lib.
- aiohttp como alternativa (más antigua, más utilities).
- Cuándo NO async: tareas CPU-bound (usar multiprocessing).

Definiciones y características

- Coroutine: función `async def`, devuelve un objeto coroutine que el event loop ejecuta.
- `await`: cede control al event loop hasta que la operación termina.
- Event loop: scheduler que ejecuta coroutines cooperativamente.
- `asyncio.gather(*coros)`: ejecuta varias coroutines concurrentemente, espera todas.
- `asyncio.as_completed(coros)`: itera resultados a medida que llegan.
- `asyncio.Semaphore(n)`: limita a `n` coroutines concurrentes.
- `httpx`: HTTP client moderno (Tom Christie, dev de Django REST Framework).
- `aiohttp`: client + server `async`. Más viejo, más maduro, sin `sync` API.

Dataset / recursos

- Una API pública lenta: <https://httpbin.org/delay/2> (espera 2 seg).
- Lista de 100 URLs para fetcher.
- Librerías: `httpx`, `aiohttp`, opcional `tenacity`.

Ejercicios

1. Sync baseline: fetcher de 50 URLs con `requests.get` en loop. Medir tiempo.
2. Async con `httpx`: `async with httpx.AsyncClient()` as `c`: `results = await asyncio.gather(*[c.get(url) for url in urls])`. Comparar tiempo (debería ser ~20-50× más rápido).
3. Semaphore: limitar a 10 concurrent. Útil para no ser bloqueado por rate limits.
4. Retry exponencial: usar `tenacity.retry(stop=stop_after_attempt(5), wait=wait_exponential(min=1, max=30))` sobre un fetcher.
5. `httpx` vs `aiohttp`: hacer el mismo benchmark con ambos. Similar performance; `httpx` tiene mejor DX.

Homework verificable

Scraper concurrente de 200 URLs (una página de Wikipedia o API pública):

1. Implementar con `httpx.AsyncClient` + `asyncio.gather`.
2. Limitar a 20 concurrent con Semaphore.
3. Retry con backoff sobre errores 5xx.
4. Reportar tiempo total y % de éxito.

Criterio de aceptación: tiempo total $\leq 1/10$ de la versión sync; success rate $\geq 95\%$.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>RuntimeError: This event loop is already r</code>	Jupyter ya tiene un loop. Fix: <code>nest_asyncio</code>
Olvido <code>await</code> → coroutine never awaited war	Fix: <code>await coro()</code> o <code>asyncio.run(coro())</code> .
Mezclar sync requests adentro de coroutine	Bloquea el event loop. Fix: usar <code>httpx.Asy</code>
Sin limit → API te banea	Demasiada concurrencia. Fix: Semaphore + d
<code>AsyncClient</code> no cerrado	Resource leak. Fix: <code>async with httpx.Async</code>

Preguntas frecuentes

`async` o `threading`?

Async para I/O-bound (HTTP, DB) — más eficiente, menos overhead. Threading para I/O-bound legacy code. Multiprocessing para CPU-bound (cálculo numérico, ML training).

`httpx` o `aiohttp`?

`httpx` es default moderno: API unificada sync/async, mejor DX, HTTP/2. `aiohttp` tiene más utilities (sessions,

file uploads) pero solo async.

async con pandas / numpy?

No tiene sentido — son CPU-bound. Async brilla cuando esperas de I/O externo.

Test de async code?

pytest-asyncio con `@pytest.mark.asyncio`.

async en Django / Flask?

Django 4+ soporta async views. Flask 2+ también. FastAPI es async-native (default moderno).

Referencias

- httpx docs
- asyncio docs
- aiohttp docs
- Beazley, D. Python Concurrency from the Ground Up (PyCon 2015).
- tenacity para retries.

Siguiente parte

Clase 050 — Panorama del ML: tipos, batch vs online, instance vs model-based

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Parte 1 — Parte 1 — Machine Learning Clásico

50 clases en esta parte.

Clase 050 — Clase 050 — Panorama del ML: tipos, batch vs online, instance vs model-based

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 1. Duración estimada: 60 min.

Objetivo

Que el alumno arme un mapa mental claro del campo de ML antes de entrar en algoritmos concretos: qué tipos de aprendizaje existen, en qué se diferencia entrenar de una sola vez (batch) vs en streaming (online), y por qué algunos modelos "memorizan ejemplos" (instance-based) y otros "abstraen una regla" (model-based). Sirve de andamio para ubicar cada algoritmo de las próximas clases dentro de esta taxonomía.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Clasificar un problema dado como supervisado, no supervisado, semi-supervisado o reforzado, justificando con la pinta de los datos (¿hay etiquetas?, ¿hay recompensa?).

2. Decidir batch vs online según el volumen de datos, la velocidad a la que cambia la distribución y el costo de reentrenar.
3. Distinguir instance-based de model-based y reconocer cuál usa scikit-learn por debajo en un algoritmo dado (KNN vs regresión lineal, por ejemplo).
4. Detectar señales de mala generalización (overfitting / data drift) en términos del marco del cap. 1 — anticipo de la clase 048.
5. Ubicar cualquier algoritmo del curso dentro de la grilla (supervisión × batch/online × instance/model) sin googlear.

Temas

#	Tema	Por qué importa
1	Qué es ML (Samuel 1959, Mitchell 1997)	Definición operativa: aprender = mejorar e
2	Aprendizaje supervisado	El 80% de lo que vas a tocar en la industr
3	Aprendizaje no supervisado	Clustering, reducción de dim, detección de
4	Semi-supervisado y auto-supervisado	Etiquetar es caro; mezclás un poco de labe
5	Aprendizaje por refuerzo	Otro paradigma (agente + recompensa); útil
6	Batch vs online learning	Reentrenar de cero vs aprender incremental
7	Instance-based vs model-based	Dos formas de generalizar: por similitud c

Definiciones y características

Aprendizaje supervisado

: El dataset trae cada instancia con su etiqueta (y). El algoritmo aprende un mapeo $f: X \rightarrow y$. Sub-divisiones: clasificación (y categórica) y regresión (y continua). Ejemplos: spam filter, predicción de precio de casa.

Aprendizaje no supervisado

: Solo hay X, sin y. El algoritmo busca estructura: clusters (k-means), componentes (PCA), densidad (DBSCAN), anomalías (isolation forest). Métricas de éxito son menos directas que en supervisado.

Aprendizaje semi-supervisado

: Mezcla — pocas instancias etiquetadas + muchas sin etiqueta. Típico cuando etiquetar es caro (imagenes médicas, audio transcripto). Google Photos lo usa para clustrear caras y pedirte el nombre solo del cluster.

Aprendizaje por refuerzo (RL)

: Un agente observa el entorno, toma acciones, recibe recompensa (positiva o negativa), aprende una política que maximiza recompensa acumulada. Distinto de supervisado: no hay "respuesta correcta", solo señal de recompensa. Ej: AlphaGo, robots, trading bots.

Batch learning (offline)

: Se entrena con todo el dataset de una sola pasada. El modelo queda congelado en producción. Para incorporar datos nuevos hay que reentrenar desde cero. Simple, reproducible, pero pesado si el dataset crece o la distribución cambia.

Online learning (incremental)

: El modelo se actualiza con mini-batches o instancias una a la vez (partial_fit en sklearn). Ideal para streaming (precios, clicks) o datasets que no caben en RAM (out-of-core). El parámetro clave es el learning rate: alto = se adapta rápido pero olvida; bajo = estable pero lento.

Instance-based learning

: El sistema "memoriza" las instancias de entrenamiento y generaliza comparando una nueva instancia con

las vistas, vía una medida de similitud. Ejemplo canónico: KNN. No hay parámetros aprendidos — el "modelo" es el propio dataset.

Model-based learning

: Se asume una forma funcional (lineal, árbol, red neuronal) con parámetros θ , y se ajustan los θ minimizando una función de costo sobre el train set. Una vez entrenado, podés tirar el dataset: el modelo es autocontenido. Ejemplos: regresión lineal/logística, random forest, redes neuronales.

Generalización

: Capacidad del modelo de funcionar bien en datos nuevos (no vistos en entrenamiento). Es el objetivo real de ML — no la performance en train. Se mide con un test set separado. Falla por dos motivos: overfitting (memorizó ruido) o underfitting (modelo demasiado simple). Tema central de la clase 048.

Dataset / recursos

Para los ejercicios prácticos: California Housing (`sklearn.datasets.fetch_california_housing`) — regresión, supervisado, ~20k filas. Y Iris (`sklearn.datasets.load_iris`) — clasificación clásica, 150 filas, perfecto para comparar KNN (instance-based) vs LogisticRegression (model-based) en pocos segundos.

Lectura de fondo: Géron, cap. 1 entero (~25 páginas). Mirá especialmente las figuras 1-13 a 1-17, que son el resumen visual de toda la taxonomía.

Ejercicios

1. Clasificá 6 problemas reales. Para cada uno indicá supervisado/no-supervisado/semi/RL y por qué: (a) detectar fraude en transacciones con tarjeta; (b) segmentar clientes de un e-commerce; (c) traducir inglés→español; (d) jugar al ajedrez; (e) detectar caras duplicadas en una galería de 10k fotos donde solo 50 están taggeadas; (f) predecir el precio del dólar a 7 días.
2. Batch o online. Decidí qué corresponde y justificá en una línea: (a) modelo de scoring crediticio que se reentrena trimestralmente; (b) recomendador de noticias que reacciona a clicks en tiempo real; (c) clasificador de imágenes médicas en un hospital; (d) detector de spam en Gmail.
3. KNN vs LogReg en Iris. Entrená un `KNeighborsClassifier(n_neighbors=5)` y un `LogisticRegression(max_iter=1000)` sobre iris. Compará: accuracy en test, tamaño del modelo serializado (`pickle.dumps`), tiempo de inferencia sobre 1000 predicciones. ¿Cuál es instance-based? Verificalo mirando el tamaño.
4. Out-of-core con `SGDRegressor`. Cargá California Housing y entrená un `SGDRegressor` en mini-batches de 500 filas usando `partial_fit` en un loop. Plotteá el MSE en train a medida que pasan los batches. Es el patrón canónico de online learning.
5. Mapa mental. En papel (o `draw.io`), dibujá una grilla 2×2×2 con los ejes supervisión / batch-online / instance-model. Ubicá: KNN, regresión lineal, k-means, random forest, `SGDClassifier`, AlphaGo, autoencoder. Algunos van a quedar en celdas raras — anotá por qué.

Homework verificable

Notebook que: (a) carga Iris y California Housing; (b) entrena 4 modelos — `KNeighborsClassifier`, `LogisticRegression`, `KNeighborsRegressor`, `SGDRegressor` con `partial_fit` en 10 batches; (c) reporta para cada uno: tipo de aprendizaje (sup/no-sup), batch o online, instance o model-based, métrica en test (accuracy o RMSE) y tamaño en bytes del modelo serializado; (d) produce una tabla resumen en markdown al final del notebook.

Criterio de aceptación: La tabla final clasifica correctamente los 4 modelos en los 3 ejes y muestra que el

KNN ocupa significativamente más bytes que LogReg para el mismo problema (evidencia empírica de que es instance-based).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Mi modelo da 99% en train y 60% en test"	Overfitting clásico. El modelo memorizó el
partial_fit da resultados muy distintos a	Online learning depende del orden de los b
KNeighborsClassifier tarda una eternidad e	Es instance-based — comparar contra todo e
"El test accuracy fluctúa cada vez que cor	No fijaste random_state en el train_test_s
Confundir "no supervisado" con "sin entren	k-means igual entrena (ajusta centroides).

Preguntas frecuentes

¿Semi-supervisado es lo mismo que self-supervised?

No. Semi-supervisado = pocas etiquetas + muchos datos sin etiquetar, en un problema supervisado clásico. Self-supervised (auto-supervisado) = el modelo se inventa la etiqueta a partir del propio dato (ej: BERT enmascara palabras y predice la enmascarada). Self-supervised es el motor de los LLMs modernos.

¿Cuándo elijo online learning en serio, no como juguete?

Tres casos: (1) streaming real donde los datos llegan continuamente y la distribución cambia (clicks, precios, sensores IoT); (2) out-of-core — el dataset no entra en RAM y entrenar offline en disco es prohibitivo; (3) adaptación rápida a concept drift sin reentrenar de cero. Para el 90% de problemas tabulares estáticos, batch es más simple y suficiente.

¿KNN realmente no entrena nada?

Técnicamente fit solo guarda el train set (con un índice como KD-tree opcional). Todo el trabajo se hace en predict. Por eso se le dice "lazy learner". Contraste total con regresión lineal, donde fit calcula los coeficientes y predict es una multiplicación.

¿Random forest es instance o model-based?

Model-based. Aunque "guarda árboles" — esos árboles son la forma funcional aprendida, no las instancias originales. Una vez fiteado, podés tirar el train set y el modelo sigue funcionando solo con los árboles. KNN no — sin el train set, KNN no existe.

¿Por qué arrancamos con este panorama antes de meternos en código?

Porque sin el mapa, cada algoritmo nuevo se siente desconectado del anterior. Con el marco del cap. 1, cuando lleguemos a SVM (clase ~055) ya sabés que es supervisado, batch, model-based — y eso te dice automáticamente qué API de sklearn esperar, cómo evaluarlo y cuáles son sus debilidades genéricas. El marco ahorra horas más adelante.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3rd ed., O'Reilly, 2022), cap. 1 — The Machine Learning Landscape.
- scikit-learn User Guide — Supervised learning
- scikit-learn User Guide — Unsupervised learning
- scikit-learn — Scaling strategies (out-of-core) — base teórica de partial_fit.
- Mitchell, T. Machine Learning (McGraw-Hill, 1997) — definición clásica de aprendizaje (T, P, E).

- Distill — A visual exploration of Gaussian Processes — bonus para entender model-based no paramétrico.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 051 — Clase 051 — Desafíos del ML: overfitting, underfitting, datos insuficientes

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 1. Duración estimada: 60 min.

Objetivo

Que el alumno identifique los seis problemas que hacen fracasar un proyecto de ML — datos insuficientes, no representativos, de mala calidad, features irrelevantes, overfitting y underfitting — y sepa qué herramienta aplicar a cada uno (más datos, mejor muestreo, limpieza, feature engineering, regularización, o un modelo más expresivo). El eje conceptual es el bias-variance tradeoff y la intuición de que "el modelo memorizó vs. generalizó".

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Distinguir overfitting de underfitting mirando la brecha entre error de entrenamiento y error de validación.
2. Diagnosticar cuál de los seis desafíos de Géron está rompiendo un pipeline concreto.
3. Aplicar regularización (L1/L2, reducir capacidad del modelo, más datos) como contramedida al overfitting.
4. Detectar sampling bias y data snooping bias antes de medir performance.
5. Justificar por qué "más datos" suele ganarle a "modelo más complejo" (Banko & Brill 2001 / "The Unreasonable Effectiveness of Data").

Temas

#	Tema	Por qué importa
1	Datos insuficientes	Hasta el algoritmo más simple necesita vol
2	Datos no representativos (sampling bias)	Si el train no se parece al deploy, el mod
3	Datos de mala calidad (outliers, NaN, ruid	Garbage in, garbage out. La limpieza es 60
4	Features irrelevantes / feature engineerin	Modelos buenos con features malos pierden
5	Overfitting	El modelo aprendió el ruido del train. Baj
6	Underfitting	El modelo es demasiado rígido para captar
7	Regularización (L1/L2, early stopping, dro	Solución estándar al overfitting cuando no

Definiciones y características

Overfitting

: El modelo performa muy bien en train y mal en test. Memorizó ejemplos en vez de extraer patrones. Síntoma típico: train_score test_score. Causas: modelo demasiado complejo, pocas observaciones, ruido en los labels. Fixes: más datos, regularización, reducir capacidad, early stopping.

Underfitting

: El modelo es demasiado simple para la estructura del dato. Performa mal tanto en train como en test. Síntoma: ambos scores bajos y parecidos. Fix: modelo más expresivo, más features, menos regularización, o aceptar que la señal no está.

Regularización

: Restricción explícita sobre los parámetros del modelo para que no se ajuste al ruido. En lineales: penalizar la norma de los coeficientes (Ridge = L2, Lasso = L1). En árboles: limitar profundidad. En redes: dropout, weight decay. Controla la variance a costa de aceptar algo más de bias.

Bias-variance tradeoff

: Descomposición del error de generalización en tres términos: $\text{error} = \text{bias}^2 + \text{variance} + \text{irreducible_noise}$. Bias = error por supuestos del modelo (un lineal sobre una curva). Variance = sensibilidad a la muestra de entrenamiento (cambias el train un poco y el modelo cambia mucho). Reducir uno suele subir el otro — el arte del ML es encontrar el sweet spot.

Sampling bias

: La muestra de entrenamiento no es representativa de la población. Caso clásico: encuesta Literary Digest 1936, predijo Landon ganador porque encuestaron a suscriptores con teléfono (sesgo socioeconómico). En ML moderno: entrenar reconocimiento facial con caras mayormente blancas y desplegarlo global.

Data snooping bias

: Mirar el test set para tomar decisiones de modelado (qué features incluir, qué hiperparámetros probar). Inflás artificialmente la performance reportada porque el "test" ya contaminó tus elecciones. Regla: separa test al inicio y no lo toques hasta el final.

Feature engineering

: Proceso de construir features informativos a partir del raw data. Incluye selección (descartar irrelevantes), extracción (combinar features existentes), y creación (fecha → día_de_semana, es_finde, mes). Históricamente, más palanca que cambiar de modelo.

Dataset / recursos

Demos sintéticas con `sklearn.datasets.make_regression` y `make_classification` para visualizar curvas de aprendizaje (train vs. validation a medida que crece N) y curvas de validación (score vs. hiperparámetro de capacidad). No hace falta dataset externo — la clase es conceptual + diagnóstica.

Ejercicios

1. Diagnóstico visual. Genera un dataset con `make_regression(n_samples=50, noise=20)`. Ajusta `PolynomialFeatures(degree=15) + LinearRegression`. Compara `train_score` y `test_score`. ¿Es overfitting o underfitting? Repetí con `degree=1` sobre datos no lineales.
2. Learning curve. Usá `sklearn.model_selection.learning_curve` sobre un dataset. Plotea `train_score` y `val_score` vs. tamaño del train. Identificá: (a) si la brecha se cierra con más datos → vale la pena conseguir más, (b) si las dos convergen bajo → modelo demasiado simple.
3. Ridge vs. Lasso. Sobre el mismo dataset polinomial del ej. 1, ajustá `Ridge(alpha=...)` para `alpha` [0.001, 0.01, 0.1, 1, 10, 100]. Plotea `train_score` y `test_score` vs. `alpha` (curva de validación). Encontrá el sweet spot.
4. Sampling bias simulado. Genera un dataset clasificación binaria balanceado (`n_samples=10000`). Entrená un modelo con un train sesgado (90% clase 0, 10% clase 1) y testéalo en un test balanceado. Reportá `accuracy` global y por clase. ¿Qué te dice el `accuracy` global solo?

5. Feature engineering manual. Para un dataset (fecha_timestamp, monto), predecí monto (a) usando solo timestamp como número, (b) extrayendo día_semana, mes, es_finde. Compará R^2 — la diferencia es el valor de feature engineering.

Homework verificable

Notebook que tome `sklearn.datasets.fetch_california_housing`, entrene tres modelos: (a) `LinearRegression` baseline, (b) `LinearRegression` sobre `PolynomialFeatures(degree=3)` sin regularizar, (c) `Ridge(alpha=10)` sobre las mismas polynomial features. Para cada uno reportá train_R^2 y test_R^2 con `train_test_split(random_state=42)`. Plotea las tres barras lado a lado.

Criterio de aceptación: El modelo (b) debe mostrar overfitting evidente (train alto, test bajo o negativo). El modelo (c) debe tener test_R^2 mejor que (b) y train_R^2 menor que (b) — evidencia de que la regularización trabajó. El test set se separa al principio y no se usa para tunear alpha (eso va por CV en el train).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
train_score = 0.99, test_score = 0.40	Overfitting clásico. Fix: regularizá (Ridg
Ambos scores bajos (0.30 train, 0.28 test)	Underfitting. Fix: modelo más expresivo, m
Reportás accuracy=0.95 en clases desbalanc	El modelo predice siempre la mayoritaria.
Tuneás alpha mirando el test set	Data snooping. El test ya no es test. Fix:
Ridge no cambia nada vs. LinearRegression	No escalaste los features. Ridge penaliza

Preguntas frecuentes

¿Más datos o modelo más complejo?

Más datos, casi siempre. Banko & Brill (2001) mostraron que algoritmos mediocres con mucho dato superan a algoritmos sofisticados con poco. Solo cuando estás bloqueado para conseguir más datos vale la pena pelear con la arquitectura.

¿Ridge o Lasso?

Ridge (L2) si querés conservar todos los features y solo bajarles la magnitud. Lasso (L1) si querés selección automática — fuerza coeficientes a 0 exactos. ElasticNet mezcla las dos.

¿Cómo sé si tengo "datos insuficientes"?

Plotea la learning curve. Si val_score sigue subiendo cuando agregás más train → te faltan datos. Si ya hizo plateau → el cuello de botella es otro (modelo, features, ruido).

¿El test set se puede usar más de una vez?

No para tomar decisiones. Sí al final, para reportar performance. Si tunear hiperparámetros y elegir features mirando el test, ese número deja de ser un estimador honesto. Para tuning usá CV sobre el train o un validation set aparte.

¿Overfitting siempre es malo?

Casi siempre. La excepción rara: cuando deploy y train son el mismo universo cerrado (overfitting a una tabla fija que nunca crece). En ML real, donde llegan datos nuevos, overfitting = modelo frágil que falla en producción.

Referencias

- Géron, cap. 1 § "Main challenges of machine learning" y § "Testing and validating".
- sklearn — Validation curves
- sklearn — Underfitting vs. overfitting
- Banko & Brill (2001), Scaling to very very large corpora for natural language disambiguation.
- Halevy, Norvig & Pereira (2009), The Unreasonable Effectiveness of Data.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 052 — Clase 052 — Testing, validación, hyperparameter tuning, no free lunch theorem

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 2 + sklearn user guide. Duración estimada: 70 min.

Objetivo

Que el alumno entienda cómo medir generalización sin engañarse — separar train/val/test correctamente, usar cross-validation para estimar performance con poca varianza, tunear hiperparámetros con GridSearchCV / RandomizedSearchCV, y aceptar el no free lunch theorem: ningún modelo gana en todos los datasets.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Separar un dataset en train/validation/test y justificar por qué el test no se toca hasta el final.
2. Aplicar KFold y StratifiedKFold con cross_val_score para estimar generalización.
3. Tunear hiperparámetros con GridSearchCV y RandomizedSearchCV, leyendo cv_results_.
4. Reconocer cuándo KFold rompe (datos temporales, grupos) y elegir el splitter correcto.
5. Explicar el no free lunch theorem y por qué siempre conviene comparar varios modelos.

Temas

#	Tema	Por qué importa
1	Train / validation / test split	Test es sagrado: una sola medición al final
2	KFold y StratifiedKFold	Estima generalización con menos varianza q
3	cross_val_score y cross_validate	Una línea, K métricas, intervalo de confia
4	GridSearchCV vs RandomizedSearchCV	Grid es exhaustivo, random escala mejor en
5	Pipeline + CV (evitar leakage del scaler)	Fitear el scaler dentro del fold, nunca af
6	No free lunch theorem	Ningún algoritmo domina en todo dataset —

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 049b — Validación temporal: TimeSeriesSplit, walk-forward, blocking

Definiciones y características

Train / validation / test split

: Train entrena, validation tunea hiperparámetros, test se mira una sola vez al final. Proporciones típicas: 60/20/20 o 70/15/15. En CV el val se reemplaza por los K folds, pero el test queda igualmente intocable.

Cross-validation (CV)

: Estimar generalización promediando K mediciones rotando qué fold es val. Reduce varianza vs un hold-out único. Costo: K veces más entrenamientos.

Hold-out

: Una sola partición train/val. Rápido pero ruidoso — con datasets chicos la métrica depende mucho de qué muestras cayeron en val.

Hyperparameter

: Configuración del modelo que no se aprende del data (ej. max_depth, C, learning_rate). Se tunea en validation, nunca en test.

Generalization gap

: Diferencia entre score en train y score en val/test. Gap grande = overfitting. Gap chico pero score bajo = underfitting.

No free lunch theorem (Wolpert, 1996)

: Promediado sobre todos los problemas posibles, ningún algoritmo de ML es mejor que otro. En la práctica: no hay un "mejor modelo" universal; siempre hay que comparar (linear, RF, GBM, etc.) sobre tu dataset específico.

Leak temporal

: Cuando el train contiene información que en producción no estaría disponible al momento de predecir (típicamente, datos del futuro respecto al target). Mata silenciosamente proyectos de forecasting.

Dataset / recursos

- sklearn.datasets.load_breast_cancer para CV estratificado.
- sklearn.datasets.fetch_california_housing para tuning con GridSearchCV.
- Serie sintética con np.cumsum(np.random.randn(500)) para TimeSeriesSplit.

Ejercicios

1. Hold-out vs CV. Sobre breast_cancer, comparar el score de un único train_test_split (varios random_state) vs cross_val_score(cv=5). Mostrar que el hold-out varía ± 0.03 entre seeds, CV mucho menos.
2. StratifiedKFold. Con un target desbalanceado 90/10, comparar KFold vs StratifiedKFold mirando la proporción de la clase minoritaria en cada fold.
3. GridSearchCV en pipeline. Pipeline([('scaler', StandardScaler()), ('svc', SVC())]) + GridSearchCV sobre C y gamma. Verificar que el scaler se fitea dentro de cada fold (no leakage).
4. RandomizedSearchCV. Mismo problema que (3) pero con RandomizedSearchCV(n_iter=30) y distribuciones (scipy.stats.loguniform). Comparar tiempo y best_score_.
5. TimeSeriesSplit. Generar serie con tendencia + ruido. Aplicar TimeSeriesSplit(n_splits=5) y entrenar Ridge con features lag-1, lag-7. Reportar score por fold. Repetir con KFold(shuffle=True) y mostrar que el score se infla artificialmente (leakage temporal).

Homework verificable

Notebook que sobre california_housing: (a) split 80/20 train/test con random_state=42; (b) GridSearchCV(cv=5) con RandomForestRegressor sobre n_estimators {100, 300} y max_depth {None, 10, 20}; (c) reportar best_params_ y best_score_; (d) evaluar el mejor modelo una sola vez sobre test; (e) comparar contra un baseline DummyRegressor(strategy='mean').

Criterio de aceptación: El test se evalúa una única vez al final del notebook. El RF supera al baseline en R^2 al menos por 0.4. cv_results_ queda exportado a CSV.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Score de CV altísimo pero en producción se	Leakage. Fix: scaler/encoder dentro del Pi
Usar KFold con datos temporales y obtener	Estás "viendo el futuro". Fix: TimeSeriesS
GridSearchCV tarda horas	Espacio de búsqueda demasiado grande. Fix:
Mirar el test set varias veces para "ajust	Estás haciendo overfitting del test → la m
Folds con clase minoritaria ausente (Value	KFold random en target desbalanceado. Fix:

Preguntas frecuentes

¿Cuándo TimeSeriesSplit en lugar de KFold?

Siempre que haya orden temporal con sentido predictivo: forecasting, logs, sensores, transacciones, métricas de negocio. Si las observaciones son intercambiables (tabular cross-sectional, ej. clasificar imágenes independientes), KFold/StratifiedKFold está bien. Regla mecánica: si tu target depende del tiempo y predecís hacia adelante, TimeSeriesSplit.

¿GridSearchCV o RandomizedSearchCV?

Grid si tenés $\leq \sim 50$ combinaciones y querés exhaustividad. Random a partir de espacios grandes (>100 combos) o cuando hay hiperparámetros continuos (C, alpha, learning_rate): con 30-60 iteraciones random llegás cerca del óptimo con una fracción del costo (resultado teórico de Bergstra & Bengio 2012).

¿K=5 o K=10 en KFold?

K=5 es el default razonable (buen compromiso varianza/costo). K=10 cuando el dataset es chico y necesitás más muestras por fold. K=N (LOOCV) casi nunca: costo absurdo y varianza alta.

¿Por qué importa el no free lunch theorem en la práctica?

Porque te recuerda no quedarte con un solo modelo "favorito". Para cualquier dataset nuevo, comparar al menos: un baseline tonto (Dummy), un lineal (Ridge/LogReg), un árbol ensamble (RandomForest/GradientBoosting). El que gane depende del problema, no de tu intuición previa.

¿Puedo usar cross_val_score y después GridSearch sobre el mismo set?

Sí, pero entendé que estás haciendo CV anidada implícita. Lo correcto cuando querés estimar la performance del proceso de tuning: cross_val_score(GridSearchCV(...), X_train, y_train, cv=outer_cv). CV externa evalúa, CV interna tunea. Caro pero honesto.

Referencias

- Géron, Hands-On ML, cap. 2 — End-to-End ML Project (sección "Better Evaluation Using Cross-Validation").
- sklearn — Cross-validation user guide
- sklearn — TimeSeriesSplit

- sklearn — GridSearchCV / RandomizedSearchCV
- Bergstra & Bengio (2012), Random Search for Hyper-Parameter Optimization, JMLR.
- Wolpert (1996), The Lack of A Priori Distinctions Between Learning Algorithms.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 053 — Clase 053 — Validación temporal: TimeSeriesSplit, walk-forward, blocking

Parte: 1 — Machine Learning Clásico · Fuente: Bergmeir & Benítez (2012) + sklearn TimeSeriesSplit docs. Duración estimada: 70 min.

Objetivo

Aplicar validación correcta para series temporales — donde KFold y train_test_split aleatorio causan leakage del futuro al pasado y métricas infladas. Cubrir TimeSeriesSplit, walk-forward validation (rolling y expanding), blocking para datos con dependencias intra-cluster, purged + embargoed CV (López de Prado, finanzas).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar sklearn.model_selection.TimeSeriesSplit(n_splits, max_train_size, test_size, gap).
- Implementar walk-forward rolling (ventana fija) y expanding (acumulativo).
- Detectar leakage cuando KFold se usa sobre datos temporales.
- Aplicar purged + embargoed K-Fold para evitar leakage por feature engineering con lags.
- Reportar métricas multi-fold con dispersión (no solo promedio).

Temas

- ¿Por qué KFold falla en series? Aleatorización mezcla pasado y futuro.
- TimeSeriesSplit: split secuencial, train siempre antes que test.
- Expanding vs rolling window.
- gap (embargo) para target leakage con lags.
- Purged CV (López de Prado): elimina overlap entre train y test.
- CombinatorialPurgedKFold para backtesting.

Definiciones y características

- TimeSeriesSplit(n_splits=5): divide la serie en N+1 chunks secuenciales; itera (train=primer N, test=último).
- Walk-forward expanding: train crece, test va avanzando.
- Walk-forward rolling: train tiene tamaño fijo, ventana se mueve.
- Embargo / gap: número de samples ignoradas entre train y test — evita leakage por features con lags.
- Purged CV: elimina del train cualquier sample cuyo target overlapped con el test.

Dataset / recursos

- Serie temporal sintética o seaborn.load_dataset('flights').
- Librerías: scikit-learn, pandas, mlxtend (alternativa con más options).

Ejercicios

1. TSSplit vs KFold leak: con serie sintética con tendencia, comparar score CV con KFold aleatorio vs

TimeSeriesSplit. KFold infla.

2. Walk-forward expanding: `tscv = TimeSeriesSplit(n_splits=5)`. Iterar y reportar score por fold.
3. Rolling window: con `max_train_size=100`, simular walk-forward de window fijo.
4. gap: con feature `y_t-1` (target lag), aplicar `gap=1` para evitar que test "vea" su propio target.
5. Score con dispersión: reportar `mean ± std` de RMSE por fold, no solo mean.

Homework verificable

Forecasting con XGBoost en serie de retail:

1. Feature engineering: lags 1, 7, 30 + rolling mean.
2. CV con `TimeSeriesSplit(5, gap=30)`.
3. Reportar RMSE por fold + global.
4. Comparar contra KFold ingenuo — mostrar inflación.

Criterio de aceptación: KFold subestima RMSE en $\geq 30\%$; TimeSeriesSplit da estimación realista que se sostiene en test final.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Métrica CV mucho mejor que producción	KFold sobre temporal. Fix: TimeSeriesSplit
Feature <code>y_t-1</code> y CV sin gap	Target leak. Fix: <code>gap = max_lag</code> .
<code>n_splits</code> muy alto con serie corta	Folds muy chicos. Fix: 3-5 folds.
Reportar solo mean	Variabilidad oculta. Fix: <code>mean ± std</code> .
Rolling con <code>max_train_size</code> mal calibrado	Train muy chico → ruido. Fix: ≥ 1 ciclo es

Preguntas frecuentes

Expanding o rolling?

Expanding usa toda la historia (más data); rolling refleja "olvido" si crees que la dinámica cambia. Probá ambos.

TimeSeriesSplit con feature engineering sobre toda la serie?

Leak. Fix: features con rolling/lag calculadas con `min_periods=window` para no usar futuro.

Para crypto / trading?

Purged + embargoed (López de Prado Advances in Financial Machine Learning).

Hyperparameter tuning con TimeSeriesSplit?

GridSearchCV con `cv=TimeSeriesSplit(5)`. Funciona idéntico.

Nested CV?

Recomendado para tuning + evaluation honesta. Outer TSSplit para reportar, inner para tunear.

Referencias

- Bergmeir & Benítez (2012), On the use of cross-validation for time series predictor evaluation.
- López de Prado (2018), Advances in Financial Machine Learning, cap. 7.
- `sklearn.model_selection.TimeSeriesSplit`.
- Hyndman & Athanasopoulos, Forecasting: Principles and Practice.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 054 — Clase 054 — Proyecto end-to-end: visión, datos, exploración, preparación

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 2 (California Housing). Duración estimada: 90 min.

Objetivo

Que el alumno recorra la primera mitad de un proyecto de ML real de punta a punta: framear el problema en términos de negocio, conseguir los datos, hacer un EDA honesto, separar train/test sin contaminarse, y dejar el pipeline de preparación (limpieza, encoding, scaling) listo para entrenar — todo sobre el dataset California Housing del capítulo 2 de Géron.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Framear el problema en términos de negocio: tipo de tarea (regresión/clasificación), métrica, baseline.
2. Hacer un EDA reproducible: describe, info, hist, corr, scatter matrix, mapas geográficos.
3. Separar train/test correctamente con `train_test_split` estratificado por una variable clave (income bucket).
4. Construir un pipeline con Pipeline + ColumnTransformer que limpie, encode (OneHotEncoder) y escale (StandardScaler) en un solo objeto.
5. Evitar data leakage: todo cálculo (medias, encodings, scalers) se ajusta solo en train y se aplica en test.

Temas

#	Tema	Por qué importa
1	Framing del problema	Sin objetivo claro y métrica, cualquier mo
2	EDA: describe, hist, corr, geo plot	Conocer los datos antes de modelar.
3	Stratified split por income bucket	Test representativo, no muestreo aleatorio
4	Limpieza: NaN, outliers, tipos	El 60% del trabajo real.
5	Encoding categórico (OneHotEncoder, Ordinal	Los modelos no comen strings.
6	Scaling (StandardScaler, MinMaxScaler)	Imprescindible para modelos sensibles a es
7	Pipelines + ColumnTransformer	Reproducible, sin leakage, deployable.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 050b — Feature Engineering avanzado + MICE imputation

Definiciones y características

Pipeline (sklearn)

: Secuencia de transformaciones (nombre, estimator) que termina opcionalmente en un modelo. `fit` ajusta cada paso con la salida del anterior; `transform/predict` aplica en orden. Encadena todo en un objeto y previene leakage cuando se usa con CV.

ColumnTransformer

: Aplica transformadores distintos a subconjuntos de columnas (numéricas → scaler, categóricas → encoder) y concatena el resultado. Es el pegamento entre EDA y modelo.

Stratified split

: Separación train/test que preserve la distribución de una variable clave (target en clasificación, o un bucket de una numérica como `income_cat`). Evita que el test caiga sesgado por azar — crítico en N chico.

OneHotEncoder

: Convierte cada valor único de una categórica en una columna binaria. Default sklearn: sparse matrix. Parámetros clave: `handle_unknown='ignore'` (para categorías nuevas en test), `drop='first'` (para evitar multicolinealidad en modelos lineales).

StandardScaler

: Resta media y divide por desvío estándar (z-score). Necesario para modelos sensibles a escala: SVM, KNN, redes neuronales, regresión regularizada (Ridge/Lasso). Árboles no lo necesitan.

Target leakage

: Cualquier feature, encoding o estadístico que contenga información del target o del test al momento de entrenar. Inflará métricas en validación y colapsará en producción. Síntoma típico: `AUC > 0.99` sospechoso.

SimpleImputer

: Imputación univariada (media/mediana/moda/constante). Default razonable para empezar; reemplazable por `KNNImputer` o `IterativeImputer` sin cambiar el resto del pipeline.

Métrica vs función de costo

: La métrica la elige el negocio (RMSE en dólares, recall en fraude); la función de costo la usa el optimizador internamente (puede ser distinta — MSE para entrenar, RMSE para reportar).

Dataset / recursos

California Housing (Géron cap. 2). 20 640 filas, 10 columnas, target = `median_house_value`. Disponible vía `sklearn.datasets.fetch_california_housing()` o el CSV del repo de Géron. Contiene una categórica (`ocean_proximity`) y una columna con NaN (`total_bedrooms`) — perfecta para practicar pipeline completo.

Ejercicios

1. EDA mínimo. Cargá el dataset y producí: `df.info()`, `df.describe()`, `df.hist(bins=50, figsize=(12,8))`. Identificá al menos 2 anomalías (cap visual de `median_house_value`, distribución skewed de population).
2. Stratified split por income bucket. Creá `income_cat = pd.cut(df['median_income'], bins=[0, 1.5, 3, 4.5, 6, np.inf])` y usá `StratifiedShuffleSplit` para train/test 80/20. Verificá que la distribución de `income_cat` sea casi idéntica en train y test.
3. Target encoding sin leakage. Tomá una variable categórica (creá `zipcode_fake` a partir de buckets de lat/long si querés). Implementá target encoding con `category_encoders.TargetEncoder` ajustado solo en train. Compará RMSE de un `RandomForestRegressor` con (a) one-hot vs (b) target encoded.
4. `KNNImputer` vs `SimpleImputer`. Sobre `total_bedrooms` (que tiene NaN reales), comparen RMSE final del pipeline usando `SimpleImputer(strategy='median')` vs `KNNImputer(n_neighbors=5)`. Reportá cuál ganó y en

cuánto.

5. Pipeline completo. Armá un ColumnTransformer con: numéricas → SimpleImputer(median) + StandardScaler; categóricas → OneHotEncoder(handle_unknown='ignore'). Envuelvelo en un Pipeline con LinearRegression al final. fit en train, RMSE en test.

Homework verificable

Notebook con: (a) carga del dataset + EDA con al menos 4 gráficos; (b) stratified split por income bucket con verificación de proporciones; (c) pipeline ColumnTransformer (numéricas con KNNImputer+StandardScaler, categóricas con OneHotEncoder); (d) feature engineering manual con al menos 2 features derivadas (rooms_per_household, bedrooms_per_room); (e) baseline LinearRegression con RMSE reportado en train y test.

Criterio de aceptación: El pipeline se entrena con pipeline.fit(X_train, y_train) sin tocar X_test antes del scoring final. RMSE de test reportado. Sin warnings de sklearn por leakage o categorías nuevas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Target encoding da AUC casi perfecto en tr	Encoding calculado sobre todo el dataset →
ValueError: Found unknown categories ['X']	El test trae una categoría nueva no vista
RMSE de test ridículamente bajo, idéntico	Imputaste/escalaste sobre pd.concat([train
fillna(df.mean()) baja drásticamente la va	Imputación univariada con media aplasta la
Modelo lineal con coeficientes raros (uno	Olvidaste el StandardScaler — features est

Preguntas frecuentes

¿Cuándo target encoding > one-hot?

Cuando la cardinalidad es alta (>~15 valores únicos) y existe relación monotónica entre categoría y target. One-hot con 1000 zipcodes te da 1000 columnas sparse y árboles que no convergen; target encoding te da 1 sola columna numérica informativa. Siempre con CV out-of-fold para no filtrar el target.

¿Necesito escalar si uso Random Forest o XGBoost?

No. Los árboles particionan por umbrales, son invariantes a transformaciones monotónicas. Sí lo necesitás para SVM, KNN, redes, regresión lineal/logística con regularización.

¿Stratified split en regresión?

Sí — pero estratificás por una versión bucketizada del target o de una feature clave (como income_cat en California Housing). StratifiedShuffleSplit no acepta target continuo directamente.

¿StandardScaler o MinMaxScaler?

StandardScaler por default (z-score, asume distribución aprox. normal). MinMaxScaler cuando necesitás bounds fijos [0,1] (redes con sigmoide, ciertos algoritmos de visión). RobustScaler si hay outliers fuertes.

¿Hago feature engineering antes o después del split?

Features que dependen solo de la fila (ratios, log, sin/cos): antes o después, da igual. Features que dependen de estadísticos agregados (target encoding, frequency encoding, z-scores globales): siempre después del split y fit solo en train.

Referencias

- Géron, cap. 2 — End-to-End Machine Learning Project (California Housing).
- sklearn — Pipeline y ColumnTransformer
- sklearn — impute (SimpleImputer, KNNImputer, IterativeImputer)
- sklearn — TargetEncoder
- category_encoders docs
- Rubin, D. B. (1976). Inference and missing data. Biometrika.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 055 — Clase 055 — Feature Engineering avanzado: target encoding + MICE imputation

Parte: 1 — Machine Learning Clásico · Fuente: Micci-Barreca (2001) target encoding + Van Buuren (2018) Flexible Imputation of Missing Data. Duración estimada: 85 min.

Objetivo

Dominar feature engineering moderno más allá de one-hot y SimpleImputer: target encoding con regularización + cross-validation (evita leakage), KNNImputer y IterativeImputer (MICE) para imputación multivariada inteligente, y category_encoders library para codificaciones modernas (CatBoost, James-Stein, hashing).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar target encoding con CV (no leak): category_encoders.TargetEncoder(cv=5, smoothing=10.0).
- Aplicar KNNImputer: imputa basado en vecinos.
- Aplicar IterativeImputer (MICE) de sklearn — predice cada feature con modelo de las demás.
- Decidir entre métodos: SimpleImputer (baseline), KNN (correlaciones locales), MICE (multivariada).
- Reconocer leakage en target encoding y evitarlo con CV interno.

Temas

- Target encoding clásico + smoothing bayesiano.
- Leak en target encoding sin CV: features ven sus propios targets.
- KNNImputer (sklearn): nearest neighbors por filas.
- IterativeImputer / MICE: estima cada feature con regresión.
- category_encoders: CatBoost, James-Stein, target, hashing.
- Pipeline-safe imputation.

Definiciones y características

- Target encoding: reemplazar categoría con su mean target. Con smoothing: combina con global mean.
- Smoothing: $\text{enc} = (n \cdot \text{mean_cat} + k \cdot \text{global}) / (n + k)$ — protege categorías raras.
- CV target encoding: cada fold usa encoding fitted en otros folds → sin leakage.
- KNNImputer: para cada NaN, promedia k vecinos en feature space.
- MICE: iterativo — predice cada feature con regresión usando las demás.
- CatBoostEncoder: variant de target encoding sin necesitar CV (ordering trick).

Dataset / recursos

- `fetch_openml('credit-g')` o California Housing con NaN inyectados.
- Librerías: `category_encoders` (pip install `category_encoders`), `scikit-learn`, `pandas`.

Ejercicios

1. Target encoding leak: encoding sobre train+test → métrica inflada. Mostrar.
2. Target encoding con CV: `TargetEncoder(cv=5, smoothing=10)` dentro de pipeline. Sin leak.
3. CatBoost encoder: alternativa sin CV. Comparar performance.
4. `KNNImputer`: con dataset con NaN, imputar con `k=5`; comparar contra `SimpleImputer` (mean).
5. MICE: `IterativeImputer(estimator=BayesianRidge(), max_iter=10)`. Comparar.

Homework verificable

Sobre `credit-g` con NaN sintéticos (10 % de missing en 3 columnas):

1. Pipeline 1: `SimpleImputer` + `OneHot` + `LogReg`.
2. Pipeline 2: `KNNImputer` + `TargetEncoder(CV)` + `LogReg`.
3. Pipeline 3: MICE + `CatBoostEncoder` + `LogReg`.
4. Comparar accuracy + tiempo.

Criterio de aceptación: pipelines modernos (2, 3) superan `SimpleImputer` baseline en AUC por ≥ 0.5 pp; sin target leakage en target encoding.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Target encoding fitted sobre X completo →	Leak. Fix: <code>TargetEncoder</code> dentro de pipelin
MICE con <code>max_iter=2</code> poco convergente	Fix: 10-20 iterations.
<code>KNNImputer</code> lento con N grande	$O(N^2)$ en distancia. Fix: KNN con <code>n_neighbo</code>
<code>Smoothing=0</code> sobre categoría con <code>n=1</code>	Encoding = ese 1 target → overfit. Fix: sm
<code>OneHot</code> para 10k+ categorías	Curse of dimensionality. Fix: target encod

Preguntas frecuentes

Target encoding mejor que one-hot?

Para alta cardinalidad (> 50 categorías), sí — mucho. Para baja, comparable o peor.

MICE o KNN?

MICE mejor con features correlated (predice con modelo). KNN mejor con clusters locales. Probar.

Smoothing value?

Empírico — 5-30 típicamente. Más smoothing = más conservador hacia global mean.

`category_encoders` integrado en `sklearn`?

No, lib externa. `sklearn` tiene `TargetEncoder` desde 1.3, pero `category_encoders` tiene más options.

Para tree-based (XGBoost)?

Mucho menos crítico — los árboles manejan categóricas razonable. Pero target encoding ayuda con cardinalidad alta.

Referencias

- Micci-Barreca (2001), A Preprocessing Scheme for High-Cardinality Categorical Attributes.

- Van Buuren (2018), Flexible Imputation of Missing Data (libro gratuito).
- category_encoders docs.
- sklearn IterativeImputer.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 056 — Clase 056 — Selección y entrenamiento de modelo

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 2 § Select and Train a Model.

Duración estimada: 60 min.

Objetivo

Que el alumno entrene varios modelos baseline sobre el pipeline ya preparado (California Housing), los compare con cross-validation en vez de un único split, identifique sub/overfitting con learning curves, y elija el candidato más prometedor para pasar a fine-tuning — sin malgastar tiempo afinando un modelo que no tiene techo.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Entrenar baselines (LinearRegression, DecisionTreeRegressor, RandomForestRegressor) sobre el `X_prepared` del pipeline de la clase anterior.
2. Evaluar con `cross_val_score` usando K-Fold y `scoring='neg_root_mean_squared_error'` en vez de un solo train/test.
3. Leer learning curves para diagnosticar bias vs varianza (underfitting vs overfitting).
4. Comparar modelos con media $\pm$ desvío de los folds y decidir cuál merece HPO.
5. Persistir el modelo elegido con `joblib.dump(...)` para retomarlo en la próxima clase.

Temas

#	Tema	Por qué importa
1	Entrenar baseline LinearRegression y medir	Punto de comparación honesto. Si el lineal
2	DecisionTreeRegressor con RMSE = 0 en tra	Caso canónico de overfitting; te enseña a
3	<code>cross_val_score(..., cv=10, scoring='neg_r</code>	Estimación robusta de error de generalizac
4	RandomForestRegressor y comparación con	Baseline fuerte en tabular; suele ganar an
5	<code>learning_curve</code> — train vs validation score	Diagnóstico visual de bias/varianza.
6	<code>validation_curve</code> — score vs un hiperparáme	Antesala del grid search de la clase sigui
7	Decidir cuándo pasar a HPO vs cuándo segui	Criterio de corte; evita el agujero del tu

Definiciones y características

Baseline model

: Modelo simple (lineal, árbol corto, dummy) contra el cual se compara cualquier modelo más complejo. Si un Random Forest no le gana al LinearRegression por margen claro, el feature engineering está fallando — no el modelo.

`cross_val_score(estimator, X, y, cv, scoring)`

: Entrena `cv` veces sobre folds disjuntos y devuelve un array de scores. En `sklearn`, `scoring` siempre es "más alto es mejor" — por eso para RMSE se usa `neg_root_mean_squared_error` (negado) y después se invierte el signo.

`scoring`

: String que selecciona la métrica. Para regresión: `'neg_root_mean_squared_error'`, `'neg_mean_absolute_error'`, `'r2'`. Para clasificación: `'accuracy'`, `'f1'`, `'roc_auc'`. La lista completa está en `Scoring parameter`.

`Learning curve`

: Gráfico de score (train y validation) en función del tamaño del training set. Diagnostica: brecha grande train ↔ val = overfitting (varianza alta); ambas curvas bajas y juntas = underfitting (bias alto); ambas convergen alto = modelo OK.

`Validation curve`

: Gráfico de score (train y validation) en función de un hiperparámetro (`max_depth`, `n_estimators`, etc.). Te muestra a ojo el rango interesante antes de tirar un `grid search`.

`Model card`

: Documento corto (markdown, una página) que registra: dataset, features, modelo, métricas en CV, hiperparámetros, fecha, supuestos y limitaciones. Práctica de Google/Hugging Face. Te salva cuando volvés al proyecto 3 meses después.

`joblib.dump` / `joblib.load`

: Serialización optimizada para objetos `sklearn` (matrices NumPy grandes). Preferido sobre `pickle` puro. Convención: `modelo.pkl` o `modelo.joblib`.

Dataset / recursos

Seguimos con California Housing y el `X_prepared` que sale del pipeline construido en las clases 049-050. Si arrancás esta clase fresca, el notebook regenera el pipeline desde `fetch_california_housing()` para que sea autocontenido.

Ejercicios

1. Baseline lineal. Entrená `LinearRegression()` sobre `X_prepared`, predecí sobre los primeros 5 ejemplos, compará con los `y` reales. Calculá RMSE sobre todo el train. Esperá algo en el orden de ~68k USD.
2. Árbol que memoriza. Entrená `DecisionTreeRegressor(random_state=42)` sin restringir profundidad. Calculá RMSE sobre train. Vas a obtener 0 (o casi). Discutí en una celda markdown por qué eso no significa que el modelo sea bueno.
3. Cross-validation honesto. Corré `cross_val_score(tree, X_prepared, y, scoring='neg_root_mean_squared_error', cv=10)`. Reportá media y desvío del RMSE (recordá negar el signo). Compará con el lineal evaluado con el mismo `cv=10`.
4. Random Forest. Entrená `RandomForestRegressor(n_estimators=100, random_state=42)` y evaluá con CV de 10 folds. Esperá que la media baje a ~50k USD. Hacé una tabla markdown con los 3 modelos: media ± desvío.
5. Learning curve. Usá `sklearn.model_selection.learning_curve` sobre el Random Forest con `train_sizes=np.linspace(0.1, 1.0, 5)`. Plotteá `train_score` y `val_score` vs tamaño. Diagnostica: ¿alta varianza,

alto bias, o convergencia?

Homework verificable

Notebook que: (a) entrene los 3 baselines sobre `X_prepared`; (b) corra CV de 10 folds para cada uno y guarde los arrays de scores; (c) genere una tabla comparativa con RMSE media, RMSE desvío, tiempo de fit; (d) elija el modelo más prometedor con justificación de 3 líneas en markdown; (e) plottee la learning curve del elegido; (f) serialice el modelo entrenado en `models/modelo_baseline.pkl` con `joblib.dump`.

Criterio de aceptación: El Random Forest aparece con RMSE-CV menor al lineal y al árbol pelado. El archivo `.pkl` se puede recargar con `joblib.load` y predice sin errores sobre los primeros 5 ejemplos de `X_prepared`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Reporto el RMSE como un número enorme posi	Olvidaste que sklearn devuelve scores nega
Decision tree con RMSE = 0 en train y lo d	Estás midiendo en el mismo set en que entr
cross_val_score tarda eternidad con Random	Cada fold reentrena el bosque entero. Fix:
Learning curve plana en train y val ambas	Underfitting. Fix: no es problema de datos
Hago cross_val_score sobre el X crudo (sin	Te salteaste el preprocessing. Fix: pasale

Preguntas frecuentes

¿Por qué CV de 10 y no un train/test split?

Un solo split te da un solo número con varianza alta — podés tener mala/buena suerte con esa partición. CV te da 10 números: media + desvío. Si el desvío entre folds es grande, el modelo es inestable y un score puntual miente.

¿neg_root_mean_squared_error o neg_mean_squared_error?

`neg_root_mean_squared_error` está disponible desde sklearn 0.22 y te devuelve directo el RMSE negado. Si tu versión es vieja, usá `neg_mean_squared_error` y aplicá `np.sqrt(-scores)` a mano.

¿Cuándo dejo de probar baselines y paso a HPO?

Cuando el mejor baseline ya le saca margen claro al segundo, y la learning curve muestra que más datos no van a mover la aguja. Si seguís en bias alto, antes de HPO sumá features o subí la familia de modelo.

¿Sirve mirar el RMSE de train?

Sí, pero solo como diagnóstico, no como métrica de selección. Train bajo + val alto = overfitting. Train alto + val alto = underfitting. La métrica que ranquea modelos es siempre la de validation (o CV).

¿Cuál es la diferencia entre `learning_curve` y `validation_curve`?

`learning_curve` varía el tamaño del training set (¿necesito más datos?). `validation_curve` varía un hiperparámetro con N fijo (¿qué `max_depth` conviene?). Las dos comparten el formato train-vs-val pero responden preguntas distintas.

Referencias

- Géron, cap. 2 § Select and Train a Model + § Better Evaluation Using Cross-Validation.
- sklearn `cross_val_score`
- sklearn `learning_curve`
- sklearn Scoring parameter

- Model Cards for Model Reporting (Mitchell et al., 2019)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 057 — Clase 057 — Fine-tuning: grid search y randomized search

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 2. Duración estimada: 70 min.

Objetivo

Que el alumno deje de tunear hiperparámetros "a ojo" y use búsquedas sistemáticas con validación cruzada — GridSearchCV cuando el espacio es chico y discreto, RandomizedSearchCV cuando es grande o continuo — integradas dentro de un Pipeline para evitar data leakage del preprocesamiento.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Distinguir parámetros entrenados (pesos del modelo) de hiperparámetros (los que vos fijás antes del fit).
2. Configurar GridSearchCV con param_grid, cv, scoring, n_jobs=-1 y leer best_params_ / best_estimator_ / cv_results_.
3. Usar RandomizedSearchCV con param_distributions (scipy.stats: randint, uniform, loguniform) y n_iter para presupuestar trials.
4. Integrar HPO dentro de un Pipeline usando claves del tipo 'step__hparam' para tunear preprocesamiento + modelo juntos.
5. Decidir grid vs random vs bayesiano según tamaño del espacio, costo por fit y continuidad de los hiperparámetros.

Temas

#	Tema	Por qué importa
1	Hiperparámetros vs parámetros	El error conceptual #1 al arrancar con skl
2	param_grid y scoring	Definís el espacio y la métrica que querés
3	cv y n_jobs=-1	CV honesto + paralelismo gratis sobre todo
4	GridSearchCV	Producto cartesiano completo. Garantiza en
5	RandomizedSearchCV con distribuciones (ra	Muestrea n_iter puntos; gana cuando pocos
6	Pipelines + HPO con sintaxis step__hparam	Evita leakage del scaler/imputer al hacer
7	Inspeccionar cv_results_	DataFrame con todos los trials; sirve para

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 052a — Optuna y HPO bayesiano dedicado

Definiciones y características

Hiperparámetro

: Parámetro que vos fijás antes del entrenamiento y que el fit no aprende. Ej.: `max_depth`, `C`, `n_estimators`. Distinto de los parámetros aprendidos (pesos, splits del árbol).

`GridSearchCV(estimator, param_grid, cv, scoring, n_jobs)`

: Evalúa el producto cartesiano de `param_grid` con CV. Total de fits = $\prod |\text{valores}| \times \text{cv}$. Determinista y exhaustivo dentro del grid.

`RandomizedSearchCV(estimator, param_distributions, n_iter, cv, scoring)`

: Muestra `n_iter` combinaciones de las distribuciones. Acepta tanto listas (como grid) como distribuciones `scipy` (`randint`, `uniform`, `loguniform`). Gana cuando algunos hparams son irrelevantes.

`param_distributions`

: Dict `{'hparam': distribución_o_lista}`. Para escalas tipo learning rate o `C` de SVM, usá `loguniform(1e-4, 1e0)` — random uniforme en log-space, no lineal.

`refit=True` (default)

: Tras encontrar la mejor combinación, re-entrena el estimador sobre todo el train set con esos hparams. Por eso `best_estimator_` viene listo para predecir.

`best_params_`

: Dict con la combinación ganadora.

`best_estimator_`

: El modelo ya re-entrenado en train completo. Es el que usás para `.predict()` en test.

`scoring`

: Métrica a maximizar. Strings como `'accuracy'`, `'f1'`, `'neg_root_mean_squared_error'` (negados porque `sklearn` maximiza siempre). También callables custom con `make_scorer`.

`cv_results_`

: Dict-of-arrays con todos los trials: parámetros, mean/std de score por fold, tiempos. Lo cargás en un `DataFrame` para decidir si refinar.

Dataset / recursos

`fetch_california_housing()` de `sklearn` (mismo que viene usando Géron en cap. 2). Para los ejercicios con clasificación, `load_breast_cancer()`.

Ejercicios

1. `GridSearchCV` sobre `RandomForestRegressor`. California Housing. `param_grid` con `n_estimators` `{50, 100, 200}` y `max_features` `{4, 6, 8}`. `cv=5`, `scoring='neg_root_mean_squared_error'`, `n_jobs=-1`. Reportá `best_params_` y RMSE en test.
2. `RandomizedSearchCV` con distribuciones. Mismo dataset, ahora con `n_estimators=randint(50, 500)`, `max_features=randint(2, 8)`, `min_samples_leaf=randint(1, 20)`. `n_iter=30`. Compará tiempo y score vs el grid del ej. 1.
3. Pipeline + HPO. Construí `Pipeline([('scaler', StandardScaler()), ('svr', SVR())])`. Tuneá `svr__C` con `loguniform(1e-1, 1e3)` y `svr__gamma` con `loguniform(1e-4, 1e-1)`. Mostrá por qué meter el `StandardScaler` afuera del CV sería data leakage.
4. Inspección de `cv_results_`. Cargá `search.cv_results_` en un `DataFrame`, ordenalo por `mean_test_score` y

plottea score vs n_estimators. ¿Hay meseta? ¿Vale subir más?

5. Optuna sobre el mismo problema. Repetí el ej. 2 pero con Optuna (n_trials=30, TPESampler, MedianPruner). Compará score final y tiempo con RandomizedSearchCV. Mostrá study.best_params y un plot de optuna.visualization.plot_optimization_history.

Homework verificable

Notebook que: (a) baseline con RandomForestRegressor sin tuneear; (b) GridSearchCV con grid chico (≤ 27 combos); (c) RandomizedSearchCV con n_iter=30 y distribuciones log; (d) Optuna con n_trials=30 y MedianPruner; (e) tabla comparativa con RMSE en test, tiempo de búsqueda y mejor combinación de cada método.

Criterio de aceptación: los tres métodos (grid/random/Optuna) mejoran sobre el baseline. La tabla final tiene RMSE_test, segundos y best_params. El Pipeline se usa en todos para evitar leakage del scaler.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ValueError: Invalid parameter 'C' for esti	Estás pasando 'C' pero dentro de un Pipeli
GridSearchCV tarda eternidades	Producto cartesiano explotó. Fix: pasá a R
RMSE de CV mucho mejor que en test	Data leakage: aplicaste StandardScaler.fit
ImportError: cannot import name 'HalvingGr	Sigue siendo experimental en sklearn. Fix:
scoring='rmse' tira ValueError: 'rmse' is	sklearn no tiene 'rmse' directo; maximiza

Preguntas frecuentes

¿GridSearch, RandomSearch u Optuna?

Regla de bolsillo: espacio chico y discreto (≤ 50 combos) → Grid, te asegura el óptimo del grid. Espacio mediano o con hparams continuos → Random con loguniform para los que viven en escala log. Cada fit cuesta caro (XGBoost grande, red neuronal) o el espacio es enorme → Optuna con TPE + MedianPruner. Para producción sería hoy, Optuna es default.

¿Cuántos n_iter en RandomizedSearchCV?

Bergstra & Bengio (2012) mostraron que 60 trials random alcanzan el top-5% del espacio con probabilidad >95% si pocos hparams dominan. Empezá con 30-60 y subí si cv_results_ muestra que el top-N todavía mejora.

¿n_jobs=-1 en el search o en el modelo?

Si lo ponés en los dos, se pelean por los cores. Recomendación: n_jobs=-1 en el search (paraleliza folds × candidatos) y n_jobs=1 dentro del estimador. Para modelos que ya paralelizan internamente y son rápidos (RandomForest chico), a veces es al revés — medilo.

¿cv=5 o cv=10?

cv=5 es el sweet spot por costo/varianza. cv=10 baja un poco la varianza pero duplica tiempo. Para datasets chicos ($\leq 1k$ filas), cv=10 o incluso LOO. Para datasets grandes, cv=3 ya alcanza.

¿Tuneo sobre train y evalúo sobre test, o necesito un set de validación aparte?

Con CV adentro de GridSearchCV ya tenés validación honesta sobre folds del train. Test set se toca una sola vez al final, con el best_estimator_ ya elegido. Si vas a hacer muchas iteraciones de exploración manual

encima, dejá un set de validación separado para no contaminar test.

Referencias

- Géron, cap. 2 § Fine-Tune Your Model.
- sklearn — GridSearchCV
- sklearn — RandomizedSearchCV
- sklearn — Successive Halving (experimental)
- Optuna docs — TPE, pruners, dashboard.
- Bergstra & Bengio (2012), Random Search for Hyper-Parameter Optimization, JMLR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 058 — Clase 058 — Optuna y HPO bayesiano dedicado

Parte: 1 — Machine Learning Clásico · Fuente: Akiba et al. (2019) + Optuna docs. Duración estimada: 80 min.

Objetivo

Profundizar en Optuna —el framework de hyperparameter optimization estándar industrial 2026— aplicado a ML clásico (sklearn, XGBoost, LightGBM, CatBoost). Pasar de Grid/Random Search (clase 052) a TPE (Tree-structured Parzen Estimator) + Hyperband Pruner + persistencia con SQLite. Aprender a interpretar `plot_optimization_history`, `plot_param_importances` y `plot_slice` para entender qué hiperparámetros mueven la aguja.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir un `objective(trial)` con `suggest_int`, `suggest_float`, `suggest_categorical`, `suggest_float('lr', 1e-5, 1e-1, log=True)`.
- Aplicar TPE (default) vs CmaEs vs NSGAIISampler (multi-objective).
- Aplicar pruners (MedianPruner, HyperbandPruner) para cortar trials malos temprano.
- Persistir el study con `storage='sqlite:///study.db'` y resumir trials.
- Visualizar e interpretar los 5 plots de Optuna.
- Comparar costo total: Grid Search (1000 trials) vs Optuna (100 trials) llegan a mismo accuracy.

Temas

- TPE: modela $P(x | y < \gamma)$ y $P(x | y \geq \gamma)$ con KDE; samplea de la primera.
- Pruning: callback que reporta progreso intermedio; si va mal vs históricos, kill.
- Multi-objective: optimizar accuracy AND latencia.
- Distributed: varios workers contra el mismo SQLite/PostgreSQL.
- Integration con sklearn, XGBoost, LightGBM, CatBoost.

Definiciones y características

- Trial: una corrida del objective.
- Study: contenedor de trials; se persiste opcional.
- TPE: Tree-structured Parzen Estimator. Modelo bayesiano con KDE bi-modal.

- HyperbandPruner: combina successive halving con varios brackets.
- storage: backend de persistencia. SQLite default; Postgres para distributed.
- plot_param_importances: importancia fANOVA — cuánto contribuye cada hiperparámetro a la varianza del objetivo.

Dataset / recursos

- sklearn.datasets.fetch_california_housing (regresión) o fetch_openml('credit-g') (clasificación).
- Librerías: optuna, optuna-integration, scikit-learn, xgboost, lightgbm.

Ejercicios

1. Objective básico: tunear LogisticRegression(C, penalty) y RandomForest(n_estimators, max_depth) con TPE. 50 trials.
2. Search space compuesto: hiperparámetros condicionales (e.g., solver=liblinear solo permite ciertos penalty). Optuna lo maneja con if.
3. Pruning en XGBoost: usar XGBoostPruningCallback que reporta validación por boosting round → mata trials malos.
4. Persistencia: optuna.create_study(study_name='exp1', storage='sqlite:///opt.db', load_if_exists=True). Re-correr y agregar trials.
5. Multi-objective: maximizar accuracy AND minimizar inference time; obtener Pareto front con optuna.create_study(directions=['maximize', 'minimize']).

Homework verificable

HPO con Optuna sobre XGBoost en credit-g:

1. Espacio: n_estimators, max_depth, lr, subsample, colsample_bytree, reg_alpha, reg_lambda.
2. 100 trials con TPE + Hyperband pruning.
3. Reportar best_params, best_value, plot history + importances.
4. Comparar contra RandomizedSearchCV(50 trials).

Criterio de aceptación: Optuna $\geq$ RandomizedSearch en AUC; plot_param_importances identifica lr y/o max_depth como las más importantes.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
LR muestreado uniforme	Fix: suggest_float('lr', 1e-5, 1e-1, log=T)
TPE no parece "inteligente" en < 20 trials	El modelo interno necesita warmup. Fix: $\geq$
study.optimize(n_jobs=4) se cuelga	Race conditions con SQLite. Fix: usar Post
Pruner muy agresivo mata trials buenos	Fix: ajustar MedianPruner(n_startup_trials
Olvido reportar valor intermedio para prun	El pruner no funciona sin trial.report(val

Preguntas frecuentes

¿TPE o CmaEs?

TPE para search spaces mezclados (cat + cont). CmaEs para puramente continuo, con poca cardinalidad — converge más rápido.

¿Cuántos trials?

Para ML clásico, 50-200 alcanza. Para DL caro (cada trial 1 h), 20-50 con pruning agresivo.

¿Distributed HPO?

Postgres storage + varios workers (Python processes en distintas máquinas). Optuna lo soporta nativo.

¿Pruning siempre?

Solo si el objective expone progreso intermedio (cada época en NN, cada boosting round en XGBoost). Para `fit().score()` directo, no aplica.

¿Visualizaciones para reportar al cliente?

`plot_param_importances` y `plot_slice` son las más comunicables. Muestran "qué cambió y cuánto".

Referencias

- Akiba et al. (2019), Optuna, KDD.
- Optuna docs — tutorials y examples.
- Bergstra, Bardenet, Bengio & Kégl (2011), Algorithms for Hyper-Parameter Optimization, NeurIPS — paper original TPE.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 059 — Clase 059 — Launch, monitoreo y mantenimiento de modelos

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 2 § Launch, Monitor, and Maintain Your System.

Duración estimada: 60 min.

Objetivo

Que el alumno entienda que entrenar el modelo es la mitad del trabajo: el resto es ponerlo en producción de forma segura, monitorearlo para detectar degradación (data drift, model drift) y mantenerlo vivo con un ciclo de retraining. Además, documentar el modelo con una Model Card para que terceros (compliance, negocio, usuarios) sepan qué hace, dónde falla y qué no hay que hacerle.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diseñar un pipeline de deploy mínimo (serialización con joblib, servicio detrás de una API, versionado del artefacto).
2. Distinguir data drift de model drift y elegir métricas para cada uno (PSI, KS, accuracy en holdout móvil).
3. Definir un retraining trigger (calendario fijo vs. trigger por drift vs. trigger por caída de KPI de negocio).
4. Comparar estrategias de release (canary, shadow deploy, A/B test) y elegir según riesgo.
5. Redactar una Model Card con secciones mínimas (uso previsto, métricas por subgrupo, limitaciones).

Temas

#	Tema	Por qué importa
---	------	-----------------

1	Pipeline de deploy: joblib.dump, contenido	El modelo serializado es el artefacto prod
2	Data drift (inputs cambian) vs. model drift	Se monitorean distinto; confundirlos te ll
3	Métricas de drift: PSI, KS-test, distancia	Cuantifican el cambio en distribución ante
4	Retraining: calendario, trigger por drift,	Cuándo reentrenar sin gastar de más ni que
5	Estrategias de release: shadow, canary, A/	Bajan el riesgo de un modelo malo en produ
6	Alertas y observabilidad	Logueo de inputs/outputs, dashboards, on-c
7	Model Cards y Datasheets	Documentación responsable del modelo y de
8	Governance y rollback	Versionar modelos como código; poder volve

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 053a — Model Cards y Responsible ML

Definiciones y características

Deployment

: Proceso de exponer un modelo entrenado como servicio consumible (REST, batch, edge). Incluye serialización del artefacto (joblib, pickle, ONNX), versionado y contenedorización.

Model drift (concept drift)

: La relación entre X e y cambia con el tiempo. Ejemplo: hábitos de compra post-pandemia. Se detecta midiendo accuracy/AUC sobre una ventana móvil con labels reales (cuando llegan).

Data drift (covariate shift)

: La distribución de los inputs X cambia, aunque la relación $X \rightarrow y$ siga estable. Se detecta sin necesidad de labels — comparás la distribución de cada feature en producción vs. la del set de entrenamiento (PSI, KS).

Retraining trigger

: Regla que decide cuándo reentrenar. Tres familias: (a) calendario fijo (cada N días), (b) drift detectado (PSI > 0.25), (c) caída de KPI de negocio (conversion baja > X%).

A/B test

: Dos versiones del modelo sirven tráfico en paralelo (e.g. 50/50). Se compara KPI de negocio con significancia estadística. Requiere volumen.

Shadow deploy

: El modelo nuevo recibe los mismos requests que el viejo pero sus predicciones no se devuelven al usuario — se loguean para comparar. Cero riesgo, ideal para validar en datos reales antes del switch.

Model card

: Documento estandarizado que describe modelo, uso previsto, métricas (overall + por subgrupo), datos, limitaciones y consideraciones éticas. Formalizado por Mitchell et al. (2019).

Governance

: Conjunto de políticas: quién aprueba un deploy, cómo se versiona el modelo, cómo se hace rollback, qué se loguea para auditoría. Equivalente del "code review" para modelos.

Dataset / recursos

Sintético: dos snapshots de un dataset tabular (mes 1 y mes 6) para simular drift. Plantilla Markdown de

Model Card en model_card_template.md.

Ejercicios

1. Serializar y cargar. Entrená un RandomForestClassifier sobre Titanic. Guardalo con joblib.dump. Cargalo en otro notebook y verificá que predice idéntico.
2. Detectar data drift con PSI. Calculá Population Stability Index entre dos snapshots de la feature edad. Interpretá: $PSI < 0.1$ estable, $0.1-0.25$ leve, > 0.25 drift significativo.
3. KS-test para drift. Aplicá `scipy.stats.ks_2samp` a la feature monto entre snapshot1 y snapshot2. ¿p-valor < 0.05 ?
4. Simular shadow deploy. Tenés modelo A (viejo) y B (nuevo). Pasá 1000 requests por ambos, logueá las predicciones y reportá tasa de desacuerdo.
5. Redactar una Model Card. Tomá tu mejor modelo de la Parte 1 hasta ahora y completá una model card con las 7 secciones del complemento. Incluí al menos una métrica desagregada por subgrupo.

Homework verificable

Notebook que: (a) entrene un clasificador, (b) lo serialice con joblib, (c) simule un mes de tráfico productivo con una feature drifteada, (d) calcule PSI por feature, (e) dispare una alerta si $PSI > 0.25$, (f) entregue un archivo MODEL_CARD.md completo en la carpeta del homework.

Criterio de aceptación: El notebook detecta el drift inyectado artificialmente. La Model Card tiene las 7 secciones mínimas con valores reales (no placeholders), incluye al menos una métrica por subgrupo y declara explícitamente un out-of-scope use.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El modelo cargado con <code>joblib.load</code> predice	Cambió la versión de sklearn entre dump y
Reentrenó cada semana "por las dudas" y el	Retraining sin trigger real introduce ruid
PSI da siempre 0 o inf	División por cero cuando un bin queda vací
El A/B test "no da significativo" después	Volumen insuficiente. Fix: hacé power anal
Model card con métrica única "accuracy = 0"	Ocultó sesgos por subgrupo y no dice para

Preguntas frecuentes

¿Cada cuánto hay que reentrenar?

No hay número mágico. Si el dominio es estable (físico, industrial), meses o años. Si es comportamiento humano online (e-commerce, fraude), semanas o incluso días. Lo correcto es dejar que el trigger lo decida (PSI, caída de KPI), no el calendario.

¿Diferencia práctica entre shadow deploy y canary?

Shadow: el modelo nuevo predice pero sus predicciones no se devuelven al usuario — cero riesgo, solo comparás. Canary: el modelo nuevo sí sirve tráfico real, pero a un % chico (1-5%) — bajo riesgo pero no nulo. Shadow para validar, canary para liberar gradualmente.

¿Model card o documentación técnica clásica?

Las dos. La doc técnica (README, API reference) es para desarrolladores. La model card es para stakeholders no técnicos: producto, legal, compliance, auditoría. Si tu modelo cae bajo EU AI Act, la model

card no es opcional.

¿pickle o joblib o ONNX?

joblib para sklearn (más eficiente con arrays NumPy grandes que pickle puro). pickle para objetos Python genéricos. ONNX cuando necesitas portabilidad cross-language (servir un modelo Python desde un backend en C# o Java).

¿PSI o KS-test?

PSI es más interpretable para negocio (umbrales 0.1 / 0.25 son estándar de la industria de scoring crediticio) y opera sobre features categóricas/binneadas. KS-test es estadísticamente más riguroso para features continuas y devuelve p-valor. En la práctica, equipos serios reportan ambas.

Referencias

- Géron, cap. 2 § Launch, Monitor, and Maintain Your System.
- Mitchell et al. (2019). Model Cards for Model Reporting. FAT* 2019.
- Gebru et al. (2018). Datasheets for Datasets.
- HuggingFace Model Cards guide.
- Google model-card-toolkit.
- EU AI Act — documentación técnica (Anexo IV).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 060 — Clase 060 — Model Cards y Responsible ML

Parte: 1 — Machine Learning Clásico · Fuente: Mitchell et al. (2018) Model Cards for Model Reporting + EU AI Act + NIST AI RMF. Duración estimada: 70 min.

Objetivo

Aprender a documentar modelos para producción y auditoría: el Model Card (Mitchell et al. 2018, adoptado por Google y luego por la industria) — ficha estandarizada con: propósito, métricas, limitaciones, distribución de datos, riesgos. Conocer el EU AI Act (en vigor 2025-2026), NIST AI RMF, y las plantillas modernas (HuggingFace model cards, Datasheets for Datasets).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un Model Card completo con las 9 secciones de Mitchell et al.
- Distinguir un Model Card (sobre el modelo) de un Datasheet (sobre el dataset, Gebru et al. 2018).
- Reportar métricas por subgrupo (no solo global) — clave en fairness.
- Reconocer los 4 tiers de riesgo del EU AI Act (prohibido, alto, transparencia, mínimo).
- Aplicar el NIST AI RMF (Map, Measure, Manage, Govern) en un proyecto real.

Temas

- Secciones de un Model Card: Model Details, Intended Use, Factors, Metrics, Evaluation Data, Training Data, Quant Analyses, Ethical Considerations, Caveats.
- Métricas por subgrupo (sexo, edad, raza, geografía).

- HuggingFace Model Card auto-generation.
- EU AI Act: clasificación de riesgo, obligaciones por tier.
- NIST AI RMF: framework de gestión.
- ISO/IEC 42001 — sistema de gestión de IA.

Definiciones y características

- Model Card: ficha estructurada que documenta un modelo para terceros.
- Datasheet for Datasets: equivalente para datasets — origen, sesgos, demográficos.
- EU AI Act: regulación europea (vigor 2024-2026). Multas hasta 35M€ o 7 % revenue.
- High-risk AI: sistemas en empleo, crédito, educación, justicia, infraestructura. Requieren conformity assessment.
- GPAI (General-Purpose AI): LLMs y similares — obligaciones de transparencia adicionales.
- NIST AI RMF: framework voluntario US, ampliamente adoptado.

Dataset / recursos

- Modelo del proyecto end-to-end (clase 050).
- Plantilla HuggingFace: <<https://huggingface.co/docs/hub/model-cards>>.
- Librerías: model-card-toolkit (Google).

Ejercicios

1. Model Card básico: para un Random Forest entrenado en California Housing, llenar las 9 secciones. Salvar como MODEL_CARD.md junto al modelo.
2. Subgroup metrics: para un clasificador de credit-g, reportar accuracy y FPR por sex y age_group. Identificar disparidades.
3. Risk classification (EU AI Act): para 5 use cases (recomendación de películas, score crediticio, recurso humano selection, marketing email, detector de spam), clasificar el tier.
4. HuggingFace Card: usar el template de HF; subirla a un repo público si tenés modelo en Hub.
5. NIST RMF: para un proyecto propio, llenar las 4 categorías (Map: contexto, Measure: métricas, Manage: mitigaciones, Govern: ownership).

Homework verificable

Model Card completo para el proyecto end-to-end (clase 050):

1. Las 9 secciones de Mitchell et al. llenadas con datos reales.
2. Métricas por al menos 2 subgrupos demográficos.
3. Sección "Ethical Considerations" con ≥ 3 riesgos identificados y mitigaciones.
4. Sección "Caveats and Recommendations" con limitaciones de validez.

Criterio de aceptación: un revisor externo puede entender propósito, performance, riesgos y cómo usarlo apropiadamente sin acceso al código.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Model Card que solo reporta accuracy global	Esconde disparidades de subgrupo. Fix: tab
"Intended use" vago ("for predictions")	Inútil. Fix: especificar exactamente qué d
Omitir "Out-of-scope use cases"	No avisás al usuario. Fix: explícito ("NO
Métricas en test, no en producción real	Distribution shift no documentado. Fix: mo
Asumir EU AI Act no aplica	Aplica si el modelo se usa en UE, independ

Preguntas frecuentes

¿Model Card obligatorio?

Por ley: depende de jurisdicción y caso de uso (EU AI Act lo requiere para high-risk). Como buena práctica: siempre.

¿Qué pasa si mi modelo es high-risk EU AI Act?

Obligaciones: conformity assessment, registro en base UE, documentación técnica, supervisión humana, robustez. Multas hasta 7 % revenue.

¿Model Cards en producción industrial?

Sí. Google, Meta, Microsoft, OpenAI, Anthropic — todas publican Model Cards. HuggingFace lo requiere para modelos en Hub.

¿Datasheet for Datasets equivalente?

Sí, Gebru et al. (2018). Documenta origen, demográficos, sesgos del dataset. HuggingFace tiene Dataset Cards.

¿Y ISO 42001?

Sistema de gestión de IA (publicado dic 2023). Certificación auditable. Análogo a ISO 27001 para seguridad.

Referencias

- Mitchell, M., et al. (2018), Model Cards for Model Reporting, FAT* 2019.
- Gebru, T., et al. (2018), Datasheets for Datasets, CACM 2021.
- EU AI Act texto completo: <<https://artificialintelligenceact.eu/>>.
- NIST AI Risk Management Framework: <<https://www.nist.gov/itl/ai-risk-management-framework>>.
- HuggingFace Model Cards: <<https://huggingface.co/docs/hub/model-cards>>.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 061 — Clase 061 — CRISP-DM como framework metodológico

Parte: 1 — Machine Learning Clásico · Fuente: CRISP-DM 1.0 spec + Géron cap. 2. Duración estimada: 50 min.

Objetivo

Que el alumno entienda CRISP-DM (Cross-Industry Standard Process for Data Mining) como esqueleto metodológico para todo proyecto de ML/DS — sus 6 fases, su naturaleza iterativa (no en cascada), y cuándo conviene complementarlo o reemplazarlo por TDSP de Microsoft o por el "ML lifecycle moderno" con MLOps. La idea es dejar de improvisar el orden del trabajo y tener un mapa al que volver cuando un proyecto se trabe.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Enumerar y explicar las 6 fases de CRISP-DM y los entregables típicos de cada una.
2. Identificar la fase actual de un proyecto real y la próxima transición esperada.
3. Definir business success criteria antes de tocar datos, distinguiéndolos de métricas técnicas (accuracy, RMSE).
4. Reconocer iteraciones legítimas (volver de Evaluation a Business Understanding) vs. retrabajo por mala planificación.
5. Comparar CRISP-DM con TDSP y el ML lifecycle moderno y elegir según contexto (PoC, equipo chico, producción seria, MLOps).

Temas

#	Tema	Por qué importa
1	Business Understanding	Sin objetivo de negocio claro, el modelo n
2	Data Understanding	EDA, calidad, volumen, disponibilidad — de
3	Data Preparation	El 60–80% del tiempo real; limpieza, featu
4	Modeling	Selección de algoritmos, tuning, validació
5	Evaluation	Compara con business success criteria, no
6	Deployment	Puesta en producción, monitoreo, retrainin
7	Iteración + comparación TDSP / ML lifecycl	CRISP-DM no es cascada; existen alternativ

Definiciones y características

Business Understanding

: Fase 1. Traduce el problema de negocio a un problema de DS. Entregables: objetivos de negocio, criterios de éxito del negocio (ej. "reducir churn 5pp en 6 meses"), inventario de recursos, plan de proyecto. Sin esto, todo lo demás es ejercicio académico.

Data Understanding

: Fase 2. Recolectar datos, describirlos, explorarlos (EDA), verificar calidad. Entregable: reporte de calidad y primeras hipótesis. Si los datos no alcanzan, se vuelve a Business Understanding a renegociar scope.

Data Preparation

: Fase 3. Limpieza, integración, formateo, feature engineering, splits (train/val/test). Suele consumir el 60–80% del proyecto. Entregable: dataset final modelable + script reproducible.

Modeling

: Fase 4. Selección de técnicas, diseño de test, entrenamiento, tuning. Entregable: modelos candidatos con métricas técnicas. Si el modelo requiere otro formato de datos, se vuelve a Data Preparation.

Evaluation

: Fase 5. ¿El modelo cumple los business success criteria definidos en fase 1? No solo accuracy — costo del falso positivo, latencia, fairness, interpretabilidad. Entregable: decisión go/no-go.

Deployment

: Fase 6. Producción, monitoreo de drift, plan de retraining, documentación, handoff. Termina con un sistema funcionando, no con un notebook.

Iteración

: CRISP-DM no es cascada — las flechas van en ambos sentidos entre fases adyacentes, y la flecha externa cierra el ciclo (Deployment → nueva ronda de Business Understanding). Iterar es la norma, no la excepción.

Business success criteria

: Métricas en términos del negocio (dinero, conversión, tiempo, NPS) acordadas con stakeholders antes de modelar. Distintas de métricas técnicas (accuracy, F1, RMSE). Sin ellas, no hay forma objetiva de cerrar la fase de Evaluation.

Dataset / recursos

No hay dataset. La clase es conceptual + un caso práctico para aplicar las 6 fases (sugerido: predicción de churn telco, o un caso del alumno).

Ejercicios

1. Identificar la fase. Dadas 6 situaciones de proyecto (ej. "estoy probando XGBoost vs Random Forest", "estoy graficando histogramas para ver outliers"), asigná cada una a su fase CRISP-DM.
2. Business success criteria. Para un caso de detección de fraude bancario, escribí 3 criterios de éxito de negocio (no técnicos) y 3 técnicos. Distinguilos claramente.
3. Aplicar las 6 fases a un caso real. Elegí un problema (churn de Netflix, recomendación de productos, predicción de demanda en una panadería). Redactá un párrafo por cada fase con entregables concretos. Mínimo 1 iteración explícita (ej. "vuelvo de Modeling a Data Preparation porque...").
4. Comparar frameworks. Tabla de 3 columnas (CRISP-DM, TDSP, ML lifecycle moderno con MLOps) y 5 filas (origen, fases, foco, herramientas, cuándo usarlo).
5. Detectar iteración legítima vs retrabajo. Dados 4 escenarios de "estoy volviendo a una fase anterior", clasificalos como iteración esperada o como síntoma de mala planificación en la fase anterior.

Homework verificable

Documento Markdown (1–2 páginas) tomando un proyecto propio (real o sintético) y desarrollando las 6 fases de CRISP-DM con: (a) objetivo de negocio en una frase; (b) 2 business success criteria cuantificables; (c) descripción mínima de datos disponibles; (d) plan de data prep en bullets; (e) 2 algoritmos candidatos justificados; (f) cómo se evaluará contra los criterios de (b); (g) cómo se desplegaría y monitorearía.

Criterio de aceptación: Las 6 fases están presentes, los success criteria son cuantificables (con número y unidad), y la fase de Evaluation mapea explícitamente a los criterios de (b) — no solo a métricas técnicas.

Errores comunes

Síntoma	Causa y cómo arreglar
El modelo tiene gran accuracy pero el nego	Saltaste Business Understanding. Fix: volv
Tratar CRISP-DM como cascada (una fase des	Malentendido del framework. Fix: el spec o
Confundir métricas técnicas con success cr	Reportás F1=0.87 al gerente; el gerente no
Saltar Deployment ("ya tengo el notebook")	El proyecto muere en el laptop del DS. Fix
Hacer EDA infinito sin avanzar a Modeling	Parálisis en Data Understanding. Fix: time

Preguntas frecuentes

¿CRISP-DM o TDSP o ML lifecycle moderno?

Depende. CRISP-DM (1999, IBM/SPSS) es agnóstico de herramientas y sigue siendo el más enseñado — bueno como esqueleto mental y para proyectos chicos/PoC. TDSP (Microsoft, 2016) es más prescriptivo, define roles (data scientist, engineer, PM), templates de carpetas y artefactos Git — bueno para equipos. ML lifecycle moderno (con MLOps: feature stores, model registry, CI/CD de modelos, monitoring de drift) es lo

que se usa cuando el modelo está en producción seria. No son excluyentes: CRISP-DM como mapa conceptual + TDSP como estructura de equipo + MLOps como tooling de producción.

¿Tengo que pasar por las 6 fases en orden estricto?

No. Las flechas entre fases adyacentes son bidireccionales y hay un loop externo. Lo que sí es no negociable: no podés saltarte Business Understanding ni Evaluation.

¿Cuánto tiempo lleva cada fase?

Regla aproximada: 10% Business, 10% Data Understanding, 50–70% Data Preparation, 10% Modeling, 10% Evaluation, variable Deployment. Si Modeling te consume el 50%, algo está mal — probablemente saltaste prep.

¿CRISP-DM aplica a deep learning?

Sí, conceptualmente. Las fases son las mismas. Cambia el detalle en Data Preparation (tensores, augmentation), Modeling (arquitecturas, GPU) y Deployment (serving de modelos pesados, ONNX). Pero el esqueleto se sostiene.

¿Y si el negocio no sabe qué quiere?

Ese es exactamente el trabajo de Business Understanding — ayudar al negocio a articularlo. Si después de varias sesiones no hay objetivo claro, el proyecto no está listo para empezar. Documentalo y volvé cuando lo esté.

Referencias

- CRISP-DM 1.0 step-by-step data mining guide (Wirth & Hipp, 2000)
- Géron, cap. 2 End-to-End Machine Learning Project.
- Microsoft Team Data Science Process (TDSP)
- Google ML Lifecycle / MLOps levels

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 062 — Clase 062 — Clasificación binaria con MNIST

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3. Duración estimada: 60 min.

Objetivo

Que el alumno arme su primer clasificador binario "de verdad" sobre MNIST (¿este dígito es un 5 o no?), entrenando un SGDClassifier, evaluándolo con cross_val_score sobre StratifiedKFold, y entendiendo por qué la accuracy sola miente cuando las clases están desbalanceadas.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Cargar MNIST con `fetch_openml('mnist_784', as_frame=False)` y separar train/test respetando el split original (60k/10k).

2. Construir un target binario `y_train_5 = (y_train == 5)` y entrenar un `SGDClassifier` sobre él.
3. Predecir y obtener scores con `predict()` y `decision_function()`, entendiendo la diferencia entre clase y score continuo.
4. Validar con `cross_val_score` sobre `StratifiedKFold` y leer los 3 valores que devuelve.
5. Detectar el accuracy paradox: comparar contra un clasificador trivial "nunca-5" y ver que también saca ~90%.

Temas

#	Tema	Por qué importa
1	<code>fetch_openml('mnist_784')</code>	Dataset estándar de entrada a CV/ML; 70k i
2	Target binario 5 vs no-5	Forma canónica de empezar antes de meterse
3	<code>SGDClassifier(random_state=42)</code>	Lineal, escalable, online; el "hola mundo"
4	<code>cross_val_score</code> + <code>StratifiedKFold</code>	Mantiene la proporción de clases en cada f
5	Accuracy paradox	90% suena bien hasta que ves que Never5Cla
6	<code>predict</code> vs <code>decision_function</code> vs <code>predict_pr</code>	Score continuo es lo que después te deja m

Definiciones y características

MNIST

: Dataset de 70.000 imágenes de dígitos manuscritos (0-9), 28×28 píxeles en escala de grises, aplanadas a vectores de 784 features. Split convencional: 60k train + 10k test, ya barajado. Es el "hello world" de ML — chico, limpio, sin sorpresas.

SGDClassifier

: Clasificador lineal entrenado con Stochastic Gradient Descent. Procesa instancias de a una (o mini-batch), lo que lo hace ideal para datasets grandes y aprendizaje online. Sensible al `random_state` (fijarlo siempre para reproducibilidad).

`decision_function(X)`

: Devuelve el score crudo (distancia firmada al hiperplano de decisión), no la clase. `Score > 0` → clase positiva, `score < 0` → negativa. Es lo que vas a necesitar después para mover el umbral y construir curvas PR/ROC.

Accuracy

: $(TP + TN) / \text{total}$. Métrica obvia pero traicionera: si el 90% de tus instancias son de la clase negativa, predecir "siempre negativo" te da 90% sin haber aprendido nada.

Baseline trivial (dummy classifier)

: Modelo que predice siempre la clase mayoritaria (o aleatorio). Sirve de piso: si tu modelo no le gana al dummy, no aprendió. En 5-vs-no-5, el dummy "nunca-5" saca ~90.9% (porque solo ~9.1% de los dígitos son 5).

StratifiedKFold

: Variante de K-Fold que conserva la proporción de clases en cada fold. Imprescindible con desbalanceo: con K-Fold común podrías terminar con un fold sin un solo 5.

`cross_val_score(clf, X, y, cv=3, scoring='accuracy')`

: Entrena y evalúa `clf` en `cv` folds y devuelve un array de `cv` scores. Por default usa `StratifiedKFold` cuando la tarea es clasificación.

Dataset / recursos

MNIST vía fetch_openml (se cachea localmente en ~/scikit_learn_data/ después de la primera descarga):

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame=False, parser='auto')
X, y = mnist.data, mnist.target.astype(int)
X_train, X_test = X[:60000], X[60000:]
y_train, y_test = y[:60000], y[60000:]
y_train_5 = (y_train == 5)
y_test_5 = (y_test == 5)
```

Ejercicios

1. Cargar y explorar. Cargá MNIST, imprimí X.shape y y.shape, mostrá un dígito con matplotlib.imshow(X[0].reshape(28, 28), cmap='binary') y verificá que y[0] == 5.
2. Target binario + SGD. Construí y_train_5, entrená SGDClassifier(random_state=42) y predecí sobre X[0]. ¿Devuelve True?
3. decision_function. Sobre la misma instancia, llamá a decision_function([X[0]]) y compará el signo con el resultado de predict.
4. Cross-validation. Corré cross_val_score(sgd, X_train, y_train_5, cv=3, scoring='accuracy'). ¿Qué tres valores te da? ¿Cuál es el promedio?
5. Baseline trampa. Implementá un Never5Classifier (clase con fit que no hace nada y predict que devuelve np.zeros(len(X), dtype=bool)) y corré el mismo cross_val_score. Comparalo con el SGD: ¿la diferencia es la que esperabas?

Homework verificable

Notebook que: (a) carga MNIST y arma el target 5-vs-no-5; (b) entrena SGDClassifier(random_state=42) sobre el train completo; (c) reporta accuracy 3-fold con cross_val_score; (d) reporta accuracy 3-fold del Never5Classifier; (e) en una celda markdown, explica en 2-3 líneas por qué ambos modelos están "cerca" en accuracy y por qué eso no significa que sean equivalentes.

Criterio de aceptación: Accuracy del SGD ≥ 0.95 en los 3 folds, accuracy del dummy ≈ 0.909 en los 3 folds, y la celda markdown menciona explícitamente el desbalanceo (~9% de 5s) como causa del paradox.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"¡Mi modelo saca 90% de accuracy, está gen"	Trampa clásica con clases desbalanceadas.
fetch_openml tarda eternidades o falla	La primera vez descarga ~55MB y a veces el
y viene como strings ('5', '0', ...) y la	Por default OpenML devuelve labels como st
Resultados distintos cada vez que entrenás	SGDClassifier es estocástico. Fix: SGDClas
decision_function no existe / AttributeError	Algunos clasificadores (como RandomForest)

Preguntas frecuentes

¿Por qué arrancar con clasificación binaria si MNIST es multiclase?

Pedagógico: binario aísla los conceptos de score, umbral, precision/recall sin el ruido de "¿es 3 o 8?". Multiclase viene en la clase 059 (One-vs-Rest, One-vs-One).

¿SGDClassifier con loss='hinge' (default) es lo mismo que SVM?

Es un SVM lineal entrenado con SGD en vez de con el optimizador cuadrático de SVC. Misma frontera teórica, distinta forma de llegar. Para datasets grandes, SGD le gana en tiempo.

¿Por qué cv=3 y no cv=10?

Géron usa 3 porque MNIST es grande (60k×784) y 10 folds tarda. En datasets chicos, 5-10 es lo habitual. La regla: más folds = menos varianza en la estimación pero más cómputo.

¿predict_proba está disponible en SGDClassifier?

Solo si entrenás con loss='log_loss' (regresión logística) o loss='modified_huber'. Con el default 'hinge' no — usás decision_function.

¿Hace falta escalar las features de MNIST?

Para SGDClassifier sí, conviene (los píxeles 0-255 son grandes y SGD es sensible a escala). Géron en el cap. 3 lo omite para simplificar; en la práctica StandardScaler o dividir por 255 ayuda a la convergencia.

Referencias

- Géron, cap. 3 § "Training a Binary Classifier" y "Measuring Accuracy Using Cross-Validation".
- sklearn SGDClassifier
- sklearn fetch_openml
- sklearn StratifiedKFold

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 063 — Clase 063 — Métricas: confusion matrix, precision, recall, F1

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3. Duración estimada: 70 min.

Objetivo

Que el alumno deje de mirar accuracy como métrica única y aprenda a leer una confusion matrix, a elegir entre precision, recall, F1 o F-beta según el costo de los errores, y a interpretar classification_report clase por clase. En particular, entender por qué en problemas desbalanceados (fraude, churn, diagnóstico) accuracy miente y qué hacer al respecto.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Construir e interpretar una confusion matrix con sklearn.metrics.confusion_matrix identificando TP, FP, TN, FN.
2. Calcular y comparar precision, recall, F1 y F-beta a mano y con precision_score, recall_score, f1_score, fbeta_score.
3. Elegir la métrica adecuada según el costo asimétrico de los errores (FP vs FN) del problema.
4. Leer classification_report y diferenciar macro avg vs weighted avg en multiclase.
5. Diagnosticar class imbalance y aplicar class_weight='balanced', SMOTE o threshold tuning según

corresponda.

Temas

#	Tema	Por qué importa
1	Confusion matrix (TP/FP/TN/FN)	Base de toda métrica de clasificación.
2	Precision = $TP / (TP+FP)$	Cuán "puras" son las predicciones positiva
3	Recall = $TP / (TP+FN)$	Qué fracción de positivos reales capturo.
4	F1 y F-beta	Media armónica; F-beta pondera recall ($\beta > 1$)
5	classification_report y macro/weighted avg	Métricas por clase en multiclase.
6	Accuracy y por qué falla con clases desbal	Paradoja del 99% inútil.
7	Class imbalance: class_weight, SMOTE, thre	Cuándo aplicar cada uno.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 056a — Class imbalance: SMOTE, ADASYN, class_weight, threshold tuning

Definiciones y características

TP / FP / TN / FN

: True Positive: positivo real predicho positivo. False Positive: negativo real predicho positivo (error tipo I). True Negative: negativo real predicho negativo. False Negative: positivo real predicho negativo (error tipo II). Toda métrica binaria se deriva de estos cuatro.

Confusion matrix

: Tabla 2×2 (o k×k en multiclase) con counts de cada combinación (real, predicho). En sklearn las filas son la clase real y las columnas la predicha: [[TN, FP], [FN, TP]].

Precision = $TP / (TP + FP)$

: De todo lo que predije positivo, ¿qué fracción acerté? Importa cuando un FP es caro (ej.: bloquear una transacción legítima, marcar email legítimo como spam).

Recall (sensitivity, TPR) = $TP / (TP + FN)$

: De todos los positivos reales, ¿qué fracción detecté? Importa cuando un FN es caro (ej.: no detectar un tumor, dejar pasar un fraude).

F1 = $2 \cdot (P \cdot R) / (P + R)$

: Media armónica de precision y recall. Penaliza fuerte cuando una de las dos es baja. Default razonable cuando no hay preferencia clara entre FP y FN.

F-beta = $(1 + \beta^2) \cdot (P \cdot R) / (\beta^2 \cdot P + R)$

: F1 generalizada. $\beta > 1$ pondera más recall ($\beta=2 \rightarrow$ recall pesa 4× más que precision). $\beta < 1$ pondera más precision ($\beta=0.5$). Útil cuando el costo de FP y FN es asimétrico pero ambos importan.

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

: Fracción total de aciertos. Engañosa con clases desbalanceadas: con 99% negativos, predecir "todo negativo" da 99% accuracy con cero recall. Reserva para problemas balanceados.

class_weight

: Hiperparámetro de la mayoría de estimadores sklearn. 'balanced' calcula pesos inversamente

proporcionales a la frecuencia. También acepta dict {0: 1, 1: 10} para control manual. Modifica la función de pérdida, no el dataset.

Dataset / recursos

- MNIST 5-vs-not-5 (binarizado): clásico de Géron cap. 3. Imbalance ~10% positivo.
- Credit card fraud (Kaggle, opcional): ~0.17% positivo, ideal para ver SMOTE en acción.
- Sintético con `make_classification(weights=[0.99, 0.01])` para experimentar sin descargar.

Ejercicios

1. Confusion matrix a mano. Dado `y_true = [0,1,1,0,1,1,0,0,1,0]` y `y_pred = [0,1,0,0,1,1,1,0,1,0]`: calculá TP/FP/TN/FN, precision, recall y F1 con lápiz y papel. Verificá con `sklearn.metrics`.
2. MNIST 5-detector. Entrená `SGDClassifier` sobre MNIST binarizado (5 vs no-5). Mostrá confusion matrix con `ConfusionMatrixDisplay` y reportá precision, recall y F1.
3. `classification_report` multiclase. Sobre MNIST completo (10 clases) con `LogisticRegression`, imprimí `classification_report`. Identificá qué dígito tiene peor recall y por qué (mirá la confusion matrix).
4. Class imbalance con `class_weight`. Generá un dataset con `make_classification(n_samples=10000, weights=[0.99, 0.01], random_state=42)`. Entrená dos `LogisticRegression`: una sin `class_weight` y otra con `class_weight='balanced'`. Compará recall de la clase minoritaria.
5. SMOTE dentro de Pipeline. Mismo dataset que el ejercicio 4. Armá un `imblearn.pipeline.Pipeline` con SMOTE + `LogisticRegression`, evaluá con `cross_val_score(scoring='f1')` y compará contra `class_weight='balanced'`. Verificá que SMOTE solo se aplica al train fold (leelo en docs).

Homework verificable

Notebook sobre el dataset sintético desbalanceado (~1% positivo) que: (a) entrene baseline `LogisticRegression` y reporte confusion matrix + `classification_report`; (b) repita con `class_weight='balanced'`; (c) arme Pipeline con SMOTE; (d) haga threshold tuning con `precision_recall_curve` sobre val set maximizando F1; (e) tabla comparativa de F1 y recall de la clase minoritaria para las 4 estrategias (baseline, `class_weight`, SMOTE, threshold tuning).

Criterio de aceptación: las 4 estrategias documentadas, F1 de la clase minoritaria del mejor enfoque $\geq 2 \times$ el del baseline, y SMOTE aplicado solo dentro de Pipeline (chequear en código que no aparece `SMOTE().fit_resample(X, y)` sobre el dataset completo antes del split).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Accuracy 99% pero el modelo "no detecta na	Clases desbalanceadas + métrica equivocada
Aplicar <code>SMOTE().fit_resample(X, y)</code> antes d	Data leakage: vecinos sintéticos se constr
<code>precision_score</code> lanza <code>UndefinedMetricWarni</code>	No hubo predicciones positivas (TP+FP=0).
Confunden ejes de la confusion matrix	En sklearn las filas son la clase real y l
Usar F1 cuando el costo de FP y FN es muy	F1 trata ambos errores como equivalentes.

Preguntas frecuentes

¿SMOTE, `class_weight` o threshold tuning?

Empezá siempre por `class_weight='balanced'` — es gratis, no toca el dataset, no genera leakage. Si no alcanza, sumá threshold tuning (también gratis, solo cambia el corte sobre `predict_proba`). Recurrí a

SMOTE/ADASYN cuando los dos anteriores no rinden y sospechás que el modelo no separa bien la frontera de decisión. Ojo: SMOTE no es magia — en datasets tabulares ruidosos a veces empeora.

¿Precision o recall?

Depende del costo asimétrico: si un FN es caro (no detectar un cáncer, dejar pasar un fraude) → priorizá recall. Si un FP es caro (bloquear cliente legítimo, marcar email serio como spam) → priorizá precision. Cuando no hay preferencia clara, usá F1.

¿macro avg o weighted avg en multiclase?

macro avg = promedio simple por clase (todas pesan igual). Útil cuando te importan todas las clases por igual, incluyendo las raras. weighted avg = promedio ponderado por soporte (clases mayoritarias pesan más). Útil cuando refleja el costo real del negocio. En problemas desbalanceados macro avg es más honesto.

¿Por qué F1 es media armónica y no aritmética?

Porque la armónica penaliza fuerte los valores bajos. Si precision=1.0 y recall=0.0, la media aritmética da 0.5 (engañoso); la armónica da 0. F1 te obliga a que ambas sean razonables.

¿El umbral 0.5 es óptimo?

Casi nunca — es el default de predict() pero no tiene base teórica. Con precision_recall_curve sobre un set de validación encontrás el umbral que maximiza F1 (o F-beta, o la métrica de negocio). En problemas desbalanceados moverlo suele dar más mejora que cambiar de modelo.

Referencias

- Géron, cap. 3 — Performance Measures, Confusion Matrix, Precision and Recall, The Precision/Recall Trade-off.
- scikit-learn — Classification metrics
- imbalanced-learn — User guide
- imbalanced-learn — SMOTE
- Chawla et al. (2002), SMOTE: Synthetic Minority Over-sampling Technique, JAIR 16.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrido desde el laboratorio del programa o desde Jupyter.

Clase 064 — Clase 064 — Class imbalance: SMOTE, ADASYN, class_weight, threshold tuning

Parte: 1 — Machine Learning Clásico · Fuente: Chawla et al. (2002) SMOTE + He et al. (2008) ADASYN + imbalanced-learn docs. Duración estimada: 80 min.

Objetivo

Tratar datasets desbalanceados —fraude (1 % positivo), churn (5 %), enfermedades raras—. Las trampas son sutiles: accuracy puede ser 99 % con un clasificador trivial. Cubrir las 4 estrategias estándar: class_weight, threshold tuning, oversampling (SMOTE, ADASYN), undersampling (Tomek, ENN). Y la decisión clave: ¿qué métrica reportar? (F1, PR-AUC, MCC, no accuracy).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Detectar imbalance: `value_counts(normalize=True)`. Decidir si $> 10:1$ amerita tratamiento.
- Aplicar `class_weight='balanced'` o pesos custom en sklearn.
- Aplicar threshold tuning: optimizar el umbral de decisión sobre la curva PR según la métrica del negocio.
- Usar SMOTE (synthetic minority over-sampling) de imbalanced-learn: `SMOTE(k_neighbors=5).fit_resample(X, y)`.
- Combinar oversampling + undersampling (SMOTETomek, SMOTEENN).
- Reportar PR-AUC y MCC (Matthews Correlation Coefficient) en lugar de accuracy.

Temas

- Imbalance ratio: $>10:1$ problemático.
- Métricas: precision, recall, F1, F-beta, PR-AUC, MCC.
- `class_weight`: penalizar más errores en minoría durante training.
- Threshold tuning: mover el umbral fuera del 0.5 default.
- SMOTE: interpola entre vecinos de la minoría.
- ADASIN: como SMOTE pero con más densidad en zonas "difíciles".
- Tomek links / ENN: remueve borderline de la mayoría.
- imbalanced-learn pipelines.

Definiciones y características

- Class imbalance: una clase mucho más frecuente que otra(s).
- `class_weight='balanced'`: pesos automáticos inversamente proporcionales a frecuencia.
- SMOTE: para cada sample minoritario, crear sintéticos en línea recta a sus k vecinos.
- ADASIN: SMOTE adaptativo — genera más cerca de la frontera de decisión.
- PR-AUC: área bajo Precision-Recall curve. Más informativa que ROC-AUC cuando hay imbalance fuerte.
- MCC: $(TP - FP) / \sqrt{...}$. Único valor en $[-1, 1]$. Robusto a imbalance.
- Threshold tuning: elegir el cutoff $p > \tau \rightarrow \text{predict } 1$ que maximiza la métrica de negocio.

Dataset / recursos

- `fetch_openml('creditcardfraud')` (Kaggle, 0.17 % positivo).
- Librerías: imbalanced-learn (pip install imbalanced-learn), scikit-learn.

Ejercicios

1. Baseline sin tratamiento: LogisticRegression en creditcardfraud. Accuracy alto, recall pésimo.
2. `class_weight`: LogisticRegression(`class_weight='balanced'`). Recall sube, precision baja.
3. Threshold tuning: con probabilidades de predict_proba, barrer thresholds y plotear F1 vs threshold. Elegir el óptimo.
4. SMOTE: `from imblearn.over_sampling import SMOTE; X_res, y_res = SMOTE().fit_resample(X_train, y_train)`. Entrenar y evaluar.
5. Pipeline imblearn: `Pipeline([('smote', SMOTE()), ('clf', LogisticRegression())])`. Importante: SMOTE solo se aplica en train (imblearn pipeline lo maneja).

Homework verificable

Sobre creditcardfraud:

1. 4 modelos: baseline, `class_weight`, SMOTE, SMOTETomek.
2. Reportar precision, recall, F1, PR-AUC y MCC para cada uno.
3. Curva PR de los 4 lado a lado.

4. Threshold tuning sobre el mejor para maximizar F2 (favorece recall).

Criterio de aceptación: al menos uno de los tratamientos supera al baseline en F1 por ≥ 0.1 ; PR-AUC ≥ 0.8 .

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Accuracy 99.5 % en fraude con clasificador	Reporta accuracy en imbalance. Fix: usar P
SMOTE aplicado antes de split	Leakage: samples sintéticos derivados de t
Threshold default 0.5 con probabilidades c	Subóptimo. Fix: tuneear sobre val.
SMOTE con features categóricas codificadas	Interpola entre 0/1, sin sentido. Fix: SMO
Oversampling para clase de 0.1 % a 50/50	Excesivo. Fix: ratio 1:3 o 1:5 suele ser s

Preguntas frecuentes

class_weight o SMOTE?

class_weight es más simple y barato. SMOTE puede ayudar en casos extremos o con árboles/ensembles. Probá ambos.

¿Cuándo PR-AUC vs ROC-AUC?

PR-AUC cuando imbalance fuerte ($>10:1$) — más sensible. ROC-AUC para casos balanceados o solo para comparar relativamente.

¿MCC vs F1?

MCC es simétrico (trata clases igual). F1 favorece la minoría. Para fraude/medical, MCC es más conservador.

¿SMOTE con DL?

Menos común — DL prefiere ajustar la loss (focal loss, weighted CE).

¿Undersampling pierde información?

Sí. Por eso es un last resort. SMOTE/oversampling sintético suele ser mejor.

Referencias

- Chawla et al. (2002), SMOTE, JAIR.
- He et al. (2008), ADASYN, IJCNN.
- Saito & Rehmsmeier (2015), The Precision-Recall Plot Is More Informative than the ROC Plot, PLOS ONE.
- imbalanced-learn docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 065 — Clase 065 — Precision/Recall tradeoff

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3. Duración estimada: 50 min.

Objetivo

Que el alumno entienda que no se puede maximizar precision y recall al mismo tiempo: mover el threshold de decisión sube uno y baja el otro. La clase enseña a usar `decision_function` + `precision_recall_curve` para elegir el threshold según el costo del negocio, no según el default de 0.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar el tradeoff entre precision y recall en términos del threshold del clasificador.
2. Obtener scores crudos con `decision_function(X)` (o `predict_proba`) en vez de quedarse con `predict`.
3. Calcular la curva con `precision_recall_curve(y_true, scores)` y graficarla.
4. Elegir un threshold que cumpla una restricción del negocio (ej: $\text{precision} \geq 90\%$).
5. Reportar `average_precision_score` como métrica resumen única de la curva.

Temas

#	Tema	Por qué importa
1	Threshold de decisión: el default 0 no es	Define cuántos FP y FN tolerás.
2	<code>decision_function</code> vs <code>predict_proba</code> vs <code>pred</code>	El primero devuelve score crudo, el último
3	<code>precision_recall_curve</code>	Te da los 3 arrays: precision, recall, thr
4	Elegir threshold según restricción de nego	"Precision $\geq 90\%$ " o "Recall $\geq 95\%$ " — depen
5	<code>average_precision_score</code> (AP)	Resumen escalar del área bajo la curva PR.
6	Cuándo PR > ROC	Datasets muy desbalanceados (la próxima cl

Definiciones y características

`decision_function(X)`

: Devuelve el score crudo del clasificador (distancia al hiperplano en SGD/SVM). Valores > threshold → clase positiva. El default de predict es threshold = 0.

Threshold de decisión

: Punto de corte sobre el score. Subirlo → menos positivos predichos → más precision, menos recall. Bajarlo → más positivos predichos → más recall, menos precision.

`precision_recall_curve(y_true, scores)`

: Devuelve (precisions, recalls, thresholds) para todos los thresholds posibles. Notar que precisions y recalls tienen un elemento más que thresholds (el extremo donde recall=0).

`precision_score(y_true, y_pred)`

: $TP / (TP + FP)$. "De los que predije positivos, cuántos lo eran". Costo de FP alto → maximizar precision.

`recall_score(y_true, y_pred)`

: $TP / (TP + FN)$. "De los positivos reales, cuántos encontré". Costo de FN alto (cáncer, fraude) → maximizar recall.

`average_precision_score(y_true, scores)` (AP)

: Área bajo la curva precision-recall, calculada como promedio ponderado de precisions en cada threshold. Métrica resumen — mejor que F1 cuando hay desbalance fuerte.

`cross_val_predict(..., method='decision_function')`

: Variante de `cross_val_predict` que devuelve scores crudos en vez de labels. Imprescindible para construir la

curva PR sin leakage.

Dataset / recursos

MNIST (clasificador binario "es el 5 vs no") — mismo dataset que la clase 056. Permite reusar el SGDClassifier ya entrenado.

Ejercicios

1. Score crudo. Entrená SGDClassifier sobre MNIST binario "es 5". Para una imagen concreta, llamá `sgd.decision_function([X[0]])` y compará con `sgd.predict([X[0]])`. Mostrá que `predict` es `decision_function > 0`.
2. Curva PR. Con `cross_val_predict(sgd, X_train, y_train_5, cv=3, method='decision_function')` obtené `y_scores`. Pasalos a `precision_recall_curve` y graficá `precision` y `recall` vs `threshold` en el mismo eje.
3. Threshold para `precision ≥ 90%`. Encontrá el `threshold` mínimo que garantice `precision ≥ 0.90`. Pista: `thresholds[np.argmax(precisions >= 0.90)]`. Aplícalo: `y_pred_90 = (y_scores >= threshold_90)` y verificá `precision` y `recall` resultantes.
4. Curva `precision` vs `recall`. Graficá `precision` (eje Y) contra `recall` (eje X) — la forma canónica de la curva PR. Marcá el punto correspondiente al `threshold` por default (0).
5. Average `precision`. Calculá `average_precision_score(y_train_5, y_scores)`. Compará con el F1 que sacaste en la clase 056. ¿Cuál te parece más informativo?

Homework verificable

Notebook con MNIST binario (es 5 vs no): (a) entrenar SGDClassifier; (b) obtener `y_scores` con `cross_val_predict(method='decision_function')`; (c) graficar las dos curvas (`precision/recall` vs `threshold` y `precision` vs `recall`); (d) encontrar el `threshold` que da `precision ≥ 90%` y reportar el `recall` en ese punto; (e) reportar `average_precision_score`.

Criterio de aceptación: El `threshold` elegido para `precision ≥ 90%` verifica esa cota cuando se aplica sobre `y_scores`. Las dos curvas están graficadas con ejes etiquetados.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>precisions</code> y <code>thresholds</code> tienen distinto la	Es por diseño: <code>precision_recall_curve</code> devu
Usás <code>predict</code> y querés mover el <code>threshold</code>	<code>predict</code> ya colapsó el <code>score</code> a <code>label</code> . Fix:
Sacaste la curva con <code>y_pred</code> en vez de <code>y_sc</code>	La curva PR necesita <code>scores</code> continuos, no
<code>Threshold</code> elegido sobre <code>train</code> , no sobre <code>va</code>	Overfitting del <code>threshold</code> . Fix: elegí <code>thre</code>
Modelos sin <code>decision_function</code> (ej: RandomF	No todos los estimadores la exponen. Fix:

Preguntas frecuentes

¿`Precision` o `recall` — cuál priorizo?

Depende del costo asimétrico. Filtro de spam: un FP (mail bueno marcado como spam) cuesta más que un FN → priorizá `precision`. Detección de cáncer: un FN (cáncer no detectado) cuesta vidas → priorizá `recall`. No hay respuesta universal, hay análisis de costos.

¿`decision_function` o `predict_proba`?

Son intercambiables a los fines de la curva PR (ambas son `scores` monótonos). `predict_proba` devuelve probabilidades calibradas en `[0,1]` (más interpretable); `decision_function` devuelve `scores` sin acotar. Para

SGDClassifier usá `decision_function`; para RandomForestClassifier usá `predict_proba(X)[:, 1]`.

¿Por qué la curva PR tiene esa forma escalonada?

Cada salto corresponde a un sample que cambia de lado del threshold. Con N muestras hay hasta N thresholds distintos. En datasets chicos se nota más; con 10k+ samples se ve suave.

¿F1 o average precision?

F1 es a un threshold fijo (típicamente 0.5) — útil cuando el threshold ya está definido. Average precision integra sobre todos los thresholds — útil para comparar modelos sin fijar threshold. Para selección de modelo, AP es más informativo.

¿Y si la clase positiva es la minoría extrema (<1%)?

Ahí la curva PR brilla y la ROC engaña. Lo cierra la clase 058 (ROC y AUC).

Referencias

- Géron, cap. 3 § Precision/Recall Tradeoff y § The ROC Curve.
- scikit-learn — `precision_recall_curve`
- scikit-learn — `average_precision_score`
- scikit-learn — Precision-Recall example

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrílo desde el laboratorio del programa o desde Jupyter.

Clase 066 — Clase 066 — Curva ROC y AUC

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3 § The ROC Curve.

Duración estimada: 50 min.

Objetivo

Que el alumno entienda qué mide la curva ROC, calcule el AUC con scikit-learn y sepa decidir cuándo ROC es la métrica adecuada y cuándo conviene usar Precision-Recall — sobre todo en datasets desbalanceados, donde ROC tiende a mentir.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Construir la curva ROC con `roc_curve` graficando TPR vs FPR a distintos umbrales.
2. Calcular el AUC con `roc_auc_score` e interpretar el valor (0.5 = azar, 1.0 = perfecto).
3. Comparar dos clasificadores superponiendo sus curvas ROC y eligiendo por AUC.
4. Decidir ROC vs PR según la prevalencia de la clase positiva.
5. Evitar la trampa del desbalance: reconocer cuándo un AUC alto esconde mala precisión.

Temas

#	Tema	Por qué importa
1	TPR (recall) y FPR	Los dos ejes de ROC; entender qué se gana

2	Curva ROC con <code>roc_curve</code>	Visualiza el trade-off completo, no un pun
3	AUC con <code>roc_auc_score</code>	Resumen escalar e independiente del umbral
4	Diagonal de azar y modelo perfecto	Referencias visuales obligatorias.
5	ROC vs Precision-Recall	En clases desbalanceadas, ROC pinta lindo
6	Comparación de modelos	Cómo elegir entre clasificadores con AUC +

Definiciones y características

TPR (True Positive Rate / Recall / Sensibilidad)

: Proporción de positivos reales correctamente detectados. $TPR = TP / (TP + FN)$. Es el eje Y de ROC.

FPR (False Positive Rate)

: Proporción de negativos reales clasificados erróneamente como positivos. $FPR = FP / (FP + TN) = 1 - \text{especificidad}$. Es el eje X de ROC.

Curva ROC (Receiver Operating Characteristic)

: Gráfico de TPR vs FPR barriendo todos los umbrales de decisión. Cada punto es un umbral. Cuanto más cerca del vértice superior izquierdo, mejor.

AUC (Area Under the Curve)

: Área bajo la curva ROC. Probabilidad de que el modelo asigne mayor score a un positivo aleatorio que a un negativo aleatorio. $0.5 = \text{azar}$, $1.0 = \text{perfecto}$, $<0.5 = \text{peor que tirar moneda}$ (invertí las etiquetas).

Diagonal de azar

: Línea $y = x$. Representa un clasificador que adivina al azar. Toda curva ROC útil va por encima de esta diagonal.

Curva Precision-Recall (PR)

: Alternativa a ROC para clases desbalanceadas. Grafica precision vs recall. No incluye TN, por lo que es insensible al inflado por la clase mayoritaria.

ROC vs PR — regla práctica

: Usá ROC cuando las clases están balanceadas o te importan ambas por igual. Usá PR cuando la clase positiva es rara (fraude, enfermedad, clicks) y los falsos positivos en una clase abundante distorsionan el FPR.

Dataset / recursos

- `sklearn.datasets.fetch_openml('mnist_784')` con el clasificador binario "es un 5" (Géron cap. 3), naturalmente desbalanceado (~10% positivos).
- Opcional: dataset sintético desbalanceado vía `make_classification(weights=[0.99, 0.01])` para ver el contraste ROC vs PR en extremo.

Ejercicios

1. Scores y curva ROC. Entrená un `SGDClassifier` sobre "es un 5", obtené scores con `cross_val_predict(..., method='decision_function')` y graficá la curva ROC con `roc_curve`.
2. AUC. Calculá `roc_auc_score(y_true, y_scores)`. Interpretá el valor en una línea.
3. Comparar dos modelos. Entrená un `RandomForestClassifier` (usá `predict_proba`, columna 1) y superponé ambas curvas ROC en un mismo plot. Elegí el mejor por AUC.
4. ROC vs PR en desbalance. Generá `make_classification(weights=[0.99, 0.01], n_samples=10_000)`,

entrená un modelo decente y graficá lado a lado curva ROC y curva PR (`precision_recall_curve`). Mostrá cómo ROC sigue "linda" mientras PR revela la pobreza real.

5. Punto operativo. Sobre la curva ROC del ejercicio 1, encontrá el umbral que maximiza TPR - FPR (índice de Youden) y reportá precision y recall en ese punto.

Homework verificable

Notebook con dataset "es un 5" de MNIST: (a) entrenar SGD y RandomForest; (b) graficar ambas ROC superpuestas con AUC en la leyenda; (c) graficar las dos curvas PR con `average_precision_score` en la leyenda; (d) tabla final con AUC y AP por modelo; (e) párrafo de 3-4 líneas eligiendo el ganador y justificando con qué métrica decidiste y por qué (pista: prevalencia $\approx 10\%$).

Criterio de aceptación: El notebook corre end-to-end. Ambas curvas en un mismo eje. La conclusión menciona explícitamente el desbalance y por qué PR/AP puede ser preferible a ROC/AUC.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
AUC = 0.99 pero la precision real es 0.10	Clase muy desbalanceada — el FPR sigue baj
<code>roc_auc_score</code> lanza <code>ValueError: y should b</code>	Le pasaste <code>predict_proba</code> entero (matriz $N \times$
Curva ROC "escalonada" rara con <code>predict</code>	Estás pasando etiquetas duras (0/1) en lug
AUC < 0.5	El modelo está invertido o pasaste la colu
Comparo modelos solo por AUC y elijo mal	AUC promedia sobre todos los umbrales, inc

Preguntas frecuentes

¿ROC o Precision-Recall?

Si la clase positiva es rara (<10-20%) o te importa específicamente cómo te va con los positivos, usá PR. ROC incluye TN en el denominador del FPR y con muchos TN el FPR queda artificialmente bajo aunque la precision sea malísima. En clases balanceadas o cuando importan ambas clases por igual, ROC está bien.

¿Qué AUC se considera "bueno"?

Depende del dominio. Regla gruesa: 0.5 azar, 0.7 aceptable, 0.8 bueno, 0.9+ excelente, 1.0 sospechá data leakage. En medicina o fraude a veces 0.95 es lo mínimo aceptable.

¿Puedo usar AUC con multiclase?

Sí: `roc_auc_score(..., multi_class='ovr')` (one-vs-rest) o `'ovo'` (one-vs-one). Pero para multiclase suele ser más informativo mirar la matriz de confusión y métricas por clase.

¿`decision_function` o `predict_proba`?

Da igual para ROC/AUC — ROC depende del orden de los scores, no de su escala. `decision_function` devuelve scores no calibrados (puede ser negativo), `predict_proba` devuelve probabilidades en $[0, 1]$. Si tu modelo no tiene `predict_proba` (como `SGDClassifier` default), usá `decision_function`.

¿Cómo elijo el umbral final si AUC es independiente del umbral?

AUC sirve para comparar modelos. Para deployar tenés que elegir umbral según el costo de FP vs FN en tu dominio. Métodos: índice de Youden ($\max(\text{TPR} - \text{FPR})$), restricción tipo "recall ≥ 0.95 y maximizá precision", o análisis de costo explícito.

Referencias

- Géron, cap. 3 § The ROC Curve y § Precision/Recall Trade-off.
- scikit-learn — roc_curve
- scikit-learn — roc_auc_score
- scikit-learn — Precision-Recall vs ROC
- Saito & Rehmsmeier (2015), The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 067 — Clase 067 — Clasificación multiclase, multilabel, multioutput

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3. Duración estimada: 60 min.

Objetivo

Que el alumno distinga los tres escenarios de clasificación más allá del binario — multiclase (una salida con $K > 2$ clases), multilabel (varias etiquetas por muestra) y multioutput (varias salidas, cada una con su propio rango de valores) — y sepa qué estrategia de sklearn (OneVsRest, OneVsOne, MultiOutputClassifier) usar en cada caso, eligiendo además la métrica correcta (accuracy global, hamming loss, macro/micro F1).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Diferenciar multiclase, multilabel y multioutput con un ejemplo concreto de cada uno.
2. Elegir entre OvR y OvO según el costo computacional del clasificador base y el tamaño del dataset.
3. Entrenar un clasificador multilabel con KNeighborsClassifier y evaluarlo con `f1_score(average='macro')` y `hamming_loss`.
4. Envolver un clasificador binario en MultiOutputClassifier para resolver un problema multioutput.
5. Interpretar las salidas de `predict_proba` en cada escenario (lista de arrays vs array 2D).

Temas

#	Tema	Por qué importa
1	Multiclase: definición y clasificadores na	Decision Tree, Random Forest, Naive Bayes,
2	OneVsRestClassifier (OvR)	K modelos binarios — escala lineal en K, d
3	OneVsOneClassifier (OvO)	$K \cdot (K-1)/2$ modelos — caro en clases, barato
4	Multilabel: y es matriz binaria K-dim	Tags de noticias, géneros de película, mul
5	Métricas multilabel: hamming loss, macro/m	Accuracy global castiga demasiado; necesit
6	Multioutput: MultiOutputClassifier / Multi	Cada salida es independiente, con sus prop

Definiciones y características

Clasificación multiclase

: Una sola etiqueta por muestra pero con $K > 2$ valores posibles (ej.: dígitos 0–9). y es vector 1D de enteros. Algunos clasificadores la soportan nativamente (RandomForest, GaussianNB, SGDClassifier); otros (SVM, Logistic puro) son binarios y sklearn los envuelve automáticamente con OvR u OvO.

One-vs-Rest (OvR / OvA)

: Estrategia que entrena K clasificadores binarios, cada uno "esta clase vs todas las demás". Predice la clase del clasificador con mayor score. Default para la mayoría — lineal en K, ideal cuando entrenar es barato pero el clasificador base no escala con N.

One-vs-One (OvO)

: Entrena $K \cdot (K-1)/2$ clasificadores binarios, uno por cada par de clases. Cada uno ve solo las muestras de sus dos clases (subsets chicos). Conviene cuando el clasificador base escala mal con N (ej.: SVM kernelizado, $O(N^2)$); sklearn lo usa por default para SVC.

Clasificación multilabel

: Cada muestra puede pertenecer a varias clases simultáneamente. y es matriz binaria ($n_samples, n_labels$) con 0/1 por etiqueta. Ejemplo: una foto puede tener ['perro', 'aire libre', 'soleado'] a la vez. KNeighborsClassifier, RandomForestClassifier y DecisionTreeClassifier lo soportan nativamente.

Clasificación multioutput

: Generalización: cada muestra tiene varias salidas y cada salida es a su vez multiclase (no binaria como en multilabel). Ejemplo típico de Géron: denoising de imagen — cada píxel es una "salida" con 256 valores posibles. Se resuelve con MultiOutputClassifier(base_estimator).

Hamming loss

: Fracción promedio de etiquetas mal predichas sobre el total ($n_samples \times n_labels$). Métrica natural para multilabel — un error en una de 5 etiquetas pesa 0.2, no 1.0 como subset accuracy. Cuanto más bajo, mejor.

Macro vs micro averaging

: Macro = promedio simple de la métrica calculada por clase (cada clase pesa igual; bueno con desbalance si querés tratar todas las clases por igual). Micro = suma global de TP/FP/FN y luego cálculo (cada muestra pesa igual; bueno cuando importa el volumen total).

MultiOutputClassifier

: Wrapper de sklearn que entrena un clasificador independiente por columna de salida. No modela correlaciones entre salidas (para eso existe ClassifierChain). Trivial: MultiOutputClassifier(RandomForestClassifier()).fit(X, Y).

Dataset / recursos

MNIST (multiclase 10 clases) vía fetch_openml('mnist_784', as_frame=False). Para multilabel/multioutput se construye Y sintético sobre MNIST: etiquetas [es_grande (≥ 7), es_impar] → multilabel; o X_noisy → X_clean → multioutput.

Ejercicios

1. Multiclase nativo vs OvR. Entrená SGDClassifier en MNIST (10 clases) y comparalo con OneVsRestClassifier(SGDClassifier()). ¿Cuántos clasificadores entrena cada uno? Mirá .estimators_.
2. OvO con SVM. Entrená SVC() sobre un subset de 5k muestras de MNIST. Verificá que clf.decision_function(X[:1]) devuelve 45 scores = $10 \cdot 9/2$. Forzá luego OneVsRestClassifier(SVC()) y compará tiempos.
3. Multilabel con KNN. Construí $Y_multilabel = np.c_[y \geq 7, y \% 2 == 1]$. Entrená KNeighborsClassifier() con ese Y. Reportá $f1_score(Y_test, Y_pred, average='macro')$ y $hamming_loss(Y_test, Y_pred)$.
4. Macro vs micro F1. Con el modelo del ejercicio 3, calculá F1 con $average='macro'$ y $average='micro'$. Si

una de las etiquetas estuviera muy desbalanceada (ej.: solo 5% de positivos), ¿cuál cambiaría más y por qué?

5. Multioutput denoising. Genera $X_{\text{train_noisy}} = X_{\text{train}} + \text{np.random.randint}(0, 100, X_{\text{train}}.\text{shape})$. Entrena `KNeighborsClassifier()` con X_{noisy} como features y X_{clean} como target (cada píxel es una salida multiclase 0–255). Predecí y visualizá una imagen denoised.

Homework verificable

Notebook sobre MNIST con: (a) clasificador multiclase con `RandomForestClassifier` y matriz de confusión 10×10; (b) versión multilabel con etiquetas [es_grande, es_impar] usando `KNeighborsClassifier` y reporte de `hamming_loss + f1_score(average='macro')`; (c) ejemplo multioutput de denoising sobre 100 imágenes con ruido uniforme, mostrando 3 pares (ruidosa, denoised, original).

Criterio de aceptación: F1 macro multilabel > 0.95 sobre test. Hamming loss < 0.05. Denoising visualmente reconocible (no hace falta métrica numérica, sí imagen comparativa lado a lado).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ValueError: y should be a 1d array, got ..	El clasificador no soporta multilabel nati
Accuracy multilabel sale carísima (1.0 es	accuracy_score en multilabel exige que tod
SVC tarda eternidades en MNIST completo	SVC con kernel es $O(N^2)$ – $O(N^3)$. Sklearn ya
predict_proba de un multilabel devuelve li	Es lo esperado: una matriz (n_samples, n_c
Macro F1 da muy distinto al micro F1	Indica desbalance fuerte entre clases/etiq

Preguntas frecuentes

¿Cuándo OvR y cuándo OvO?

Default: OvR (lineal en K, simple). Usá OvO solo si el clasificador base escala mal con N (SVM kernelizado es el caso típico) y K es chico. Sklearn ya elige bien por default — rara vez hace falta override manual.

Multilabel vs multioutput, ¿en qué se diferencian exactamente?

Multilabel = caso particular de multioutput donde cada salida es binaria (0/1 = "tiene/no tiene esa etiqueta"). Multioutput general = cada salida puede ser multiclase (ej.: 256 niveles de gris por píxel). Sklearn los trata casi igual; la diferencia es solo el rango de valores que puede tomar cada columna de Y.

¿MultiOutputClassifier modela correlaciones entre las salidas?

No — entrena un modelo independiente por columna. Si las etiquetas están correlacionadas (ej.: "es perro" y "tiene cola"), `ClassifierChain` aprende esa estructura: pasa la predicción de la primera etiqueta como feature extra al modelo de la siguiente.

¿Por qué `cross_val_score` con `SGDClassifier` en MNIST da accuracy alta pero la matriz de confusión muestra confusión entre 3 y 5?

Accuracy global enmascara errores localizados entre pocas clases. La matriz de confusión normalizada por filas es indispensable en multiclase — la clase 060 (la próxima) se dedica exactamente a eso.

¿Puedo combinar `class_weight` / SMOTE con multilabel?

`class_weight='balanced'` sí, etiqueta por etiqueta. SMOTE multilabel es más delicado (existe `MLSMOTE` en `imbalanced-learn`, no en `sklearn core`). Para el curso: usá `class_weight` y métricas macro.

Referencias

- Géron, cap. 3 § Multiclass / Multilabel / Multioutput Classification.
- sklearn — Multiclass and multioutput algorithms
- sklearn — MultiOutputClassifier
- sklearn — Métricas de clasificación (hamming_loss, f1_score)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 068 — Clase 068 — Análisis de errores

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 3 § Error Analysis. Duración estimada: 60 min.

Objetivo

Que el alumno deje de mirar el accuracy global y empiece a auditar dónde se equivoca un clasificador: confusion matrix normalizada por fila, pares de clases confundidas, inspección visual de ejemplos mal clasificados, y el loop de error analysis como puerta de entrada al data-centric AI (mejorar datos, no solo modelos).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Construir y normalizar una confusion matrix por fila (recall por clase) y leerla sin confundir filas con columnas.
2. Identificar pares de clases confundidas ordenando los off-diagonals normalizados de mayor a menor.
3. Inspeccionar visualmente ejemplos mal clasificados (hard examples) para formular hipótesis de causa raíz.
4. Decidir si la próxima iteración mejora el modelo (features, regularización, capacidad) o mejora los datos (relabel, augmentación, balancear).
5. Ejecutar el error analysis loop: entrenar → matriz → slices → hipótesis → fix → re-entrenar.

Temas

#	Tema	Por qué importa
1	Confusion matrix cruda vs normalizada por	Sin normalizar, las clases mayoritarias ta
2	Off-diagonals: qué clase se confunde con c	El error no es uniforme; suele haber 2-3 p
3	Inspección visual de hard examples	Te dice si es ruido de label, ambigüedad r
4	Slice analysis (error por subgrupo)	Accuracy global oculta sesgos por subpobla
5	Data-centric AI: cuándo arreglar datos	Más barato y efectivo que tunear hiperpará
6	Error analysis loop	Workflow iterativo y reproducible.

Definiciones y características

Confusion matrix normalizada por fila

: Matriz donde $C[i,j] = P(\text{predicho}=j \mid \text{real}=i)$. La diagonal es el recall por clase; los off-diagonals son la

distribución de errores condicional al label real. Siempre dividí por la suma de fila, no por el total: lo que querés ver es "del total de los i verdaderos, qué fracción cayó en j ".

Error rate por clase

: 1 - recall _{i} . Útil para rankear clases por dificultad. Una clase con 60% recall en un modelo de 95% accuracy global es un problema invisible al accuracy.

Hard examples

: Instancias mal clasificadas con alta confianza, o cerca del borde de decisión. Inspeccionarlas a mano (50-100 alcanza) revela el 80% de las causas raíz: labels equivocados, imágenes borrosas, ambigüedad ontológica, dominio fuera de distribución.

Slice analysis

: Computar métricas no en el set completo sino en subconjuntos definidos por una variable (edad, región, tipo de cámara, longitud del texto). Detecta sesgos que el promedio entierra.

Data-centric AI

: Paradigma (Andrew Ng) que pone el foco en mejorar la calidad y consistencia de los datos en vez de iterar modelos. Mucho del error analysis es la herramienta operacional del data-centric.

Error analysis loop

: Ciclo train → confusion matrix → top-k confusiones → inspeccionar ejemplos → hipótesis (modelo/dato/feature) → intervenir → repetir. Convierte el debugging de ML de arte a proceso.

Top-k confusiones

: Los k pares (i, j) con $i \neq j$ ordenados por $C_norm[i, j]$ descendente. Son la lista priorizada de qué atacar primero.

Dataset / recursos

MNIST (sklearn.datasets.fetch_openml('mnist_784') o keras.datasets.mnist). Es el ejemplo canónico de Géron cap. 3: 10 clases, errores no uniformes (los $4 \leftrightarrow 9$, $3 \leftrightarrow 5$, $7 \leftrightarrow 9$ son los pares clásicos que aparecen en cualquier modelo decente). Alternativa más densa: Fashion-MNIST (shirt vs t-shirt vs pullover es un caos pedagógicamente útil).

Ejercicios

1. Matriz cruda y normalizada. Entrená un SGDClassifier sobre MNIST. Calculá confusion_matrix(y_true, y_pred) y normalizá por fila (cm / cm.sum(axis=1, keepdims=True)). Plotealá con plt.matshow y poné ceros en la diagonal para que los errores se vean.

2. Top-5 confusiones. Del array normalizado, extraé los 5 pares (i, j) con mayor valor off-diagonal. Imprimí "real= i → pred= j : XX%".

3. Galería de errores. Para el par (real, pred) peor del ejercicio 2, mostrá una grilla 5×5 de imágenes mal clasificadas. Anotá si te parecen ambiguas, mal labeladas o claramente del label real.

4. Slice por grosor de trazo. Calculá la suma de píxeles por imagen como proxy de "grosor". Dividí el test set en terciles y reportá accuracy por tercil. ¿El modelo es peor con dígitos finos o gruesos?

5. Intervención data-centric. Tomá el par confundido del ejercicio 2. Augmentá el set de entrenamiento solo con esa clase (shifts de 1px) y re-entrená. Reportá el cambio en el recall de esa clase y el accuracy global.

Homework verificable

Notebook con MNIST que entregue: (a) confusion matrix normalizada por fila como heatmap; (b) tabla con los 3 pares de clases más confundidos y su porcentaje; (c) grilla de 16 ejemplos mal clasificados del par peor, con título `real=X pred=Y`; (d) tabla de accuracy por slice según una variable derivada (intensidad media, posición del centro de masa, lo que elijas); (e) un párrafo de hipótesis: ¿el error es de modelo o de datos? ¿qué probarías en la siguiente iteración?

Criterio de aceptación: Las filas de la matriz normalizada suman 1. El top-3 de confusiones está justificado numéricamente. La galería muestra ejemplos reales del par identificado. La hipótesis distingue explícitamente "mejoro modelo" vs "mejoro datos".

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
La matriz "se ve toda igual" / no se disti	No normalizaste, o no pusiste ceros en la
Normalicé por columna y los números no cie	Normalización por columna te da precision
Hago error analysis sobre el train set	Te miente: el modelo memorizó. Fix: siempr
"Tuneo hiperparámetros 3 días y no mejora"	Probablemente el techo es la calidad del l
Reporto solo accuracy global	Esconde clases minoritarias y subgrupos. F

Preguntas frecuentes

¿Cuándo mejoro el modelo y cuándo mejoro los datos?

Heurística operativa: si los ejemplos mal clasificados te confunden a vos también (ambiguos, mal labelados, fuera de dominio) → datos (relabel, limpiar, aumentar, recolectar más de esa clase). Si los errores son obvios para un humano pero el modelo los falla sistemáticamente → modelo (más capacidad, mejores features, menos regularización). En la práctica, el 70% de las veces son datos; por eso Ng popularizó el data-centric.

¿Por fila o por columna se normaliza?

Por fila (`axis=1`) para análisis de errores estándar: te da $P(\text{pred} | \text{real})$ = distribución de errores condicional a la verdad. Por columna sirve para diagnosticar precision (cuando el modelo dice j, ¿qué tan seguido es realmente j?). Géron usa por fila en cap. 3.

¿Cuántos ejemplos mal clasificados hay que mirar?

50 a 100 suele alcanzar para ver patrones. Más allá tenés rendimientos decrecientes. Lo importante es que estén estratificados por el par de confusión que te interesa, no muestreados al azar.

¿`cross_val_predict` o un split fijo?

`cross_val_predict` te da predicciones out-of-fold para todo el train set sin leakage, ideal para análisis de errores con N moderado. Para producción usá un test set holdout fijo.

¿Esto reemplaza el ROC/PR curve?

No. Las curvas ROC/PR son para threshold tuning y comparación de modelos a nivel agregado. El error analysis es para debugging cualitativo y priorización. Son complementarias.

Referencias

- Géron, Hands-On ML, cap. 3 § "Error Analysis".
- Andrew Ng — A Chat with Andrew on MLOps: From Model-centric to Data-centric AI.
- scikit-learn — `confusion_matrix` y `ConfusionMatrixDisplay`.

- scikit-learn — cross_val_predict.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 069 — Clase 069 — Regresión lineal: ecuación normal vs gradient descent

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4 § Linear Regression. Duración estimada: 60 min.

Objetivo

Que el alumno entienda regresión lineal desde adentro: la hipótesis $\hat{y} = \theta^T x$, por qué se usa MSE como costo, las dos formas de resolverla (ecuación normal cerrada vs gradient descent iterativo), y cuándo conviene cada una según el tamaño del dataset y la cantidad de features.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Escribir la hipótesis lineal $\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$ en forma matricial $X\theta$.
2. Derivar la ecuación normal $\theta = (X^T X)^{-1} X^T y$ y resolverla con NumPy.
3. Usar LinearRegression de sklearn y entender que internamente usa pseudoinversa SVD (más estable que la ecuación normal).
4. Comparar complejidad: ecuación normal $O(n^3)$ en features vs gradient descent $O(n)$ por iteración.
5. Justificar la elección entre forma cerrada y GD según n features y m muestras.

Temas

#	Tema	Por qué importa
1	Hipótesis lineal y notación vectorial	Base de todo modelo lineal (incluso logist
2	MSE como función de costo	Convexa, derivable, mínimo global garantiz
3	Ecuación normal (forma cerrada)	Solución exacta en un solo paso.
4	Pseudoinversa SVD	Lo que sklearn usa por debajo — funciona a
5	Gradient descent (intuición)	Cuando n es grande, la ecuación normal n
6	Complejidad computacional	$O(n^3)$ vs $O(n)$ por iteración — define

Definiciones y características

Hipótesis lineal

: Predicción $\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x$ (agregando $x_0 = 1$ para absorber el bias). En forma matricial sobre todo el dataset: $\hat{y} = X\theta$, donde X tiene shape $(m, n+1)$.

MSE (Mean Squared Error)

: Costo $\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$. Se elige porque es convexa (un único mínimo global), diferenciable en todo el dominio y penaliza más errores grandes. RMSE es

la raíz, en unidades del target.

Ecuación normal

: Solución cerrada que sale de igualar el gradiente del MSE a cero: $\hat{\theta} = (X^T X)^{-1} X^T y$. Da el óptimo exacto sin iteraciones. Requiere invertir una matriz $(n+1) \times (n+1)$.

Pseudoinversa de Moore-Penrose (X^+)

: Generalización de la inversa que siempre existe, incluso si $X^T X$ es singular (features colineales) o no cuadrada. Se computa vía SVD: $X = U \Sigma V^T \rightarrow X^+ = V \Sigma^+ U^T$. Sklearn usa `np.linalg.lstsq` (basado en SVD) — más estable numéricamente que invertir $X^T X$.

Complejidad $O(n^3)$

: Invertir $X^T X$ cuesta entre $O(n^{2.4})$ y $O(n^3)$ según el algoritmo. Con $n = 100$ features va instantáneo; con $n = 100,000$ es inviable. La SVD es del mismo orden. En m (muestras) la ecuación normal es lineal, así que escala bien en filas, mal en columnas.

Gradient descent (intuición)

: Algoritmo iterativo: arrancás con θ random y vas restando $\eta \cdot \nabla_{\theta} \text{MSE}(\theta)$ hasta converger. Cada paso cuesta $O(mn)$ (batch) — mucho menos que invertir cuando n es grande. La tasa de aprendizaje η es el hiperparámetro crítico.

LinearRegression (sklearn)

: Estimador en `sklearn.linear_model`. No usa ecuación normal — usa `scipy.linalg.lstsq` (pseudoinversa SVD). Sin hiperparámetros relevantes salvo `fit_intercept`. Atributos post-fit: `coef_` (los $\theta_1 \dots \theta_n$) e `intercept_` (θ_0).

`fit_intercept=True`

: Agrega automáticamente la columna de unos. Si tus features ya están centradas en cero ($\text{mean}=0$), podés ponerlo en `False`. Default `True` — dejalo así salvo que sepas lo que hacés.

`coef_`

: Array con los pesos aprendidos, una entrada por feature. Su signo y magnitud son interpretables solo si las features están en la misma escala (de ahí la importancia de `StandardScaler` previo).

Dataset / recursos

Dataset sintético generado con NumPy: $y = 4 + 3x + \text{ruido gaussiano}$, $m = 100$ muestras. Permite comparar el θ recuperado con el verdadero $[4, 3]$.

Ejercicios

1. Hipótesis a mano. Generá X sintético ($m=100$, $n=1$) con $y = 4 + 3x + \text{mathcal{N}}(0, 1)$. Resolvé $\hat{\theta}$ con la ecuación normal usando solo NumPy (`np.linalg.inv`, `@`). Verificá que $\hat{\theta} \approx [4, 3]$.
2. Pseudoinversa. Repetí el ejercicio 1 con `np.linalg.pinv(X_b) @ y`. Compará el resultado con la ecuación normal — deberían dar lo mismo en este caso bien-condicionado.
3. sklearn. Ajustá `LinearRegression` al mismo dataset. Verificá que `lin_reg.intercept_  $\approx 4$  y lin_reg.coef_  $\approx [3]$ . Predecí en  $x = [[0], [2]]$ .`
4. Caso singular. Construí X con dos features colineales ($x_2 = 2 x_1$). Intentá la ecuación normal — `np.linalg.inv` tira `LinAlgError` o devuelve basura. Usá `np.linalg.pinv` y observá que sí funciona (distribuye el peso entre las dos features).

5. Complejidad empírica. Cronométrá `LinearRegression().fit(X, y)` con $n = 100, 1000, 10000$ features (y m fijo). Graficá el tiempo — debería crecer cúbicamente.

Homework verificable

Notebook con dataset California Housing (`sklearn.datasets.fetch_california_housing`): (a) split 80/20 con `random_state=42`; (b) ajustar `LinearRegression`; (c) reportar R^2 y RMSE en test; (d) imprimir `coef_` mapeado a nombres de feature; (e) resolver manualmente la ecuación normal en los datos escalados y verificar que da los mismos coeficientes que sklearn (módulo escalado del intercept).

Criterio de aceptación: RMSE en test < 0.75 (en unidades de la variable target). Los coeficientes manuales coinciden con sklearn con tolerancia $1e-6$.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>LinAlgError: Singular matrix</code> al hacer <code>np.linalg.pinv(X)</code>	Features colineales o $m < n$. Fix: usá <code>np.linalg.pinv(X, rcond=1e-10)</code>
<code>coef_</code> enormes y de signo contrario a lo es	Features no escaladas y multicolinealidad.
Olvido de la columna de unos en la ecuación	Resolvés sin intercept y el modelo pasa por el origen
<code>LinearRegression</code> da resultado distinto al	Olvidaste agregar la columna de unos en <code>X</code>
"Mi modelo lineal tarda horas con 50k feat"	Ecuación normal $O(n^3)$ no escala. Fix: c

Preguntas frecuentes

¿Sklearn usa la ecuación normal por dentro?

No. Usa `scipy.linalg.lstsq`, que internamente factoriza con SVD (pseudoinversa). Es más estable numéricamente que $(X^T X)^{-1} X^T y$ y funciona aunque la matriz sea singular o rectangular.

¿Por qué MSE y no MAE como costo?

MSE es diferenciable en todo el dominio (MAE no lo es en cero) y convexa, así que el mínimo es único y se llega con gradient descent sin sustos. MAE es más robusto a outliers pero requiere métodos no-suaves (programación lineal o subgradiente).

¿Cuándo elijo ecuación normal vs gradient descent?

Regla práctica: $n < \sim 10,000$ features $\rightarrow$ ecuación normal/SVD (un shot, sin tuneear hiperparámetros). $n \gg 10,000$ o no entra en RAM $\rightarrow$ gradient descent (típicamente SGD). En m (muestras) ambos escalan bien, así que millones de filas con pocas columnas $\rightarrow$ ecuación normal sin drama.

¿Necesito escalar features para regresión lineal?

Para que el modelo funcione, no — la ecuación normal da exactamente la misma predicción con o sin escalado. Para interpretar `coef_` (comparar importancia entre features), sí. Para gradient descent, sí o sí (sin escalar la convergencia es lentísima).

¿Qué pasa si tengo más features que muestras ($n > m$)?

$X^T X$ es singular $\rightarrow$ la ecuación normal falla. La pseudoinversa SVD devuelve la solución de norma mínima entre las infinitas posibles, pero el modelo va a sobreajustar feísimo. Fix real: regularización (Ridge/Lasso, clase 063) o reducción de dimensionalidad.

Referencias

- Géron, cap. 4 § Linear Regression, The Normal Equation, Computational Complexity.

- sklearn LinearRegression
- NumPy linalg.pinv
- Wikipedia — Moore-Penrose pseudoinverse

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 070 — Clase 070 — Gradient Descent: batch, stochastic, mini-batch

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 60 min.

Objetivo

Que el alumno entienda gradient descent como motor de optimización para entrenar modelos lineales cuando la ecuación normal no escala, y sepa elegir entre batch (BGD), stochastic (SGD) y mini-batch GD según tamaño del dataset, ruido tolerable y costo por iteración. Que además dimensione el rol del learning rate y del feature scaling, y use SGDRegressor de scikit-learn.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar el gradiente de la MSE en regresión lineal y por qué moverse en - minimiza el costo.
2. Diferenciar BGD vs SGD vs mini-batch en términos de costo por iteración, varianza del paso y memoria.
3. Diagnosticar la curva de costo (divergente, oscilante, lenta, suave) e inferir si el learning_rate está mal seteado.
4. Aplicar feature scaling (StandardScaler) antes de cualquier GD y justificar por qué sin escalado SGD diverge o tarda 100×.
5. Entrenar SGDRegressor con learning_rate='invscaling' y comparar coeficientes contra LinearRegression (ecuación normal).

Temas

#	Tema	Por qué importa
1	Gradiente de la MSE y regla de update $\theta :=$	Es el núcleo de todo ML moderno (incluye r
2	Batch GD: usa todo el dataset por step	Convergencia suave, pero $O(m)$ por iteració
3	Stochastic GD: 1 muestra por step	Muy rápido y escala, pero ruidoso; nunca c
4	Mini-batch GD: lotes de 32–256	Equilibrio práctico, aprovecha vectorizaci
5	Learning rate η y learning schedule	Demasiado alto diverge, demasiado bajo no
6	Feature scaling como prerequisite	Sin escalar, las curvas de nivel son elips
7	SGDRegressor de sklearn	API estándar, soporta partial_fit (online

Definiciones y características

Gradiente $MSE(\theta)$

: Vector de derivadas parciales del costo respecto a cada parámetro. Apunta en la dirección de máximo crecimiento del error $\rightarrow$ restarlo ($-\eta \cdot$) reduce el costo. Para MSE: $= (2/m) \cdot X^T(X\theta - y)$.

Learning rate η (eta)

: Tamaño del paso. Hiperparámetro crítico. Muy alto: el costo diverge o oscila. Muy bajo: tarda eternidades. Valores típicos: 0.001 a 0.1.

Batch Gradient Descent (BGD)

: Cada update usa todas las m muestras. Costo por iteración $O(m \cdot n)$. Determinista, converge suave al mínimo global (en problemas convexos como MSE). Inviabile con $m > 10^6$.

Stochastic Gradient Descent (SGD)

: Cada update usa una sola muestra (elegida al azar). Costo $O(n)$ por iteración. Camino ruidoso, "rebota" cerca del mínimo sin asentarse $\rightarrow$ requiere `learning_schedule` decreciente para que el ruido baje con el tiempo.

Mini-batch Gradient Descent

: Usa lotes de tamaño b (típicamente 32, 64, 128, 256). Combina lo mejor: vectorizable (rápido en GPU), menos ruidoso que SGD, mucho más liviano que BGD. Es el estándar de facto en deep learning.

Epoch

: Una pasada completa sobre el dataset. En SGD, una epoch = m updates. En mini-batch con `batch_size` b , una epoch = m/b updates.

Learning schedule

: Función que baja η con el tiempo. Patrón común: $\eta_t = \eta / (1 + \text{decay} \cdot t)$. En sklearn: `learning_rate='invscaling'` con `eta0` y `power_t`.

Feature scaling

: Llevar las features a escala comparable (StandardScaler: media 0, std 1). Sin esto, una feature con valores $[0, 10^6]$ domina el gradiente y otra con $[0, 1]$ es invisible $\rightarrow$ GD zigzaguea por un valle alargado.

Dataset / recursos

`sklearn.datasets.fetch_california_housing` (20.640 filas, 8 features, escalas muy distintas $\rightarrow$ caso ideal para mostrar la necesidad de scaling).

Ejercicios

1. BGD a mano. Implementá BGD en NumPy para regresión lineal sobre un dataset sintético ($y = 4 + 3x + \text{ruido}$). Loopeá 1000 iteraciones con $\eta=0.1$. Graficá la trayectoria de θ , θ y la curva de costo.
2. SGD a mano. Mismo dataset. Implementá SGD con `learning_schedule` $\eta_t = 5 / (50 + t)$. Compará la trayectoria contra BGD: tendría que ser visiblemente más ruidosa pero más rápida en wall-clock.
3. Efecto del learning rate. Corré BGD con $\eta \in \{0.001, 0.01, 0.1, 0.5, 1.0\}$. Graficá las 5 curvas de costo en un mismo plot. Identificá cuál diverge y cuál es absurdamente lenta.
4. Scaling sí/no. Sobre California Housing, entrená SGDRegressor (a) sin escalar y (b) con StandardScaler. Reportá `n_iter_` y score en cada caso. Tiene que haber un orden de magnitud de diferencia.
5. SGDRegressor vs LinearRegression. Entrená ambos sobre California Housing escalado. Compará coeficientes y R^2 . Tienen que dar muy parecidos (SGD es aproximación estocástica de la solución cerrada).

Homework verificable

Notebook con California Housing: (a) train/test split 80/20; (b) pipeline StandardScaler + SGDRegressor(`max_iter=1000`, `tol=1e-3`, `learning_rate='invscaling'`, `eta0=0.01`); (c) reportá R^2 en test y

`n_iter_` real; (d) repetí sin scaler y mostraré que `n_iter_` se dispara o que R^2 cae; (e) graficé la curva de loss vs epoch usando `partial_fit` en loop manual.

Criterio de aceptación: R^2 test ≥ 0.55 con scaler. Sin scaler, R^2 baja al menos 0.1 puntos o aparece `ConvergenceWarning`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>ConvergenceWarning: Maximum number of iter</code>	Olvidaste el <code>StandardScaler</code> . Fix: meté SGD
El costo diverge (sube en lugar de bajar)	<code>learning_rate</code> demasiado alto. Fix: bajalo
SGD nunca "se asienta", el costo oscila pa	Falta <code>learning_schedule</code> que baje η con el
Coefficientes de <code>SGDRegressor</code> distintos a L	Si entrenaste con datos no escalados, los
<code>partial_fit</code> parece arrancar de cero en cad	Si reinstanciás el modelo dentro del loop,

Preguntas frecuentes

¿BGD, SGD o mini-batch?

Mini-batch en el 90% de los casos. BGD solo si el dataset entra cómodo en RAM y $m < \sim 10^4$. SGD puro (`batch_size=1`) casi nunca en la práctica — sirve para entender el concepto y para online learning donde llegan muestras una por una. `SGDRegressor` de sklearn internamente puede comportarse como mini-batch dependiendo del solver.

¿Por qué `LinearRegression` no usa GD?

Porque para MSE existe solución cerrada (ecuación normal: $\theta = (X^T X)^{-1} X^T y$). Es exacta y no tiene hiperparámetros. Pero invertir $X^T X$ es $O(n^3)$ en features y $O(m \cdot n^2)$ en muestras $\rightarrow$ con muchas features ($n > 10^4$) o $X^T X$ mal condicionada, GD gana.

¿Cómo elijo η ?

Empezá con 0.01. Si diverge, dividilo por 10. Si converge pero lento, multiplícalo por 3. Mejor aún: grid search sobre `eta0` con cross-validation. En deep learning hay schedulers más sofisticados (Adam, cosine annealing), pero para ML clásico invscaling alcanza.

¿Qué tamaño de mini-batch uso?

32, 64, 128 o 256. Potencias de 2 porque la GPU las prefiere. Más chico $\rightarrow$ más ruido, más updates por epoch. Más grande $\rightarrow$ más estable, pero menos pasos. 32 es el default histórico (Bengio); 256 es común con datasets grandes.

¿`StandardScaler` o `MinMaxScaler` antes de GD?

`StandardScaler` por default — centra en 0 (importante para que el gradiente no tenga sesgo direccional) y normaliza varianza. `MinMaxScaler` lo usás si necesitás un rango acotado (ej. imágenes a $[0, 1]$).

Referencias

- Géron, cap. 4 — Training Models, sección "Gradient Descent".
- scikit-learn — `SGDRegressor`
- scikit-learn — Stochastic Gradient Descent user guide
- Sebastian Ruder — An overview of gradient descent optimization algorithms

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 071 — Clase 071 — Regresión polinomial

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4 § Polynomial Regression. Duración estimada: 50 min.

Objetivo

Que el alumno ajuste modelos lineales a relaciones no lineales usando PolynomialFeatures de scikit-learn, entienda la combinatoria de features que esto genera, y reconozca el riesgo de overfitting cuando el grado crece.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Transformar features con PolynomialFeatures(degree=d) y entender qué columnas produce.
2. Ajustar un LinearRegression sobre features polinómicas y graficar la curva resultante.
3. Calcular cuántas features genera grado d con n variables originales (combinatoria con repetición).
4. Diagnosticar overfitting comparando RMSE en train vs test al subir el grado.
5. Decidir cuándo usar interaction_only=True vs incluir potencias.

Temas

#	Tema	Por qué importa
1	Modelo lineal sobre features no lineales	Linealidad es en los coeficientes, no en l
2	PolynomialFeatures(degree, include_bias, i	API para expandir el feature space.
3	Combinatoria de features: C(n+d, d)	Explosión cuadrática/cúbica con n features
4	Overfitting con grado alto	Curva oscila para pasar por todos los punt
5	Validación con train/test split	Sin split, grado alto siempre "gana" en tr
6	Interaction-only vs full polynomial	Cuándo importan las potencias y cuándo sol

Definiciones y características

PolynomialFeatures

: Transformer de sklearn.preprocessing que genera todas las combinaciones polinómicas de las features de entrada hasta grado degree. Para x con grado 2: [1, x, x²]. Para [x, x] grado 2: [1, x, x, x², xx, x²].

degree

: Grado máximo del polinomio. Grado 1 no transforma (más bias). Grado 2–3 cubre la mayoría de curvaturas suaves. Grado ≥10 casi siempre es overfit.

interaction_only=True

: Genera solo productos cruzados entre features distintas, sin potencias (x², x² quedan fuera). Útil cuando el efecto no lineal viene de interacciones, no de curvatura individual.

include_bias=True (default)

: Agrega columna de 1's. Conviene ponerlo en False si después usás LinearRegression (que ya tiene

`fit_intercept=True`) para no duplicar.

Explosión combinatoria

: La cantidad de features generadas es $C(n+d, d) = (n+d)! / (n! d!)$. Con $n=10$ y $d=5$: 3003 features. Con $n=100$ y $d=3$: 176851. Escala feo.

Generalización

: Capacidad del modelo de funcionar en datos no vistos. Subir grado mejora train pero llega un punto donde test empeora — es el síntoma clásico de overfitting.

Modelo lineal sobre features no lineales

: Ajustar $y = w + wx + wx^2$ es resolver un problema lineal en (w, w, w) aunque la curva en x sea una parábola. Por eso `LinearRegression` basta tras transformar.

Dataset / recursos

Sintético: $y = 0.5 x^2 + x + 2 + \text{ruido_gaussiano}$, con $x \in [-3, 3]$, $n=100$. Ideal para visualizar curva ajustada y comparar grados.

Ejercicios

1. Generar dataset cuadrático. $x = \text{np.linspace}(-3, 3, 100) + \text{ruido}$. Graficar scatter.
2. Ajustar grado 2. `PolynomialFeatures(degree=2, include_bias=False) + LinearRegression`. Imprimir `coef_` e `intercept_` y compararlos con los del DGP (0.5, 1, 2).
3. Grafo de curvas. Ajustar grados 1, 2, 5, 30 sobre el mismo dataset y plotear las 4 curvas superpuestas al scatter. Observar oscilaciones en grado 30.
4. Curva train/test vs grado. Para grado 1 a 20, calcular RMSE en train y en test. Graficar ambas curvas en función del grado. Identificar el punto donde test empieza a subir.
5. Combinatoria. Con $n_features=3$ de entrada, contar columnas que devuelve `PolynomialFeatures(degree=4, include_bias=False)`. Verificar con la fórmula $C(n+d, d) - 1$.

Homework verificable

Notebook con: (a) dataset sintético $y = \sin(x) + \text{ruido}$ en $x \in [0, 2\pi]$; (b) ajustar polinomios de grado 1 a 15; (c) split 80/20; (d) graficar RMSE train vs test por grado; (e) reportar el grado óptimo según test; (f) graficar curva del grado óptimo encima del scatter.

Criterio de aceptación: El grado óptimo reportado está entre 3 y 7 (zona razonable para sin con ruido). Curva train decrece monótona; curva test tiene forma de U.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>LinearRegression</code> da <code>coef_</code> con 2 columnas d	Dejaste <code>include_bias=True</code> y <code>LinearRegression</code>
RMSE train baja pero test explota al subir	Overfitting clásico. Fix: bajá el grado, o
<code>MemoryError</code> con grado 10 y 50 features	Combinatoria: $C(60, 10) \approx 75M$. Fix: reducí
Coefficientes enormes ($1e8$) con grado alto	Síntoma de overfitting + colinealidad entr
Olvidaste transformar el test set	Aplicaste <code>.fit_transform</code> en train y <code>.fit_t</code>

Preguntas frecuentes

¿PolynomialFeatures es un modelo no lineal?

No. Es una transformación del input. El modelo (LinearRegression) sigue siendo lineal en los coeficientes. Lo que es no lineal es la relación entre x original e y .

¿Qué grado elegir por default?

Empezá con 2. Si el residual muestra patrón curvo, subí a 3. Más allá de 4–5 rara vez es necesario en problemas reales — si lo necesitás, probablemente quieras un modelo no lineal (árboles, kernel) en vez de subir grado.

¿Cuándo `interaction_only=True`?

Cuando sabés (o sospechás) que el efecto no lineal viene de combinaciones entre features (ej: precio $\times$ cantidad), no de curvatura individual de cada una. Reduce muchísimo la cantidad de columnas.

¿Por qué siempre escalar antes de PolynomialFeatures?

Porque x^2 y x^3 agrandan rangos: si $x \in [0, 1000]$, entonces $x^3 \in [0, 1e9]$. Eso degrada el condicionamiento numérico y mata cualquier regularización posterior. Escalá primero con `StandardScaler`.

¿Cómo se relaciona esto con la clase 064?

Polinomial es el ejemplo canónico de modelo con alto bias en grado bajo y alta varianza en grado alto. La 064 formaliza ese trade-off con curvas de aprendizaje.

Referencias

- Géron, cap. 4 § Polynomial Regression y § Learning Curves.
- sklearn PolynomialFeatures
- sklearn user guide § Polynomial regression

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 072 — Clase 072 — Curvas de aprendizaje y bias-variance tradeoff

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 60 min.

Objetivo

Diagnosticar si un modelo sufre de alto sesgo o alta varianza leyendo curvas de aprendizaje (`sklearn.model_selection.learning_curve`) y decidir, con criterio, si conviene conseguir más datos, aumentar la capacidad del modelo o regularizar.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

1. Graficar una curva de aprendizaje (RMSE train vs. RMSE validación en función de `train_size`) con `learning_curve`.
2. Identificar patrones canónicos: underfitting (curvas altas y juntas) vs. overfitting (gap persistente).
3. Descomponer conceptualmente el error esperado en $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.
4. Decidir acción correctiva apropiada: más datos, más capacidad, features nuevas, o regularización.

5. Justificar por qué "más datos" no siempre es la solución (caso de high bias).

Temas

- Curva de aprendizaje: qué se plotea y cómo se lee.
- Patrón de underfitting: ambas curvas convergen alto → más datos no ayuda.
- Patrón de overfitting: gap grande train/val → más datos sí ayuda, o regularizar.
- Descomposición bias-variance del error de generalización.
- Error irreducible (ruido de Bayes): cota inferior inevitable.
- Tradeoff: aumentar capacidad ↓ bias pero ↑ variance.
- Diagnóstico operacional con `learning_curve` y `validation_curve`.

Definiciones y características

- Learning curve (curva de aprendizaje) — gráfico del error de entrenamiento y validación en función del tamaño del conjunto de entrenamiento. Permite diagnosticar capacidad vs. datos.
- Bias (sesgo) — error por supuestos erróneos del modelo (p. ej., asumir lineal lo no-lineal). Modelos con alto sesgo subajustan.
- Variance (varianza) — sensibilidad del modelo a pequeñas variaciones en los datos de entrenamiento. Modelos con alta varianza sobreajustan.
- Irreducible error — ruido intrínseco de los datos ($\text{Var}(\epsilon)$); ninguna mejora de modelo lo elimina. Es la cota inferior.
- Diagnóstico high-bias — `train_error` alto y `val_error` alto, ambas curvas convergen a un valor alto cuando `m` crece. Síntoma: más datos no mueven la aguja.
- Diagnóstico high-variance — `train_error` bajo, `val_error` notablemente más alto, gap persistente. Síntoma: el modelo memoriza; más datos o regularización deberían cerrar el gap.
- Bias-variance tradeoff — $E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. Reducir uno suele aumentar el otro; el óptimo está en el medio.
- Capacidad del modelo — grados de libertad efectivos (grado del polinomio, profundidad del árbol, nº de parámetros). Más capacidad menos bias, más variance.

Dataset / recursos

- Dataset sintético tipo "noisy quadratic": $y = 0.5 \cdot x^2 + x + 2 + \epsilon$ con `np.random.randn`. Permite controlar la complejidad.
- `sklearn.datasets.fetch_california_housing(subset)` para una corrida sobre datos reales.
- API clave: `sklearn.model_selection.learning_curve`, `validation_curve`.

Ejercicios

- Curva base. Genera 200 puntos del dataset cuadrático ruidoso. Ajusta una regresión lineal y grafica la curva de aprendizaje.
- Aumentar capacidad. Repetí con `PolynomialFeatures(degree=2) + LinearRegression`. Comparalo con `degree=10`. Identificá cuál es el patrón de ajuste.
- ¿Más datos ayudan? Para el polinomio de grado 10, extendé el dataset a 2000 puntos y volvé a plotear. ¿Se cierra el gap?
- Validation curve. Usá `validation_curve` para barrer `degree` de 1 a 15 sobre el mismo dataset. Encontrá el "sweet spot".

1. Bias-variance empírico. Entrená 100 modelos $\text{degree}=10$ sobre bootstraps del dataset y calculá, para una grilla

de x de test

$\text{bias}^2 + \text{var}$ se acerca al MSE total menos σ^2 .

Homework verificable

Sobre el dataset `california_housing`:

1. Entrená `DecisionTreeRegressor(max_depth=d)` para $d \in \{2, 5, 10, \text{None}\}$.
2. Para cada uno, generá la curva de aprendizaje con `learning_curve(cv=5, scoring='neg_root_mean_squared_error')`.
3. Clasificá cada modelo como underfit, fit razonable o overfit justificando con el gap final y el nivel

de error.

1. Recomendá explícitamente, para cada caso, una de tres acciones: ["más datos", "más capacidad", "regularizar/podar"].

Criterio de aceptación: un .py o notebook que, al ejecutarse, imprima una tabla con columnas

`depth | train_rmse_final | val_rmse_final | gap | diagnostico | accion_recomendada` y guarde los 4 gráficos en `figs/learning_curve_depth_{d}.png`.

Errores comunes

- Confundir curva de aprendizaje con validation curve. La primera varía m (tamaño de train); la segunda varía un hiperparám
- Leer el error de train absoluto como diagnóstico. Lo relevante es el gap y la tendencia asintótica, no un punto ai
- Recomendar "más datos" frente a high bias. Si ambas curvas ya convergieron alto, sumar filas no baja el error; hay que ca
- No promediar sobre CV. Una sola partición es ruidosa; `learning_curve` ya hace CV interno — usalo.
- Olvidar el irreducible error. Apuntar a $\text{val_rmse} = 0$ es absurdo si σ del ruido es positivo; siempre hay piso.

Preguntas frecuentes

- ¿Más datos o modelo más complejo? Mirá el gap. Gap grande con train bajo overfit más datos o regularización ayudan. Ga
- ¿Cuántos puntos uso para `train_sizes`? Por defecto `np.linspace(0.1, 1.0, 10)` está bien. Para datasets grandes, usá menos
- ¿Por qué el `train_error` sube al aumentar m ? Con pocos datos el modelo memoriza ($\text{train_error} \approx 0$); al sumar filas le cues
- ¿Sirve para clasificación? Sí. Usá `scoring='accuracy'` o `'neg_log_loss'`; la interpretación del gap es análoga.
- ¿Bias-variance se mide en la práctica? Conceptualmente sí (vía bootstrap, como el ejercicio 5), pero en producció

Referencias

- Géron, A. Hands-On Machine Learning, 3ª ed., cap. 4 — "Learning Curves" y cap. 7 — "Bias/Variance Tradeoff".
- scikit-learn user guide: Validation curves: plotting scores to evaluate models.
- API: `sklearn.model_selection.learning_curve`.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 073 — Clase 073 — Regularización: Ridge, Lasso, Elastic Net

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 70 min.

Objetivo

Aprender a controlar el overfitting en modelos lineales mediante regularización L2 (Ridge), L1 (Lasso) y su combinación (Elastic Net), entendiendo el rol del hiperparámetro alpha, la importancia del scaling, y cuándo conviene cada variante.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar qué es la regularización y por qué reduce la varianza de un modelo lineal.
- Implementar Ridge, Lasso y ElasticNet de scikit-learn sobre un dataset escalado.
- Tunear alpha con RidgeCV / LassoCV / ElasticNetCV y leer los coeficientes resultantes.
- Justificar la elección entre L1, L2 y L1+L2 según el problema (multicolinealidad, sparsity, número de features).
- Diagnosticar por qué Lasso "anula" features y Ridge solo las "encoge".

Temas

- Sesgo–varianza y motivación de la regularización.
- Ridge (L2): penalización $\alpha \cdot \sum \beta^2$. Encoge coeficientes hacia cero sin anularlos.
- Lasso (L1): penalización $\alpha \cdot \sum |\beta|$. Produce soluciones sparse (selección de features).
- Elastic Net: combinación L1+L2 con `l1_ratio`.
- Hiperparámetro alpha: efecto en bias/varianza; $\alpha=0 \equiv \text{OLS}$; $\alpha \rightarrow \infty \equiv \text{modelo nulo}$.
- Scaling obligatorio (StandardScaler) antes de regularizar.
- Selección de α con CV: RidgeCV, LassoCV, ElasticNetCV.

Definiciones y características

- Ridge (regresión L2): añade $\alpha \cdot \sum \beta^2$ a la función de costo. Tiene solución cerrada. No anula coeficientes; los acerca a cero proporcionalmente. Robusto frente a multicolinealidad.
- Lasso (regresión L1): añade $\alpha \cdot \sum |\beta|$. No tiene solución cerrada (se resuelve con coordinate descent). Produce soluciones esparsas: muchos coeficientes quedan exactamente en 0 → hace selección automática de features.
- Elastic Net: combina L1 y L2: $\alpha \cdot (l1_ratio \cdot \sum |\beta| + (1 - l1_ratio)/2 \cdot \sum \beta^2)$. Útil cuando hay más features que muestras o features muy correlacionadas (Lasso puro "elige una al azar" del grupo correlacionado).
- alpha (α): fuerza de la regularización. Más alto → más penalización → coeficientes más chicos → más bias, menos varianza. Se tunea por CV.

- L1 vs L2: L1 promueve sparsity (esquinas del rombo $|\beta|$); L2 promueve coeficientes pequeños y suaves (círculo β^2). L1 hace selección, L2 no.
- Sparsity: propiedad de un vector de coeficientes con muchos ceros. Útil para interpretabilidad y costo computacional en predicción.
- RidgeCV / LassoCV / ElasticNetCV: variantes que internamente buscan alpha (y l1_ratio) por cross-validation sobre una grilla. Más eficiente que GridSearchCV para estos modelos.
- Scaling: como la penalización suma cuadrados/absolutos de coeficientes, una feature con escala grande recibe penalización injustamente alta. Siempre escalar antes.

Dataset / recursos

- Dataset sugerido: `sklearn.datasets.fetch_california_housing` (regresión continua, 8 features con escalas distintas → ideal para mostrar scaling).
- Alternativa sintética: `make_regression(n_features=50, n_informative=10, noise=10)` para ver cómo Lasso anula las 40 no informativas.
- Stack: `numpy`, `pandas`, `scikit-learn` (Ridge, Lasso, ElasticNet, RidgeCV, LassoCV, ElasticNetCV, StandardScaler, Pipeline).

Ejercicios

1. OLS baseline: entrená `LinearRegression` sobre California Housing escalado. Reportá RMSE en train y test. Anotá los coeficientes.
2. Ridge: entrená `Ridge(alpha=1.0)` sobre los mismos datos. Compará RMSE y la magnitud de los coeficientes vs OLS.
3. Lasso: entrená `Lasso(alpha=0.1)`. Contá cuántos coeficientes quedaron exactamente en 0. Subí alpha a 1.0 y a 10.0; observá la sparsity creciente.
4. Elastic Net: entrená `ElasticNet(alpha=0.1, l1_ratio=0.5)`. Compará con Ridge y Lasso puros.
5. Tuning con CV: usá `RidgeCV(alphas=np.logspace(-3, 3, 50))` y `LassoCV(cv=5)` para encontrar el mejor alpha. Graficá el path de coeficientes vs alpha (Lasso path).

Homework verificable

Sobre `make_regression(n_samples=200, n_features=50, n_informative=10, noise=10, random_state=42)`:

1. Entrená OLS, Ridge, Lasso y Elastic Net (todos con `StandardScaler` en Pipeline).
2. Tuneá alpha con la variante *CV correspondiente.
3. Reportá: RMSE en test (split 80/20, `random_state=42`), número de coeficientes $\neq 0$, y top-5 features por `|coef|` en Lasso.

Criterio de verificación: Lasso debe identificar al menos 8 de las 10 features informativas (`coef  $\neq$  0`) y dejar en 0 al menos 30 de las 40 ruidosas. RMSE de Ridge y Elastic Net dentro de $\pm 10\%$ del de Lasso.

Errores comunes

1. No escalar features antes de Ridge/Lasso/Elastic Net: la penalización castiga a las features de mayor escala. Siempre `StandardScaler` (o equivalente) en un Pipeline.
2. Usar `alpha=0`: equivale a OLS y además dispara warnings/inestabilidad numérica en Lasso. Si querés OLS, usá `LinearRegression`.
3. Confundir alpha de `sklearn` con λ de la teoría: en `sklearn` alpha ya incluye el factor $1/(2n)$ o $1/n$ según el modelo. No es directamente comparable entre Ridge y Lasso.
4. Tinear alpha sobre el test set: usá CV (`RidgeCV`, `LassoCV`) o `GridSearchCV` sobre train. El test se toca una sola vez al final.
5. Esperar que Ridge haga selección de features: no las anula, solo las encoge. Si querés sparsity, usá

Lasso o Elastic Net.

Preguntas frecuentes

1. ¿Ridge, Lasso o Elastic Net?
 - Ridge: pocas features, todas potencialmente relevantes, posible multicolinealidad.
 - Lasso: muchas features, sospechás que muchas son irrelevantes, querés un modelo interpretable.
 - Elastic Net: muchas features correlacionadas entre sí, o $p > n$ (más features que muestras).
1. ¿Por qué Lasso anula coeficientes y Ridge no?
1. ¿Qué pasa si alpha es muy grande?
1. ¿Cómo elijo el rango de alpha para buscar?
1. ¿Sirve regularización con árboles o solo con modelos lineales?

Geométrica
restricción
sobre un eje

Todos los
Underfitting

Una grilla lo

Ridge/Lasso
regularizaci

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3.^a ed.), cap. 4 — sección "Regularized Linear Models".
- scikit-learn — Linear Models: Ridge, Lasso, Elastic Net.
- Hastie, Tibshirani, Friedman — The Elements of Statistical Learning, cap. 3.4 (Shrinkage Methods).
- Zou & Hastie (2005), "Regularization and variable selection via the elastic net".

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 074 — Clase 074 — Early stopping

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 45 min.

Objetivo

Aplicar early stopping como técnica de regularización implícita en entrenamientos iterativos: monitorear la pérdida de validación durante el descenso por gradiente y detener el ajuste cuando deja de mejorar, conservando el mejor modelo visto.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar por qué detener el entrenamiento antes del óptimo de train actúa como regularización.
- Configurar SGDRegressor(early_stopping=True) con validation_fraction, n_iter_no_change y tol.
- Graficar curvas de train loss vs. validation loss e identificar la best epoch.
- Implementar manualmente un loop con paciencia y snapshot del mejor modelo (partial_fit).
- Decidir cuándo early stopping reemplaza o complementa a Ridge/Lasso.

Temas

- Sobreajuste en entrenamientos iterativos (SGD, gradient boosting, redes neuronales).
- Curva de aprendizaje por época: train baja, validación baja y luego sube.
- Mecánica del early stopping: monitorear val loss + criterio de paciencia.
- API de scikit-learn: SGDRegressor / SGDClassifier con early_stopping=True.
- Implementación manual con partial_fit + copy.deepcopy del mejor estimador.
- Relación con otras regularizaciones (L1, L2, dropout en deep learning).

Definiciones y características

- Early stopping: detener el entrenamiento iterativo cuando una métrica de validación deja de mejorar durante un número fijo de iteraciones, devolviendo los parámetros del mejor punto observado.
- Validation loss: error medido sobre un split separado del de entrenamiento; es la señal que decide cuándo cortar. No debe usarse el test set para esta decisión.
- Patience (paciencia) / n_iter_no_change: cantidad de épocas consecutivas sin mejora superior a tol que se toleran antes de detener. Valores típicos: 5–20.
- Best epoch snapshot: copia de los parámetros (coef_, intercept_) en la época con menor val loss. Sin este snapshot, al cortar nos quedaríamos con un modelo ya degradado.
- tol: umbral mínimo de mejora para considerar que hubo progreso. Si loss_actual > loss_best - tol, la época no cuenta como mejora.
- Regularización implícita: a diferencia de Ridge/Lasso (que penalizan la magnitud de los pesos en la función objetivo), early stopping limita cuánto se ajustan los pesos, controlando capacidad efectiva.

Dataset / recursos

- sklearn.datasets.fetch_california_housing para regresión.
- sklearn.datasets.make_regression(n_samples=2000, n_features=50, noise=20) para experimentos controlados.
- Géron, Hands-On ML (3ª ed.), cap. 4 § "Early Stopping" (figura de la "U" en val loss).

Ejercicios

1. Curva clásica: entrenar SGDRegressor(max_iter=1, warm_start=True, learning_rate='constant', eta0=0.0005) por 500 épocas sobre California Housing escalado. Graficar RMSE de train y validación por época. Marcar la best epoch.
2. Early stopping automático: comparar SGDRegressor(early_stopping=True, validation_fraction=0.2, n_iter_no_change=10, tol=1e-4) contra el modelo sin early stopping. Reportar n.º de épocas reales (n_iter_) y RMSE en test.
3. Paciencia: barrer n_iter_no_change {1, 5, 20, 100} y mostrar cómo afecta la época final y el error de test.
4. Implementación manual: escribir un loop con partial_fit que mantenga best_loss, best_model = deepcopy(sgd) y un contador de paciencia. Devolver el mejor modelo.
5. Comparación con Ridge: sobre make_regression con ruido, comparar (a) SGD sin regularización + early stopping, (b) Ridge con alpha tuneado por CV. Discutir cuál generaliza mejor y por qué.

Homework verificable

Sobre California Housing (split 60/20/20 train/val/test, features escaladas con StandardScaler):

1. Entrenar un SGDRegressor con early_stopping=True, validation_fraction=0.2, n_iter_no_change=15, tol=1e-4, random_state=42.
2. Guardar la curva de loss_curve reconstruida con partial_fit (paralela, sin early stopping, 1000 épocas)

en `curva_val.png`.

3. Reportar: épocas usadas por el modelo con early stopping (`n_iter_`), RMSE en test, y época óptima vista en la curva manual.

Criterio de aceptación: $\text{RMSE}_{\text{test}} \leq 0.75$ (en variable target sin escalar, en cientos de miles de USD), y la época con early stopping debe estar dentro de $\pm 10\%$ de la best epoch detectada manualmente.

Errores comunes

- Monitorear train loss en vez de val loss: train siempre baja; nunca dispara el corte. Hay que usar un split de validación interno.
- No guardar el snapshot del mejor modelo: si cortás en la época k después de patience épocas malas, los pesos finales no son los mejores. SGDRegressor lo hace por dentro; en implementación manual hay que hacerlo explícito.
- Confundir validación con test: usar el test set para decidir el corte filtra información y arruina la estimación de generalización. El test se toca una sola vez al final.
- Patience demasiado baja: con ruido en val loss, `n_iter_no_change=1` corta prematuro por fluctuaciones aleatorias. Subir a 5–20.
- Olvidar escalar features: SGD es muy sensible a la escala; sin StandardScaler el descenso oscila y la curva de validación no muestra la "U" clara.

Preguntas frecuentes

- ¿Early stopping reemplaza a Ridge/Lasso? No siempre. En modelos lineales con muchos features correlacionados, Ridge suele dar mejor sesgo-varianza. Early stopping brilla en modelos iterativos costosos (boosting, redes) donde tunear alpha por CV es caro.
- ¿Sirve para modelos cerrados como LinearRegression o Ridge con `solver='cholesky'`? No: esos resuelven en un paso, no hay "épocas" que detener. Aplica a algoritmos iterativos: SGD, gradient boosting (`n_estimators` con `early_stopping_rounds`), redes neuronales.
- ¿Cuánto debería ser `validation_fraction`? 10–20 % suele alcanzar. Con datasets chicos (<1000 filas) usar CV externa en lugar de un split interno fijo.
- ¿`n_iter_no_change=5` es estándar? Es el default de sklearn y razonable. Subilo si la val loss es ruidosa o si la curva baja muy lento.
- ¿Por qué la val loss puede subir después de cierto punto? Porque el modelo empieza a memorizar ruido del train: train sigue bajando pero la capacidad efectiva se gastó en patrones espurios que no generalizan.

Referencias

- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3ª ed.), cap. 4, sección "Early Stopping".
- scikit-learn docs: SGDRegressor (parámetros `early_stopping`, `validation_fraction`, `n_iter_no_change`, `tol`).
- Prechelt, L. (1998). Early Stopping — But When? en Neural Networks: Tricks of the Trade.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 075 — Clase 075 — Regresión logística binaria y softmax

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 70 min.

Objetivo

Que el alumno entienda la regresión logística como modelo lineal para clasificación: cómo la sigmoide convierte un score lineal en probabilidad, por qué se entrena minimizando log-loss (cross-entropy), y cómo se generaliza a multiclase con softmax. Además, que sepa diagnosticar si las probabilidades que devuelve un clasificador están bien calibradas y cómo corregirlas si no.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Derivar la sigmoide $\sigma(z) = 1/(1+e^{-z})$ como puente entre score lineal y probabilidad, y explicar por qué no se usa MSE en clasificación.
2. Entrenar LogisticRegression binaria de sklearn, interpretar coeficientes como log-odds y la frontera de decisión.
3. Extender a multiclase con multi_class='multinomial' (softmax) y diferenciar de 'ovr' (one-vs-rest).
4. Evaluar con log-loss y Brier score, no solo accuracy.
5. Diagnosticar y corregir calibración con calibration_curve y CalibratedClassifierCV (Platt / isotonic).

Temas

#	Tema	Por qué importa
1	Sigmoide y log-odds	Conecta regresión lineal con probabilidad
2	Log-loss (cross-entropy binaria)	Función de costo convexa, derivable, penal
3	Regularización (C, penalty)	sklearn regulariza por default — $C=1/\lambda$.
4	Softmax para multiclase	Generaliza sigmoide a K clases con probabi
5	multinomial vs ovr	El primero es softmax real; el segundo ent
6	Predict_proba y calibración	El score no siempre es probabilidad confia

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 067a — Calibración de probabilidades: Platt, isotonic, temperature scaling

Definiciones y características

Sigmoide $\sigma(z)$

: Función $1/(1+\exp(-z))$ que mapea $\rightarrow (0,1)$. Su inverso es el logit $\log(p/(1-p))$. La regresión logística modela $\logit(p) = w \cdot x + b$ (linealidad en los log-odds).

Log-loss (cross-entropy binaria)

: $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$. Función de costo convexa de la logística. Penaliza fuerte la confianza alta cuando te equivocas (predecir 0.99 y que sea 0 cuesta ~ 4.6).

Softmax

: Generalización de la sigmoide a K clases: $\text{softmax}(z_k) = \exp(z_k) / \sum \exp(z_j)$. Probabilidades positivas que suman 1. Es la activación final de logística multinomial y de la última capa en clasificadores neuronales.

Cross-entropy categórica

: Generalización del log-loss a K clases: $-\sum_k y_k \cdot \log(p_k)$ con y one-hot. Pareja natural de softmax.

Calibración

: Propiedad de que $P(y=1 | \hat{y}=p) \approx p$ para todo p . Un modelo calibrado al 0.7 acierta como positivo el 70% de las veces que dice "0.7".

Reliability diagram

: Gráfico de fracción observada de positivos vs score promedio en bins. Diagonal = perfecto. Curva en S = típica de árboles; curva inversa = sobre-confianza.

Brier score

: $\text{mean}((p - y)^2)$. MSE entre probabilidades y labels 0/1. Resume calibración + discriminación en un escalar. Menor = mejor.

CalibratedClassifierCV

: Wrapper de sklearn que envuelve un clasificador base y calibra sus probabilidades vía cross-validation interno. Métodos: 'sigmoid' (Platt) o 'isotonic'.

Dataset / recursos

- Iris (3 clases) para softmax — `sklearn.datasets.load_iris()`.
- Breast cancer Wisconsin (binario) para logística y calibración — `load_breast_cancer()`.
- Sintético desbalanceado con `make_classification(weights=[0.9, 0.1])` para visualizar mis-calibración de un RandomForest.

Ejercicios

1. Logística binaria desde cero. Entrená `LogisticRegression()` sobre breast cancer. Reportá accuracy, log-loss y matriz de confusión. Imprimí los 5 coeficientes con mayor $|w|$ e interpretá uno como odds-ratio ($\exp(w)$).
2. Frontera de decisión. Con 2 features de iris (solo 2 clases primero), graficá la frontera lineal de la logística y los puntos. Cambiá C entre 0.01 y 100 y observá cómo la frontera se vuelve más/menos rígida.
3. Softmax sobre iris. `LogisticRegression(multi_class='multinomial', solver='lbfgs')`. Comparalo con `multi_class='ovr'` en log_loss y accuracy. Imprimí `predict_proba` de 3 muestras y verificá que sumen 1.
4. Reliability diagram de un RandomForest. Entrená `RandomForestClassifier(n_estimators=100)` sobre el dataset sintético desbalanceado. Computá `calibration_curve` con `n_bins=10` y graficá vs diagonal. Reportá Brier score.
5. Calibrar con `CalibratedClassifierCV`. Sobre el mismo RF: aplicá `method='sigmoid'` y `method='isotonic'` (`cv=5`). Re-grficá los tres reliability diagrams (RF crudo, +Platt, +isotonic) y compará Brier scores. Reportá cuál calibra mejor y por qué te parece.

Homework verificable

Notebook que sobre `make_classification(n_samples=20000, weights=[0.9, 0.1], random_state=42)`: (a) entrena `LogisticRegression` y `RandomForestClassifier`; (b) reporta accuracy, log-loss y Brier score de ambos en test; (c) grafica reliability diagram de ambos en la misma figura; (d) calibra el RF con `CalibratedClassifierCV(method='isotonic', cv=5)` y reporta el nuevo Brier; (e) escribe 2-3 líneas interpretando: ¿quedó el RF mejor calibrado que la logística cruda?

Criterio de aceptación: Brier score del RF calibrado debe ser menor que el del RF crudo. El reliability diagram del calibrado debe estar visiblemente más cerca de la diagonal.

Errores comunes

Síntoma / mensaje

Causa y cómo arreglar

Usar predict_proba directamente para tomar	Asumir que el score es probabilidad. Fix:
ConvergenceWarning: lbfgs failed to conver	Features sin escalar o max_iter bajo. Fix:
Coefficientes enormes con C muy alto en dat	Sin regularización, la logística diverge c
multi_class='ovr' y log-loss raro en multi	OvR entrena K binarios independientes — la
Calibrar sobre el mismo set de entrenamien	Leakage — los scores ya están sobreajustad

Preguntas frecuentes

¿Por qué log-loss y no MSE para clasificación?

Con MSE sobre una sigmoide la superficie de costo es no convexa (varios mínimos locales) y los gradientes se saturan en los extremos. Log-loss + sigmoide da costo convexo y gradientes limpios ($p - y$).

¿Platt o isotonic?

Platt si tenés ~1000 ejemplos de calibración y la curva de mis-calibración parece sigmoidea (típico SVM, NN pequeñas). Isotonic si tenés ~10k+ y/o la mis-calibración no es monótona-sigmoidea (RF, boosting). Regla práctica: probá ambos en validación y quedate con el de menor Brier.

¿predict_proba de LogisticRegression está calibrado?

Generalmente sí, porque la logística optimiza log-loss directamente — sus probabilidades suelen ser razonables. Igual chequealo con calibration_curve antes de usarlas para decisiones críticas; regularización fuerte (C chico) puede sub-confiar el output.

¿Softmax y sigmoide son lo mismo en binario?

Equivalentes: softmax con K=2 colapsa a sigmoide. sklearn con multi_class='multinomial' y 2 clases hace lo mismo que el modo binario por default.

¿Calibrar afecta el accuracy o solo las probabilidades?

Platt e isotonic son monótonas no-decrecientes → no cambian el ranking ni el argmax → accuracy y AUC quedan iguales. Lo que cambia es el log-loss, el Brier score, y el threshold óptimo cuando elegís uno distinto de 0.5.

Referencias

- Géron, cap. 4 § Logistic Regression y § Softmax Regression.
- sklearn — Probability calibration
- sklearn — LogisticRegression
- Niculescu-Mizil & Caruana (2005), Predicting Good Probabilities With Supervised Learning, ICML — paper canónico sobre Platt vs isotonic en RF, SVM, boosting, NB.
- Guo et al. (2017), On Calibration of Modern Neural Networks — temperature scaling.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 076 — Clase 076 — Calibración de probabilidades: Platt, isotonic, temperature scaling

Parte: 1 — Machine Learning Clásico · Fuente: Platt (1999) + Niculescu-Mizil & Caruana (2005) + Guo et al. (2017). Duración estimada: 75 min.

Objetivo

Saber cuándo las probabilidades que devuelve predict_proba son calibradas — es decir, si el modelo dice "70 %" para un grupo, ¿realmente el 70 % es positivo? Modelos como Random Forest y SVM suelen estar mal calibrados; XGBoost mejor. Aplicar Platt scaling (sigmoid) y isotonic regression para corregir. Evaluar con Brier score, ECE (Expected Calibration Error) y reliability diagrams.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Generar un reliability diagram: agrupar predicciones por bin de probabilidad y plotear "predicho vs real".
- Calcular Brier score = $\text{mean}((p - y)^2)$. Más bajo = mejor calibración.
- Calcular ECE: $\sum (n_b/N) \cdot |\text{acc}_b - \text{conf}_b|$.
- Aplicar `sklearn.calibration.CalibratedClassifierCV(estimator, method='sigmoid' | 'isotonic', cv=5)`.
- Decidir: Platt cuando muestra chica ($n < 1000$), isotonic cuando hay datos.

Temas

- ¿Por qué importa? Decisiones que dependen de $\text{threshold} \neq 0.5$ requieren probs reales.
- Reliability diagram: predicción vs frecuencia real.
- Platt scaling: ajustar $\sigma(A \cdot \text{logit} + B)$ con MLE sobre val set.
- Isotonic regression: monótono pero más flexible (puede sobreajustar).
- Temperature scaling: solo divide logits por T aprendido — para multiclase, eficiente.
- Modelos típicamente calibrados (logistic regression) vs no (RF, SVM).

Definiciones y características

- Calibración: $P(y=1 | \hat{p}=p) = p$ para todo p.
- Reliability diagram: histograma de calibración.
- Brier score: $(1/N) \sum (p_i - y_i)^2$. Combina calibración + accuracy.
- ECE: weighted gap entre confidence y accuracy por bin.
- Platt: sigmoid-fit; parametric, 2 params (A, B). Bueno con n chico.
- Isotonic: monotonic non-parametric; flexible pero needs más data.
- Temperature scaling: $\text{softmax}(z / T)$ con T learnable. Para multiclase.

Dataset / recursos

- `fetch_openml('credit-g')` o `load_breast_cancer`.
- Librerías: `sklearn.calibration`, `matplotlib`.

Ejercicios

1. Reliability diagram: entrenar RandomForest, generar predict_proba, bin en 10 grupos, plotear curva calibration vs $y=x$. Suele desviar.
2. Brier + ECE: implementar ambas y comparar entre RF (mala calibración) y LogReg (buena).
3. CalibratedClassifierCV: `CalibratedClassifierCV(RF, method='sigmoid', cv=5).fit(X, y)`. Re-evaluar Brier.
4. Isotonic vs Platt: comparar ambos sobre el mismo modelo. Con $n=10_000$ ambos similares; con $n=300$ Platt mejor.
5. Threshold tuning post-calibración: con probs calibradas, F1 vs threshold es más interpretable.

Homework verificable

Sobre credit-g:

1. RandomForest + reliability diagram + Brier + ECE.

2. Calibrar con Platt y con isotonic (CV).
3. Comparar Brier/ECE pre y post.
4. Reliability diagrams superpuestos.

Criterio de aceptación: calibración reduce ECE en $\geq 30\%$; reliability diagram post se acerca a la diagonal.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Calibrar sobre train (overfit)	Leak. Fix: usar CV o held-out calibration
Isotonic con < 500 ejemplos por clase	Overfit. Fix: Platt.
Reportar accuracy post-calibración como mé	No mide calibración. Fix: Brier o ECE.
Asumir LogReg está calibrado siempre	Con regularización fuerte o features extra
Calibración en multiclase con sigmoid	Sigmoid es binario. Fix: temperature scali

Preguntas frecuentes

¿Calibración cambia accuracy?

No (mucho). Reordena las probs pero no cambia el argmax típicamente. La accuracy queda igual; las probs son más interpretables.

¿Cuándo importa?

Cuando hacés decisiones $\text{threshold} \neq 0.5$ (e.g., "alertar si $P > 0.8$ "), reportes de "confianza" al usuario, ensemble por probabilidades.

¿RF tan mal calibrado?

Sí, hacia probs intermedias (0.2-0.8). Boosting (XGBoost) suele estar mejor.

¿DL necesita calibración?

Sí — DL modernos tienden a sobre-confiar. Temperature scaling (Guo 2017) es el estándar.

¿En producción cómo monitoreo calibración?

Logueá (prob_predicha, y_real). Cada N días, calculá Brier y ECE. Si drift, re-calibrar.

Referencias

- Platt (1999), Probabilistic Outputs for Support Vector Machines.
- Niculescu-Mizil & Caruana (2005), Predicting Good Probabilities With Supervised Learning, ICML.
- Guo et al. (2017), On Calibration of Modern Neural Networks, ICML.
- sklearn calibration docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 077 — Clase 077 — SVM lineal

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 5. Duración estimada: 60 min.

Objetivo

Entender el principio de maximización del margen que define a las Support Vector Machines lineales, distinguir entre hard margin y soft margin, y entrenar un clasificador con LinearSVC controlando el trade-off bias/varianza mediante el hiperparámetro C.

Resultados de aprendizaje

Al finalizar la clase, podrás:

- Explicar qué es el margen y por qué SVM busca el hiperplano que lo maximiza.
- Diferenciar hard margin (datos linealmente separables, sin tolerancia) de soft margin (admite violaciones).
- Interpretar el hiperparámetro C y su efecto sobre el ancho del margen y las violaciones permitidas.
- Entrenar un LinearSVC en scikit-learn con un Pipeline que incluya StandardScaler.
- Identificar los vectores soporte y entender por qué son los únicos puntos que definen la frontera.

Temas

- Intuición geométrica: hiperplano separador y margen.
- Hard margin: condiciones y limitaciones (sensibilidad a outliers, exige separabilidad).
- Soft margin: introducción de variables de holgura (slack).
- Hiperparámetro C: regularización, trade-off margen ancho vs. violaciones.
- Función de pérdida hinge loss.
- API de scikit-learn: LinearSVC, loss, C, dual, max_iter.
- Importancia crítica del escalado de features en SVM.

Definiciones y características

- Margen (margin): distancia perpendicular entre el hiperplano separador y los puntos más cercanos de cada clase. SVM busca maximizarlo.
- Vectores soporte (support vectors): instancias que tocan o violan el margen. Son los únicos puntos que determinan la frontera; el resto del dataset no influye.
- Hard margin classification: exige que todas las instancias estén del lado correcto del margen. Solo funciona si los datos son linealmente separables y es muy sensible a outliers.
- Soft margin classification: permite violaciones del margen (instancias dentro del margen o del lado equivocado) a cambio de un margen más ancho y robusto.
- Hiperparámetro C: controla cuánto se penalizan las violaciones del margen. C alto → poco tolerante, margen estrecho, riesgo de overfitting. C bajo → más tolerante, margen ancho, más regularización.
- LinearSVC: implementación eficiente de SVM lineal en scikit-learn, basada en liblinear. Escala mejor que SVC(kernel="linear") en datasets grandes.
- Hinge loss: función de pérdida que SVM minimiza, definida como $\max(0, 1 - t \cdot y)$ donde t es la etiqueta (± 1) e y la salida del modelo. Es cero cuando la predicción está del lado correcto y fuera del margen.
- Escalado: SVM es sensible a la escala de los features; sin StandardScaler el margen queda dominado por las variables de mayor rango.

Dataset / recursos

- Iris (sklearn.datasets.load_iris) — filtrado a dos clases (Iris-Virginica vs. resto) usando los features petal length y petal width. Es el ejemplo canónico del capítulo 5 de Géron.
- Opcional: dataset sintético con make_classification o make_blobs para visualizar el efecto de C y de outliers.

Ejercicios

1. Cargá Iris, quedate con dos features (petal length, petal width) y la clase Virginica como problema

binario. Entrená un Pipeline([StandardScaler, LinearSVC(C=1, loss="hinge")]) y reportá accuracy sobre un split train/test.

2. Repetí el entrenamiento con C=0.1, C=1, C=100. Compará accuracy, número de vectores soporte estimados y ancho del margen. ¿Qué pasa en los extremos?
3. Graficá la frontera de decisión y el margen para los tres valores de C del ejercicio anterior (scatter de los datos + línea + márgenes punteados).
4. Agregá un outlier artificial a la clase minoritaria y volvé a entrenar con C alto y C bajo. Mostrá cómo C bajo absorbe mejor el outlier.
5. Compará tiempo de entrenamiento de LinearSVC vs. SVC(kernel="linear") sobre un dataset de ~10.000 muestras (make_classification). Confirmá que LinearSVC es más rápido.

Homework verificable

Entregá un script tarea_068.py que:

1. Cargue Iris binarizado (Virginica vs. resto) con petal length y petal width.
2. Construya un Pipeline con StandardScaler + LinearSVC(C=1, loss="hinge", random_state=42).
3. Haga train_test_split(test_size=0.2, random_state=42, stratify=y) y entrene.
4. Imprima accuracy sobre test y los coeficientes del clasificador (coef_, intercept_).

Criterio de aceptación: accuracy $\geq$ 0.93 sobre el test set y los coeficientes deben provenir del modelo escalado (no del crudo).

Errores comunes

1. No escalar features. SVM es sensible a la escala; sin StandardScaler el margen lo domina la variable de mayor rango y el modelo rinde mal.
2. Confundir la dirección de C. C alto = menos regularización (margen estrecho, más overfitting). C bajo = más regularización. Es inversa a alpha de Ridge/Lasso.
3. Usar SVC(kernel="linear") en datasets grandes. Es $O(m^2)$ a $O(m^3)$; preferí LinearSVC (basado en liblinear) que escala mejor.
4. Esperar predict_proba de LinearSVC. No lo soporta directamente. Usá CalibratedClassifierCV o decision_function si necesitás scores.
5. Olvidar loss="hinge". El default de LinearSVC es "squared_hinge", que no es la pérdida SVM canónica del libro.

Preguntas frecuentes

1. ¿C alto o bajo? Depende del problema. Empezá con C=1 y ajustá con GridSearchCV. Si hay overfitting, bajá C; si hay underfitting o el modelo es demasiado tolerante, subí C.
2. ¿Por qué se llaman "vectores soporte"? Porque son los únicos puntos que "sostienen" la frontera: si los movés, el hiperplano cambia. Los demás puntos no afectan al modelo.
3. ¿SVM lineal sirve si los datos no son linealmente separables? Con soft margin sí, tolera violaciones. Si la separación claramente no es lineal, pasá a kernels (clase 069).
4. ¿LinearSVC vs. LogisticRegression? Ambos producen fronteras lineales. SVM optimiza margen (hinge loss), LogReg optimiza log-likelihood. En la práctica suelen dar resultados similares; SVM tiende a ser más robusto a outliers cuando C es moderado.
5. ¿Funciona para multiclase? Sí, vía one-vs-rest automáticamente en LinearSVC.

Referencias

- Géron, A. Hands-On Machine Learning, 3ra ed., cap. 5 — "Support Vector Machines", sección "Linear SVM Classification".

- scikit-learn user guide: 1.4. Support Vector Machines.
- API: sklearn.svm.LinearSVC.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 078 — Clase 078 — SVM no lineal: kernel polinomial y RBF

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 5. Duración estimada: 70 min.

Objetivo

Que el alumno entrene SVMs sobre datos no linealmente separables usando el kernel trick: en lugar de generar features polinómicas a mano (caro en memoria), SVC calcula el producto interno en el espacio expandido vía una función kernel. Foco en kernel polinomial y RBF (Gaussian), y cómo gamma y C controlan el bias-variance trade-off.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar el kernel trick: por qué $K(x, x')$ evita materializar el feature map $\phi(x)$.
2. Entrenar $SVC(kernel='poly')$ y $SVC(kernel='rbf')$ en datasets no lineales (moons, circles).
3. Tunear gamma y C con GridSearchCV entendiendo el efecto en la frontera.
4. Elegir kernel según geometría del problema (polinomial vs RBF vs lineal).
5. Reconocer el límite computacional de SVC: complejidad entre $O(n^2)$ y $O(n^3)$, no escala a $>100k$ filas.

Temas

#	Tema	Por qué importa
1	Datos no linealmente separables (moons, ci	Motivación: lineal no alcanza.
2	Polynomial features manuales vs kernel tri	Kernel evita explosión combinatoria.
3	Kernel polinomial: degree, coef0, gamma	Captura interacciones de orden d.
4	Kernel RBF (Gaussian): gamma como inverso	Default robusto en sklearn.
5	C (regularización) × gamma (forma)	Grid 2D clásico.
6	Complejidad $O(n^2)$ – $O(n^3)$ y LinearSVC / SGD	Cuándo NO usar SVC.

Definiciones y características

Kernel trick

: Truco matemático que reemplaza el producto interno $\phi(x) \cdot \phi(x')$ en el espacio expandido por una función $K(x, x')$ calculable directamente en el espacio original. Permite trabajar en espacios de dimensión infinita (RBF) sin materializarlos. Requisito: K debe ser definida positiva (condición de Mercer).

Kernel RBF (Gaussian)

: $K(x, x') = \exp(-\gamma \cdot \|x - x'\|^2)$. Mide similitud por distancia: 1 si $x = x'$, $\rightarrow 0$ cuando se alejan. Equivale a un feature map infinito-dimensional. Default razonable cuando no sabés qué kernel usar.

gamma (γ)

: Inverso del ancho del kernel RBF. γ alto $\rightarrow$ cada punto influye solo en su vecindad inmediata $\rightarrow$ frontera muy ondulada $\rightarrow$ overfitting. γ bajo $\rightarrow$ influencia amplia $\rightarrow$ frontera suave $\rightarrow$ underfitting. En sklearn: `gamma='scale'` (default) = $1 / (n_features \cdot X.var())$.

Kernel polinomial

: $K(x, x') = (\gamma \cdot x \cdot x' + \text{coef0})^{\text{degree}}$. Captura interacciones hasta orden degree. coef0 controla el peso de los términos de orden alto vs bajo. Típicamente degree=2 o 3; más alto suele overfittear.

Kernel sigmoid

: $K(x, x') = \tanh(\gamma \cdot x \cdot x' + \text{coef0})$. Rara vez supera a RBF en la práctica; se mantiene por razones históricas (similitud con redes neuronales).

Complejidad $O(n^2)$ – $O(n^3)$

: SVC resuelve un QP cuadrático sobre los n ejemplos. En la práctica, libsvm escala entre $O(n^2)$ y $O(n^3)$ según la fracción de support vectors. Para >50k–100k filas: usar LinearSVC (si es lineal) o SGDClassifier(loss='hinge').

Condición de Mercer

: Para que una función K sea un kernel válido, su matriz de Gram $[K(x_i, x_j)]$ debe ser semidefinida positiva para cualquier conjunto finito de puntos. RBF, polinomial y lineal la cumplen; sigmoid solo bajo ciertos parámetros.

C (regularización)

: Inverso del parámetro de regularización L2. C alto $\rightarrow$ poca tolerancia a violaciones del margen $\rightarrow$ margen estrecho, posible overfit. C bajo $\rightarrow$ margen amplio, más violaciones permitidas, modelo más simple.

Dataset / recursos

- `sklearn.datasets.make_moons(n_samples=500, noise=0.15)` — clásico no lineal.
- `sklearn.datasets.make_circles(n_samples=500, noise=0.1, factor=0.4)` — radial puro, ideal para mostrar RBF.

Ejercicios

1. Lineal falla. Entrená `SVC(kernel='linear')` sobre `make_moons`. Reportá accuracy y graficá la frontera. Mostrá visualmente que es inadecuada.
2. Polynomial kernel. `SVC(kernel='poly', degree=3, coef0=1, C=5)` sobre `moons`. Comparar accuracy y forma de la frontera vs lineal.
3. RBF kernel. `SVC(kernel='rbf', gamma=5, C=1)` sobre `circles`. Variar gamma {0.1, 1, 10, 100} con el mismo C y graficar las 4 fronteras lado a lado.
4. Grid search 2D. `GridSearchCV` sobre gamma {0.01, 0.1, 1, 10} $\times$ C {0.1, 1, 10, 100} con cv=5 sobre `moons`. Reportar el mejor par y matriz de scores como heatmap.
5. Pipeline con `StandardScaler`. SVMs son sensibles a escala. Armá `Pipeline([('sc', StandardScaler()), ('svc', SVC(kernel='rbf'))])` y comparar accuracy con y sin scaler en un dataset con features de magnitudes muy distintas.

Homework verificable

Notebook con `make_moons(n_samples=1000, noise=0.2)`. Train/test split 80/20. Entrenar tres modelos: (a) `SVC(kernel='linear')`, (b) `SVC(kernel='poly', degree=3)`, (c) `SVC(kernel='rbf')` con gamma y C tuneados vía `GridSearchCV(cv=5)`. Reportar accuracy en test de los tres y graficar las tres fronteras de decisión en una

grilla 1×3.

Criterio de aceptación: El modelo RBF tuneado supera al lineal en al menos +15 puntos de accuracy. El notebook incluye el heatmap de GridSearchCV y las tres fronteras visualizadas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Usar SVC en datasets >100k filas y que el	Complejidad $O(n^2)$ – $O(n^3)$. Fix: si el proble
Accuracy pésima sin escalar features	RBF y polinomial dependen de distancias. U
gamma alto da 100% en train y 60% en test	Overfitting clásico: γ alto = frontera ond
SVC(kernel='poly', degree=10) tarda eterni	Polinomios de alto grado explotan numérica
No setear random_state y resultados irrepr	SVC con probability=True o algunos solvers

Preguntas frecuentes

¿RBF o polynomial?

Si no sabés nada del problema: RBF (default robusto, un solo hiperparámetro relevante gamma). Polynomial conviene si tenés razones para creer que las interacciones son de orden bajo y fijo (ej. features físicas que se multiplican). RBF tiende a ganar en benchmarks tabulares.

¿Qué pasa si $\gamma \rightarrow 0$ en RBF?

El kernel tiende a constante $\rightarrow$ todos los puntos parecen iguales $\rightarrow$ modelo equivale a clasificar por mayoría. Frontera trivial (clase mayoritaria en todo el espacio).

¿Cuántos support vectors es "muchos"?

Si $\text{len}(\text{svm.support_}) > 0.5 \cdot n_{\text{train}}$, el modelo está overfitteando o C es muy alto. SVMs eficientes tienen pocos SVs (los del borde).

¿Por qué gamma='scale' y no 'auto'?

'scale' = $1 / (n_{\text{features}} \cdot X.\text{var}())$ — tiene en cuenta la varianza de los datos, robusto a escala. 'auto' = $1 / n_{\text{features}}$ (default viejo, deprecado conceptualmente). Usá 'scale' o un valor explícito tuneado.

¿Puedo usar SVM kernelizada con 1M de filas?

No con SVC. Alternativas: (a) LinearSVC si el problema es separable linealmente tras feature engineering; (b) Nystroem + LinearSVC para aproximar RBF en $O(n)$; (c) SGDClassifier(loss='hinge'); (d) cambiar de modelo (Gradient Boosting suele ganar en tabular grande).

Referencias

- Géron, cap. 5 — Nonlinear SVM Classification y The Kernel Trick.
- sklearn — SVC
- sklearn — RBF kernel intuition
- sklearn — Kernel approximation (Nystroem)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 079 — Clase 079 — SVM para regresión (SVR)

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 5. Duración estimada: 50 min.

Objetivo

Aplicar Support Vector Machines al problema de regresión: entender el truco del epsilon-insensitive loss (ajustar dentro de un tubo de tolerancia en vez de minimizar el error puntual), entrenar LinearSVR y SVR con kernel, y elegir cuándo conviene SVR sobre regresión lineal clásica.

Resultados de aprendizaje

Al finalizar la clase, vas a poder:

- Explicar la lógica de SVR: maximizar el ancho del tubo ϵ mientras se contienen la mayoría de los puntos dentro.
- Entrenar un LinearSVR y un SVR(kernel="rbf") de scikit-learn con sus hiperparámetros (epsilon, C, gamma).
- Interpretar el efecto de epsilon (ancho del tubo) y C (penalización por puntos fuera del tubo) en el bias-variance trade-off.
- Comparar SVR vs LinearRegression / Ridge en un dataset con outliers.
- Justificar cuándo SVR escala mal (SVR es $O(m^2-m^3)$) y conviene LinearSVR u otro modelo.

Temas

1. De SVM clasificación a SVM regresión: invertir el objetivo.
2. Epsilon-insensitive loss: errores menores a ϵ no se penalizan.
3. LinearSVR: caso lineal, escala bien ($O(m)$).
4. SVR con kernel RBF: regresión no lineal, costo cuadrático.
5. Hiperparámetros clave: epsilon, C, gamma, kernel.
6. Robustez frente a outliers comparada con OLS.

Definiciones y características

- SVR (Support Vector Regression): algoritmo de regresión que busca una función que se desvíe a lo sumo ϵ de cada target, manteniéndola lo más "plana" posible (margen máximo).
- Epsilon-tube (tubo ϵ): banda de ancho 2ϵ alrededor de la predicción donde los errores no cuentan. Solo los puntos fuera del tubo aportan a la pérdida.
- LinearSVR: implementación lineal de SVR basada en liblinear. Escala linealmente con el tamaño del dataset; no soporta kernels.
- Kernel SVR: SVR de scikit-learn usa libsvm; soporta kernels (rbf, poly, sigmoid) para capturar no linealidad, pero su complejidad es entre $O(m^2)$ y $O(m^3)$.
- Hiperparámetro C: controla la penalización por puntos fuera del tubo. C chico $\rightarrow$ modelo más regularizado (tubo "flexible"); C grande $\rightarrow$ ajuste más estricto.
- Hiperparámetro epsilon: define el ancho del tubo de insensibilidad. ϵ grande $\rightarrow$ menos vectores de soporte, modelo más simple; ϵ chico $\rightarrow$ ajuste más fino, más vectores.
- Robust regression: SVR es menos sensible que OLS a outliers porque la pérdida es lineal fuera del tubo (no cuadrática).

Dataset / recursos

- California Housing (sklearn.datasets.fetch_california_housing) para regresión realista.
- Dataset sintético con outliers (make_regression + ruido pesado) para mostrar robustez.
- Géron, Hands-On ML (3ª ed.), cap. 5, sección "SVM Regression".

Ejercicios

1. Tubo ϵ en 2D. Generá $y = 0.5x + \text{ruido}$, entrená `LinearSVR(epsilon=0.5)` y graficá la recta junto al tubo $\pm\epsilon$. Marcá los vectores de soporte (puntos fuera del tubo).
2. Efecto de epsilon. Repetí el ejercicio 1 con $\epsilon \in \{0.1, 0.5, 1.5\}$. ¿Cómo cambia la cantidad de vectores de soporte y el MSE en test?
3. Kernel RBF. En un dataset no lineal ($y = \sin(x) + \text{ruido}$), compará `LinearSVR` vs `SVR(kernel="rbf", gamma="scale")`. Reportá MAE y graficá ambas curvas.
4. Grid search. Sobre California Housing, hacé `GridSearchCV` con `SVR` variando $C \in \{0.1, 1, 10\}$ y $\text{gamma} \in \{\text{"scale"}, 0.01, 0.1\}$. No uses más de 5 000 muestras (cuidado con el costo cuadrático).
5. Robustez vs OLS. Inyectá 5% de outliers en un dataset lineal y compará `LinearRegression`, `Ridge` y `LinearSVR`. ¿Cuál degrada menos?

Homework verificable

Entregar un script `svr_california.py` que:

1. Cargue California Housing y aplique `StandardScaler` (¡SVR exige escalado!).
2. Entrene `LinearSVR(epsilon=0.5, C=1.0, random_state=42)` y un `SVR(kernel="rbf", C=10, gamma="scale")` sobre un subset de 5 000 muestras.
3. Reporte RMSE en test para ambos modelos.

Criterio de aceptación: `LinearSVR RMSE < 0.85` y `SVR-RBF RMSE < 0.65` sobre el split de test (`train_test_split(random_state=42, test_size=0.2)`).

Errores comunes

- No escalar los features. SVR es extremadamente sensible a la escala: sin `StandardScaler`, los hiperparámetros pierden sentido y el entrenamiento no converge.
- Usar SVR con kernel sobre datasets grandes. Para más de ~10 000 muestras, SVR se vuelve impracticable. Usá `LinearSVR` o `SGDRegressor(loss="epsilon_insensitive")`.
- Confundir epsilon de SVR con epsilon de optimizadores. Acá ϵ es el ancho del tubo de tolerancia, no una tolerancia numérica de convergencia.
- Dejar C en el default sin tunear. El default $C=1.0$ rara vez es óptimo; siempre validá con CV.
- Comparar tiempos sin `dual="auto"`. En `LinearSVR`, el parámetro dual cambia drásticamente el tiempo según n_{samples} vs n_{features} .

Preguntas frecuentes

- ¿SVR vs Linear Regression cuándo conviene cuál? OLS minimiza el error cuadrático medio (sensible a outliers, óptimo si el ruido es gaussiano). SVR ignora errores chicos (dentro del tubo ϵ) y penaliza linealmente los grandes: más robusto a outliers y suele generalizar mejor con pocos datos ruidosos. Si tu dataset es limpio y grande, OLS/Ridge suele ser más simple y rápido.
- ¿Cómo elijo epsilon? Empezá con una fracción del desvío estándar del target (p. ej. $\epsilon \approx 0.1 \cdot \sigma(y)$) y ajustá con CV. Un ϵ muy chico convierte SVR en algo cercano a regresión cuadrática; uno muy grande genera un modelo casi constante.
- ¿SVR devuelve probabilidades o intervalos? No. SVR predice un valor puntual. Para intervalos de predicción usá `quantile regression`, `conformal prediction` o un modelo bayesiano.
- ¿Por qué SVR con kernel es tan lento? Calcula la matriz kernel ($m \times m$) y resuelve un QP. La complejidad va de $O(m^2)$ a $O(m^3)$. Para datasets grandes, `LinearSVR` o `Nystroem + LinearSVR` (kernel approximation) son alternativas viables.
- ¿Conviene SVR para deep learning / problemas con millones de muestras? No. Para esas escalas, `gradient boosting` (XGBoost, LightGBM) o redes neuronales superan a SVR tanto en performance como

en costo.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3ª ed.), O'Reilly, cap. 5 — "SVM Regression".
- Smola, A. & Schölkopf, B. A Tutorial on Support Vector Regression (2004).
- scikit-learn docs: `sklearn.svm.SVR` y `sklearn.svm.LinearSVR`.
- scikit-learn user guide: Support Vector Machines — Regression.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 080 — Clase 080 — Árboles de decisión: entrenamiento, visualización, CART

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 6 § Training and Visualizing a Decision Tree.

Duración estimada: 60 min.

Objetivo

Que el alumno entrene un `DecisionTreeClassifier` con el algoritmo CART, entienda cómo se elige cada split (criterio Gini/Entropy), y sepa leer el árbol — tanto el dibujo (`plot_tree`) como el código (`export_graphviz`) — para auditar las decisiones del modelo.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Entrenar un `DecisionTreeClassifier` de scikit-learn sobre un dataset tabular (Iris) y predecir clases / probabilidades.
2. Explicar el algoritmo CART: greedy, binario, busca el par (feature, threshold) que minimiza la impureza ponderada de los dos hijos.
3. Calcular Gini y entropía a mano sobre un nodo con k clases y elegir el split óptimo entre candidatos.
4. Visualizar el árbol entrenado con `plot_tree` (matplotlib) y `export_graphviz` (DOT → PNG/SVG), interpretando `samples`, `value`, `class`, `gini`.
5. Identificar la decision boundary axis-aligned: cada split es ortogonal a un eje, lo que limita al árbol frente a fronteras oblicuas.

Temas

#	Tema	Por qué importa
1	<code>DecisionTreeClassifier</code> API	El estimador base del capítulo y de Random
2	Algoritmo CART	Saber qué hace el <code>.fit()</code> por debajo evita
3	Criterio Gini vs Entropy	Casi siempre dan el mismo árbol; saber cuál
4	Visualización: <code>plot_tree</code> y <code>Graphviz</code>	Auditar el modelo y comunicarlo a no-técni
5	Interpretación de nodos	Leer <code>value=[n0, n1, n2]</code> , <code>samples</code> , <code>class pr</code>
6	Decision boundary axis-aligned	Limitación geométrica que explica por qué

Definiciones y características

CART (Classification And Regression Trees)

: Algoritmo que entrena árboles binarios (cada split tiene exactamente 2 hijos). Greedy: en cada nodo elige el par (feature k , threshold t_k) que minimiza el costo $J = (m_{\text{left}}/m) \cdot G_{\text{left}} + (m_{\text{right}}/m) \cdot G_{\text{right}}$. No vuelve atrás (no es óptimo global).

Impureza Gini

: $G_i = 1 - \sum p_{\{i,k\}}^2$ sobre las k clases del nodo i . Vale 0 si el nodo es puro (una sola clase) y máximo cuando las clases están balanceadas. Es el default en sklearn.

Entropía (information gain)

: $H_i = - \sum p_{\{i,k\}} \cdot \log(p_{\{i,k\}})$. Mide desorden en bits. Se activa con `criterion='entropy'`. En la práctica produce árboles muy similares a Gini; Gini es marginalmente más rápido.

Nodo

: Punto del árbol donde se evalúa una condición $\text{feature} \leq \text{threshold}$. Si se cumple, va al hijo izquierdo; si no, al derecho.

Hoja (leaf)

: Nodo terminal, sin hijos. Predice la clase mayoritaria de las muestras de entrenamiento que cayeron en él. La probabilidad estimada es la proporción de cada clase en la hoja.

Profundidad (max_depth)

: Distancia desde la raíz hasta la hoja más lejana. Controla complejidad: sin tope, CART crece hasta que cada hoja es pura $\rightarrow$ overfitting casi garantizado. Se ataca en la Clase 072.

criterion

: Hiperparámetro de sklearn: 'gini' (default) o 'entropy'. Define la función de impureza que minimiza CART al elegir splits.

Decision boundary axis-aligned

: Cada split parte el espacio con un hiperplano ortogonal a un eje ($x_k = t_k$). La frontera resultante es una unión de rectángulos. Por eso un árbol no puede aprender una diagonal de forma compacta — necesita escaleras.

Dataset / recursos

- Iris (sklearn.datasets.load_iris): 150 muestras, 4 features, 3 clases. Géron lo usa porque permite dibujar el árbol completo en una página y la frontera en 2D (pétalo largo $\times$ ancho).
- Opcional: moons (make_moons) para ver la limitación axis-aligned.

Ejercicios

1. Fit + score baseline. Entrená DecisionTreeClassifier(max_depth=2, random_state=42) sobre Iris (todas las features). Reportá accuracy en train. Probabilidades de la primera flor con .predict_proba.
2. Gini a mano. Para el nodo raíz de Iris (50/50/50), calculá Gini. Verificá contra tree_.impurity[0] del estimador entrenado.
3. Gini vs Entropy. Entrená dos árboles max_depth=3, uno con cada criterio. Compará accuracy y el set de features usadas en los splits (tree_.feature). ¿Cambia algo material?
4. plot_tree. Renderizá el árbol del ejercicio 1 con sklearn.tree.plot_tree(clf, feature_names=..., class_names=..., filled=True). Identificá: feature del root split, threshold, y la clase predicha por cada hoja.

5. Boundary axis-aligned. Entrená un árbol `max_depth=4` sobre `make_moons(n_samples=300, noise=0.2)`. Graficá la decision boundary con un `meshgrid`. Observá los rectángulos.

Homework verifiable

Notebook que: (a) carga Iris (solo pétalo largo y ancho), (b) entrena `DecisionTreeClassifier(max_depth=2)`, (c) reporta `accuracy` y `predict_proba` para una flor nueva `[5, 1.5]`, (d) exporta el árbol con `export_graphviz` a un archivo `iris_tree.dot` y lo convierte a PNG (o usa `plot_tree`), (e) responde por escrito: "¿por qué la hoja izquierda tiene `Gini=0`?".

Criterio de aceptación: `accuracy ≥ 0.95` en train, el árbol exportado tiene exactamente 3 hojas (porque `max_depth=2` con el split que CART elige sobre pétalos), y la respuesta menciona que esa hoja contiene solo setosa (pétalos chicos → linealmente separable).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>accuracy = 1.0</code> en train y mucho menos en t	No pusiste <code>max_depth</code> ni regularización. CA
<code>graphviz: command not found</code> al correr expo	El paquete Python <code>graphviz</code> no trae el bina
Cambio <code>criterion='gini' ↔ 'entropy'</code> y el á	Casi nunca pasa en datasets reales — si pa
Resultados distintos en cada <code>.fit()</code> con el	CART desempata splits con un componente al
<code>plot_tree</code> sale ilegible (texto pisado, min	Árbol grande sin <code>figsize</code> . Fix: <code>plt.figure(</code>

Preguntas frecuentes

¿Hay que escalar las features antes de un árbol?

No. CART compara `feature ≤ threshold` por feature individualmente — escalar/centrar no cambia el orden, así que el árbol es invariante a transformaciones monótonas de cada feature. Esto lo diferencia de SVM, KNN y regresión regularizada.

¿Gini o Entropy, cuál uso?

Gini (default). Es marginalmente más rápido (no calcula log) y produce árboles casi idénticos. Géron lo dice explícito: la elección rara vez importa en la práctica.

¿Por qué los árboles de sklearn son siempre binarios? ID3 hacía multi-way.

Porque sklearn implementa CART, no ID3/C4.5. CART es binario por diseño: un split por nodo, dos hijos. Es más simple, se generaliza a regresión sin cambios, y los multi-way se simulan encadenando binarios.

¿Los árboles manejan features categóricas?

sklearn no nativamente (hasta versión 1.4 hay soporte experimental). Hay que codificar: ordinal si hay orden natural, one-hot si no. Cuidado con one-hot de alta cardinalidad: el árbol nunca puede agrupar dos categorías en el mismo split, lo que sesga la selección hacia features numéricas.

¿Qué predice una hoja cuando empatan dos clases?

Devuelve la clase con índice menor entre las empatadas (orden de `clf.classes_`). En `predict_proba` te muestra el empate real (`[0.5, 0.5, 0]`), así que si te importa, usá probabilidades en vez de la clase `argmax`.

Referencias

- Géron, cap. 6 — Decision Trees, secciones Training and Visualizing y Making Predictions.
- sklearn — Decision Trees user guide

- DecisionTreeClassifier API
- Graphviz — para export_graphviz + dot.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 081 — Clase 081 — Regularización de árboles

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 6. Duración estimada: 50 min.

Objetivo

Que el alumno controle el overfitting de un DecisionTreeClassifier/Regressor usando hiperparámetros de regularización (pre-pruning) y cost-complexity pruning (post-pruning), y sepa elegir entre ambas estrategias en función del problema.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Identificar overfitting en un árbol sin regularizar (train accuracy ≈ 1 , test bajo).
2. Tunear max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, max_features con GridSearchCV.
3. Aplicar cost-complexity pruning vía ccp_alpha y leer la curva cost_complexity_pruning_path.
4. Diferenciar pre-pruning (frenar el crecimiento) de post-pruning (podar después).
5. Justificar la elección de hiperparámetros con curvas de validación, no a ojo.

Temas

#	Tema	Por qué importa
1	Árboles sin regularizar = overfitting gara	Un árbol crece hasta hojas puras.
2	max_depth y max_leaf_nodes	Los dos frenos más efectivos y baratos.
3	min_samples_split / min_samples_leaf	Frenan splits sobre muestras chicas (ruido)
4	max_features	Aleatoriza el split; antesala de Random Fo
5	ccp_alpha (cost-complexity pruning)	Post-pruning principled, basado en α .
6	Pre-pruning vs post-pruning	Trade-off: rapidez vs óptimo bias-variance

Definiciones y características

max_depth

: Profundidad máxima del árbol. Default None (crece hasta hojas puras $\rightarrow$ overfit). Valor típico: 3–10. El hiperparámetro más impactante y barato de tunear.

min_samples_split

: Mínimo de muestras que un nodo necesita para ser candidato a split. Default 2 (cualquier nodo se parte). Subirlo (ej. 20) bloquea splits sobre muestras chicas, que suelen ser ruido.

min_samples_leaf

: Mínimo de muestras que debe quedar en cada hoja resultante del split. Default 1 (hojas con 1 sample $\rightarrow$

memorización). Subirlo a 5–20 suaviza la frontera.

`max_leaf_nodes`

: Tope absoluto de hojas. El árbol crece en best-first (mejor reducción de impureza primero) hasta alcanzarlo. Equivalente funcional a `max_depth` pero más fino — el árbol no tiene que ser balanceado.

`max_features`

: Cantidad de features consideradas en cada split. Default = todas. Bajarlo (`sqrt`, `log2`) introduce aleatoriedad, reduce varianza y acelera el fit. Es la idea base de Random Forest.

`ccp_alpha` (cost-complexity pruning)

: Hiperparámetro $\alpha \geq 0$ que penaliza la complejidad del árbol al podar. Se minimiza $R(T) + \alpha \cdot |T|$ donde $|T| = n^\circ$ de hojas. $\alpha=0$ no poda; α grande poda agresivo. Se elige con `cost_complexity_pruning_path()` + CV.

Pre-pruning

: Frenar el crecimiento durante el fit con `max_depth`, `min_samples_*`, `max_leaf_nodes`. Rápido y simple — es lo que más se usa.

Post-pruning

: Dejar crecer el árbol completo y después podarlo con `ccp_alpha`. Más fundamentado teóricamente, pero requiere doble pasada (fit completo + path + CV).

Dataset / recursos

moons de `sklearn.datasets.make_moons(n_samples=1000, noise=0.4)` — clásico para visualizar fronteras de decisión y el efecto de regularizar.

Ejercicios

1. Overfit baseline. Entrená un `DecisionTreeClassifier()` sin tocar nada sobre moons. Reportá train vs test accuracy. ¿Cuánto gap hay?
2. `max_depth` sweep. Para `max_depth` {1, 2, 3, 5, 10, None} graficá train y test accuracy. Identificá el punto donde empieza el overfit.
3. GridSearch. Buscá con `GridSearchCV(cv=5)` la mejor combinación de `max_depth`, `min_samples_leaf` y `max_leaf_nodes`. Reportá mejores params y test score.
4. Cost-complexity path. Llamá `tree.cost_complexity_pruning_path(X_train, y_train)` para obtener `ccp_alphas`. Entrená un árbol por cada α y graficá test accuracy vs α . Elegí el óptimo.
5. Visualización de fronteras. Ploteá la decision boundary de (a) árbol sin regularizar y (b) árbol con `max_depth=4`. Compará overfit visual.

Homework verificable

Notebook con moons (noise=0.4, 1000 samples, split 70/30): (a) baseline sin regularizar — reportar gap train/test; (b) `GridSearchCV` sobre `max_depth` y `min_samples_leaf` con 5-fold; (c) cost-complexity pruning — graficar `ccp_alphas` vs test accuracy y elegir el mejor α ; (d) comparar test accuracy de los 3 modelos (baseline, grid, pruned) en tabla.

Criterio de aceptación: El modelo regularizado tiene gap train-test ≤ 5 pp y test accuracy $\geq$ baseline. La elección de α está justificada con la curva, no a ojo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Train accuracy = 1.0, test = 0.75	Árbol sin regularizar memorizó train. Fix:
GridSearchCV tarda eternamente	Grilla enorme sobre 4+ hiperparámetros. Fi
ccp_alpha no hace nada	Default es 0.0 (sin pruning). Fix: corré c
min_samples_split=2 "no es default" — sí l	Confusión típica: 2 es el default y signif
Tuneás max_features=sqrt y los resultados	max_features < n_features introduce aleato

Preguntas frecuentes

¿Pre-pruning o post-pruning?

Pre-pruning (max_depth, min_samples_leaf) en el 90% de los casos: rápido, simple, suficiente. Post-pruning (ccp_alpha) cuando querés un árbol único, interpretable, óptimo bias-variance — típico en problemas tabulares chicos donde el árbol es el modelo final, no un building block. Para ensembles (RF, GBM), pre-pruning y listo.

¿max_depth o max_leaf_nodes?

max_depth es más fácil de interpretar ("profundidad ≤ 5 "). max_leaf_nodes es más flexible — deja que el árbol sea asimétrico y crezca solo donde realmente reduce impureza. Si te importa interpretabilidad: max_depth. Si te importa performance: max_leaf_nodes.

¿Cómo elijo min_samples_leaf?

Regla de dedo: 1–5% de n_samples. Con 1000 filas, probá 10–50. Cuanto más ruidoso el dataset, más alto.

¿max_features regulariza?

Sí, indirectamente: al limitar features candidatas por split, fuerza al árbol a usar features sub-óptimas y reduce varianza. Es el mecanismo central de Random Forest, no tanto de árbol único.

¿Por qué ccp_alpha se usa poco en la práctica?

Porque (a) requiere doble pasada (path + CV), (b) en ensembles ya tenés regularización implícita por agregación, (c) max_depth + min_samples_leaf suele alcanzar. Pero es la podada con mejor fundamento teórico (Breiman et al., CART).

Referencias

- Géron, cap. 6 § "Regularization Hyperparameters".
- scikit-learn — Decision Trees user guide
- scikit-learn — Post pruning decision trees with cost complexity pruning
- Breiman, Friedman, Olshen & Stone (1984), Classification and Regression Trees — el paper original de CART y cost-complexity pruning.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 082 — Clase 082 — Regresión con árboles

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 6. Duración estimada: 45 min.

Objetivo

Entrenar árboles de decisión para problemas de regresión con `DecisionTreeRegressor`, entender por qué sus predicciones son escalonadas (constantes por hoja) y reconocer su incapacidad para extrapolar fuera del rango de entrenamiento.

Resultados de aprendizaje

Al finalizar la clase, el estudiante podrá:

- Ajustar un `DecisionTreeRegressor` de scikit-learn y predecir sobre datos nuevos.
- Explicar cómo el criterio MSE (squared error) decide los splits en regresión.
- Visualizar la predicción escalonada del árbol contra el target real en 1D.
- Identificar el problema de no-extrapolación y contrastarlo con regresión lineal.
- Regular `max_depth` / `min_samples_leaf` para balancear bias y varianza.

Temas

- `DecisionTreeRegressor`: API e hiperparámetros principales.
- Criterio de split: `squared_error` (MSE) y `friedman_mse`.
- Predicción como media del target dentro de cada hoja → función escalonada.
- Sobreajuste con árboles profundos y regularización vía `max_depth`, `min_samples_leaf`, `min_samples_split`.
- Limitación clave: el árbol no extrapola — predice el último valor visto en los extremos.
- Comparación rápida con regresión lineal en datos con tendencia.

Definiciones y características

- `DecisionTreeRegressor`: variante de árbol que en cada hoja predice un escalar (la media del target de las muestras que cayeron ahí), no una clase.
- `squared_error` (MSE): criterio por defecto; en cada split elige el corte que minimiza la suma ponderada de varianzas en los hijos. Es el análogo de Gini/entropía pero para regresión.
- `friedman_mse`: variante de MSE que ajusta una mejora propuesta por Friedman; suele dar splits levemente distintos, útil con boosting.
- Predicción escalonada (step prediction): como cada hoja devuelve una constante, la función $\hat{y}(x)$ es piecewise constant. En 1D se ve como una escalera, nunca como una curva suave.
- No extrapolación: fuera del rango $[\min(x_{\text{train}}), \max(x_{\text{train}})]$ el árbol devuelve la constante de la hoja del extremo. No hay pendiente, no hay tendencia: lineal con `slope = 0` afuera.
- Regularización: `max_depth` limita profundidad, `min_samples_leaf` exige un mínimo de muestras por hoja (suaviza); sin estos límites el árbol llega a `MSE = 0` en train (una hoja por muestra) y sobreajusta.
- Sensibilidad a rotaciones: igual que en clasificación, los splits son axis-aligned; si la relación es diagonal, el árbol necesita muchos cortes para aproximarla.

Dataset / recursos

- Dataset sintético 1D: `x = np.linspace(-3, 3, 200)`, `y = np.sin(x) + ruido_gaussiano(0, 0.1)` — sirve para ver la escalera y la no-extrapolación.
- Alternativa real: `sklearn.datasets.fetch_california_housing` (regresión, 8 features) para evaluar con `cross_val_score`.
- Géron, Hands-On ML, cap. 6, sección "Regression" (incluye figura 6-5 con la predicción escalonada).

Ejercicios

1. Ajuste básico: generá los datos `sin(x) + ruido`, entrená `DecisionTreeRegressor(max_depth=2)` y `max_depth=5`, y graficá ambas predicciones sobre los puntos. Observá las escaleras.
2. MSE en train vs test: hacé `train_test_split(test_size=0.3)`, calculá `mean_squared_error` en train y test

para `max_depth` {1, 2, 4, 8, None}. ¿Dónde empieza el sobreajuste?

3. No extrapolación: predecí con el modelo entrenado sobre `x_new = np.linspace(-5, 5, 200)` (rango más ancho que el train). Graficá: vas a ver mesetas planas en los extremos.
4. Árbol vs lineal: entrená `LinearRegression` sobre los mismos datos. Comparalo con el árbol fuera del rango de entrenamiento. ¿Cuál extrapola "bien" y por qué?
5. California Housing: corré `cross_val_score(DecisionTreeRegressor(max_depth=6), X, y, scoring='neg_mean_squared_error', cv=5)` y compará contra `max_depth=None`.

Homework verificable

Entregar un script `tarea_073.py` que:

1. Genere $y = 0.5 * x + \sin(x) + \text{ruido}$ con x en $[0, 10]$.
2. Entrene `DecisionTreeRegressor(max_depth=4, random_state=42)` y `LinearRegression`.
3. Prediga sobre `x_new` en $[0, 15]$ (excede el rango de train).
4. Imprima el MSE en test (dentro del rango) y el valor predicho por el árbol en $x=15$.

Criterio de aceptación: el script corre sin error, el MSE del árbol en test es menor al de la lineal dentro del rango, y la predicción del árbol en $x=15$ es idéntica a su predicción en $x=10$ (demuestra no-extrapolación).

Errores comunes

- Esperar una curva suave: el árbol nunca produce una recta ni una curva — siempre escalones. Si necesitás suavidad, usá regresión lineal, polinómica o un ensamble como Gradient Boosting.
- Olvidar regularizar: dejar `max_depth=None` con pocos datos da MSE = 0 en train y MSE altísimo en test. Siempre validar con CV.
- Usar el árbol para forecasting con tendencia: si la serie tiene drift, el árbol queda clavado en el último valor del rango de train. Para tendencia, primero diferenciar o usar otro modelo.
- Confundir el criterio: `criterion='gini'` es para clasificación; en regresión usá `squared_error` o `absolute_error` (este último es más robusto a outliers pero más lento).
- Comparar MSE entre datasets: el MSE no es adimensional. Para comparar entre problemas usá R^2 o RMSE relativo al desvío del target.

Preguntas frecuentes

- ¿Árbol o regresión lineal? Si la relación es monótona y aproximadamente lineal y querés extrapolar, lineal. Si hay no-linealidades, interacciones y umbrales y trabajás dentro del rango de entrenamiento, árbol (o mejor: un ensamble).
- ¿Por qué la predicción es constante por tramos? Porque cada hoja almacena un único número (la media del target del subconjunto que cayó ahí). Toda muestra que termine en esa hoja recibe el mismo valor.
- ¿Puedo usar `absolute_error` en lugar de MSE? Sí, optimiza la mediana por hoja en vez de la media; es más robusto a outliers pero entrena más lento y suele dar splits distintos.
- ¿Cómo elijo `max_depth`? Con validación cruzada sobre una grilla pequeña ([2, 4, 6, 8, 10, None]). En general entre 4 y 10 alcanza para la mayoría de datasets tabulares.
- ¿Sirve un solo árbol en producción? Casi nunca. Su varianza es alta. En la práctica se usa como bloque base de Random Forest o Gradient Boosting (próximas clases).

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, 3ra ed., cap. 6 — sección "Regression".
- scikit-learn: `DecisionTreeRegressor`.

- scikit-learn user guide: Decision Trees — Regression.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 083 — Clase 083 — Voting classifiers: hard y soft

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7. Duración estimada: 45 min.

Objetivo

Combinar varios modelos heterogéneos en un ensemble por votación y entender cuándo conviene hard voting (voto por mayoría) versus soft voting (promedio de probabilidades), apoyándonos en el principio de wisdom of the crowd.

Resultados de aprendizaje

Al finalizar la clase, el estudiante podrá:

- Explicar por qué un ensemble de clasificadores diversos suele superar al mejor modelo individual.
- Implementar un VotingClassifier de scikit-learn con voting="hard" y voting="soft".
- Decidir entre hard y soft voting según si los modelos base estiman probabilidades calibradas.
- Comparar la accuracy del ensemble contra la de cada modelo base en un dataset de clasificación.
- Identificar errores frecuentes al armar el ensemble (modelos correlacionados, probabilidades mal calibradas, pesos sin justificar).

Temas

- Wisdom of the crowd: por qué combinar predicciones reduce error.
- Hard voting: predicción final = clase con más votos entre los modelos base.
- Soft voting: predicción final = clase con mayor promedio de probabilidades estimadas.
- Requisitos para soft voting: que cada modelo exponga predict_proba y esté bien calibrado.
- Diversidad entre modelos base (algoritmos distintos, no sólo hiperparámetros distintos).
- VotingClassifier(estimators=[...], voting=..., weights=...).
- Limitaciones: si los modelos se equivocan en los mismos ejemplos, el ensemble no ayuda.

Definiciones y características

- Hard voting: cada clasificador del ensemble emite una predicción discreta y se elige la clase con más votos (moda). No requiere predict_proba.
- Soft voting: se promedian las probabilidades estimadas por cada modelo (opcionalmente ponderadas) y se elige la clase con mayor probabilidad promedio. Suele rendir mejor que hard voting cuando las probabilidades son confiables.
- VotingClassifier (sklearn.ensemble): meta-estimador que envuelve una lista de (nombre, estimador) y expone fit, predict, predict_proba (sólo si voting="soft").
- Weak learner / strong learner: un weak learner apenas supera el azar; al combinar muchos weak learners diversos se obtiene un strong learner. Ley de los grandes números aplicada a clasificadores independientes.
- Wisdom of the crowd: si los errores de los modelos son independientes, la probabilidad de que la mayoría se equivoque cae exponencialmente con el número de modelos.
- Diversidad: condición clave para que el ensemble funcione. Se logra usando algoritmos distintos

(logística, SVM, árbol, kNN, etc.), no copias del mismo modelo con seeds distintos.

- Calibración: un modelo está calibrado si `predict_proba` $\approx$ frecuencia empírica. SVM con `probability=True` y árboles puros suelen estar mal calibrados, lo que degrada el soft voting.

Dataset / recursos

- Dataset sugerido: `make_moons(n_samples=500, noise=0.30, random_state=42)` (mismo que usa Géron en el cap. 7).
- Alternativa: `load_breast_cancer()` de `sklearn.datasets` para un caso real.
- Imports clave: `LogisticRegression`, `RandomForestClassifier`, `SVC`, `VotingClassifier`, `train_test_split`, `accuracy_score`.

Ejercicios

1. Cargar `make_moons` y partir en `train/test` (80/20, `random_state=42`). Entrenar por separado `LogisticRegression`, `RandomForestClassifier` y `SVC(probability=True)`. Reportar `accuracy` de cada uno.
2. Armar un `VotingClassifier` con esos tres modelos y `voting="hard"`. Comparar `accuracy` contra cada modelo base.
3. Repetir con `voting="soft"` (recordá `SVC(probability=True)`). ¿Mejora? ¿Por qué?
4. Probar `weights=[1, 2, 1]` favoreciendo al Random Forest. ¿Cómo cambia la performance? Justificar.
5. Reemplazar uno de los modelos por una copia casi idéntica de otro (ej.: dos regresiones logísticas con C muy parecido). Observar y explicar por qué el ensemble deja de ganar.

Homework verificable

Entregar un script `voting.py` que:

1. Cargue `make_moons(n_samples=500, noise=0.30, random_state=42)`.
2. Entrene tres modelos base distintos y un `VotingClassifier` en modo `hard` y otro en modo `soft`.
3. Imprima la `accuracy` en test de los cinco (3 base + 2 ensembles).

Criterio de aceptación: el `VotingClassifier` en modo `soft` debe alcanzar `accuracy` $\geq$ que el mejor modelo base individual sobre el test split. Si no lo logra, justificar en un comentario qué modelo está rompiendo la diversidad o la calibración.

Errores comunes

- Usar `voting="soft"` con modelos no calibrados (ej.: `SVC` sin `probability=True`, o árboles muy profundos). Las probabilidades estimadas son ruido y el promedio empeora la predicción.
- Olvidar `probability=True` en `SVC`: sin eso, `SVC` no expone `predict_proba` y `voting="soft"` falla con `AttributeError`.
- Modelos base correlacionados (tres árboles, o tres regresiones logísticas con hiperparámetros parecidos): no hay diversidad, el ensemble no aporta nada.
- Pesos arbitrarios en `weights=` sin validación cruzada que los respalde. Mejor empezar con pesos iguales y ajustar con evidencia.
- No fijar `random_state` en los modelos estocásticos: los resultados no son reproducibles y dificultan la comparación.

Preguntas frecuentes

- ¿Soft voting es siempre mejor que hard? No. Es mejor si los modelos base están bien calibrados. Con modelos mal calibrados, hard voting puede ganarle.
- ¿Cuántos modelos conviene meter? Géron muestra que con 3-5 modelos diversos ya hay ganancia notoria. Más allá, los rendimientos decrecen.

- ¿Puedo mezclar modelos lineales y no lineales? Sí, y es lo recomendado: la diversidad de sesgos inductivos es justo lo que hace funcionar al ensemble.
- ¿Diferencia con bagging? Bagging usa el mismo algoritmo entrenado en distintos bootstraps. Voting usa algoritmos distintos sobre el mismo dataset. Se cubre en la clase 075.
- ¿Y si un modelo base es mucho peor que los demás? Suele convenir sacarlo o bajarle el peso; arrastra al ensemble, sobre todo en hard voting.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3ª ed.), cap. 7 — sección "Voting Classifiers".
- scikit-learn — VotingClassifier.
- scikit-learn — Ensemble methods: voting classifier.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 084 — Clase 084 — Bagging y pasting

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7. Duración estimada: 55 min.

Objetivo

Que el alumno entrene ensembles entrenando el mismo algoritmo sobre distintos subsets del training set —bagging (con reemplazo) y pasting (sin reemplazo)— y evalúe sin held-out usando out-of-bag (OOB). Cierre con sampling de features (random patches y random subspaces) como puente conceptual a Random Forests.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Distinguir bagging vs pasting y justificar cuándo elegir uno u otro.
2. Entrenar un BaggingClassifier de scikit-learn con un base estimator y n_estimators razonable.
3. Usar oob_score=True para estimar el error de generalización sin tocar el test set.
4. Aplicar sampling de features (max_features < 1.0, bootstrap_features=True) para random patches / random subspaces.
5. Comparar bias/variance del ensemble vs un único árbol, en accuracy y en frontera de decisión.

Temas

#	Tema	Por qué importa
1	Bagging (bootstrap aggregating)	Reduce varianza promediando predictores en
2	Pasting	Igual idea pero sin reemplazo — útil cuand
3	BaggingClassifier API	estimator, n_estimators, max_samples, boot
4	OOB evaluation	Cada predictor ve ~63% de las instancias;
5	Random patches	Sampling de instancias y features.
6	Random subspaces	Sampling solo de features (bootstrap=False
7	Paralelización con n_jobs=-1	Los predictores son independientes — escal

Definiciones y características

Bagging (bootstrap aggregating)

: Entrenar el mismo algoritmo sobre múltiples muestras con reemplazo del training set y agregar predicciones por voto mayoritario (clasificación) o promedio (regresión). Reduce varianza sin aumentar bias.

Pasting

: Igual que bagging pero el sampling es sin reemplazo. Cada instancia aparece a lo sumo una vez por predictor. Bagging suele ganar porque introduce más diversidad.

Bootstrap

: Sampling con reemplazo del mismo tamaño que el original. En promedio cubre ~63.2% de las instancias únicas; el resto son repetidas.

Out-of-bag (OOB) score

: Para cada predictor, las instancias no muestreadas (~37%) actúan como validation set. Promediando OOB scores se obtiene una estimación del error de generalización sin separar test set. Activado con `oob_score=True`.

BaggingClassifier

: Meta-estimador de sklearn. Params clave: `estimator` (base learner), `n_estimators` (cuántos), `max_samples` (tamaño de cada muestra, default 1.0 = igual al training set), `bootstrap=True` (bagging) / `False` (pasting), `oob_score`, `n_jobs`.

Random patches

: Sampling simultáneo de instancias y features. `bootstrap=True`, `bootstrap_features=True`, `max_features<1.0`. Útil cuando hay muchas features (imágenes, alta dimensionalidad).

Random subspaces

: Sampling solo de features, manteniendo todas las instancias. `bootstrap=False`, `max_samples=1.0`, `bootstrap_features=True`, `max_features<1.0`.

`n_estimators`

: Número de predictores del ensemble. Más estimators = menos varianza, más cómputo. Típico: 100–500. La curva de mejora se aplanará rápido.

Dataset / recursos

`make_moons(n_samples=500, noise=0.30)` de sklearn — frontera no lineal, ideal para visualizar el efecto smoothing del ensemble vs un árbol único.

Ejercicios

1. Bagging vs árbol único. Entrená un `DecisionTreeClassifier` y un `BaggingClassifier(DecisionTreeClassifier(), n_estimators=500, max_samples=100, bootstrap=True)` sobre `make_moons`. Compará accuracy en test.
2. Bagging vs pasting. Repetí (1) con `bootstrap=False`. Reportá diferencia de accuracy y discutí.
3. OOB. Entrená con `oob_score=True`, `bootstrap=True`. Imprimí `bag.oob_score_` y compará con accuracy en test — deberían ser parecidos.
4. Curva de `n_estimators`. Variá `n_estimators` {1, 10, 50, 100, 500, 1000}. Plotteá accuracy test vs `n_estimators`.
5. Random subspaces. Sobre `load_digits()` (64 features), entrená con `bootstrap=False`, `max_samples=1.0`,

`bootstrap_features=True, max_features=0.5`. Compará con bagging clásico.

Homework verificable

Notebook que: (a) cargue `make_moons(500, noise=0.30)`, split 80/20; (b) entrene árbol único y `BaggingClassifier(n_estimators=500, max_samples=100, oob_score=True, n_jobs=-1)`; (c) reporte accuracy test de ambos y `oob_score_` del bag; (d) grafique las dos fronteras de decisión lado a lado; (e) repita con `bootstrap=False` (pasting) y compare en 1 párrafo.

Criterio de aceptación: Bagging supera al árbol único en test. `oob_score_` $\approx$ accuracy test (diferencia < 0.05). La frontera del bag es visiblemente más suave.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>ValueError: Out of bag estimation only ava</code>	Pediste <code>oob_score=True</code> con <code>bootstrap=False</code>
<code>n_estimators=10</code> y el ensemble no mejora al	Pocos predictores — la varianza no se prom
<code>BaggingClassifier</code> lento con árboles profun	Cada predictor crece sin podar. Fix: <code>n_job</code>
Confundir <code>max_samples</code> con <code>max_features</code>	<code>max_samples</code> sortea filas, <code>max_features</code> sor
OOB score muy distinto al test score	Dataset chico (OOB tiene alta varianza) o

Preguntas frecuentes

¿Bagging o pasting?

Bagging casi siempre. El reemplazo introduce más diversidad entre predictores, que es exactamente lo que reduce la varianza del ensemble. Pasting puede ganar marginalmente si el dataset es enorme y querés evitar repetir instancias, pero la diferencia es chica. Como bonus, bagging te da OOB gratis.

¿Cuántos `n_estimators` uso?

Empezá con 100. Subí a 500 si tenés cómputo. Más allá de eso, retornos decrecientes — la mejora marginal por estimator se aplana.

¿OOB reemplaza al test set?

Reemplaza al validation set durante tuning. Igual conviene reservar un test set final para reportar el número honesto al stakeholder.

¿Bagging sirve con cualquier base estimator?

Sí, pero brilla con learners de alta varianza (árboles profundos sin podar). Con learners de bajo varianza (regresión logística, naive Bayes) el bagging casi no mueve la aguja.

¿Esto es lo mismo que Random Forest?

Casi. Random Forest = bagging de árboles + sampling de features en cada split (no por árbol). Lo vemos en la clase siguiente.

Referencias

- Géron, Hands-On ML, cap. 7 § "Bagging and Pasting", "Out-of-Bag Evaluation", "Random Patches and Random Subspaces".
- sklearn `BaggingClassifier`
- sklearn user guide — Bagging
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 085 — Clase 085 — Random Forests y Extra Trees

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7. Duración estimada: 60 min.

Objetivo

Entender Random Forests y Extra Trees como ensambles de árboles decorrelacionados, diferenciar sus mecanismos de aleatoriedad y elegir hiperparámetros razonables para problemas reales de clasificación y regresión.

Resultados de aprendizaje

Al terminar la clase vas a poder:

1. Explicar por qué un Random Forest reduce varianza respecto a un árbol único, usando los conceptos de bagging y subsampling de features.
2. Diferenciar Random Forest vs Extra Trees (thresholds óptimos vs aleatorios) y argumentar cuándo conviene cada uno.
3. Configurar los hiperparámetros clave (`n_estimators`, `max_features`, `max_depth`, `bootstrap`, `min_samples_leaf`) con criterios fundados.
4. Entrenar y comparar `RandomForestClassifier` y `ExtraTreesClassifier` de scikit-learn en un dataset tabular, midiendo accuracy y tiempo.
5. Interpretar `oob_score_` como estimación honesta del error de generalización sin necesidad de validación cruzada.

Temas

- Repaso de bagging: bootstrap + agregación → reducción de varianza.
- Random Forest: bagging de árboles + subsampling de features en cada split.
- Extra Trees (Extremely Randomized Trees): thresholds aleatorios en lugar de óptimos.
- Hiperparámetros: `n_estimators`, `max_features`, `max_depth`, `min_samples_leaf`, `bootstrap`, `oob_score`.
- Out-of-Bag (OOB) score y su uso como reemplazo de CV.
- Comparativa empírica: bias, varianza, tiempo de entrenamiento.

Definiciones y características

- Random Forest: ensamble de árboles de decisión entrenados sobre muestras bootstrap del dataset; en cada split solo se considera un subconjunto aleatorio de features. La predicción final es voto mayoritario (clasificación) o promedio (regresión).
- Extra Trees: variante de Random Forest donde, para cada feature candidata en un split, el threshold se elige al azar en lugar de buscar el óptimo. Esto añade más aleatoriedad → mayor reducción de varianza a costa de un poco más de sesgo, y es más rápido de entrenar.
- `n_estimators`: cantidad de árboles. Más árboles → mejor performance y mayor estabilidad, pero rendimiento decreciente. Valores típicos: 100–500.
- `max_features`: cantidad de features consideradas por split. Default `sqrt(n_features)` para clasificación, `n_features` (o 1.0) para regresión. Bajarlo aumenta decorrelación entre árboles.
- `bootstrap`: si True (default en RF), cada árbol se entrena sobre una muestra bootstrap; si False (default en ExtraTrees), sobre todo el dataset. Solo con `bootstrap=True` se puede calcular OOB.

- Feature subsampling: técnica clave que diferencia RF de un simple bagging de árboles; obliga a cada árbol a explorar diferentes combinaciones, decorrelacionando sus errores.
- Decorrelación: cuanto menos correlacionados estén los errores de los árboles, mayor la reducción de varianza del ensamble (el promedio de variables correlacionadas reduce varianza más lento que el de independientes).
- OOB score (oob_score_): precisión calculada sobre las muestras que no entraron en el bootstrap de cada árbol; estima generalización sin gastar un split de validación.

Dataset / recursos

- Dataset principal: `sklearn.datasets.load_breast_cancer()` (569 muestras, 30 features, binario).
- Dataset secundario (regresión): `sklearn.datasets.fetch_california_housing()`.
- Librerías: `scikit-learn >= 1.3`, `numpy`, `pandas`, `matplotlib`.
- Notebook: `notebook.ipynb` con ejemplos guiados.

Ejercicios

1. Baseline: entrená un `DecisionTreeClassifier` único sobre `load_breast_cancer` con `random_state=42` y reportá `accuracy` en test (split 80/20).
2. Random Forest: entrená un `RandomForestClassifier(n_estimators=200, random_state=42)` y compará `accuracy` y tiempo contra el árbol único.
3. Extra Trees: entrená un `ExtraTreesClassifier(n_estimators=200, random_state=42)` y compará contra RF en `accuracy` y tiempo de entrenamiento.
4. OOB: entrená un RF con `oob_score=True`, `bootstrap=True` y compará `oob_score_` contra el `accuracy` de test; ¿se parecen?
5. Sensibilidad a `max_features`: variá `max_features` en `[1, 'sqrt', 'log2', 0.5, 1.0]` y graficá `accuracy` de CV; identificá el óptimo.

Homework verificable

Sobre `fetch_california_housing`:

1. Entrená `RandomForestRegressor` y `ExtraTreesRegressor` con `n_estimators=300`, `random_state=42`.
2. Reportá R^2 en test (split 80/20) y tiempo de entrenamiento de cada uno.
3. Tuneá `max_features` con `GridSearchCV (cv=3)` sobre `[0.3, 0.5, 0.7, 1.0]` para el mejor de los dos.

Criterio de aprobación: el mejor modelo tuneado alcanza $R^2 \geq 0.80$ en test, y entregás una tabla comparativa con `accuracy`, tiempo y `max_features` óptimo.

Errores comunes

- Confundir `bootstrap=False` con desactivar el ensamble: los árboles siguen siendo distintos por el subsampling de features y, en `ExtraTrees`, por los thresholds aleatorios.
- Pedir `oob_score=True` con `bootstrap=False`: `scikit-learn` lanza error; el OOB requiere muestras fuera del bootstrap.
- Inflar `n_estimators` sin medir: pasar de 500 a 2000 árboles rara vez mueve la aguja y multiplica tiempo/memoria. Medí la curva.
- No fijar `random_state`: hace que los resultados no sean reproducibles entre corridas; siempre fijalo en ejercicios y benchmarks.
- Usar `max_features=1.0` en clasificación: anula el subsampling de features y degrada la decorrelación; quedate con 'sqrt' salvo evidencia en contra.

Preguntas frecuentes

- ¿RF o `ExtraTrees`? Si tenés mucho ruido en los features o el dataset es grande y el tiempo importa,

probá ExtraTrees (más rápido, más reducción de varianza). Si querés OOB score o el dataset es chico/limpio, RF suele ser un poco mejor. En la práctica: probá los dos, la diferencia suele ser $< 1\%$.

- ¿Cuántos árboles necesito? Empezá con 100, subí a 300–500 si ves margen. Más allá, ganancias marginales.
- ¿Random Forest sufre overfitting? Mucho menos que un árbol único, pero sí puede sobreajustar con datasets muy chicos o con árboles demasiado profundos sin regularización (`min_samples_leaf`).
- ¿Sirven para features categóricas? Sí, pero scikit-learn requiere encoding previo (OneHot o Ordinal). Otros frameworks (LightGBM, CatBoost) los manejan nativamente.
- ¿Por qué el OOB es una buena estimación? Cada muestra queda fuera de $\sim 37\%$ de los árboles (por la matemática del bootstrap), lo que da un estimador casi insesgado del error de generalización, gratis.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3ra ed.), cap. 7 — Ensemble Learning and Random Forests, sección "Random Forests".
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely Randomized Trees. Machine Learning, 63(1), 3–42.
- Documentación scikit-learn: RandomForestClassifier y ExtraTreesClassifier.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 086 — Clase 086 — Feature importance

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7 + sklearn permutation_importance docs.

Duración estimada: 70 min.

Objetivo

Aprender a medir y comunicar qué variables aportan a un modelo basado en árboles, distinguiendo entre Mean Decrease in Impurity (MDI) —el `feature_importances_` por default de scikit-learn— y permutation importance, entendiendo los sesgos de cada método. Como complemento, conocer las herramientas modernas de interpretabilidad SHAP y LIME para explicar predicciones individuales.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Calcular `feature_importances_` en un RandomForestClassifier / GradientBoostingRegressor y explicar qué mide MDI.
- Aplicar `sklearn.inspection.permutation_importance` sobre el set de validación y comparar el ranking con MDI.
- Identificar los sesgos de MDI (favorece features de alta cardinalidad y continuas) y de permutation (problemas con features correlacionadas).
- Generar explicaciones locales con `shap.TreeExplainer` e interpretar `summary_plot` y `waterfall_plot`.
- Decidir cuándo usar MDI, permutation, SHAP o LIME según contexto (auditoría, debugging, comunicación).

Temas

- Recordatorio: cómo los árboles eligen splits (impurity / variance reduction).
- MDI: suma ponderada de la reducción de impureza por feature, promediada sobre los árboles.
- Permutation importance: caída del score al permutar aleatoriamente los valores de una feature.
- Sesgos: cardinalidad alta infla MDI; correlación infla/desinfla permutation.
- MDI se calcula sobre train (riesgo de overfitting); permutation se calcula sobre test/valid.
- Interpretabilidad global vs local.
- Complemento moderno: SHAP, LIME, PDP, ICE.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 077a — SHAP en profundidad: TreeExplainer, KernelExplainer, DeepExplainer

Definiciones y características

- `feature_importances_`: atributo de los estimadores tree-based de sklearn que devuelve un array normalizado (suma 1) con la importancia MDI de cada feature.
- MDI (Mean Decrease in Impurity): para cada feature, suma de las reducciones de impureza (Gini/entropy/MSE) en cada split que la usa, ponderadas por la fracción de muestras que pasan por ese nodo, promediado sobre todos los árboles del ensemble.
- Sesgo MDI por cardinalidad: features con muchos valores únicos (IDs, continuas) ofrecen más splits candidatos y suelen aparecer infladas aunque no tengan poder predictivo real.
- Permutation importance: diferencia entre el score del modelo con la columna original y el score tras permutar aleatoriamente esa columna. Se calcula sobre datos no vistos (test/valid).
- SHAP value: contribución de una feature a la predicción de una instancia específica, promediada sobre todos los órdenes posibles de inclusión (Shapley values de teoría de juegos).
- LIME: aproximación local con un modelo interpretable (regresión lineal/árbol pequeño) entrenado sobre perturbaciones de la instancia ponderadas por cercanía.
- PDP (Partial Dependence Plot): efecto marginal promedio de una o dos features sobre la predicción, marginalizando sobre el resto.
- ICE (Individual Conditional Expectation): como PDP pero una curva por instancia, revela heterogeneidad oculta por el promedio.

Dataset / recursos

- Dataset: `sklearn.datasets.fetch_california_housing` (regresión) y/o `load_breast_cancer` (clasificación).
- Librerías: `scikit-learn`, `shap`, `lime`, `matplotlib`.
- Instalación: `pip install shap lime`.

Ejercicios

1. Entrená un `RandomForestRegressor` sobre California Housing. Imprimí `feature_importances_` ordenado y graficalo como barh.
2. Calculá `permutation_importance` sobre el set de test con `n_repeats=10`. Comparalo con MDI en un DataFrame lado a lado. ¿Coincide el top-3?
3. Agregá una columna `random_id = np.arange(len(X))` y reentrená. Mostrá cómo MDI le asigna importancia espuria mientras permutation la ignora.
4. Generá `shap.summary_plot` con `TreeExplainer` y explicá la diferencia entre el plot tipo beeswarm (impacto + dirección) y un bar plot de MDI.
5. Elegí una instancia mal clasificada (o con error grande en regresión) y explicala con `shap.waterfall_plot`. Anotá las 3 features que más empujaron la predicción.

Homework verificable

Sobre el dataset `load_breast_cancer`, entrená un `RandomForestClassifier` y entregá un notebook que:

1. Reporte el top-5 de features según MDI y según permutation (sobre test).
2. Genere `shap.summary_plot` y guarde el PNG.
3. Explique con SHAP una predicción correcta y una incorrecta (`waterfall_plot`).

Criterio de aceptación: los rankings MDI y permutation pueden diferir, pero el alumno debe justificar la diferencia en 2-3 líneas mencionando cardinalidad/correlación. El SHAP debe mostrar que las contribuciones suman (aproximadamente) la diferencia entre `expected_value` y la predicción.

Errores comunes

1. Interpretar MDI con features de alta cardinalidad: IDs, fechas como int, o continuas con muchos decimales aparecen infladas. Siempre validá con permutation.
2. Calcular permutation importance sobre train: si el modelo overfittea, el ranking es inútil. Siempre sobre test/valid.
3. Confiar en permutation con features correlacionadas: si dos features llevan info redundante, permutar una sola subestima ambas (el modelo se apoya en la otra). Considerá agrupar features correlacionadas.
4. Usar `KernelExplainer` de SHAP sobre un Random Forest: lento e innecesario. Usá `TreeExplainer`, que es exacto y ~100x más rápido.
5. Reportar feature importance sin escalado/contexto: un valor de 0.15 no significa nada si no decís contra qué se compara. Mostrá ranking o porcentajes acumulados.

Preguntas frecuentes

1. ¿SHAP o LIME? SHAP si trabajás con tabular y querés rigor (especialmente con árboles, por `TreeExplainer`). LIME si necesitás explicar texto/imagen o un modelo donde SHAP es prohibitivamente lento.
2. ¿Por qué MDI suma 1 y permutation no? MDI está normalizado por construcción; permutation devuelve la caída absoluta de score, que depende de la métrica usada.
3. ¿Puedo usar feature importance para selección de features? Sí, pero con cuidado: usá permutation sobre validación, no MDI sobre train. Mejor aún: `SelectFromModel` o RFE con CV.
4. ¿Qué pasa si dos features están perfectamente correlacionadas? MDI las reparte arbitrariamente; permutation puede mostrar ambas como poco importantes. SHAP también distribuye entre ellas. Detectalas con `corr()` antes de entrenar.
5. ¿SHAP funciona con XGBoost/LightGBM/CatBoost? Sí, los tres tienen integración nativa con `TreeExplainer`. De hecho, XGBoost y LightGBM exponen `pred_contribs` que son SHAP values calculados internamente.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3rd ed.), cap. 7 "Ensemble Learning and Random Forests" — sección "Feature Importance".
- scikit-learn docs: Permutation feature importance y `partial_dependence`.
- SHAP docs: <<https://shap.readthedocs.io/>> — Lundberg & Lee (2017), A Unified Approach to Interpreting Model Predictions, NeurIPS.
- LIME paper: Ribeiro, Singh & Guestrin (2016), "Why Should I Trust You?": Explaining the Predictions of Any Classifier, KDD. Repo: <<https://github.com/marcotcr/lime>>.
- Molnar, C. Interpretable Machine Learning (libro online gratuito): <<https://christophm.github.io/interpretable-ml-book/>>.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 087 — Clase 087 — SHAP en profundidad: TreeExplainer, KernelExplainer, DeepExplainer

Parte: 1 — Machine Learning Clásico · Fuente: Lundberg & Lee (2017) + shap docs. Duración estimada: 90 min.

Objetivo

Dominar SHAP (SHapley Additive exPlanations) en profundidad: teoría de Shapley values (teoría de juegos cooperativos), TreeExplainer (rápido y exacto para árboles), KernelExplainer (model-agnostic, lento), DeepExplainer (para NN), y los plots clave: summary_plot, waterfall_plot, force_plot, dependence_plot, decision_plot.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar Shapley value intuitivamente: contribución marginal promedio sobre todos los órdenes de inclusión.
- Aplicar TreeExplainer en XGBoost/LightGBM/RF (segundos para millones de samples).
- Generar e interpretar los 5 plots SHAP principales.
- Diferenciar explicación global (summary_plot beeswarm) de local (waterfall de una predicción).
- Reconocer las limitaciones: SHAP asume cierta forma de "feature attribution" pero no es causal.

Temas

- Shapley value: $\phi_i = \sum \frac{|S|!(M-|S|-1)!}{M!} \cdot [v(S \cup \{i\}) - v(S)]$.
- 4 propiedades únicas: efficiency, symmetry, dummy, additivity.
- TreeExplainer: exacto para tree-based, $O(TLD^2)$.
- KernelExplainer: LIME-style con kernel especial → SHAP values aproximados.
- DeepExplainer: para Keras/PyTorch.
- Permutation explainer: alternativa moderna sin necesidad de árbol/red.

Definiciones y características

- Shapley value: única atribución que satisface las 4 propiedades.
- shap_values: matriz (n_samples, n_features) con contribución de cada feature a cada predicción.
- expected_value: predicción promedio del modelo (baseline). $\sum \text{shap_values}[i] + \text{expected_value} \approx \text{predicción}[i]$.
- summary_plot beeswarm: para cada feature, distribución de SHAP values; color = valor de la feature.
- waterfall_plot: para una predicción específica, contribución acumulada feature por feature.
- dependence_plot: feature en x, SHAP value en y; revela no-linearidades e interacciones.

Dataset / recursos

- fetch_california_housing o load_breast_cancer.
- Librerías: shap (pip install shap), xgboost, matplotlib.

Ejercicios

1. TreeExplainer: XGBoost en California Housing → explainer = shap.TreeExplainer(model);

shap_values = explainer(X_test).

2. Summary plot: shap.summary_plot(shap_values, X_test). Identificar las 3 features más importantes y su dirección.
3. Waterfall: elegir 1 muestra concreta → shap.waterfall_plot(shap_values[0]). Sumar contribuciones y verificar que reconstruye la predicción.
4. Dependence plot: shap.dependence_plot('MedInc', shap_values.values, X_test). Detectar non-linearity.
5. Interaction values: shap_interaction = explainer.shap_interaction_values(X_test). Identificar par de features con mayor interacción.

Homework verificable

XGBoost sobre California Housing + análisis SHAP completo:

1. Modelo entrenado.
2. SHAP values con TreeExplainer.
3. Summary plot + force_plot de 3 predicciones (alta, media, baja).
4. Dependence plots para top-3 features.
5. Reporte de 1 página: insights de qué mueve el precio (en lenguaje no técnico).

Criterio de aceptación: el reporte identifica correctamente las features dominantes; las explicaciones individuales suman al valor del modelo (± 0.01).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
KernelExplainer sobre tree → muy lento	Usa el equivocado. Fix: TreeExplainer.
Asumir SHAP es causal	No lo es — atribución es estadística, no c
Interpretar SHAP sobre train data	Sesgo por overfit. Fix: explicar sobre val
force_plot no renderiza en Jupyter	Sin shap.initjs(). Fix: ejecutar al inicio
Features muy correlacionadas → SHAP distri	Limitación inherente. Fix: agrupar correla

Preguntas frecuentes

SHAP vs LIME?

SHAP: fundamento teórico, exacto en tree-based, determinista. LIME: model-agnostic, rápido pero inestable. SHAP gana hoy.

¿explainer(X) o explainer.shap_values(X)?

API moderna (≥ 0.40): explainer(X) devuelve Explanation object. Compatible con todos los plots. Recomendado.

¿SHAP para clasificación multiclase?

shap_values es lista/tensor con valores por clase. Plot por clase con shap.summary_plot(shap_values[class_idx]).

¿SHAP en LLM?

Existe pero costoso por la combinatoria. Para LLMs se usan otras técnicas (attention rollout, integrated gradients).

¿En producción reporto SHAP por predicción?

Sí — para decisiones high-stake (crédito, medicina), el SHAP por predicción ayuda al human-in-the-loop a entender qué pasó.

Referencias

- Lundberg & Lee (2017), A Unified Approach to Interpreting Model Predictions, NeurIPS.
- Lundberg et al. (2020), From local explanations to global understanding with explainable AI for trees, Nature Machine Intelligence.
- SHAP docs.
- Molnar, C. Interpretable Machine Learning — cap. SHAP.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 088 — Clase 088 — Boosting: AdaBoost y Gradient Boosting

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7. Duración estimada: 70 min.

Objetivo

Entender la idea de boosting como combinación secuencial de aprendices débiles, dominar los dos enfoques clásicos —AdaBoost (reponderar errores) y Gradient Boosting (ajustar al residuo)— y saber tunear `learning_rate`, `n_estimators` y aplicar early stopping con `staged_predict`.

Resultados de aprendizaje

Al terminar la clase vas a poder:

1. Explicar la diferencia conceptual entre bagging (paralelo, varianza) y boosting (secuencial, sesgo).
2. Entrenar un `AdaBoostClassifier` y un `GradientBoostingClassifier` de `scikit-learn`, leyendo correctamente sus hiperparámetros.
3. Entender por qué `learning_rate` y `n_estimators` se mueven en sentidos opuestos (`shrinkage`).
4. Implementar early stopping en boosting usando `staged_predict` sobre un set de validación.
5. Decidir cuándo conviene AdaBoost, cuándo Gradient Boosting clásico y cuándo saltar directo a las librerías de la 079.

Temas

- Intuición: muchos clasificadores débiles → uno fuerte.
- AdaBoost: peso a las muestras mal clasificadas; SAMME y SAMME.R.
- Gradient Boosting: cada árbol ajusta el residuo (gradiente de la loss) del ensemble previo.
- Hiperparámetros clave: `n_estimators`, `learning_rate`, `max_depth`, `subsample` (stochastic gradient boosting).
- `staged_predict` / `staged_predict_proba`: predicciones intermedias para early stopping.
- Curvas de error vs. número de estimadores: detectar el "codo".
- Limitaciones: entrenamiento secuencial (no se paraleliza tan bien como bagging).

Definiciones y características

- Boosting: meta-algoritmo que entrena modelos en serie, donde cada uno corrige los errores del anterior. Reduce sesgo (a diferencia del bagging, que reduce varianza).
- AdaBoost (Adaptive Boosting): en cada iteración aumenta el peso de las muestras mal clasificadas y

entrena un nuevo weak learner sobre la distribución reponderada. La predicción final es un voto ponderado.

- Gradient Boosting: en cada iteración entrena un árbol nuevo para predecir el residuo (gradiente negativo de la loss) del ensemble actual. Generaliza AdaBoost a cualquier función de pérdida diferenciable.
- learning_rate (η): factor de shrinkage que escala la contribución de cada árbol nuevo. Valores chicos (0.01–0.1) regularizan; obligan a usar más `n_estimators`.
- `n_estimators`: cantidad de aprendices débiles. En boosting puede haber sobreajuste si crece demasiado (a diferencia de Random Forest).
- Weak learner: modelo apenas mejor que el azar. En sklearn, por defecto, un árbol de profundidad 1 (decision stump) para AdaBoost y profundidad 3 para Gradient Boosting.
- Residuo: diferencia entre el target real y la predicción acumulada del ensemble; lo que falta por explicar.
- `staged_predict`: iterador que devuelve la predicción del ensemble usando 1, 2, ... N estimadores. Sirve para graficar la curva de error y detectar el número óptimo.
- Shrinkage: técnica de regularización que multiplica cada contribución por `learning_rate < 1`; baja el riesgo de overfitting al costo de más iteraciones.

Dataset / recursos

- `sklearn.datasets.make_moons(n_samples=500, noise=0.30, random_state=42)` para la parte visual.
- `sklearn.datasets.load_breast_cancer()` para comparar AdaBoost vs. Gradient Boosting en un problema real.
- `sklearn.ensemble.AdaBoostClassifier`, `GradientBoostingClassifier`, `GradientBoostingRegressor`.
- Géron, cap. 7, sección "Boosting" (AdaBoost + Gradient Boosting + Early Stopping).

Ejercicios

1. AdaBoost desde cero conceptual: entrená un `AdaBoostClassifier(n_estimators=200, learning_rate=0.5)` sobre `make_moons`. Graficá la frontera de decisión con 1, 10, 50 y 200 estimadores.
2. Gradient Boosting paso a paso: usá `GradientBoostingRegressor(max_depth=2, n_estimators=3, learning_rate=1.0)` sobre datos sintéticos $y = x^2 + \text{ruido}$. Mostrá las predicciones intermedias entrenando 3 árboles a mano sobre residuos sucesivos y verificá que coinciden con el ensemble.
3. Trade-off learning_rate $\leftrightarrow$ n_estimators: comparé ($lr=1.0$, $n=50$) vs. ($lr=0.1$, $n=500$) vs. ($lr=0.01$, $n=5000$) en `breast_cancer`. Reportá accuracy y tiempo de fit.
4. Early stopping con `staged_predict`: entrené `GradientBoostingClassifier(n_estimators=500, learning_rate=0.05)` y, usando `staged_predict` sobre validación, encontrá el `n_estimators` óptimo. Re-entrené con ese valor.
5. Stochastic Gradient Boosting: agregale `subsample=0.5` al modelo de (4). ¿Mejora la generalización? ¿Por qué?

Homework verificable

Sobre `load_breast_cancer()` con `train_test_split(test_size=0.2, random_state=42, stratify=y)`:

1. Entrené `AdaBoostClassifier(n_estimators=200, learning_rate=0.5, random_state=42)` y `GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42)`.
2. Para el Gradient Boosting, encontrá el `best_n` usando `staged_predict_proba` sobre el set de test y `log_loss`.
3. Reportá accuracy de los tres modelos (AdaBoost, GB completo, GB con `best_n`).

Criterio de aprobación: accuracy ≥ 0.95 para el GB con early stopping y `best_n < 200` (debe recortar de

verdad).

Errores comunes

1. Subir `n_estimators` sin bajar `learning_rate`: termina sobreajustando feo. Si subís uno, bajá el otro.
2. Usar árboles profundos en AdaBoost: el algoritmo asume weak learners. Stumps (`max_depth=1`) suelen funcionar mejor que árboles grandes.
3. Comparar boosting con random forest a igualdad de `n_estimators`: no son comparables; en boosting cada árbol depende del anterior.
4. Olvidar `random_state`: tanto AdaBoost como Gradient Boosting son deterministas dado el seed, pero al cambiar la versión de sklearn los defaults se mueven.
5. No escalar... no hace falta: los métodos basados en árboles no necesitan escalado. El error inverso (escalar de más por las dudas) infla el pipeline sin beneficio.

Preguntas frecuentes

1. ¿AdaBoost o Gradient Boosting? En la práctica, Gradient Boosting gana casi siempre en tabular: es más flexible (cualquier loss), maneja mejor el ruido y tiene early stopping natural. AdaBoost queda como curiosidad histórica y para casos muy específicos con pocos features.
2. ¿`learning_rate` alto o bajo? Bajo (0.05–0.1) con muchos estimadores regulariza mejor y suele dar el mejor test score, a costa de tiempo de entrenamiento. Alto (0.5–1.0) entrena rápido pero sobreajusta.
3. ¿Cuántos `n_estimators` poner? Empezá con 500–1000 y dejá que early stopping lo recorte. No es como Random Forest, acá "más" no es siempre mejor.
4. ¿Por qué `staged_predict` y no `validation_fraction` directamente? `GradientBoostingClassifier` ya trae `n_iter_no_change` y `validation_fraction` desde sklearn 0.20; `staged_predict` te sirve para graficar la curva y entender qué pasa, no solo para parar.
5. ¿Es paralelizable? No de forma trivial: cada árbol depende del anterior. Por eso XGBoost / LightGBM / CatBoost (clase 079) usan trucos a nivel de construcción de cada árbol para acelerar.

Referencias

- Géron, Hands-On Machine Learning, 3ra ed., cap. 7 — "Ensemble Learning and Random Forests", sección Boosting.
- scikit-learn user guide: Ensemble methods → Boosting.
- Friedman, J. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics.
- Freund & Schapire (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 089 — Clase 089 — XGBoost, LightGBM y CatBoost

Parte: 1 — Machine Learning Clásico · Fuente: docs XGBoost/LightGBM/CatBoost + Géron, cap. 7.
Duración estimada: 80 min.

Objetivo

Conocer las tres librerías de gradient boosting moderno que dominan tabular ML —XGBoost, LightGBM y CatBoost—, entender sus diferencias algorítmicas (level-wise vs leaf-wise, manejo de categóricas, regularización) y saber elegir la adecuada según el problema, usando sus APIs sklearn-compatibles con

early_stopping_rounds.

Resultados de aprendizaje

Al terminar la clase, vas a poder:

1. Instalar y usar xgboost, lightgbm y catboost con la API sklearn (fit/predict/predict_proba).
2. Explicar la diferencia entre crecimiento level-wise (XGBoost por defecto) y leaf-wise (LightGBM) y sus implicancias en velocidad y overfitting.
3. Configurar early stopping con eval_set y early_stopping_rounds para evitar overfitting y ahorrar tiempo.
4. Manejar features categóricas: encoding manual para XGBoost, categorical_feature en LightGBM, cat_features nativo en CatBoost con ordered boosting.
5. Elegir la librería apropiada según tamaño de dataset, presencia de categóricas y necesidad de velocidad de entrenamiento o inferencia.

Temas

1. Repaso: gradient boosting como ensemble secuencial (de clase 078).
2. XGBoost: histogram-based, level-wise, regularización L1+L2, sparse-aware.
3. LightGBM: leaf-wise growth, GOSS (Gradient-based One-Side Sampling), EFB (Exclusive Feature Bundling).
4. CatBoost: ordered boosting, manejo nativo de categóricas, symmetric trees.
5. Tabla comparativa de los 3.
6. Instalación: pip install xgboost lightgbm catboost.
7. API sklearn unificada: XGBClassifier, LGBMClassifier, CatBoostClassifier.
8. early_stopping_rounds con eval_set.
9. Categorical features en cada uno.
10. Cuándo cada uno: heurísticas prácticas.

Tabla comparativa

Aspecto	XGBoost	LightGBM	CatBoost
Crecimiento del árbol	Level-wise (default)	Leaf-wise	Symmetric (oblivious)
Velocidad entrenamiento	Media	Muy rápida	Media
Velocidad inferencia	Rápida	Rápida	Muy rápida
Categóricas nativas	(requiere encoding)	(índices, ordinal)	(ordered boosting)
Manejo de NaN			
Overfitting en datasets chicos	Bajo	Más riesgo (leaf-wise)	Bajo
Hiperparámetro clave	max_depth	num_leaves	depth
GPU			

Definiciones y características

- XGBoost (eXtreme Gradient Boosting): implementación optimizada de gradient boosting con regularización L1/L2 explícita, manejo de sparsity y construcción de árboles level-wise (todos los nodos de un nivel antes de bajar). Paper de Chen & Guestrin (2016).
- LightGBM: librería de Microsoft (2017) que crece árboles leaf-wise (expande la hoja con mayor pérdida), histogram-based, mucho más rápida en datasets grandes. Más propensa a overfit en datos chicos.
- CatBoost: librería de Yandex (2017) optimizada para features categóricas; usa ordered boosting para

evitar target leakage y symmetric trees (todos los splits del mismo nivel usan la misma condición) que aceleran inferencia.

- Level-wise growth: estrategia de XGBoost; expande todos los nodos de un nivel antes de pasar al siguiente. Árbol balanceado, menos overfitting, más lento.
- Leaf-wise growth: estrategia de LightGBM; expande siempre la hoja con mayor reducción de pérdida. Árbol desbalanceado, más rápido, mayor riesgo de overfit si no se controla num_leaves.
- Histogram-based splitting: bucketea features continuas en max_bin histogramas (típicamente 255) para evaluar splits en $O(\text{bins})$ en lugar de $O(n)$. Usado por los tres.
- GOSS (Gradient-based One-Side Sampling): técnica de LightGBM que retiene todas las instancias con gradiente grande y submuestra aleatoriamente las de gradiente chico, acelerando sin perder precisión.
- EFB (Exclusive Feature Bundling): en LightGBM, agrupa features sparse mutuamente excluyentes en una sola, reduciendo dimensionalidad efectiva.
- Ordered boosting: técnica de CatBoost; para cada muestra usa un modelo entrenado sin esa muestra para calcular su residual, evitando el target leakage que ocurre al encodear categóricas con target mean usando la misma muestra.
- Target leakage en categóricas: cuando se codifica una categoría usando el target de las mismas filas que se entrenan, inflando artificialmente la performance en train; CatBoost lo evita con ordered TS (target statistics).

Dataset / recursos

- sklearn.datasets.fetch_openml('adult') o 'credit-g' — datasets con mezcla de numéricas y categóricas, ideales para comparar las 3 librerías.
- Alternativa: cualquier dataset tabular con >10k filas y columnas categóricas.
- Notebook: notebook.ipynb (no editar — se entrega).

Ejercicios

1. Instalación y smoke test: instalar las 3 librerías, importarlas e imprimir versiones. Confirmar que cargan sin error.
2. XGBoost básico: entrenar XGBClassifier sobre adult (con OneHotEncoder para categóricas), usar eval_set y early_stopping_rounds=20, reportar accuracy en test y la mejor iteración.
3. LightGBM con leaf-wise: entrenar LGBMClassifier pasando categorical_feature con los índices de columnas categóricas (encoding ordinal previo). Comparar tiempo de entrenamiento vs XGBoost.
4. CatBoost nativo: entrenar CatBoostClassifier pasando cat_features con los nombres de columnas — sin encoding manual. Verificar que la accuracy se mantiene o mejora respecto a 2 y 3.
5. Comparativa final: entrenar los 3 modelos sobre el mismo dataset con los mismos splits, reportar en una tabla: accuracy test, tiempo de fit, tiempo de predict y mejor iteración. Concluir cuál elegirías.

Homework verificable

Construí un script boosting_showdown.py que reciba --dataset <nombre openml> y entrene los tres modelos (XGBoost, LightGBM, CatBoost) con early_stopping_rounds=50, los mismos train/test_split(random_state=42), y emita un CSV resultados.csv con columnas: modelo, accuracy_test, tiempo_fit_seg, tiempo_predict_seg, mejor_iter.

Criterio de aprobado:

- Los 3 modelos entrenan sin error sobre al menos un dataset con categóricas.
- CatBoost se entrena sin OneHotEncoder previo (usa cat_features).
- El CSV contiene las 3 filas con valores no nulos.
- En el README del homework, una conclusión de 3 líneas justificando cuál elegirías.

Errores comunes

1. Olvidar `early_stopping_rounds` con `eval_set`: si pasás `eval_set` pero no `early_stopping_rounds`, entrenás las `n_estimators` completas sin parar — desperdiciás tiempo y podés overfittear.
2. Usar `max_depth` alto en LightGBM: como crece leaf-wise, `num_leaves` es el parámetro de control real; `max_depth=-1` (sin límite) con `num_leaves` alto explota en overfit.
3. Pasar strings a XGBoost: XGBoost no maneja categóricas como strings por default (sí desde 1.5 con `enable_categorical=True`, pero limitado); olvidarse y obtener `ValueError: could not convert string to float`.
4. OneHotEncodear para CatBoost: anula su ventaja principal —el ordered boosting—; pasale las categóricas crudas con `cat_features`.
5. Comparar con `n_estimators` distintos: si XGBoost para en 200 y LightGBM en 800 por early stopping, comparar tiempos absolutos sin normalizar es injusto; reportá también iteraciones.

Preguntas frecuentes

1. ¿XGBoost, LightGBM o CatBoost?
 - LightGBM si tenés dataset grande (>100k filas) y querés velocidad.
 - CatBoost si tenés muchas categóricas de alta cardinalidad.
 - XGBoost si necesitás máxima madurez del ecosistema, integración con Spark/Dask, o ya tenés pipeline montado.
1. ¿Cuál gana competencias de Kaggle? Históricamente XGBoost, hoy mezclado: LightGBM y CatBoost ganan terreno. Stacking de los tres suele rendir mejor que cualquiera solo.
1. ¿Son sklearn-compatibles? Sí — todos exponen `XGBClassifier/LGBMClassifier/CatBoostClassifier` con `fit/predict/predict_proba/score`, usables en Pipeline, GridSearchCV, `cross_val_score`.
1. ¿Soportan GPU? Sí los tres. XGBoost con `tree_method='gpu_hist'`, LightGBM con `device='gpu'`, CatBoost con `task_type='GPU'`. Útil con datasets grandes.
1. ¿Cómo elijo `num_leaves` en LightGBM? Regla práctica: `num_leaves < 2^max_depth`. Empezá con 31 (default) y subí gradualmente monitoreando val loss.

Referencias

- XGBoost docs — <<https://xgboost.readthedocs.io/>>
- LightGBM docs — <<https://lightgbm.readthedocs.io/>>
- CatBoost docs — <<https://catboost.ai/docs/>>
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD. <<https://arxiv.org/abs/1603.02754>>
- Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS.
- Prokhorenkova, L. et al. (2018). CatBoost: unbiased boosting with categorical features. NeurIPS. <<https://arxiv.org/abs/1706.09516>>
- Géron, A. Hands-On Machine Learning, cap. 7 (Ensemble Learning — mención a XGBoost).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 090 — Clase 090 — Stacking (stacked generalization)

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 7. Duración estimada: 60 min.

Objetivo

Que el alumno combine modelos heterogéneos vía stacking: entrenar varios modelos base, generar predicciones out-of-fold, y entrenar un meta-modelo (blender) sobre esas predicciones — todo con StackingClassifier / StackingRegressor de sklearn, sin leakage.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar la idea de stacking como ensamble donde un meta-modelo aprende a combinar las predicciones de los base learners.
2. Construir un StackingClassifier con varios estimadores base y un blender (típicamente regresión logística).
3. Justificar las predicciones out-of-fold (CV interna) como mecanismo para evitar que el blender vea predicciones in-sample.
4. Comparar stacking contra voting y contra un solo modelo bien tuneado, en accuracy y costo computacional.
5. Usar passthrough=True para que el blender vea también las features originales, no solo las predicciones de los base.

Temas

#	Tema	Por qué importa
1	Idea: ensamble de dos niveles	El meta-modelo aprende los errores correla
2	Base learners heterogéneos	Diversidad reduce varianza del ensamble.
3	Meta-modelo (blender)	Suele ser simple (logística, lineal) para
4	Out-of-fold predictions	Sin esto hay leakage: el blender ve scores
5	StackingClassifier / StackingRegressor	API sklearn que hace el CV interno por vos
6	passthrough	Concatenar features originales al input de
7	Costo y cuándo conviene	N entrenamientos × K folds — caro, no siem

Definiciones y características

Stacking (stacked generalization)

: Técnica de ensamble donde se entrenan M modelos base sobre el train set; sus predicciones (out-of-fold) se usan como features para entrenar un meta-modelo que aprende a combinarlas. Introducido por Wolpert (1992).

Base learners (nivel 0)

: Los modelos individuales del ensamble. Conviene que sean heterogéneos (p. ej. RandomForest + SVM + KNN) — si todos cometen los mismos errores, stackearlos no agrega información.

Meta-modelo / blender (nivel 1)

: Modelo que toma como input las predicciones de los base learners y produce la predicción final. Suele ser simple (LogisticRegression, Ridge) porque su input ya es altamente predictivo y un modelo complejo overfitearía.

StackingClassifier / StackingRegressor

: Implementación de sklearn (sklearn.ensemble). API: StackingClassifier(estimators=[(nombre, modelo), ...], final_estimator=Logistic(), cv=5). Internamente entrena los base con CV para generar predicciones out-of-fold, después reentrena cada base sobre todo el train, y entrena el blender sobre las predicciones

OOF.

Out-of-fold predictions (OOF)

: Para cada fila del train, su predicción del base learner se genera cuando esa fila estuvo en el fold de validación (no en el de entrenamiento). Esto garantiza que el blender ve predicciones generadas por un modelo que no vio esa fila — evita el leakage.

cv en stacking

: Número de folds para generar las predicciones OOF. Default 5. Más folds → menos varianza en las features del blender, pero más costo (cada base se entrena cv veces).

passthrough=True

: Concatena las features originales X al input del blender, junto con las predicciones de los base. El blender puede así aprovechar tanto los meta-features como las features crudas. Útil cuando los base learners no capturan toda la señal.

Stacking vs voting

: Voting (hard/soft) promedia o vota las predicciones de los base con pesos fijos. Stacking aprende los pesos (y combinaciones no lineales) vía el blender. Más expresivo pero más caro y más propenso a overfit si el blender es complejo.

Dataset / recursos

load_breast_cancer o make_classification(n_samples=2000) de sklearn — suficiente para mostrar el patrón sin tiempos largos de CV.

Ejercicios

1. Stacking básico. Construí un StackingClassifier con tres base learners (RandomForest, SVC con probability=True, KNN) y LogisticRegression como blender. Reportá accuracy con cross_val_score(cv=5).
2. Comparación contra base learners solos. Evaluá los tres base learners individualmente con el mismo CV. ¿El stacking supera al mejor solo? ¿Por cuánto?
3. passthrough=True. Repetí el ejercicio 1 con passthrough=True. ¿Mejora la accuracy? Pensá por qué (el blender ve features originales además de las predicciones).
4. Variando cv. Probá cv=3, cv=5, cv=10 en el stacking. Mirá accuracy y tiempo de entrenamiento. Trade-off típico.
5. Stacking vs Voting. Entrená un VotingClassifier(voting='soft') con los mismos tres base. Compará accuracy contra stacking. Discutí costo computacional.

Homework verifiable

Notebook con make_classification(n_samples=3000, n_features=20, n_informative=10, random_state=42): (a) entrenar 3 base learners individuales y reportar CV-accuracy; (b) entrenar StackingClassifier con esos 3 + LogisticRegression blender; (c) repetir con passthrough=True; (d) entrenar VotingClassifier(voting='soft') con los mismos base; (e) tabla comparativa final con accuracy, std y tiempo de entrenamiento.

Criterio de aceptación: El stacking iguala o supera al mejor base learner individual. La tabla comparativa muestra los 5 modelos (3 base + stacking + voting) con accuracy media $\pm$ std y tiempo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El stacking da peor que el mejor base learner	Base learners poco diversos (todos árboles)
Blender entrenado sobre las mismas predicciones	Pasaste predicciones in-sample al blender
SVC no se puede usar en StackingClassifier	SVC por default no expone probas. Fix: SVC
Stacking tarda 10× más que un solo modelo	Es esperable: $M \text{ base} \times K \text{ folds} + 1 \text{ blender}$
passthrough=True empeora el resultado	El blender se confunde con muchas features

Preguntas frecuentes

¿Stacking vs voting — cuándo cuál?

Voting si querés algo rápido, simple y los base learners son razonablemente parejos: promedia probas (soft) o vota clases (hard) con pesos fijos. Stacking si tenés tiempo de CV y los base learners cometen errores distintos — el blender aprende a explotar esa diversidad. Stacking suele ganar ~1-3 pp de accuracy a costa de 5-10× más cómputo.

¿Qué modelo conviene como blender?

Algo simple y regularizado: LogisticRegression (clasificación) o Ridge (regresión) son la elección estándar. Modelos complejos (RandomForest, XGBoost) como blender suelen overfittear las M predicciones OOF.

¿Cuántos base learners?

3-5 está bien. Más allá, la ganancia marginal cae rápido y el costo escala lineal. Lo importante es que sean heterogéneos (familias distintas), no que sean muchos.

¿Puedo apilar stackings (multinivel)?

Técnicamente sí (Kaggle lo hace), pero la ganancia marginal vs costo es brutal y el riesgo de overfit explota. Para producción: un solo nivel de stacking es el sweet spot.

¿Sirve stacking si tengo poca data?

Mal. Con N chico, las predicciones OOF tienen mucha varianza y el blender no encuentra señal estable. Bajo ese régimen, un modelo solo bien tuneado o un VotingClassifier suelen ganarle.

Referencias

- Géron, cap. 7 § Stacking.
- Wolpert, D. (1992). Stacked Generalization. Neural Networks 5(2).
- sklearn StackingClassifier
- sklearn StackingRegressor
- sklearn user guide — Stacked generalization

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 091 — Clase 091 — La maldición de la dimensionalidad

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 8 § The Curse of Dimensionality.

Duración estimada: 45 min.

Objetivo

Que el alumno entienda por qué los algoritmos basados en distancia (kNN, k-means, SVM-RBF) degradan en alta dimensión: el espacio se vuelve mayormente vacío, las distancias entre puntos colapsan a un mismo valor, y los modelos overfittean. Esto motiva la reducción de dimensionalidad (PCA, manifold learning) que viene en las próximas clases.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Explicar la sparsity exponencial: por qué llenar uniformemente un hipercubo unitario requiere n^d puntos.
2. Calcular numéricamente que en $d=100$ la razón $(d_{\max} - d_{\min}) / d_{\min}$ entre distancias euclidianas tiende a 0.
3. Identificar qué algoritmos sufren la maldición (basados en distancia/densidad) y cuáles menos (árboles, modelos lineales con regularización).
4. Reconocer la manifold hypothesis como justificación de PCA, t-SNE, UMAP: los datos reales viven en un subespacio de baja dimensión.
5. Decidir cuándo reducir dimensionalidad vs. cuándo regularizar o usar otro modelo.

Temas

#	Tema	Por qué importa
1	Intuición geométrica: hipercubo y volumen	En $d=100$, el 99.99% del volumen está pegado
2	Sparsity exponencial	Para cubrir el espacio uniformemente, los
3	Concentración de la medida	Distancias entre puntos aleatorios converg
4	Distancia euclidiana pierde sentido	nearest neighbor deja de ser informativo.
5	Hubness	Algunos puntos aparecen como vecinos de to
6	Manifold hypothesis	Los datos reales no llenan el espacio: viv
7	Implicaciones prácticas para ML	Overfitting, kNN colapsa, k-means inestabl

Definiciones y características

Maldición de la dimensionalidad

: Conjunto de fenómenos contraintuitivos que aparecen al crecer la cantidad de features: el espacio se vuelve mayormente vacío, las distancias se igualan, y la cantidad de datos necesaria para densidad constante crece exponencialmente en d . Acuñada por Bellman (1961).

Sparsity exponencial

: Para mantener una densidad constante de puntos en un hipercubo $[0,1]^d$, hace falta n^d muestras. Con $d=100$ y $n=10$ por eje son 10^{100} puntos — más que átomos en el universo observable. Consecuencia: en alta dimensión todos los datasets son chicos.

Concentración de la medida

: En distribuciones de alta dimensión (uniforme, gaussiana), la distancia entre dos puntos aleatorios se concentra fuertemente alrededor de su media. Formalmente, $\text{Var}(\|X-Y\|) / E[\|X-Y\|]^2 \rightarrow 0$ cuando $d \rightarrow \infty$. Resultado: $d_{\min} \approx d_{\max}$, y la noción de "más cercano" se vuelve ruido.

Distancia euclidiana en alta dimensión

: Pierde poder discriminativo porque acumula varianza por cada feature irrelevante. Alternativas: distancia coseno (escala-invariante), Mahalanobis (decorrela), o reducir antes con PCA.

Hubness

: Fenómeno por el cual, en alta dimensión, ciertos puntos aparecen entre los k vecinos más cercanos de muchísimos otros (hubs), mientras otros nunca son vecinos de nadie (anti-hubs). Rompe el supuesto de simetría que asume kNN.

Manifold hypothesis

: Suposición de que los datos reales de alta dimensión (imágenes, texto, audio) no llenan el espacio ambiente sino que viven en un subespacio (manifold) de dimensión intrínseca mucho menor. Justifica PCA, autoencoders, t-SNE, UMAP. Sin esta hipótesis, reducir dimensionalidad sería destruir información.

Dimensión intrínseca

: Cantidad mínima de variables latentes necesarias para representar los datos sin pérdida apreciable. MNIST tiene $d=784$ features (28×28 pixels) pero dimensión intrínseca estimada $\sim 10-15$.

Dataset / recursos

Sintético: puntos uniformes en $[0,1]^d$ para $d \in \{2, 10, 100, 1000\}$. Permite calcular numéricamente sparsity, $\text{ratio } d_{\text{max}}/d_{\text{min}}$, y fracción de volumen cerca del borde. Para la parte aplicada, `sklearn.datasets.load_digits` ($d=64$) como ejemplo de manifold hypothesis empírica.

Ejercicios

1. Volumen del borde. Calculá la fracción de volumen del hipercubo $[0,1]^d$ que está a menos de 0.01 del borde, para $d=2, 10, 100$. Fórmula: $1 - 0.98^d$.
2. Concentración de distancias. Sampleá 1000 puntos uniformes en $[0,1]^d$. Para $d \in \{2, 10, 100, 1000\}$, calculá $(d_{\text{max}} - d_{\text{min}}) / d_{\text{min}}$ sobre todas las distancias pairwise. Verificá que tiende a 0.
3. Distancia al vecino más cercano. Con $n=1000$ puntos uniformes y d variable, graficá la distancia media al 1-NN. Mostrá que crece con d (el "vecino" está cada vez más lejos).
4. kNN degradada. Generá clasificación sintética con $n=500$, agregando d features de ruido puro (irrelevantes). Evaluá accuracy de `KNeighborsClassifier` para $d = 2, 10, 50, 200$. Curva debería caer.
5. Manifold hipótesis empírica. Cargá `sklearn.datasets.load_digits`. Calculá cuántos componentes PCA explican el 95% de la varianza vs. $d=64$ originales — estimación grosera de dimensión intrínseca.

Homework verificable

Notebook que: (a) genere puntos uniformes en $[0,1]^d$ para $d \in [1, 200]$; (b) calcule y grafique $(d_{\text{max}} - d_{\text{min}})/d_{\text{min}}$ vs d ; (c) entrene `KNeighborsClassifier` con $n=1000$ y features añadidas de ruido $N(0,1)$, reporte accuracy vs d ; (d) sobre `load_digits`, calcule cuántos componentes PCA explican 90%, 95%, 99% de varianza.

Criterio de aceptación: El ratio de distancias tiende a 0 para $d > 50$. Accuracy de kNN cae monotónicamente al agregar ruido. PCA muestra que digits ($d=64$) tiene dimensión intrínseca ≤ 30 al 95%.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Agregué más features y el modelo empeoró"	Features irrelevantes inyectan ruido y emp
kNN con $k=1$ da resultados aleatorios en al	Concentración de medida: $d_{\text{min}} \approx d_{\text{max}}$, "e
"PCA me bajó accuracy"	Te quedaste con muy pocos componentes, o e
Confundir d (features) con n (muestras)	"Tengo 10M filas, no aplica la maldición"

Pensar que árboles también sufren

Random Forest y boosting son mucho más rob

Preguntas frecuentes

¿A partir de qué d empiezo a preocuparme?

Heurística: con kNN/k-means/SVM-RBF, ya con $d > 20-30$ y n modesto se nota la degradación. Con árboles y modelos lineales regularizados, podés escalar a $d = 10000$ sin drama (texto TF-IDF, genómica).

¿Más datos resuelve la maldición?

Solo asintóticamente, y la cantidad necesaria crece exponencialmente. Para $d=100$ no hay suficientes datos en el universo. La salida real es reducir d (PCA, feature selection) o usar modelos que no dependan de densidad global.

¿Por qué deep learning funciona con d enorme (imágenes $224 \times 224 \times 3 = 150k$)?

Por la manifold hypothesis: las imágenes naturales no llenan $\mathbb{R}^{150000}$, viven en un manifold de dimensión intrínseca mucho menor. Las redes profundas aprenden esa estructura jerárquicamente. La maldición sigue aplicando si hacés kNN sobre pixels crudos — y por eso no se hace.

¿Distancia coseno me salva?

Mitiga, no salva. Es invariante a escala y funciona mejor en texto (TF-IDF, embeddings), pero también sufre concentración si las features son ruido puro. Mejor combinar coseno + reducción + selección.

¿Cómo estimo la dimensión intrínseca?

Métodos: (a) PCA + curva de varianza explicada (rápido, lineal); (b) `sklearn.manifold` (Isomap, LLE); (c) estimadores específicos como MLE de Levina–Bickel o el paquete `skdim`. Para empezar, PCA al 95% alcanza como aproximación grosera.

Referencias

- Géron, cap. 8 § The Curse of Dimensionality y § Main Approaches for Dimensionality Reduction.
- Bellman, R. (1961). Adaptive Control Processes: A Guided Tour — origen del término.
- Beyer, K. et al. (1999). When Is "Nearest Neighbor" Meaningful? — paper clásico sobre concentración de distancias.
- Radovanović, M. (2010). Hubs in space: Popular nearest neighbors in high-dimensional data — fenómeno de hubness.
- scikit-learn — Manifold learning

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 092 — Clase 092 — PCA: proyección, varianza explicada, incremental, randomized, kernel

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 8. Duración estimada: 80 min.

Objetivo

Dominar PCA como técnica de reducción de dimensionalidad lineal: entender la proyección al subespacio de máxima varianza vía SVD, elegir el número de componentes con `explained_variance_ratio_`, y aplicar las variantes (Incremental, Randomized, Kernel) según el tamaño y la geometría del dataset.

Resultados de aprendizaje

Al finalizar la clase, el estudiante podrá:

- Explicar geoméricamente qué hace PCA (proyección al hiperplano que maximiza la varianza).
- Ajustar PCA de scikit-learn y leer `components_`, `explained_variance_ratio_` y `singular_values_`.
- Elegir el número de componentes vía umbral de varianza acumulada (ej. 95%) o codo del scree plot.
- Decidir entre PCA, IncrementalPCA y PCA(`svd_solver="randomized"`) según memoria y velocidad.
- Aplicar KernelPCA (RBF/poly) para datasets con estructura no-lineal y tunear gamma con GridSearchCV.

Temas

1. Maldición de la dimensionalidad y motivación de PCA.
2. Proyección al subespacio de máxima varianza: intuición geométrica.
3. SVD como motor algebraico de PCA.
4. Varianza explicada y elección de `n_components` (entero, ratio, "mle").
5. IncrementalPCA para datasets que no entran en RAM (`partial_fit`).
6. Randomized PCA (`svd_solver="randomized"`) para acelerar en alta dimensión.
7. KernelPCA con kernels RBF, polinómico y sigmoide; tuning de gamma.
8. Pipeline completo: StandardScaler → PCA → modelo.

Definiciones y características

- PCA (Principal Component Analysis): técnica lineal que proyecta los datos sobre los ejes ortogonales que capturan máxima varianza.
- Componentes principales: vectores ortonormales (filas de `components_`) que definen el nuevo sistema de coordenadas; el primero apunta a la dirección de máxima varianza, el segundo a la siguiente ortogonal, etc.
- SVD (Singular Value Decomposition): factorización $X = U \Sigma V^T$; las columnas de V son los componentes principales y $\Sigma^2/(n-1)$ da la varianza explicada.
- `explained_variance_ratio_`: array con la fracción de varianza total capturada por cada componente; su suma acumulada guía la elección de `n_components`.
- IncrementalPCA: versión que procesa mini-batches con `partial_fit`; útil cuando X no entra en memoria. Costo: ligeramente más lenta y aproximada.
- Randomized PCA (`svd_solver="randomized"`): algoritmo estocástico que encuentra los primeros d componentes en $O(m \cdot d^2) + O(d^3)$ en vez de $O(m \cdot n^2) + O(n^3)$. Default cuando `n_components << min(m, n)`.
- KernelPCA: aplica PCA en el espacio de features inducido por un kernel (RBF, poly, sigmoid); permite capturar estructura no-lineal (ej. swiss roll).
- gamma: hiperparámetro del kernel RBF que controla el ancho de la gaussiana; valor chico = kernel suave, valor alto = kernel angosto. Se tunea con GridSearchCV.
- Scaling previo: PCA es sensible a la escala — siempre estandarizar (StandardScaler) antes, salvo que todas las variables ya estén en la misma unidad.

Dataset / recursos

- MNIST (`fetch_openml("mnist_784")`) — 70.000 × 784, ideal para mostrar reducción a ~150 componentes preservando 95% de varianza.
- Swiss roll (`sklearn.datasets.make_swiss_roll`) — para contrastar PCA lineal vs KernelPCA RBF.
- Géron, Hands-On ML, cap. 8 — Dimensionality Reduction, sección "PCA".

Ejercicios

1. Cargá MNIST, aplicá StandardScaler + PCA(`n_components=0.95`) y reportá cuántos componentes quedaron.
2. Graficá la curva de varianza acumulada (`cumsum(explained_variance_ratio_)`) y marcá el codo.
3. Compará tiempos de PCA(`svd_solver="full"`) vs "randomized" sobre MNIST con `%timeit`.
4. Usá IncrementalPCA con `n_batches=100` y verificá que el resultado se aproxima al PCA full (cosine similarity entre componentes > 0.99).
5. Sobre `make_swiss_roll(n_samples=1000)`, aplicá KernelPCA(`kernel="rbf"`, `gamma=0.04`, `n_components=2`) y comparalo con PCA lineal en un scatter 2D.

Homework verificable

Construí un pipeline StandardScaler → PCA → LogisticRegression sobre MNIST (subset 10k) y:

1. Reportá accuracy con `n_components=50, 100, 154` ($154 \approx 95\%$ varianza).
2. Mostrá la matriz `pipe.named_steps["pca"].components_.shape` y el `explained_variance_ratio_.sum()`.
3. Entregá notebook + un párrafo explicando el trade-off accuracy vs nº de componentes.

Criterio de aceptación: accuracy ≥ 0.90 con `n_components=100` y curva de varianza acumulada graficada.

Errores comunes

1. No escalar antes de PCA — variables con varianza grande (por unidad, no por información) dominan los componentes. Siempre StandardScaler primero.
2. Usar PCA para reducir dimensionalidad antes de un modelo basado en árboles (RF, XGBoost) — no aporta, los árboles son invariantes a rotaciones de features.
3. Interpretar los componentes como "features originales seleccionadas" — son combinaciones lineales, no subconjuntos.
4. Aplicar `PCA().fit(X_train_and_test)` — fuga de datos. Hacer fit solo sobre train y transform sobre test.
5. Olvidar que KernelPCA no tiene `inverse_transform` por default — hay que pasar `fit_inverse_transform=True`.

Preguntas frecuentes

1. ¿Cuántos componentes elegir? Regla práctica: el menor `d` tal que `cumsum(explained_variance_ratio_)  $\geq 0.95$` . También se puede usar `n_components="mle"` (Minka) o ver el codo del scree plot.
2. ¿PCA mejora siempre la accuracy? No. Reduce ruido y acelera el entrenamiento, pero puede perder información discriminativa. Validá con CV.
3. ¿Cuándo usar Incremental vs Randomized? Incremental si X no entra en RAM; Randomized si entra pero querés velocidad y `n_components` es chico vs dimensión original.
4. ¿KernelPCA reemplaza a t-SNE/UMAP? No. KernelPCA preserva varianza global en el espacio kernel; t-SNE/UMAP preservan estructura local. Son herramientas distintas (ver clases 083-084).
5. ¿Por qué PCA es sensible a outliers? Porque maximiza varianza y los outliers tienen varianza desproporcionada. Considerá RobustScaler o TruncatedSVD si hay sparse + outliers.

Referencias

- Géron, A. Hands-On Machine Learning, 3ª ed., cap. 8 — Dimensionality Reduction, secciones "PCA", "Randomized PCA", "Incremental PCA", "Kernel PCA".
- scikit-learn user guide: <https://scikit-learn.org/stable/modules/decomposition.html#pca>
- Halko, Martinsson & Tropp (2011), Finding structure with randomness — paper original de Randomized SVD.
- Schölkopf, Smola & Müller (1998), Nonlinear component analysis as a kernel eigenvalue problem.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 093 — Clase 093 — LLE (Locally Linear Embedding)

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 8. Duración estimada: 45 min.

Objetivo

Entender LLE como técnica no lineal de reducción de dimensionalidad basada en manifold learning: preservar las relaciones lineales locales entre cada punto y sus vecinos para "desenrollar" estructuras curvas (Swiss roll, S-curve) donde PCA falla.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la intuición de LLE: cada punto se reconstruye como combinación lineal de sus k vecinos, y esa relación se preserva en baja dimensión.
- Aplicar `sklearn.manifold.LocallyLinearEmbedding` sobre un dataset no lineal (Swiss roll) y visualizar el resultado en 2D.
- Elegir el hiperparámetro `n_neighbors` y discutir su impacto (sub/sobre-ajuste local).
- Comparar LLE contra PCA y otras técnicas no lineales (Isomap, t-SNE) en términos de qué preservan.
- Identificar cuándo LLE es apropiado y cuándo conviene usar la variante Modified LLE.

Temas

- Motivación: limitaciones de PCA en manifolds curvos.
- Algoritmo LLE en dos pasos: (1) pesos de reconstrucción local, (2) embedding que preserva esos pesos.
- Hiperparámetros: `n_neighbors`, `n_components`, `method` (standard, modified, hessian, ltsa).
- Variantes: Modified LLE (MLLE), Hessian LLE, LTSA.
- Visualización del Swiss roll antes y después.
- Limitaciones: costo computacional $O(m \log(m) \cdot n \cdot k^3 + m \cdot n \cdot k^2)$ y sensibilidad al ruido.

Definiciones y características

- LLE (Locally Linear Embedding): técnica no lineal de reducción de dimensionalidad. No usa proyecciones; descubre cómo cada instancia se relaciona linealmente con sus vecinos más cercanos y busca una representación de baja dimensión donde esas relaciones locales se preserven al máximo.
- Manifold learning: familia de técnicas que asume que los datos de alta dimensión yacen sobre una variedad (manifold) de menor dimensión embebida en el espacio original. LLE, Isomap, t-SNE y UMAP son ejemplos.
- k vecinos (`n_neighbors`): cantidad de vecinos más cercanos que se usan para reconstruir cada punto. Valor bajo $\rightarrow$ ruido / desconexiones; valor alto $\rightarrow$ pierde la estructura local y se parece a PCA.
- Reconstruction weights (W): matriz de pesos que minimiza $\sum \|x - \sum_j w_{ij} x_j\|^2$ con $\sum_j w_{ij} = 1$ y $w_{ij} = 0$ si j no es vecino de i . Captura la geometría local.
- Modified LLE (MLLE): variante que usa múltiples vectores de pesos por vecindario para evitar el problema de regularización del LLE estándar cuando `n_neighbors` > `n_components`. Más estable y suele dar mejores embeddings.
- Hessian LLE / LTSA: otras variantes que reemplazan el criterio de reconstrucción por restricciones

geométricas más fuertes (curvatura local, tangentes locales).

Dataset / recursos

- `sklearn.datasets.make_swiss_roll(n_samples=1000, noise=0.2)` — dataset clásico para visualizar reducción no lineal.
- `sklearn.datasets.make_s_curve(n_samples=1000)` — manifold alternativo en forma de S.
- `sklearn.manifold.LocallyLinearEmbedding`.
- Géron, cap. 8 — sección "LLE" y figuras del Swiss roll.

Ejercicios

1. Swiss roll básico: generá un Swiss roll de 1000 puntos, aplicá `LocallyLinearEmbedding(n_neighbors=10, n_components=2)` y graficá el resultado coloreando por la coordenada original `t`. Verificá que el rollo quede "desenrollado".
2. Comparación con PCA: sobre el mismo Swiss roll, aplicá `PCA(n_components=2)` y compará visualmente. Discutí por qué PCA aplasta el rollo en lugar de desenrollarlo.
3. Barrido de `n_neighbors`: probá `n_neighbors` {5, 10, 30, 100} y graficá los 4 embeddings en una grilla 2x2. Describí qué pasa en cada extremo.
4. Modified LLE: repetí el ejercicio 1 con `method='modified'` y `n_neighbors=12`. Compará con el LLE estándar — ¿qué embedding se ve más limpio?
5. LLE sobre datos reales: cargá `load_digits()` (64 dimensiones) y proyectá a 2D con LLE. Coloreá por dígito. ¿Se separan las clases?

Homework verificable

Generá un Swiss roll con `n_samples=1500` y `random_state=42`. Aplicá LLE estándar y MLLE (ambos con `n_neighbors=12, n_components=2`). Guardá los dos arrays resultantes en `embeddings.npz` con keys `lle` y `mle`.

Criterio de aceptación: ambos arrays tienen shape (1500, 2), ningún NaN, y la correlación de Spearman entre la primera coordenada del embedding de MLLE y la variable `t` del Swiss roll original es $|\rho| > 0.95$ (el embedding preservó el orden a lo largo del rollo).

Errores comunes

- No escalar los datos cuando las features tienen magnitudes muy distintas — los "vecinos" pasan a estar dominados por una feature.
- Elegir `n_neighbors` muy bajo (ej. 3-4): el grafo de vecindad queda desconectado y LLE devuelve embeddings rotos o con componentes colapsados.
- Elegir `n_neighbors` muy alto: se pierde el carácter local y el resultado tiende a parecerse a PCA, perdiendo la ventaja no lineal.
- Esperar que LLE preserve distancias globales: no lo hace. Para distancias geodésicas usar Isomap; para clusters bien separados, t-SNE o UMAP.
- Usar LLE estándar con `n_neighbors > n_components` sin regularización: la solución se vuelve degenerada. Para eso existe MLLE.

Preguntas frecuentes

- ¿PCA o LLE? PCA si los datos viven en un subespacio lineal o si querés interpretar componentes / hacer compresión rápida. LLE (y otras técnicas de manifold) si la estructura es curva y querés visualizar. En la práctica, primero PCA para reducir ruido a ~50 dimensiones y después LLE/t-SNE a 2D.
- ¿LLE sirve para preprocesar antes de un clasificador? Rara vez. No es eficiente sobre datos nuevos (no

tiene una función transform natural y rápida) y t-SNE/UMAP suelen visualizar mejor. Para preprocesar, PCA o autoencoders.

- ¿Qué diferencia hay con Isomap? Isomap preserva distancias geodésicas globales sobre el grafo de vecinos; LLE preserva sólo relaciones lineales locales. Isomap suele dar embeddings más fieles a la geometría global; LLE es más barato.
- ¿Por qué a veces LLE colapsa todo en una línea? Suele ser un `n_neighbors` mal elegido o falta de regularización. Probá MLLE o ajustá `reg` (parámetro de regularización del solver).
- ¿Es determinístico? No del todo: depende del solver de eigendecomposición y de `random_state`. Fijá `random_state` y `eigen_solver='dense'` para reproducibilidad estricta.

Referencias

- Géron, A. Hands-On Machine Learning, 3ra ed., cap. 8 — Dimensionality Reduction, sección "LLE".
- Roweis, S. & Saul, L. (2000). Nonlinear Dimensionality Reduction by Locally Linear Embedding. Science, 290(5500).
- scikit-learn docs: LocallyLinearEmbedding y manifold learning comparison.
- Zhang, Z. & Wang, J. (2007). MLLE: Modified Locally Linear Embedding Using Multiple Weights.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 094 — Clase 094 — MDS, Isomap, t-SNE, UMAP, LDA

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 8 + UMAP docs. Duración estimada: 80 min.

Objetivo

Conocer y aplicar técnicas de reducción de dimensionalidad más allá de PCA: MDS, Isomap, t-SNE, UMAP y LDA. Entender qué preserva cada una (distancias, geodésicas, vecindarios locales, estructura global, separación entre clases) y cuándo elegir cada método según el problema (visualización 2D, preprocesamiento para clasificación, datos en variedades no lineales).

Resultados de aprendizaje

1. Distinguí entre métodos lineales (PCA, LDA) y no lineales (Isomap, t-SNE, UMAP) y sabés cuándo usar cada uno.
2. Aplicás MDS, Isomap, TSNE, umap.UMAP y LinearDiscriminantAnalysis de scikit-learn / umap-learn sobre datasets reales.
3. Configurás los hiperparámetros clave: `perplexity` (t-SNE), `n_neighbors` y `min_dist` (UMAP), `n_neighbors` (Isomap).
4. Interpretás correctamente embeddings 2D: qué significan los clusters, qué NO significan las distancias entre clusters en t-SNE.
5. Justificás por qué UMAP suele ser preferible a t-SNE: más rápido, preserva mejor la estructura global y es determinista con `random_state`.

Temas

Método	Tipo	Preserva	Uso típico	Hiperparámetros clave
MDS	No lineal	Distancias pairwise	Visualización de matrices de	<code>n_components</code> , <code>metric</code>

Isomap	No lineal	Distancias geodésicas (sol	Datos en manifold curvo (S	n_neighbors
t-SNE	No lineal	Vecindarios locales (proba	Visualización 2D/3D de clu	perplexity, learning_rate
UMAP	No lineal	Estructura local y global	Visualización + preprocesa	n_neighbors, min_dist
LDA	Lineal supervisado	Separación entre clases	Reducción previa a clasific	n_components ($\leq n_clases - 1$)

Definiciones y características

- MDS (Multidimensional Scaling): proyecta a baja dimensión intentando que las distancias entre puntos en el espacio reducido sean lo más parecidas posible a las distancias originales. No asume linealidad pero es costoso: $O(n^2)$ en memoria.
- Isomap: variante de MDS que reemplaza la distancia euclídea por la distancia geodésica (camino más corto sobre el grafo de k vecinos más cercanos). Ideal cuando los datos viven en una variedad curva (ej: Swiss roll).
- t-SNE (t-distributed Stochastic Neighbor Embedding): convierte similitudes en distribuciones de probabilidad y minimiza la divergencia KL entre el espacio original y el reducido. Excelente para visualizar clusters; no sirve como preprocesamiento general porque es estocástico y no preserva distancias globales.
- Perplexity (t-SNE): controla cuántos vecinos efectivos considera cada punto. Valores típicos: 5–50. Datasets grandes $\rightarrow$ perplexity mayor.
- UMAP (Uniform Manifold Approximation and Projection): alternativa moderna a t-SNE basada en topología algebraica. Más rápido, escalable, preserva mejor la estructura global y soporta transform sobre datos nuevos.
- n_neighbors (UMAP): balance entre estructura local (valores bajos, ~ 5 –15) y global (valores altos, ~ 50 –200). min_dist controla qué tan "apretados" se ven los clusters.
- LDA (Linear Discriminant Analysis): método supervisado que proyecta los datos maximizando la separación entre clases y minimizando la varianza dentro de cada clase. Limitado a $n_clases - 1$ dimensiones.
- Supervisado vs no supervisado: LDA usa las etiquetas y ; PCA, MDS, Isomap, t-SNE y UMAP no. Esto hace que LDA sea óptimo como preprocesamiento de clasificación cuando hay etiquetas.

Dataset / recursos

- `sklearn.datasets.make_swiss_roll` — clásico para Isomap vs PCA.
- `sklearn.datasets.load_digits` — $8 \times 8 = 64$ dims, ideal para visualizar con t-SNE/UMAP.
- `sklearn.datasets.fetch_openml('mnist_784')` — $70k \times 784$, para comparar tiempos t-SNE vs UMAP.
- Librerías: `scikit-learn` (MDS, Isomap, TSNE, LDA) + `umap-learn` (`pip install umap-learn`).

Ejercicios

1. Generá un Swiss roll con `make_swiss_roll(n_samples=1500)` y reducí a 2D con PCA, MDS e Isomap. Graficá los tres y comentá cuál "desenrolla" la variedad.
2. Cargá `load_digits` y aplicá t-SNE con perplexity `{5, 30, 50, 100}`. Mostrá los 4 plots y explicá el efecto.
3. Sobre `load_digits`, compará t-SNE vs UMAP: medí tiempo de ejecución con `time.perf_counter()` y reportá ambos embeddings 2D.
4. Aplicá LDA a `load_digits` reduciendo a 2D y a 9D. Entrená un `LogisticRegression` sobre cada versión y compará accuracy con el original (64 dims).
5. Con UMAP sobre `load_digits`, probá `n_neighbors {2, 15, 100}` con `min_dist=0.1`. Graficá los tres y explicá el trade-off local vs global.

Homework verificable

Tomá load_digits y producí un script compare_dimred.py que:

1. Aplique PCA(2), Isomap(2), t-SNE(2) y UMAP(2).
2. Para cada embedding, entrene un KNeighborsClassifier(n_neighbors=5) con cross_val_score (cv=5) sobre el embedding 2D.
3. Imprima una tabla con método, tiempo de ajuste y accuracy media.

Criterio: UMAP debe terminar al menos 3× más rápido que t-SNE sobre los 1797 dígitos, y la accuracy 2D de UMAP debe superar 0.90.

Errores comunes

1. Interpretar las distancias entre clusters en t-SNE como reales. t-SNE preserva vecindarios locales, no distancias globales: dos clusters cercanos en el plot no son necesariamente similares.
2. Usar t-SNE como preprocesamiento de un modelo. No tiene transform confiable sobre datos nuevos; usá UMAP o PCA para eso.
3. No escalar antes de Isomap o t-SNE. Estos métodos dependen de distancias; sin StandardScaler las features con escala grande dominan.
4. Pedirle a LDA más componentes que n_clases - 1. scikit-learn lanza error; LDA está topeado por la cantidad de clases.
5. Olvidar random_state en t-SNE/UMAP. Son estocásticos: sin semilla el embedding cambia en cada corrida y los plots no son reproducibles.

Preguntas frecuentes

1. ¿t-SNE o UMAP? UMAP en casi todos los casos: más rápido, preserva estructura global, tiene transform, escala a millones de puntos. t-SNE sigue siendo válido para visualización pequeña cuando ya tenés pipelines hechos.
2. ¿Cuándo usar Isomap en vez de PCA? Cuando sospechás que los datos viven en una variedad curva (ej: imágenes de un objeto rotando). PCA proyecta linealmente y aplanar mal la curvatura.
3. ¿LDA o PCA antes de clasificar? Si tenés etiquetas, LDA suele dar mejor separación con menos dimensiones. PCA es agnóstico al target y puede tirar info útil.
4. ¿Por qué MDS es tan lento? Calcula la matriz de distancias $O(n^2)$ y resuelve un problema de optimización. Para $n > 5000$ es impráctico; usá Isomap o UMAP.
5. ¿Puedo usar UMAP como preprocesamiento supervisado? Sí: UMAP acepta y en fit y aprende un embedding que respeta las etiquetas (semi-supervisado).

Referencias

- Géron, Hands-On ML, cap. 8 — "Other Dimensionality Reduction Techniques".
- McInnes, Healy & Melville (2018). UMAP: Uniform Manifold Approximation and Projection. arXiv:1802.03426.
- van der Maaten & Hinton (2008). Visualizing Data using t-SNE. JMLR.
- Documentación: [<https://umap-learn.readthedocs.io/>](https://umap-learn.readthedocs.io/)
- [<https://scikit-learn.org/stable/modules/manifold.html>](https://scikit-learn.org/stable/modules/manifold.html)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 095 — Clase 095 — Clustering K-Means: selección de K, MiniBatch

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 9. Duración estimada: 70 min.

Objetivo

Aplicar K-Means para segmentar datos no etiquetados, elegir el número de clusters K con criterios reproducibles (elbow, silhouette) y escalar el algoritmo con MiniBatchKMeans cuando el dataset no entra cómodo en memoria.

Resultados de aprendizaje

Al finalizar, vas a poder:

- Explicar el algoritmo de Lloyd y por qué K-Means++ mejora la inicialización aleatoria.
- Ajustar KMeans de scikit-learn fijando `n_init` y `random_state` para resultados estables.
- Elegir K combinando elbow method (inercia vs K) y silhouette score.
- Reemplazar KMeans por MiniBatchKMeans y discutir el trade-off velocidad / calidad.
- Diagnosticar por qué K-Means falla sin escalado o frente a clusters no esféricos.

Temas

1. Clustering no supervisado: planteo del problema.
2. Algoritmo de Lloyd: asignación + actualización de centroides.
3. Inicialización: random vs K-Means++; rol de `n_init`.
4. Inercia (within-cluster sum of squares) como objetivo.
5. Selección de K: elbow method, silhouette score, gap statistic (mención).
6. MiniBatchKMeans para datasets grandes / streaming.
7. Limitaciones: clusters no convexos, densidades distintas, sensibilidad al escalado.

Definiciones y características

- K-Means — algoritmo iterativo que parte el espacio de features en K regiones de Voronoi minimizando la suma de distancias cuadradas a los centroides.
- K-Means++ — esquema de inicialización que elige centroides iniciales separados entre sí; baja la varianza entre corridas y suele converger en menos iteraciones.
- Inertia — suma de distancias cuadradas de cada punto a su centroide asignado (`model.inertia_`). Monótona decreciente con K; sirve para el elbow.
- Elbow method — graficar `inertia_` vs K y elegir el K donde la curva "se quiebra" (la mejora marginal se aplana).
- Silhouette score — métrica en $[-1, 1]$ que combina cohesión intra-cluster y separación inter-cluster. Más alto = mejor; útil para comparar K.
- MiniBatchKMeans — variante que actualiza centroides con mini-lotes en vez de pasar por todo X en cada iteración. ~5-10× más rápido, inercia levemente peor.
- Centroides — vectores (K, `n_features`) que representan el "centro" de cada cluster; en KMeans es la media de los puntos asignados.
- `n_init` — cantidad de corridas con distintas semillas; scikit-learn se queda con la de menor inercia. Default 10 (sklearn ≥ 1.4 : "auto").

Dataset / recursos

- `sklearn.datasets.make_blobs(n_samples=2000, centers=5, cluster_std=0.8, random_state=42)` para los ejercicios de exploración.
- `sklearn.datasets.load_digits()` (1797 × 64) para el ejercicio de MiniBatchKMeans.
- Opcional: dataset Mall_Customers.csv o cualquier CSV tabular numérico ya escalado.

Ejercicios

1. Genera blobs con 5 centros, ajusta `KMeans(n_clusters=5, n_init=10, random_state=42)` y grafica los puntos coloreados por etiqueta junto con `cluster_centers_`.
2. Para `K` en `range(2, 11)`, calcula `inertia_` y `silhouette_score`. Grafica ambas curvas y justifica el `K` elegido.
3. Repetí el ajuste sin escalar un dataset donde una feature tenga escala $100\times$ mayor que la otra. Compara con `StandardScaler` previo y comenta el cambio en las etiquetas.
4. Sobre `load_digits()`, ajusta `KMeans(n_clusters=10)` y `MiniBatchKMeans(n_clusters=10, batch_size=256)`. Cronometra ambos con `%timeit` y compara inercias.
5. Probá `K-Means` sobre `make_moons(noise=0.05)`. Muestra visualmente por qué falla y proponé qué algoritmo usarías en su lugar (te lo vamos a contestar en la clase 086).

Homework verificable

Entregá un script `homework_085.py` que:

1. Cargue `make_blobs` con `random_state=42`, 4 centros reales.
2. Recorra `K = 2..8` y guarde inercia + `silhouette` en un `DataFrame`.
3. Imprima el `K` óptimo según `silhouette` y genere `elbow.png` + `silhouette.png`.

Criterio de aceptación: el script corre sin errores con `python homework_085.py`, imprime `K` óptimo = 4, y produce los dos PNG con ejes y título legibles.

Errores comunes

1. No escalar features — `K-Means` usa distancia euclídea; una feature en miles domina a otra en décimas. Aplicá `StandardScaler` salvo que tengas razón explícita para no hacerlo.
2. No fijar `n_init` (o dejarlo en 1) — una sola corrida puede caer en un mínimo local malo. Mantené `n_init` ≥ 10 y `random_state` fijo para reproducibilidad.
3. Elegir `K` mirando solo la inercia — la inercia siempre baja al subir `K`. Sin `elbow` visible ni `silhouette`, el criterio queda arbitrario.
4. Asumir clusters esféricos — `K-Means` no sirve para "lunas", anillos o densidades muy distintas; ahí necesitás `DBSCAN` o `GMM`.
5. Usar `MiniBatchKMeans` con `batch_size` muy chico — la varianza entre updates explota y los centroides oscilan. Mantenelo en 256–1024 salvo benchmarks específicos.

Preguntas frecuentes

1. ¿Elbow o `silhouette`? `Silhouette` es más objetivo cuando no hay codo claro; usá ambos y, si discrepan, priorizá `silhouette` + criterio de negocio.
2. ¿Cuántas iteraciones máximas? El default `max_iter=300` alcanza casi siempre. Si convergés antes, `sklearn` corta solo.
3. ¿`K-Means` es determinístico? No: depende de la inicialización. Fijá `random_state` y `n_init` para resultados reproducibles.
4. ¿Cuándo usar `MiniBatchKMeans`? Cuando `n_samples > 10^5` o el dataset llega en streaming. Para datasets chicos, el `K-Means` clásico es más preciso y suficientemente rápido.
5. ¿Sirve para detectar outliers? No directamente; los outliers distorsionan los centroides. Para outliers usá `DBSCAN` o `IsolationForest`.

Referencias

- Géron, A. Hands-On Machine Learning (3ra ed.), cap. 9 — sección "K-Means".
- scikit-learn user guide: Clustering — K-Means.

- Arthur, D. & Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding.
- Sculley, D. (2010). Web-Scale K-Means Clustering (paper original de MiniBatchKMeans).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 096 — Clase 096 — DBSCAN

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 9. Duración estimada: 55 min.

Objetivo

Que el alumno aplique DBSCAN (Density-Based Spatial Clustering of Applications with Noise) para descubrir clusters de forma arbitraria e identificar outliers nativamente, sin tener que predefinir k como en K-Means. Que sepa elegir eps con un k-distance plot y entienda cuándo conviene escalar a HDBSCAN.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Ejecutar DBSCAN con `sklearn.cluster.DBSCAN` y tunear `eps` y `min_samples` para un dataset 2D.
2. Elegir `eps` mirando el codo del k-distance plot (no a ojo).
3. Identificar outliers vía la etiqueta -1 que DBSCAN asigna a los puntos ruido.
4. Distinguir core / border / noise points y entender la noción de density-reachable.
5. Comparar DBSCAN vs K-Means y saber cuándo usar HDBSCAN (`eps` variable, clusters de densidad mixta).

Temas

#	Tema	Por qué importa
1	Intuición de densidad vs centroides	DBSCAN encuentra "regiones densas" — no ne
2	Hiperparámetros <code>eps</code> y <code>min_samples</code>	Son los dos botones del modelo. Mal puesto
3	Core / border / noise points	Vocabulario base para leer la salida y dep
4	k-distance plot para elegir <code>eps</code>	Método estándar para no ir a ciegas.
5	Etiqueta -1 y detección de outliers	DBSCAN es de los pocos clusterers que dete
6	Limitaciones: densidad uniforme, curse of	Por qué falla en datasets reales con clust
7	HDBSCAN como evolución	<code>eps</code> jerárquico: maneja densidad variable y

Definiciones y características

DBSCAN

: Algoritmo de clustering basado en densidad. Agrupa puntos que están densamente conectados y marca como ruido los que quedan en regiones de baja densidad. No requiere k. Complejidad $\sim O(n \log n)$ con índice espacial.

`eps` (epsilon)

: Radio de la vecindad alrededor de cada punto. Define qué se considera "cerca". Es el hiperparámetro más sensible — un cambio chico colapsa o explota los clusters.

min_samples

: Cantidad mínima de puntos (incluyendo el propio) dentro de eps para que un punto se considere core. Regla práctica: $\text{min_samples} \geq \text{dim} + 1$, típicamente 4–10 para 2D.

Core point

: Punto con al menos min_samples vecinos dentro de eps. Es el "núcleo" desde el cual crece el cluster.

Border point

: Punto que está dentro del eps de un core, pero él mismo no tiene min_samples vecinos. Pertenece al cluster pero no propaga.

Noise point (outlier)

: Punto que no es core ni border. DBSCAN lo etiqueta como -1. Sale gratis del modelo, sin entrenar un detector aparte.

Density-reachable

: Relación que conecta dos puntos vía una cadena de core points. Es la definición formal de "pertenecer al mismo cluster" en DBSCAN.

HDBSCAN (Hierarchical DBSCAN)

: Evolución de DBSCAN que reemplaza eps por una jerarquía. Maneja clusters de densidades distintas, expone un parámetro más intuitivo (min_cluster_size) y devuelve probabilidades de pertenencia. Es el default moderno para clustering no supervisado.

Dataset / recursos

- make_moons(n_samples=1000, noise=0.05) de scikit-learn — dos lunas entrelazadas, el caso canónico donde K-Means falla y DBSCAN brilla.
- make_blobs con densidades distintas para mostrar la limitación de DBSCAN y motivar HDBSCAN.

Ejercicios

1. DBSCAN sobre moons. Entrená DBSCAN(eps=0.2, min_samples=5) sobre make_moons. Graficá los clusters y contá cuántos puntos quedaron como -1.
2. K-distance plot. Calculá la distancia al k-ésimo vecino más cercano ($k=\text{min_samples}$) para todos los puntos, ordenala y graficala. Identificá el codo y usalo como eps.
3. Sensibilidad a eps. Probá eps {0.05, 0.1, 0.2, 0.5} y reportá número de clusters y % de ruido en cada caso. Mostrá cómo eps chico = todo ruido y eps grande = un cluster.
4. DBSCAN vs K-Means. Sobre el mismo make_moons, corré ambos con $k=2$. Mostrá visualmente que K-Means parte las lunas por la mitad y DBSCAN las separa bien.
5. HDBSCAN sobre densidades mixtas. Generá blobs con cluster_std distinto por blob. Mostrá que un eps único en DBSCAN no puede capturar ambos, y que HDBSCAN(min_cluster_size=20) sí.

Homework verificable

Notebook que sobre un dataset 2D sintético (moons + outliers inyectados a mano) haga: (a) k-distance plot y elección razonada de eps; (b) DBSCAN con esos hiperparámetros; (c) reporte de % de outliers detectados vs inyectados; (d) comparación visual con K-Means; (e) corrida con HDBSCAN sobre blobs de densidad mixta.

Criterio de aceptación: eps justificado con el codo del k-distance plot (no a ojo). DBSCAN recupera al menos el 80% de los outliers inyectados. La comparación con K-Means muestra explícitamente la falla de los

centroides en datos no convexos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Elegir eps a ojo y obtener un solo cluster	El parámetro es altamente sensible y depen
DBSCAN sobre features sin escalar	eps es una distancia euclidiana; si una fe
min_samples=1 o min_samples=2	Todo termina siendo core point, no hay rui
Aplicar DBSCAN en alta dimensión (>10)	Curse of dimensionality: las distancias se
Pedirle predict() a DBSCAN	DBSCAN no tiene predict — no hay forma nat

Preguntas frecuentes

¿K-Means o DBSCAN?

K-Means si los clusters son aproximadamente esféricos, de tamaño parecido, y sabés (o podés estimar) k. DBSCAN si los clusters tienen forma arbitraria, no sabés cuántos hay, o necesitás detectar outliers en la misma pasada. Para producción moderna: HDBSCAN suele ganar a ambos cuando no tenés intuición previa.

¿Por qué DBSCAN devuelve -1?

Es la etiqueta convencional para noise points — puntos que no pertenecen a ningún cluster. No es un cluster más, es la salida natural del algoritmo para outliers.

¿Cómo elijo min_samples?

Regla práctica: $\text{min_samples} = 2 * \text{dim}$. Para 2D, usá 4. Para 3D, 6. Más grande = clusters más robustos pero más ruido. En la práctica, min_samples mueve menos la aguja que eps.

¿DBSCAN escala a millones de puntos?

Con algorithm='ball_tree' o 'kd_tree' queda en $\sim O(n \log n)$. Para >1M puntos en alta dimensión, considerá HDBSCAN o aproximaciones tipo sklearn.cluster.OPTICS.

¿Cuándo HDBSCAN vence a DBSCAN?

Siempre que los clusters tengan densidades distintas entre sí. DBSCAN usa un eps global; HDBSCAN construye una jerarquía y elige el corte óptimo por cluster. Además expone min_cluster_size que es más intuitivo de tunear que eps.

Referencias

- Géron, cap. 9 § "DBSCAN".
- scikit-learn DBSCAN user guide
- HDBSCAN docs
- Ester et al. (1996), A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise — paper original.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 097 — Clase 097 — Agglomerative, BIRCH, Mean Shift, Affinity Propagation, Spectral

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 9 § Other Clustering Algorithms. Duración estimada: 70 min.

Objetivo

Que el alumno conozca el zoológico de algoritmos de clustering más allá de K-Means y DBSCAN — Agglomerative (jerárquico, lee dendrogramas), BIRCH (escalable a millones de filas), Mean Shift (denso, sin especificar k), Affinity Propagation (elige exemplars por message-passing) y Spectral Clustering (clustering vía autovectores del grafo de similitud) — y sepa cuándo elegir cada uno según tamaño del dataset, forma de los clusters y necesidad de jerarquía.

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Construir un dendrograma con `scipy.cluster.hierarchy.linkage` y cortarlo a una altura dada para obtener clusters.
2. Elegir un linkage (ward, complete, average, single) según la forma esperada de los clusters y conocer el efecto del chaining en single.
3. Usar BIRCH para datasets que no entran en memoria, ajustando `threshold` y `branching_factor`.
4. Aplicar Mean Shift ajustando `bandwidth` (con `estimate_bandwidth`) y entender por qué descubre el número de clusters automáticamente.
5. Decidir entre Affinity Propagation y Spectral Clustering según escalabilidad (AP es $O(n^2)$ en memoria) y geometría (Spectral funciona en clusters no convexos).

Temas

#	Algoritmo	Hiperparámetro clave	k a priori	Complejidad	Cuándo usarlo
1	Agglomerative	linkage, n_clusters o d	Sí (o threshold)	$O(n^3)$ / $O(n^2 \log n)$	Querés jerarquía + dendrograma; n s
2	BIRCH	threshold, branching_f	Opcional	$O(n)$	Datasets grandes (millones) que no e
3	Mean Shift	bandwidth	No	$O(n^2)$ por iter	Modos de densidad; clusters de tama
4	Affinity Propagation	damping, preference	No	$O(n^2 \cdot T)$	Pocos puntos, querés exemplars rea
5	Spectral	n_clusters, affinity (rbf,	Sí	$O(n^3)$ (eigen)	Clusters no convexos, manifolds, gra

Definiciones y características

Clustering aglomerativo (jerárquico)

: Algoritmo bottom-up: arranca con cada punto como su propio cluster y en cada paso fusiona los dos clusters más cercanos hasta llegar a uno solo. Produce una jerarquía completa que se visualiza como dendrograma. En sklearn: `AgglomerativeClustering(n_clusters=k)` o `distance_threshold=d` (entonces `n_clusters=None`).

Linkage

: Criterio para medir la distancia entre dos clusters durante la fusión. Las cuatro opciones que importan:

- ward (default sklearn): minimiza la varianza intra-cluster al fusionar. Tiende a producir clusters de tamaño parecido. Solo con distancia euclidiana.
- complete (max-linkage): distancia entre los dos puntos más lejanos. Produce clusters compactos.
- average: promedio de todas las distancias entre puntos de ambos clusters. Compromiso razonable.
- single (min-linkage): distancia entre los dos puntos más cercanos. Sufre chaining — un puente fino de puntos une dos clusters que deberían quedar separados.

Dendrograma

: Diagrama de árbol invertido que muestra el orden y la altura (= distancia entre clusters fusionados) de cada merge. Cortar con una línea horizontal a altura h da los clusters resultantes. Se grafica con `scipy.cluster.hierarchy.dendrogram(Z)` donde $Z = \text{linkage}(X, \text{method}='ward')$.

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)

: Algoritmo de dos pasos para datasets enormes. (1) Recorre los datos una sola vez construyendo un CF-tree (Clustering Feature tree) que comprime puntos en sub-clusters compactos. (2) Aplica un clustering tradicional sobre los sub-clusters. Streaming-friendly (online). Hiperparámetros: `threshold` (radio máximo de un sub-cluster — más chico, más sub-clusters) y `branching_factor` (hijos máximos por nodo).

Mean Shift

: Algoritmo basado en densidad. Cada punto "trepa" iterativamente hacia el modo (máximo local) de la densidad estimada con un kernel gaussiano de ancho `bandwidth`. Puntos que convergen al mismo modo forman un cluster. No requiere k . Sensible al `bandwidth`: chico $\rightarrow$ muchos clusters chiquitos; grande $\rightarrow$ todo un cluster. `sklearn.cluster.estimate_bandwidth(X)` da un valor de arranque razonable.

Bandwidth

: Ancho del kernel en Mean Shift. Análogo conceptual al `eps` de DBSCAN — define qué tan "lejos" mira cada punto para estimar densidad.

Affinity Propagation

: Algoritmo de message passing entre puntos. Cada punto manda dos tipos de mensajes (`responsibility` y `availability`) a los demás hasta que emerge un subconjunto de exemplars (puntos representativos reales del dataset) y cada otro punto se asigna al exemplar más afín. No requiere k . Caro: $O(n^2)$ en memoria y $O(n^2 \cdot T)$ en tiempo — no escala más allá de unos miles de puntos. Hiperparámetros: `damping` (entre 0.5 y 1, para evitar oscilaciones) y `preference` (controla cuántos exemplars emergen).

Spectral Clustering

: Construye una matriz de similitud S (típicamente con kernel RBF o k -NN), calcula el grafo Laplaciano $L = D - S$, toma los k autovectores de menor autovalor, los apila como nuevas features y corre K-Means en ese embedding. Funciona donde K-Means falla: clusters no convexos, dos lunas entrelazadas, círculos concéntricos. Cuello de botella: eigendecomposición $O(n^3)$.

Similarity matrix

: Matriz $n \times n$ donde $S[i,j]$ mide qué tan parecidos son los puntos i y j (alta similitud = cerca). Es la entrada de Spectral y Affinity Propagation. Para Spectral con kernel RBF: $S[i,j] = \exp(-\gamma \cdot \|x_i - x_j\|^2)$.

Dataset / recursos

- `make_blobs` (sklearn) — para Agglomerative y BIRCH (clusters convexos).
- `make_moons` y `make_circles` — clusters no convexos, donde Spectral brilla y K-Means/Agglomerative-ward fallan.
- Dataset opcional escalable: generará 1M de puntos sintéticos para probar BIRCH vs K-Means en tiempo y memoria.

Ejercicios

1. Dendrograma sobre `make_blobs`. Generá 50 puntos en 4 blobs. Calculá $Z = \text{linkage}(X, \text{method}='ward')$ y graficá el dendrograma. Cortá a una altura que dé 4 clusters. Comparalo con `AgglomerativeClustering(n_clusters=4)`.

2. Linkage comparison. Sobre `make_moons(n_samples=300, noise=0.05)`, ajustá `AgglomerativeClustering(n_clusters=2)` con `linkage='ward', 'complete', 'average', 'single'`. Ploteá los 4

resultados lado a lado. ¿Cuál recupera las dos lunas? ¿Por qué?

3. BIRCH escalable. Generá 100k puntos con `make_blobs`. Medí tiempo y memoria de `BIRCH(n_clusters=5)` vs `KMeans(n_clusters=5)`. Variá `threshold` {0.1, 0.5, 1.0} y mostrá cómo cambia el número de sub-clusters internos del CF-tree.

4. Mean Shift sin saber k. Sobre 3 blobs claros, corré `MeanShift(bandwidth=estimate_bandwidth(X, quantile=0.2))`. Verificá que recupera 3 clusters automáticamente. Después poné `quantile=0.5` y observá cómo colapsa a menos clusters.

5. Spectral vs K-Means en `make_circles`. Generá dos círculos concéntricos con `make_circles(n_samples=500, factor=0.5, noise=0.05)`. Ajustá `KMeans(n_clusters=2)` y `SpectralClustering(n_clusters=2, affinity='nearest_neighbors')`. Graficá ambos. Documentá por qué Spectral funciona y K-Means no.

Homework verificable

Notebook con: (a) dendrograma de `load_iris().data` usando ward linkage, cortado para obtener 3 clusters; (b) tabla comparativa de Adjusted Rand Index entre las labels verdaderas y las predichas por Agglomerative, BIRCH, Mean Shift, Affinity Propagation y Spectral — corriendo los 5 sobre el mismo Iris; (c) gráfico 2D (con las 2 primeras PCs) coloreado por las labels de cada método; (d) párrafo justificando qué método ganó y por qué.

Criterio de aceptación: el dendrograma se visualiza correctamente con eje y = distancia. La tabla muestra ARI [-1, 1] para los 5 métodos. Spectral y Agglomerative-ward deberían superar 0.7 en Iris.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
AgglomerativeClustering con <code>linkage='ward'</code>	ward solo soporta distancia euclidiana. Fix: <code>Fi</code>
Dendrograma con miles de hojas, ilegible	Demasiados puntos. Fix: pasale <code>truncate</code>
MeanShift corre eterno o devuelve 1 solo c	bandwidth mal calibrado (default = estimat
AffinityPropagation no converge — warning	damping muy bajo (oscilaciones). Fix: subí
SpectralClustering con <code>affinity='rbf'</code> da c	gamma del RBF mal escalado. Fix: usá affin

Preguntas frecuentes

¿Cuándo Agglomerative en lugar de K-Means?

Cuando querés (a) explorar la estructura jerárquica del dataset con un dendrograma antes de decidir k, (b) no asumir clusters esféricos (con average o complete linkage), o (c) reproducibilidad determinística — Agglomerative no depende de inicialización aleatoria. Costo: $O(n^3)$, inviable para $n > \sim 10k$.

¿BIRCH reemplaza a MiniBatch K-Means?

Casi. Ambos son escalables. BIRCH gana cuando los datos llegan en streaming o no entran ni siquiera en lotes (es one-pass). MiniBatch K-Means es más simple y suele ser suficiente para datasets que entran en disco. Si dudás, empezá por MiniBatch K-Means.

¿Cómo elijo el bandwidth de Mean Shift sin probar a mano?

Usá `sklearn.cluster.estimate_bandwidth(X, quantile=0.2, n_samples=500)`. El quantile controla el ancho del kernel basado en distancias entre pares de puntos muestreados. Empezá con 0.2 y bajá si querés clusters más finos.

¿Affinity Propagation sirve para algo en la práctica?

Para datasets chicos (< 1000 puntos) donde necesitás exemplars reales — por ejemplo, elegir 50 reviews representativas de 800. Para clustering general, los demás algoritmos lo superan en velocidad y robustez.

¿Spectral Clustering es lo mismo que clustering sobre PCA?

No. PCA proyecta sobre direcciones de máxima varianza (lineal). Spectral usa autovectores del grafo Laplaciano de la matriz de similitud — captura estructura no lineal (manifolds). Por eso recupera dos lunas entrelazadas, donde PCA + K-Means falla.

Referencias

- Géron, Hands-On ML, cap. 9 § Other Clustering Algorithms.
- scikit-learn — Clustering overview (tabla comparativa)
- scipy linkage + dendrogram
- Frey & Dueck (2007) — Affinity Propagation paper
- von Luxburg (2007) — A Tutorial on Spectral Clustering

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 098 — Clase 098 — Gaussian Mixture Models

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 9. Duración estimada: 70 min.

Objetivo

Entender los Gaussian Mixture Models (GMM) como modelo probabilístico de soft clustering, ajustarlos con el algoritmo EM en scikit-learn, elegir el número de componentes con BIC/AIC, y conocer las variantes (covariance_type, BayesianGaussianMixture) para aplicarlas en clustering, densidad y detección de anomalías.

Resultados de aprendizaje

Al finalizar la clase vas a poder:

1. Explicar qué es un GMM y cómo se diferencia de K-Means (asignación dura vs. probabilística).
2. Ajustar un GaussianMixture con scikit-learn y obtener predict, predict_proba y score_samples.
3. Seleccionar el número óptimo de componentes comparando BIC y AIC en una grilla.
4. Elegir el covariance_type apropiado (full, tied, diag, spherical) según supuestos y tamaño del dataset.
5. Usar BayesianGaussianMixture para que el modelo descarte componentes innecesarios automáticamente.

Temas

- Modelos de mezcla: intuición y fórmula.
- Algoritmo Expectation-Maximization (EM): paso E (responsabilidades) + paso M (actualizar medias, covarianzas, pesos).
- API de sklearn.mixture.GaussianMixture: n_components, covariance_type, n_init, tol.
- Métodos: predict_proba (soft), score_samples (log-densidad), sample (generar datos).
- Selección de modelo con BIC y AIC.
- Variantes de covarianza y su impacto en parámetros / sesgo / varianza.

- Bayesian GMM con prior de Dirichlet: aprende cuántos componentes hacen falta.
- Detección de anomalías por umbral sobre densidad.

Definiciones y características

- Gaussian Mixture Model (GMM): modelo generativo que asume que los datos provienen de una mezcla ponderada de K distribuciones gaussianas multivariadas. Cada punto tiene probabilidad de pertenecer a cada componente.
- Mixture (mezcla): combinación convexa $p(x) = \sum \pi_k \cdot N(x | \mu_k, \Sigma_k)$ con pesos π_k que suman 1.
- EM algorithm (Expectation-Maximization): procedimiento iterativo que alterna entre estimar las responsabilidades de cada componente sobre cada punto (E) y reestimar parámetros maximizando la verosimilitud esperada (M). Converge a un óptimo local.
- Soft clustering: asignación probabilística — cada punto recibe un vector de probabilidades de pertenencia, no una etiqueta única. Útil cuando los clusters se solapan.
- covariance_type: controla la forma de las matrices de covarianza.
- full: cada componente tiene su propia matriz completa (más flexible, más parámetros).
- tied: todos comparten la misma matriz.
- diag: matrices diagonales (ejes alineados).
- spherical: una sola varianza escalar por componente (clusters esféricos).
- BIC (Bayesian Information Criterion): $-2 \cdot \log L + k \cdot \log n$. Penaliza fuerte la complejidad; preferí el modelo con BIC mínimo. Tiende a elegir modelos más parsimoniosos.
- AIC (Akaike Information Criterion): $-2 \cdot \log L + 2 \cdot k$. Penaliza menos que BIC; suele elegir modelos algo más complejos.
- BayesianGaussianMixture: variante con prior de Dirichlet sobre los pesos. Si fijás n_components alto, el modelo "apaga" los componentes sobrantes (pesos ≈ 0), evitando elegir K manualmente.

Dataset / recursos

- sklearn.datasets.make_blobs con clusters de varianzas distintas (para ver el aporte vs. K-Means).
- sklearn.datasets.load_iris para validar agrupamiento contra etiquetas reales.
- Opcional: dataset Old Faithful (geyser) — clásico ejemplo de mezcla bimodal.

Ejercicios

1. Ajuste básico: generá make_blobs con 3 centros y cluster_std variable. Ajustá GaussianMixture(n_components=3) y compará predict con las etiquetas reales (ARI).
2. Soft vs. hard: sobre el mismo dataset, mostrá predict_proba de 5 puntos cerca de la frontera. Compará con la asignación dura de K-Means.
3. Selección de K con BIC/AIC: ajustá GMMs con n_components de 1 a 10. Graficá BIC y AIC vs. K e identificá el mínimo.
4. covariance_type: repetí el ajuste con los 4 tipos sobre un dataset con clusters elípticos rotados. Compará BIC y visualizá las elipses de covarianza.
5. Bayesian GMM: ajustá BayesianGaussianMixture(n_components=10, weight_concentration_prior=0.01) sobre datos con 3 clusters reales y mostrá que los pesos efectivos son ≈ 3 .

Homework verificable

Sobre load_iris (sin usar la etiqueta para entrenar):

1. Estandarizá las features.
2. Ajustá GMMs con n_components de 1 a 8 y covariance_type en ['full', 'tied', 'diag', 'spherical'].
3. Elegí el (n_components, covariance_type) con BIC mínimo.

4. Calculá el Adjusted Rand Index entre predict del mejor modelo y y real.

Criterio: $ARI \geq 0.85$ y el mejor modelo elegido por BIC tiene `n_components` {2, 3}.

Errores comunes

1. No estandarizar las features. GMM con `covariance_type='spherical'` o `'diag'` es muy sensible a la escala.
2. Usar `n_init=1` (default). EM converge a óptimos locales; subí a `n_init=10` para resultados estables.
3. Elegir K mirando solo la log-verosimilitud. Siempre aumenta con K; usá BIC/AIC que penalizan complejidad.
4. `covariance_type='full'` con pocos datos y alta dimensión: explota los parámetros ($K \cdot d \cdot (d+1)/2$) y sobreajusta. Bajá a `diag` o `tied`.
5. Asumir que `predict_proba` da incertidumbre calibrada. Da responsabilidades dentro del modelo; si el modelo está mal especificado, las probas pueden ser engañosas.

Preguntas frecuentes

1. ¿GMM o K-Means? K-Means es más rápido y simple, pero asume clusters esféricos de igual tamaño y asigna duro. GMM permite clusters elípticos, tamaños distintos, solapamiento y da probabilidades. Si tus clusters son claramente esféricos y no se solapan, K-Means alcanza.
2. ¿BIC o AIC? Si querés el modelo más parsimonioso y tenés muchos datos, usá BIC. Si priorizás capacidad predictiva y no te molesta un modelo algo más complejo, usá AIC. En la práctica, mostrá los dos y mirá si coinciden.
3. ¿Cómo detecto anomalías con GMM? Calculá `score_samples(X)` (log-densidad) y marcá como anomalía los puntos con score bajo un percentil (ej. 4%).
4. ¿Sirve para datos de alta dimensión? Con `covariance_type='full'` no escala bien (muchos parámetros). Reducí dimensión con PCA o usá `diag/tied`.
5. ¿Qué hace `BayesianGaussianMixture` que no haga GMM? Aprende cuántos componentes son necesarios: si fijás `n_components` alto, los sobrantes quedan con peso ≈ 0 . Evita la búsqueda en grilla de K.

Referencias

- Géron, Hands-On ML (3ª ed.), cap. 9 § "Gaussian Mixtures".
- scikit-learn — Gaussian Mixture Models.
- scikit-learn — GaussianMixture y BayesianGaussianMixture.
- Bishop, Pattern Recognition and Machine Learning, cap. 9 (EM y mixtures).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 099 — Clase 099 — Detección de anomalías: Isolation Forest, LOF, One-Class SVM

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 9 + sklearn outlier detection. Duración estimada: 70 min.

Objetivo

Que el alumno detecte puntos anómalos (fraude, fallas, outliers) en datos sin etiquetas, eligiendo entre

Isolation Forest, LOF, One-Class SVM y Elliptic Envelope según la geometría del problema, y entendiendo la diferencia entre outlier detection (entrenar con datos sucios) y novelty detection (entrenar limpio, predecir sobre nuevos).

Resultados de aprendizaje

Al finalizar la clase, el alumno podrá:

1. Distinguir outlier detection vs novelty detection y elegir el algoritmo acorde.
2. Entrenar IsolationForest y ajustar el hiperparámetro contamination.
3. Usar LocalOutlierFactor en modo novelty=False (fit_predict) y novelty=True (predict).
4. Aplicar OneClassSVM y EllipticEnvelope, reconociendo sus supuestos (kernel, gaussianidad).
5. Evaluar detectores de anomalías con score_samples, ROC-AUC y reglas de negocio (top-k).

Temas

#	Tema	Por qué importa
1	Outlier vs novelty detection	Define cómo se entrena y qué se predice.
2	Isolation Forest	Default sólido en alta dimensión, escala b
3	Local Outlier Factor (LOF)	Detecta anomalías locales por densidad.
4	One-Class SVM	Frontera no lineal con kernel RBF; sensibl
5	Elliptic Envelope	Asume distribución gaussiana; útil en dato
6	score_samples y umbrales	Salida continua > etiqueta binaria.
7	Evaluación sin labels	Top-k, inspección manual, ROC si hay groun

Definiciones y características

Anomaly / outlier detection

: Tarea no supervisada de identificar instancias que difieren significativamente de la mayoría. Entrena sobre un dataset contaminado (con algunas anomalías) y las marca dentro del mismo dataset. Salida sklearn: +1 (inlier) / -1 (outlier).

Novelty detection

: Variante donde el entrenamiento se hace sobre datos limpios (solo inliers), y luego en inferencia se predice si nuevos puntos son normales o novedosos. LocalOutlierFactor(novelty=True) y OneClassSVM operan en este modo.

Isolation Forest

: Ensemble de árboles que aíslan puntos eligiendo features y splits al azar. Las anomalías quedan aisladas con menos splits (path corto). Escala bien a alta dimensión y N grande. Default recomendado.

contamination

: Hiperparámetro que indica la fracción esperada de anomalías en los datos (entre 0 y 0.5, o 'auto'). Define el umbral del score. Si te equivocás mucho, calibrá con score_samples y elegí el umbral a mano.

Local Outlier Factor (LOF)

: Compara la densidad local de un punto con la de sus k vecinos. Si la densidad es mucho menor que la de sus vecinos, es outlier. Bueno para anomalías locales (un punto raro dentro de un cluster). No escala bien a N muy grande.

One-Class SVM

: Aprende una frontera (típicamente con kernel RBF) que envuelve la región "normal" del espacio. Sensible a

escala (estandarizar siempre) y al hiperparámetro ν (cota superior de la fracción de outliers). Lento en N grande; preferir `SGDOneClassSVM` para datasets grandes.

Elliptic Envelope

: Ajusta una gaussiana robusta a los datos y marca como outliers los puntos fuera de un elipsoide de confianza. Asume distribución unimodal aproximadamente gaussiana — si los datos son multimodales o tienen estructura compleja, falla.

`score_samples(X)`

: Devuelve un score continuo de "normalidad" por instancia (más alto = más normal). Útil para rankear top-k anomalías o elegir el umbral a mano en lugar de confiar en `contamination`.

Dataset / recursos

Sintético con `make_blobs` + outliers uniformes inyectados, o `sklearn.datasets.fetch_kddcup99` (detección de intrusiones, real y con labels para evaluar).

Ejercicios

1. Isolation Forest baseline. Genera 2 blobs + 5% de outliers uniformes. Ajusta `IsolationForest(contamination=0.05)`. Plot 2D con inliers vs outliers detectados.
2. LOF local. Usá el mismo dataset pero metiendo un outlier dentro de uno de los blobs (anomalía local). Comparar Isolation Forest vs `LocalOutlierFactor(n_neighbors=20)`: ¿cuál lo agarra?
3. One-Class SVM con escalado. Entrená `OneClassSVM(kernel='rbf', nu=0.05)` con y sin `StandardScaler`. Comparar fronteras de decisión.
4. Top-k con `score_samples`. Usá `score_samples` de Isolation Forest, ordená ascendente, y devolvé los 10 puntos más anómalos. Inspeccionálos visualmente.
5. Evaluación con labels. Sobre `fetch_kddcup99` (subset), entrená Isolation Forest sin usar labels. Después calculá ROC-AUC contra las labels reales (normal. vs ataque). Reportá AUC.

Homework verificable

Notebook con dataset de transacciones sintéticas (montos, hora, comercio): (a) inyectar 2% de transacciones anómalas (montos extremos, horarios raros); (b) entrenar Isolation Forest, LOF y One-Class SVM; (c) generar tabla comparativa con `precision@k=20`, `recall` y tiempo de entrenamiento; (d) elegir el ganador y justificar.

Criterio de aceptación: los tres modelos están entrenados sobre los mismos datos escalados, la tabla compara las 3 métricas, y la justificación menciona supuestos (densidad local vs global, escalabilidad).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>LocalOutlierFactor</code> no tiene predict	Por default <code>novelty=False</code> y solo expone <code>fi</code>
One-Class SVM marca todo como outlier (o t	Datos sin escalar — el kernel RBF es ultra
<code>IsolationForest</code> con <code>contamination</code> muy off	Si la fracción real difiere mucho del valo
Elliptic Envelope falla con datos multimod	Asume una sola gaussiana. Fix: cambiá a ls
Evaluar con <code>accuracy</code> sobre clases desbalan	El modelo "todo normal" da 99% <code>accuracy</code> y

Preguntas frecuentes

¿Isolation Forest, LOF o One-Class SVM?

Isolation Forest primero, casi siempre: escala bien, pocos hiperparámetros, robusto en alta dimensión. LOF si sospechás anomalías locales (raros dentro de un cluster denso). One-Class SVM si el dataset es chico/mediano y la frontera "normal" es claramente no lineal — pero estandarizá sí o sí. Elliptic Envelope solo si los datos son unimodales y aproximadamente gaussianos.

¿Cómo elijo contamination?

Si conocés la prevalencia esperada (ej. 1% de fraude), poné ese valor. Si no, dejá 'auto' y después calibrá mirando la distribución de score_samples — buscá un quiebre natural en el histograma.

¿Necesito escalar las features?

Para One-Class SVM y LOF, sí (ambos usan distancias). Para Isolation Forest, no es estrictamente necesario porque parte features con splits — pero no hace daño.

¿Sirve para series de tiempo?

No directamente — estos modelos asumen i.i.d. Para series, mirá descomposición + residuos, ARIMA, Prophet, o modelos específicos como PyOD con ventanas deslizantes.

¿Cómo evalúo si no tengo labels?

Inspección manual del top-k con score_samples. Si hay labels parciales o un sample anotado, ROC-AUC y precision@k. Sin nada de eso, sanity check: ¿los outliers detectados tienen sentido para el negocio?

Referencias

- Géron, cap. 9 § Anomaly Detection.
- sklearn — Novelty and Outlier Detection
- sklearn — IsolationForest
- sklearn — LocalOutlierFactor
- Liu, Ting & Zhou (2008), Isolation Forest, ICDM.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Parte 2 — Parte 2 — Deep Learning — Keras, TensorFlow, Transformers, RL y Despliegue

75 clases en esta parte.

Clase 100 — Clase 100 — Perceptrón, MLP y backpropagation

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § From Biological to Artificial Neurons. Duración estimada: 80 min.

Objetivo

Que el alumno entienda la unidad fundamental del Deep Learning: la neurona artificial (perceptrón de Rosenblatt 1957), por qué un solo perceptrón no puede aprender XOR, cómo el MLP (Multi-Layer Perceptron)

lo resuelve apilando capas con activaciones no lineales, y cómo backpropagation (regla de la cadena) permite calcular gradientes en cualquier grafo computacional — el algoritmo que destrabó el Deep Learning moderno.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar a mano un perceptrón ($y = \text{step}(w \cdot x + b)$) y mostrar por qué no separa XOR.
- Construir un MLP con `keras.Sequential([Dense(...), Dense(...)])` y entrenarlo sobre un dataset simple.
- Explicar forward pass (compute layer by layer) y backward pass (propagar gradientes con la regla de la cadena).
- Calcular a mano los gradientes para un MLP de 2 capas con una sola muestra.
- Reconocer que la diferenciación automática (autograd) hace innecesario derivar a mano para modelos arbitrarios.

Temas

#	Tema	Por qué importa
1	Perceptrón clásico (Rosenblatt 1957)	La unidad atómica; ver sus límites motiva
2	El problema XOR	Por qué necesitamos no linealidad y profun
3	MLP: input $\rightarrow$ hidden $\rightarrow$ output	La arquitectura mínima útil.
4	Activación no lineal	Sin activación, N capas = 1 capa lineal.
5	Backpropagation (Rumelhart, Hinton & Willi	La regla de la cadena aplicada a un grafo.
6	Autograd / autodiff	Lo que hace que TF/PyTorch/JAX te liberen

Definiciones y características

- Perceptrón: $y = \text{step}(\sum w_i \cdot x_i + b)$. Regla de aprendizaje: $w_i \leftarrow w_i + \eta \cdot (y_{\text{true}} - y_{\text{pred}}) \cdot x_i$. Converge si los datos son linealmente separables.
- MLP (Multi-Layer Perceptron): red feedforward fully-connected con ≥ 1 capa oculta y activaciones no lineales (sigmoid, tanh, ReLU).
- Forward pass: $a^l = \sigma(W^l \cdot a^{l-1} + b^l)$. Se compute layer-by-layer hasta la salida.
- Loss function: $L(y_{\text{pred}}, y_{\text{true}})$ — MSE para regresión, cross-entropy para clasificación.
- Backward pass: aplica la regla de la cadena para calcular $\partial L / \partial W^l$ propagando el gradiente de la salida hacia la entrada.
- Autograd: TF (GradientTape), PyTorch (backward()), JAX (grad) — construyen el grafo y aplican backprop automáticamente.
- Universal Approximation Theorem (Cybenko 1989, Hornik 1991): un MLP con una sola capa oculta y "suficientes" neuronas puede aproximar cualquier función continua. En la práctica, profundo es más eficiente que ancho.

Dataset / recursos

- `sklearn.datasets.make_moons(n_samples=500, noise=0.2)` — clásico para mostrar no linealidad.
- XOR como dataset de 4 puntos (ejercicio motivacional).
- Librerías: tensorflow, keras (incluido), numpy, matplotlib.

Ejercicios

1. Perceptrón a mano: implementá un perceptrón en numpy y entrenalo en AND/OR (separables). Después probá con XOR; mostrará que no converge.
2. MLP con Keras: `model = keras.Sequential([Dense(8, activation='relu', input_shape=(2,)), Dense(1,`

activation='sigmoid'))). Entrenalo sobre XOR; debería llegar a accuracy 1.0.

3. Visualización decision boundary: con make_moons, entrená un MLP [16, 8] y graficá la frontera con un meshgrid.
4. Backprop a mano: para un MLP con 1 input, 1 hidden (2 neuronas), 1 output, con MSE, calculá $\partial L / \partial w$ para una muestra y comparalo con tf.GradientTape.
5. ¿Y sin activación no lineal?: cambiá las activaciones a linear en el MLP de XOR y mostrá que ya no puede aprenderlo.

Homework verificable

Notebook que:

1. Carga make_moons(noise=0.2, random_state=42).
2. Entrena 2 modelos: regresión logística (Parte 1) vs MLP [16, 8] con ReLU.
3. Reporta accuracy en test para ambos.
4. Grafica las dos decision boundaries lado a lado.
5. Explica en 3 líneas por qué el MLP captura la curvatura y la regresión logística no.

Criterio de aceptación: el MLP debe lograr ≥ 0.95 accuracy y la frontera debe ser claramente curva; la regresión logística queda en ≈ 0.85 con frontera recta.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MLP no aprende XOR aunque la arquitectura	Probablemente sin activación no lineal, o
Loss nan desde la primera época	Inicialización mala + activación + LR. Fix
Modelo sigmoid de salida con loss='mse'	Lento y mal calibrado. Fix: loss='binary_c
input_shape mal especificado: (2,) vs 2	Tiene que ser tupla. Fix: Dense(8, input_s
Modelo memoriza train y mal en test con XO	XOR son 4 puntos — no hay nada para genera

Preguntas frecuentes

¿Una sola neurona equivale a regresión logística?

Sí, exactamente: $\text{sigmoid}(w \cdot x + b)$ con cross-entropy = logistic regression. Por eso un MLP "es regresión logística apilada con no linealidades en medio".

¿Por qué no se podía entrenar redes profundas hasta los 2000s?

Por tres problemas: vanishing gradients (sigmoid satura), datasets chicos, hardware limitado. Lo resolvieron ReLU (2010), GPUs y ImageNet (2012). Lo verás en Clase 096.

¿Cuántas capas y neuronas pongo?

Empezá chico: 1-2 capas ocultas, 16-128 neuronas. Si underfittea, ensanchá o profundizá. Si overfittea, achicá o regularizá. La arquitectura se busca empíricamente con Keras Tuner (Clase 095) u Optuna.

¿Qué es el "Universal Approximation Theorem" en la práctica?

Que existe un MLP que aproxima tu función. No dice nada sobre cuán difícil es encontrarlo (el problema de optimización). Por eso "1 capa muy ancha" en teoría alcanza, pero "10 capas más angostas" en práctica converge mucho mejor.

¿Forward + backward = una época?

No. Una iteración (= un batch) es 1 forward + 1 backward + 1 update. Una época es haber visto todo el dataset una vez (= $n_samples / batch_size$ iteraciones).

Referencias

- Géron, cap. 10 — Introduction to Artificial Neural Networks with Keras.
- Rumelhart, Hinton & Williams (1986), Learning representations by back-propagating errors, Nature — paper original de backprop.
- Goodfellow, Bengio & Courville (2016), Deep Learning, cap. 6 (online: <https://www.deeplearningbook.org/>).
- 3Blue1Brown — serie "But what is a neural network?" (4 videos, intuición visual de backprop).
- Keras docs — Sequential model.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 101 — Clase 101 — Regresión y clasificación con MLP

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § Building an Image Classifier y § Building a Regression MLP. Duración estimada: 75 min.

Objetivo

Saber construir y entrenar un MLP para los tres tipos de problemas tabulares estándar — regresión, clasificación binaria y clasificación multiclase — eligiendo correctamente la activación de salida y la loss para cada caso. Hacer un train/val/test split adecuado y leer las curvas de entrenamiento.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Mapear problema → activación de salida → loss: regresión → linear + MSE; binario → sigmoid + binary_crossentropy; multiclase → softmax + sparse_categorical_crossentropy.
- Hacer split train / validation / test con train_test_split y pasar validation_data a model.fit.
- Leer history.history['loss'] y ['val_loss'], identificar overfitting (val sube mientras train baja).
- Aplicar EarlyStopping y ModelCheckpoint callbacks como protección estándar.
- Diferenciar sparse_categorical_crossentropy (labels enteros) de categorical_crossentropy (labels one-hot).

Temas

- Mapeo problema → arquitectura de salida + loss.
- Activación de salida: linear, sigmoid, softmax.
- Train/val/test split — por qué hace falta los tres (val para selección, test para reporte final).
- Curvas de aprendizaje: lectura visual (subfitting / overfitting).
- Callbacks: EarlyStopping(patience=5, restore_best_weights=True), ModelCheckpoint.
- Normalización de inputs con Normalization() layer o StandardScaler.

Definiciones y características

- linear activation: sin transformación. Salida no acotada. Para regresión.
- sigmoid: $1/(1+e^x)$, sale en (0, 1). Una neurona = probabilidad binaria.

- softmax: $e^{x_i} / \sum e^{x_j}$. Convierte logits en distribución de probabilidad sobre K clases.
- MSE: $\text{mean}((y - \hat{y})^2)$. Para regresión. Sensible a outliers (cuadrado).
- MAE: $\text{mean}(|y - \hat{y}|)$. Robusto a outliers.
- Binary Cross-Entropy: $-(y \cdot \log(p) + (1-y) \cdot \log(1-p))$. Para 2 clases.
- Sparse Categorical Cross-Entropy: como categorical pero con labels enteros ([0, 2, 1, ...]). Más eficiente y común.
- EarlyStopping: corta entrenamiento cuando val_loss deja de bajar por patience épocas.

Dataset / recursos

- Regresión: `sklearn.datasets.fetch_california_housing()`.
- Clasificación binaria: `sklearn.datasets.load_breast_cancer()`.
- Multiclase: `keras.datasets.fashion_mnist.load_data()` (10 clases).
- Librerías: tensorflow, keras, scikit-learn, matplotlib.

Ejercicios

1. MLP regresión: California Housing, MLP [64, 32] con `Normalization()`, salida linear, loss mse. Reportar MAE en test.
2. MLP binario: breast_cancer, MLP [32, 16], salida sigmoid, loss binary_crossentropy. Reportar accuracy y AUC.
3. MLP multiclase: Fashion-MNIST (aplastado a 784), MLP [256, 128], salida softmax(10), loss sparse_categorical_crossentropy. Reportar accuracy.
4. Curvas y overfitting: entrenar el modelo 3 por 50 épocas sin early stopping. Graficar loss y val_loss; identificar la época donde arranca overfitting.
5. EarlyStopping: repetir con `EarlyStopping(patience=5, restore_best_weights=True)`. Verificar que cortó antes y los pesos guardados son los mejores.

Homework verificable

Fashion-MNIST end-to-end:

1. Cargar y normalizar (/ 255).
2. Split train (50 000) / val (10 000) / test (10 000).
3. MLP [300, 100], ReLU, salida softmax.
4. Compilar con `optimizer='adam'`, `loss='sparse_categorical_crossentropy'`, `metrics=['accuracy']`.
5. Entrenar con `EarlyStopping(patience=5)` y `ModelCheckpoint('best.keras', save_best_only=True)`.
6. Reportar accuracy en test y matriz de confusión.

Criterio de aceptación: accuracy en test ≥ 0.87 ; matriz de confusión muestra que las prendas más confundidas son Shirt / T-shirt / Pullover.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
loss = nan desde la época 1	Inputs sin escalar + LR alta. Fix: Standar
softmax + binary_crossentropy	Mismatch. Fix: softmax va con (sparse_)cat
categorical_crossentropy con labels entero	Espera one-hot. Fix: usar sparse_categoric
accuracy = 0.10 en Fashion-MNIST sin entre	Predicción aleatoria. Fix: <code>model.fit(...)</code> .
Reportar accuracy en val como "test final"	Validation está contaminada por la búsqueda

Preguntas frecuentes

¿Cuántas neuronas por capa?

Primera capa entre `n_features` y `4·n_features`; capas posteriores más angostas (pirámide invertida). Ej.: 784 → 256 → 128 → 10. Refinar con tuning (Clase 095).

¿Qué optimizador uso?

Adam con LR default ($1e-3$) es el caballito de batalla. Para producción, AdamW con LR ajustada (Clase 102).

¿Cuántas épocas?

Las que decidan tus callbacks. Setear `epochs=100 + EarlyStopping(patience=10)`.

¿Hay que normalizar siempre?

Siempre para features con escalas distintas. Para imágenes basta / 255. Sin normalizar, el optimizador zigzaguea.

¿Por qué la última capa de Fashion-MNIST tiene 10 neuronas?

Una neurona por clase. Softmax convierte los 10 logits en probabilidades que suman 1. El `argmax` es la predicción.

Referencias

- Géron, cap. 10 — Building an Image Classifier Using the Sequential API y Building a Regression MLP.
- Keras docs — Training & evaluation.
- Keras docs — Callbacks API.
- Glorot & Bengio (2010), Understanding the difficulty of training deep feedforward neural networks.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 102 — Clase 102 — Keras Sequential API

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § The Sequential API. Duración estimada: 60 min.

Objetivo

Dominar la Sequential API de Keras — la forma más simple y declarativa de construir un modelo cuando es una pila lineal de capas. Saber cuándo NO alcanza (cualquier topología con ramas, skip connections, multi-input/multi-output → Functional API, clase 093).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un modelo con `keras.Sequential([...])` o `model.add(...)` incrementalmente.
- Inspeccionar la arquitectura con `model.summary()` (parámetros por capa, output shape, total).
- Calcular a mano el número de parámetros de una `Dense(n)` ($= \text{input_dim} * n + n$ por el bias).
- Compilar (`compile`), entrenar (`fit`), evaluar (`evaluate`) y predecir (`predict`).
- Guardar y cargar con el formato moderno `.keras` (HDF5 legacy).

Temas

- Dos formas equivalentes: lista en el constructor vs `.add()` incremental.

- Input layer explícito vs `input_shape` en la primera capa.
- `model.summary()`: leer parámetros por capa + total trainable / non-trainable.
- `compile(optimizer, loss, metrics)`.
- `fit(X, y, epochs, batch_size, validation_split, callbacks, verbose)`.
- `model.save('m.keras')` (formato nativo Keras 3+) y `keras.models.load_model('m.keras')`.

Definiciones y características

- Sequential model: stack lineal — la salida de una capa es la entrada de la siguiente. No permite branching, skip connections, ni múltiples inputs/outputs.
- Parámetros trainable: pesos + bias que el optimizador actualiza. `Dense(n_out)` aplicada a entrada `n_in`: $n_{in} * n_{out} + n_{out}$ params.
- Parámetros non-trainable: capas como `BatchNormalization` tienen estadísticas (running mean/var) que no se actualizan por backprop sino por moving average.
- .keras format: ZIP con `config.json` + `metadata.json` + pesos. Reemplaza al .h5 desde Keras 3.
- `batch_size`: cuántas muestras procesa antes de actualizar pesos. Default 32. Más grande = más estable pero menos updates por época.

Dataset / recursos

- Reutilizar Fashion-MNIST de la clase anterior.
- Librerías: tensorflow, keras (≥ 3.0).

Ejercicios

1. Dos sintaxis: construir el mismo modelo dos veces — una con `Sequential([...])` y otra con `model = Sequential(); model.add(...)`. Verificar que `summary()` produce el mismo resultado.
2. Conteo de parámetros: para `Sequential([Dense(128, input_shape=(784,)), Dense(64), Dense(10)])`, calcular a mano los parámetros y verificar contra `model.summary()`.
3. Guardado/carga: entrenar 5 épocas, `model.save('m.keras')`, recargar con `load_model`, verificar que `predict` da idéntico.
4. Predict en batch vs individual: predecir 1 sola muestra (¿cómo cambia la shape?) vs predecir 100. Cuidado con la dimensión batch.
5. Verbose: probá `fit(..., verbose=0)`, `verbose=1` (barra), `verbose=2` (1 línea por época). Útil para notebooks vs scripts.

Homework verificable

Entrenar tres arquitecturas distintas sobre Fashion-MNIST y comparar:

1. Tiny: `Dense(64) → Dense(10)`.
2. Medium: `Dense(256) → Dense(128) → Dense(10)`.
3. Wide: `Dense(1024) → Dense(10)`.

Reportar: # parámetros, accuracy en test, tiempo de entrenamiento.

Criterio de aceptación: el medium debe ganar en accuracy/tiempo balanceado; el wide tiene más parámetros que el medium pero NO necesariamente mejor accuracy (lección: profundo > ancho).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>ValueError: Input shape mismatch</code> al cargar	El modelo guardado tiene un input shape di
<code>model.summary()</code> muestra ? en output shape	No definiste el input shape de la primera

Olvido compile() y fit() falla	Sin compile, no hay optimizer/loss. Fix: m
Modelo guardado en .h5 y se quiere cargar	.h5 está deprecado. Fix: model.save('m.keras')
model.add() después de compile() y fit() y	No se puede modificar la arquitectura post

Preguntas frecuentes

¿Cuándo Sequential no alcanza?

Cuando hay branching (skip connections de ResNet), multiple inputs (modelos de fusión), multiple outputs (multi-task), o capas compartidas. Para todo eso → Functional API (clase 093).

¿Es más lento Sequential que Functional?

No, son la misma cosa por dentro. La diferencia es solo sintáctica.

¿Por qué Input separado y no input_shape=?

Equivalentes funcionalmente. Input(shape=(...)) es más explícito y es lo único que funciona en Functional API; mantenerlo es buena costumbre.

¿batch_size=32 siempre?

Depende. Imágenes: 32–128. Texto/NLP: a menudo 8–16 por memoria. Tabular grande: 256–1024. Probá; el batch_size afecta no solo velocidad sino también la generalización (batches grandes tienden a converger a minima "más planos" pero peor generalizables).

¿validation_split=0.2 o validation_data=(X_val, y_val)?

validation_split=0.2 toma el último 20 % del array; útil si los datos están bien mezclados. validation_data=(...) es explícito y obligatorio si los datos están ordenados.

Referencias

- Géron, cap. 10 — The Sequential API.
- Keras — The Sequential model.
- Keras 3 release notes.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 103 — Clase 103 — Keras Functional API y Subclassing

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § Building Complex Models Using the Functional API y § Building Dynamic Models Using the Subclassing API. Duración estimada: 70 min.

Objetivo

Construir modelos con topologías no lineales —skip connections, multi-input, multi-output, capas compartidas— usando la Functional API (estilo "grafo de capas"), y modelos con flujo de control dinámico (loops, ifs) usando Subclassing (estilo class MyModel(Model) con call()). Saber elegir entre las tres APIs según el caso.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un modelo Wide & Deep (clásico de Cheng et al. 2016) con Functional API.
- Construir un modelo multi-output con dos Dense finales y dos losses.
- Implementar un MyResBlock con Subclassing que tiene una skip connection.
- Reconocer el trade-off: Sequential (simple) → Functional (la mayoría de los casos) → Subclassing (cuando hace falta control dinámico).
- Convertir un modelo Functional en JSON / cargar con `model_from_json` (útil para serialización separada de pesos).

Temas

- Functional API: `x = layer(prev_x)`; al final `Model(inputs=[...], outputs=[...])`.
- Multi-input / multi-output: pasar listas o dicts a `inputs=` / `outputs=`.
- Capas compartidas (siamese networks): aplicar la misma instancia de capa a dos entradas distintas.
- Subclassing: heredar de `keras.Model`, definir `__init__` (capas) y `call(self, inputs, training=False)`.
- Cuándo subclassing: control de flujo dinámico, modelos imperativos estilo PyTorch.

Definiciones y características

- Functional API: cada capa se llama como función `output = LayerInstance()(input)`; permite cualquier DAG. Visualizable con `keras.utils.plot_model`.
- Subclassing: Model custom donde `call()` define el forward pass arbitrariamente. Más flexible pero más difícil de serializar y debuggear.
- Skip connection: `y = Add()(x, F(x))` — clave en ResNet (clase 115).
- Capa compartida: `shared = Dense(32)`; `a = shared(x1)`; `b = shared(x2)`. Misma matriz de pesos en ambos.
- `call(self, inputs, training=False)`: el flag `training` distingue forward de entrenamiento (con dropout activo) vs inferencia.

Dataset / recursos

- California Housing para Wide & Deep.
- Librerías: tensorflow, keras.

Ejercicios

1. Wide & Deep: implementá el modelo de Cheng et al. con Functional API — input → wide path (directo a salida) + deep path (`Dense × 2` → salida) → `Add()` → output.
2. Multi-output: predecir simultáneamente precio (regresión) Y rango de precio (clasificación 3 clases) sobre California Housing. Compilar con dict de losses.
3. Capa compartida (siamese): dos imágenes de entrada → mismo encoder `Dense(64)` aplicado a ambas → concatenar embeddings → clasificación "same/different".
4. ResBlock con Subclassing: implementá una clase `ResBlock(Layer)` con dos Dense y skip connection. Usala en un modelo Sequential.
5. `plot_model`: graficá el modelo Wide & Deep con `keras.utils.plot_model(model, show_shapes=True)`. Identificá visualmente las dos rutas.

Homework verificable

Implementar un modelo multi-output en California Housing:

1. Predecir `median_house_value` (regresión, loss MSE).
2. Predecir `expensive` (binario: above/below mediana, loss BCE).
3. Compartir las 2 primeras capas Dense (representación común) y luego ramificar.

4. Compilar con `loss=[mse, bce]`, `loss_weights=[0.7, 0.3]`.
5. Entrenar y reportar MAE + accuracy.

Criterio de aceptación: el modelo entrena sin errores, MAE razonable ($< \$50k$), $accuracy \geq 0.85$; `plot_model` muestra la rama compartida + dos heads.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
ValueError: A merge layer should be called	<code>Add()(x, y)</code> en vez de <code>Add()([x, y])</code> . Fix:
Subclass <code>call()</code> no respeta <code>training</code> y <code>drop</code>	Fix: aceptar <code>training=False</code> como argumento
<code>model.save()</code> falla en subclass	Subclassing requiere implementar <code>get_config</code>
Multi-output con <code>loss='mse'</code> (un solo strin	Aplica MSE a las dos salidas. Fix: pasar l
Capa creada dentro de <code>call()</code> en cada forwa	Crea pesos nuevos cada vez → no aprende. F

Preguntas frecuentes

¿Functional o Subclassing?

Functional por default. Es más fácil de serializar, visualizar y debuggear. Subclassing solo si necesitás control de flujo dinámico (loops, ifs sobre el input — raro en práctica salvo RNN custom).

¿Puedo mezclar Sequential dentro de Functional?

Sí. `inner = Sequential([Dense(64), Dense(32)])`; `out = inner(x)` — Sequential es un Layer válido.

¿Functional es más lento que Sequential?

No, mismo grafo por debajo.

¿Cuándo necesito multi-input?

Cuando tenés modalidades distintas (imagen + metadatos del usuario, texto + features numéricas). Cada modalidad tiene su input y su encoder; al final se concatenan.

¿Add() o Concatenate()?

`Add()` para skip connections (preserva dimensión). `Concatenate()` para fusión de modalidades (suma dimensiones).

Referencias

- Géron, cap. 10 — The Functional API y The Subclassing API.
- Cheng et al. (2016), Wide & Deep Learning for Recommender Systems, DLRS.
- Keras — The Functional API.
- Keras — Making new layers and models via subclassing.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 104 — Clase 104 — Callbacks, TensorBoard, guardar/restaurar modelos

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § Using Callbacks y § Using TensorBoard for Visualization. Duración estimada: 60 min.

Objetivo

Inyectar lógica al loop de entrenamiento sin modificarlo, mediante callbacks (EarlyStopping, ModelCheckpoint, ReduceLROnPlateau, custom). Visualizar el progreso del training en TensorBoard (loss, métricas, histogramas de pesos, embeddings). Saber guardar y restaurar correctamente — arquitectura + pesos + estado del optimizador.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar los 4 callbacks más usados: EarlyStopping, ModelCheckpoint, ReduceLROnPlateau, TensorBoard.
- Configurar TensorBoard: `keras.callbacks.TensorBoard(log_dir='./logs')` y abrirlo con `tensorboard --logdir=./logs`.
- Escribir un callback custom heredando de `keras.callbacks.Callback` con hooks como `on_epoch_end`.
- Distinguir guardado completo (`model.save('m.keras')`) vs solo pesos (`model.save_weights('w.weights.h5')`).
- Restaurar y continuar entrenamiento desde checkpoint sin pérdida.

Temas

- Callbacks: hooks (`on_train_begin`, `on_epoch_end`, `on_batch_end`, ...).
- EarlyStopping(`monitor='val_loss'`, `patience=10`, `restore_best_weights=True`).
- ModelCheckpoint(`filepath`, `save_best_only=True`, `monitor='val_accuracy'`, `mode='max'`).
- ReduceLROnPlateau(`factor=0.5`, `patience=5`) — bajar LR cuando se estanca.
- TensorBoard: scalars, histograms, distributions, images, projector (embeddings).
- Custom callbacks: `class MyCallback(keras.callbacks.Callback): def on_epoch_end(self, epoch, logs): ....`

Definiciones y características

- Callback: objeto con métodos hook que Keras invoca en momentos específicos del training. Permite logging, early stopping, LR scheduling, etc.
- EarlyStopping: monitorea una métrica y corta cuando deja de mejorar. `restore_best_weights=True` revierte a los mejores pesos vistos.
- ModelCheckpoint: guarda el modelo (o solo pesos) cuando una métrica mejora.
- ReduceLROnPlateau: reduce el LR multiplicándolo por factor cuando la métrica se estanca por `patience` épocas.
- TensorBoard: la herramienta de visualización oficial de TF. Sirve via `tensorboard --logdir=....`
- logs dict: contiene loss, val_loss, métricas, etc. Disponible en hooks `on_epoch_end`.

Dataset / recursos

- Fashion-MNIST o cualquier modelo entrenable de clases anteriores.
- Librerías: tensorflow, keras, tensorboard (incluido).

Ejercicios

1. EarlyStopping + Checkpoint: entrenar Fashion-MNIST con ambos callbacks. Verificar que cortó cuando `val_loss` se estancó y que 'best.keras' contiene los mejores pesos.
2. TensorBoard: agregar `TensorBoard(log_dir=f'./logs/run-{time}')`, entrenar 10 épocas. Lanzar `tensorboard --logdir=./logs` y revisar scalars + histograms.
3. ReduceLROnPlateau: configurar `factor=0.5`, `patience=3`, `min_lr=1e-6`. Graficar el LR a lo largo de las épocas (usar el logs del callback).

4. Custom callback: escribir uno que loggee a un CSV el (epoch, loss, val_loss, lr_actual) para análisis offline.
5. Restaurar y continuar: entrenar 10 épocas, guardar, recargar y continuar 5 épocas más. Verificar que el optimizer state (momentum de Adam) se preservó.

Homework verificable

Entrenamiento "production-ready" sobre Fashion-MNIST:

1. MLP [300, 100].
2. Callbacks: EarlyStopping(patience=10), ModelCheckpoint('best.keras', save_best_only=True), ReduceLROnPlateau(patience=3), TensorBoard('./logs/').
3. Entrenar epochs=100 (sabiendo que EarlyStopping cortará antes).
4. Recargar best.keras y evaluar en test.
5. Capturar un screenshot del TensorBoard mostrando loss y val_loss a lo largo del training.

Criterio de aceptación: el modelo final restaurado debe ser el del epoch con menor val_loss; accuracy en test ≥ 0.87 .

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
EarlyStopping corta en la primera época	Default es patience=0. Fix: patience=5-10
ModelCheckpoint guarda en cada época con s	Llena el disco. Fix: save_best_only=True.
model.save_weights('w.h5') y al cargar dic	Hay que reconstruir la arquitectura antes
TensorBoard no muestra nada en localhost:6	El log_dir está mal o aún no se escribió n
ReduceLROnPlateau y LearningRateScheduler	Conflicto, ambos cambian el LR. Fix: elegí

Preguntas frecuentes

¿save_best_only=True con qué métrica?

monitor='val_loss' por default (modo min). Si monitoreás accuracy, agregá mode='max'.

¿Puedo usar varios ModelCheckpoint?

Sí. Útil para guardar el mejor por val_loss y el mejor por val_accuracy en dos archivos.

¿TensorBoard funciona en Colab?

Sí: %load_ext tensorboard + %tensorboard --logdir=./logs. Funciona inline.

¿Custom callback necesita herencia explícita?

Sí: class MyCB(keras.callbacks.Callback). La base provee self.model y manejo de logs.

¿Format .keras vs .h5?

.keras (Keras 3+) es zip transparente, futuro-proof. .h5 aún funciona pero deprecado para nuevos proyectos.

Referencias

- Géron, cap. 10 — Using Callbacks y Using TensorBoard.
- Keras callbacks.
- TensorBoard quickstart.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 105 — Clase 105 — Keras Tuner (+ Optuna, Ray Tune)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 10 § Fine-Tuning Neural Network Hyperparameters + docs Keras Tuner / Optuna / Ray Tune. Duración estimada: 85 min.

Objetivo

Hacer hyperparameter tuning sistemático en redes neuronales — buscar `n_layers`, `units`, `lr`, `dropout`, etc. con estrategias modernas: Random Search, Hyperband y Bayesian Optimization. Conocer las tres herramientas estándar de Python — Keras Tuner (TF-native, simple), Optuna (multi-framework, default industrial) y Ray Tune (distribuido, escalable a clusters).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir un model-building function con hiperparámetros declarados vía `hp.Int`, `hp.Float`, `hp.Choice`.
- Lanzar tuners de Keras Tuner: `RandomSearch`, `Hyperband`, `BayesianOptimization`.
- Migrar el mismo problema a Optuna con `optuna.create_study(direction='minimize')` y `trial.suggest_float`.
- Lanzar Ray Tune con `tune.run` para distribuir trials en múltiples GPUs/nodos.
- Comparar las 3 estrategias (Random vs Hyperband vs Bayesian) en términos de eficiencia.

Temas

- ¿Por qué tuning? Los defaults (Adam `lr=1e-3`, `dropout 0.5`) rara vez son óptimos.
- Random Search vs Grid Search: Bergstra & Bengio (2012) — random gana casi siempre por el "curse of low effective dimensionality".
- Hyperband (Li et al. 2017): asignar más cómputo a configs prometedoras (successive halving).
- Bayesian Optimization (TPE, GP): construye un modelo del paisaje y elige el siguiente trial inteligentemente.
- Trade-off cómputo vs ganancia: típicamente 50-200 trials para una primera optimización.
- Complemento moderno: Optuna y Ray Tune como alternativas multi-framework.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 052a — Optuna y HPO bayesiano dedicado (ML clásico)
- Clase 095b — Ray Tune: HPO distribuido y a escala

Definiciones y características

- Search space: el rango/conjunto donde busca cada hiperparámetro.
- Trial: una corrida de entrenamiento con una config específica.
- Random Search: muestrea trials al azar del espacio.
- Hyperband: corre muchos trials cortos, mata a los peores, sigue con los buenos por más épocas (successive halving en múltiples brackets).
- Bayesian Optimization / TPE (Tree-structured Parzen Estimator): modela $P(\text{config} \mid \text{resultado bueno})$ y muestrea de ahí.

- Pruner: lógica para matar trials sin terminar (callback en cada época).
- log=True en suggest_float: muestreo log-uniform — esencial para LR.

Dataset / recursos

- Fashion-MNIST como dataset chico para iterar rápido.
- Librerías: keras-tuner (pip install keras-tuner), optuna, ray[tune].

Ejercicios

1. Keras Tuner — RandomSearch: tunear units {32, 64, 128} y lr [1e-4, 1e-2] log con 20 trials. Reportar mejores hiperparámetros y val_accuracy.
2. Keras Tuner — Hyperband: el mismo espacio, 50 trials con Hyperband. Comparar tiempo total vs RandomSearch.
3. Optuna: traducir el espacio a Optuna; correr 50 trials con HyperbandPruner; graficar plot_optimization_history y plot_param_importances.
4. Multi-objective: optimizar simultáneamente val_accuracy (max) y n_params (min) con optuna.create_study(directions=['maximize', 'minimize']). Inspeccionar la Pareto front.
5. Visualización: con Optuna, generar plot_parallel_coordinate(study) y entender qué dimensiones son las más sensibles.

Homework verificable

Tunear un MLP sobre Fashion-MNIST con Optuna:

1. Espacio: n_layers [1, 4], units {32, 64, 128, 256}, dropout [0, 0.5], lr [1e-5, 1e-2] log, optimizer {Adam, SGD}.
2. 100 trials con HyperbandPruner, n_jobs=2.
3. Reportar best_params, best_value, y guardar el modelo final entrenado con esos params.
4. Generar los 3 plots de visualización.

Criterio de aceptación: val_accuracy del mejor modelo ≥ 0.89 ; plot_param_importances debe mostrar lr y/o units como las más importantes (típico).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
LR muestreado uniforme entre 1e-5 y 1e-2 y	LR debe ser log-uniform. Fix: trial.suggests
Bayesian optimization parece random	Necesita >10-20 trials para que el modelo
Hyperband poda trials demasiado pronto	factor muy alto o max_epochs muy bajo. Fix
optuna se cuelga al usar n_jobs > 1 con Ke	TF tiene problemas con multi-threading. Fi
Reportar el resultado del mejor trial del	Eso es el resultado de búsqueda, no de eva

Preguntas frecuentes

¿Cuántos trials?

Mínimo 20–50 para Random / Hyperband. Bayesian se beneficia de más (100+). Si cada trial cuesta 1h → 100 trials = ~4 días en 1 GPU; con Ray Tune en 4 GPUs, ~1 día.

¿Tuning del LR primero o del resto?

LR primero, siempre. Es de lejos el hiperparámetro más sensible. Después arquitectura, después regularización.

¿Optuna o Keras Tuner para alguien que recién empieza?

Keras Tuner para entender la idea (más simple). Optuna apenas el problema es serio. La transición es rápida.

¿Cómo guardo el resultado del estudio Optuna?

`optuna.create_study(study_name='exp1', storage='sqlite:///optuna.db', load_if_exists=True)`. Persistencia gratis; podés agregar trials después.

¿Ray Tune en una sola máquina vale la pena?

Sí si tenés ≥ 4 cores o ≥ 2 GPUs. Optuna `n_jobs` también paraleliza pero Ray maneja mejor recursos heterogéneos y crashes.

Referencias

- Géron, cap. 10 — Fine-Tuning Neural Network Hyperparameters.
- Bergstra & Bengio (2012), Random Search for Hyper-Parameter Optimization, JMLR.
- Li et al. (2017), Hyperband, JMLR.
- Akiba et al. (2019), Optuna: A Next-Generation Hyperparameter Optimization Framework, KDD.
- Liaw et al. (2018), Tune: A Research Platform for Distributed Model Selection and Training.
- Keras Tuner docs, Optuna docs, Ray Tune docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 106 — Clase 106 — Ray Tune: HPO distribuido y a escala

Parte: 2 — Deep Learning · Fuente: Liaw et al. (2018) Ray Tune + Ray docs. Duración estimada: 75 min.

Objetivo

Escalar hyperparameter tuning de DL a cluster con Ray Tune — el framework distribuido que la industria (Uber, Anyscale, OpenAI) usa cuando los trials toman horas y se necesitan decenas en paralelo. Cubrir orquestación, schedulers modernos (ASHA, PBT Population Based Training), integración con W&B, MLflow, y combinación con Optuna como search algorithm.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir trainable(config) y reportar progreso con `tune.report(loss=...)`.
- Configurar ASHA (Async Successive Halving) para podar trials malos.
- Aplicar Population Based Training (PBT) para evolución de hyperparams durante training.
- Asignar recursos: `resources_per_trial={'cpu': 2, 'gpu': 1}`.
- Usar OptunaSearch como search algorithm + Ray Tune como orchestrator.

Temas

- Ray cluster: local vs multi-node.
- Trainable: function-based vs class-based.
- ASHA scheduler — async successive halving.

- PBT — evoluciona hyperparams + checkpoints.
- BOHB — Bayesian opt + Hyperband.
- Loggers: TensorBoard, W&B, MLflow.

Definiciones y características

- Trial: una corrida del trainable.
- Scheduler: decide qué trial para, cuál sigue.
- ASHA: variante asíncrona de Successive Halving — mata trials malos rápido.
- PBT: evolutionary — copia weights de mejores + perturba hyperparams.
- Search algorithm: cómo se eligen configs (random, TPE via Optuna, etc.).

Dataset / recursos

- Fashion-MNIST o cualquier dataset DL.
- Librerías: ray[tune], opcional optuna, lightning.

Ejercicios

1. Trainable básico: train de CNN con metric report each epoch. `tune.run(trainable, num_samples=20)`.
2. ASHA: `ASHAScheduler(metric='val_loss', mode='min', max_t=20, grace_period=3)`. Verificar pruning.
3. PBT: 8 workers, copy + perturb each 5 epochs. Plot evolution de LR.
4. OptunaSearch + ASHA: combination — Optuna sugiere, ASHA poda.
5. Resources: `gpus_per_trial=0.5` (fractional GPU sharing).

Homework verificable

Sobre Fashion-MNIST con CNN simple:

1. Tunear LR, batch_size, dropout con Ray Tune.
2. 50 trials, ASHA, OptunaSearch.
3. W&B logging.
4. Reportar mejor config + tiempo total.

Criterio de aceptación: convergencia visible en W&B; pruning de ASHA mata ≥ 30 % trials malos temprano; mejor config supera baseline.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Ray cluster crash con OOM	Workers piden mucha RAM. Fix: <code>resources_pe</code>
PBT requiere checkpointing	Si no lo implementás, PBT falla. Fix: <code>tune</code>
Trainable con loop infinito sin <code>tune.report</code>	Tune no sabe progreso. Fix: report cada ep
ASHA + <code>grace_period</code> bajo	Mata trials antes de aprender. Fix: <code>grace_</code>
Multi-node sin configurar Ray cluster	Trials van a 1 node. Fix: <code>ray.init(address</code>

Preguntas frecuentes

Ray Tune vs Optuna directo?

Optuna: single-machine, simple. Ray Tune: cluster, mejor orquestación de recursos.

PBT cuándo?

LLM training, modelos largos donde LR decay schedule matter. Para HPO simple, ASHA basta.

Cuántos workers?

Tantos como GPUs disponibles. Local: `cpu_count - 2`.

Combinar con Lightning?

Sí — `ray_lightning` para trainer distribuido + Tune para HPO. Más compleja la setup.

Anyscale (Ray managed)?

Cloud service from Ray creators. Para producción serio sin manejar infra propia.

Referencias

- Liaw et al. (2018), Tune: A Research Platform for Distributed Model Selection and Training.
- Li et al. (2018), ASHA.
- Jaderberg et al. (2017), Population Based Training, DeepMind.
- Ray Tune docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 107 — Clase 107 — Vanishing/exploding gradients

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § The Vanishing/Exploding Gradients Problems.
Duración estimada: 70 min.*

Objetivo

Entender el problema central que estancó el Deep Learning hasta 2010: cuando un gradiente atraviesa muchas capas, se desvanece (sigmoid/tanh saturadas $\rightarrow$ multiplicas números < 1 muchas veces $\rightarrow \approx 0$) o explota (pesos grandes $\rightarrow > 1$ muchas veces $\rightarrow \infty$). Identificar los 4 culpables (activación, inicialización, profundidad, LR) y conocer las soluciones que destrabaron el campo: Glorot/He init (097), ReLU y variantes (098), BatchNorm (099), Gradient clipping (100).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diagnosticar vanishing gradient: gradients en capas tempranas con norma $\sim 1e-8$.
- Diagnosticar exploding: loss = nan o gradients con norma $\sim 1e+10$.
- Inspeccionar gradientes con `tf.GradientTape + tf.norm`.
- Mapear cada solución al problema: init para arrancar bien, ReLU para no saturar, BN para estabilizar, clipping para evitar explosión.
- Explicar por qué sigmoid en MLPs profundos no escala (derivada máx. 0.25 $\rightarrow$ gradiente decae rápido).

Temas

- Backprop como producto de derivadas a lo largo de capas.
- Sigmoid: $\sigma(x) \cdot (1 - \sigma(x))$ máxima 0.25 en $x=0$; satura a 0 en colas.
- Tanh: derivada máx. 1, pero también satura.
- ReLU: derivada 0 o 1 — no decae al multiplicar, pero genera "dying ReLU".
- Inicialización mala: pesos $N(0, 1) \rightarrow$ activaciones explotan; pesos $N(0, 0.01) \rightarrow$ vanishing.
- BatchNorm: normaliza dentro del forward pass para que cada capa reciba inputs con varianza

controlada.

Definiciones y características

- Vanishing gradient: las derivadas se acercan a 0 al propagar hacia atrás → capas tempranas no aprenden.
- Exploding gradient: las derivadas crecen sin límite → pesos divergen, loss = nan.
- Saturación: una activación está saturada cuando su derivada ≈ 0 (sigmoid en $|x| > 5$).
- Dying ReLU: una neurona ReLU queda con salida siempre 0 → gradiente 0 → no se actualiza más.
Solución: Leaky ReLU, ELU, init He.
- Gradient norm: $||\partial L / \partial W||$. Indicador visualizable durante el entrenamiento.

Dataset / recursos

- Fashion-MNIST.
- Librerías: tensorflow, keras, matplotlib.

Ejercicios

1. Diagnóstico vanishing: entrenar un MLP de 10 capas con sigmoid activations e init default. Inspeccionar gradientes de la primera capa vs la última. Verificar que los de la primera son órdenes de magnitud más chicos.
2. Mismo experimento con ReLU: comparar gradients. Mejor pero aún heterogéneo.
3. Exploding: forzar init RandomNormal(stddev=5). Observar loss = nan en pocas iteraciones.
4. Norma del gradiente por capa: con tf.GradientTape, calcular tf.norm(g) para cada peso y graficar a lo largo del entrenamiento.
5. Solución sencilla: cambiar a He init + ReLU + BatchNorm y mostrar que el problema desaparece (anticipa las siguientes 4 clases).

Homework verificable

Construir un MLP de 20 capas y reportar entrenamiento bajo 4 condiciones:

1. Sigmoid + Glorot init.
2. ReLU + Glorot init.
3. ReLU + He init.
4. ReLU + He init + BatchNorm.

Para cada uno: graficar loss durante 20 épocas + reportar val_accuracy final.

Criterio de aceptación: (1) prácticamente no aprende; (2) aprende lento; (3) aprende razonable; (4) aprende rápido y mejor. La diferencia visual debe ser dramática.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
loss = nan	Exploding. Fix: bajar LR, agregar gradient
loss se queda en valor constante alto	Vanishing. Fix: ReLU + He init.
Accuracy en train baja, val baja	El modelo no aprende. Diagnóstico: imprimir
50 % de las neuronas tienen salida 0 todo	Dying ReLU. Fix: Leaky ReLU o ELU.
Después de unas épocas la loss salta a nan	Exploding tardío. Fix: gradient clipping (

Preguntas frecuentes

¿Por qué sigmoid no funciona en MLPs profundos pero sí en la última capa?

Porque la última capa tiene solo 1 multiplicación; las capas profundas multiplican N veces. $(0.25)^{10} \approx 10^6 \rightarrow$ desaparece. En la última capa, sigmoid sí sirve para probabilidades.

¿Cuánto es "profundo"?

A partir de 5-10 capas Dense (más con BN). En CNNs, con BN o residuals se llega a 100+. En Transformers, decenas pueden no usar residuals + LayerNorm = vanishing.

¿BatchNorm "soluciona" todo?

Resuelve el vanishing/exploding casi completamente al normalizar las activaciones. Pero tiene problemas propios (batch chico, dependencia entre muestras) — clase 099 entra en detalle.

¿GELU/Swish ayudan?

Sí. Son activaciones más suaves que ReLU (sin "muerte"), usadas en transformers modernos (BERT, GPT). Clase 098.

¿Esto sigue siendo relevante con LLMs?

Sí — los LLMs usan combinación de LayerNorm + residual + GELU + init cuidadoso (Xavier/Glorot custom) precisamente para esto. Sin estas piezas, no se podrían entrenar.

Referencias

- Géron, cap. 11 — Training Deep Neural Networks.
- Hochreiter (1991), thesis — primera descripción del vanishing gradient.
- Bengio, Simard & Frasconi (1994), Learning Long-Term Dependencies with Gradient Descent is Difficult.
- Glorot & Bengio (2010), Understanding the difficulty of training deep feedforward neural networks, AISTATS.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 108 — Clase 108 — Inicialización (Glorot, He)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Glorot and He Initialization. Duración estimada: 55 min.

Objetivo

Saber inicializar los pesos de cada capa para que la varianza de las activaciones y de los gradientes se mantenga estable a lo largo del forward y backward pass. Diferenciar Glorot (Xavier) —para sigmoid/tanh— de He (Kaiming) —para ReLU y variantes—. Saber cuál usa Keras por default y cuándo cambiarlo.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la idea: $\text{Var}(W) \approx 1 / \text{fan_in}$ (o promedio $\text{fan_in}/\text{fan_out}$) para preservar varianza.
- Aplicar `kernel_initializer='glorot_uniform'` (default Keras), `'he_normal'`, `'he_uniform'`.
- Calcular a mano los límites de la distribución para Glorot uniform: $\pm\sqrt{6/(\text{fan_in}+\text{fan_out})}$.
- Reconocer que la combinación correcta es He init + ReLU, Glorot + tanh/sigmoid.

- Inspeccionar el efecto visualmente: histogramas de activaciones por capa.

Temas

- ¿Por qué importa la varianza? Productos de N capas amplifican o atenúan exponencialmente.
- Glorot (2010): $\text{Var}(W) = 2/(\text{fan_in} + \text{fan_out})$. Asume activación lineal/simétrica.
- He (2015): $\text{Var}(W) = 2/\text{fan_in}$. Compensa que ReLU "mata" la mitad de las salidas.
- Distribuciones: uniform o normal. Equivalentes prácticamente.
- LeCun init: $\text{Var}(W) = 1/\text{fan_in}$. Para SELU.

Definiciones y características

- fan_in: número de entradas a la capa (= units de la capa anterior).
- fan_out: número de salidas (= units de la capa).
- Glorot uniform: $U(-\sqrt{6/(\text{fan_in}+\text{fan_out})}, +\sqrt{6/(\text{fan_in}+\text{fan_out})})$. Default de Keras Dense.
- He normal: $N(0, \sqrt{2/\text{fan_in}})$. Recomendado para ReLU, Leaky ReLU, ELU.
- LeCun normal: $N(0, \sqrt{1/\text{fan_in}})$. Para SELU (self-normalizing networks).
- Inicialización por capa: `Dense(64, kernel_initializer='he_normal', bias_initializer='zeros')`.

Dataset / recursos

- Fashion-MNIST.
- Librerías: tensorflow, keras, matplotlib.

Ejercicios

1. Inspección de defaults: para `Dense(128, input_shape=(784,))`, imprimir `model.layers[0].kernel.numpy()`. Calcular la varianza empírica y compararla con la teórica de Glorot.
2. Comparación: entrenar MLP [512, 256, 128, 64, 10] con ReLU. Probar 3 inits: Glorot, He, `RandomNormal(stddev=0.01)`. Graficar `val_loss` en las 3.
3. Histogramas de activaciones: para cada capa del modelo bien inicializado, plot del histograma de salidas para un batch. Verificar que la varianza se mantiene similar entre capas.
4. He init + Tanh: probar la combinación incorrecta (He con tanh). Comparar contra Glorot + tanh. Verificar que importa.
5. Reset y reproducibilidad: con `tf.random.set_seed(42) + np.random.seed(42)`, entrenar 2 veces y verificar que da idéntico. Sin seed, varía.

Homework verificable

Sobre MLP [300, 200, 100, 50, 10] con ReLU sobre Fashion-MNIST:

1. Entrenar con 3 inits: default Glorot, He uniform, He normal.
2. Reportar `val_accuracy` tras 20 épocas para cada uno.
3. Para el mejor (He init), inspeccionar la norma de cada kernel antes y después del entrenamiento.

Criterio de aceptación: He init debe igualar o superar Glorot por ≥ 0.5 pp; la norma de los kernels post-entrenamiento debe ser similar entre capas (no orders of magnitude apart).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Modelo arranca lento con ReLU	Estás usando Glorot por default. Fix: kern
<code>RandomNormal(stddev=0.01)</code> por costumbre de	Vanishing inmediato. Fix: usar Glorot/He s
Reset de pesos no funciona con <code>model.set_w</code>	Hay que guardar <code>initial = [w.numpy()]</code> for w
Bias init glorot_uniform por error	Bias debe arrancar en zeros (default). Ini

Inicializar igual una capa convolucional q

fan_in para Conv es $\text{kernel_h} \times \text{kernel_w} \times$

Preguntas frecuentes

¿Glorot uniform o normal?

Para Glorot, casi indistinguible en práctica. Keras default es `glorot_uniform`. Para He, `he_normal` es ligeramente preferido pero las diferencias son ínfimas.

¿LeCun init cuándo?

Solo con SELU (activación que se auto-normaliza). Esto fue un experimento de 2017 (Klambauer et al.) que perdió tracción frente a BatchNorm + ReLU/GELU.

¿BatchNorm hace innecesaria la inicialización?

Casi. BN normaliza el output dentro del forward → init importa menos. Pero un mal init aún hace que las primeras épocas sean inestables.

¿Init para Transformers?

Combinación cuidadosa: linears con `truncated_normal(stddev=0.02)` (GPT/BERT-style), embeddings con normales escaladas. Lo verás en clase 126.

¿Por qué `fan_in` + `fan_out` en Glorot?

Es el promedio armónico de "preservar varianza en forward ($1/\text{fan_in}$)" y "preservar varianza en backward ($1/\text{fan_out}$)". Compromiso óptimo bajo activación lineal.

Referencias

- Géron, cap. 11 — Glorot and He Initialization.
- Glorot & Bengio (2010), Understanding the difficulty of training deep feedforward neural networks, AISTATS.
- He, Zhang, Ren & Sun (2015), Delving Deep into Rectifiers, ICCV — paper original de He init.
- Keras initializers.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 109 — Clase 109 — Activaciones: ReLU, ELU, GELU, Swish, Mish

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Better Activation Functions. Duración estimada: 60 min.

Objetivo

Conocer la familia de activaciones modernas — desde ReLU (Krizhevsky et al. 2012) hasta GELU (BERT, GPT) y Swish/SiLU (EfficientNet) — entendiendo qué problema resuelve cada una y por qué los Transformers modernos usan GELU y no ReLU. Saber elegir según arquitectura.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir matemáticamente las 5 activaciones: ReLU, Leaky ReLU, ELU, GELU, Swish (SiLU), Mish.
- Identificar dying ReLU y aplicar Leaky ReLU / ELU como mitigación.
- Reconocer que GELU es la activación default en Transformers (BERT, GPT, ViT) y Swish/SiLU en EfficientNet, modelos modernos de visión.
- Aplicar cada activación en Keras: Dense(64, activation='relu' | 'gelu' | 'swish' | 'elu' | LeakyReLU()).
- Saber que el costo computacional de GELU/Swish es mayor (sigmoid/erf internos) pero el beneficio supera en arquitecturas profundas.

Temas

- ReLU: $\max(0, x)$. Rápida, simple, default histórico. Dying ReLU.
- Leaky ReLU: $\max(\alpha x, x)$ con $\alpha \approx 0.01$. Sin dying.
- ELU (Clevert et al. 2015): x si $x > 0$, $\alpha(e^x - 1)$ si $x < 0$. Suave, sin dying, pero más cara.
- GELU (Hendrycks & Gimpel 2016): $x \cdot \Phi(x)$ (Φ = CDF gaussiana). Suave, no monótona. Default en Transformers.
- Swish / SiLU (Ramachandran et al. 2017): $x \cdot \text{sigmoid}(x)$. Encontrada por NAS. Casi idéntica a GELU.
- Mish (Misra 2019): $x \cdot \tanh(\text{softplus}(x))$. Marginalmente mejor en algunos benchmarks.

Definiciones y características

- Función de activación: no linealidad aplicada elementwise tras la transformación lineal de una capa.
- Saturación: cuando la derivada se acerca a 0 $\rightarrow$ vanishing.
- Monótona: $f(x)$ creciente en todo el dominio. ReLU/ELU/Leaky lo son; GELU/Swish/Mish no estrictamente.
- Bounded: salida acotada. Sigmoid (0,1), tanh (-1,1), softmax. La mayoría modernas no son bounded.
- Smooth: derivable continuamente. ReLU no en $x=0$; las demás sí.

Dataset / recursos

- Fashion-MNIST.
- Librerías: tensorflow, keras.

Ejercicios

1. Plot de funciones: graficar las 6 activaciones en $x \in [-3, 3]$.
2. Comparación empírica: entrenar MLP [256, 128, 64] con cada activación, mismo init He, mismo LR. Comparar val_accuracy tras 15 épocas.
3. Dying ReLU: con LR alto (0.1), entrenar con ReLU. Contar cuántas neuronas tienen $\text{mean}(\text{activation}) = 0$ al final.
4. Leaky ReLU al rescate: repetir con Leaky. Verificar que el % de neuronas muertas baja.
5. GELU vs ReLU en profundidad: armar un MLP de 12 capas. Comparar GELU vs ReLU. GELU suele ganar.

Homework verificable

Sobre Fashion-MNIST, MLP [300, 200, 100, 50, 10]:

1. Entrenar 4 modelos con: ReLU, Leaky ReLU ($\alpha=0.1$), ELU, GELU.
2. Reportar val_accuracy y tiempo por época.
3. Para el mejor, inspeccionar % de neuronas "muertas" por capa.

Criterio de aceptación: GELU o ELU debe ganar en accuracy; Leaky/ELU/GELU deben tener $< 5\%$ neuronas muertas; ReLU posiblemente más.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cambio de ReLU a sigmoid en capa oculta pa	Vanishing inmediato en redes profundas. Fi
Leaky ReLU con $\alpha=0.5$	Casi lineal, pierde la no linealidad útil.
activation='swish' no reconocido	Keras antiguo lo llama 'swish', TF moderno
GELU en CPU es lento	Es $\approx 2\times$ más caro que ReLU por la erf. En C
Mish en GPU sin implementación optimizada	TF puede no tener kernel cuda eficiente pa

Preguntas frecuentes

¿Qué activación uso por default en 2026?

- MLP/CNN clásica: ReLU o GELU.
- Transformers: GELU (es lo que usan BERT, GPT, ViT).
- EfficientNet/MobileNet: Swish (= SiLU).
- Modelos profundos sin BN: ELU o GELU (más estables).

Swish y SiLU son lo mismo?

Sí, mismo formato ($x \cdot \text{sigmoid}(x)$). SiLU es el nombre PyTorch; Swish es el nombre de Google.

¿Por qué GELU en Transformers?

Históricamente: BERT (2018) la eligió porque parecía funcionar mejor empíricamente. Hoy es estándar más por inercia/compatibilidad que por una ventaja categórica. En benchmarks recientes, Swish es competitiva.

¿GELU exacta o aproximada?

`tf.keras.activations.gelu(x, approximate=False)` usa erf exacta. True usa la aproximación tanh, $\approx 10\times$ más rápida en GPU. La diferencia numérica es < 0.001 .

¿La salida del modelo también lleva GELU?

No. La activación final depende del problema (linear, sigmoid, softmax). GELU/ReLU son para capas ocultas.

Referencias

- Géron, cap. 11 — Better Activation Functions.
- Krizhevsky et al. (2012), AlexNet — ReLU al mainstream.
- Clevert et al. (2015), ELU, ICLR.
- Hendrycks & Gimpel (2016), Gaussian Error Linear Units (GELUs).
- Ramachandran et al. (2017), Searching for Activation Functions, ICLR — Swish.
- Misra (2019), Mish: A Self Regularized Non-Monotonic Activation Function.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 110 — Clase 110 — Batch Normalization, Layer Normalization

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Batch Normalization. Duración estimada: 75 min.

Objetivo

Entender BatchNorm (Ioffe & Szegedy 2015) — la técnica que destrabó el entrenamiento de redes muy

profundas estandarizando las activaciones en cada capa — y su variante LayerNorm (Ba, Kiros & Hinton 2016) — usada en Transformers y RNN porque no depende del batch. Saber dónde poner BN en la arquitectura, qué problemas tiene (batch chico, distribución entre train/inference) y cuándo preferir LN.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar BatchNormalization() antes o después de la activación (debate clásico — moderno: antes suele ser mejor para ReLU, después para GELU).
- Explicar qué hace BN en train (normaliza con stats del batch) vs inference (usa moving averages acumulados).
- Aplicar LayerNormalization() en RNN y Transformers; saber por qué allí BN falla.
- Reconocer las 3 variantes: BN, LN, GroupNorm (Wu & He 2018, para batch chico en visión).
- Diagnosticar el problema de "train-test mismatch" cuando el batch en inference es muy distinto al de train.

Temas

- BN: $y = \gamma \cdot (x - \mu_{\text{batch}}) / \sigma_{\text{batch}} + \beta$. γ , β trainables.
- Beneficios: convergencia más rápida, regularización mild, permite LR más altos.
- BN en train vs inference: moving avg de μ , σ acumulados.
- LN: normaliza sobre los features de una sola muestra (no sobre el batch).
- GroupNorm: agrupa canales, normaliza dentro de cada grupo. Para batch chico (segmentación, detección).
- ¿Antes o después de la activación? Géron y la práctica moderna: antes funciona mejor con ReLU, después con GELU/Swish.

Definiciones y características

- BatchNorm: normaliza cada feature usando la media y varianza del batch actual durante training.
- γ (scale), β (shift): parámetros learnables que permiten al modelo deshacer la normalización si necesita.
- Moving averages: durante training, mantiene EMAs de μ , σ . En inference usa esos valores fijos.
- LayerNorm: normaliza sobre features de una sola muestra, independiente del batch. Default en NLP/Transformers.
- GroupNorm: agrupa N canales y normaliza dentro de cada grupo. Funciona con batch_size=1.
- InstanceNorm: como BN pero por sample individual (estilo style transfer).

Dataset / recursos

- Fashion-MNIST + un modelo profundo ([512, 256, 128, 64, 10]).
- Librerías: tensorflow, keras.

Ejercicios

1. BN vs sin BN: entrenar el mismo MLP con y sin BN. Comparar curvas de val_loss y tiempo hasta llegar a accuracy 0.85.
2. ¿Antes o después de la activación?: probar las dos variantes (Dense → BN → ReLU vs Dense → ReLU → BN). Comparar.
3. Inference mode: entrenar con BN, cambiar a training=False, predecir un batch y comparar con training=True. Las predicciones cambian (correcto).
4. Batch chico: forzar batch_size=4 y entrenar con BN. Observar inestabilidad. Cambiar a LayerNormalization y verificar que se estabiliza.
5. LayerNorm en RNN: aplicar keras.layers.LSTM con recurrent_activation y un LayerNormalization

previo. Útil como anticipo de Transformers.

Homework verificable

Sobre MLP [512, 256, 128, 64, 10] en Fashion-MNIST:

1. Entrenar 3 versiones: sin norm, con BN, con LN.
2. Reportar val_accuracy + épocas hasta val_loss < 0.4.
3. Entrenar BN con batch_size=128 y luego inferir batch por batch de tamaño 1. Verificar que el accuracy no se rompe (gracias a moving averages).

Criterio de aceptación: BN debe acelerar la convergencia ≥ 30 % (menos épocas para mismo loss); LN debe ser estable también pero típicamente algo peor en MLP feedforward.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
BN en batch_size=1 → entrenamiento inestab	μ del batch es la muestra misma → $\sigma=0$. Fix
Métricas en val muy distintas de train par	Las stats de BN en inference son distintas
Olvidar pasar training=True/False en custo	Por default es False → BN usa moving avg i
BN en una RNN	Las estadísticas cambian con la longitud d
Aplicar BN a la última capa antes de softm	Generalmente innecesario y a veces dañino

Preguntas frecuentes

¿BN reemplaza dropout?

Parcialmente. BN tiene efecto regularizador mild (el ruido del batch). En CNNs/MLPs profundos suele alcanzar; en Transformers se usa LN + dropout juntos.

¿LN o BN en Transformers?

LN siempre. BN falla porque (a) batch chico es común, (b) longitudes variables rompen las estadísticas.

¿Por qué LN no es default también en CNN?

Empíricamente BN gana en visión (los canales se benefician de stats compartidos). LN gana en NLP donde el orden temporal importa más.

¿fused=True qué hace?

BatchNormalization(fused=True) usa una implementación CUDA optimizada que fusiona ops. Default auto lo elige cuando puede. Velocidad ~ 20 % más rápido.

¿Cuánto cuesta BN en cómputo?

Casi gratis en GPU (ops elementwise + pocas reducciones). En CPU es notable pero negligible en práctica.

Referencias

- Géron, cap. 11 — Batch Normalization.
- Ioffe & Szegedy (2015), Batch Normalization, ICML.
- Ba, Kiros & Hinton (2016), Layer Normalization.
- Wu & He (2018), Group Normalization, ECCV.
- Santurkar et al. (2018), How Does Batch Normalization Help Optimization?, NeurIPS — explicación moderna.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 111 — Clase 111 — Gradient clipping

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Gradient Clipping + Pascanu, Mikolov & Bengio (2013). Duración estimada: 45 min.

Objetivo

Aplicar gradient clipping —limitar la norma o el valor de los gradientes antes de actualizar pesos— como protección contra exploding gradients, especialmente crítico en RNN/LSTM (clase 120) y en entrenamiento de LLMs. Diferenciar clipnorm (preserva dirección) de clipvalue (clipea por elemento).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Configurar clipping en cualquier optimizer Keras: Adam(clipnorm=1.0) o Adam(clipvalue=0.5).
- Saber cuándo clipnorm es preferible (default moderno): preserva dirección del gradiente.
- Implementar clipping manual en custom training loop con `tf.clip_by_global_norm`.
- Detectar exploding monitoreando la norma del gradiente.
- Reconocer que en Transformers de LLM, clipnorm=1.0 es estándar.

Temas

- Exploding revisitado: ¿qué pasa cuando $\|grad\|$ crece exponencialmente?
- clipnorm: si $\|g\| > c$, escalar $g \leftarrow g \cdot c/\|g\|$. Preserva dirección.
- clipvalue: $g_i \leftarrow \text{clip}(g_i, -c, +c)$ por elemento. Cambia dirección.
- Global norm vs per-variable: `clip_by_global_norm` mira el norm del tensor concatenado de todos los pesos.

Definiciones y características

- Gradient clipping: limitar el tamaño del gradiente antes de aplicarlo.
- clipnorm: norma euclídea L2 máxima permitida. Si excede, se reescala manteniendo dirección.
- clipvalue: máximo valor absoluto por elemento.
- Global norm: $\|g\|$ calculado sobre todos los gradientes concatenados como un solo vector.
- `tf.clip_by_global_norm(grads, clip_norm=1.0)`: API moderna para clipping en custom loops.

Dataset / recursos

- Fashion-MNIST + un MLP propenso a exploding (LR alto + sin BN).
- Librerías: tensorflow, keras.

Ejercicios

1. Forzar exploding: entrenar MLP con Adam(lr=10.0) sobre Fashion-MNIST. loss = nan rápido.
2. Clipping al rescate: repetir con Adam(lr=10.0, clipnorm=1.0). Verificar que no explota (aunque sigue malo el LR — clipping no es solución a LR mal calibrado, solo a explosión).
3. clipnorm vs clipvalue: comparar las dos con LR razonable. Para problemas estables son ~equivalentes; diferencias aparecen en patrones específicos.
4. Custom loop: implementar el paso con `gradients = tape.gradient(loss, model.trainable_variables)`;


```
gradients, _ = tf.clip_by_global_norm(gradients, 1.0); optimizer.apply_gradients(...).
```

5. Monitoreo: graficar $\|grad\|$ por step. Verificar que no excede el clipnorm configurado.

Homework verificable

Reentrenar el modelo del ejercicio anterior con LR razonable + clipnorm=1.0 como práctica estándar:

1. MLP [300, 100] Fashion-MNIST.
2. Adam(learning_rate=1e-3, clipnorm=1.0).
3. Graficar la norma del gradiente en cada step (custom loop o callback).
4. Verificar que la curva está acotada a 1.0 cuando el modelo aún no convergió.

Criterio de aceptación: el modelo entrena estable ($val_accuracy \geq 0.87$) y la norma del gradiente está acotada como esperado.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
clipnorm=0.01 y modelo no aprende	Clipping muy agresivo enmascara gradientes
Configuro clipvalue y los gradientes peque	Si el valor es chico y los gradientes son
Custom loop sin clipping → loss=nan ocasio	Sin clipping RNN explota. Fix: tf.clip_by_
Clipping pasa pero el problema es otro (LR	Clipping no arregla LR mal calibrado. Diag
clipnorm y clipvalue simultáneamente	Keras los aplica en cascada — comportamien

Preguntas frecuentes

¿clipnorm=1 o clipnorm=5?

1.0 para Transformers/LLMs (estándar). 5.0 para RNN/LSTM clásicos. Para MLPs/CNNs con BN, en general no hace falta clipping; un default de 1.0 no hace daño.

¿Clipping deteriora el modelo final?

Si el clipping nunca se activa (rara vez sobrepasás), no hace nada. Si se activa todo el tiempo, es una banda-aid sobre un problema más serio. Ideal: clipnorm por si las moscas pero no debería gatillarse seguido.

¿Cuándo monitorear la norma del gradiente?

Siempre que entrenes algo nuevo / no probado. Es el indicador más rápido de exploding/vanishing.

¿GradientTape global vs por variable?

Global norm (clip_by_global_norm) es lo correcto: trata el conjunto de pesos como un vector único. Per-variable distorsiona la dirección.

¿Por qué en LLMs es tan importante?

Transformers profundos + sequences largas → gradientes pueden ser muy heterogéneos. clipnorm=1.0 y Adam(beta1=0.9, beta2=0.95) son la receta default de modelos como GPT-3 entrenados desde cero.

Referencias

- Géron, cap. 11 — Gradient Clipping.
- Pascanu, Mikolov & Bengio (2013), On the difficulty of training recurrent neural networks, ICML.
- tf.clip_by_global_norm.
- Keras Optimizer base — clipnorm/clipvalue/global_clipnorm.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 112 — Clase 112 — Transfer learning, unsupervised pretraining

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Reusing Pretrained Layers. Duración estimada: 70 min.

Objetivo

Aplicar transfer learning — el patrón dominante en producción cuando hay pocos datos: tomar un modelo preentrenado (ImageNet para visión, BERT/GPT para texto), reemplazar la cabeza, congelar las capas base, fine-tunear la cabeza, y opcionalmente descongelar gradualmente para una segunda fase con LR muy bajo. Conocer unsupervised pretraining como hermano histórico (autoencoders, contrastive learning).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Cargar un modelo preentrenado: `keras.applications.MobileNetV3Small(weights='imagenet', include_top=False)`.
- Congelar capas: `base.trainable = False`.
- Construir un modelo nuevo con la base + un head custom (`GlobalAveragePooling2D + Dense`).
- Fine-tunear en dos etapas: (a) solo head con LR normal, (b) toda la red con LR 10× más bajo.
- Reconocer cuándo NO usar transfer (dataset muy distinto al de origen).

Temas

- Por qué funciona: las capas tempranas aprenden features generales (bordes, texturas); las tardías son task-specific.
- Pipeline estándar: `load` → `freeze` → `new head` → `fit` → `unfreeze` → `fit con LR bajo`.
- LR diferencial: capas tempranas más bajo ($1e-5$), capas tardías más alto ($1e-3$).
- Unsupervised pretraining: autoencoders (clase 130), contrastive (SimCLR, MoCo, CLIP).
- Self-supervised: cómo BERT/GPT se pre-entrenan sin labels (masked LM / autoregressive).

Definiciones y características

- Transfer learning: reusar pesos preentrenados en una tarea como punto de partida para otra.
- Feature extraction: usar el modelo base como extractor fijo (`trainable=False`) y entrenar solo la cabeza.
- Fine-tuning: descongelar (parte o todo) el modelo base y continuar entrenando con LR bajo.
- Catastrophic forgetting: si descongelás y usás LR alto, el modelo "olvida" lo aprendido. Por eso LR bajo en fine-tuning.
- Frozen / Trainable: `layer.trainable = False` excluye los pesos del backprop.
- Self-supervised: pre-entrenar con un objetivo derivado de los propios datos (predecir palabra masked, contrastar aumentaciones).

Dataset / recursos

- Visión: dataset chico de 2-5 clases (perros/gatos, flores). `tf.keras.utils.image_dataset_from_directory`.
- Modelo base: `MobileNetV3Small` o `EfficientNetB0` (más liviano que `ResNet50`).
- Librerías: `tensorflow`, `keras`, `keras.applications`.

Ejercicios

1. Carga: `base = MobileNetV3Small(weights='imagenet', include_top=False, input_shape=(224,224,3)); base.trainable = False.`
2. Modelo completo: `model = Sequential([base, GlobalAveragePooling2D(), Dropout(0.2), Dense(num_classes, activation='softmax')])`.
3. Etapa 1: compilar con Adam(1e-3) y entrenar 10 épocas. Reportar accuracy.
4. Etapa 2: `base.trainable = True` y recompilar con Adam(1e-5). Entrenar 10 épocas más. Verificar mejora.
5. Sin transfer: entrenar la misma arquitectura desde cero (`weights=None`). Comparar cuántas épocas necesita para igualar accuracy de transfer (típicamente: nunca lo iguala con dataset chico).

Homework verificable

Clasificación de un dataset propio de imágenes (≥ 3 clases, ≥ 100 imágenes por clase):

1. Pipeline con `image_dataset_from_directory` + augmentation (RandomFlip, RandomRotation).
2. Transfer learning con EfficientNetB0.
3. Reportar accuracy de las dos etapas (frozen y unfreeze).
4. Comparar contra entrenar desde cero.

Criterio de aceptación: la etapa 2 (unfreeze + LR bajo) debe mejorar la etapa 1 por ≥ 2 pp; el modelo desde cero queda al menos 10 pp por debajo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Olvido <code>base.trainable = False</code> y la etapa 1	Está entrenando millones de params. Fix: c
Recompilar entre etapas pero trainable cam	El cambio de trainable requiere recompilar
Imágenes en [0, 255] cuando el modelo espe	Pre-procesar con <code>keras.applications.<model</code>
BN en <code>training=True</code> se actualiza en fine-t	Sin cuidado, las moving averages se siguen
Catastrophic forgetting	LR demasiado alto al descongelar. Fix: 10-

Preguntas frecuentes

¿Cuántas imágenes mínimas para transfer learning?

Para fine-tuning: 100-500 por clase. Para feature extraction (solo head): incluso 20-50 por clase puede funcionar.

¿Qué modelo base elijo?

Empezá con MobileNetV3Small o EfficientNetB0 (chicos, rápidos). Si necesitás más capacidad y tenés GPU, EfficientNetB3-B5 o ConvNeXt.

¿Cuántas capas descongelo?

Empezá con todo descongelado + LR bajo. Si overfittea, congelar las primeras N capas (las más genéricas).

¿Transfer funciona para datos médicos / satelitales?

Sí, pero menos: las features de ImageNet son menos relevantes. Aún así, suele superar entrenar desde cero. Para datasets muy específicos, considerar pretraining con MAE o SimCLR sobre los datos propios.

¿Y para texto?

Misma idea: cargar bert-base o distilbert desde Hugging Face (clase 127), agregar head, fine-tunar. Estándar industrial.

Referencias

- Géron, cap. 11 — Reusing Pretrained Layers.
- Keras Applications — catálogo de modelos preentrenados.
- Pan & Yang (2010), A Survey on Transfer Learning, IEEE TKDE.
- Chen et al. (2020), A Simple Framework for Contrastive Learning of Visual Representations (SimCLR), ICML.
- He et al. (2022), Masked Autoencoders Are Scalable Vision Learners (MAE), CVPR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 113 — Clase 113 — Optimizadores: Momentum, Nesterov, AdaGrad, RMSProp, Adam, AdamW (+ Lion, Sophia)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Faster Optimizers + papers Lion (Chen et al. 2023), Sophia (Liu et al. 2023). Duración estimada: 80 min.

Objetivo

Conocer la evolución de los optimizadores —SGD → Momentum → Nesterov → AdaGrad → RMSProp → Adam → AdamW— entendiendo qué problema resuelve cada uno. Aplicar los optimizadores 2023+ (Lion, Sophia) que están reemplazando a Adam en LLMs grandes por mejor performance y memoria. Saber elegir según contexto (Adam para casi todo, SGD+momentum para visión clásica, Lion para LLMs).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la fórmula de cada uno: SGD ($w \leftarrow w - \eta \cdot g$), Momentum (acumulación), Nesterov (lookahead), Adam (mom 1er + 2do orden).
- Diferenciar Adam vs AdamW — la corrección de weight decay que Loshchilov & Hutter (2019) demostraron esencial.
- Usar Lion (tf.keras.optimizers.Lion en Keras 3+) con LR 3-10× más bajo que Adam.
- Reconocer cuándo SGD+Momentum supera a Adam: visión clásica con datasets grandes (ImageNet), donde el modelo final generaliza mejor.
- Inspeccionar y entender los hiperparámetros beta_1, beta_2, epsilon, weight_decay.

Temas

- SGD vanilla y por qué es lento en cañones (zigzaguea).
- Momentum (Polyak 1964): acelera en direcciones consistentes.
- Nesterov (1983): "miro hacia adelante" antes de calcular gradiente.
- AdaGrad: tasa adaptativa por parámetro; bueno para sparse data, malo para LR que decae a 0.
- RMSProp (Hinton, sin publicar): suaviza AdaGrad con EMA.
- Adam (Kingma & Ba 2014): Momentum + RMSProp = caballito industrial.
- AdamW (Loshchilov & Hutter 2019): weight decay separado del gradiente.
- Complemento moderno: Lion (Chen et al. 2023, signo en lugar de gradient adaptive), Sophia (Liu et al.

2023, Hessian aproximada).

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 102b — Optimizadores modernos: Lion, Sophia, Schedule-Free

Definiciones y características

- SGD: $w \leftarrow w - \eta \cdot g$. La base.
- Momentum: $v \leftarrow \beta \cdot v + g$; $w \leftarrow w - \eta \cdot v$. β típicamente 0.9.
- Nesterov: como momentum, pero calcula el gradiente en el punto adelantado.
- AdaGrad: $s += g^2$; $w \leftarrow w - \eta \cdot g / \sqrt{s}$. LR efectivo decrece para parámetros frecuentes.
- RMSProp: como AdaGrad pero con EMA: $s \leftarrow \beta \cdot s + (1-\beta) \cdot g^2$.
- Adam: combina momentum (1er momento) + RMSProp (2do momento) + bias correction.
- AdamW: aplica weight decay como una operación separada del gradiente (decoupled), no como L2.
- Lion: gradient sign + momentum. Menos memoria.
- beta_1: decay del primer momento (default 0.9).
- beta_2: decay del segundo momento (default 0.999 Adam / 0.99 Lion).
- epsilon: estabilidad numérica del $\sqrt{\cdot}$. Default 1e-7. En LLMs a veces 1e-8 / 1e-5.
- weight_decay: en AdamW, el coeficiente de "tirar pesos hacia 0" — regularización L2 implícita.

Dataset / recursos

- Fashion-MNIST + un modelo razonable.
- Librerías: tensorflow, keras (Lion incluido en Keras 3+).

Ejercicios

1. Comparar 5 optimizadores: SGD(0.01), SGD+Momentum(0.9), Adam(1e-3), AdamW(1e-3, wd=1e-2), Lion(1e-4, wd=0.1). Mismo modelo, mismo dataset, 20 épocas. Graficar val_loss.
2. Tuning del LR: para Adam y Lion, hacer un sweep de LR [1e-5, 1e-2] log. Encontrar el LR óptimo de cada uno. Verificar que el de Lion es ~5× más chico.
3. AdamW vs Adam con L2: comparar Adam + keras.regularizers.L2(1e-2) en cada capa vs AdamW con weight_decay=1e-2. AdamW gana en val_loss.
4. Inspección de buffer: imprimir optimizer.variables. Adam tiene m y v por parámetro; Lion solo m. Verificar memoria total.
5. LR alto + Momentum: SGD con LR=0.1 explota; SGD+Momentum(0.9) con LR=0.1 puede funcionar. Probar.

Homework verificable

Sobre Fashion-MNIST + un MLP [300, 100, 10]:

1. Encontrar el LR óptimo para 3 optimizadores: SGD, AdamW, Lion (sweep de 5 valores cada uno).
2. Con el LR óptimo, entrenar 30 épocas y reportar val_accuracy.
3. Comparar wall time y memoria.

Criterio de aceptación: AdamW debe igualar o superar a Adam por ≥ 0.3 pp en val_accuracy; Lion con buen LR debe ser competitivo y consumir menos memoria del optimizer.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cambiar de Adam a Lion sin cambiar LR	Lion necesita LR mucho más chico. Fix: div
Adam con weight_decay en Keras viejo	Lo aplica como L2 (mal). Fix: usar AdamW.
SGD plano sin schedule en redes profundas	Convergencia lentísima. Fix: SGD + Momentu
epsilon=1e-7 produce inestabilidad en bf16	Para mixed precision en LLMs, epsilon=1e-5
Asumir que Adam es siempre mejor	En visión clásica con suficiente data, SGD

Preguntas frecuentes

¿Adam o AdamW por default?

AdamW siempre que uses weight decay. Adam con weight_decay > 0 está mal implementado en muchos frameworks antiguos.

¿Lion en producción ya?

Sí, desde 2023. Google lo usa internamente. Estable y bien probado.

¿Cuándo SGD gana a Adam?

Cuando podés permitirte LR + momentum + cosine schedule bien tuneados, sobre datasets grandes (ImageNet). El modelo final generaliza ~0.5-1 pp mejor. Pero requiere más tuning.

¿epsilon cuándo lo toco?

Casi nunca con fp32. En mixed precision (bf16/fp16), subir a 1e-4 para estabilidad.

¿beta_2 por qué 0.999 en Adam y 0.99 en Lion?

Adam el 2do momento debe ser estable (decay muy lento). Lion no tiene 2do momento — beta_2 define cómo se mezcla m_{t-1} con g_t en el lookahead, similar a Nesterov.

Referencias

- Géron, cap. 11 — Faster Optimizers.
- Kingma & Ba (2014), Adam, ICLR.
- Loshchilov & Hutter (2019), Decoupled Weight Decay Regularization (AdamW), ICLR.
- Chen et al. (2023), Symbolic Discovery of Optimization Algorithms (Lion), NeurIPS.
- Liu et al. (2023), Sophia: A Scalable Stochastic Second-order Optimizer.
- Keras optimizers.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 114 — Clase 114 — Optimizadores modernos: Lion, Sophia, Schedule-Free

Parte: 2 — Deep Learning · Fuente: Chen et al. (2023) Lion + Liu et al. (2023) Sophia + Defazio et al. (2024) Schedule-Free. Duración estimada: 75 min.

Objetivo

Conocer la nueva generación de optimizadores 2023-2024 que está reemplazando a AdamW en LLM training a escala: Lion (Google, signo del gradiente), Sophia (Stanford, segundo orden aproximado), Schedule-Free

(Meta, sin LR scheduling). Saber cuándo justifican el cambio.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar Lion con LR 3-10× más chico que AdamW y weight_decay 3-10× más grande.
- Aplicar Sophia con estimación diagonal del Hessiano (Hutchinson sampling).
- Aplicar Schedule-Free (schedulefree.AdamWScheduleFree) sin warmup/cosine.
- Comparar memoria, velocidad y calidad final.
- Reconocer cuándo Lion supera AdamW (modelos grandes, ViT, CLIP) y cuándo no.

Temas

- Lion: $\text{update} = \text{sign}(\beta \cdot m + (1-\beta) \cdot g)$. 1 buffer en lugar de 2.
- Sophia: pre-condicionador diagonal del Hessiano vía Hutchinson.
- Schedule-Free: aprende sin schedule explícito, sin warmup.
- Memory: Lion ahorra 50 % vs AdamW.
- Trade-off: Lion + LR alto explota fácilmente.

Definiciones y características

- Lion: signo del gradiente como dirección. $\beta=0.9$, $\beta=0.99$ típicos.
- Sophia: actualiza $\theta \leftarrow \theta - \eta \cdot \text{clip}(m/h, -p, p)$. Robusto a hessianas mal condicionadas.
- Schedule-Free: averaging interpolation entre z (live) y x (averaged); no necesita LR schedule.
- Memory cost: Adam 2× params, AdamW 2× params, Lion 1× params, Schedule-Free 2× params.

Dataset / recursos

- Fashion-MNIST o CIFAR-10 + ViT-Tiny.
- Librerías: torch, torch.optim, schedulefree (pip), implementaciones Lion/Sophia community.

Ejercicios

1. AdamW baseline: ViT-Tiny en CIFAR-10. LR=1e-3, wd=0.05.
2. Lion: misma red, LR=1e-4, wd=0.5. Comparar accuracy y memoria.
3. Sophia: con Hutchinson cada 10 steps. Comparar convergencia.
4. Schedule-Free: AdamWScheduleFree(lr=1e-3, warmup_steps=500). Sin cosine.
5. Memory: para modelo grande, medir VRAM con cada uno.

Homework verificable

Comparar 4 optimizadores en ViT-Tiny + CIFAR-100:

1. AdamW (baseline).
2. Lion.
3. Sophia.
4. Schedule-Free AdamW.

Reportar: accuracy final, wall-time, peak VRAM.

Criterio de aceptación: Lion debe ahorrar ≥ 30 % VRAM vs AdamW; al menos uno de los modernos iguala o supera AdamW en accuracy.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
-------------------	-----------------------

Lion con LR de AdamW → loss=nan	LR debe ser 3-10× menor. Fix: 1e-4 a 3e-4
Sophia muy lento	Hutchinson costoso. Fix: estimar hessiano
Schedule-Free + cosine schedule	Conflicto. Fix: NO usar scheduler externo.
Comparar wall-time sin igual cantidad de e	Trampa. Fix: mismo epochs o early-stopping
Lion con weight_decay bajo	Resultados peores. Fix: subir wd 3-10× res

Preguntas frecuentes

Lion o AdamW en 2026?

Para modelos < 100M params: AdamW sigue siendo default seguro. Para LLM training a escala (>1B), Lion ahorra mucha memoria y gana papers (Chen 2023).

Sophia en producción?

Aún experimental. Google reportó 2× speedup en LLM pretraining (770M). Vale probar.

Schedule-Free realmente funciona sin warmup?

Recomienda warmup corto (500-1000 steps). El resto sin cosine.

Implementaciones?

Lion: implementaciones community (lucidrains/lion-pytorch). Sophia: similares. Schedule-Free: schedulefree package oficial Meta.

Lion en CV / fine-tuning?

Sí — paper original lo demostró en ViT, CLIP, modelos de difusión. Especialmente bueno para fine-tuning grande.

Referencias

- Chen et al. (2023), Symbolic Discovery of Optimization Algorithms (Lion), NeurIPS.
- Liu et al. (2023), Sophia: A Scalable Stochastic Second-order Optimizer.
- Defazio et al. (2024), The Road Less Scheduled (Schedule-Free).
- schedulefree.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrido desde el laboratorio del programa o desde Jupyter.

Clase 115 — Clase 115 — Learning rate scheduling

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Learning Rate Scheduling. Duración estimada: 60 min.

Objetivo

Saber variar el LR durante el entrenamiento —no dejarlo fijo— porque ningún LR es óptimo en todas las fases. Aplicar las 4 estrategias estándar: step decay, exponential decay, cosine annealing (default moderno), y warmup + decay (estándar en Transformers).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Configurar `keras.optimizers.schedules.CosineDecay` y pasarlo como `learning_rate=` al optimizer.
- Diferenciar `ExponentialDecay`, `PiecewiseConstantDecay` y `CosineDecayRestarts`.
- Implementar warmup lineal + cosine — receta default en BERT/GPT.
- Usar `ReduceLROnPlateau` (reactivo) vs `schedule` (proactivo).
- Graficar la curva de LR a lo largo del entrenamiento para verificar.

Temas

- LR fijo: arrancás bien, terminás demasiado alto para refinar.
- Step decay: cortar LR cada N épocas. Simple, anticuado.
- Exponential decay: $lr = lr_0 \cdot \gamma^t$.
- Cosine annealing (Loshchilov & Hutter 2017): $lr = 0.5 \cdot lr_0 \cdot (1 + \cos(\pi t/T))$.
- Warmup: empezar bajo y subir linealmente las primeras X steps. Esencial en Transformers.
- One-cycle policy (Smith 2018): warmup + cosine descent + tail decay.

Definiciones y características

- Warmup: subir LR linealmente desde 0 (o muy bajo) hasta el `lr_max` en los primeros `warmup_steps`. Estabiliza el inicio cuando los gradientes son ruidosos.
- Cosine annealing: bajar LR siguiendo una media curva coseno desde `lr_max` hasta `lr_min`. Suave, sin saltos.
- Restarts (SGDR): cada vez que llega al mínimo, reinicia con `lr_max` — exploración periódica.
- `ReduceLROnPlateau`: callback reactivo; baja LR cuando una métrica deja de mejorar.
- `OneCycle`: variante moderna que sube hasta `lr_max` en la primera mitad y baja en la segunda, terminando 100× por debajo del inicial.

Dataset / recursos

- Fashion-MNIST con un MLP medio.
- Librerías: `tensorflow`, `keras`, `matplotlib`.

Ejercicios

1. Schedule básica: `lr = CosineDecay(initial_learning_rate=1e-3, decay_steps=10_000); Adam(learning_rate=lr)`. Entrenar y graficar `val_loss`.
2. Visualizar el LR: para una schedule, evaluarla en steps 0, 100, 1000, 5000, 10000 y graficar.
3. Warmup + Cosine: implementar custom callback (o usar `CosineDecay(warmup_steps=...)` en Keras 3) y entrenar un Transformer chico (anticipo). Comparar contra sin warmup.
4. `ReduceLROnPlateau`: alternativa reactiva — `ReduceLROnPlateau(factor=0.5, patience=3)`. Comparar con cosine.
5. One-cycle: implementar con `LearningRateScheduler` callback. Probar y comparar.

Homework verificable

Sobre Fashion-MNIST:

1. Entrenar 3 modelos con la misma arquitectura: (a) LR fijo 1e-3, (b) `ExponentialDecay`, (c) `CosineDecay(warmup_steps=100)`.
2. Reportar `val_accuracy` y graficar las curvas de loss + LR.
3. Concluir qué schedule produjo mejor accuracy.

Criterio de aceptación: cosine con warmup debe ganar o empatar contra fijo. La curva de `val_loss` del schedule debe ser visualmente más suave hacia el final.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Schedule decae demasiado rápido	decay_steps está mal calibrado vs total st
Combinar schedule + ReduceLROnPlateau	Conflicto: ambos modifican LR. Fix: elegí
LearningRateScheduler callback no funciona	Conflicto. Fix: usar uno u otro.
Sin warmup, transformer diverge en las pri	Gradientes iniciales son ruidosos. Fix: wa
Reiniciar entrenamiento desde checkpoint p	optimizer.iterations cuenta steps; al reca

Preguntas frecuentes

¿Cuál schedule por default?

Cosine con warmup. Es el estándar 2022+ en visión y NLP.

¿Cuántos warmup steps?

5-10 % del total. Para 100 épocas de 500 steps cada una = 50 000 steps → warmup 2 500-5 000.

¿Schedule en steps o en épocas?

Steps casi siempre. Cuanto más granular, más suave.

¿CosineDecayRestarts cuándo?

Cuando entrenás muy largo y querés exploraciones periódicas. Útil en redes muy profundas y datasets grandes; raro en proyectos chicos.

¿Schedule depende del optimizer?

Casi no — Adam(lr=schedule) funciona igual que SGD(lr=schedule). La diferencia es en los valores absolutos.

Referencias

- Géron, cap. 11 — Learning Rate Scheduling.
- Loshchilov & Hutter (2017), SGDR: Stochastic Gradient Descent with Warm Restarts, ICLR.
- Smith (2018), Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates.
- Keras LR schedules.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 116 — Clase 116 — Regularización: L1/L2, dropout, max-norm, MC dropout (+ Stochastic Depth, DropPath)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 11 § Regularization + Huang et al. (2016) Deep Networks with Stochastic Depth. Duración estimada: 80 min.

Objetivo

Conocer las técnicas de regularización en DL —L1/L2, dropout (Srivastava et al. 2014), max-norm, MC dropout para incertidumbre— y las técnicas modernas que se usan en arquitecturas profundas (ResNets, ViT,

Transformers): Stochastic Depth, DropPath y LayerDrop.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar `keras.regularizers.L1(...)`, `L2(...)`, `L1_L2(...)` en una capa.
- Aplicar `Dropout(rate=0.5)` y entender qué hace en train vs en inference (default desactivado).
- Implementar Monte Carlo dropout (`Dropout(0.5)` activo en inference → predicciones diferentes → incertidumbre).
- Aplicar Stochastic Depth en una ResNet: dropear bloques residuales completos al azar durante training.
- Aplicar DropPath (estándar en ViT, Swin Transformer, ConvNeXt).

Temas

- L1/L2 como penalización en la loss. λ típicamente $1e-4$ a $1e-2$.
- Dropout: enmascarar fracción r de las activaciones por batch.
- Inverted dropout: en inference no se hace nada porque train ya escala por $1/(1-r)$.
- Max-norm constraint: $\|w\| \leq c$ por neurona después de cada update.
- MC Dropout (Gal & Ghahramani 2016): incertidumbre bayesiana aproximada.
- Complemento moderno: Stochastic Depth, DropPath (= Stochastic Depth aplicado a paths de attention/FFN), LayerDrop (Fan et al. 2020).

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 104b — Regularización moderna: Stochastic Depth, DropPath, LayerDrop

Definiciones y características

- L1 regularization: agrega $\lambda \cdot \sum |w|$ a la loss. Promueve sparsity.
- L2 regularization (weight decay): agrega $\lambda \cdot \sum w^2$. Mantiene pesos chicos.
- Dropout: enmascara fracción r de neuronas por batch. Forzar redundancia.
- MC Dropout: hacer N predicciones con dropout activo → distribución de predicciones → incertidumbre.
- Max-norm: constraint sobre la norma de los pesos por unidad.
- Stochastic Depth: dropear bloques residuales enteros durante training.
- DropPath: como Stochastic Depth pero para paths en transformer (attention o FFN).

Dataset / recursos

- Fashion-MNIST + un MLP propenso a overfit.
- Librerías: tensorflow, keras, matplotlib.

Ejercicios

1. Sin regularización: entrenar un MLP grande ([512, 256, 128]) en Fashion-MNIST y observar overfitting (gap train/val ≥ 5 pp).
2. L2: agregar `kernel_regularizer=keras.regularizers.L2(1e-3)` a cada Dense. Comparar.
3. Dropout: agregar `Dropout(0.3)` entre Dense layers. Comparar.
4. MC Dropout: para 1 sample de test, hacer 100 predicciones con `model(x, training=True)`. Calcular mean $\pm$ std de las probabilidades. Interpretar la incertidumbre.
5. Stochastic Depth simulado: en un mini ResNet con 8 bloques, dropear cada bloque con prob 0.1 lineal. Comparar contra sin stochastic depth.

Homework verificable

Sobre Fashion-MNIST con MLP [512, 256, 128, 64]:

1. Entrenar 4 versiones: sin regularización; L2(1e-3); Dropout(0.3); L2 + Dropout combinados.
2. Reportar train_acc y val_acc; calcular el gap.
3. Para el mejor modelo, hacer MC Dropout con 50 muestras sobre 5 imágenes ambiguas y reportar incertidumbre.

Criterio de aceptación: el modelo regularizado tiene gap train-val menor a 3 pp (vs ~6 pp del baseline) y val_acc igual o mejor. MC dropout debe asignar mayor std a las imágenes ambiguas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Dropout en inferencia da resultados distín	Pasaste training=True por error. Fix: en i
L2 con $\lambda=1.0$ y modelo no aprende	Penalización demasiado fuerte. Fix: λ típí
Dropout(0.5) en la última capa antes de so	Distorsiona logits. Fix: dropout en capas
L2 + AdamW con weight_decay → doble penali	Usar uno: AdamW(wd=...) o kernel_regulariz
Stochastic Depth con p_i constante en luga	Funciona pero menos óptimo. Fix: p_i = i/N

Preguntas frecuentes

¿Dropout 0.5 siempre?

0.5 para capas Dense grandes. Para capas Conv: 0.1-0.2. Para embeddings y attention en Transformers: 0.1.

¿BN ya regulariza, necesito dropout también?

Depende. En CNNs/MLPs con BN, dropout a veces ya no aporta. En Transformers, sí (BN no se usa allí; LN + dropout + DropPath).

¿MC Dropout es bayesiano "de verdad"?

Aproxima un proceso gaussiano variacional. No es bayesiano riguroso pero es una excelente aproximación práctica para incertidumbre.

¿Stochastic Depth en CNN no residual?

No tiene sentido — Stochastic Depth necesita la skip connection para que dropear no rompa el forward.

¿Cuánta dropout/droppath en ViT base?

ViT-Base original: dropout=0.1 en attention, droppath=0.1 lineal en cada bloque. Para fine-tuning, suele bajarse a 0.0.

Referencias

- Géron, cap. 11 — Regularization Using Dropout.
- Srivastava et al. (2014), Dropout, JMLR.
- Gal & Ghahramani (2016), Dropout as a Bayesian Approximation, ICML — MC dropout.
- Huang et al. (2016), Deep Networks with Stochastic Depth, ECCV.
- Fan et al. (2020), Reducing Transformer Depth on Demand with Structured Dropout (LayerDrop).
- keras DropPath / StochasticDepth.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 117 — Clase 117 — Regularización moderna: Stochastic Depth, DropPath, LayerDrop

Parte: 2 — Deep Learning · Fuente: Huang et al. (2016) Stochastic Depth + Fan et al. (2020) LayerDrop + DropPath en ViT/Swin/ConvNeXt. Duración estimada: 70 min.

Objetivo

Aplicar regularización por paths/bloques —más allá del dropout clásico— en arquitecturas profundas modernas (ResNet, ViT, ConvNeXt, Swin Transformer). Cubrir Stochastic Depth (drop bloque residual), DropPath (drop path en transformer), LayerDrop (drop layer completa). Beneficio doble: regularización + reducción de cómputo durante training.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar `keras.layers.StochasticDepth(rate)` o `DropPath(rate)` en bloques residuales.
- Diseñar rate lineal por profundidad: capa 0 $\rightarrow$ 0.0, capa N $\rightarrow$ 0.2.
- Aplicar LayerDrop durante pretraining para permitir inference con menos capas.
- Diferenciar Dropout (neurona) vs Stochastic Depth (bloque) vs LayerDrop (layer).
- Reconocer el speedup de training: 25 % más rápido en ResNet-110 (Huang 2016).

Temas

- Dropout clásico vs Stochastic Depth.
- DropPath = Stochastic Depth para Transformers.
- Rate lineal: $p_i = i/N \cdot p_{\text{max}}$.
- LayerDrop para compresión de Transformers.
- Combinaciones: Dropout + DropPath + Label Smoothing.

Definiciones y características

- Stochastic Depth: $y = x + \text{drop}(b(x))$ con prob p_i . En inference, scale por $1 - p_i$.
- DropPath: igual idea aplicada a attention y FFN paths.
- LayerDrop: dropear layer entera del Transformer.
- Linear rate schedule: capas tempranas seguras ($p \approx 0$), tardías regularizadas ($p \approx 0.2$).

Dataset / recursos

- CIFAR-10/100 con ResNet propia.
- ViT-Tiny preentrenado.
- Librerías: torch, timm (donde DropPath es default).

Ejercicios

1. ResNet con Stochastic Depth: implementar BasicBlock con `StochasticDepth(p)`. Train CIFAR-10.
2. Rate lineal: aplicar $p_i = i/N \cdot 0.2$ en cada bloque. Comparar contra rate constante.
3. ViT con DropPath: `timm.create_model('vit_tiny_patch16_224', drop_path_rate=0.1)`. Comparar contra 0.0.
4. LayerDrop: simular con 12-layer BERT mini — drop 50 % layers. Verificar accuracy aún razonable.
5. Speed: medir wall-clock training con vs sin Stochastic Depth.

Homework verificable

ResNet-50 en CIFAR-100:

1. Entrenar con y sin Stochastic Depth (rate lineal a 0.2 final).
2. Reportar accuracy + tiempo total.
3. Verificar regularización: gap train-val.

Criterio de aceptación: Stochastic Depth reduce gap train-val ≥ 1 pp; speedup en wall-time visible (~10-20 %).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Rate constante 0.5 en todas las capas	Mata bloques iniciales (críticos). Fix: li
Olvido scale en inference	Sesgo. Fix: librerías lo manejan; verifica
DropPath sin residual	Sin sentido — no hay path para droppear. F
Aplicar a 1ª y última capa	Generalmente innecesario. Fix: layers inte
Combinar Dropout + Stochastic Depth + Labe	Overregularization. Fix: ajustar individua

Preguntas frecuentes

Stochastic Depth o Dropout?

Para CNN/ViT profundas: ambos. Stochastic Depth en bloques residuales, Dropout en MLP final.

Rate final 0.1 o 0.2 o 0.5?

0.1-0.2 para ResNet/ViT base. 0.3-0.5 para modelos enormes (LayerDrop en BERT-Large).

LayerDrop sirve para inference?

Sí — train con LayerDrop=0.5, inference con k random layers. Habilita modelos compactos sin re-training.

timm vs implementación propia?

Usá timm siempre que se pueda — DropPath bien implementado, rate scheduling automático.

En LLMs modernos (Llama)?

Less común. Llama no usa DropPath. BERT/RoBERTa sí.

Referencias

- Huang et al. (2016), Deep Networks with Stochastic Depth, ECCV.
- Fan et al. (2020), Reducing Transformer Depth on Demand with Structured Dropout (LayerDrop), ICLR.
- Dosovitskiy et al. (2020), ViT — DropPath aplicado.
- timm DropPath.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 118 — Clase 118 — TensorFlow: tensores, variables, operaciones

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § Using TensorFlow like NumPy. Duración estimada: 60 min.

Objetivo

Bajar un nivel por debajo de Keras: trabajar directamente con tensores (`tf.Tensor`) y variables (`tf.Variable`), entender la API NumPy-like de TF (`tf.matmul`, `tf.reduce_*`, `tf.cast`, `tf.reshape`), y diferenciar inmutable (`Tensor`) de mutable (`Variable`, base de los pesos).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Crear tensores con `tf.constant`, `tf.zeros`, `tf.ones`, `tf.random.normal`.
- Aplicar operaciones: aritméticas, `matmul`, broadcasting, indexing/slicing, `reduce_mean/sum/max`.
- Convertir entre TF y NumPy (`.numpy()`, `tf.convert_to_tensor`).
- Crear y modificar `tf.Variable` con `.assign`, `.assign_add`.
- Reconocer dtypes (`float32`, `float64`, `int32`, `bool`) y forzar cast cuando hace falta.

Temas

- `tf.Tensor`: inmutable, similar a `np.ndarray`.
- `tf.Variable`: mutable, base de los pesos.
- Broadcasting (idéntico a NumPy).
- Operaciones reduce con `axis=`.
- `tf.function` (anticipo clase 107) — convierte una función Python en grafo.
- TF en GPU: ops sobre tensores van a GPU automáticamente si está disponible.

Definiciones y características

- `tf.constant`: crea un Tensor inmutable.
- `tf.Variable`: tensor mutable, registrable como peso de un modelo.
- Broadcasting: $(3, 1) + (1, 4) \rightarrow (3, 4)$. Reglas idénticas a NumPy.
- dtype: tipo numérico. Mismatches disparan errores; usar `tf.cast(x, tf.float32)`.
- `.assign()`: actualizar el contenido de una Variable in-place. `v.assign_add(g)` es `v += g`.
- `.shape` y `.numpy()`: forma del tensor y conversión a array NumPy.

Dataset / recursos

- Operaciones a mano (no requiere dataset).
- Librerías: tensorflow, numpy.

Ejercicios

1. Tensores básicos: crear `t = tf.constant([[1.0, 2.0], [3.0, 4.0]])`. Imprimir `shape`, `dtype`, `t.numpy()`.
2. Operaciones: `tf.matmul(t, t)`, `tf.transpose(t)`, `tf.reduce_sum(t, axis=0)`. Verificar shapes.
3. Broadcasting: `a = tf.constant([[1.], [2.], [3.]])` (shape 3,1), `b = tf.constant([10., 20., 30.])` (3,). `a + b` → ¿qué shape?
4. Variable y assign: `v = tf.Variable([1., 2., 3.])`; `v.assign([4., 5., 6.])`; `v.assign_add([1., 1., 1.])`. Verificar.
5. dtype mismatch: `tf.constant([1, 2, 3]) + tf.constant([1.0, 2.0, 3.0])` → error. Arreglar con `tf.cast`.

Homework verificable

Implementar regresión lineal manualmente con TF (sin Keras):

1. Generar `X = tf.random.normal((100, 2))`, `y_true = X @ tf.constant([[1.], [2.]]) + 3 + ruido`.
2. Variables `w = tf.Variable(tf.random.normal((2, 1)))`, `b = tf.Variable(0.0)`.
3. Loss = MSE.
4. Loop manual: calcular pred, loss, gradientes (con GradientTape — anticipo clase 108), update con

assign_sub.

- Reportar w, b finales, comparar con verdaderos.

Criterio de aceptación: tras 500 iteraciones con LR=0.01, $w \approx [1, 2]$ (± 0.1) y $b \approx 3$ (± 0.1).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
InvalidArgumentError: cannot compute Add a	dtype mismatch. Fix: <code>tf.cast(x, tf.float32)</code>
AttributeError: 'EagerTensor' object has n	Dentro de <code>@tf.function</code> los tensores son si
Asignar a <code>tf.constant</code>	Immutable. Fix: usar <code>tf.Variable</code> .
Operaciones tipo <code>arr[mask] = 0</code> en TF	TF tensors son inmutables. Fix: <code>tf.where(m</code>
<code>t.numpy()</code> lento dentro de un loop de entre	Forza sync GPU → CPU. Fix: mantener todo e

Preguntas frecuentes

¿TF tensors o NumPy arrays?

TF cuando trabajás con un modelo y querés que las operaciones corran en GPU + sean diferenciables. NumPy para análisis CPU de datos. Conversión es barata pero no gratis.

¿`tf.Variable` o `tf.constant` para datos de entrada?

Constant (los datos no cambian). Variables son para pesos que se entrenan.

¿Por qué float32 y no float64?

GPUs son MUCHO más rápidas en float32 (y aún más en bfloat16). Default ML.

¿Cómo seteo el device manualmente?

`with tf.device('/GPU:0'): ...` o `tf.config.set_visible_devices(...)`. En la mayoría de los casos no hace falta — TF lo elige bien.

¿Equivalencia con PyTorch?

`tf.constant` ↔ `torch.tensor(..., requires_grad=False)`. `tf.Variable` ↔ `torch.Parameter` o `nn.Parameter`. Operaciones casi 1:1.

Referencias

- Géron, cap. 12 — Custom Models and Training with TensorFlow.
- TF Tensor guide.
- TF Variable guide.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 119 — Clase 119 — Losses, métricas, capas, modelos custom

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § Customizing Models and Training Algorithms.
Duración estimada: 70 min.

Objetivo

Crear losses, métricas y capas custom cuando los builtins de Keras no alcanzan: focal loss, métrica F1 macro, una capa con normalización custom, modelo subclassed con train_step propio. Diferenciar stateless (función) de stateful (clase con estado acumulado por época).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir loss custom como función: `def my_loss(y_true, y_pred): return ...`
- Definir métrica stateful heredando `keras.metrics.Metric` con `update_state`, `result`, `reset_state`.
- Crear capa custom heredando `keras.layers.Layer` con `build` (declara pesos) y `call` (forward).
- Overridar `train_step` de un modelo subclassed (`Model`) para custom training logic.
- Saber cuándo usar custom vs cuándo los builtins de Keras alcanzan (casi siempre).

Temas

- Loss como función simple: 2 args (`y_true`, `y_pred`), devuelve un tensor.
- Métrica stateless (función) vs stateful (`Metric` class).
- Capa custom: `__init__` (config), `build` (pesos), `call` (forward).
- Model subclass con `train_step(self, data)` custom.

Definiciones y características

- Loss: función que devuelve un escalar (o vector por sample) que minimizamos.
- Métrica stateful: acumula a lo largo del epoch (necesario para F1, precision, recall, AUC).
- `build(input_shape)`: hook donde Keras te dice las dims; ahí declararás pesos con `self.add_weight`.
- `get_config()`: serialización — necesario si quieres `model.save()`.
- `train_step`: lo que Keras llama por cada batch durante fit. Default es: forward, compute loss, backward, optimizer step.

Dataset / recursos

- Fashion-MNIST o cualquier dataset previo.
- Librerías: tensorflow, keras.

Ejercicios

1. Loss custom: implementar Focal Loss $FL(p_t) = -\alpha(1-p_t)^\gamma \log(p_t)$ (Lin et al. 2017, útil para imbalance). Aplicar a Fashion-MNIST y comparar contra cross-entropy.
2. Métrica F1 macro: heredar `keras.metrics.Metric`, mantener confusion matrix acumulada por época, calcular F1 macro en `result()`.
3. Capa custom: `class L2Normalize(Layer)` que normaliza cada vector a norma 1. Probarla en un modelo.
4. Modelo con `train_step` custom: subclass que en cada batch aplica gradient clipping manual + logging extra.
5. `get_config`: agregar a una capa custom; verificar que `model.save()` y `load_model(custom_objects=...)` funciona.

Homework verificable

Sobre un dataset desbalanceado (simular o usar uno con clase minoritaria al 5 %):

1. Entrenar con cross-entropy estándar; reportar F1 macro.
2. Entrenar con Focal Loss custom ($\gamma=2.0$, $\alpha=0.25$); reportar F1.
3. Implementar una métrica F1 macro custom y usarla durante el training.

Criterio de aceptación: Focal Loss debe mejorar F1 macro en ≥ 2 pp respecto a cross-entropy en el caso desbalanceado.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Loss custom devuelve shape incorrecto	Debe ser (batch,) o escalar. Fix: revisar
Métrica custom devuelve siempre el mismo v	Olvidaste reset_state entre épocas. Keras
model.save() falla con capa custom	Falta get_config. Fix: implementarlo y cls
Pesos creados en __init__ en vez de build	Funciona pero pierde la flexibilidad de sh
Override train_step y se pierde el logging	Hay que actualizar self.compiled_metrics.u

Preguntas frecuentes

¿Custom loss o tf.keras.losses.Loss subclass?

Función para casos simples; subclass cuando tenés estado o configs complejas.

¿Métrica función vs subclass?

Función para batch-wise (accuracy). Subclass para cualquier cosa que necesite acumular (precision, recall, F1, AUC).

¿call o __call__ en capa custom?

Definí call; Keras envuelve para llamar a través de __call__ agregando training/masking.

¿Custom training loop o subclass con train_step?

Subclass train_step cuando podés (preserva fit, callbacks, etc.). Custom loop completo solo si necesitás control de flujo realmente complejo (clase 108).

¿Por qué add_weight y no tf.Variable?

add_weight registra automáticamente la variable como peso entrenable del modelo, gestiona dtype, initializer, regularizer y nombre.

Referencias

- Géron, cap. 12 — Custom Loss Functions, Metrics, Layers and Models.
- Lin et al. (2017), Focal Loss for Dense Object Detection, ICCV.
- Keras — Making new layers and models.
- Keras — Customize what happens in fit().

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 120 — Clase 120 — Funciones y grafos (autograph)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § TF Functions and Graphs. Duración estimada: 55 min.

Objetivo

Entender qué hace `@tf.function` —compila una función Python a un grafo TF estático, acelerando 2-10× y permitiendo deploy en TF Serving / TFLite—. Conocer AutoGraph (traduce automáticamente `if/for/while` Python a operaciones TF), saber los gotchas clásicos (efectos colaterales, prints, listas Python) y cuándo el decorator deteriora la experiencia de debugging.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar `@tf.function` a una función custom y verificar speedup.
- Identificar cuándo NO usarlo (debugging, lógica con efectos colaterales no determinísticos).
- Entender retracing: cada vez que cambias la shape o dtype del input, TF reconstruye el grafo.
- Usar `tf.function(input_signature=...)` para evitar retracing.
- Diferenciar eager mode (default, dinámico) de graph mode (compilado).

Temas

- Eager vs graph execution.
- `@tf.function` y AutoGraph.
- Retracing: por qué pasar Python ints vs tensors causa retraces.
- Print en grafo: `tf.print` (corre en graph) vs `print` (solo en tracing).
- Cuándo `@tf.function` vale la pena (loops, training step) y cuándo no (one-shot, debugging).

Definiciones y características

- Eager mode: cada op se ejecuta inmediatamente. Default en TF 2+. Como NumPy.
- Graph mode: las ops se ensamblan en un grafo y se ejecutan después. Más rápido, deployable.
- `@tf.function`: convierte una función Python en un ConcreteFunction (grafo).
- AutoGraph: subsistema que traduce `if/for/while` a `tf.cond/tf.while_loop`.
- Retracing: rebuild del grafo cuando cambian shape/dtype de inputs. Caro.
- `input_signature`: tupla de TensorSpec que fija las shapes esperadas → no retracing.

Dataset / recursos

- Operaciones aisladas para medir velocidad.
- Librerías: tensorflow, time.

Ejercicios

1. Speedup básico: definir `def f(x): return tf.reduce_sum(x * 2 + 3x + 1)`. Medir tiempo eager vs `@tf.function`-wrapped en un loop de 10 000 iteraciones.
2. Retracing: definir una función con `@tf.function`. Llamarla con tensores de shapes distintas; usar `tf.config.experimental_run_functions_eagerly(True)` y `print` para detectar retraces.
3. AutoGraph: una función con un `for` Python y un `if`. Verificar que se convierte correctamente (`tf.autograph.to_code(f)`).
4. `tf.print` vs `print`: dentro de `@tf.function`, demostrar que `print` solo se ejecuta en tracing (1ª llamada), `tf.print` siempre.
5. `input_signature`: fijar `input_signature` para evitar retracing con shape (None, 784).

Homework verificable

Implementar un training loop minimalista (sin Keras) con y sin `@tf.function`:

1. Modelo simple en numpy/TF (regresión lineal).
2. Loop manual de 1 000 iteraciones.
3. Mismo loop wrapped con `@tf.function`.

4. Comparar tiempo total.

Criterio de aceptación: la versión `@tf.function` debe ser $\geq 2\times$ más rápida (en CPU, más en GPU).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
print no aparece dentro de <code>@tf.function</code> de	Solo corre en tracing. Fix: <code>tf.print</code> para
Función retrase en cada llamada	Pasás Python ints/strings como argumentos.
<code>list.append()</code> dentro de <code>@tf.function</code> no fu	Listas Python no se preservan en grafo. Fi
if <code>x > 0</code> : con tensor <code>x</code> → error o warning	AutoGraph debería convertirlo a <code>tf.cond</code> . F
<code>np.random</code> dentro de <code>@tf.function</code> siempre d	Se evalúa una vez en tracing. Fix: usar <code>tf</code>

Preguntas frecuentes

¿Cuándo `@tf.function`?

Para funciones que se llaman muchas veces (training step, custom layers complejas). NO para funciones one-shot, debugging, o muy simples.

¿`@tf.function` y debugging?

Hacé el debugging en eager (sin decorator). Una vez que funciona, agregás `@tf.function` para producción.

¿Cómo veo el grafo generado?

`tf.autograph.to_code(f.python_function)` te da el Python equivalente que generó AutoGraph. Útil para entender qué hizo.

¿`jit_compile=True` qué hace?

`@tf.function(jit_compile=True)` activa XLA, otra capa de optimización (fusión de ops). Mejora velocidad otro 1.5-3×, especialmente en TPU.

¿En Keras hace falta?

No para `model.fit` (ya está jitteado). Sí para custom training loops (clase 108) y para custom layers/models complejos.

Referencias

- Géron, cap. 12 — TF Functions and Graphs.
- TF — Better performance with `tf.function`.
- TF — AutoGraph.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 121 — Clase 121 — Custom training loops (+ PyTorch & PyTorch Lightning)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 12 § Custom Training Loops + docs PyTorch, Lightning.
Duración estimada: 85 min.

Objetivo

Escribir un training loop manual en TF con GradientTape — control absoluto sobre cada paso (útil para GANs, RL, multi-step optimizers, debugging). Conocer el equivalente en PyTorch (el framework dominante de la industria en 2026) y cómo PyTorch Lightning abstrae los boilerplate del loop, devolviendo la productividad de Keras con la flexibilidad de PyTorch.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un training loop TF con for batch in dataset: with GradientTape() as tape: ...; grads = tape.gradient(loss, vars); optimizer.apply_gradients(...).
- Hacer el equivalente en PyTorch: optimizer.zero_grad(); loss.backward(); optimizer.step().
- Usar PyTorch Lightning para el mismo problema con boilerplate mínimo (LightningModule.training_step).
- Reconocer cuándo el loop manual es necesario (modelo con dos optimizadores, schedule custom step-wise, RL).
- Comparar productividad y flexibilidad de los 3 frameworks.

Temas

- Loop básico TF: epochs → batches → tape → grads → apply.
- tape.watch(...) para watch tensors que no son tf.Variable.
- Métricas y logging manual.
- Equivalente PyTorch.
- Lightning como capa de abstracción.
- Complemento moderno: PyTorch + Lightning en paralelo a TF/Keras.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 108a — PyTorch fundamentos: tensores, autograd, nn.Module
- Clase 108b — PyTorch Lightning: Trainer + distributed

Definiciones y características

- GradientTape: contexto TF que registra operaciones para autodiff.
- tape.watch(t): forzar a tape a registrar un tensor no-Variable.
- tape.gradient(loss, vars): calcular gradientes.
- optimizer.apply_gradients(zip(grads, vars)): aplicar la update.
- PyTorch .backward(): aplica autodiff sobre el grafo dinámico construido en el forward.
- requires_grad: flag PyTorch que indica si un tensor necesita gradientes.

Dataset / recursos

- Fashion-MNIST o MNIST.
- Librerías: tensorflow, torch, lightning (pip install lightning).

Ejercicios

1. TF loop manual: entrenar un MLP en Fashion-MNIST con GradientTape. Implementar logging de loss y métrica manual.
2. PyTorch equivalente: reimplementar el mismo loop en PyTorch.
3. Lightning: mismo problema con LightningModule.
4. Speedup con jit: tf.function en TF; torch.compile en PyTorch. Medir.

5. Multi-optimizer: con TF, escribir un loop que aplica un optimizer para las capas frozen-ish (LR bajo) y otro para las nuevas (LR alto). Esto en `model.fit` requiere mucho boilerplate.

Homework verificable

Implementar el mismo problema (Fashion-MNIST, MLP [256, 128]) en los 3 frameworks:

1. TF con custom training loop.
2. PyTorch puro.
3. PyTorch Lightning.

Reportar para cada uno: # líneas de código, tiempo de training, accuracy.

Criterio de aceptación: las 3 versiones llegan a accuracy similar (± 0.5 pp). Lightning $\approx$ Keras en LOC; PyTorch puro tiene más boilerplate; TF custom loop también.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Loss no baja en TF custom loop	Olvidaste <code>tape.gradient(loss, model.traina</code>
En PyTorch, gradientes acumulados batch a	Olvidaste <code>optimizer.zero_grad()</code> al inicio
<code>RuntimeError: cudnn error</code> en PyTorch	Versión de PyTorch incompatible con CUDA i
Lightning espera <code>LightningDataModule</code> cuand	Lightning acepta ambos; verificar firma de
BatchNorm no se actualiza en TF custom loo	Hay que pasar <code>training=True</code> explícitamente

Preguntas frecuentes

¿TF o PyTorch para empezar?

Keras (TF) es más amigable para empezar. PyTorch es más demandado en producción 2026. Saber ambos es el estándar profesional.

¿Lightning vs Keras de cuál sale mejor?

Lightning te abre el ecosistema PyTorch (Hugging Face). Keras 3 ahora soporta backends multi (TF, JAX, PyTorch) — la diferencia disminuye.

¿`zero_grad()` por qué existe en PyTorch?

Para permitir gradient accumulation (varios `.backward()` sin step para batches grandes virtuales). Si no necesitas eso, llama `zero_grad()` al inicio del batch siempre.

¿Distributed training más fácil en cuál?

Lightning, por mucho. `trainer = L.Trainer(strategy='ddp', devices=4)` y ya. En TF necesitas `tf.distribute.MirroredStrategy` con `strategy.scope()`. PyTorch puro requiere `torch.distributed.init_process_group` manual.

¿Hugging Face usa PyTorch o TF?

Casi todo PyTorch desde 2022. Los modelos modernos (Llama, Mistral, Mixtral, etc.) están solo en PyTorch.

Referencias

- Géron, cap. 12 — Custom Training Loops.
- PyTorch tutorials.
- PyTorch Lightning docs.

- Falcon et al. (2019), PyTorch Lightning.
- Hugging Face — Trainer (built on Lightning concepts).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 122 — Clase 122 — PyTorch fundamentos: tensores, autograd, nn.Module

Parte: 2 — Deep Learning · Fuente: PyTorch tutorials + Howard & Gugger, Deep Learning for Coders with fastai & PyTorch. Duración estimada: 90 min.

Objetivo

Aprender PyTorch —el framework dominante en research y en LLMs/multimodal 2026—. Cubrir: tensores (similar a NumPy, en GPU), autograd (requires_grad, .backward()), nn.Module (forma de definir modelos), Dataset/DataLoader para data pipelines. Equivalencias 1:1 con Keras/TF de las clases anteriores.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Crear tensores: torch.tensor, torch.zeros, torch.randn, device='cuda'.
- Aplicar autograd: x.requires_grad_(True); y = f(x); y.backward(); x.grad.
- Definir un MLP custom: class Net(nn.Module): def __init__(self): ...; def forward(self, x):
- Escribir el loop manual: optimizer.zero_grad(); loss.backward(); optimizer.step().
- Usar Dataset y DataLoader para pipelines de datos.

Temas

- Tensors vs ndarray, .to(device).
- Computation graph dinámico (vs estático TF1) — define-by-run.
- requires_grad y autograd.
- nn.Module, nn.Linear, nn.Sequential, nn.functional.
- Loss funcs: nn.CrossEntropyLoss, nn.MSELoss.
- Optim: torch.optim.Adam(model.parameters(), lr=...).
- Dataset + DataLoader(num_workers, pin_memory).

Definiciones y características

- torch.tensor: similar a np.ndarray pero con GPU + autograd.
- nn.Module: clase base de todos los modelos. __init__ declara capas, forward define forward.
- nn.Parameter: tensor con requires_grad=True registrado automáticamente.
- optimizer.zero_grad(): OBLIGATORIO al inicio de cada batch (sino se acumulan).
- model.train() / model.eval(): cambia comportamiento de BatchNorm y Dropout.
- with torch.no_grad(): contexto para inference, no construye grafo.

Dataset / recursos

- Fashion-MNIST vía torchvision.datasets.
- Librerías: torch, torchvision.

Ejercicios

1. Tensores: crear, mover a GPU, operaciones básicas. Comparar con NumPy.

2. Autograd: `x = torch.tensor([2.0], requires_grad=True)`; `y = x**3`; `y.backward()`; `print(x.grad)` → debe ser 12.
3. MLP custom: definir clase con 2 `nn.Linear` + `ReLU`. Verificar `model.parameters()`.
4. Training loop manual: Fashion-MNIST, 1 época, reportar loss.
5. DataLoader: `DataLoader(dataset, batch_size=32, shuffle=True, num_workers=2)`. Iterar.

Homework verificable

Reproducir el modelo de Fashion-MNIST de Clase 091 en PyTorch:

1. MLP [300, 100, 10].
2. CrossEntropy loss, Adam(1e-3).
3. EarlyStopping manual (cuando `val_loss` no baja por 5 épocas).
4. Reportar test accuracy.

Criterio de aceptación: accuracy ≥ 0.87 (igual que la versión Keras); código compacto y legible.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Olvido <code>optimizer.zero_grad()</code>	Gradientes acumulados. Fix: agregar al ini
Modelo no aprende	Olvidaste <code>.to(device)</code> en model o data. Fix
<code>RuntimeError: Trying to backward through t</code>	Reusar el grafo. Fix: <code>loss.backward(retain</code>
Eval con <code>model.train()</code> (BN/Dropout activos	Métricas distorsionadas. Fix: <code>model.eval()</code>
DataLoader lento	<code>num_workers=0</code> . Fix: <code>num_workers=4</code> , <code>pin_mem</code>

Preguntas frecuentes

¿PyTorch o TF/Keras?

PyTorch para research, LLMs, multimodal, HuggingFace ecosystem. Keras para tabular/imagen estándar. Saber ambos.

¿`torch.compile` mejora velocidad?

Sí — `model = torch.compile(model)` en PyTorch 2.0+: 2-5× speedup automático sin cambiar código.

¿Equivalente de Keras `fit`?

PyTorch puro no lo tiene. Lo provee Lightning (clase 108b) y HuggingFace Trainer.

¿`nn.Sequential` o `nn.Module` custom?

Sequential para pilas lineales. Custom Module para cualquier topología no trivial.

¿GPU mac (Apple Silicon)?

`device='mps'` desde PyTorch 1.12. Funcional, ~50-80 % de NVIDIA performance.

Referencias

- PyTorch tutorials.
- Howard & Gugger, Deep Learning for Coders with fastai & PyTorch.
- Paszke et al. (2019), PyTorch: An Imperative Style, High-Performance Deep Learning Library, NeurIPS.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.

- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrirlo desde el laboratorio del programa o desde Jupyter.

Clase 123 — Clase 123 — PyTorch Lightning: Trainer, callbacks, distributed

Parte: 2 — Deep Learning · Fuente: Lightning docs. Duración estimada: 80 min.

Objetivo

Aprender PyTorch Lightning — la capa de abstracción que convierte PyTorch puro (mucho boilerplate) en algo tan productivo como Keras pero conservando flexibilidad. Cubrir LightningModule, Trainer, callbacks, logging (W&B/TensorBoard), distributed training con un solo kwarg, mixed precision automática.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Subclasear LightningModule con training_step, validation_step, configure_optimizers.
- Usar Trainer(max_epochs, accelerator='auto', devices='auto', precision='bf16-mixed', logger=...).
- Aplicar callbacks: EarlyStopping, ModelCheckpoint, LearningRateMonitor.
- Activar distributed con strategy='ddp' o 'fsdp' para multi-GPU sin reescribir nada.
- Loggear a W&B / TensorBoard / MLflow vía 1 línea.

Temas

- LightningModule vs nn.Module puro.
- Trainer args: max_epochs, devices, precision, strategy, accumulate_grad_batches.
- Callbacks integrados.
- LightningDataModule para data pipelines.
- Distributed training: ddp, fsdp, deepspeed.
- Logging multi-backend.

Definiciones y características

- LightningModule: extiende nn.Module con hooks (training_step, etc.).
- Trainer: orquesta el entrenamiento. Maneja loops, device, distributed.
- self.log(...): registra métrica al logger; se agrega en epoch/batch.
- strategy: 'auto' | 'ddp' | 'fsdp' | 'deepspeed_stage_2'.
- precision: '32-true' | '16-mixed' | 'bf16-mixed'.

Dataset / recursos

- Fashion-MNIST o cualquier dataset previo.
- Librerías: lightning, torch, torchmetrics.

Ejercicios

1. LightningModule básico: convertir el MLP de 108a a Lightning.
2. Callbacks: agregar EarlyStopping(patience=5) + ModelCheckpoint(save_top_k=3).
3. Mixed precision: Trainer(precision='bf16-mixed'). Comparar tiempo.
4. W&B logging: Trainer(logger=WandbLogger(project='test')). Ver curvas online.
5. DDP: si tenés 2+ GPUs, strategy='ddp', devices=2. Verificar speedup.

Homework verificable

Re-entrenar el modelo Fashion-MNIST en Lightning + W&B/TensorBoard:

1. LightningModule con train/val/test steps.
2. Trainer con EarlyStopping, ModelCheckpoint, mixed precision.
3. Logging a TensorBoard.
4. Reportar accuracy + screenshot del dashboard.

Criterio de aceptación: accuracy ≥ 0.87 ; dashboard muestra curvas de train/val.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
self.log no aparece en logger	Falta llamar a self.log('val_loss', loss)
Olvido return loss en training_step	Lightning no puede backprop. Fix: devolver
Multi-GPU con DDP rompe en Jupyter	DDP no funciona en notebooks. Fix: usar dd
Trainer(max_epochs=100) con datasets gigan	Sin max_steps. Fix: max_steps=10_000 como
Mixed precision con OOM	A veces empeora. Fix: bajar batch o usar b

Preguntas frecuentes

Lightning o PyTorch puro?

Lightning para producción / experimentos serios — quita boilerplate, agrega features (distributed, callbacks). Puro para tutoriales y casos muy custom.

Lightning vs HuggingFace Trainer?

Trainer (HF) está más integrado con transformers. Lightning es más general. Si trabajás con LLMs/HF, Trainer; sino Lightning.

FSDP cuándo?

Cuando el modelo no entra en una GPU. Lightning lo activa con strategy='fsdp'.

lightning o pytorch-lightning?

Mismo proyecto. lightning es el nuevo nombre desde 2023.

Logger recomendado?

W&B (Weights & Biases) — mejor DX, comparación de experimentos. TensorBoard si tenés que ser local-only.

Referencias

- Lightning docs.
- Falcon et al. (2019), PyTorch Lightning.
- W&B: <<https://wandb.ai/>>.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 124 — Clase 124 — tf.data API

Parte: 2 — Deep Learning · Fuente: Géron, cap. 13 § The Data API. Duración estimada: 65 min.

Objetivo

Construir pipelines de datos eficientes con `tf.data.Dataset`: leer desde memoria/archivos/CSV, transformar (`map`, `filter`), mezclar (`shuffle`), batchear (`batch`), prefetch (paraleliza CPU↔GPU). Saber por qué un buen pipeline de datos es la diferencia entre "GPU al 30 %" y "GPU al 95 %".

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Crear datasets desde varias fuentes: `from_tensor_slices`, `list_files`, `TextLineDataset`.
- Encadenar transformaciones: `.map(fn, num_parallel_calls=tf.data.AUTOTUNE)`, `.filter`, `.shuffle(buffer)`, `.batch(N)`, `.prefetch(tf.data.AUTOTUNE)`.
- Reconocer el orden correcto: `cache` → `shuffle` → `batch` → `prefetch`.
- Usar `tf.data.AUTOTUNE` y profilear con TensorBoard Profiler.
- Saber cuándo `cache()` vale la pena (datasets que caben en RAM).

Temas

- Lazy evaluation: el dataset es un grafo, no datos cargados.
- `shuffle(buffer_size)`: buffer chico → mal mezclado; `buffer = dataset_size` → perfecto pero RAM.
- `batch(N)` → cada elemento es ahora un mini-batch.
- `prefetch`: solapamiento CPU (loading) con GPU (training).
- `interleave` para leer múltiples archivos en paralelo.
- Métricas: `tf.data.experimental.assert_cardinality`.

Definiciones y características

- `Dataset.from_tensor_slices`: convierte un array NumPy en dataset.
- `map(fn, num_parallel_calls=AUTOTUNE)`: aplica `fn` a cada elemento; `AUTOTUNE` elige hilos.
- `shuffle(buffer_size)`: mezclado con buffer rotativo.
- `batch(N, drop_remainder=False)`: agrupa en batches.
- `prefetch(n)`: precarga `n` batches mientras la GPU procesa el actual.
- `cache()`: guarda en memoria (o archivo) después de la primera época.
- `AUTOTUNE`: TF mide y ajusta el número de hilos automáticamente.

Dataset / recursos

- Fashion-MNIST cargado vía `tf.data`.
- CSV grande para testear pipelines a archivo (anticipa 110 `TfRecord`).
- Librerías: `tensorflow`.

Ejercicios

1. Dataset desde NumPy: `ds = tf.data.Dataset.from_tensor_slices((x_train, y_train)).shuffle(1024).batch(32).prefetch(tf.data.AUTOTUNE)`. Iterar y verificar shapes.
2. Map con normalización: `.map(lambda x, y: (tf.cast(x, tf.float32)/255., y), num_parallel_calls=tf.data.AUTOTUNE)`.
3. Cache: comparar tiempo del 1er epoch vs 2do epoch con y sin `.cache()`.
4. Buffer chico: comparar `shuffle` con `buffer_size=10` vs `buffer_size=len(data)`. Inspeccionar el primer batch.
5. Profilear: usar `tf.profiler` (vía TensorBoard) y verificar dónde está el bottleneck — data loading vs compute.

Homework verificable

Pipeline production-ready para Fashion-MNIST:

1. `from_tensor_slices` con `shuffle`, augmentation simple (random flip), normalización, `batch=128`, `prefetch`.
2. Cachear el dataset (entra en RAM).
3. Train un MLP por 10 épocas; medir tiempo total.
4. Comparar contra pasar `x`, `y` directo a `model.fit`.

Criterio de aceptación: el pipeline `tf.data` debe igualar o ser un poco más rápido que pasar arrays directos; con cache, el 2do epoch en adelante es notablemente más rápido (skip de I/O y map).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>shuffle().batch()</code> muy mal mezclado	Buffer = 1000 con dataset de 60 000 → veci
Cache + <code>shuffle</code> en orden incorrecto	<code>.cache().shuffle(...)</code> : ok. <code>.shuffle(...).c</code>
<code>.batch(32, drop_remainder=False)</code> y el últi	A veces deseado, a veces rompe modelos cus
GPU idle 70 %	El pipeline es el bottleneck. Fix: <code>prefetc</code>
<code>.map(py_function(...))</code> lento	Las <code>py_function</code> no se vectorizan ni se eje

Preguntas frecuentes

¿Cuál es el orden canónico de las ops?

Dataset → map → cache → shuffle → batch → prefetch. Cache después de map (para no remap), antes de shuffle (cache de elementos individuales).

¿`prefetch(AUTOTUNE)` vs `prefetch(N)`?

AUTOTUNE casi siempre. Mide y ajusta. Solo fijá N si tenés constraints muy específicos.

¿`batch(32)` o `batch(128)`?

Depende del modelo y la GPU. Imágenes en VRAM grande: 128. Modelos grandes/Transformers: 8-32. Probá y mirá VRAM.

¿`tf.data` o PyTorch DataLoader?

Si usás TF: `tf.data`. PyTorch: `DataLoader` (similar concepto, `num_workers`). En 2026, los pipelines de Hugging Face usan `datasets library` que tiene buen interfaz para ambos.

¿Pipeline en GPU?

Algunas ops sí (decode JPG en GPU con `nvidia.dali` o `tf.image`). En general el pipeline corre en CPU y el modelo en GPU; `prefetch` solapa.

Referencias

- Géron, cap. 13 — The Data API.
- TF — `tf.data`: Build TensorFlow input pipelines.
- TF — Better performance with the `tf.data` API.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.

- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 125 — Clase 125 — TFRecord

Parte: 2 — Deep Learning · Fuente: Géron, cap. 13 § The TFRecord Format. Duración estimada: 60 min.

Objetivo

Aprender el formato TFRecord — el formato binario nativo de TF, optimizado para datasets grandes (cientos de GB) que no caben en RAM. Saber escribirlo (tf.io.TFRecordWriter), parsearlo (tf.io.parse_single_example), y por qué es estándar en TPU/Vertex AI para training a escala.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Serializar un tf.train.Example con Features ↔ Feature (ByteList, Int64List, FloatList).
- Escribir TFRecord shards: with tf.io.TFRecordWriter('file.tfrecord') as w: w.write(serialized).
- Leer con tf.data.TFRecordDataset('file.tfrecord').map(parse_fn).
- Splitear datasets grandes en múltiples shards (*.tfrecord-00000-of-00010) para paralelizar reads.
- Reconocer cuándo TFRecord vale la pena vs alternativas modernas (Parquet, WebDataset).

Temas

- tf.train.Example: estructura protobuf con Features (dict de strings a Feature).
- Feature types: ByteList, Int64List, FloatList.
- Serialize → escribir → leer → parse.
- Sharding: data.tfrecord-NNNNN-of-MMMMM.
- tf.data.experimental.bucket_by_sequence_length para batches eficientes por longitud (NLP).

Definiciones y características

- TFRecord: archivo binario de records seriales protobuf. Eficiente, splitteable, compresible.
- tf.train.Example: proto que envuelve un dict de features.
- SerializeToString(): convierte el Example en bytes para escribir.
- parse_single_example(serialized, feature_spec): parsea con un schema declarado.
- Sharding: dividir un dataset en N archivos → reads paralelos vía interleave.

Dataset / recursos

- Fashion-MNIST exportado a TFRecord.
- Librerías: tensorflow.

Ejercicios

1. Escribir: convertir Fashion-MNIST (60 000 imágenes) a 10 shards TFRecord. Cada Example tiene image (bytes) y label (int).
2. Leer y parsear: ds = tf.data.TFRecordDataset(glob.glob('shards/*.tfrecord')).map(parse_fn). Iterar y verificar shapes.
3. Compresión: escribir con options=tf.io.TFRecordOptions(compression_type='GZIP'). Comparar tamaño.
4. Reads paralelos: ds = ds.interleave(lambda f: tf.data.TFRecordDataset(f), num_parallel_calls=AUTOTUNE). Medir speedup vs lectura serial.
5. Schema: usar tf.io.FixedLenFeature (tamaño fijo) vs VarLenFeature (variable, returns SparseTensor).

Homework verificable

Pipeline completo de Fashion-MNIST con TFRecord:

1. Escribir 10 shards.
2. Leer con interleave + map + batch + prefetch.
3. Entrenar un modelo y comparar tiempo vs cargar desde NumPy.

Criterio de aceptación: el pipeline TFRecord debe escalar mejor cuando los datos no caben en RAM (simular limitando RAM si es necesario); para Fashion-MNIST en RAM, los tiempos deben ser similares.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
parse_single_example falla con "Key not fo	Schema no coincide con lo escrito. Fix: im
Tamaño TFRecord enorme	No comprimiste. Fix: compression_type='GZI
tf.io.parse_tensor vs parse_single_example	Para tensores serializados con tf.io.seria
Reads serializados secuenciales en un únic	El throughput está limitado por 1 disco. F
Sin drop_remainder los últimos batches tie	Algunos modelos rompen. Fix: batch(N, drop

Preguntas frecuentes

¿TFRecord o Parquet?

TFRecord para TF/JAX. Parquet para data science general (Polars, Spark, DuckDB). PyTorch tiene WebDataset que es similar a TFRecord pero más portable (tarballs).

¿Cuándo TFRecord es necesario?

Cuando los datos no caben en RAM (>10 GB) y querés training rápido. Para datasets chicos (< 1 GB), arrays NumPy alcanzan.

¿Cuántos shards?

Regla práctica: 100-1000 archivos, cada uno 100 MB - 1 GB. Para TPU training, ≥ 8 shards por nodo.

¿Comprimir o no?

Comprimir si I/O es el bottleneck. Si CPU es el cuello (decode lento), no comprimir. Probá ambos.

¿Hugging Face datasets?

Sí, formato moderno (basado en Arrow/Parquet) con caching automático y API más amigable. Para LLMs, default industrial. Saber TFRecord es útil legacy pero datasets lo está reemplazando.

Referencias

- Géron, cap. 13 — The TFRecord Format.
- TF — TFRecord and tf.train.Example.
- WebDataset — alternativa PyTorch.
- Hugging Face datasets — alternativa moderna multi-framework.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 126 — Clase 126 — Keras preprocessing layers

Parte: 2 — Deep Learning · Fuente: Géron, cap. 13 § The Keras Preprocessing Layers. Duración estimada: 65 min.

Objetivo

Hacer preprocesamiento dentro del modelo con las preprocessing layers de Keras (Normalization, StringLookup, IntegerLookup, Discretization, CategoryEncoding, Hashing, TextVectorization). Beneficio: el preprocesamiento viaja con el modelo (.keras), no como código separado — elimina el clásico "train-serve skew" en producción.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Usar Normalization() con .adapt(data) para escalar features tabulares.
- Usar StringLookup para encoding de categóricas.
- Aplicar TextVectorization para tokenización + indexing.
- Construir un modelo "todo en uno": preprocesamiento + red en el mismo keras.Model.
- Reconocer la ventaja: model.predict(raw_data) funciona, sin necesidad de scaler separado.

Temas

- Normalization: subtrae mean, divide por std. .adapt(data) aprende los stats.
- StringLookup / IntegerLookup: mapea categorías a índices enteros.
- CategoryEncoding: one-hot, multi-hot, count.
- Discretization: convierte continua en bucket (bin_boundaries=...).
- Hashing: mapea categorías a buckets via hash (sin necesidad de vocabulario).
- TextVectorization: tokeniza + lookup en una capa.

Definiciones y características

- .adapt(data): aprende los parámetros (mean/std, vocabulario, etc.) desde un dataset. Reemplaza un fit-then-transform separado.
- Normalization: $(x - \text{mean}) / \text{std}$. Mean/std aprendidos con .adapt.
- StringLookup(vocabulary=..., output_mode='int'): dict {categoría: índice}. 'multi_hot' para one-hot.
- Hashing(num_bins=1024): para vocabulario gigante o variable. Hash mod num_bins.
- TextVectorization(max_tokens=10_000, output_mode='int', output_sequence_length=100): tokeniza + indexa.

Dataset / recursos

- California Housing (tabular).
- IMDB reviews (texto).
- Librerías: tensorflow, keras.

Ejercicios

1. Normalization tabular: norm = Normalization(); norm.adapt(X_train); X_norm = norm(X_test). Verificar que mean ≈ 0 , std ≈ 1 .
2. StringLookup: con un array de categorías, lookup = StringLookup(); lookup.adapt(categorias); lookup(['A', 'B', 'C']) $\rightarrow$ tensor de ints.
3. Modelo end-to-end tabular: inputs = Input((n,)); x = Normalization()(inputs); x = Dense(64, ...)(x); Después .adapt(...) la capa norm con X_train.

4. TextVectorization: sobre IMDB, tokenizar, entrenar un modelo de sentimiento.
5. Hashing: con un dataset que tiene 100 000 categorías únicas, Hashing(1024) lo mapea a 1024 buckets. Comparar accuracy vs StringLookup con vocabulario truncado.

Homework verificable

Modelo end-to-end sobre California Housing:

1. Pipeline tf.data con CSV.
2. Modelo con Normalization layer adaptada al train set.
3. Entrenar y guardar con model.save('m.keras').
4. Recargar y predecir sobre datos RAW (sin pre-escalar). Verificar que funciona.

Criterio de aceptación: predicción sobre raw data debe dar el mismo resultado que pre-escalar a mano y pasar al modelo "sin Normalization layer".

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Olvido .adapt() y los stats de Normalizati	Normalization no aprende. Fix: .adapt(trai
.adapt con val/test data	Leakage. Fix: solo train.
StringLookup no encuentra una categoría en	OOV (out-of-vocabulary). Por default se ma
TextVectorization muy lento	Suele ser por max_tokens enorme o output_s
Capa preprocessing fuera del modelo y al s	Anti-pattern. Fix: meter las capas DENTRO

Preguntas frecuentes

¿Normalization o StandardScaler de sklearn?

Si vas a deployar el modelo: Normalization de Keras (viaja con el modelo). Para análisis offline: StandardScaler es igual de bueno y más sklearn-idiomático.

¿Cuándo Hashing vs StringLookup?

Hashing para vocabularios gigantes (>100k) o dinámicos (categorías nuevas aparecen continuamente). StringLookup para vocabulario fijo y manejable.

¿Preprocessing en GPU?

Las preprocessing layers corren en CPU por default. Para mover a GPU: with tf.device('/GPU:0'). La normalización es barata; image augmentation puede ser bueno en GPU.

¿TextVectorization reemplaza a Hugging Face tokenizers?

Para tokenización word-level simple, sí. Para BERT/GPT necesitás los tokenizers específicos (subword/BPE) de Hugging Face — clase 127.

¿Y para LLMs?

LLMs llevan su propio tokenizer (BPE, SentencePiece, tiktoken). TextVectorization no aplica.

Referencias

- Géron, cap. 13 — Preprocessing Data with Keras Preprocessing Layers.
- Keras — Preprocessing layers.
- Keras — Working with preprocessing layers.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 127 — Clase 127 — TensorFlow Datasets (TFDS)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 13 § The TensorFlow Datasets (TFDS) Project.
Duración estimada: 40 min.

Objetivo

Conocer TFDS —catálogo de datasets prearmados (CIFAR, ImageNet, IMDB, COCO, MNIST, GLUE, etc.)— y la alternativa moderna Hugging Face datasets (estándar en NLP/LLMs). Cargar datasets de prueba, hacer splits, y entender por qué TFDS es práctico para benchmarks reproducibles.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Listar datasets disponibles con `tfds.list_builders()`.
- Cargar con `tfds.load('cifar10', split=['train', 'test'], as_supervised=True)`.
- Hacer splits custom con la slicing API: `'train[:80%]', 'train[80%:]'`.
- Reconocer cuando usar `tfds` vs `huggingface_hub.datasets`.

Temas

- Catálogo TFDS: 200+ datasets, descarga automática + cache.
- `as_supervised=True` → tuplas (x, y).
- Splits: `'train[:80%]', 'train[-20%:]', 'all'`.
- `dataset.info` con metadata (shape, num_classes, etc.).
- Hugging Face datasets: estándar moderno multi-framework.

Definiciones y características

- TFDS: librería de Google con datasets pre-procesados en formato TFRecord.
- `as_supervised`: si True, devuelve (input, label). Si False, devuelve dict con todas las features.
- Slicing API: notación tipo Python sobre los splits.
- Hugging Face datasets: equivalente moderno, basado en Arrow, soporta PyTorch/TF/JAX, comunidad enorme.

Dataset / recursos

- CIFAR-10 vía TFDS.
- IMDB vía HF datasets.
- Librerías: `tensorflow-datasets` (pip install tensorflow-datasets), opcional `datasets` (HF).

Ejercicios

1. Listar: `tfds.list_builders()` → primeros 20 datasets.
2. CIFAR-10: `(ds_train, ds_test), info = tfds.load('cifar10', split=['train', 'test'], as_supervised=True, with_info=True)`. Imprimir info.
3. Slicing: cargar `train[:90%]` + `train[90%:]` como train/val split.
4. Pipeline: `ds_train.map(preprocess).cache().shuffle(1024).batch(32).prefetch(AUTOTUNE)`.
5. HF datasets: `from datasets import load_dataset; ds = load_dataset('imdb')`. Inspeccionar; convertir a

tf.data con `ds.to_tf_dataset(...)`.

Homework verificable

Entrenar un MLP simple en CIFAR-10 cargado vía TFDS:

1. Cargar con `tfds.load(..., as_supervised=True)`.
2. Pipeline con preprocessing, batch, prefetch.
3. MLP [1024, 512, 256, 10] con BN + Dropout.
4. Reportar accuracy en test.

Criterio de aceptación: accuracy en test ≥ 0.45 (MLP no es ideal para CIFAR — CNN gana — pero debe llegar a 45 %; clase 113+ lo mejora con CNN).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Primera carga muy lenta	TFDS descarga + procesa. Cachea en ~/tenso
OutOfRangeError al iterar	Te quedaste sin batches sin pasar <code>.repeat()</code>
Olvidar <code>as_supervised=True</code>	El dataset devuelve dicts, modelo espera t
<code>tfds.load(split='train+test')</code>	Concatena ambos splits. Para train/val: us
Disco lleno por datasets grandes (ImageNet)	TFDS cachea todo. Fix: borrar ~/tensorflow

Preguntas frecuentes

¿TFDS o HF datasets?

Para benchmarks tradicionales (CIFAR, IMDB, MNIST) ambos sirven. Para NLP moderno (text generation, instruction datasets), HF. Para TF nativo + GCP/Vertex AI, TFDS.

¿TFDS funciona con PyTorch?

Sí, con `tfds.as_numpy(...)` o `tfds.load(..., builder_kwargs={'as_dataset_kwargs': ...})`. Pero HF es más natural.

¿Datasets propios cargo cómo?

Para datasets locales no listados: `tfds.folder_dataset.ImageFolder('/path')`, o construir un `DatasetBuilder` custom. En HF: `load_dataset('csv', data_files='...')`.

¿Datasets de imagen grandes (ImageNet)?

TFDS los maneja bien (shardea, cache). Para TPU + Vertex AI, ya está optimizado.

¿Versionado de datasets?

TFDS versiona por defecto ('cifar10:3.0.2'). HF también. Es importante reportarlo en papers/reports.

Referencias

- Géron, cap. 13 — The TensorFlow Datasets (TFDS) Project.
- TFDS catalog.
- Hugging Face datasets.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 128 — Clase 128 — Capas convolucionales, filtros, feature maps

Parte: 2 — Deep Learning · Fuente: Géron, cap. 14 § Convolutional Layers. Duración estimada: 75 min.

Objetivo

Entender la operación de convolución 2D — un filtro $K \times K$ se desliza sobre la imagen produciendo un feature map —, los hiperparámetros (filters, kernel_size, strides, padding), por qué las CNN son parameter-efficient vs MLPs (sharing + locality + translation invariance) y cómo aprenden jerarquías visuales (bordes → texturas → partes → objetos).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar Conv2D(filters=32, kernel_size=3, strides=1, padding='same', activation='relu').
- Calcular el shape de la salida: $H' = (H - K + 2P)/S + 1$.
- Visualizar feature maps (activaciones intermedias) y filtros aprendidos.
- Diferenciar padding='same' (preserva tamaño) de 'valid' (reduce).
- Calcular el # de parámetros de una Conv2D: $K \times K \times C_{in} \times C_{out} + C_{out}$ (mucho menos que Dense equivalente).

Temas

- Operación convolución 2D paso a paso.
- Filters / kernels como features detectables.
- Feature maps como representación espacial.
- Stride: paso del filtro.
- Padding: bordes; 'same' agrega zeros para preservar shape.
- Parameter sharing: el mismo filtro se aplica en toda la imagen.

Definiciones y características

- Filter / kernel: matriz pequeña (3×3 , 5×5) de pesos aprendibles.
- Feature map: salida de aplicar un filtro a toda la imagen. Una capa con 32 filtros produce 32 feature maps.
- Stride: cuántos píxeles se mueve el filtro entre aplicaciones. 1 = denso; 2 = subsampling.
- padding='same': agrega zeros para que la salida tenga la misma altura/ancho que la entrada.
- padding='valid': sin padding. Salida más chica.
- Receptive field: porción de la imagen original que afecta una neurona específica del feature map.

Dataset / recursos

- MNIST / Fashion-MNIST (1 canal) o CIFAR-10 (3 canales).
- Librerías: tensorflow, keras, matplotlib.

Ejercicios

1. Conv básica: Conv2D(8, 3, padding='same', activation='relu')(input_28x28x1). Verificar shape de salida: (batch, 28, 28, 8).
2. Conteo parámetros: para Conv2D(32, kernel_size=5) aplicado a entrada (28, 28, 1), calcular params ($551 \times 32 + 32 = 832$).
3. Stride 2: Conv2D(32, 3, strides=2, padding='same')(x). Shape de salida: (batch, $H/2$, $W/2$, 32).
4. Visualizar feature maps: entrenar una mini-CNN en MNIST; tomar las activaciones intermedias para una imagen y visualizar.

5. Filtros aprendidos: visualizar `model.layers[0].kernel.numpy()` para los primeros filtros. Para CIFAR, suelen verse bordes/colores básicos.

Homework verificable

Mini-CNN para Fashion-MNIST:

1. Conv(32, 3) → MaxPool(2) → Conv(64, 3) → MaxPool(2) → Flatten → Dense(128) → Dense(10).
2. Entrenar y reportar accuracy.
3. Visualizar feature maps de la primera Conv para 3 imágenes test.
4. Calcular # parámetros total y compararlo con un MLP equivalente en tamaño (Dense(128) → Dense(64) → Dense(10) sin Conv).

Criterio de aceptación: accuracy de la CNN ≥ 0.91 (mejor que MLP); # parámetros de la CNN MLP equivalente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Input shape error expected ndim=4, got ndi	Olvidaste el canal: MNIST viene (60000, 28
Conv2D + Dense sin Flatten o GlobalAverage	Dense espera 2D. Fix: agregá Flatten() o G
padding='valid' con muchas capas → shape s	Cada conv reduce. Fix: usar 'same' por def
Stride alto pierde información	strides=4 con kernel=3 salta píxeles. Fix:
Filtros aprendidos no se ven nada	Inicialización mala o modelo no entrenado.

Preguntas frecuentes

¿Cuántos filtros?

Empieza chico y duplicá al profundizar. Patrón estándar: 32 → 64 → 128 → 256.

¿Kernel 3×3 o 5×5?

3×3 es default moderno (VGG, ResNet). Dos 3×3 apilados tienen el mismo receptive field que 5×5 con menos parámetros.

¿stride o MaxPool para downsampling?

Tradicionalmente MaxPool. ResNet moderno usa stride 2 en Convs y elimina pooling intermedio. Ambos funcionan.

¿Conv1D para series, Conv3D para video?

Sí. Conv1D para secuencias temporales (clase 121 WaveNet). Conv3D para video ($T \times H \times W \times C$).

¿Convs vs Transformers en visión?

CNN domina en visión clásica (eficiente, buen prior espacial). ViT (Vision Transformer) supera con datasets muy grandes (>10M imágenes). ConvNeXt (2022) recuperó terreno para CNN. Ver clase 126.

Referencias

- Géron, cap. 14 — Deep Computer Vision Using Convolutional Neural Networks.
- LeCun et al. (1998), Gradient-based learning applied to document recognition — LeNet, paper canónico.
- Keras Conv2D.
- CS231n CNN notes — referencia clásica de Stanford.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 129 — Clase 129 — Pooling

Parte: 2 — Deep Learning · Fuente: Géron, cap. 14 § Pooling Layers. Duración estimada: 45 min.

Objetivo

Conocer pooling — operación sin parámetros que reduce dimensiones espaciales: MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D. Saber que max-pool agrega invariancia local a translación y reduce cómputo de capas posteriores. Comparar contra el approach moderno (stride > 1 en Conv).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar MaxPooling2D(2), AveragePooling2D(2), GlobalAveragePooling2D() y conocer sus shapes de salida.
- Diferenciar max-pool (preserva la característica más fuerte) de average-pool (promedio espacial).
- Aplicar GlobalAveragePooling2D antes de la cabeza Dense (estándar en CNN modernas — reemplaza Flatten).
- Reconocer que pooling no tiene parámetros entrenables (es solo una reducción).
- Saber que ResNet/ConvNeXt usan stride en lugar de pool intermedio.

Temas

- MaxPool: ventana $k \times k \rightarrow$ toma el máximo. Default pool_size=2, strides=2 $\rightarrow$ halve H y W.
- AvgPool: promedio de la ventana. Suaviza.
- GlobalAvgPool: una sola activación por feature map $\rightarrow$ (batch, channels).
- Invariancia a translación local: si el feature se mueve 1 px, el max no cambia.
- Pool vs stride: equivalentes en muchos casos; tendencia moderna es eliminar pool intermedio.

Definiciones y características

- MaxPooling2D(pool_size=2, strides=2, padding='valid'): ventana $2 \times 2 \rightarrow$ 1 valor (el máximo).
- AveragePooling2D: como max pero con promedio.
- GlobalAveragePooling2D: agrega sobre toda la spatial dim $\rightarrow$ (batch, channels). Sin pool size.
- GlobalMaxPooling2D: similar, con max.
- Output shape: para pool_size=2 stride=2: (H/2, W/2, C).

Dataset / recursos

- Fashion-MNIST o CIFAR-10.
- Librerías: tensorflow, keras.

Ejercicios

1. MaxPool shape: aplicar MaxPool(2) a tensor (1, 28, 28, 32). Verificar salida (1, 14, 14, 32).
2. MaxPool vs AvgPool: con la misma CNN, intercambiar y comparar accuracy en Fashion-MNIST. MaxPool suele ganar marginal.
3. GlobalAvgPool reemplaza Flatten: arquitectura Conv $\rightarrow$ Conv $\rightarrow$ GlobalAvgPool $\rightarrow$ Dense(10) vs Conv $\rightarrow$ Conv $\rightarrow$ Flatten $\rightarrow$ Dense(128) $\rightarrow$ Dense(10). Comparar params y accuracy.

4. Pool vs stride: comparar $\text{Conv}(\text{stride}=1) \rightarrow \text{Pool}(2)$ vs $\text{Conv}(\text{stride}=2)$ (sin pool). En modelos chicos son prácticamente equivalentes.
5. `padding='same'` en pool: comportamiento si H es impar. `valid` recorta, `same` agrega zeros.

Homework verificable

Comparar 3 arquitecturas sobre Fashion-MNIST:

1. $\text{Conv}(32) \rightarrow \text{MaxPool} \rightarrow \text{Conv}(64) \rightarrow \text{MaxPool} \rightarrow \text{Flatten} \rightarrow \text{Dense}(128) \rightarrow \text{Dense}(10)$.
2. $\text{Conv}(32) \rightarrow \text{MaxPool} \rightarrow \text{Conv}(64) \rightarrow \text{MaxPool} \rightarrow \text{GlobalAvgPool} \rightarrow \text{Dense}(10)$.
3. $\text{Conv}(32, \text{stride}=2) \rightarrow \text{Conv}(64, \text{stride}=2) \rightarrow \text{GlobalAvgPool} \rightarrow \text{Dense}(10)$ (sin pool).

Reportar accuracy y # parámetros.

Criterio de aceptación: la opción 2 tiene muchos menos parámetros y accuracy similar; la 3 también es competitiva (validando el approach moderno).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>pool_size=3, strides=1</code> recorta poco	Posible pero raro. Fix: usual es <code>pool=2</code> , s
MaxPool antes del primer Conv	Pierde detalle al inicio. Fix: pool después
Olvido pool y modelo Dense tiene millones	Flatten de feature maps grandes. Fix: pool
<code>padding='same'</code> en pool con H impar genera	TF agrega un row/col de zeros. Fix: pre-cr
Usar pooling en NLP / Transformers	No tiene sentido. Fix: pool es espacial; e

Preguntas frecuentes

¿MaxPool o AvgPool?

MaxPool por default — captura "el más fuerte" (útil en clasificación). AvgPool en regresión espacial (segmentación final) o cuando el promedio es semánticamente relevante.

¿GlobalAvgPool obligatorio en ResNet?

Sí, es la última capa antes del FC final. Reduce el riesgo de overfit del Flatten + Dense gigante.

¿Pool intermedio aún se usa?

CNNs clásicas (VGG, LeNet) sí. ResNet/ConvNeXt usan stride en conv (sin pool intermedio).

¿`pool_size=4` o más grande?

Raro. Casi siempre 2. Más grande pierde información rápido.

¿Pool diferenciable?

MaxPool tiene gradiente subdiferencial (= 1 en el max, 0 en el resto). Avg es diferenciable normal. TF/PyTorch los manejan transparente.

Referencias

- Géron, cap. 14 — Pooling Layers.
- Keras MaxPooling2D.
- Springenberg et al. (2015), Striving for Simplicity: The All Convolutional Net — argumenta eliminar pool intermedio.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 130 — Clase 130 — Arquitecturas CNN: LeNet, AlexNet, VGG, GoogLeNet, ResNet, Xception, SNet, EfficientNet, ConvNeXt

Parte: 2 — Deep Learning · Fuente: Géron, cap. 14 § CNN Architectures + papers originales. Duración estimada: 95 min.

Objetivo

Conocer la historia y evolución de las arquitecturas CNN desde LeNet-5 (1998) hasta ConvNeXt (2022) — qué innovación introdujo cada una, por qué importaba, y cuál usar hoy. Identificar 3 patrones clave: profundidad creciente, módulos con paths múltiples, eficiencia paramétrica.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Trazar la línea temporal: LeNet → AlexNet → VGG → GoogLeNet (Inception) → ResNet → Xception → SNet → EfficientNet → ConvNeXt.
- Reconocer qué innovación trajo cada una: ReLU + dropout (AlexNet), kernels 3×3 (VGG), módulos Inception (GoogLeNet), skip connections (ResNet), depthwise-separable (Xception), squeeze-and-excite (SNet), compound scaling (EfficientNet), modernización con receta ConvNeXt.
- Implementar un mini-ResNet con Add() y skip connections.
- Cargar arquitecturas de keras.applications y leer sus tamaños (MobileNetV3, EfficientNetB0, ConvNeXtTiny).
- Elegir la arquitectura adecuada según constraint (latency, accuracy, memory).

Temas

- LeNet-5 (1998): 7 capas, MNIST. La cuna.
- AlexNet (2012): GPU + ReLU + Dropout. Ganó ImageNet → revolución DL.
- VGG (2014): muy uniforme — solo conv 3×3 + maxpool. 138M parámetros.
- GoogLeNet / Inception (2014): módulos paralelos con kernels 1×1, 3×3, 5×5.
- ResNet (2015): skip connections → entrenamiento de redes de 152 capas posible. Cambió el juego.
- Xception (2017): depthwise-separable convolutions → eficiencia.
- SNet (2017): attention sobre canales (squeeze-and-excite).
- EfficientNet (2019): scaling balanceado de depth/width/resolution.
- ConvNeXt (2022): "modernizar ResNet" con trucos de ViT.

Definiciones y características

- Skip / residual connection: $y = x + F(x)$. Permite gradiente fluir y entrenar redes muy profundas.
- Bottleneck: 1×1 conv → 3×3 conv → 1×1 conv (reduce → opera → expande). Reduce parámetros.
- Depthwise-separable conv: una conv que actúa por canal, seguida de 1×1 conv que mezcla canales. ~10× menos parámetros.
- Squeeze-and-Excite: re-pesa los canales aprendido (GlobalAvgPool → Dense → sigmoid → multiply).
- Compound scaling (EfficientNet): escalar depth, width, resolution juntos con coeficientes balanceados.
- keras.applications: catálogo con pesos ImageNet preentrenados de todas estas arquitecturas.

Dataset / recursos

- ImageNet (vía transfer) o un dataset propio chico.
- Librerías: tensorflow, keras.applications.

Ejercicios

1. Cargar varios modelos: `keras.applications.{ResNet50, EfficientNetB0, ConvNeXtTiny}(weights='imagenet')`. Comparar `model.count_params()` y `model.summary()`.
2. Mini-ResNet: implementar 4 bloques residuales: `def res_block(x): return x + Conv(64,3,padding='same')(ReLU()(Conv(64,3,padding='same')(x)))`. Apilar y entrenar.
3. Skip connection a mano: comparar entrenamiento de un "ResNet" sin skip vs con skip a 50 capas. Sin skip no entrena.
4. Depthwise-separable: usar `SeparableConv2D` en lugar de `Conv2D` en el mismo modelo. Comparar params y accuracy.
5. Squeeze-Excite: implementar un bloque SE manualmente: `s = GlobalAvgPool(x); s = Dense(C//r)(s); s = Dense(C, sigmoid)(s); return x * Reshape((1,1,C))(s)`.

Homework verificable

Comparar 4 arquitecturas en un dataset propio de imágenes (≥ 5 clases) con transfer learning:

1. ResNet50 (clásico).
2. EfficientNetB0 (eficiente).
3. MobileNetV3Small (móvil).
4. ConvNeXtTiny (moderno).

Para cada uno: feature extraction (frozen) + fine-tune. Reportar accuracy, # parámetros, tiempo de inference.

Criterio de aceptación: ConvNeXt o EfficientNet deben ganar en accuracy; MobileNet en velocidad de inference. ResNet sirve de baseline.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cargar ResNet50 sin <code>include_top=False</code> y pa	El head ImageNet espera 1000 clases. Fix:
Cargar EfficientNet y pasarle imágenes en	Espera preprocesamiento específico. Fix: k
Implementar ResNet desde cero y <code>Add()</code> fall	Skip connection requiere shapes iguales —
MobileNet para batch grande en CPU lento	Está optimizado para GPU/mobile. Fix: para
ConvNeXt en TF Lite no exporta	Algunos ops de ConvNeXt aún no portados a

Preguntas frecuentes

¿Cuál arquitectura uso en 2026?

- Default: EfficientNetB0 o ConvNeXtTiny.
- Movil/embedded: MobileNetV3Small.
- Máxima accuracy con budget: ConvNeXt-L o EfficientNet-L2.
- Visión + texto unificado: CLIP / SigLIP (clase 129).

¿ResNet aún se usa?

Sí, como baseline universal. Confiable, bien entendida, todo framework la soporta.

¿ViT supera a CNNs?

Con datasets muy grandes ($>10M$ imágenes), ViT (clase 126) gana. Con datasets típicos del cliente ($<100k$

imágenes), CNNs ganan o empatan. ConvNeXt mostró que CNNs bien modernizadas igualan a ViT.

¿"Compound scaling" qué significa?

EfficientNet showed que escalar $\text{depth} \times \text{width} \times \text{resolution}$ con coeficientes balanceados (ϕ) es más eficiente que escalar uno solo. Por eso B0, B1, B2... son la misma red escalada.

¿Tantas opciones por qué?

Cada una optimizó una métrica distinta (accuracy puro, params, latencia, eficiencia mobile). No hay "una mejor" — depende del constraint.

Referencias

- Géron, cap. 14 — CNN Architectures.
- Krizhevsky et al. (2012), AlexNet, NeurIPS.
- Simonyan & Zisserman (2014), VGG.
- Szegedy et al. (2015), Inception (GoogLeNet), CVPR.
- He et al. (2015), Deep Residual Learning (ResNet), CVPR.
- Chollet (2017), Xception.
- Hu et al. (2018), Squeeze-and-Excitation Networks, CVPR.
- Tan & Le (2019), EfficientNet, ICML.
- Liu et al. (2022), A ConvNet for the 2020s (ConvNeXt), CVPR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 131 — Clase 131 — Transfer learning con CNNs preentrenadas

Parte: 2 — Deep Learning · Fuente: Géron, cap. 14 § Using Pretrained Models from Keras. Duración estimada: 70 min.

Objetivo

Aplicar transfer learning específicamente en visión — el caso de uso más común del campo. Profundizar la clase 101 con receta industrial: data augmentation, fine-tuning gradual, learning rate diferencial, y manejo de BatchNorm en fine-tuning (un gotcha clásico).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir pipeline con `image_dataset_from_directory` + augmentation (RandomFlip, RandomRotation, RandomZoom).
- Aplicar el `preprocess_input` específico del modelo base.
- Hacer fine-tuning en 2 fases con LR diferencial.
- Manejar correctamente BatchNorm en fine-tuning (mantenerlo en inference mode si descongelaste pocas capas).
- Comparar feature extraction (frozen base + nueva head) vs fine-tune (descongelar todo) según tamaño del dataset.

Temas

- Augmentation como capa: RandomFlip, RandomRotation, RandomZoom, RandomCrop, RandomContrast.
- preprocess_input por modelo (cada red espera su escalado).
- 2-stage training: freeze + head warmup → unfreeze + LR bajo.
- BN gotcha: cuando descongelás, BN sigue actualizando moving averages → puede dañar pretraining.
- MixUp, CutMix, RandAugment como augmentations modernas (anticipo).

Definiciones y características

- Augmentation layer: capa Keras que transforma random las imágenes durante training, no en inference.
- preprocess_input: función específica del modelo (keras.applications.<modelo>.preprocess_input).
- Feature extraction: base frozen, solo entreno head. Bueno para $n < 1000$.
- Fine-tune: descongelar parte/total y reentrenar con LR muy bajo.
- LR diferencial: capas tempranas LR bajo ($1e-5$), tardías más alto ($1e-4$).

Dataset / recursos

- Dataset chico propio o tfds.load('cats_vs_dogs') o tfds.load('tf_flowers').
- Modelo base: EfficientNetB0 o MobileNetV3Small.
- Librerías: tensorflow, keras, keras.applications.

Ejercicios

1. Pipeline con augmentation: RandomFlip + RandomRotation(0.1) + RandomZoom(0.1) como capas. Visualizar imágenes augmentadas.
2. Modelo con base + head: base = EfficientNetB0(include_top=False, weights='imagenet'); base.trainable = False; model = Sequential([data_aug, preprocess_input, base, GlobalAvgPool, Dropout, Dense(num_classes)]).
3. Etapa 1 (head warmup): Adam($1e-3$), entrenar 5 épocas.
4. Etapa 2 (fine-tune): base.trainable = True, recompilar con Adam($1e-5$). Entrenar 10 épocas. Verificar mejora.
5. BN gotcha: con base.trainable = True pero pasando training=False a la base durante fine-tuning → BN no se actualiza. Comparar.

Homework verificable

Sobre un dataset de 5 categorías (e.g., flores):

1. Pipeline completo con augmentation.
2. Transfer learning con EfficientNetB0 en 2 fases.
3. Reportar accuracy de cada fase.
4. Comparar contra entrenar EfficientNet desde cero (sin transfer).

Criterio de aceptación: la etapa 2 mejora la 1 en ≥ 2 pp; transfer learning supera "desde cero" por mucho margen con dataset chico.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Imágenes pasadas en [0, 255] cuando el mod	Fix: agregar preprocess_input correcto.
Augmentation aplicada en val/test también	Augmentation debe ser solo en train. Fix:
Fine-tuning con LR= $1e-3$ → catastrophic for	Fix: LR 10-100× más bajo.
Forgot to recompilar después de cambiar tra	El optimizer no ve los nuevos pesos como t
BatchNorm en fine-tuning actualiza moving	Para datasets chicos, mantener BN en infer

Preguntas frecuentes

¿Cuántas imágenes mínimas?

50-100 por clase para feature extraction. 200-500 por clase para fine-tuning completo. < 50 → considerar data augmentation agresiva o few-shot learning.

¿Qué proporción de capas descongelo?

Empezá con todo. Si overfittea, congelar las primeras N. Si underfittea, descongelar todo.

¿include_top=False siempre?

Sí para transfer learning (querés head custom).

¿Augmentation en GPU o CPU?

GPU es más rápido si está disponible (with `tf.device('/GPU:0')`). Augmentation pesada es lo más comúnmente bottleneck en CPU.

¿Test-time augmentation (TTA)?

Predecir varias versiones aumentadas de la misma imagen y promediar. Mejora accuracy 0.5-2 pp. Buen truco para Kaggle.

Referencias

- Géron, cap. 14 — Using Pretrained Models from Keras.
- Yosinski et al. (2014), How transferable are features in deep neural networks?, NeurIPS.
- Cubuk et al. (2020), RandAugment.
- Keras — Image data preprocessing.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 132 — Clase 132 — Localización, detección, segmentación (+ DETR, Segment Anything, YOLOv11)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 14 § Object Detection y § Semantic Segmentation + papers DETR, SAM, YOLOv11. Duración estimada: 90 min.

Objetivo

Saber detectar y segmentar objetos en imágenes — la tarea de visión más compleja y más comercial. Conocer la evolución: Faster R-CNN (two-stage, lento + preciso) → YOLO (one-stage, rápido) → DETR (Transformer, end-to-end) → YOLOv11 (estado del arte 2024) → Segment Anything (SAM/SAM 2) (foundation model para segmentación).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Distinguir las 4 tareas: clasificación, localización (1 objeto), detección (N objetos + cajas), segmentación (pixel-wise mask).
- Usar un modelo YOLOv11 preentrenado con ultralytics: `model = YOLO('yolo11n.pt')`; `results =`

model('imagen.jpg').

- Aplicar Segment Anything (SAM 2) para segmentación promptable con puntos o cajas como input.
- Reconocer cuándo elegir DETR (mejor formal, lento) vs YOLO (rápido, default industrial) vs SAM (pre-entrenado universal).
- Métricas: IoU, mAP, COCO AP@[.5:.05:.95].

Temas

- Localización vs detección vs instance segmentation vs semantic segmentation.
- IoU, mAP, COCO benchmark.
- Anchors vs anchor-free.
- One-stage (YOLO, SSD) vs two-stage (Faster R-CNN).
- Complemento moderno: DETR (Carion et al. 2020), SAM/SAM 2 (Meta 2023/2024), YOLOv11 (Ultralytics 2024).
- Pre-trained pipelines: ultralytics, transformers (DETR), segment_anything.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 117b — Segment Anything (SAM / SAM 2)
- Clase 117c — YOLOv11 práctico: detección, segmentación, pose, tracking

Definiciones y características

- Bounding box: (x_min, y_min, x_max, y_max) o (cx, cy, w, h).
- IoU (Intersection over Union): $\text{area}(\text{intersección}) / \text{area}(\text{unión})$. Match si $\text{IoU} > 0.5$.
- mAP (mean Average Precision): promedio del AP sobre clases y umbrales IoU. COCO usa IoU 0.5 $\rightarrow$ 0.95 en pasos de 0.05.
- NMS (Non-Maximum Suppression): post-procesado que elimina cajas superpuestas dejando la de mejor score. DETR no la necesita.
- Anchor-based: pre-define cajas en cada posición y aprende offsets.
- Anchor-free: predice directamente, sin pre-definir cajas (FCOS, DETR, YOLOX+).
- Semantic vs instance segmentation: semantic = mismo color para todas las instancias de "persona"; instance = cada persona con su color.
- Promptable segmentation (SAM): dado un punto/caja/texto, devolver la máscara correspondiente.

Dataset / recursos

- COCO (detección/seg estándar): `tlds.load('coco/2017')`.
- Pascal VOC: más chico, bueno para experimentos.
- Custom: anotaciones en YOLO format (txt por imagen) o COCO format (JSON).
- Librerías: ultralytics (pip install ultralytics), transformers, segment-anything.

Ejercicios

1. YOLO inference: cargar yolo11n.pt y detectar sobre 3 imágenes propias. Visualizar boxes + labels.
2. DETR inference: usar facebook/detr-resnet-50 desde HF. Comparar resultados con YOLO sobre las mismas imágenes.
3. SAM segmentation: dar un punto sobre un objeto en una imagen, obtener máscara. Probar con cajas.
4. mAP a mano: dado un set de predicciones y GT, implementar IoU y calcular AP@0.5 manualmente.
5. YOLO fine-tune: dataset propio (≥ 100 imágenes anotadas), `model.train(data='dataset.yaml', epochs=50)`. Reportar mAP.

Homework verificable

Detector custom de 2-3 clases con YOLOv11:

1. Etiquetar ~100 imágenes con CVAT, LabelImg o Roboflow → YOLO format.
2. dataset.yaml con paths + clases.
3. model.train(data='dataset.yaml', epochs=100, imgsz=640).
4. Reportar mAP@0.5 en val + 3 imágenes con bounding boxes superpuestos.

Criterio de aceptación: mAP@0.5 $\geq$ 0.7 sobre val set; el modelo detecta correctamente las clases en las 3 imágenes de inspección visual.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
YOLO devuelve cajas raras tras fine-tuning	Pocos datos o etiquetado inconsistente. Fi
imgsz=320 para mejor velocidad pierde much	Trade-off speed/accuracy. Fix: 640 default
SAM lento sin GPU	El modelo ViT-H tiene 636M params. Fix: vi
DETR lento en training	DETR converge muy lento (500 epochs en COC
NMS no aplicado → muchas cajas superpuesta	YOLO lo aplica internamente (conf=0.25, io

Preguntas frecuentes

¿YOLO o DETR para producción?

YOLO es default industrial (rápido, bien soportado, fácil deploy). DETR es preferido en investigación.

¿SAM puede reemplazar a un segmentador clásico?

Para tareas donde quieras "segmentar lo que el usuario apunta", sí. Para clases específicas sin intervención humana, fine-tunear un segmentador es más eficiente.

¿Open-vocabulary detection?

Detectores que aceptan clases nuevas vía texto sin reentrenar (CLIP-based). OWL-ViT, GroundingDINO. Útil cuando las clases cambian.

¿Anotación manual cuántas imágenes?

Para 2-3 clases simples con YOLO: 100-500 ya da resultados decentes. Para escenarios complejos: miles. Roboflow / CVAT facilitan el workflow.

¿Tracking de objetos en video?

YOLO + tracker (ByteTrack, BoT-SORT) integrados en ultralytics. Para video más complejo, SAM 2.

Referencias

- Géron, cap. 14 — Object Detection y Semantic Segmentation.
- Carion et al. (2020), End-to-End Object Detection with Transformers (DETR), ECCV.
- Kirillov et al. (2023), Segment Anything, ICCV.
- Ravi et al. (2024), SAM 2: Segment Anything in Images and Videos, Meta.
- Ultralytics YOLOv11 docs.
- Segment Anything project.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 133 — Clase 133 — Segment Anything (SAM / SAM 2): foundation model para segmentación

Parte: 2 — Deep Learning · Fuente: Kirillov et al. (2023) + Ravi et al. (2024) SAM 2. Duración estimada: 85 min.

Objetivo

Usar Segment Anything (Meta AI 2023) y SAM 2 (2024) — el foundation model para segmentación: entrenado en 11M imágenes + 1.1B máscaras (SAM 2 agrega video). Segmenta cualquier objeto dado un prompt (punto, caja, máscara, "todo"). Zero-shot — no requiere training para empezar.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Instalar y cargar SAM/SAM 2 con pip install 'git+https://github.com/facebookresearch/segment-anything-2'.
- Aplicar prompts: punto, caja, multi-punto positivo/negativo.
- Generar máscaras en modo "everything" (segmenta cada objeto automáticamente).
- Tracking de máscaras en video con SAM 2 (memoria temporal).
- Combinar SAM con detector (YOLO) para pipeline detection → segmentation.

Temas

- Arquitectura SAM: ViT encoder + prompt encoder + mask decoder.
- Promptable: una sola red, múltiples interfaces.
- SAM 2: adds memory + tracking en video.
- Variants: vit_h (mejor calidad), vit_l, vit_b (más rápido).
- Pipeline detección + segmentación: YOLO/Grounding DINO → boxes → SAM → masks.
- Fine-tuning SAM (rara vez necesario, casos médicos / dominios muy específicos).

Definiciones y características

- Promptable segmentation: input = imagen + prompt (punto/caja/mask) → output = mask.
- Mask decoder: producirá hasta 3 máscaras por prompt (handles ambigüedad).
- SamPredictor: API estándar para una imagen.
- SamAutomaticMaskGenerator: modo "everything" — segmenta toda la imagen automáticamente.
- SAM 2 memory: bank de features previas para tracking en video.

Dataset / recursos

- Imágenes propias o cualquier dataset visual.
- Modelos pretrained: <<https://github.com/facebookresearch/segment-anything-2>>.
- Librerías: segment-anything-2 (PyTorch).

Ejercicios

1. SAM setup: descargar checkpoint vit_h. Cargar con SamPredictor.
2. Punto prompt: predictor.set_image(img); masks, scores, _ = predictor.predict(point_coords=[[x,y]], point_labels=[1]).

- 3. Box prompt: pasar box=[x1,y1,x2,y2]; útil después de YOLO.
- 4. Everything mode: SamAutomaticMaskGenerator(sam).generate(img) → lista de masks.
- 5. SAM 2 video tracking: tomar un punto en frame 0, propagar a través del video.

Homework verificable

Pipeline YOLO + SAM para anotación semi-automática:

- 1. YOLOv11 detecta objects → boxes.
- 2. Para cada box, SAM produce máscara precisa.
- 3. Visualizar overlay sobre imagen.
- 4. Reportar tiempo por imagen.

Criterio de aceptación: las máscaras son visualmente correctas (alineadas a contornos); pipeline corre en GPU < 1s por imagen.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
SAM lento en CPU	Es enorme. Fix: usar vit_b o GPU.
Máscara incluye background	Prompt ambiguo. Fix: agregar puntos negati
Out-of-memory con ViT-H	4-8 GB VRAM necesarios. Fix: vit_l o ViT-B
Mask decoder devuelve 3 masks, ¿cuál uso?	El score más alto suele ser la correcta. F
SAM 2 tracking pierde objeto en video	Re-prompt cada N frames.

Preguntas frecuentes

¿SAM reemplaza segmentación supervisada?

Para uso interactivo / anotación, sí. Para producción con clases específicas sin intervención humana, fine-tune un segmentador clásico (DeepLabV3+, YOLO-seg) es más eficiente.

¿Open vocabulary?

SAM solo: NO. Para "segmenta el perro" con texto: combinar con Grounding DINO (detector text-grounded) → caja → SAM → mask. O usar Grounded-SAM end-to-end.

¿SAM en mobile?

Hay versiones distilled: MobileSAM, EfficientSAM — 10× más rápidas, ligera pérdida de calidad.

¿Fine-tune SAM?

Posible pero rara vez vale la pena. Casos médicos / dominios muy distintos.

¿Licencia?

Apache 2.0. Comercialmente OK.

Referencias

- Kirillov et al. (2023), Segment Anything, ICCV.
- Ravi et al. (2024), SAM 2: Segment Anything in Images and Videos.
- Segment Anything project.
- Grounded-SAM.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 134 — Clase 134 — YOLOv11 práctico: detección, segmentación, pose, tracking

Parte: 2 — Deep Learning · Fuente: Ultralytics YOLOv11 docs. Duración estimada: 85 min.

Objetivo

Dominar YOLOv11 (Ultralytics, 2024) — el detector default industrial 2026: inference real-time, fine-tuning sencillo, export a ONNX/TensorRT/CoreML/TFLite con un kwarg. Cubrir las 4 tareas: detection, segmentation, pose estimation, oriented bounding boxes (OBB).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Inferir con un modelo COCO-pretrained: `YOLO('yolo11n.pt')(img)`.
- Fine-tunear con dataset propio en formato YOLO (txt por imagen).
- Aplicar las 4 tareas: detection (`yolo11n.pt`), segmentation (`yolo11n-seg.pt`), pose (`yolo11n-pose.pt`), OBB (`yolo11n-obbb.pt`).
- Tracking de objetos en video con ByteTrack / BoT-SORT integrado.
- Exportar a ONNX/TensorRT para deploy.

Temas

- Modelos: n (nano), s, m, l, x. Trade-off speed vs accuracy.
- Formato YOLO de anotación: `class cx cy w h` (normalizado).
- `dataset.yaml`: paths + nombres de clases.
- Fine-tuning: `model.train(data='dataset.yaml', epochs=100, imgsz=640)`.
- Modes: predict, val, train, export, benchmark.
- Tracking integrado.

Definiciones y características

- YOLO: clase Ultralytics que carga cualquier variante.
- `results[0]`: objeto con boxes, masks (seg), keypoints (pose), probs.
- OBB (Oriented Bounding Box): caja rotada — útil para aerial, documents.
- `model.export(format='onnx')`: 1 línea para ONNX/TensorRT/CoreML/TFLite.
- `mAP@0.5`: métrica standard, threshold IoU 0.5.

Dataset / recursos

- COCO pre-trained para inference.
- Roboflow Universe para dataset propio.
- Librerías: `ultralytics` (`pip install ultralytics`).

Ejercicios

1. Inference: `model = YOLO('yolo11n.pt');` `results = model('zidane.jpg');` `results[0].show()`.
2. Segmentation: `YOLO('yolo11n-seg.pt')`. Visualizar máscaras.
3. Pose: `YOLO('yolo11n-pose.pt')`. Detectar keypoints en una foto.
4. Fine-tune: dataset propio (50-100 imágenes anotadas). `model.train(data='ds.yaml', epochs=50, imgsz=640)`.

- Tracking: `model.track('video.mp4', tracker='botsort.yaml')`.

Homework verificable

Detector custom de 2 clases:

- Etiquetar ~100 imágenes con Roboflow / LabelImg → formato YOLO.
- `dataset.yaml` correcto.
- Train YOLOv11n por 100 épocas.
- Reportar `mAP@0.5`; visualizar 3 inferencias.
- Exportar a ONNX y verificar que predice igual.

Criterio de aceptación: `mAP@0.5` ≥ 0.7 ; ONNX produce predicciones consistentes.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>dataset.yaml</code> paths mal	Errores de path. Fix: usar paths absolutos
<code>mAP</code> bajo con dataset chico	Pocos datos. Fix: ≥ 100 ejemplos por clase
Inference lento en CPU	YOLO es para GPU. Fix: <code>device='cuda'</code> o exp
Class names mismatched	Orden en yaml debe coincidir con anotacion
Anotaciones con coords absolutas	YOLO espera normalizadas [0,1]. Fix: conve

Preguntas frecuentes

YOLOv11 vs versiones anteriores?

v11 mejor accuracy/latency. v8 sigue siendo válido y muy usado. v5 legacy pero funciona.

Tareas además de detección?

Segmentation, pose estimation (COCO keypoints), classification, OBB. Una sola API.

Producción cómo deploy?

Export → ONNX → ONNX Runtime / TensorRT. O TF Lite para mobile. Soportado nativo.

Licencia?

AGPL para uso open-source. Para uso comercial cerrado, license enterprise.

Augmentation automática?

Sí, mosaic/mixup/etc. Built-in. Tunear con `model.train(augment=True)`.

Referencias

- Ultralytics YOLOv11.
- Redmon et al. (2016), You Only Look Once — paper original.
- Roboflow Universe — datasets pre-anotados.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 135 — Clase 135 — RNNs: neuronas recurrentes, BPTT

Parte: 2 — Deep Learning · Fuente: Géron, cap. 15 § Recurrent Neurons and Layers. Duración estimada: 70 min.

Objetivo

Entender las redes recurrentes (RNN) — la primera arquitectura para secuencias: misma celda aplicada en cada timestep, estado oculto h_t que acumula contexto, y BPTT (Backpropagation Through Time) que entrena estos modelos. Reconocer sus limitaciones (vanishing en secuencias largas) que motivaron LSTM (clase 120) y eventualmente Transformers.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la ecuación $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$.
- Usar `keras.layers.SimpleRNN(units=N, return_sequences=False|True)`.
- Diferenciar `return_sequences=False` (un único output al final) vs `True` (output por timestep).
- Implementar BPTT truncado (longitud máxima de window).
- Reconocer cuándo una RNN simple es suficiente (secuencias cortas, < 20 pasos) y cuándo necesitas LSTM/Transformer.

Temas

- Estado oculto h_t como memoria.
- Unfolding: la RNN como red feedforward muy profunda en el tiempo.
- BPTT: gradientes a través de cada paso temporal.
- Vanishing/exploding en secuencias largas (compounding multiplicativo).
- BPTT truncado.

Definiciones y características

- RNN cell: función $(h_{t-1}, x_t) \rightarrow h_t$.
- `return_sequences=True`: output (batch, T, units) — para apilar otra RNN o aplicar Dense a cada timestep.
- `return_state=True`: devuelve también el estado final (útil para encoder-decoder).
- BPTT: backprop estándar aplicado al grafo unfolded.
- Truncated BPTT: cortar el gradiente cada K pasos para evitar costo + vanishing.

Dataset / recursos

- Serie temporal sintética (sumas, sine).
- Librerías: tensorflow, keras, numpy, matplotlib.

Ejercicios

1. SimpleRNN básico: predecir el siguiente valor de $\sin(t)$. Modelo: `SimpleRNN(20) → Dense(1)`. Entrenar y graficar predicciones.
2. `return_sequences=True`: apilar 2 RNN, primera con `return_sequences=True`, segunda sin. Verificar shapes.
3. Predicción de N pasos adelante: iterar prediciendo, alimentando la predicción anterior como nuevo input.
4. BPTT truncado: con secuencia de 200 pasos, comparar BPTT completo vs truncado a 20.
5. Vanishing demo: usar SimpleRNN con secuencias de 100 pasos. Las primeras observaciones casi no influyen en la última predicción.

Homework verificable

Forecasting con SimpleRNN sobre $\sin(t)$ + ruido:

1. Generar 5 000 pasos de $\sin(0.01 \cdot t) + N(0, 0.05)$.
2. Construir samples de longitud 50 → predecir el paso 51.
3. SimpleRNN [20] + Dense(1).
4. Evaluar MAE en test y graficar predicción vs realidad.

Criterio de aceptación: $MAE < 0.1$; gráfico muestra la predicción siguiendo la sinusoide.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Input shape error en RNN	RNN espera (batch, timesteps, features). F
Olvido return_sequences=True al apilar	Capa intermedia debe devolver toda la secu
Loss explota con secuencia larga	Exploding gradients. Fix: clipnorm=1.0 + L
Predicción autoregresiva diverge	Error acumulado. Fix: usar teacher forcing
mask_zero no soportado en SimpleRNN	Es para Embedding + LSTM/GRU. Fix: usar LS

Preguntas frecuentes

¿RNN vs LSTM vs Transformer en 2026?

Para secuencias cortas y simples: SimpleRNN o LSTM. Para >50 pasos o NLP serio: Transformer. SimpleRNN ya casi no se usa en producción salvo casos muy específicos.

¿return_state para qué?

Para encoder-decoder: el estado final del encoder inicializa el decoder.

¿Bidireccional?

Bidirectional(SimpleRNN(...)) corre la RNN forward y backward, concatenando. Mejor para tareas non-causales (clasificación, NER).

¿Cuántas units?

Empezá con 32-128. Más complejidad de secuencia → más units. RNNs no son tan parameter-hungry como Transformers.

¿Predecir 1 paso o varios?

Modelos de "1 paso" suelen ser más precisos. Para "varios", entrenar con return_sequences=True y target shifted, o seq2seq (clase 124).

Referencias

- Géron, cap. 15 — Recurrent Neural Networks.
- Pascanu et al. (2013), On the difficulty of training RNNs.
- Keras RNN guide.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 136 — Clase 136 — Forecasting de series con RNN

Parte: 2 — Deep Learning · Fuente: Géron, cap. 15 § Forecasting a Time Series. Duración estimada: 80 min.

Objetivo

Aplicar RNN/LSTM/GRU a un problema real de forecasting de series temporales. Hacer split temporal (NO aleatorio), preparar windows, comparar contra baselines (naive, MA, ARIMA), y reportar métricas estándar (MAE, MAPE, RMSE).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Hacer split temporal (train: período antiguo, val/test: período más reciente).
- Construir samples de longitud T con `tf.keras.utils.timeseries_dataset_from_array`.
- Comparar contra el baseline naïve ($\hat{y}_t = y_{t-1}$) y media móvil.
- Reportar MAE, MAPE, RMSE.
- Reconocer cuándo Deep Learning es mejor que ARIMA / Prophet / XGBoost para series.

Temas

- Split temporal estricto (no shuffle).
- Stationarity / diferenciación.
- Baseline naïve y por qué siempre comparar contra él.
- Windowing: cómo elegir tamaño de window (T) y horizonte.
- Forecasting multi-step: directo vs recursivo vs seq2seq.

Definiciones y características

- Window / lookback: cuántos pasos pasados usás para predecir.
- Horizon: cuántos pasos hacia el futuro predecís.
- Naïve forecast: predicción = último valor observado.
- MAPE: $\text{mean}(|y - \hat{y}| / |y|) \times 100$ — error porcentual.
- `timeseries_dataset_from_array`: helper Keras para crear ds de windows automáticamente.

Dataset / recursos

- `seaborn flights` dataset o cualquier serie pública (e.g., consumo eléctrico).
- `tfds.load('electricity_load_diagrams')` o sintético.
- Librerías: tensorflow, keras, pandas, matplotlib.

Ejercicios

1. Split temporal: separar primer 70 % train, 15 % val, último 15 % test. No mezclar.
2. Baseline naïve: `y_pred = y_test.shift(1)`. Reportar MAE.
3. LSTM forecasting: `Sequential([LSTM(32), Dense(1)])`. Comparar contra naïve.
4. GRU: igual con GRU. Comparar performance y velocidad.
5. Multi-step: predecir 7 pasos directamente (`Dense(7)` al final). Comparar contra predicción recursiva.

Homework verificable

Forecasting de consumo eléctrico:

1. Dataset diario, 5 años; split 70/15/15 temporal.
2. Baselines: naïve y MA(7).

3. LSTM [64], lookback=30 días, horizon=7 días.
4. Reportar MAE en test para los 3.

Criterio de aceptación: LSTM debe superar a naive en MAE; comparable o mejor que MA(7). Documentar dónde gana y dónde pierde.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
train_test_split con shuffle por costumbre	Filtración de futuro al pasado. Fix: split
LSTM mejor que naïve por 0.001 → declarar	Sin significancia clara. Fix: usar pingou
Normalizar usando stats de todo el dataset	Leakage. Fix: fitear solo en train.
Predicción multi-step recursiva diverge	Error acumulado. Fix: entrenar con teacher
Reportar MAPE cuando hay valores cerca de	MAPE explota. Fix: usar SMAPE o reportar M

Preguntas frecuentes

¿DL siempre supera a ARIMA?

No. Para series cortas, regulares, sin features exógenas: ARIMA / ETS suelen igualar o ganar. DL gana con series largas + features adicionales + complejidad no lineal.

¿Lookback óptimo?

Empezar con 1-2 ciclos (e.g., 14 días si hay estacionalidad semanal). Tunear con val.

¿RNN o Transformer para series?

Transformers de series temporales (TFT, Informer, PatchTST 2023) son state-of-the-art para series largas con muchas features. Para series chicas, LSTM/GRU + features manuales gana.

¿Multi-step directo o recursivo?

Directo: 1 modelo predice todos los horizontes. Recursivo: feedback de cada paso. Directo más simple y a veces mejor en horizonte largo.

¿Cómo manejo estacionalidad?

Features explícitas (mes, día de semana, hora) ayudan mucho. Diferenciar la serie también.

Referencias

- Géron, cap. 15 — Forecasting a Time Series.
- Hyndman & Athanasopoulos (2021), Forecasting: Principles and Practice (libro gratuito).
- Lim et al. (2021), Temporal Fusion Transformers, IJF.
- Nie et al. (2023), PatchTST, ICLR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 137 — Clase 137 — LSTM, GRU

Parte: 2 — Deep Learning · Fuente: Géron, cap. 15 § Tackling Short-Term Memory Problems. Duración estimada: 70 min.

Objetivo

Entender LSTM (Hochreiter & Schmidhuber 1997) y GRU (Cho et al. 2014) — celdas recurrentes con gates que solucionan el vanishing gradient de SimpleRNN: la información puede fluir sin atenuación por la cell state, y los gates aprenden qué olvidar, recordar y emitir.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar los 3 gates de LSTM: forget, input, output, y la cell state que viaja "horizontal".
- Diferenciar LSTM (3 gates + 2 estados) de GRU (2 gates + 1 estado, más simple, casi igual de bueno).
- Usar `keras.layers.LSTM` y `keras.layers.GRU` con `return_sequences`, `return_state`, `recurrent_dropout`.
- Aplicar Bidirectional cuando la tarea lo permite.
- Reconocer que con $T > 100$, incluso LSTM lucha — preferir Transformers.

Temas

- Cell state c_t (memoria larga) vs hidden state h_t (memoria corta).
- Forget gate: cuánto borrar de c_{t-1} .
- Input gate: cuánto agregar nuevo a c_t .
- Output gate: cuánto exponer en h_t .
- GRU: combina forget e input en una sola "update gate".
- Bidirectional + stacked LSTMs como receta clásica pre-Transformers.

Definiciones y características

- LSTM cell: 3 gates con sigmoid (0-1) que controlan flujo, + tanh para candidatos.
- GRU cell: update + reset gate. Menos parámetros que LSTM.
- `recurrent_dropout`: dropout aplicado a las conexiones recurrentes (entre timesteps).
- CuDNN kernel: LSTM/GRU tienen implementación cuDNN ultra rápida si cumplís restricciones (default activation tanh, recurrent activation sigmoid, no `recurrent_dropout`).
- Stacked LSTM: apilar varios → mejor capacidad pero más caro.

Dataset / recursos

- IMDB para sentimiento.
- Serie temporal del ejercicio anterior.
- Librerías: tensorflow, keras.

Ejercicios

1. LSTM vs SimpleRNN: en una serie de 100 pasos con dependencia long-range, comparar accuracy.
2. GRU vs LSTM: misma tarea, comparar. GRU ~ 25 % menos params; accuracy casi igual.
3. Stacked: 2-3 capas LSTM apiladas con `return_sequences=True` en las primeras.
4. Bidirectional: para sentimiento IMDB, comparar LSTM vs Bidirectional(LSTM).
5. cuDNN check: medir velocidad LSTM vanilla vs con `recurrent_dropout` (desactiva cuDNN).

Homework verificable

Clasificación de sentimiento sobre IMDB con LSTM:

1. Tokenizar con `TextVectorization(max_tokens=20_000, output_sequence_length=200)`.
2. `Embedding(20_000, 128) → Bidirectional(LSTM(64)) → Dense(64) → Dense(1, sigmoid)`.
3. Entrenar 5 épocas.
4. Reportar accuracy y comparar contra una baseline Dense sin RNN.

Criterio de aceptación: accuracy ≥ 0.85 ; el modelo recurrente claramente supera al baseline Dense.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
LSTM lento en GPU	Tal vez recurrent_dropout > 0 o unroll=Tru
Padding/masking ignorado	mask_zero=True en Embedding + LSTM que res
Bidirectional rompe en task causal (foreca	Bidirectional ve el futuro. Fix: solo Forw
Stacked LSTM overfittea	Demasiado expresivo. Fix: dropout entre ca
GRU vs LSTM comparados sin suficientes see	Diferencia chica puede ser ruido. Fix: ≥ 3

Preguntas frecuentes

¿LSTM o GRU?

Empezá con LSTM (más conocido). GRU si necesitás eficiencia. La diferencia de accuracy es típicamente < 0.5 pp.

¿LSTM aún se usa en 2026?

Sí, para forecasting de series temporales chicas, sistemas embedded, NER simple. Para NLP serio → Transformers.

¿Cuántas capas?

1-3. Más de 3 raramente justifica el costo. Para tareas serias, mejor un Transformer.

¿recurrent_dropout vs dropout?

dropout se aplica a inputs (entre capas); recurrent_dropout entre timesteps (intra-secuencia). El segundo desactiva cuDNN.

¿Cómo manejo secuencias de longitud variable?

Embedding(..., mask_zero=True) + padding con 0s. LSTM/GRU respetan la máscara automáticamente.

Referencias

- Géron, cap. 15 — Tackling Short-Term Memory Problems.
- Hochreiter & Schmidhuber (1997), Long Short-Term Memory, Neural Computation.
- Cho et al. (2014), Learning Phrase Representations using RNN Encoder–Decoder (GRU), EMNLP.
- Chung et al. (2014), Empirical Evaluation of Gated Recurrent Neural Networks.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 138 — Clase 138 — 1D CNNs y WaveNet

Parte: 2 — Deep Learning · Fuente: Géron, cap. 15 § Handling Long Sequences + WaveNet paper (van den Oord et al. 2016). Duración estimada: 60 min.

Objetivo

Conocer la alternativa a RNN para secuencias: Conv1D y WaveNet (dilated causal convolutions). Más rápido

que LSTM (paralelizable), receptive field amplio con pocas capas (dilated convolutions). Útil para audio, series temporales y como capa de preprocesamiento.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar Conv1D(filters, kernel_size, padding='causal') sobre secuencias.
- Implementar dilated convolutions (kernel salta posiciones).
- Reconocer causal convolution: el output en t depende solo de inputs $\leq t$.
- Construir un mini-WaveNet con dilation rates exponenciales (1, 2, 4, 8, ...).
- Comparar Conv1D vs LSTM en speed/accuracy.

Temas

- Conv1D fundamentos: stride, padding, dilation.
- Causal padding: para no ver el futuro.
- Dilated convolutions: receptive field grande sin más layers.
- WaveNet: stack de dilated causal convolutions (1, 2, 4, ..., 512).
- Conv1D vs LSTM: paralelización, receptive field, parameter count.

Definiciones y características

- Conv1D: convolución sobre una sola dimensión espacial/temporal.
- padding='causal': padding asimétrico de modo que output[t] depende solo de input[0..t].
- dilation_rate: gap entre los elementos del kernel. dilation=2 con kernel=3 $\rightarrow$ ve t, t-2, t-4.
- Receptive field: rango temporal que afecta el output. Con dilated stack 1,2,4,8,16: $2^N + 1$.
- WaveNet: red de Conv1D dilated causal stackeadas, originalmente para generación de audio.

Dataset / recursos

- Serie temporal del ejercicio 119.
- Audio simple (sintético: superposición de senos).
- Librerías: tensorflow, keras.

Ejercicios

1. Conv1D vs LSTM: forecasting de serie. Conv1D(32, 5, padding='causal') $\rightarrow$ Conv1D(32, 5, padding='causal') $\rightarrow$ Flatten $\rightarrow$ Dense(1). Comparar con LSTM equivalente.
2. Dilation rates: stack de 4 Conv1D con dilation_rate {1, 2, 4, 8}. Calcular receptive field.
3. Speed test: medir tiempo de training Conv1D vs LSTM para misma data. Conv1D suele ser 5-20x más rápido en GPU.
4. WaveNet mini: implementar stack de 10 Conv1D causal con dilations {1, 2, 4, ..., 512} para una serie larga.
5. Visualización del receptive field: para un output [t], marcar qué inputs lo afectan.

Homework verificable

Forecasting del dataset eléctrico (clase 119) con mini-WaveNet:

1. Stack de 6 Conv1D causal dilated con rates 1, 2, 4, 8, 16, 32.
2. Cada capa con 32 filtros, kernel 2.
3. Comparar MAE en test vs LSTM del ejercicio 119.

Criterio de aceptación: el WaveNet debe ser competitivo o mejor que LSTM, y notablemente más rápido en GPU.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
padding='same' en forecasting causal	Ve el futuro. Fix: padding='causal'.
Stack sin dilation → receptive field chico	Necesitas muchas capas. Fix: dilations exp
Conv1D + Flatten → Dense con muchos params	Modelo grande. Fix: GlobalAvgPool1D antes
Mezclar Conv1D no causal y causal	Inconsistente. Fix: si forecasting, todo c
Audio en [-1, 1] con Conv1D sin normalizar	Funcionar funciona, pero converge mejor co

Preguntas frecuentes

¿Conv1D supera a Transformer en algún caso?

Sí, en audio raw y series largas (>1000 pasos) con patrones locales. Transformer atención $O(N^2)$ en memoria.

¿WaveNet sigue siendo state-of-the-art?

En generación de audio, fue reemplazada por modelos de difusión (clase 133) y autoregresivos sobre tokens de audio (EnCodec + Llama-style).

¿Combinar Conv1D + LSTM/Transformer?

Sí. Patrón "convs como tokenizer" + Transformer arriba es estándar en audio (Wav2Vec, Whisper).

¿dilation_rate y kernel_size cómo se combinan?

receptive_field = (kernel_size - 1) * dilation_rate + 1 para una sola capa. Stack: suma de eso por capa.

¿Por qué Conv1D paralelizable?

Todas las posiciones se computan en paralelo (no hay dependencia temporal explícita como RNN). Ideal para GPU.

Referencias

- Géron, cap. 15 — Using 1D Convolutional Layers + WaveNet.
- van den Oord et al. (2016), WaveNet.
- Bai et al. (2018), An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 139 — Clase 139 — Generación de texto char-RNN

Parte: 2 — Deep Learning · Fuente: Géron, cap. 16 § Generating Shakespearean Text Using a Character RNN. Duración estimada: 70 min.

Objetivo

Construir un modelo de lenguaje autoregresivo a nivel carácter — el ejercicio canónico de Karpathy 2015 sobre Shakespeare. Entender next-token prediction como tarea de pre-training (la base de todo LLM

moderno), sampling con temperatura, y por qué char-RNN fue importante históricamente aunque hoy se hace con tokens BPE y Transformers.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un vocabulario de caracteres (stoi, itos dicts).
- Generar samples (window, target) donde el target es el siguiente carácter.
- Entrenar un modelo Embedding → LSTM → Dense(vocab_size) con cross-entropy.
- Implementar sampling autoregresivo: softmax → multinomial → next char → feed back.
- Aplicar temperatura: logits / T antes de softmax — bajo T = más determinista, alto T = más random.

Temas

- Tokenización a nivel carácter vs word vs BPE.
- Stateful vs stateless RNN: stateful=True para mantener h entre batches.
- Loss: cross-entropy sobre vocab_size.
- Sampling: greedy vs categorical multinomial vs top-k vs nucleus.
- Temperatura como control de creatividad.

Definiciones y características

- Next-token prediction: predecir x_{t+1} dado $x_1, \dots, x_t$. Self-supervised.
- Vocabulario: lista única de caracteres del corpus.
- Stateful RNN: hidden state persiste entre batches consecutivos. Útil para corpus largo.
- Sampling temperatura: $p_i = \text{softmax}(\text{logits} / T)$. $T < 1$ más confiado; $T > 1$ más diverso.
- Top-k sampling: solo considerar los k tokens más probables. Default moderno con k=40-50.

Dataset / recursos

- Tiny Shakespeare (~1 MB de obras):
<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>
- Librerías: tensorflow, keras, numpy.

Ejercicios

1. Vocab + encoding: tokenizar el texto a ints. Reportar vocab_size.
2. Modelo: Embedding(vocab_size, 64) → GRU(128, return_sequences=True) → Dense(vocab_size).
3. Train: pasar batches de longitud 100; loss = sparse_categorical_crossentropy.
4. Sample: implementar función que genera N caracteres con T=1.0, T=0.5, T=1.5. Comparar outputs.
5. Top-k: implementar np.argpartition(logits, -k) antes de muestrear.

Homework verificable

Generar Shakespeare:

1. Entrenar Embedding(64) → GRU(256, return_sequences=True, stateful=True) → Dense(vocab_size) por 10 épocas.
2. Generar 1000 caracteres comenzando con "ROMEO:" a temperaturas 0.3, 0.7, 1.2.
3. Reportar val_loss y muestras de output.

Criterio de aceptación: val_loss < 1.5 (vs ~3.5 random); las muestras a T=0.7 tienen palabras reconocibles y estructura de obra teatral.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Output es gibberish total	Modelo no entrena (loss alta) o T demasiad
Output es siempre la misma frase	T = 0 o muy bajo. Fix: T = 0.5 - 1.0.
OOM al generar 10 000 chars	Stateful + sin trincar window. Fix: usar w
argmax greedy → loops ("the the the the")	Greedy sin diversidad. Fix: sampling con t
Vocabulario incluye chars raros que aparec	Aumenta vocab innecesariamente. Fix: filtr

Preguntas frecuentes

¿char-RNN vs word-RNN vs BPE?

Char es simple, no tiene problema de OOV pero los outputs son largos. Word es eficiente pero pierde con palabras nuevas. BPE / SentencePiece (clase 127) es el estándar moderno.

¿Esto es un "LLM"?

Conceptualmente sí: next-token prediction. La diferencia con GPT es escala (parámetros, datos) y arquitectura (Transformer vs RNN). Mismo objetivo.

¿Sampling estrategias avanzadas?

Top-k, nucleus / top-p (Holtzman et al. 2020), beam search. Default moderno: top-p con p=0.9 + temperatura.

¿Cómo evaluar generación?

Perplexity (exp(loss)) automática. Para calidad subjetiva, evaluadores humanos o métricas como BLEU, ROUGE, BERTScore.

¿Char-RNN sirve para algo en 2026?

Sí: NER muy específico, dominios con vocabulario muy chico, o como baseline. Para generación de texto serio → Transformer + BPE.

Referencias

- Géron, cap. 16 — Generating Shakespearean Text.
- Karpathy (2015), The Unreasonable Effectiveness of Recurrent Neural Networks (blog).
- Holtzman et al. (2020), The Curious Case of Neural Text Degeneration (nucleus sampling), ICLR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 140 — Clase 140 — Análisis de sentimiento

Parte: 2 — Deep Learning · Fuente: Géron, cap. 16 § Sentiment Analysis. Duración estimada: 65 min.

Objetivo

Aplicar un modelo de clasificación de texto sobre IMDB reviews — la tarea NLP más clásica para benchmarks. Pipeline completo: TextVectorization → Embedding → arquitectura (Dense / CNN / RNN / Transformer) → Dense(1, sigmoid). Comparar el zoo de approaches y reconocer que con Hugging Face hoy se hace en 3 líneas (clase 127).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Tokenizar y vectorizar texto con `TextVectorization(max_tokens=20_000, output_sequence_length=200)`.
- Aplicar `Embedding(vocab_size, dim)` y entender que es una lookup table aprendible.
- Construir 4 arquitecturas: bag-of-embeddings (sin orden), `Conv1D`, `LSTM`, `Bidirectional LSTM`.
- Comparar accuracy de las 4 vs un baseline `TfidfVectorizer + LogisticRegression`.
- Usar `Embedding(..., mask_zero=True)` para manejar padding correctamente.

Temas

- `TextVectorization` moderno (Keras 3+).
- `Embedding`: lookup table inicializada random y entrenable.
- `BagOfEmbeddings` (mean pooling) como baseline DL.
- `Conv1D` para texto: capta n-gramas.
- `LSTM + Bidirectional`: captura contexto largo y bidireccional.
- Pre-trained embeddings (GloVe, Word2Vec) — históricamente importantes; hoy reemplazados por embeddings de transformers.

Definiciones y características

- `Embedding`: matriz (`vocab_size`, `embed_dim`). La fila `i` es la representación aprendida del token `i`.
- `Pooling`: tras `Embedding` tenés (`batch`, `T`, `dim`); pooling reduce a (`batch`, `dim`). Mean, max, attention.
- `Masking`: ignorar posiciones con padding (0 por convención).
- `Bidirectional`: corre `LSTM` forward + backward y concatena.

Dataset / recursos

- `keras.datasets.imdb.load_data()` o `tfds.load('imdb_reviews')`.
- Librerías: tensorflow, keras.

Ejercicios

1. Baseline ML clásico: `TfidfVectorizer + LogisticRegression`. Accuracy de referencia (~0.88).
2. Bag-of-embeddings: `Embedding` → `GlobalAveragePooling1D` → `Dense(1, sigmoid)`. Reportar accuracy.
3. `Conv1D`: `Embedding` → `Conv1D(64, 5)` → `GlobalMaxPool1D` → `Dense(1, sigmoid)`.
4. `Bidirectional LSTM`: `Embedding` → `Bidirectional(LSTM(64))` → `Dense(1, sigmoid)`.
5. Pre-trained: cargar GloVe 100d y inicializar la matriz de `Embedding` con ellos. Comparar accuracy contra inicialización random.

Homework verificable

IMDB sentiment classifier con 3 arquitecturas:

1. Pipeline `TextVectorization(20_000, 200)`.
2. Tres modelos: `Conv1D`, `Bidirectional LSTM`, `BagOfEmbeddings`.
3. Reportar accuracy en test para cada uno.
4. Comparar contra `TFIDF + LogReg`.

Criterio de aceptación: al menos uno de los modelos DL debe ≥ 0.88 . `Bidirectional LSTM` suele ganar pero por margen pequeño vs `Conv1D`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>output_sequence_length</code> muy chico	Trunca reviews largos. Fix: 200-500 para <code>r</code>

Olvido mask_zero=True en Embedding	LSTM/GRU aún ven el padding como token vál
Sin pooling antes del Dense → shapes incom	Fix: GlobalAveragePooling1D().
GloVe sin escalado de dimensiones	Embeddings vienen pre-trained con norma es
Reviews en otros idiomas con tokenizer ent	Mal performance. Fix: usar un tokenizer mu

Preguntas frecuentes

¿Sentimiento en 2026: TextVectorization o HF?

Para producción serio: HuggingFace + distilbert-base-uncased-finetuned-sst-2-english (94 % accuracy, 5 líneas de código). Esta clase es pedagógica.

¿Embeddings pre-trained todavía importan?

Para tokens "raw" no — todo modelo moderno tiene su propio embedding integrado. Para visualizar relaciones semánticas (analogías), sí.

¿Cuántos tokens y cuánta longitud?

IMDB: max_tokens=20_000 cubre vocab; output_sequence_length=200 cubre el 90 % de reviews.

¿Bidirectional siempre mejor?

Para clasificación (non-causal), sí. Para generación o real-time, NO (necesitarías ver el futuro).

¿Sentimiento multi-clase (estrellas 1-5)?

Cambiar última capa a Dense(5, softmax) + sparse_categorical_crossentropy. Para sentiment ordinal, considerar ordinal regression.

Referencias

- Géron, cap. 16 — Sentiment Analysis.
- Maas et al. (2011), Learning Word Vectors for Sentiment Analysis — IMDB dataset.
- Pennington et al. (2014), GloVe, EMNLP.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 141 — Clase 141 — Encoder-Decoder para traducción

Parte: 2 — Deep Learning · Fuente: Géron, cap. 16 § Encoder-Decoder Network for Neural Machine Translation. Duración estimada: 80 min.

Objetivo

Implementar la arquitectura seq2seq (Sutskever, Vinyals & Le 2014) — un encoder que comprime la oración fuente en un vector de contexto + un decoder que genera la oración destino token por token. Conocer teacher forcing (durante training, el decoder ve los targets reales como input) vs inference autoregresiva. Esta arquitectura es la antesala de atención (clase 125) y de Transformers (clase 126).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un seq2seq con dos LSTM: encoder devuelve state, decoder usa ese state como inicialización.
- Aplicar teacher forcing: en training, decoder input = target shifted; en inference, autoregresivo.
- Tokens especiales: <start>, <end>, <pad>, <unk>.
- Reconocer la limitación del cuello de botella (todo el meaning de la oración fuente en un vector fijo) — motivación para atención.
- Evaluar traducción con BLEU (nltk.translate.bleu_score).

Temas

- Arquitectura clásica seq2seq con 2 LSTM.
- Teacher forcing vs scheduled sampling.
- Inference autoregresiva con generate loop.
- Beam search (mejor que greedy).
- BLEU como métrica.
- Limitación del bottleneck → motivó atención (Bahdanau 2014).

Definiciones y características

- Encoder: LSTM que procesa la fuente, devuelve final_state = (h_T, c_T).
- Decoder: LSTM inicializado con final_state del encoder; genera target.
- Teacher forcing: durante training, decoder recibe el target real (shifted) en lugar de su propia predicción.
- <start> / <end> tokens: marcadores para iniciar/parar generación.
- BLEU: métrica de calidad de traducción basada en n-grams overlapping. Va de 0 a 100.
- Beam search: durante inference, mantiene los k mejores prefijos en lugar de greedy.

Dataset / recursos

- Tatoeba English-Spanish (small): <https://www.manythings.org/anki/>
- Librerías: tensorflow, keras, nltk (para BLEU).

Ejercicios

1. Preparar datos: tokenizar source y target, agregar <start> y <end> al target, padding.
2. Encoder: Embedding → LSTM(256, return_state=True). Mantener state_h, state_c.
3. Decoder en training: Embedding(decoder_input) → LSTM(256, initial_state=encoder_state) → Dense(target_vocab, softmax).
4. Inference loop: feed <start>, predecir, alimentar prediction como next input, hasta <end> o max_len.
5. BLEU: calcular sobre el test set.

Homework verificable

Traducción inglés → español con seq2seq:

1. Dataset Tatoeba (~10k frases).
2. Encoder + decoder LSTM con 256 units.
3. Train 30 épocas con teacher forcing.
4. Inference autoregresiva + BLEU.

Criterio de aceptación: BLEU > 10 en test (modesto pero válido para LSTM básico); muestras de traducción reconocibles aunque imperfectas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
-------------------	-----------------------

Modelo genera <end> inmediatamente	El bias del decoder lleva siempre a <end>.
Greedy inference repite palabras	"the the the". Fix: beam search o repetiti
Inference más lento que esperado	Loop Python en cada token. Fix: usar tf.nn
BLEU = 0	Mismatch en tokens entre prediction y refe
Vocabulario target muy grande	Sample softmax o tied embeddings. Fix: lim

Preguntas frecuentes

¿seq2seq aún se usa?

Conceptualmente sí (Transformer es seq2seq con atención). Como RNN-seq2seq pure: solo histórico o casos muy específicos. Para traducción sería → Transformer + Hugging Face (clase 127).

¿Bottleneck del estado fijo?

Una sola tupla (h, c) para resumir 50 tokens de input es muy pobre. Atención (clase 125) resuelve dando al decoder acceso a TODOS los estados del encoder.

¿Teacher forcing vs scheduled sampling?

Teacher forcing = siempre target real. Funciona, pero hay mismatch entre train y inference. Scheduled sampling alterna entre target y predicción propia → mejor.

¿Cómo manejo vocabulario grande?

BPE / SentencePiece (clase 127). Vocab típico 32k subwords cubre cualquier idioma con OOV manejable.

¿BLEU es buena métrica?

Aceptable para traducción "literal". Falla con paráfrasis. Para evaluación humana o LLM-as-judge, mejores opciones.

Referencias

- Géron, cap. 16 — Encoder-Decoder Network for Neural Machine Translation.
- Sutskever, Vinyals & Le (2014), Sequence to Sequence Learning with Neural Networks, NeurIPS.
- Cho et al. (2014), Learning Phrase Representations using RNN Encoder-Decoder.
- Papineni et al. (2002), BLEU, ACL.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 142 — Clase 142 — Mecanismos de atención

Parte: 2 — Deep Learning · Fuente: Géron, cap. 16 § Attention Mechanisms + Bahdanau et al. (2015), Luong et al. (2015). Duración estimada: 75 min.

Objetivo

Entender la atención — el mecanismo que destrabó NLP moderno. Bahdanau (2015): permite al decoder mirar todos los hidden states del encoder, ponderando dinámicamente. Luego self-attention (Vaswani 2017): tokens dentro de la misma secuencia se atienden entre sí → Transformer (clase 126).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar scaled dot-product attention: $\text{softmax}(QK^T / \sqrt{d}) V$.
- Diferenciar cross-attention (decoder→encoder, clásico Bahdanau) de self-attention (intra-secuencia, base del Transformer).
- Implementar attention a mano en numpy/TF para una secuencia corta.
- Aplicar `keras.layers.Attention` o `keras.layers.MultiHeadAttention`.
- Interpretar attention weights como "qué tokens fuente miró el decoder al generar cada token destino".

Temas

- Motivación: bottleneck del encoder en seq2seq.
- Bahdanau (additive) vs Luong (multiplicative / dot-product).
- Q, K, V: queries, keys, values.
- Scaled dot-product attention.
- Self-attention vs cross-attention.
- Multi-head: paralelizar varias attention heads con subspaces distintos.
- Attention weights visualizables.

Definiciones y características

- Query (Q): "lo que estoy buscando" (e.g., el token del decoder).
- Key (K): "lo que cada elemento ofrece como índice" (e.g., los hidden del encoder).
- Value (V): "el contenido a recuperar" (normalmente igual o relacionado a K).
- Attention weights: $\text{softmax}(QK^T / \sqrt{d})$, matrix (seq_len_q, seq_len_k).
- Scaled: dividir por $\sqrt{d}$ para que el softmax no saturate.
- Multi-head: dividir d en h heads, calcular attention por separado, concatenar.

Dataset / recursos

- Reusar dataset de traducción de 124.
- Librerías: tensorflow, keras.

Ejercicios

1. Attention a mano: Q, K, V random (seq, d); calcular $\text{softmax}(QK^T / \sqrt{d}) V$. Verificar shapes.
2. Visualizar attention map: tras entrenar un seq2seq con atención, plot heatmap de los pesos (target, source) para una traducción.
3. MultiHeadAttention: `mha = MultiHeadAttention(num_heads=8, key_dim=64)`; `output = mha(query, value)`.
4. Self-attention: aplicar `mha(x, x)` (query = key = value). Esto es el bloque de Transformer.
5. Causal mask: para generación autoregresiva, `MultiHeadAttention(..., use_causal_mask=True)`.

Homework verificable

Traducción con atención sobre el dataset de 124:

1. Encoder LSTM con `return_sequences=True`.
2. Decoder con cross-attention sobre todos los outputs del encoder.
3. Train y evaluar BLEU.
4. Visualizar attention map para 2 frases.

Criterio de aceptación: BLEU debe mejorar respecto al seq2seq sin atención (de 124) por ≥ 5 pp; el attention map muestra alineamiento source-target razonable.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Olvido scale / $\sqrt{d}$	Softmax satura con d grande. Fix: usar Mul
Attention sin masking sobre padding	Atiende a tokens <pad>. Fix: attention_mas
Causal mask al revés	Mira el futuro. Fix: use_causal_mask=True
Cross-attention con Q de un tamaño y K/V d	OK si $d_q \neq d_{kv}$, pero MultiHeadAttention
Visualizar attention de un modelo no conve	Mapas ruidosos. Fix: entrenar bien primero

Preguntas frecuentes

¿Self vs cross attention?

Self: $Q = K = V$ (la secuencia se atiende a sí misma). Cross: Q de un origen, K/V de otro (decoder mira encoder).

¿Por qué $\sqrt{d}$ y no d en el scaled?

QK^T tiene varianza $\sim d$. Dividir por $\sqrt{d}$ mantiene varianza estable y softmax no satura.

¿Multi-head qué aporta?

Cada head puede aprender un patrón distinto (one por sintaxis, otro por semántica, ...). Empíricamente importante.

¿Attention $O(N^2)$?

Sí, en memoria y compute. Es el principal limitante para secuencias largas. Flash Attention (clase 126) lo optimiza; Linformer/Performer aproximan.

¿Atención biológica?

Inspiración loose. El mecanismo no es realmente cómo funciona el cerebro, pero la metáfora "prestar atención a partes relevantes" es intuitiva.

Referencias

- Géron, cap. 16 — Attention Mechanisms.
- Bahdanau, Cho & Bengio (2015), Neural Machine Translation by Jointly Learning to Align and Translate, ICLR.
- Luong, Pham & Manning (2015), Effective Approaches to Attention-based Neural Machine Translation, EMNLP.
- Vaswani et al. (2017), Attention Is All You Need, NeurIPS — paper del Transformer.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 143 — Clase 143 — Transformers: arquitectura, BERT, GPT (+ Flash Attention, RoPE, GQA)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 16 § The Transformer Architecture + Vaswani et al. (2017) + papers BERT, GPT, FlashAttention. Duración estimada: 100 min.

Objetivo

Dominar la arquitectura Transformer —encoder, decoder, ambas variantes (BERT encoder-only, GPT

decoder-only, T5 encoder-decoder)— a nivel de poder implementarla a mano. Conocer las mejoras clave 2022-2024 que hacen a los LLMs modernos rápidos y eficientes: Flash Attention v2/v3, RoPE (Rotary Position Embeddings), Grouped-Query Attention (GQA).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Dibujar el block de Transformer: LayerNorm → MultiHeadAttention → +residual → LayerNorm → FFN → +residual.
- Implementar positional encoding sinusoidal o aprendible.
- Reconocer 3 variantes: encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5, Whisper).
- Saber qué hace cada mejora moderna: Flash Attention ($O(N)$ memoria, 2-3× speedup), RoPE (mejor extrapolación a secuencias largas), GQA (compartir KV heads → menos KV cache en inference).
- Cargar y usar un Transformer chico con `keras.layers.MultiHeadAttention` o desde Hugging Face.

Temas

- Block Transformer: LN → MHA → res → LN → FFN → res.
- Positional encoding: por qué se necesita (attention es permutation-invariant) — sin → aprendible → RoPE.
- BERT: encoder-only, MLM + NSP, bidireccional.
- GPT: decoder-only, next-token, causal mask.
- T5 / BART: encoder-decoder, span-corruption / denoising.
- Complemento moderno: Flash Attention, RoPE, GQA, MQA.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 126b — Flash Attention v2/v3, RoPE, GQA: el motor de los LLMs modernos

Definiciones y características

- Transformer block: bloque básico repetido N veces.
- MHA (Multi-Head Attention): H heads paralelos, cada uno con Q, K, V proyectados.
- Positional encoding: información de orden inyectada en embeddings.
- BERT: encoder-only, bidirectional, masked LM pre-training.
- GPT: decoder-only, causal mask, autoregressive pre-training.
- T5: encoder-decoder, span-corruption pre-training.
- Flash Attention: implementación memory-efficient de scaled-dot-product attention.
- RoPE: rotación de Q/K en función de posición.
- GQA: heads de KV compartidos entre grupos de heads de Q.
- KV cache: almacenamiento de K y V de tokens previos para inference autoregresiva eficiente.

Dataset / recursos

- WikiText-2 o similar.
- Modelos preentrenados desde HF (clase 127).
- Librerías: tensorflow, keras, transformers.

Ejercicios

1. Transformer block desde cero: implementar `def transformer_block(x): x = x + mha(LN(x)); x = x +`

`ffn(LN(x)); return x` con MultiHeadAttention y FFN.

2. Positional encoding sinusoidal: implementar la fórmula original de Vaswani; visualizar como heatmap.
3. Mini-GPT: 4 capas Transformer con causal mask. Entrenar en next-token sobre Tiny Shakespeare. Comparar con char-RNN (122).
4. RoPE manual: implementar la rotación. Aplicar y verificar que `attention(Q_rot, K_rot)` da diferencia relativa.
5. HuggingFace check: cargar `bert-base-uncased` y `gpt2`; imprimir `model.config`. Identificar `num_attention_heads`, `hidden_size`, `intermediate_size`.

Homework verificable

Entrenar un mini-GPT desde cero sobre Tiny Shakespeare:

1. 4 capas Transformer decoder con causal mask.
2. `d_model=128`, `n_heads=4`, `vocab=char-level`.
3. Train 20 épocas; generar 1000 caracteres con temperatura 0.7.
4. Comparar `val_loss` y muestras vs char-RNN (clase 122).

Criterio de aceptación: `val_loss < 1.3` (mejor que char-RNN); samples más coherentes con estructura de obra teatral.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Modelo entrena pero outputs son random	Olvido del causal mask en decoder. Fix: us
LayerNorm antes vs después: dudas	"Pre-LN" (antes) entrena mucho mejor que "
Positional encoding aprendible explota	Init mal. Fix: small std (0.02) o usar RoP
Sin gradient checkpointing → OOM en secuen	Fix: <code>tf.recompute_grad</code> o <code>gradient checkpoi</code>
FFN sin expansión	Default es <code>d_ff = 4 * d_model</code> . Fix: respet

Preguntas frecuentes

¿Encoder-only o decoder-only o ambos?

Encoder-only (BERT): clasificación, NER, similaridad. Decoder-only (GPT): generación, LLMs modernos. Encoder-decoder (T5, Whisper): traducción, transcripción.

¿GPT-style "es todo lo que necesitas" hoy?

Para LLMs generales sí. BERT-style sigue siendo eficiente para clasificación.

¿LayerNorm o RMSNorm?

RMSNorm (Zhang & Sennrich 2019) es estándar moderno — más rápida sin pérdida de calidad. Lo usan Llama, Mistral, etc.

¿Por qué RoPE en lugar de aprendible?

(a) Mejor extrapolación a longitudes no vistas, (b) propiedad relativa útil, (c) no agrega parámetros.

¿GQA solo para inference?

Se entrena con GQA desde el principio. Beneficio principal es inference; training también es marginalmente más rápido.

Referencias

- Géron, cap. 16 — The Transformer Architecture.
- Vaswani et al. (2017), Attention Is All You Need, NeurIPS.
- Devlin et al. (2018), BERT, NAACL.
- Radford et al. (2018-2019), GPT, GPT-2, OpenAI.
- Dao et al. (2022, 2023, 2024), FlashAttention v1/v2/v3.
- Su et al. (2021), RoFormer: Enhanced Transformer with Rotary Position Embedding.
- Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 144 — Clase 144 — Flash Attention v2/v3, RoPE, GQA: el motor de los LLMs modernos

Parte: 2 — Deep Learning · Fuente: Dao et al. (2022, 2023, 2024) FlashAttention + Su et al. (2021) RoPE + Ainslie et al. (2023) GQA. Duración estimada: 90 min.

Objetivo

Entender en profundidad las 3 piezas técnicas que hacen que un LLM moderno (Llama 3, Mistral, Qwen, Gemma) sea rápido y memory-efficient: Flash Attention v2/v3 ($O(N)$ memoria + 2-3× speedup), Rotary Position Embeddings (RoPE) (mejor extrapolación), Grouped-Query Attention (GQA) (menos KV cache en inference).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar por qué attention naïve es $O(N^2)$ en memoria y cómo FlashAttention lo reduce a $O(N)$ con online softmax + tiling.
- Implementar RoPE: rotar pares de dimensiones de Q, K por ángulo función de posición.
- Diferenciar MHA, MQA, GQA — y por qué GQA es el compromiso default 2026.
- Aplicar `torch.nn.functional.scaled_dot_product_attention(q, k, v, is_causal=True)` que elige Flash auto.
- Reconocer combinación moderna: RMSNorm + GQA + RoPE + SwiGLU + Flash Attention.

Temas

- Attention cost: matriz $(N, N) \rightarrow 64$ MB por head con $N=8192$, fp16.
- FlashAttention: bloques en SRAM, no materializa la matriz completa.
- v1 (2022), v2 (2023, 2× speedup), v3 (2024, optimizado H100).
- Positional encoding: sinusoidal $\rightarrow$ learnable $\rightarrow$ RoPE.
- RoPE: rotación bidimensional, propiedad relativa.
- MHA / MQA / GQA: trade-off entre calidad y memoria.
- KV cache: por qué crece en inference.

Definiciones y características

- FlashAttention: algoritmo IO-aware. Reformula $\text{softmax}(QK^T)V$ en bloques que caben en SRAM.
- Online softmax: actualización incremental del softmax sin materializar todo.
- RoPE: $q' = R_{\theta} q$ donde R_{θ} rota pares de dims. $\theta_i = 10000^{(-2i/d)}$.
- MHA: $H_q = H_{kv}$ (clásico).
- GQA: $H_{kv} = H_q / G$. G grupos. Llama 2 70B usa $G=8$.

- MQA: $H_{kv} = 1$. Extremo de GQA.

Dataset / recursos

- HuggingFace modelos: Llama 3, Mistral 7B.
- Librerías: flash-attn, torch ≥ 2.0 (SDPA), transformers.

Ejercicios

1. SDPA vs naïve: implementar attention naïve y `F.scaled_dot_product_attention`. Benchmark.
2. RoPE: implementar rotation function, verificar propiedad $\text{attention}(R_{\theta} q, R_{\phi} k) = \text{f}(\theta - \phi)$.
3. GQA Vs MHA: con Llama config (`n_heads=32`, `kv_heads=8`), inspeccionar shapes.
4. KV cache: medir VRAM en inference con secuencia 8192 — comparar MHA vs GQA.
5. FlashAttention v3 en H100: si tenés H100, benchmark vs v2.

Homework verificable

Mini-GPT con piezas modernas:

1. 6-layer Transformer con: RMSNorm, GQA (4 KV heads / 8 Q heads), RoPE, SwiGLU FFN.
2. Train next-token sobre Tiny Shakespeare.
3. Comparar contra mini-GPT clásico (LayerNorm + MHA + Sin PE + GELU FFN).

Criterio de aceptación: mini-GPT moderno entrena más estable + menor memoria; quality comparable.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
flash-attn no instala en CUDA viejo	Requiere CUDA 11.6+, GPU Ampere+. Fix: usa
RoPE con base distinta a 10000	Para extrapolación a contextos largos (32k
MQA da peor calidad	Esperado. Fix: GQA es el compromiso.
KV cache OOM en context largo	Inherente. Fix: GQA + quantization (Q8 KV)
is_causal=True en SDPA solo aplica si tens	Fix: passing attn_mask cuando shapes asimé

Preguntas frecuentes

FlashAttention v2 o v3?

v3 si tenés H100. v2 estable para todo lo demás. SDPA de PyTorch elige el mejor disponible.

RoPE absoluto o relativo?

RoPE codifica posición absoluta pero produce comportamiento relativo en attention. Lo mejor de ambos.

GQA en training?

Sí — entrenar con GQA desde el principio. Llama 2 70B y todos los modernos lo hacen.

Combina con sliding window attention?

Sí — Mistral 7B usa GQA + sliding window. Para contextos infinitos.

¿Y para CV (ViT)?

ViT moderno también usa Flash Attention (timm support). RoPE en algunos (DiT). GQA menos común en CV.

Referencias

- Dao et al. (2022, 2023, 2024), FlashAttention v1/v2/v3.

- Su et al. (2021), RoFormer: Enhanced Transformer with Rotary Position Embedding.
- Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models.
- Touvron et al. (2023), Llama 2.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 145 — Clase 145 — Hugging Face Transformers (uso práctico)

Parte: 2 — Deep Learning · Fuente: HuggingFace docs + Géron, cap. 16 § Pretrained Transformer Models. Duración estimada: 80 min.

Objetivo

Dominar Hugging Face Transformers — la librería estándar de la industria para usar modelos preentrenados (BERT, GPT, T5, Llama, Whisper, ViT...). Aprender el pipeline API (one-liner para 90 % de los casos), el Trainer API (fine-tuning con muy poco código) y los componentes manuales (Tokenizer + Model + DataLoader).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Usar pipeline('sentiment-analysis'), pipeline('summarization'), pipeline('zero-shot-classification'), etc. en 1 línea.
- Cargar manualmente: tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased'), model = AutoModelForSequenceClassification.from_pretrained('...').
- Tokenizar con tokenizer(texts, padding=True, truncation=True, return_tensors='pt').
- Fine-tunear con Trainer sobre un dataset propio.
- Reconocer el Hub (huggingface.co/models) como catálogo de + de 500k modelos.

Temas

- pipeline API: lo más fácil. Tareas: sentiment, NER, QA, summarization, translation, fill-mask, zero-shot, etc.
- AutoTokenizer + AutoModel*: API manual flexible.
- Tokenizers: BPE, WordPiece, SentencePiece, tiktoken.
- Trainer + TrainingArguments: fine-tuning con 20 líneas.
- Hub: descubrir modelos, datasets, spaces.
- datasets library: cargar datasets, preprocesar.

Definiciones y características

- AutoTokenizer: detecta el tokenizer adecuado para el modelo.
- AutoModelFor<Task>: cabezas específicas (SequenceClassification, TokenClassification, CausalLM, Seq2SeqLM).
- Subword tokenization (BPE/WP/SP): tokens son piezas de palabras → vocab fijo (~32-64k), nada de OOV.
- Special tokens: [CLS], [SEP], <s>, </s>, [PAD], <|im_start|>, etc. Dependen del modelo.
- Hub: GitHub para modelos — cualquiera puede subir/descargar.

Dataset / recursos

- HuggingFace Hub: <<https://huggingface.co/models>>.
- Dataset: `load_dataset('imdb')`, `load_dataset('squad')`, etc.
- Librerías: `transformers`, `datasets`, `accelerate`, `evaluate`.

Ejercicios

1. pipeline one-liner: `pipe = pipeline('sentiment-analysis');` `pipe('I loved this movie!')`. Inspeccionar output.
2. Zero-shot classification: `pipeline('zero-shot-classification')(text, candidate_labels=['sports', 'politics', 'tech'])`. Sin training específico.
3. Manual: tokenizar texto con `bert-base-uncased`. Inspeccionar `input_ids`, `attention_mask`, `token_type_ids`.
4. Modelo + forward: `outputs = model(**inputs)`. Inspeccionar `outputs.logits`.
5. Tokenizer especiales: `tokenizer.decode([101, 7592, 102])` → `'[CLS] hello [SEP]'`.

Homework verificable

Fine-tuning de `distilbert-base-uncased` sobre IMDB:

1. `load_dataset('imdb')`.
2. Tokenizar con `tokenizer(texts, truncation=True, padding='max_length', max_length=256)`.
3. `AutoModelForSequenceClassification.from_pretrained(..., num_labels=2)`.
4. `TrainingArguments(output_dir, num_train_epochs=2, per_device_train_batch_size=16, eval_strategy='epoch')`.
5. `Trainer(...).train()`.
6. Reportar accuracy en test.

Criterio de aceptación: accuracy ≥ 0.92 (DistilBERT con 2 épocas alcanza ~ 0.93 en IMDB).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>OSError: Can't load tokenizer</code>	Modelo no existe o sin internet. Fix: veri
<code>RuntimeError: CUDA out of memory</code>	Modelo + batch demasiado grandes. Fix: red
Tokenizer no agrega special tokens automáticamente	Pasar <code>add_special_tokens=True</code> (default). S
Trainer no usa GPU	Default lo detecta; sino <code>TrainingArguments</code>
Fine-tuning de un modelo grande catastroph	LR alto. Fix: <code>learning_rate=2e-5</code> o menor p

Preguntas frecuentes

¿pipeline o Trainer o AutoModel?

pipeline para usar un modelo preentrenado tal cual. AutoModel para custom training loop. Trainer para fine-tuning estándar (95 % de los casos).

¿PyTorch o TF backend?

PyTorch es estándar en HF — más modelos disponibles, mejor soportado. transformers soporta ambos pero los modelos modernos (Llama, Mistral) son solo PyTorch.

¿Tokenizer rápido?

`AutoTokenizer.from_pretrained(..., use_fast=True)` (default). Implementado en Rust, 10-100× más rápido que la versión Python.

¿Cómo descubro qué modelo usar?

<<https://huggingface.co/models>> con filtros por task, language, license. Modelos más descargados suelen ser buena apuesta.

¿Hugging Face Inference API en producción?

Sí, para prototipos. Para producción serio, self-host con transformers o vLLM / TGI (clase 139).

Referencias

- Hugging Face Transformers docs.
- Hugging Face course — el mejor recurso gratuito para empezar.
- Wolf et al. (2020), Transformers: State-of-the-Art Natural Language Processing, EMNLP demo.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 146 — Clase 146 — CLIP, SigLIP: multimodal embeddings (visión + texto)

Parte: 2 — Deep Learning · Fuente: Radford et al. (2021) CLIP + Zhai et al. (2023) SigLIP. Duración estimada: 80 min.

Objetivo

Conocer CLIP (OpenAI 2021) y su evolución SigLIP (Google 2023) — los foundation models que mapean imágenes y texto al mismo espacio vectorial, entrenados con contrastive learning sobre 400M-4B pares. Aplicaciones: zero-shot classification, image search por texto, content moderation, embeddings para RAG multimodal.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Cargar CLIP/SigLIP desde HuggingFace: `CLIPModel.from_pretrained('openai/clip-vit-base-patch32')`.
- Calcular embeddings de imágenes y de texto; cosine similarity entre ambos.
- Implementar zero-shot classification: predecir clase con la mayor similaridad a "a photo of a [class]".
- Hacer image retrieval por texto sobre un corpus de imágenes.
- Diferenciar CLIP (softmax contrastive) de SigLIP (sigmoid pairwise, mejor escalabilidad).

Temas

- Contrastive Language-Image Pre-training: matchear pares correctos, separar incorrectos.
- Arquitectura: image encoder (ViT) + text encoder (Transformer).
- Cosine similarity como métrica.
- Zero-shot vs few-shot.
- Variantes modernas: SigLIP (sigmoid loss), EVA-CLIP, OpenCLIP, Apple AIM.

Definiciones y características

- CLIP: dual encoder, contrastive softmax.
- SigLIP: dual encoder, sigmoid pairwise — escala mejor, no necesita global batch.
- Embedding space: cosine similarity para comparar.
- Zero-shot: clasificar sin training en la tarea, solo con prompt textual.
- Variantes: ViT-B/32 (rápido), ViT-L/14 (mejor), ViT-H/14 (top).

Dataset / recursos

- Imágenes propias o `tfds.load('cats_vs_dogs')`.
- HuggingFace: `openai/clip-vit-base-patch32`, `google/siglip-base-patch16-224`.
- Librerías: `transformers`, `torch`, `PIL`.

Ejercicios

1. CLIP setup: cargar processor + model. Embed una imagen y un texto. Cosine similarity.
2. Zero-shot classification: dada una imagen, comparar contra ["a photo of a cat", "a photo of a dog"]. Predict argmax.
3. Image search: corpus de 100 imágenes; query texto "a sunset over the ocean" → top-5 más similares.
4. SigLIP: misma tarea, comparar accuracy.
5. Fine-tune ligero: con dataset chico custom, fine-tunear CLIP con LoRA para un dominio específico.

Homework verificable

Buscador de imágenes por texto:

1. 200 imágenes diversas.
2. Embed todas con CLIP ViT-B/32.
3. UI simple (notebook): input texto → top-5 imágenes.
4. Probar 10 queries; reportar precisión subjetiva.

Criterio de aceptación: queries claras devuelven imágenes relevantes en top-3 con frecuencia alta.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Similitud baja para imagen y texto que par	Prompt mal estructurado. Fix: "a photo of
Lento en CPU	Modelos grandes. Fix: ViT-B/32 (chico) o G
OOM con ViT-L/14	Fix: ViT-B/32 o batch=1.
Embeddings no normalizados → cosine raro	Fix: normalizar con <code>outputs.image_embeds /</code>
Imágenes en formato wrong	CLIP espera RGB 224×224. Fix: usar el proc

Preguntas frecuentes

CLIP o SigLIP?

SigLIP entrena más rápido y escala mejor; calidad similar. En 2026, SigLIP / SigLIP-2 es default moderno.

¿En LLM multimodal (LLaVA)?

CLIP/SigLIP es el image encoder. La LLM (Llama) recibe los embeddings.

Open vocabulary detection con CLIP?

OWL-ViT, Grounding DINO usan CLIP como base. Hacen detección con prompts texto.

CLIP en producción de búsqueda?

Sí — Pinterest, Adobe, Shopify. Vector DB (Qdrant, Pinecone) con embeddings CLIP.

Train CLIP desde cero?

400M+ pares necesarios. Open-source: OpenCLIP, MetaCLIP — alternativas abiertas con datasets públicos.

Referencias

- Radford et al. (2021), CLIP, ICML.
- Zhai et al. (2023), Sigmoid Loss for Language Image Pre-Training (SigLIP), ICCV.
- OpenCLIP.
- LAION-5B — dataset abierto.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 147 — Clase 147 — Whisper: ASR, transcripción, traducción de audio

Parte: 2 — Deep Learning · Fuente: Radford et al. (2022) Whisper + OpenAI release notes. Duración estimada: 70 min.

Objetivo

Usar Whisper (OpenAI 2022, open-source) — el modelo de ASR (Automatic Speech Recognition) multilinguaje que destronó a Google STT y AWS Transcribe en accuracy. Cubrir transcripción, traducción a inglés, timestamps, word-level timing. Alternativas modernas: Whisper-large-v3, distil-whisper (4× más rápido), insanely-fast-whisper.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Cargar Whisper con transformers (openai/whisper-large-v3) o openai-whisper (lib oficial).
- Transcribir audio en cualquier idioma (99+ soportados).
- Aplicar task='translate' para traducir directo a inglés.
- Obtener timestamps a nivel palabra para subtítulos.
- Usar distil-whisper para inference 4-6× más rápida con calidad similar.

Temas

- Arquitectura: encoder-decoder Transformer + spectrogram input.
- Tamaños: tiny, base, small, medium, large-v3.
- Languages: detectado automático o explícito.
- Tareas: transcribe (en idioma origen), translate (→ inglés).
- Diarization (quién habla): no built-in, requiere pyannote.audio separado.
- Long-form audio: chunking con overlap.

Definiciones y características

- Whisper: encoder-decoder Transformer entrenado en 680k horas multi-lenguaje.
- pipeline('automatic-speech-recognition'): API HF más simple.
- Distil-Whisper: destilación 4× más rápida; calidad casi igual.
- WER (Word Error Rate): métrica estándar ASR.
- Timestamps: por segmento (default) o por word (con flag).

Dataset / recursos

- Cualquier audio: voz, podcasts, llamadas.
- HuggingFace: openai/whisper-large-v3, distil-whisper/distil-large-v3.
- Librerías: transformers, torch, librosa (procesamiento audio).

Ejercicios

1. Transcripción básica: cargar pipeline('asr', model='openai/whisper-base'); pasar un audio.
2. Multilenguaje: audio en español → transcribir; verificar.
3. Traducción: pipe(audio, task='translate') → texto en inglés.
4. Timestamps: pipe(audio, return_timestamps='word') → palabras con start/end seconds.
5. Distil-Whisper: comparar tiempo y WER vs full Whisper.

Homework verificable

Sistema de subtítulos:

1. Audio de 5-10 min (clase grabada, podcast).
2. Whisper-large-v3 con return_timestamps='word'.
3. Generar SRT (formato subtítulos) a partir del output.
4. Verificar manualmente la calidad en 1 minuto random.

Criterio de aceptación: SRT bien formado; timestamps razonables; ≥ 90 % de palabras correctas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Audio en formato no soportado	Whisper espera 16kHz mono. Fix: librosa.io
Lento en CPU	Whisper-large es big. Fix: medium o distil
OOM en GPU	Fix: chunk_length_s=30 para procesar por c
Hallucinations en silencios	Whisper a veces "inventa" texto en silenci
Idioma detectado mal	Fix: language='es' explícito.

Preguntas frecuentes

Whisper vs Google STT / Azure?

Whisper es gratis y open. Calidad comparable o mejor en muchos idiomas. Google/Azure mejor en real-time streaming.

Distil-Whisper cuándo?

Cuando latencia importa o tenés CPU. Calidad ~1-2 WER points peor que full Whisper-large. Worth it.

Diarization (multiple speakers)?

Whisper NO la hace. Combinar con pyannote.audio.

Real-time streaming?

Whisper es batch (audio completo). Para streaming: faster-whisper con VAD, o insanely-fast-whisper (BetterTransformer + Flash Attention).

Hallucination prevention?

condition_on_previous_text=False, temperature alta cuando no hay confianza, VAD para skip silencios.

Referencias

- Radford et al. (2022), Robust Speech Recognition via Large-Scale Weak Supervision, OpenAI.
- Whisper repo.
- Distil-Whisper.

- Insanely Fast Whisper.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 148 — Clase 148 — LLMs aplicados: fine-tuning, prompting (+ LoRA / QLoRA, DPO, vLLM)

Parte: 2 — Deep Learning · Fuente: docs HuggingFace PEFT + TRL + vLLM + papers LoRA (Hu et al. 2021), QLoRA (Dettmers et al. 2023), DPO (Rafailov et al. 2023). Duración estimada: 110 min.

Objetivo

Manejar Large Language Models (Llama 3, Mistral, Qwen, etc.) en flujos reales: prompting técnico (zero-shot, few-shot, chain-of-thought), fine-tuning eficiente con LoRA / QLoRA (entrenar solo 0.1-1 % de parámetros), alineamiento con DPO (preferencias, en lugar de RLHF clásico), e inference de producción con vLLM (continuous batching, PagedAttention).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diseñar prompts con system prompts, few-shot examples y chain-of-thought.
- Aplicar LoRA con peft library: agregar adapters chicos a un LLM grande, entrenar solo ~0.5 % de params.
- Aplicar QLoRA (cuantización 4-bit) para fine-tunear Llama 7B en una sola GPU de 24 GB.
- Alinear con preferencias usando DPO (trl library) — más simple y estable que RLHF.
- Servir un modelo con vLLM y medir throughput.

Temas

- LLM stack en 2026: base → SFT (Supervised Fine-Tuning) → DPO/RLHF → inference optimizada.
- Prompting técnico: structure, few-shot, chain-of-thought, function calling.
- PEFT (Parameter-Efficient Fine-Tuning): LoRA, QLoRA, adapters.
- DPO vs RLHF.
- vLLM: continuous batching, PagedAttention, OpenAI-compatible API.
- Evaluación: MMLU, HumanEval, MT-Bench, LMSys Arena.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 128a — LoRA / QLoRA: fine-tuning eficiente de LLMs
- Clase 128b — DPO y RLHF: alineamiento de LLMs
- Clase 128c — vLLM y TGI: serving de LLMs en producción

Definiciones y características

- Prompting: arte de estructurar el input para que el modelo dé buenos outputs.
- Few-shot: incluir 2-5 ejemplos del task en el prompt.
- Chain-of-thought (CoT): pedirle al modelo que "piense paso a paso" — mejora dramática en tareas razonamiento.
- System prompt: instrucciones generales antes del diálogo (rol, restricciones).

- PEFT: técnica de fine-tuning con pocos parámetros.
- LoRA: rank-r decomposition para los deltas.
- QLoRA: LoRA + 4-bit quantization del base.
- DPO: loss directa sobre preferencias, sin reward model.
- vLLM: framework de inference optimizada open-source.
- KV cache: cache de K, V de tokens previos, crítico para inference autoregresiva.

Dataset / recursos

- HuggingFace Hub para modelos.
- Datasets de instrucción: Alpaca, ShareGPT, OpenOrca.
- Datasets de preferencia: Anthropic HH-RLHF, UltraFeedback.
- Librerías: transformers, peft, trl, bitsandbytes, vllm.

Ejercicios

1. Prompt engineering: para una tarea (e.g., clasificación de quejas), iterar 5 versiones de prompt (zero-shot, few-shot, CoT, etc.). Medir accuracy en un set chico.
2. LoRA SFT: fine-tune un Mistral 7B Instruct con LoRA sobre un dataset propio chico (1k-10k ejemplos).
3. QLoRA: el mismo experimento pero con quantization 4-bit. Comparar memoria y velocidad.
4. DPO: con un dataset de preferencias mínimo (e.g., 500 pairs), aplicar DPO sobre el modelo SFT.
5. vLLM serving: levantar vLLM con el modelo + LoRA adapter, medir throughput vs HF pipeline.

Homework verificable

Fine-tune un LLM open con LoRA para una tarea propia:

1. Elegir Llama 3 8B Instruct o Mistral 7B Instruct.
2. Dataset propio: 500-2000 pares (instrucción, respuesta).
3. QLoRA config: r=16, target=['q_proj','k_proj','v_proj','o_proj'], 4-bit.
4. Entrenar 3 épocas; evaluar perplexity en val.
5. Servir con vLLM y testear 10 ejemplos.

Criterio de aceptación: perplexity en val mejora vs base; el modelo responde apropiadamente al dominio para los 10 ejemplos test.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
RuntimeError: CUDA out of memory con Llama	Sin quantization. Fix: QLoRA con bnb_4bit.
LoRA con target_modules='all-linear' lento	Demasiados adapters. Fix: solo q/v en attn
Modelo SFT sobre dataset chico pierde capa	Catastrophic forgetting parcial. Fix: subi
DPO sin ref_model actualizado	El ref model debe ser SFT init. Fix: usar
vLLM no carga LoRA adapter	Hay que mergear LoRA + base primero o usar

Preguntas frecuentes

¿Open-source o API (GPT-4, Claude)?

API para prototipos, casos non-críticos. Self-host con open-source cuando importan: privacidad, costo a escala, control total. Ambos coexisten.

¿LoRA o full fine-tune?

LoRA en 99 % de los casos. Full fine-tune solo si tenés cluster grande y querés cambiar significativamente la base.

¿DPO o RLHF?

DPO por default — más simple y casi tan bueno. RLHF para tareas muy específicas donde necesitás un reward model fino.

¿vLLM o TGI?

vLLM ligeramente más rápido en benchmarks. TGI (HuggingFace) más integrado con HF Hub. Ambos buenos.

¿Cuánto cuesta entrenar un LLM desde cero?

Llama 3 70B: estimado \$10M-50M en compute. Para custom: forget about it — fine-tunea uno existente.

Referencias

- Hu et al. (2021), LoRA: Low-Rank Adaptation of Large Language Models, ICLR.
- Dettmers et al. (2023), QLoRA: Efficient Finetuning of Quantized LLMs, NeurIPS.
- Rafailov et al. (2023), Direct Preference Optimization, NeurIPS.
- Kwon et al. (2023), Efficient Memory Management for LLM Serving with PagedAttention, SOSP.
- PEFT docs, TRL docs, vLLM docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 149 — Clase 149 — LoRA / QLoRA: fine-tuning eficiente de LLMs

Parte: 2 — Deep Learning · Fuente: Hu et al. (2021) LoRA + Dettmers et al. (2023) QLoRA. Duración estimada: 100 min.

Objetivo

Hacer fine-tuning de LLMs (Llama 3, Mistral, Qwen 2) en una GPU consumer con LoRA y QLoRA. Cubrir hiperparámetros (r, alpha, dropout, target_modules), inspección de trainable params, merge LoRA con base, y deployment.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar LoRA con peft.LoraConfig y get_peft_model.
- Configurar QLoRA con bitsandbytes 4-bit + BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type='nf4').
- Elegir rank r (típico 8-64), alpha (típico 2×r), target_modules.
- Mergear el LoRA adapter con el base para deploy: model.merge_and_unload().
- Calcular trainable params vs total: ~0.1-1 % es el ratio típico.

Temas

- LoRA matemáticamente: $W' = W + (B \cdot A) \cdot \alpha / r$.
- QLoRA: NF4 quantization + double quant + paged optimizers.

- target_modules: ['q_proj', 'k_proj', 'v_proj', 'o_proj'] clásico; o all-linear.
- Save / load: solo el adapter (~10-50 MB) en lugar del full model.
- Merge + serve vs separated adapter.

Definiciones y características

- LoRA: matrices rank-r entrenables agregadas a Linear layers.
- r (rank): 8-16 para tareas chicas; 32-64 para datos grandes/cambios fuertes.
- alpha: scaling. Convención: $\alpha = 2r$.
- NF4: NormalFloat 4-bit, optimizado para weights de redes pre-trained.
- Double quantization: cuantizar los scales también; ahorra ~0.4 bits/param.
- merge_and_unload(): devuelve un modelo "normal" con LoRA aplicado.

Dataset / recursos

- HuggingFace dataset de instrucción (Alpaca, Dolly, OpenAssistant).
- Modelo base: Llama 3 8B Instruct, Mistral 7B Instruct, Qwen 2 7B Instruct.
- Librerías: transformers, peft, bitsandbytes, accelerate, trl.

Ejercicios

1. LoRA básico: cargar Mistral 7B Instruct sin quantization; aplicar LoRA $r=16$. Inspeccionar trainable params (~0.5 %).
2. QLoRA: ahora con load_in_4bit=True. Verificar uso VRAM (~6 GB).
3. Train con TRL: SFTTrainer sobre 500 ejemplos custom. ~30 min en GPU consumer.
4. Inference con adapter: cargar base + LoRA y predecir.
5. Merge: model.merge_and_unload() → save como modelo normal.

Homework verificable

Fine-tune Mistral 7B Instruct con QLoRA para un dominio propio:

1. Dataset: 200-500 pares (instrucción, respuesta).
2. QLoRA $r=16$, target=all-linear, 3 épocas.
3. Inference con el adapter: 10 ejemplos test.
4. Reportar mejora cualitativa vs base.

Criterio de aceptación: respuestas más alineadas al dominio que el base; uso de VRAM < 16 GB durante training.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OOM en Llama 7B + Adam	Sin quantization. Fix: QLoRA + paged_adamw
LoRA target_modules='all-linear' lento	Demasiados adapters. Fix: solo q/v y luego
Trainable params 0	Olvidaste get_peft_model. Fix: aplicarlo a
Resultados peor que base	LR mal, target_modules incompleto, o pocos
merge_and_unload modifica el base	Por design, en place. Fix: si necesitás ba

Preguntas frecuentes

rank r óptimo?

Empieza con 16. Tareas pequeñas: 8. Datos grandes / cambios sustanciales: 32-64.

¿LoRA o QLoRA?

QLoRA si la GPU no aguanta. Sino LoRA, ligeramente mejor calidad final.

Multi-adapter?

Sí — entrenar varios adapters (uno por tarea/idioma) y switchear en inference con `model.set_adapter('name')`.

¿LoRA en visión / difusión?

Sí, mismo concepto. diffusers tiene LoRA support para SDXL fine-tuning.

DoRA, AdaLoRA?

Variantes: DoRA (weight-decomposed) y AdaLoRA (rank adaptativo). Mejoras marginales. LoRA "vanilla" sigue siendo default.

Referencias

- Hu et al. (2021), LoRA, ICLR.
- Dettmers et al. (2023), QLoRA, NeurIPS.
- PEFT docs.
- TRL docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 150 — Clase 150 — DPO y RLHF: alineamiento de LLMs

Parte: 2 — Deep Learning · Fuente: Ouyang et al. (2022) InstructGPT/RLHF + Rafailov et al. (2023) DPO + papers IPO/KTO/ORPO. Duración estimada: 95 min.

Objetivo

Alinear LLMs con preferencias humanas (helpful, harmless, honest). Cubrir RLHF clásico (SFT → Reward Model → PPO, complejo) y DPO (Direct Preference Optimization, moderno y simple). Conocer variantes 2023-2024: IPO, KTO, ORPO.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar el pipeline RLHF de 3 etapas (SFT, RM, PPO).
- Aplicar DPO con `trl.DPOTrainer` sobre un dataset de preferencias.
- Diferenciar DPO (single-step) de KTO (no requiere pairs) y ORPO (combina SFT + alineamiento).
- Crear un dataset de preferencias: (prompt, chosen, rejected).
- Evaluar alineamiento con MT-Bench, AlpacaEval, o LLM-as-judge.

Temas

- Por qué alineamiento: LLM pretrained → genera pero no sigue instrucciones bien ni evita harm.
- SFT (Supervised Fine-Tuning): instruction tuning.
- Reward Model: regression entrenada con human preferences.
- PPO: optimizar el LLM contra el RM.

- DPO: derivación matemática que elimina el RM.
- IPO: variante con identity link, más estable.
- KTO: solo chosen o rejected (no necesita pairs).
- ORPO: alineamiento desde SFT directamente.

Definiciones y características

- SFT: training supervisado con (prompt, response_humana).
- Reward Model: predican $r(\text{prompt}, \text{response})$.
- Bradley-Terry: modelo probabilístico $P(A > B) = \sigma(r(A) - r(B))$.
- DPO loss: $-\log \sigma(\beta \cdot (\log \pi(\text{chosen}) / \pi_{\text{ref}}(\text{chosen}) - \log \pi(\text{rejected}) / \pi_{\text{ref}}(\text{rejected})))$.
- β (DPO): control del KL respecto al ref model. 0.1-0.5 típico.
- Reference model: copia frozen del SFT, usada para regularizar.

Dataset / recursos

- Anthropic HH-RLHF, UltraFeedback, Argilla DPO datasets.
- Modelo base: SFT propio (clase 128a) o Mistral 7B Instruct.
- Librerías: transformers, trl, peft, bitsandbytes.

Ejercicios

1. Dataset de preferencias: cargar Anthropic/hh-rlhf. Inspeccionar chosen y rejected.
2. DPO con TRL: `DPOTrainer(model, ref_model, ...)` con LoRA encima. Train 1 época.
3. Eval pre/post: generar respuestas a 20 prompts antes y después; comparar manualmente.
4. β sensitivity: probar $\beta \in \{0.1, 0.3, 1.0\}$. β alto $\rightarrow$ menos cambio; β bajo $\rightarrow$ más agresivo.
5. KTO: dataset con solo chosen (no pairs). Aplicar `KTOTrainer`.

Homework verificable

DPO sobre un dominio propio:

1. Dataset de 200-500 pares (puede ser sintético con LLM como judge).
2. Base: modelo SFT propio (de 128a).
3. DPO con LoRA, $\beta=0.1$.
4. Comparar 20 respuestas pre/post; evaluar con LLM-as-judge (e.g., Claude).

Criterio de aceptación: $\geq 60\%$ de los outputs post-DPO son juzgados mejores que pre.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
DPO degrada calidad	β muy bajo o demasiadas épocas. Fix: $\beta=0.1$
Reward hacking en RLHF	Modelo aprende a hacer trampas al RM. Fix:
ref_model consume mucha VRAM	Copia frozen del modelo. Fix: usar ref_mod
Dataset de pairs muy ruidoso	Annotators inconsistentes. Fix: filtering,
Resultados malos sin SFT previo	DPO asume modelo razonable. Fix: SFT prime

Preguntas frecuentes

DPO o RLHF clásico?

DPO por default 2024+ — más simple, casi igual calidad. RLHF si tenés team y reward model bueno (e.g., GPT-4 / Claude para producción).

¿IPO, KTO, ORPO cuál?

DPO sigue siendo default. IPO si DPO inestable. KTO si solo tenés chosen. ORPO combina SFT+pref → más eficiente, en alza.

¿Dataset de cuántos pairs?

10k-50k para alineamiento serio. 500-2000 para experimentos.

Evaluación cómo?

LLM-as-judge (GPT-4/Claude evalúa pares), MT-Bench, AlpacaEval. Human eval para gold standard.

¿DPO en LLMs > 70B?

Sí, con FSDP / DeepSpeed. Trabajo "expensive" pero factible.

Referencias

- Ouyang et al. (2022), Training language models to follow instructions with human feedback (InstructGPT), NeurIPS.
- Rafailov et al. (2023), Direct Preference Optimization, NeurIPS.
- Ethayarajh et al. (2024), KTO, NeurIPS.
- Hong et al. (2024), ORPO.
- TRL docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 151 — Clase 151 — vLLM y TGI: serving de LLMs en producción

Parte: 2 — Deep Learning · Fuente: Kwon et al. (2023) vLLM + HuggingFace TGI docs. Duración estimada: 80 min.

Objetivo

Servir LLMs eficientemente en producción con vLLM (Berkeley) o TGI (HuggingFace). Cubrir: PagedAttention, continuous batching, prefill/decode, quantization (AWQ, GPTQ, FP8), structured output (JSON, function calling), streaming, OpenAI-compatible API.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Levantar vLLM serving: `python -m vllm.entrypoints.openai.api_server --model X`.
- Hacer requests con OpenAI client apuntando a vLLM.
- Aplicar continuous batching y entender ganancia de throughput.
- Servir modelos cuantizados (AWQ, GPTQ, FP8) para reducir VRAM.
- Activar structured outputs con `guided_json` (JSON schema enforced).

Temas

- KV cache: por qué crece con secuencia.
- PagedAttention (vLLM): page table como OS → no fragmentación.
- Continuous batching: nuevos requests entran sin esperar al batch.

- Prefill (compute KV) vs decode (1 token/step).
- Speculative decoding: predecir N tokens, verificar con modelo grande.
- Quantization: FP8 (H100), AWQ/GPTQ (4-bit weight-only).

Definiciones y características

- vLLM: framework serving open-source. PyTorch-based. ~5-20× throughput vs HF pipeline.
- TGI (Text Generation Inference): HF Hub-integrado. Similar a vLLM.
- PagedAttention: KV cache paginado.
- AWQ / GPTQ: 4-bit weight-only quantization, mantiene calidad ~98 %.
- FP8: 8-bit float native en H100. Mejor que int8 numéricamente.
- Speculative decoding: 2-3× speedup en decode con modelo draft.

Dataset / recursos

- Modelo: Mistral 7B Instruct, Llama 3 8B Instruct, o uno cuantizado AWQ.
- Librerías: vllm, transformers, openai (client).

Ejercicios

1. vLLM básico: `python -m vllm.entrypoints.openai.api_server --model mistralai/Mistral-7B-Instruct-v0.2`. Cliente OpenAI.
2. Continuous batching benchmark: 100 requests paralelos vs 1 a la vez. Comparar throughput.
3. AWQ quantization: cargar TheBloke/Mistral-7B-Instruct-v0.2-AWQ. VRAM ~5 GB vs 14 GB fp16.
4. Structured JSON output: `extra_body={'guided_json': {schema}}` → forced JSON valid.
5. TGI: `docker run --gpus all ghcr.io/huggingface/text-generation-inference:latest --model-id X`.

Homework verificable

Servir Mistral 7B Instruct AWQ con vLLM + cliente OpenAI:

1. Levantar server vLLM.
2. 50 prompts batch en paralelo desde cliente.
3. Medir throughput (tokens/sec total).
4. Comparar contra transformers.pipeline.generate.
5. Activar guided_json para una tarea específica.

Criterio de aceptación: vLLM $\geq 5\times$ throughput vs HF pipeline; structured output válido al 100 %.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OOM al cargar	Modelo + KV cache no entra. Fix: quantizat
Throughput bajo en una sola request	Con batch=1, vLLM es similar a HF. Fix: pa
LoRA adapter no carga	Fix: vLLM soporta LoRA con <code>--enable-lora</code> -
Structured output formato inválido	guided_json schema mal. Fix: JSON Schema v
TGI lento	Versión vieja. Fix: usar imagen latest.

Preguntas frecuentes

vLLM vs TGI?

vLLM ligeramente más rápido en benchmarks; mejor LoRA support. TGI más integrado con HF Hub. Ambos buenos.

Ollama / llama.cpp?

llama.cpp / Ollama: CPU-friendly, GGUF format. Para desktop, local, low traffic. vLLM/TGI para servers con GPU.

Estructura JSON garantizada?

Sí con guided_json (vLLM uses outlines library). Mucho más confiable que prompt engineering.

Speculative decoding?

--speculative-model X --num-speculative-tokens 5. 2-3× decode speedup con calidad idéntica.

Multi-GPU?

--tensor-parallel-size N para shard del modelo en N GPUs.

Referencias

- Kwon et al. (2023), Efficient Memory Management for LLM Serving with PagedAttention, SOSP.
- vLLM docs.
- TGI docs.
- Outlines (structured output).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 152 — Clase 152 — RAG básico y embeddings (+ hybrid search, re-ranking, MCP)

Parte: 2 — Deep Learning · Fuente: Lewis et al. (2020) RAG paper + LlamaIndex / LangChain docs + Anthropic MCP spec. Duración estimada: 100 min.

Objetivo

Construir un sistema RAG (Retrieval-Augmented Generation) — pipeline que enriquece a un LLM con conocimiento externo: documentos propios, base de datos, web. Pipeline: embedding los docs → almacenar en vector DB → al query, hacer retrieval de los k más relevantes → inyectar como contexto al LLM → respuesta basada en docs. Conocer mejoras modernas: hybrid search (denso + sparse), cross-encoder re-ranking, y el Model Context Protocol (MCP) que estandariza la conexión LLM-herramientas.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Generar embeddings de texto con sentence-transformers o modelos HF.
- Almacenar y buscar embeddings con FAISS, Chroma, Pinecone, Qdrant, o pgvector.
- Construir el flujo query → embed → top-k → context → LLM.
- Aplicar hybrid search: combinar BM25 (sparse) con embeddings (dense) → mejor recall.
- Aplicar cross-encoder re-ranking sobre los top-100 retornados para promover los top-10 reales.
- Reconocer el MCP como protocolo abierto para conectar LLMs a herramientas (filesystems, DBs, APIs).

Temas

- Por qué RAG: LLMs no saben de documentos privados; entrenar fine-tune no escala para conocimiento dinámico.

- Embeddings: vectores de dimensión ~768-1536.
- Vector DBs: FAISS (in-memory), Chroma (local), Qdrant/Weaviate (server), pgvector (Postgres extension).
- Chunking: dividir docs en piezas de ~200-1000 tokens.
- Top-k retrieval + context window.
- Complemento moderno: hybrid search, cross-encoder rerank, MCP.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 129a — MCP (Model Context Protocol)
- Clase 129b — Agentes: tool use, ReAct, multi-agent
- Clase 129c — LLM Evaluation: MMLU, MT-Bench, LLM-as-judge

Definiciones y características

- Embedding model: red que mapea texto a vector denso. all-MiniLM-L6-v2 (small, free), text-embedding-3-large (OpenAI, mejor calidad).
- Vector DB: base optimizada para similarity search aproximada (HNSW, IVF).
- Chunking: dividir documentos largos. Trade-off: chunks chicos → contexto perdido; grandes → ruido.
- Top-k: cuántos docs retorna el retriever. Típico 5-20.
- Reranker: modelo que re-puntúa pares (query, doc).
- MCP: protocolo estandarizado de Anthropic para tool use.
- RAG-Fusion: variante que genera N variantes del query y agrega resultados.

Dataset / recursos

- Cualquier corpus de docs propios (PDFs, markdown, HTML).
- Wikipedia dump para experimentos.
- Librerías: sentence-transformers, chromadb / faiss-cpu, rank_bm25, langchain / llama-index, mcp.

Ejercicios

1. Embed + index: con sentence-transformers/all-MiniLM-L6-v2, embeber 100 párrafos y guardarlos en Chroma. Query y top-5.
2. BM25 baseline: con rank_bm25, query y comparar resultados vs dense.
3. Hybrid (RRF): combinar ambos. Verificar que hybrid > dense > BM25 sola en queries técnicas.
4. Cross-encoder rerank: tomar top-50 del paso 3, rerankar con cross-encoder/ms-marco-MiniLM-L-6-v2. Comparar nDCG.
5. RAG con LLM: top-5 → contexto + query → Claude o GPT → respuesta con citas.

Homework verificable

Mini RAG sobre 100-1000 documentos propios:

1. Chunking + embedding + indexado en Chroma.
2. Pipeline retrieval (dense), generate (LLM via API o vLLM local).
3. 10 queries de prueba.
4. Evaluar manualmente: ¿cuántas respuestas correctas con citas válidas?
5. Bonus: agregar BM25 + cross-encoder rerank y medir mejora.

Criterio de aceptación: al menos 7/10 respuestas correctas con citas válidas; hybrid + rerank mejora el ratio.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Retrieval devuelve docs irrelevantes	Chunks muy grandes o embeddings malos. Fix
LLM alucina aunque hay contexto	Prompt no instruye a "responder solo con e
Recall bajo en queries con nombres propios	Dense pierde. Fix: hybrid con BM25.
Vector DB lento con millones de docs	Sin índice ANN. Fix: HNSW (default en Chro
Citas falsas (LLM inventa números)	El modelo no respeta formato. Fix: usar fu

Preguntas frecuentes

¿RAG o fine-tuning?

RAG para conocimiento dinámico, citable, separable del modelo. Fine-tune para skills (formato de respuesta, tono). A menudo ambos: fine-tune para how, RAG para what.

¿Cuál vector DB?

- Prototipo local: Chroma o FAISS.
- Producción small: Qdrant (Rust, rápido).
- Postgres existente: pgvector.
- Enterprise: Pinecone (managed, sin maintenance).

¿Embeddings open o API?

Open: bge-large-en-v1.5, Nomic-embed-text-v1.5. API: text-embedding-3-large (OpenAI), voyage-3 (Voyage AI). API es mejor pero costo recurrente.

¿Cómo evalúo el RAG?

Métricas: Recall@k (¿el doc correcto está entre top-k?), MRR, nDCG. Para end-to-end: RAGAS library, LLM-as-judge.

¿MCP en producción?

Sí, ya. Anthropic y comunidad ofrecen MCP servers para muchas tools comunes. Adopt earlier para ganar portabilidad cross-LLM.

Referencias

- Lewis et al. (2020), Retrieval-Augmented Generation, NeurIPS — paper original.
- Reimers & Gurevych (2019), Sentence-BERT, EMNLP.
- Robertson & Zaragoza (2009), BM25 and Beyond.
- Anthropic (2024), Model Context Protocol — <<https://modelcontextprotocol.io/>>.
- LlamaIndex docs, LangChain docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 153 — Clase 153 — MCP (Model Context Protocol): herramientas y datos para LLMs

Parte: 2 — Deep Learning · Fuente: MCP spec (Anthropic 2024). Duración estimada: 80 min.

Objetivo

Aprender MCP (Model Context Protocol) — estándar abierto publicado por Anthropic en noviembre 2024 que define cómo un LLM se conecta a herramientas externas (filesystems, databases, APIs, search engines, etc.). Antes de MCP, cada framework (LangChain, LlamaIndex, OpenAI plugins) tenía API propia. MCP unifica → portabilidad entre LLMs y clients.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la arquitectura MCP: client, server, resources, tools, prompts.
- Conectar MCP servers existentes a Claude Desktop, Cursor, Zed.
- Escribir un MCP server propio en Python (con fastmcp).
- Diferenciar MCP de tool use clásico de OpenAI / LangChain.
- Usar MCP servers populares: filesystem, postgres, git, slack, brave-search.

Temas

- Cliente (LLM app) vs Server (provee tools/resources/prompts).
- Transport: stdio (local) y SSE (network).
- Resources: read-only data (archivos, DB rows).
- Tools: funciones invocables (search, write).
- Prompts: templates reusables.
- Discovery: el client descubre dinámicamente qué hay disponible.

Definiciones y características

- MCP server: programa que expone resources/tools/prompts via JSON-RPC.
- MCP client: typically un LLM app (Claude Desktop, Cursor, IDE plugins).
- Resource URI: e.g., file:///path/to/x, postgres://db/table.
- Tool schema: JSON Schema describing inputs/outputs.
- fastmcp: librería Python para escribir servers MCP rápido.

Dataset / recursos

- MCP servers oficiales.
- Librerías: mcp (Python SDK), fastmcp.

Ejercicios

1. Conectar server existente: instalar mcp-server-filesystem en Claude Desktop. Hacer queries sobre archivos del file system.
2. Server propio: con fastmcp, exponer un tool search_docs(query) sobre un corpus local.
3. Resource: exponer archivos .md como resources lectura.
4. Prompt template: definir summarize(file_uri) como prompt reusable.
5. Multi-server: conectar 2-3 servers simultáneos a Claude Desktop; usar conjuntamente.

Homework verificable

MCP server propio para RAG sobre documentación:

1. fastmcp con tool search(query) que usa el RAG de clase 129.
2. Resource list_documents() que lista archivos indexados.
3. Conectar a Claude Desktop; usarlo en una conversación.
4. Comparar UX vs hacer RAG manual en código.

Criterio de aceptación: el LLM cliente puede llamar al search tool naturalmente y trae resultados relevantes.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Server no aparece en client	Config mal en ~/.config/claude/.... Fix: v
Tool schema rejected	JSON Schema inválido. Fix: validar con un
stdio server no inicia	Path al executable mal. Fix: paths absolut
Server lento	Llamadas síncronas. Fix: usar async def to
Cliente no encuentra resource	URI mal formado. Fix: schema correcto.

Preguntas frecuentes

¿MCP vs LangChain tools?

LangChain tools son Python objects acoplados al framework. MCP es un protocol — funciona con cualquier LLM client que lo soporte. Más portable.

¿OpenAI plugins / GPTs?

Más cerrados, especificos a OpenAI. MCP open + multi-vendor.

Servers existentes utiles?

filesystem, git, postgres, sqlite, slack, brave-search, fetch (HTTP), memory, github, gdrive — muchos en el repo oficial.

Seguridad?

Cada server corre con permisos del usuario. Cuidado con qué exponés. Auditar antes de instalar de terceros.

¿OpenAI agrega MCP?

Anthropic open-sourced el spec; otros vendors lo están adoptando (Microsoft Copilot Studio, etc.). Está volviéndose estándar.

Referencias

- MCP spec.
- MCP servers official repo.
- fastmcp.
- Anthropic blog (Nov 2024), Introducing the Model Context Protocol.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 154 — Clase 154 — Agentes: tool use, ReAct, multi-agent

Parte: 2 — Deep Learning · Fuente: Yao et al. (2023) ReAct + Schick et al. (2023) Toolformer + Anthropic agent patterns (2024). Duración estimada: 95 min.

Objetivo

Construir agentes con LLMs: el LLM planifica, invoca tools (funciones, MCP servers, APIs), observa los resultados y decide el siguiente paso. Cubrir el paradigma ReAct (Reasoning + Acting), tool use moderno

(function calling structured), y patrones multi-agent (workflows, swarms, supervisores).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir tools con JSON schema; pasar a la API del LLM.
- Implementar loop ReAct manual: prompt → LLM → tool call → execute → observation → prompt.
- Usar LangGraph o AutoGen o CrewAI para orquestar multi-agent.
- Diseñar patrones: workflow (lineal), router (decide ruta), evaluator-optimizer (loop con auto-crítica).
- Reconocer cuándo NO usar agentes (tareas determinísticas → workflow simple; agentes solo si hace falta razonamiento dinámico).

Temas

- Tool use con OpenAI/Anthropic function calling.
- ReAct prompt template.
- Loops: while LLM produces tool_call, execute and feed observation.
- LangGraph: state machines para agentes.
- AutoGen / CrewAI: multi-agent frameworks.
- Patrones (Anthropic 2024): prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
- Token cost: agentes con loops pueden gastar mucho.

Definiciones y características

- Tool: función con schema + descripción que el LLM puede invocar.
- Function calling: el LLM devuelve structured {tool_name, arguments} en lugar de texto.
- ReAct: prompt template Thought → Action → Observation → Thought → ...
- Agent: LLM en loop con tools y memoria de pasos previos.
- Supervisor pattern: 1 LLM coordina varios sub-agents.
- Token bound: agentes pueden hacer N iteraciones; siempre poner max_iterations.

Dataset / recursos

- Anthropic Claude API o OpenAI API (function calling).
- Librerías: anthropic, openai, langgraph, crewai, autogen.

Ejercicios

1. Tool básico: definir weather(city) y search(query) tools en Claude API. Pedirle que use ambos.
2. ReAct loop manual: implementar el loop sin librería, en Python puro.
3. LangGraph: definir un grafo: router → tool → evaluator → end. Compilar y ejecutar.
4. Multi-agent (CrewAI): 3 agents (researcher, writer, editor) para producir un blog post.
5. Evaluator-optimizer: agent que escribe + agent que critica + loop hasta approved=True.

Homework verificable

Agente de investigación con tools:

1. Tools: web_search, fetch_url, extract_text, save_note.
2. Prompt: "investigá X y produce un resumen de 200 palabras citando fuentes".
3. Loop ReAct hasta produce el resumen.
4. Reportar # de iteraciones, # de tool calls, costo en tokens.

Criterio de aceptación: resumen producido con ≥ 3 fuentes citadas; max_iterations no excedido.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Loop infinito	LLM repite mismas tool calls. Fix: max_ite
Tool falla, LLM no maneja	Fix: pasar el error como observation.
Costo explota	Agentes son caros. Fix: usar Haiku/GPT-min
Tool schema mal	LLM no llama. Fix: descriptions claras, sc
Output no estructurado	Fix: forzar JSON schema en respuesta final

Preguntas frecuentes

Agentes vs workflows simples?

Workflow simple si la secuencia es fija. Agente cuando el path depende del input. Anthropic recomienda: empieza simple, agregá agentic complexity solo cuando justifica.

LangGraph vs CrewAI vs AutoGen?

- LangGraph: state machines explícitos, control fino.
- CrewAI: roles + delegation, fácil para multi-agent estilo "team".
- AutoGen: conversaciones entre agents, research-oriented.

MCP en agentes?

Sí — los MCP servers son una fuente de tools estándar. Combinar MCP + agente.

Eval de agentes?

Difícil. AgentBench, τ -Bench, SWE-Bench (coding). Para custom: LLM-as-judge + human spot check.

Memoria persistent?

Vector DB + tool recall(query). O frameworks como mem0.

Referencias

- Yao et al. (2023), ReAct: Synergizing Reasoning and Acting, ICLR.
- Schick et al. (2023), Toolformer, NeurIPS.
- Anthropic (2024), Building Effective Agents — patterns guide.
- LangGraph docs, CrewAI, AutoGen.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 155 — Clase 155 — LLM Evaluation: MMLU, MT-Bench, LLM-as-judge, evals propios

Parte: 2 — Deep Learning · Fuente: Hendrycks et al. (2021) MMLU + Zheng et al. (2023) MT-Bench + LMSys Arena. Duración estimada: 85 min.

Objetivo

Evaluar LLMs (propios o terceros) con rigor: benchmarks estándar (MMLU, HumanEval, GSM8K, MT-Bench, LMSys Arena), LLM-as-judge para casos open-ended, y evals propios específicos al dominio. Reconocer las

trampas (data contamination, reward hacking, leaderboard hacking).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Correr MMLU con lm-evaluation-harness sobre un modelo propio.
- Implementar LLM-as-judge con (prompt, response_A, response_B) → "cuál es mejor".
- Diseñar evals custom: cobertura por tema, casos edge, regresiones.
- Diferenciar classification metrics (accuracy on MCQs) de generation metrics (BLEU, ROUGE, BERTScore, LLM-judge).
- Reconocer data contamination (test set en pretraining) y leaderboard hacking.

Temas

- MMLU: 57 dominios, multiple choice. Standard 2020-2023.
- HumanEval: 164 problemas Python codegen.
- GSM8K: math word problems.
- MT-Bench: 80 multi-turn questions evaluadas por GPT-4 judge.
- LMSys Arena: head-to-head humanos. Standard moderno (ELO ranking).
- LLM-as-judge: usar GPT-4/Claude como evaluador.
- Custom evals: críticos para producción.

Definiciones y características

- MMLU: 15k MCQs, 57 categorías. Score 0-100.
- MT-Bench: 80 prompts multi-turno; GPT-4 score 1-10 cada respuesta.
- LMSys Arena: humanos votan A vs B; ELO global.
- Pass@k: codegen — % de problemas resueltos en k intentos.
- LLM-as-judge: criterios estructurados, comparison pairs.
- Data contamination: el modelo memorizó el test → metrics infladas.

Dataset / recursos

- HuggingFace: lm-eval-harness, HELM, lighteval.
- Modelo a evaluar: cualquier LLM open o API.
- Librerías: lm-evaluation-harness, inspect_ai, promptfoo.

Ejercicios

1. MMLU con lm-eval-harness: `lm_eval --model hf --model_args pretrained=mistralai/Mistral-7B-v0.1 --tasks mmlu --num_fewshot 5`. Reportar score.
2. HumanEval: code generation, `pass@1`.
3. MT-Bench: usar GPT-4 / Claude como judge. Reportar score promedio.
4. LLM-as-judge propio: 20 pairs (model_A vs model_B); judge devuelve A/B/tie + reasoning.
5. Custom eval: 50 prompts específicos a tu use case + criterios de aceptación.

Homework verificable

Evaluar 2 modelos (e.g., Mistral 7B vs Llama 3 8B) en una tarea propia:

1. 30 prompts específicos al dominio.
2. Generar respuestas con ambos.
3. LLM-as-judge (Claude o GPT-4) comparando.
4. Reportar win rate de cada uno.

Criterio de aceptación: judge entrega resultado consistente (inter-rater agreement > 0.7 entre 2 ejecuciones).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MMLU score muy alto sospechoso	Posible data contamination. Fix: verificar
LLM-as-judge sesgado por longitud	Tiende a preferir respuestas largas. Fix:
LLM-judge sesgado por orden A/B	Si A siempre se evalúa primero, sesgo. Fix
Eval gameable	Optimizar en el test → degrada producción.
GPU OOM con lm-eval	Modelo grande. Fix: --batch_size auto:1 y

Preguntas frecuentes

MMLU sigue siendo válido en 2026?

Sí pero saturado — top modelos > 88 %. Mejores: MMLU-Pro, GPQA, BBH, MATH.

LMSys Arena reliable?

Sí, gold standard humano. Caro (necesita usuarios). Para producción usar como ground truth.

LLM-as-judge confiable?

GPT-4 / Claude como judge correlatan ~85 % con humanos en MT-Bench. Bias conocidos (length, position). Mitigar con prompts cuidadosos.

Evals para agentes?

SWE-Bench (coding), t-Bench (tool use), AgentBench. Custom para tu workflow.

Eval continuo en producción?

Sample logs → LLM-as-judge daily → alertar si win-rate baja vs baseline.

Referencias

- Hendrycks et al. (2021), MMLU, ICLR.
- Zheng et al. (2023), Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, NeurIPS.
- LM Evaluation Harness.
- LMSys Arena.
- Inspect AI — UK AISI eval framework.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 156 — Clase 156 — Autoencoders: undercomplete, stacked, denoising, sparse

Parte: 2 — Deep Learning · Fuente: Géron, cap. 17 § Autoencoders, GANs, and Diffusion Models.
Duración estimada: 70 min.

Objetivo

Entender autoencoders — red Encoder → bottleneck → Decoder entrenada a reconstruir su input. Variantes que cubrimos: undercomplete (dim_latent < dim_input, fuerza compresión), stacked (deep), denoising (input

ruidoso → output limpio), sparse (penaliza activaciones latentes). Saber qué problemas resuelven (compresión, anomaly detection, pretraining) y cuándo VAEs/GANs/Diffusion los superan en generación.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un undercomplete AE con MLP/CNN y entrenarlo con MSE.
- Diferenciar $\text{latent_dim} < \text{input_dim}$ (compresión real) de $\text{latent_dim} > \text{input_dim}$ con regularización (sparse).
- Implementar Denoising AE: input x + noise, target x .
- Aplicar AE como anomaly detector: alta reconstruction error → anomalía.
- Reconocer que autoencoders no son buenos generadores (latent space irregular) → motivó VAE (clase 131).

Temas

- Encoder + Decoder simétricos.
- Bottleneck: latent space.
- Undercomplete: dimensión chica.
- Stacked: varias capas Dense/Conv.
- Denoising: robusto a ruido.
- Sparse: penalizar $\| \text{latent} \|_1$ para que pocas neuronas activas.
- AE para anomaly detection.

Definiciones y características

- Bottleneck: capa con menor dimensión que input, fuerza al modelo a comprimir.
- Reconstruction loss: MSE o BCE pixel-wise.
- Denoising AE: input contaminado → target original. Aprende invariancias.
- Sparse AE: agrega L1 sobre las activaciones latentes a la loss.
- Tied weights: usar W^T del encoder como decoder. Reduce parámetros.

Dataset / recursos

- Fashion-MNIST / MNIST.
- Librerías: tensorflow, keras.

Ejercicios

1. AE simple: Encoder: $784 \rightarrow 64$; Decoder: $64 \rightarrow 784$. Entrenar en MNIST. Visualizar reconstrucciones.
2. Latent space 2D: $\text{latent_dim}=2$. Plot scatter de las representaciones de 1000 imágenes coloreadas por clase.
3. Denoising: $\text{noise} = 0.5 * \text{rng.normal}(x.\text{shape})$, target = x . Mostrar que reconstruye limpio aunque input está ruidoso.
4. Sparse: agregar $\text{keras.regularizers.l1}(1e-3)$ sobre la capa latente. Inspeccionar activaciones.
5. Anomaly detection: entrenar AE solo sobre clase "normal"; calcular reconstruction error en clase "anomalía"; usar como score.

Homework verificable

AE como anomaly detector en Fashion-MNIST:

1. Entrenar AE convolucional solo sobre clase 0 (T-shirt).
2. Calcular MSE de reconstrucción para todas las imágenes test.
3. Plotear histograma del MSE separado por "T-shirt" vs "no T-shirt".

4. ROC-AUC de "es T-shirt" usando MSE como score (negativo).

Criterio de aceptación: ROC-AUC ≥ 0.85 ; el histograma muestra clara separación.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
latent_dim muy grande → AE copia el input	No comprime nada. Fix: latent_dim < input_
MSE pierde detalle	Lo borrona. Fix: BCE pixel-wise o percept
Decoder con Dense para imágenes → mucho pa	Fix: Conv2DTranspose para upsampling.
Visualizar latent de latent_dim=64 directa	No se puede visualizar 64D. Fix: t-SNE / U
AE para generar nuevas imágenes → outputs	Latent space no es continuo. Fix: VAE (131

Preguntas frecuentes

¿AE moderna?

Sí, especialmente como encoder backbone para auto-supervised pretraining (MAE — Masked Autoencoders en visión, He et al. 2022).

¿AE vs PCA?

Linear AE con MSE = PCA. AE con activaciones no lineales = PCA no lineal. Buena ganancia con datos no lineales.

¿Tied weights?

Reducen params 2× sin perder mucho. Usado históricamente, hoy raro porque GPUs y datos son abundantes.

¿Denoising AE en producción?

Sí, para imagen denoising, audio. Pero modelos de difusión (133) lo hacen mejor.

¿Sparse AE relevante?

Más histórico. Hoy se usan Vector Quantized AE (VQ-VAE) para representaciones discretas (audio, video → tokens).

Referencias

- Géron, cap. 17 — Autoencoders.
- Vincent et al. (2008), Extracting and Composing Robust Features with Denoising Autoencoders.
- He et al. (2022), Masked Autoencoders Are Scalable Vision Learners (MAE), CVPR.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 157 — Clase 157 — Variational Autoencoders (VAE)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 17 § Variational Autoencoders + Kingma & Welling (2014). Duración estimada: 80 min.

Objetivo

Construir un VAE (Variational Autoencoder, Kingma & Welling 2014) — variante probabilística del AE que aprende una distribución sobre el latent en lugar de un punto: encoder outputs μ , σ de una gaussiana; sampling + reparametrization trick para mantener gradientes. Resultado: latent space continuo y estructurado → permite generación, interpolación entre samples.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar encoder que devuelve $(\mu, \log \sigma^2)$; sample con $z = \mu + \sigma \cdot \epsilon$ (reparametrization).
- Loss = reconstruction_loss + $\beta \cdot \text{KL}(N(\mu, \sigma^2) \parallel N(0, I))$.
- Generar samples nuevos: muestrear $z \sim N(0, I)$, pasar por el decoder.
- Interpolan en el latent space y verificar transiciones suaves.
- Reconocer que VAE produce outputs borrosos (consecuencia del MSE/BCE) — motivó GANs (132).

Temas

- ELBO (Evidence Lower BOund): $\log p(x) \geq E[\log p(x|z)] - \text{KL}(q(z|x) \parallel p(z))$.
- Reparametrization trick: para back-propagar a través del sample.
- β -VAE: subir β fuerza latent más disentangled.
- Posterior collapse: cuando el decoder ignora z .

Definiciones y características

- $q(z|x)$: encoder, devuelve $\mu(x)$, $\sigma(x)$.
- $p(z)$: prior, típicamente $N(0, I)$.
- $p(x|z)$: decoder.
- ELBO: lower bound de log-likelihood que se maximiza.
- KL divergence: $\text{KL}(N(\mu, \sigma^2) \parallel N(0, I)) = 0.5 \cdot \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ con signo.
- β -VAE: scale del término KL, controla trade-off reconstrucción vs estructura latente.

Dataset / recursos

- Fashion-MNIST / MNIST / Celeb-A (cara).
- Librerías: tensorflow, keras.

Ejercicios

1. VAE básico: encoder $\rightarrow (z_mean, z_log_var) \rightarrow$ sample $\rightarrow$ decoder. Loss combinada. Entrenar en MNIST.
2. Sampling: muestrear $z \sim N(0, I)$ de tamaño (100, latent_dim). Pasar por decoder. Visualizar las 100 imágenes generadas.
3. Interpolación: dos imágenes A y B $\rightarrow z_A, z_B$. Generar 10 imágenes en interpolación lineal entre z_A y z_B . Visualizar.
4. β -VAE: probar $\beta=1$, $\beta=5$, $\beta=10$. Comparar disentanglement vs blurriness.
5. Posterior collapse: con LR alto, el encoder colapsa a $\mu=0$, $\sigma=1$. Diagnosticar mirando $z_mean.std()$ cerca de 0.

Homework verificable

VAE sobre Fashion-MNIST:

1. Encoder convolucional, latent_dim=10.
2. Decoder simétrico.
3. Loss: BCE + KL.
4. Entrenar 30 épocas; generar 64 muestras nuevas y visualizar grid.

5. Interpolación entre 2 prendas distintas.

Criterio de aceptación: muestras generadas son reconocibles como prendas (aunque borrosas); interpolación es suave.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Posterior collapse: encoder predice $\mu=0, \sigma$	KL domina. Fix: KL annealing (subir β grad
Outputs borrosos	Consecuencia inherente de VAE + MSE/BCE. F
$KL = 0 \rightarrow$ AE puro	$\beta=0 \rightarrow$ no es VAE. Fix: $\beta > 0$.
Sampleo con $z \sim N(\mu, \sigma)$ en lugar de $N(0, I)$	Eso es reconstrucción + ruido, no generaci
Generación muestra solo 1 modo	Posterior collapse parcial. Fix: arquitect

Preguntas frecuentes

¿VAE en 2026?

Como generador end-to-end: superado por difusión. Como encoder dentro de Stable Diffusion (latent space comprimido), sí.

¿Por qué reparametrization trick?

Sin él, no se puede back-propagar a través del sample (operación estocástica). Con $z = \mu + \sigma \cdot \epsilon$, ϵ es noise externo independiente, gradientes fluyen por μ y σ .

¿Qué es disentanglement?

Cada dimensión latente captura un factor independiente (rotación, color, tamaño). β -VAE y FactorVAE lo persiguen explícitamente.

¿VAE para texto?

Sí, VAE de texto histórico. Hoy LLMs autoregresivos lo cubren mejor.

¿VQ-VAE?

VAE con codebook discreto. Base de muchos modelos modernos: DALL-E 1, Jukebox, EnCodec (audio). Importante.

Referencias

- Géron, cap. 17 — Variational Autoencoders.
- Kingma & Welling (2014), Auto-Encoding Variational Bayes, ICLR.
- Higgins et al. (2017), β -VAE, ICLR.
- van den Oord et al. (2017), Neural Discrete Representation Learning (VQ-VAE), NeurIPS.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 158 — Clase 158 — GANs: DCGAN, Progressive GAN, StyleGAN

Parte: 2 — Deep Learning · Fuente: Géron, cap. 17 § Generative Adversarial Networks + Goodfellow (2014). Duración estimada: 80 min.

Objetivo

Construir un GAN (Generative Adversarial Network, Goodfellow 2014) — dos redes en juego adversarial: Generador crea muestras desde noise; Discriminador distingue real de falso. El equilibrio Nash de este juego = generador que genera muestras indistinguibles de la distribución real. Conocer las variantes principales: DCGAN, Progressive GAN, StyleGAN (caras hiperrealistas).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar un DCGAN sencillo con Generator + Discriminator convolucionales sobre MNIST/Fashion.
- Escribir custom training loop alternando G y D updates.
- Diagnosticar 3 problemas clásicos: mode collapse (G produce 1 sola cosa), vanishing G gradient (D demasiado bueno), training inestable.
- Aplicar label smoothing (real labels = 0.9 en lugar de 1.0), noise en D inputs, y WGAN-GP (Wasserstein) para mejor estabilidad.
- Reconocer que GANs perdieron terreno frente a Diffusion (clase 133) en 2022+, pero StyleGAN sigue siendo competitivo para caras.

Temas

- Loss original (min-max): $\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$.
- DCGAN: arquitecturas convolucionales (Radford et al. 2015).
- Modos de fallo: mode collapse, D demasiado fuerte/débil.
- WGAN, WGAN-GP, spectral norm.
- Progressive GAN: empezar con 4×4, ir subiendo resolución.
- StyleGAN: AdaIN, style mixing, latent W intermedio.
- Métricas: FID (Fréchet Inception Distance), IS.

Definiciones y características

- Generator: red que toma $z \sim N(0, I) \rightarrow$ genera muestra.
- Discriminator: clasificador binario real/fake.
- Mode collapse: G genera diversidad reducida (1 o pocos modos).
- Vanishing gradient (G): cuando D distingue perfectamente, su gradiente sobre G se desvanece.
- WGAN-GP: cambia la loss a Wasserstein-1 + gradient penalty. Estabilidad superior.
- FID: distancia entre features de InceptionV3 de reales vs falsas; menor = mejor.
- StyleGAN: latent z se mapea a w y se inyecta vía AdaIN en cada capa del generador.

Dataset / recursos

- MNIST / Fashion-MNIST como playground.
- Celeb-A para caras (más serio).
- Librerías: tensorflow, keras.

Ejercicios

1. DCGAN básico: G y D convolucionales. Custom training loop con dos `tf.function` (`train_d`, `train_g`).
2. Diagnóstico: graficar `D_loss` y `G_loss` por step. Si `D_loss` $\rightarrow 0$, G está perdiendo.
3. Mode collapse: tras N épocas, generar 64 samples. Si todos son la misma cosa $\rightarrow$ collapse.
4. WGAN-GP: implementar gradient penalty en la D loss. Comparar estabilidad vs DCGAN.
5. FID: implementar (o usar `tensorflow_gan.eval.fid`) y reportar FID del modelo.

Homework verificable

DCGAN en Fashion-MNIST:

1. Generator: Dense(77128) → reshape → Conv2DTranspose × 3 hasta 28×28.
2. Discriminator: Conv2D × 3 → Dense(1).
3. Entrenar 50 épocas; generar grid 8×8.
4. Reportar D_loss y G_loss finales; visual del grid.

Criterio de aceptación: las muestras generadas son reconocibles como prendas (aunque imperfectas); el training es razonablemente estable (sin colapsar a un solo modo).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Mode collapse	D muy fuerte, G no puede engañar. Fix: WGA
Training inestable, oscilaciones	LR alto en ambos. Fix: Adam(2e-4, beta_1=0
D_loss = 0	D resolvió. Fix: hacer D más débil o agreg
Outputs muy borrosos	Underfit de G. Fix: G más expresivo, más é
Outputs de baja resolución bien, alta reso	Sin Progressive Growing. Fix: usar StyleGA

Preguntas frecuentes

¿GAN sigue relevante en 2026?

StyleGAN3 sigue siendo competitivo para caras y generación en general "rápida" (1 forward pass). Difusión gana en calidad para texto-to-image complejo.

¿FID o IS?

FID es mejor (más robusta a artifacts). IS está deprecada.

¿Conditional GAN?

cGAN (Mirza & Osindero 2014): condicionar en label o texto. Permite control sobre qué generar.

¿GAN inversion?

Encontrar z tal que $G(z) \approx \text{imagen_dada}$. Útil para edición. Difícil; StyleGAN tiene buenas implementaciones.

¿Por qué Diffusion ganó?

Más estable de entrenar (sin min-max), mejor calidad de outputs, controllable con text. La explicación más simple: optimiza $\log p(x)$ directamente, GAN optimiza un divergence relacionado pero no la log-likelihood.

Referencias

- Géron, cap. 17 — Generative Adversarial Networks.
- Goodfellow et al. (2014), Generative Adversarial Networks, NeurIPS.
- Radford et al. (2015), DCGAN, ICLR.
- Karras et al. (2018), Progressive Growing of GANs, ICLR.
- Karras et al. (2019, 2020, 2021), StyleGAN1/2/3, NeurIPS/CVPR.
- Gulrajani et al. (2017), WGAN-GP, NeurIPS.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.

- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 159 — Clase 159 — Modelos de difusión (+ Stable Diffusion XL, ControlNet, LCM)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 17 § Diffusion Models + Ho et al. (2020) DDPM + papers SDXL, ControlNet, LCM. Duración estimada: 105 min.

Objetivo

Entender modelos de difusión — la familia que destronó a GANs en 2022 (DALL-E 2, Stable Diffusion, Midjourney). Idea: forward process agrega ruido gaussiano gradualmente; reverse process (aprendido) lo elimina paso a paso. Conocer DDPM clásico, latent diffusion (Stable Diffusion), ControlNet para condicionamiento espacial, LCM (Latent Consistency Models) para inference rápida.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar un DDPM sencillo: forward $q(x_t | x_0)$, U-Net que predice el ruido $\epsilon_\theta(x_t, t)$.
- Aplicar el sampling DDPM: 1000 steps de denoising desde $x_T \sim N(0, I)$.
- Reconocer latent diffusion: aplicar difusión en el espacio comprimido de un VAE (1/64 del tamaño) → 8× más rápido.
- Usar Stable Diffusion XL con diffusers library: prompt → imagen en 3 líneas.
- Aplicar ControlNet para condicionar generación en pose, edge, depth, segmentation.
- Acelerar inference con LCM o Turbo (1-4 steps en lugar de 50).

Temas

- Forward process: $q(x_t | x_{t-1}) = N(\sqrt{1-\beta_t} x_{t-1}, \beta_t I)$.
- Closed form: $q(x_t | x_0) = N(\sqrt{\alpha_t} x_0, (1-\alpha_t) I)$.
- Loss simple (Ho 2020): $MSE(\epsilon, \epsilon_\theta(x_t, t))$ — predecir el ruido.
- U-Net architecture (encoder-decoder con skip connections).
- Sampling DDPM vs DDIM vs DPM-Solver: 1000 → 50 → 20 steps.
- Complemento moderno: Stable Diffusion XL, ControlNet, LCM/Turbo.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 133a — Stable Diffusion XL + ControlNet en profundidad

Definiciones y características

- Forward (diffusion) process: $q(x_t | x_{t-1})$ agrega ruido. Markov chain.
- Reverse process: $p_\theta(x_{t-1} | x_t)$ aprendido, denoise.
- Schedule: β_t o α_t — cuánto ruido en cada step. Linear o cosine.
- U-Net: arquitectura encoder-decoder con skip connections, usada como denoiser.
- DDIM: sampler deterministic más rápido que DDPM (Song et al. 2021).
- Classifier-Free Guidance (CFG): en inference, mezcla condicional + incondicional para control.
- ControlNet: adapter para control espacial.
- LCM: distillation a 1-4 steps.

Dataset / recursos

- Fashion-MNIST como playground.

- HF Hub para modelos pre-trained.
- Librerías: diffusers, transformers, accelerate, controlnet_aux.

Ejercicios

1. DDPM en MNIST: U-Net chico + DDPM scheduler. Entrenar 50 épocas; samplear 64 imágenes.
2. SDXL inference: cargar SDXL y generar 4 imágenes desde prompts.
3. CFG: variar guidance_scale {1, 5, 15}. Ver cómo afecta fidelidad al prompt vs diversidad.
4. ControlNet Canny: tomar una foto, extraer bordes con OpenCV, generar variantes condicionadas con SDXL+ControlNet.
5. LCM: comparar SDXL base (30 steps) vs SDXL + LCM-LoRA (4 steps). Tiempo y calidad.

Homework verificable

Pipeline de generación SDXL controlado:

1. Imagen base (e.g., una foto de un edificio).
2. Extraer Canny con OpenCV.
3. Generar 3 variantes con SDXL+ControlNet con prompts distintos.
4. Comparar tiempo SDXL vs SDXL+LCM.

Criterio de aceptación: las 3 variantes preservan la estructura del edificio según los Canny; LCM es $\geq 5\times$ más rápido con calidad razonable.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OOM al cargar SDXL en GPU 8 GB	Modelo muy grande. Fix: fp16, enable_attn
Imágenes borrosas con LCM	Steps insuficientes. Fix: subir a 4-8; aju
ControlNet no obedece el control	controlnet_conditioning_scale muy bajo. Fi
Generación con prompts pobres da resultado	Prompt engineering es real. Fix: estilo "a
Difusión propia entrenada genera ruido	No converge. Fix: verificar la schedule, U

Preguntas frecuentes

¿GAN o Diffusion en 2026?

Diffusion gana en calidad y controlabilidad. GAN (StyleGAN3) sigue siendo competitivo para caras (1 forward pass).

¿DDPM o DDIM o DPM-Solver?

Para inference: DPM-Solver / DPM++ (Lu et al. 2022) — 20 steps con misma calidad que DDPM 1000. Default en diffusers.

¿Cómo fine-tunear SDXL en estilo propio?

LoRA sobre el U-Net con diffusers examples. 20-50 imágenes alcanza para un estilo. ~1 hora en GPU consumer.

¿Difusión para video?

Sí. Sora, Veo, Stable Video Diffusion. Más caro computacionalmente.

¿Difusión para texto?

Sí pero menos competitivo que LLMs autoregresivos. Áreas activas de investigación.

Referencias

- Géron, cap. 17 — Diffusion Models.
- Ho, Jain & Abbeel (2020), Denoising Diffusion Probabilistic Models, NeurIPS.
- Rombach et al. (2022), Stable Diffusion, CVPR.
- Zhang, Rao & Agrawala (2023), ControlNet, ICCV.
- Luo et al. (2023), Latent Consistency Models.
- diffusers docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 160 — Clase 160 — Stable Diffusion XL + ControlNet en profundidad

Parte: 2 — Deep Learning · Fuente: Rombach et al. (2022) SD + Podell et al. (2023) SDXL + Zhang et al. (2023) ControlNet. Duración estimada: 90 min.

Objetivo

Dominar Stable Diffusion XL (Stability AI 2023) en producción: pipeline completo (text encoder dual, U-Net, VAE), schedulers modernos (DPM-Solver++, Euler ancestral, UniPC), CFG (Classifier-Free Guidance), prompt weighting. Combinar con ControlNet para condicionamiento espacial (Canny, depth, pose, segmentation, lineart). Conocer Flux (Black Forest Labs 2024) como sucesor open-source.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Cargar SDXL: `StableDiffusionXLPipeline.from_pretrained('stabilityai/stable-diffusion-xl-base-1.0')`.
- Aplicar refiner opcional para detalle final.
- Usar schedulers distintos y entender trade-off speed/quality.
- Agregar ControlNet (canny, depth, openpose) sobre SDXL.
- Aplicar LoRA para estilo custom (`pipe.load_lora_weights('path')`).
- Reconocer Flux y SD3 como sucesores.

Temas

- Pipeline SDXL: text_encoder_1 (CLIP-L), text_encoder_2 (CLIP-G), U-Net 2.6B params, VAE.
- Schedulers: DDIM (50 steps), DPM-Solver++ (20 steps), Euler ancestral (creative), UniPC (10-20 steps).
- CFG: guidance_scale=7.5 default; > 12 over-fitting al prompt.
- Negative prompts.
- Refiner (paso opcional adicional).
- ControlNet variantes: Canny, Depth, OpenPose, Scribble, Lineart, MLSD, Tile.

Definiciones y características

- SDXL: 2.6B params U-Net, latent space 128×128×4 desde imagen 1024×1024.
- Dual text encoder: CLIP-L (768d) + CLIP-G (1280d) concatenados.
- CFG: output = uncond + scale · (cond - uncond).
- Negative prompt: lo que NO querés (ugly, blurry, low quality).
- ControlNet: red adicional, freezeada o trained, que inyecta condicionamiento espacial.

- LoRA: rank-low adapter, ~50-200 MB, para estilos.
- Flux 1: sucesor open-source SDXL (2024), mejor calidad y prompt following.

Dataset / recursos

- HuggingFace: stabilityai/stable-diffusion-xl-base-1.0, ControlNets en diffusers/controlnet-*-sdxl-1.0.
- Librerías: diffusers, transformers, accelerate, controlnet_aux.

Ejercicios

1. SDXL básico: prompt → imagen 1024². Comparar schedulers (DPM-Solver++, Euler a, UniPC).
2. CFG sweep: guidance_scale {3, 7.5, 12, 18}. Ver trade-off creativity/fidelity.
3. Negative prompt: agregar "blurry, watermark, low quality" y comparar.
4. ControlNet Canny: extraer Canny de una foto, generar variante manteniendo estructura.
5. LoRA estilo: cargar un LoRA de estilo (CivitAI), aplicar. Variar cross_attention_kwargs={'scale': 0.8}.

Homework verificable

Pipeline visual: SDXL + ControlNet Canny + LoRA estilo:

1. Foto base → Canny.
2. SDXL + ControlNet, prompt + LoRA estilo.
3. Generar 4 variantes; comparar.
4. Reportar tiempo y memoria.

Criterio de aceptación: estructura preservada por Canny; estilo del LoRA visible; outputs de buena calidad.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OOM en GPU 8 GB	Modelo grande. Fix: pipe.enable_attention_
Outputs raros, deformados	CFG demasiado alto. Fix: guidance_scale=5-
ControlNet no obedece estructura	controlnet_conditioning_scale muy bajo. Fi
Manos deformadas	Bug clásico de difusión. Fix: LoRA especí
LoRA no se siente	Scale muy bajo. Fix: cross_attention_kwarg

Preguntas frecuentes

SDXL o Flux?

Flux (2024) mejor calidad y prompt following. SDXL más maduro, más LoRAs/ControlNets disponibles. Ambos viables.

SD3 dónde está?

Stability AI lo publicó pero con licencia restrictiva → comunidad migró a Flux. SD3.5 medium open license.

Refiner vale la pena?

A veces. Toma más tiempo. Probar con vs sin para tu caso.

Cómo entreno LoRA para SDXL?

diffusers examples + dataset de 10-30 imágenes + script de training. 1-2 horas GPU.

Inpainting?

StableDiffusionXLInpaintPipeline. Modificar regiones específicas con máscara.

Referencias

- Podell et al. (2023), SDXL.
- Zhang et al. (2023), ControlNet, ICCV.
- diffusers docs.
- Flux (Black Forest Labs).
- CivitAI — comunidad LoRA/checkpoints.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 161 — Clase 161 — RL: aprendizaje por recompensa, Gymnasium (Farama)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Introduction to Reinforcement Learning + docs Gymnasium. Duración estimada: 75 min.

Objetivo

Entender el paradigma de Reinforcement Learning — un agente interactúa con un environment, observa states, toma actions, recibe rewards, y aprende una policy que maximiza recompensa acumulada. Conocer Gymnasium (Farama Foundation, fork del antiguo OpenAI Gym que dejó de mantenerse en 2022) — la librería estándar de environments para benchmarks.

Nota: el nombre "OpenAI Gym" es histórico. Desde 2022, OpenAI dejó de mantenerlo y la Farama Foundation tomó el relevo creando Gymnasium — drop-in replacement con mejor mantenimiento.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir los 5 componentes RL: agent, environment, state, action, reward.
- Usar gymnasium: `env = gym.make('CartPole-v1')`; `obs, _ = env.reset()`; `obs, reward, terminated, truncated, info = env.step(action)`.
- Implementar un policy random como baseline.
- Reconocer la diferencia entre on-policy (PPO, A2C) y off-policy (DQN, SAC).
- Saber dónde RL es apropiado (juegos, robótica, navegación) vs donde no (clasificación, regresión típicas).

Temas

- Bloque básico: `state` → `action` → `reward` → `next_state`.
- Episodic vs continuous tasks.
- Discrete vs continuous action spaces.
- Discount factor γ : balance entre recompensa inmediata y futura.
- Gymnasium API estándar (`reset`, `step`).
- Environments populares: `CartPole`, `MountainCar`, `Atari`, `MuJoCo`, `BipedalWalker`.

Definiciones y características

- Policy $\pi(a|s)$: la estrategia — distribución sobre acciones dado state.
- Trajectory / episode: secuencia (`s_0`, `a_0`, `r_0`, `s_1`, `a_1`, ...) hasta done.
- Return: suma de recompensas futuras $G_t = \sum \gamma^k r_{t+k}$.

- Value function $V(s)$: expected return desde s siguiendo π .
- Q-function $Q(s, a)$: expected return tras tomar a desde s .
- Gymnasium step: devuelve 5-tuple (obs, reward, terminated, truncated, info) (Gym original devolvía 4 — done combinaba terminated/truncated).
- Action space: Discrete(N) o Box(low, high, shape).

Dataset / recursos

- gymnasium library (pip install gymnasium).
- Environments built-in: CartPole-v1, LunarLander-v3, MountainCar-v0.
- Librerías: gymnasium, numpy, matplotlib.

Ejercicios

1. Entorno básico: `env = gym.make('CartPole-v1', render_mode='human')`. Correr 10 episodios con acciones random; reportar duración promedio.
2. Estructura del state: imprimir `env.observation_space` y `env.action_space` para CartPole, MountainCar, LunarLander.
3. Policy heurística: para CartPole, `action = 0` if `pole_angle < 0` else `1` (push opuesto al tilt). Comparar contra random.
4. Return discounted: calcular $G_t = \sum \gamma^k r_{t+k}$ con $\gamma=0.99$ sobre un episodio.
5. Render: usar `render_mode='rgb_array'` y guardar frames para crear un gif.

Homework verificable

Sobre CartPole-v1:

1. Implementar 3 policies: random, heurística simple, "siempre derecha".
2. Correr 100 episodios de cada; reportar return promedio y desviación.
3. Verificar que heurística supera random.

Criterio de aceptación: random ≈ 20 steps; heurística ≥ 80 steps; "siempre derecha" muere rápido.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>gym.make(...)</code> deprecated warning	Estás usando el viejo OpenAI Gym. Fix: pip
<code>env.step(action)</code> devuelve 4 valores	Gym viejo. Gymnasium devuelve 5. Fix: actu
<code>env.reset()</code> devuelve un solo valor en tuto	Gym pre-2022 devolvía obs. Gymnasium devue
Render no muestra ventana	<code>render_mode</code> no especificado al crear env.
Atari environments fail	Requieren pip install gymnasium[atari,acce

Preguntas frecuentes

¿RL vs supervised?

Supervised: aprende $f(x) = y$ con labels. RL: aprende $\pi(s) = a$ con feedback indirecto (rewards). Más cercano a humanos pero más difícil.

¿Cuándo RL?

Juegos (AlphaGo), robótica, navegación, sistemas de recomendación con feedback de usuarios. NO para clasificación / regresión típicas.

¿Gymnasium o RLlib o Stable-Baselines?

Gymnasium = environments. Stable-Baselines3 (PyTorch) o RLlib (Ray) = algoritmos. Estándar 2026: SB3 para experimentos, RLlib para producción a escala.

¿RL con LLMs?

Sí — RLHF y DPO (clase 128) son aplicaciones de RL a LLMs.

¿Cuánto cuesta entrenar?

CartPole: 1 minuto. Atari: horas-días. AlphaGo: semanas en cluster. RL es muy costoso en datos (interacciones).

Referencias

- Géron, cap. 18 — Reinforcement Learning.
- Sutton & Barto (2018), Reinforcement Learning: An Introduction (2nd ed.) — biblia del RL.
- Gymnasium docs.
- Stable-Baselines3 docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 162 — Clase 162 — Policy gradients

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Policy Gradients + Sutton & Barto, cap. 13.
Duración estimada: 70 min.*

Objetivo

Implementar policy gradient —REINFORCE (Williams 1992)—: parametrizar la policy con una red neuronal $\pi_{\theta}(a|s)$, optimizar directamente la expected return via gradiente. Es el método más simple de RL que usa redes y la base conceptual de PPO/A2C/A3C (clase 138).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir la policy como red state $\rightarrow$ softmax(actions).
- Calcular el gradiente REINFORCE: $\theta J = E[\theta \log \pi_{\theta}(a|s) \cdot G_t]$.
- Implementar el training loop: rollout $\rightarrow$ calcular returns $\rightarrow$ gradient ascent.
- Aplicar baseline (substraer $V(s)$ de G_t) para reducir varianza.
- Reconocer la limitación: alta varianza, lento (motiva A2C/PPO).

Temas

- Expected return $J(\theta) = E_{\pi} [G]$.
- Policy gradient theorem: $\theta J = E[\theta \log \pi \cdot Q]$.
- REINFORCE algorithm: rollout completo + apply gradient.
- Baseline para reducir varianza.
- Discounted returns con γ .

Definiciones y características

- Policy network: $\pi_{\theta}(a|s)$, típicamente MLP con softmax (discrete actions) o gaussiana (continuas).

- Log-prob: $\log \pi(a|s)$, lo que multiplicamos por G_t para el gradient.
- Discounted return $G_t: \sum \gamma^k r_{t+k}$.
- Baseline: cualquier función de s (e.g., $V(s)$) que reduce varianza sin sesgo.
- Advantage $A = G - V(s)$: cuán mejor o peor fue la acción comparada con el valor esperado.

Dataset / recursos

- CartPole-v1.
- Librerías: gymnasium, tensorflow, keras.

Ejercicios

1. Policy network: $\text{Dense}(32) \rightarrow \text{Dense}(32) \rightarrow \text{Dense}(2, \text{softmax})$ para CartPole.
2. Rollout: ejecutar 1 episodio, guardar (s, a, r) por timestep.
3. Returns: calcular G_t para cada timestep con $\gamma=0.99$.
4. Gradient step: $\text{loss} = -\sum \log \pi(a_t|s_t) \cdot G_t$; backward; apply.
5. Con baseline: agregar $V(s)$ head, restar de G antes del gradient.

Homework verificable

REINFORCE en CartPole:

1. Policy $\text{Dense}(64) \rightarrow \text{Dense}(64) \rightarrow \text{Dense}(2, \text{softmax})$.
2. Train 500 episodios.
3. Reportar return medio por época.
4. Comparar con/sin baseline.

Criterio de aceptación: episode return llega a ≥ 195 (resuelto) en ≤ 300 episodios; baseline acelera convergencia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Alta varianza en gradientes	Inherente a REINFORCE. Fix: baseline, batc
Policy collapse a una acción	LR alto o sin entropy bonus. Fix: bajar LR
Recompensas muy chicas $\rightarrow$ updates débiles	Normalizar G (mean=0, std=1) por episode.
Action probs nan	Inputs sin normalizar + softmax. Fix: clip
El env tiene rewards muy variables $\rightarrow$ entre	Probar con env más simple primero.

Preguntas frecuentes

¿REINFORCE en producción?

Casi no — pero es la base. PPO es el default industrial moderno (clase 138).

¿On-policy o off-policy?

REINFORCE es on-policy (entrena con datos generados por la policy actual). Off-policy (DQN, SAC) reutiliza datos viejos.

¿Cómo elijo γ ?

0.99 default. Más alto = más visión a largo plazo, pero más varianza. Para tareas episódicas cortas, 0.95-0.99.

¿Entropy regularization?

Agregar $-\beta \cdot H(\pi)$ a la loss promueve exploración (policy no determinística temprano). Estándar.

¿Cómo veo si converge?

Plot del return promedio por epoch (smoothed). Sube → converge.

Referencias

- Géron, cap. 18 — Policy Gradients.
- Williams (1992), Simple Statistical Gradient-Following Algorithms (REINFORCE).
- Sutton & Barto (2018), cap. 13.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 163 — Clase 163 — Markov Decision Processes

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Markov Decision Processes + Sutton & Barto cap. 3. Duración estimada: 60 min.

Objetivo

Formalizar el marco teórico de RL: un Markov Decision Process (MDP) = tupla (S, A, P, R, γ). Conocer la Bellman equation que define V y Q óptimos, y los algoritmos clásicos Value Iteration y Policy Iteration que los resuelven (cuando el MDP es conocido).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir un MDP: states, actions, transition prob, reward function, discount.
- Escribir la Bellman equation para V: $V(s) = \max_a \sum P(s'|s,a)[R + \gamma V^*(s')]$.
- Implementar Value Iteration: actualizar iterativamente hasta convergencia.
- Implementar Policy Iteration: alternar policy evaluation + policy improvement.
- Reconocer la limitación: requiere conocer P y R (no aplicable a entornos reales) → motivó model-free (Q-learning, clase 137).

Temas

- Propiedad de Markov: $P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_t, a_t, s_{t-1}, \dots)$.
- Componentes MDP.
- Bellman optimality equation.
- Value Iteration (synchronous update).
- Policy Iteration (alternar eval + improve).
- Convergencia garantizada (contractive operator).

Definiciones y características

- State s: representación del entorno.
- Action a: lo que el agente puede hacer.
- Transition $P(s' | s, a)$: probabilidad de llegar a s' desde s tomando a.
- Reward $R(s, a, s')$: recompensa inmediata.
- Discount γ : peso de recompensas futuras.

- $V^*(s)$: máximo expected return desde s .
- $Q^*(s, a)$: máximo expected return tras tomar a en s .
- Policy óptima: $\pi(s) = \operatorname{argmax}_a Q(s, a)$.

Dataset / recursos

- MDPs sintéticos chicos (FrozenLake, Taxi de Gymnasium).
- Librerías: gymnasium, numpy.

Ejercicios

1. MDP de juguete: definir un MDP de 4 estados con P, R manualmente.
2. Value Iteration: implementar $V[s] = \max_a \sum P(s'|s,a)(R + \gamma V[s'])$ hasta $\max \text{change} < 1e-6$.
3. Policy Iteration: alternar evaluación (V^π) con mejora ($\pi' = \text{greedy}(V)$) hasta estabilidad.
4. FrozenLake: cargar `gym.make('FrozenLake-v1')`, extraer `env.unwrapped.P` (modelo del MDP), resolver con VI.
5. Compare: # iteraciones VI vs PI para llegar a misma policy.

Homework verificable

Sobre FrozenLake-v1 (`is_slippery=True`):

1. Extraer modelo P y R desde `env.unwrapped.P`.
2. Implementar Value Iteration; reportar V y π greedy.
3. Evaluar la policy con 1000 episodios random; reportar success rate.

Criterio de aceptación: success rate ≥ 0.7 sobre FrozenLake slippery; V^* y policy claramente prefieren caminos seguros.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Value Iteration no converge	$\gamma \geq 1.0$. Fix: $\gamma < 1.0$ (típicamente 0.95-0.99).
Policy Iteration loops infinito	Numerical issues en evaluación. Fix: limit
Acceder a <code>env.P</code> falla	Hay que usar <code>env.unwrapped.P</code> .
Asumir Markov en problemas non-Markov	Por ejemplo, control con velocidad escondi
VI vs PI: cuándo usar?	VI: más simple, más iters pero más baratas

Preguntas frecuentes

¿MDPs en problemas reales?

Casi nunca conocés P y R exactamente. Por eso $\rightarrow$ Q-learning (clase 137) y métodos model-free.

¿POMDPs?

Partially Observable: cuando no ves el state completo. Mucho más difícil. Usar agente con memoria (LSTM, Transformer).

¿Continuous state space?

VI/PI no escalan. Usar function approximation: DQN (clase 137) o policy gradient (135).

¿Bellman equation conexión con DP?

Sí, VI es dynamic programming clásico aplicado a MDPs.

¿Aprender el modelo P, R desde data?

Model-based RL: aprender un modelo, planificar con él. Más sample-efficient pero más complejo.

Referencias

- Géron, cap. 18 — Markov Decision Processes.
- Sutton & Barto, cap. 3-4.
- Bellman (1957), Dynamic Programming.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 164 — Clase 164 — TD Learning, Q-Learning, Deep Q-Networks

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Q-Learning y § Deep Q-Learning. Duración estimada: 80 min.

Objetivo

Implementar Q-Learning clásico (Watkins 1989) y su versión moderna DQN (Mnih et al. 2015, Nature paper que aprendió Atari desde pixels). Off-policy, model-free, bootstrap. Conocer los 2 trucos que hicieron a DQN funcionar: experience replay y target network.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la update Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.
- Implementar Q-learning tabular en FrozenLake.
- Construir un DQN: red state $\rightarrow Q(a)$ para todas las actions, MSE entre Q predicted y $r + \gamma \max_{a'} Q(s',a')$.
- Implementar replay buffer (almacenar transitions, samplear batch para training).
- Implementar target network (copia frozen actualizada cada N steps).

Temas

- TD (Temporal Difference) error: $\delta = r + \gamma V(s') - V(s)$.
- Q-learning: off-policy, sigue greedy de Q^* .
- ϵ -greedy exploration: con prob ϵ explora random, sino greedy.
- Replay buffer: deque de (s, a, r, s', done).
- Target network: estabilidad (sino, target se mueve mientras estimás).
- DQN sobre CartPole y Atari.

Definiciones y características

- Q-value $Q(s, a)$: expected return tras tomar a desde s.
- Bootstrap: update usando estimación actual (sin esperar fin de episodio).
- Off-policy: aprende sobre policy óptima aunque exploración sea ϵ -greedy.
- Replay buffer: almacena experiencias para sampleo iid.
- Target network Q' : copia de Q usada para calcular el target. Actualizada cada N steps.
- ϵ -greedy: balance exploration vs exploitation.

Dataset / recursos

- FrozenLake (tabular).
- CartPole (DQN).
- Librerías: gymnasium, tensorflow, keras, numpy.

Ejercicios

1. Q-learning tabular: en FrozenLake, mantener $Q[s, a]$ numpy array. Update con ϵ -greedy. Reportar success rate tras 5000 episodios.
2. DQN básico: red Dense(64) $\rightarrow$ Dense(64) $\rightarrow$ Dense(2) para CartPole. Sin replay buffer ni target network $\rightarrow$ ver inestabilidad.
3. Replay buffer: from collections import deque; buffer = deque(maxlen=10_000). Sample batch=32 random.
4. Target network: copiar Q.weights cada 100 steps a Q_target.
5. ϵ decay: empezar $\epsilon=1.0$, decaer linealmente a 0.01 en 10 000 steps.

Homework verificable

DQN sobre CartPole-v1:

1. Red Dense(64, relu) $\rightarrow$ Dense(64, relu) $\rightarrow$ Dense(2).
2. Replay buffer 50 000, batch 64.
3. Target network sync cada 100 steps.
4. ϵ : linear decay 1.0 $\rightarrow$ 0.05 sobre 10 000 steps.
5. Train hasta mean_reward(100 episodios) ≥ 195 .

Criterio de aceptación: alcanza ≥ 195 en ≤ 500 episodios; gráfico de return promedio muestra convergencia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Q values explotan	Sin target network, bootstrap inestable. F
Convergencia lenta	Replay buffer muy chico o batch muy chico.
No explora $\rightarrow$ policy subóptima	ϵ muy chico desde el inicio. Fix: $\epsilon=1.0$ al
done confundido con truncated	Gymnasium 5-tuple. Fix: tratar terminated,
OOM con Atari	Stacks de 4 frames + buffer enorme. Fix: r

Preguntas frecuentes

¿DQN supera Q-learning tabular cuándo?

Cuando state space es enorme (continuous o pixels). Para FrozenLake (16 states), tabular gana.

¿Variantes de DQN?

- Double DQN (van Hasselt 2016): reduce overestimación.
- Dueling DQN (Wang 2016): separa V y advantage.
- Prioritized Experience Replay: muestreo no uniforme.
- Rainbow (Hessel 2018): combina todas.

¿DQN vs Policy Gradient?

DQN: off-policy, sample-efficient, action space discreto. PG (PPO): on-policy, más estable, action space continuo. Ambos en el zoo moderno.

¿Por qué replay buffer rompe la correlación temporal?

Sin él, transitions consecutivas están correlacionadas → batches no iid → gradiente sesgado.

¿Cuál env empezar?

CartPole para entender. LunarLander para algo más complejo. Atari para serio (requiere días).

Referencias

- Géron, cap. 18 — Q-Learning y Deep Q-Learning.
- Watkins (1989), Learning from Delayed Rewards (PhD thesis).
- Mnih et al. (2015), Human-level control through deep reinforcement learning, Nature — DQN paper.
- van Hasselt et al. (2016), Deep Reinforcement Learning with Double Q-learning.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 165 — Clase 165 — RL moderno: A3C, PPO, SAC (vista general)

Parte: 2 — Deep Learning · Fuente: papers A3C (Mnih 2016), PPO (Schulman 2017), SAC (Haarnoja 2018) + docs Stable-Baselines3. Duración estimada: 65 min.

Objetivo

Vista general (sin implementación desde cero) de los 3 algoritmos modernos de RL: A3C (Asynchronous Advantage Actor-Critic), PPO (Proximal Policy Optimization — el default industrial), y SAC (Soft Actor-Critic — off-policy, continuous actions). Saber cuál elegir y cómo usarlos con Stable-Baselines3.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar on-policy (A3C, PPO) de off-policy (SAC, DQN).
- Reconocer la idea de Actor-Critic: dos redes — actor (policy) + critic (value).
- Entender el clipped objective de PPO: limita updates a $[1-\epsilon, 1+\epsilon]$ veces el old policy → estabilidad.
- Usar Stable-Baselines3: `PPO('MlpPolicy', env).learn(total_timesteps=100_000)`.
- Elegir: PPO para discreto/continuo, on-policy. SAC para continuo, off-policy, sample-efficient.

Temas

- Actor-Critic: actor da policy, critic da $V(s)$. Advantage = $G - V(s)$.
- A3C: async + advantage actor-critic. Paralelización con multiples workers.
- A2C: variante sincrónica (más simple).
- PPO: clipped surrogate objective.
- SAC: actor-critic off-policy + entropy regularization.
- Stable-Baselines3 como librería estándar.

Definiciones y características

- Actor: red de policy $\pi_\theta(a|s)$.
- Critic: red de valor $V_\phi(s)$ o $Q_\phi(s,a)$.
- Advantage $A(s, a)$: cuán mejor es a que el promedio.
- GAE (Generalized Advantage Estimation): estimador robusto del advantage usado en PPO.

- PPO clip: $\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot A)$ con $\text{ratio} = \pi_{\text{new}}/\pi_{\text{old}}$.
- SAC entropy bonus: $H(\pi)$ se maximiza junto al return $\rightarrow$ exploración natural.

Dataset / recursos

- LunarLander, CartPole, Pendulum.
- Librerías: gymnasium, stable-baselines3 (pip install stable-baselines3[extra]).

Ejercicios

1. PPO con SB3: `from stable_baselines3 import PPO; model = PPO('MlpPolicy', 'CartPole-v1', verbose=1); model.learn(50_000)`. Evaluar.
2. SAC en Pendulum: `SAC('MlpPolicy', 'Pendulum-v1').learn(20_000)`.
3. Comparar: PPO vs SAC en LunarLander; reportar return y sample efficiency.
4. TensorBoard: `PPO(..., tensorboard_log='./tb/')` y ver curvas.
5. Custom callback: EvalCallback para evaluar cada N steps y guardar mejor modelo.

Homework verificable

Entrenar PPO en LunarLander-v3:

1. `PPO('MlpPolicy', 'LunarLander-v3', verbose=1, tensorboard_log='./tb/')`.
2. `model.learn(total_timesteps=500_000)`.
3. Evaluar 100 episodios; reportar mean reward.

Criterio de aceptación: mean reward ≥ 200 (problema resuelto); TensorBoard muestra curva ascendente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
PPO no converge	LR alto o env muy complejo. Fix: learning_
SAC en discrete actions	SAC es continuous. Fix: PPO o DQN para dis
Reward "stuck" en ~0	El env tal vez no es resoluble con MLP. Fi
Training muy lento	Pocos parallel envs. Fix: <code>make_vec_env('Ca</code>
Modelo guardado no carga	Versión de SB3 incompatible. Fix: <code>model =</code>

Preguntas frecuentes

¿PPO o SAC?

- PPO: discreto o continuo, on-policy, más estable, default industrial.
- SAC: continuo, off-policy, sample-efficient, mejor performance pico.
- Estándar: PPO primero; SAC si necesitas más sample efficiency.

¿Cuántos steps?

CartPole: 50k. LunarLander: 500k. Atari: 10M+. Robótica MuJoCo: 1M-10M.

¿RL para LLM (RLHF)?

PPO es el algo estándar de RLHF. DPO (clase 128) lo reemplazó por simplicidad.

¿Cuándo NO usar RL?

Si el problema es supervised (clasificación, regresión), no uses RL. RL es sample-inefficient y caro.

¿Hyperparameter tuning?

Optuna sobre SB3 o rl_zoo3 que tiene hyperparams pre-tuneados para muchos envs.

Referencias

- Mnih et al. (2016), Asynchronous Methods for Deep Reinforcement Learning (A3C), ICML.
- Schulman et al. (2017), PPO.
- Haarnoja et al. (2018), Soft Actor-Critic, ICML.
- Stable-Baselines3 docs.
- Sutton & Barto, capítulos avanzados.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 166 — Clase 166 — TF Serving + gRPC (+ ONNX, TensorRT, vLLM/TGI)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Deploying a Model to a Service + docs ONNX, TensorRT, vLLM, TGI. Duración estimada: 100 min.

Objetivo

Servir un modelo entrenado a producción. Aprender TF Serving (oficial de TensorFlow, gRPC/REST, batching) y las alternativas modernas multi-framework: ONNX + ONNX Runtime (portable a cualquier framework / runtime), TensorRT (NVIDIA, máxima velocidad en GPU NVIDIA), y para LLMs: vLLM y TGI (Text Generation Inference) — específicos del caso autoregresivo con continuous batching.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Exportar un modelo Keras a SavedModel: `model.save('servable/1/', save_format='tf')`.
- Levantar TF Serving con Docker: `docker run -p 8501:8501 -v $PWD/servable:/models/m tensorflow/serving --model_name=m`.
- Hacer requests REST/gRPC.
- Exportar a ONNX con `tf2onnx` / `torch.onnx.export`, servir con ONNX Runtime.
- Conocer cuándo usar TensorRT (latencia mínima en GPU NVIDIA) o vLLM/TGI (LLMs).

Temas

- SavedModel format (TF nativo).
- TF Serving: configuración, versioning, model warm-up, batching.
- gRPC vs REST: gRPC más rápido (binary protobuf); REST más simple.
- Complemento moderno: ONNX/ONNX Runtime, TensorRT, vLLM/TGI.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 139a — ONNX y ONNX Runtime: portabilidad e inference optimizada

Definiciones y características

- SavedModel: formato TF para modelos servibles (variables + graph + signature).
- Signature: contrato del modelo (predict con inputs/outputs nombrados).

- TF Serving: server escrito en C++, batching automático, versioning.
- gRPC: protocolo binario sobre HTTP/2. Más rápido que REST.
- ONNX: formato intermedio standard.
- TensorRT: optimizador NVIDIA específico.
- Triton: servidor multi-framework.

Dataset / recursos

- Un modelo ya entrenado de las clases previas (Fashion-MNIST MLP).
- Librerías: tensorflow, tensorflow-serving-api, tf2onnx, onnxruntime, opcional tensorrt.

Ejercicios

1. Export SavedModel: `model.export('servable/1/')` (Keras 3). Inspeccionar la estructura `assets`, `variables`, `saved_model.pb`.
2. TF Serving con Docker: levantar el container con el modelo montado, hacer una request REST a `localhost:8501/v1/models/m:predict`.
3. ONNX export: convertir el TF model a ONNX con `tf2onnx`. Cargar con ONNX Runtime y verificar misma predicción.
4. vLLM: levantar vllm con `mistralai/Mistral-7B-Instruct`. Cliente OpenAI-style. Medir tokens/sec.
5. Latencia comparada: TF Serving REST vs gRPC vs ONNX Runtime CPU vs TensorRT GPU sobre el mismo modelo. Reportar latencia P50 y P99.

Homework verificable

Servir un modelo de Fashion-MNIST con TF Serving y comparar con ONNX:

1. `model.export('servable/1/')`.
2. Docker run TF Serving.
3. Convert TF → ONNX.
4. Inference con ambos sobre los mismos 100 inputs; verificar outputs idénticos ($\pm 1e-5$).
5. Comparar latencia.

Criterio de aceptación: predicciones de TF Serving y ONNX Runtime coinciden; latencia reportada para ambos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
TF Serving no encuentra modelo	Estructura de carpetas mal. Fix: <code>model_name</code>
Outputs cambian entre TF y ONNX	Conversión incompleta (capas custom). Fix:
TensorRT engine no portable entre GPUs	Es específico al hardware. Fix: rebuild en
vLLM no carga modelo cuantizado	Algunas quantizations no soportadas. Fix:
Latencia P99 alta	Sin warm-up. Fix: <code>warmup_count=10</code> en TF Se

Preguntas frecuentes

¿REST o gRPC?

gRPC para producción serio (más rápido). REST para debugging y compatibilidad universal.

¿ONNX es siempre más rápido que TF Serving?

No siempre. ONNX brilla en CPU + casos específicos. TF Serving está más optimizado para batching en GPU.

¿TensorRT necesario?

Solo si latencia es crítica. Para 99 % de los casos, ONNX Runtime + GPU es suficiente.

¿Cómo manejo versioning de modelos?

TF Serving por default sirve el "latest" entre las carpetas numeradas. Para A/B: serve varias versiones, configurar weights en model_config_list.

¿LLM en TF Serving o vLLM?

vLLM siempre para LLMs. TF Serving es ineficiente para autoregresivo.

Referencias

- Géron, cap. 19 — Deploying a Model to a Service.
- TF Serving docs.
- ONNX docs, ONNX Runtime.
- Triton Inference Server.
- vLLM docs, TGI docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 167 — Clase 167 — ONNX y ONNX Runtime: portabilidad e inference optimizada

Parte: 2 — Deep Learning · Fuente: ONNX docs + ONNX Runtime team blogs. Duración estimada: 80 min.

Objetivo

Dominar ONNX (formato intermedio) y ONNX Runtime (runtime cross-platform) — la solución portable para inference: entrenás en TF/PyTorch/JAX, exportás a ONNX, corrés en cualquier hardware (CPU, GPU NVIDIA, GPU AMD, mobile, browser, edge). Conocer TensorRT (NVIDIA-specific, mayor performance).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Exportar modelos: torch.onnx.export, tf2onnx.convert.
- Cargar e inferir con onnxruntime.InferenceSession.
- Elegir execution provider: CPU, CUDA, TensorRT, OpenVINO, DirectML, CoreML.
- Optimizar modelos: graph optimization, quantization int8/fp16.
- Reconocer cuándo ONNX vs TF Serving vs vLLM (LLMs).

Temas

- ONNX como protocolo (protobuf): operator set + tensor types.
- Conversión: cada framework tiene su exporter.
- Verificación: outputs deben coincidir framework ↔ ONNX ($\pm 1e-5$).
- Optimization: graph simplification, layer fusion.
- Quantization: dynamic, static (con calibration), QAT.
- Execution providers: priority order.

Definiciones y características

- ONNX: formato; archivo .onnx.
- Opset version: versión del set de ops. Higher = newer ops (ej. opset 17 incluye attention).
- InferenceSession: objeto runtime que carga modelo y ejecuta.
- Execution Provider (EP): backend de hardware. ORT elige el primero disponible.
- onnxruntime-gpu: pip package específico para CUDA.
- onnxruntime-directml: Windows GPU (AMD/Intel via DirectX).

Dataset / recursos

- Modelo de clases anteriores (Fashion-MNIST o ResNet preentrenado).
- Librerías: onnx, onnxruntime o onnxruntime-gpu, tf2onnx (TF), torch.onnx built-in.

Ejercicios

1. PyTorch → ONNX: `torch.onnx.export(model, dummy_input, 'model.onnx', opset_version=17, input_names=['input'], output_names=['output'])`.
2. TF → ONNX: `python -m tf2onnx.convert --saved-model dir/ --output model.onnx --opset 17`.
3. Inference: `sess = ort.InferenceSession('model.onnx', providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])`. Verificar outputs.
4. Graph optimization: `sess_options.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL`.
5. Quantization: `onnxruntime.quantization.quantize_dynamic('model.onnx', 'model_q.onnx', weight_type=QuantType.QInt8)`.

Homework verificable

Exportar y deployar un modelo CNN ya entrenado:

1. PyTorch ResNet50 preentrenado → ONNX.
2. Quantization dynamic.
3. Comparar latencia + accuracy: PyTorch CUDA vs ORT CUDA vs ORT CPU vs ORT CPU quantized.
4. Verificar correctness (output diff < 1e-4 vs PyTorch).

Criterio de aceptación: ORT CUDA ≥ PyTorch CUDA en velocidad; quantization reduce tamaño 4× con < 1 pp accuracy loss.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
opset version X not supported	EP version vieja. Fix: actualizar onnxrunt
Output diff vs framework grande	Conversión incompleta (custom ops). Fix: v
CUDA provider no disponible	Falta onnxruntime-gpu install. Fix: pip in
Dynamic axes mal	Mal export. Fix: <code>dynamic_axes={'input': {0</code>
Quantization rompe accuracy	Modelo sensible. Fix: usar static quantiza

Preguntas frecuentes

ONNX vs TensorRT?

ONNX Runtime + TensorRT EP combina ambos: ORT como interfaz, TensorRT como backend para GPUs NVIDIA. Best of both.

ONNX en mobile?

ONNX Runtime Mobile: optimizado para Android/iOS. Soporta CoreML, NNAPI delegates.

ONNX en browser?

ONNX.js — corre en WebGL/WebGPU/WASM.

LLMs con ONNX?

Posible (especialmente con onnxruntime-genai). vLLM/TGI siguen siendo mejores para LLMs grandes.

Triton vs ORT?

Triton (NVIDIA) puede servir modelos ONNX como uno de sus backends. Triton para infra multi-modelo; ORT solo para inference.

Referencias

- ONNX.
- ONNX Runtime.
- ONNX Runtime Mobile.
- ONNX Runtime GenAI.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 168 — Clase 168 — Despliegue en Vertex AI

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Deploying a Model to Vertex AI + docs Vertex AI.
Duración estimada: 60 min.*

Objetivo

Desplegar un modelo a Vertex AI (GCP) — el servicio managed de Google para servir modelos sin mantener infraestructura. Conocer alternativas: AWS SageMaker, Azure ML, Modal, Replicate, HuggingFace Inference Endpoints.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Subir un modelo a Vertex AI Model Registry.
- Crear un Endpoint y deployar el modelo.
- Hacer requests a un endpoint Vertex AI.
- Comparar managed (Vertex/SageMaker) vs self-hosted (TF Serving + GKE/EKS).
- Conocer alternativas modernas (Modal, Replicate) para deploy serverless.

Temas

- Vertex AI Model Registry.
- Endpoint creation + traffic split (A/B testing).
- gcloud CLI: gcloud ai models upload, gcloud ai endpoints deploy-model.
- Pricing: pay per CPU/GPU hour + requests.
- Alternativas: SageMaker, Azure ML, Modal, Replicate.

Definiciones y características

- Model Registry: catálogo de modelos versionados.
- Endpoint: URL HTTP para inference.
- Traffic split: routing parcial entre versiones (A/B).
- Auto-scaling: replicas automáticas según carga.
- Prebuilt containers: TF/PyTorch/sklearn containers oficiales.

Dataset / recursos

- Modelo de clases previas exportado.
- GCP account con billing habilitado (free tier limitado).
- Librerías: google-cloud-aiplatform.

Ejercicios

1. Setup: gcloud init, gcloud auth application-default login. Crear bucket GCS.
2. Upload model: gcloud ai models upload --display-name=fashion --container-image-uri=....
3. Deploy endpoint: con n1-standard-4. Min/max replicas 1-3.
4. Predict request: from google.cloud import aiplatform; ep = aiplatform.Endpoint(...); ep.predict(...).
5. A/B traffic: deploy v2 con 20 % traffic, observar logs.

Homework verificable

Deploy del modelo Fashion-MNIST a Vertex AI:

1. Subir a GCS.
2. Upload model en Vertex Model Registry.
3. Crear endpoint y deploy.
4. Hacer 10 predicciones desde notebook local.
5. (Opcional) cleanup para no gastar.

Criterio de aceptación: predicciones llegan correctamente; cost report < \$1.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
PERMISSION_DENIED al deploy	IAM mal configurado. Fix: rol aiplatform.a
Endpoint queda corriendo y factura \$\$\$	Olvidaste undeploy. Fix: gcloud ai endpoin
Modelo no carga	Formato no soportado. Fix: SavedModel form
Predicciones lentas (cold start)	Auto-scale a 0 y arranque toma 30-60s. Fix
Output incomprensible	Signature mal definida. Fix: documentar in

Preguntas frecuentes

¿Vertex AI o SageMaker?

Depende del ecosistema. Si ya estás en AWS → SageMaker. En GCP → Vertex. Funcionalmente similares.

¿Modal / Replicate cuándo?

Para deployment rápido (serverless, pay-per-request) y para LLMs/diffusion específicamente. Modal es excelente DX, Replicate tiene modelos pre-built.

¿Self-host con K8s en lugar de managed?

Si tenés equipo de ops y volumen alto, sale más barato. Para empezar o equipos chicos, managed gana.

¿Costos típicos?

Endpoint con n1-standard-4 24/7 ≈ \$100/mes. Con GPU T4 ≈ \$250/mes. Si auto-scale a 0 entre requests, mucho menos.

¿Inferencia batch para datasets grandes?

gcloud ai batch-predict-jobs create — más barato que endpoint para volúmenes grandes sin necesidad real-time.

Referencias

- Géron, cap. 19 — Deploying a Model to Vertex AI.
- Vertex AI docs.
- SageMaker docs.
- Modal, Replicate.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 169 — Clase 169 — TF Lite (mobile/embedded)

Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Deploying a Model to a Mobile or Embedded Device. Duración estimada: 55 min.

Objetivo

Convertir y desplegar un modelo a TensorFlow Lite (LiteRT) — runtime optimizado para móviles (Android/iOS), IoT y embedded (Raspberry Pi, microcontroladores). Aplicar quantization (int8) para reducir tamaño 4× y acelerar 2-4× en CPU móvil.

Nota: TF Lite fue renombrado a LiteRT en 2024 (mismo proyecto, nuevo branding bajo el paraguas de "AI Edge").

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Convertir SavedModel a .tflite: `converter = tf.lite.TFLiteConverter.from_saved_model('servable/1/'); tflite_model = converter.convert().`
- Aplicar quantization post-training (dynamic range, int8 full).
- Cargar y ejecutar inference con `tf.lite.Interpreter`.
- Conocer alternativas modernas: ONNX Runtime Mobile, CoreML (iOS), NNAPI (Android).
- Reconocer trade-off accuracy vs tamaño/velocidad.

Temas

- TF Lite vs TF: subset de ops, runtime minimal.
- Conversión SavedModel → tflite.
- Quantization types: dynamic range (weights int8), int8 full (weights + activations), float16.
- `tf.lite.Interpreter` API.
- Mobile delegates: NNAPI, GPU, CoreML.

Definiciones y características

- TF Lite: runtime para edge devices.
- Quantization: convertir float32 → int8/float16 → modelo más chico y rápido.
- Calibration dataset: ~100 ejemplos para calibrar quantization de activaciones.
- Delegate: hardware-specific accelerator (NNAPI en Android, CoreML en iOS, Edge TPU).
- .tflite: archivo binario compacto.

Dataset / recursos

- Modelo Fashion-MNIST.
- Librerías: tensorflow.

Ejercicios

1. Convert: `TFLiteConverter.from_saved_model(...).convert()` → guardar .tflite. Comparar tamaño vs SavedModel.
2. Inference: cargar .tflite y hacer predict con Interpreter. Verificar misma predicción que el modelo original.
3. Dynamic range quant: `converter.optimizations = [tf.lite.Optimize.DEFAULT]`. Tamaño 4× menor.
4. Full int8: definir `representative_dataset` y `converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]`. Comparar tamaño y accuracy.
5. Latencia: medir tiempo de inference en CPU laptop. Comparar versión float32 vs int8.

Homework verificable

Pipeline de Fashion-MNIST a TF Lite:

1. Convertir + 3 quantizations (none, dynamic, int8 full).
2. Para cada uno: tamaño + accuracy en test + latencia.
3. Reportar tabla comparativa.

Criterio de aceptación: int8 reduce tamaño ~4× con < 1 pp accuracy loss; latencia 2-3× más rápida.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
op not supported al convertir	Algunas ops Keras no son TFLite-compatible
int8 full quant con representative dataset	Mala calibración. Fix: ≥ 100 ejemplos dive
Tamaño del .tflite no baja con quant	El modelo era pequeño. Quantization brilla
Inference más lenta de lo esperado	Sin delegate. Fix: NNAPI (Android), CoreML
Predicciones distintas float vs int8	Esperable; verificar accuracy en test set.

Preguntas frecuentes

¿TFLite o ONNX Runtime Mobile?

TFLite si entrenas en TF; ONNX para portabilidad multi-framework. Funcionalmente comparable.

¿iOS — TFLite o CoreML?

CoreML mejor integración con iOS (Neural Engine de Apple). Convertir TFLite → CoreML con `coremltools`.

¿Microcontroladores ARM Cortex-M?

TF Lite Micro (TFLM) o MicroPython TF — para inference en MCU con KB de RAM. Modelos diminutos (< 100 KB).

¿Quantization durante training?

QAT (Quantization-Aware Training) — entrenar simulando los efectos de int8. Mejor accuracy que post-training quant.

¿Cuándo NO desplegar a mobile?

Modelos grandes (>100 MB) o que requieren GPU server (LLMs grandes). Para LLMs en mobile, ver MLC LLM o llama.cpp con quantization GGUF.

Referencias

- Géron, cap. 19 — Deploying a Model to a Mobile or Embedded Device.
- TensorFlow Lite (LiteRT) docs.
- ONNX Runtime Mobile.
- CoreML Tools.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 170 — Clase 170 — TensorFlow.js (navegador)

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Running a Model in a Web Page + TF.js docs.
Duración estimada: 55 min.*

Objetivo

Servir modelos directamente en el navegador con TensorFlow.js — corre client-side (privacidad, sin server cost, sin latency network). Alternativas modernas: ONNX Runtime Web, WebGPU-accelerated inference, transformers.js (Hugging Face) para LLMs en el browser.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Convertir un Keras model a TF.js format con tensorflowjs_converter.
- Cargar y hacer inference desde JavaScript en el browser.
- Conocer WebGL backend (default TF.js) y WebGPU (nuevo, más rápido).
- Usar transformers.js para correr modelos NLP/visión en browser.
- Reconocer cuándo conviene client-side vs server-side.

Temas

- Conversion: tensorflowjs_converter --input_format=keras model.keras tfjs_model/.
- JS API: const model = await tf.loadLayersModel('tfjs_model/model.json').
- Backends: WebGL (default), WASM, WebGPU.
- transformers.js: ONNX en browser, soporta BERT, GPT-2, Whisper.
- Edge cases: tamaño del modelo (~5-50 MB ideal), latencia primer load.

Definiciones y características

- TF.js: implementación de TF en JavaScript.
- WebGL backend: usa GPU del browser, default.
- WebGPU: API moderna, 2-5× más rápida que WebGL.

- WASM backend: CPU-only pero portable.
- transformers.js: HuggingFace en browser, ONNX-based.

Dataset / recursos

- Modelo Fashion-MNIST.
- Librerías: tensorflowjs, tensorflowjs-converter.

Ejercicios

1. Convert: `tensorflowjs_converter --input_format=keras_saved_model servable/ tfjs_model/`. Inspeccionar archivos.
2. HTML page: pagina simple que carga model.json, dibuja una imagen 28×28 en canvas, predice.
3. WebGPU: `await tf.setBackend('webgpu')`. Comparar velocidad vs WebGL.
4. transformers.js: `import { pipeline } from '@xenova/transformers'`. Correr sentiment-analysis en browser. 1 línea.
5. PWA: empaquetar como Progressive Web App con service worker para offline inference.

Homework verificable

Demo de Fashion-MNIST en browser:

1. Convertir modelo a TF.js.
2. Página HTML con canvas donde el usuario dibuja.
3. Botón "Predict" → muestra clase + probabilidades.

Criterio de aceptación: la demo funciona en Chrome/Firefox; predicciones son razonables para dibujos sencillos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
model.json not found	Path mal o servidor no sirve archivos está
OOM en browser con modelo grande	TF.js no es para GB-scale models. Fix: mod
WebGL fail en algunos browsers	Falta WebGL2. Fix: fallback a WASM.
Predicciones diferentes vs server	Diferencia en preprocesamiento. Fix: hacer
Modelo carga lento	Sin caching. Fix: Service Worker + cache.

Preguntas frecuentes

¿TF.js o ONNX Runtime Web?

TF.js para modelos TF; ORT Web para portabilidad. ORT Web también soporta WebGPU.

¿LLMs en browser?

Sí — modelos chicos (DistilBERT, TinyLlama, Whisper tiny) con quantization. WebLLM proyecto usa MLC y WebGPU para Llama 2 7B en browser.

¿Privacidad?

Client-side = data nunca sale del navegador. Ideal para healthcare, finanzas.

¿WebAssembly (WASM) vs WebGL?

WASM es CPU portable, lento. WebGL usa GPU. WebGPU es el futuro (5× WebGL).

¿Cuándo NO usar client-side?

Modelos grandes (>100 MB), datasets propietarios (no quieres exponerlos), latencia inaceptable en primer load.

Referencias

- Géron, cap. 19 — Running a Model in a Web Page.
- TensorFlow.js docs.
- transformers.js.
- ONNX Runtime Web.
- MLC WebLLM.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 171 — Clase 171 — Aceleración con GPU

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Using GPUs to Speed Up Computations.
Duración estimada: 60 min.*

Objetivo

Configurar GPU para DL: drivers, CUDA, cuDNN, verificación. Conocer mixed precision (bfloat16/float16) que duplica throughput en GPUs modernas (Ampere/Hopper). Profilear con TensorBoard Profiler para identificar bottlenecks (data loading vs compute vs sync).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Verificar GPU: `tf.config.list_physical_devices('GPU')`, `torch.cuda.is_available()`.
- Activar mixed precision: `keras.mixed_precision.set_global_policy('mixed_float16')` (Volta+) o `'mixed_bfloat16'` (Ampere+).
- Limitar memoria GPU: `set_memory_growth(True)` para no consumir toda al inicio.
- Multi-GPU básico con `tf.distribute.MirroredStrategy` (clase 144 profundiza).
- Profilear con TensorBoard Profiler.

Temas

- CUDA toolkit + cuDNN versions matching.
- `nvidia-smi` para monitor.
- Mixed precision: float16 (Volta+, 16 GB max) vs bfloat16 (Ampere+, mismo exponente que float32, más estable).
- Memory growth vs allocated all.
- Profiling con TensorBoard.
- GPUs típicos en 2026: H100, H200 (server); RTX 5090 (consumer).

Definiciones y características

- CUDA: API de NVIDIA para GPU compute.
- cuDNN: librería de primitivas DL (Conv, RNN, MatMul).
- Mixed precision: usar float16/bfloat16 para forward+backward, float32 para weights master copy.

- mixed_float16: requiere loss scaling para evitar underflow.
- mixed_bfloat16: mismo rango que float32, no necesita scaling.
- TPU: ASIC de Google. Diferente API (XLA).

Dataset / recursos

- Modelo + dataset cualquier de las clases previas.
- Librerías: tensorflow, opcional nvidia-smi.

Ejercicios

1. GPU check: imprimir `tf.config.list_physical_devices('GPU')`, `tf.config.list_logical_devices('GPU')`, `nvidia-smi`.
2. Memory growth: `tf.config.experimental.set_memory_growth(gpu, True)` para evitar reservar 100 % al inicio.
3. Mixed precision: `keras.mixed_precision.set_global_policy('mixed_float16')`. Re-entrenar modelo. Verificar speedup (1.5-2× en V100; 2-3× en A100/H100).
4. Profile: `keras.callbacks.TensorBoard(log_dir=..., profile_batch=(5, 10))`. Abrir Profiler tab.
5. Bottleneck: si la GPU está al 30 %, el bottleneck es data loading. Optimizar pipeline (clase 109).

Homework verificable

Optimizar el training de un modelo:

1. Baseline con default config.
2. Activar mixed precision + memory_growth.
3. Profile y identificar bottleneck.
4. Aplicar fix (más prefetch, batch más grande, etc.).
5. Reportar speedup wall-clock.

Criterio de aceptación: speedup $\geq 1.5\times$ con mixed precision; identificado el bottleneck correctamente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Could not find cuda drivers	Drivers no instalados o versión incompatib
OOM al usar mixed precision	A veces el ahorro de memoria no es total.
mixed_float16 loss = NaN	Loss scaling automático debe estar activo.
GPU al 100 % CPU al 100 % también	Data loading no es el bottleneck → OK. Si
Múltiples GPU pero usa 1	No configuraste strategy. Fix: MirroredStr

Preguntas frecuentes

¿float16 o bfloat16?

bfloat16 si GPU lo soporta (Ampere+, A100/H100, TPU). Más estable, sin loss scaling. float16 en GPUs viejas (V100, T4).

¿GPU consumer (RTX) para DL profesional?

Sí — RTX 4090 / 5090 tienen 24 GB VRAM y excelente performance. Para LLMs grandes (>30B), considerar A100/H100.

¿Cuántas GPU necesito?

1 GPU para experimentos. Multi-GPU para datasets grandes (ImageNet) o modelos grandes (LLMs).

¿Apple Silicon (M-chips)?

TF tiene soporte vía tensorflow-metal. PyTorch vía MPS backend. Para experimentos chicos OK; producción serio sigue siendo NVIDIA o TPU.

¿GPU en cloud?

AWS p4/p5, GCP A2/A3, Lambda Labs, Vast.ai. Lambda/Vast son más baratos para experimentos.

Referencias

- Géron, cap. 19 — Using GPUs to Speed Up Computations.
- TF GPU guide.
- Mixed precision guide.
- NVIDIA H100 / Hopper architecture whitepaper.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 172 — Clase 172 — Entrenamiento multi-dispositivo, tf.distribute

*Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Training Models Across Multiple Devices.
Duración estimada: 75 min.*

Objetivo

Escalar el training a múltiples GPUs (y multi-nodos). Conocer las 3 estrategias TF: MirroredStrategy (1 nodo, varias GPUs), MultiWorkerMirroredStrategy (varios nodos), TPUStrategy. Conocer equivalentes PyTorch (DDP, FSDP) que son estándar para LLMs grandes.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar MirroredStrategy en un único nodo con N GPUs.
- Entender data parallelism: cada GPU procesa su mini-batch, gradients se promedian (all-reduce).
- Diferenciar de model parallelism (modelo dividido entre GPUs) y pipeline parallelism.
- Conocer FSDP (Fully Sharded Data Parallel) para modelos demasiado grandes para 1 GPU (LLMs).
- Saber que PyTorch Lightning abstrae todo esto cambiando un kwarg.

Temas

- Data parallelism: misma red replicada, distinto batch por GPU.
- Model parallelism: red dividida (tensor parallel, pipeline parallel).
- FSDP / DeepSpeed ZeRO: shard de parámetros, gradientes, optimizer states.
- TF: MirroredStrategy, MultiWorkerMirroredStrategy, TPUStrategy.
- PyTorch: DistributedDataParallel, FullyShardedDataParallel, DeepSpeed.
- Lightning / Accelerate: abstracciones de alto nivel.

Definiciones y características

- Data Parallelism: cada device procesa un slice del batch.
- All-reduce: operación colectiva que promedia gradients entre devices.
- MirroredStrategy: data parallelism single-node.

- MultiWorkerMirroredStrategy: data parallelism multi-node.
- FSDP: shards de parámetros entre devices — para modelos demasiado grandes.
- Gradient accumulation: simular batch grande acumulando gradients en GPUs chicas.

Dataset / recursos

- Acceso a ≥ 2 GPUs (Colab Pro+, cloud).
- Librerías: tensorflow, opcional torch.distributed.

Ejercicios

1. MirroredStrategy: `strategy = tf.distribute.MirroredStrategy(). with strategy.scope(): model = build_model(); model.compile(...). Entrenar.`
2. Batch effective: si tenés 4 GPUs y `batch_size=32`, el batch global es 128. Adjust LR (LR scales linearly with batch).
3. Gradient accumulation: simular `batch=512` acumulando 4 mini-batches de 128. Manual con custom training loop.
4. PyTorch DDP: comando `torchrun --nproc_per_node=4 train.py` con `DistributedDataParallel(model)`.
5. PyTorch Lightning: `trainer = L.Trainer(strategy='ddp', devices=4)`. Listo.

Homework verificable

Si tenés ≥ 2 GPUs:

1. Mismo modelo en single-GPU vs MirroredStrategy(2 GPUs).
2. Reportar wall-time por epoch.
3. Verificar que la accuracy final es similar.

Criterio de aceptación: speedup wall-clock $\approx 1.5-1.9\times$ con 2 GPUs (no es $2\times$ porque hay overhead de comm); accuracy similar.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MirroredStrategy no acelera	Bottleneck en data loading. Fix: optimizar
LR no ajustado con batch global mayor	Convergencia subóptima. Fix: <code>lr_new = lr_o</code>
OOM en multi-GPU con LLM grande	Data parallelism replica el modelo. Si el
All-reduce muy lento	Network bandwidth limitado en multi-nodo.
Mismatch en BatchNorm stats entre replicas	Sync BN: <code>tf.keras.layers.experimental.Sync</code>

Preguntas frecuentes

¿Multi-GPU cuándo necesario?

Dataset masivo (días en 1 GPU) o modelo grande (no entra en 1 GPU). Para experimentos chicos, 1 GPU.

¿FSDP o DeepSpeed?

Ambas implementan shard de pesos. FSDP es PyTorch nativo. DeepSpeed (Microsoft) tiene más features (ZeRO-Offload, CPU offloading).

¿Train LLM 70B en 1 nodo?

8× H100 (640 GB VRAM total) con FSDP es factible para Llama 70B. Cluster necesario para más grande.

¿Cloud para multi-GPU?

AWS p4d (8× A100), GCP A3 (8× H100). Caro (\$30-60/hora).

¿MultiWorkerMirroredStrategy real-world?

Sí, con TF_CONFIG env var y orquestación (K8s, Slurm, Vertex AI). Vertex AI hace esto transparente.

Referencias

- Géron, cap. 19 — Training Models Across Multiple Devices.
- tf.distribute guide.
- PyTorch DDP, FSDP.
- DeepSpeed.
- Rajbhandari et al. (2020), ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 173 — Clase 173 — JAX y Flax: el stack moderno de Google para DL

Parte: 2 — Deep Learning · Fuente: JAX docs + Flax NNX docs. Duración estimada: 85 min.

Objetivo

Aprender JAX (Google 2018) y Flax (NN library on top of JAX) — el stack que sostiene AlphaFold, Gemini, MaxText, AlphaCode y muchos modelos modernos. Cubrir jit, vmap, pmap, grad, transformaciones funcionales, y Flax NNX (la nueva API 2024, similar a PyTorch).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar JAX (NumPy + autodiff + XLA) de NumPy plano.
- Usar jax.jit para compilación XLA (2-10× speedup automático).
- Aplicar jax.vmap (vectorización automática) y jax.grad (autodiff funcional).
- Construir modelos con Flax NNX (API moderna similar a PyTorch).
- Reconocer cuándo elegir JAX sobre PyTorch (TPU, escala extrema, research).

Temas

- Functional programming: funciones puras, no mutación.
- jit, grad, vmap, pmap como transformaciones.
- XLA: compilación a hardware específico (CPU, GPU, TPU).
- PRNG explícito (jax.random.PRNGKey).
- Flax: NN library. Antes Linen (functional), ahora NNX (stateful, más parecido a PyTorch).
- Optax: optimizadores.

Definiciones y características

- jax.numpy: drop-in NumPy con autodiff + XLA.
- jit: compila la función a XLA. Primera vez lento, después rápido.
- grad: devuelve función que computa gradiente.
- vmap: vectoriza sobre un eje. Como agregar batch dim.
- pmap: paraleliza sobre dispositivos (multi-GPU/TPU).

- PRNGKey: random determinista. `jax.random.split(key)` para usar varias veces.
- NNX: API stateful de Flax 2024, reemplaza Linen.

Dataset / recursos

- Fashion-MNIST.
- Librerías: `jax`, `jaxlib`, `flax`, `optax`.

Ejercicios

1. JAX basic: `import jax.numpy as jnp; x = jnp.array([1.,2.,3.]); jnp.sum(x**2)`. Comparar contra NumPy.
2. `grad`: `grad_f = jax.grad(lambda x: x**3); grad_f(2.)` → 12.
3. `jit speedup`: definir función numérica, medir tiempo con y sin `@jax.jit`. ≥ 5× speedup.
4. `vmap`: función para una muestra → `vmap` para procesar batch.
5. Flax NNX MLP: definir modelo, training step, entrenar Fashion-MNIST.

Homework verificable

Re-entrenar Fashion-MNIST en JAX/Flax:

1. Modelo Flax NNX con 2 capas Dense.
2. `Optax adam(1e-3)`.
3. Training loop con `jit`.
4. Reportar accuracy + tiempo vs equivalente PyTorch.

Criterio de aceptación: accuracy ≥ 0.87; JIT activo (segunda llamada mucho más rápida que primera).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Primera ejecución muy lenta	Compilación XLA. Fix: paciencia, después e
TracerError	Mutación dentro de jit. Fix: pure function
OOM en TPU	Modelo grande sin sharding. Fix: <code>pmap</code> o <code>pj</code>
Random no determinista	Olvidaste <code>split</code> del PRNGKey. Fix: <code>key</code> , <code>sub</code>
Comparar PyTorch vs JAX wall-clock sin JIT	Sin JIT, JAX es lento. Fix: siempre con <code>@j</code>

Preguntas frecuentes

JAX o PyTorch?

PyTorch ecosystem es más grande. JAX brilla en research / TPU / extrema escala. Para 99 % de DL aplicado, PyTorch.

Linen o NNX?

NNX (2024) — más fácil, stateful, parecido a PyTorch. Linen es legacy.

TPU en cloud?

Google Cloud TPU v4/v5e/v5p. Vertex AI lo wrappea. Caro pero performance excelente para batch grande.

Hugging Face soporta JAX?

Sí, muchos modelos tienen versión JAX. Pero la mayoría de la actividad es PyTorch.

¿AlphaFold en JAX?

Sí. JAX brilla en numerical computing (física, chemistry, biology).

Referencias

- JAX docs.
- Flax docs.
- Optax docs.
- Bradbury et al. (2018), JAX: composable transformations of Python+NumPy programs.
- MaxText — LLM training en JAX.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 174 — Clase 174 — Entrenamiento a escala con Vertex AI

Parte: 2 — Deep Learning · Fuente: Géron, cap. 19 § Running Large Training Jobs on Vertex AI + docs Vertex AI Training. Duración estimada: 65 min.

Objetivo

Cierre del bloque de despliegue: lanzar training jobs a escala en Vertex AI (GCP managed) — cluster automático, GPUs/TPUs on-demand, hyperparameter tuning distribuido. Conocer alternativas: AWS SageMaker Training, Azure ML Jobs, y plataformas dedicadas a LLMs (Modal, Together AI).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Empaquetar un training script como Docker container.
- Lanzar un Custom Training Job en Vertex AI con `gcloud ai custom-jobs create`.
- Configurar HP tuning con Vertex AI Vizier.
- Usar TPU Pods para training distribuido extremo.
- Conocer alternativas cloud-agnostic (Modal, Together, RunPod).

Temas

- Custom container vs prebuilt container en Vertex.
- WorkerPoolSpec: master + workers + parameter servers.
- TPU pods para training a escala.
- Hyperparameter tuning con Vizier (Bayesian search).
- Costos: GPU/TPU hours.
- Alternativas: SageMaker, Modal, Together, RunPod, Lambda Labs.

Definiciones y características

- Custom Training Job: corre tu container en Vertex.
- WorkerPool: cluster spec (`machine_type`, `count`, `accelerator`).
- Vizier: black-box optimizer integrado para HP tuning.
- TPU: tensor processing unit, ASIC de Google.
- Spot instances / preemptible: ~70 % más barato pero pueden ser killed.

Dataset / recursos

- GCP account.
- Librerías: `google-cloud-aiplatform`.

Ejercicios

1. Dockerize: escribir Dockerfile con TF + tu training script.
2. Lanzar job: `aiplatform.CustomJob(display_name='exp1', worker_pool_specs=[...]).run()`.
3. HP tuning: HyperparameterTuningJob con Vizier; 20 trials.
4. Multi-GPU spec: `machine_type='a2-highgpu-4g'` (4× A100).
5. TensorBoard integration: Vertex TB para monitor.

Homework verificable

Lanzar un training job de Fashion-MNIST en Vertex:

1. Containerizar.
2. Job de 1 GPU (n1-standard-4 + T4).
3. Logs y output a GCS.
4. Verificar accuracy similar a entrenamiento local.

Criterio de aceptación: el job termina con `accuracy ≥ 0.87` y el modelo queda guardado en GCS.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Job falla al iniciar	Image no accesible. Fix: push a Artifact R
Olvidé apagar y factura \$1000	Vertex jobs son time-limited pero HP tunin
Logs no aparecen	Tarda 1-2 min en stream. Fix: paciencia o
Multi-GPU pero job usa 1	TF no detecta GPUs en el container. Fix: i
Spot interrumpe a mitad	Aceptable para experimentos. Fix: checkpoi

Preguntas frecuentes

¿Cuándo Vertex vs local?

Local para iteración. Cloud cuando: dataset no entra en RAM local, training > 1 día, HP tuning con muchos trials, multi-GPU/TPU.

¿TPU vs GPU?

TPU brilla en modelos TF/JAX grandes (LLMs, transformers de visión). GPU es más universal. PyTorch en TPU funciona (vía XLA) pero menos óptimo.

¿Spot/preemptible?

Para training con checkpointing, ahorra 70 %. Para inference o jobs cortos, evitar.

¿Modal vs Vertex?

Modal: DX excelente, billing por segundo, ideal para LLMs e inference serverless. Vertex: más enterprise features (compliance, IAM, audit).

¿Together AI / Replicate cuándo?

Cuando solo necesitás API a modelos open-source preentrenados (LLMs, difusión). No para training desde cero.

Referencias

- Géron, cap. 19 — Running Large Training Jobs on Vertex AI.

- Vertex AI Training docs.
- SageMaker Training.
- Modal, Together AI, RunPod.

Siguiente parte

Clase 175 — Distribuciones: normal, binomial, Poisson, exponencial

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Parte 3 — Parte 3 — Estadística Inferencial y Causal

19 clases en esta parte.

Clase 175 — Clase 175 — Distribuciones: normal, binomial, Poisson, exponencial

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 2 + Bruce & Bruce, cap. 2 Data and Sampling Distributions. Duración estimada: 70 min.

Objetivo

Reconocer las cuatro distribuciones de probabilidad que aparecen en el 90 % de los problemas reales de data science —normal, binomial, Poisson, exponencial— sabiendo qué fenómeno modela cada una, cuáles son sus parámetros, cómo simularlas con `scipy.stats` / `numpy.random`, y cómo verificar empíricamente si los datos realmente siguen esa distribución antes de aplicar un test que la asuma.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Identificar la distribución apropiada para un fenómeno descrito en lenguaje natural (conteos raros → Poisson, éxitos/fracasos → binomial, tiempos entre eventos → exponencial, sumas/promedios → normal por TCL).
- Calcular media, varianza y cuantiles teóricos con `scipy.stats.{norm, binom, poisson, expon}` (`.mean()`, `.var()`, `.ppf()`, `.pdf()`/`.pmf()`).
- Simular muestras con `rng = np.random.default_rng(seed)` y comparar histograma vs PDF/PMF teórica.
- Aplicar un Q-Q plot (`scipy.stats.probplot`) y un Kolmogorov-Smirnov (`scipy.stats.kstest`) para validar normalidad.
- Reconocer cuándo el Teorema Central del Límite justifica usar normal aunque los datos crudos no lo sean.

Temas

#	Tema	Por qué importa
1	Distribución normal $N(\mu, \sigma^2)$	Base de t-test, ANOVA, intervalos de confi
2	Distribución binomial $\text{Bin}(n, p)$	Conversiones A/B, click-through rate, prop
3	Distribución de Poisson $\text{Poi}(\lambda)$	Eventos raros por unidad de tiempo/área (f

4	Distribución exponencial $\text{Exp}(\lambda)$	Tiempos entre eventos Poisson (churn, time)
5	Teorema Central del Límite (TCL)	Por qué la normal aparece aunque los datos
6	Verificación empírica: Q-Q plot + KS test	Antes de asumir, mirá.

Definiciones y características

- PDF (Probability Density Function): para variables continuas. $f(x)$ no es probabilidad; la probabilidad es $\int f(x) dx$ sobre un intervalo. $f(x)$ puede ser > 1 .
- PMF (Probability Mass Function): para variables discretas. $P(X = k)$ directo. Siempre en $[0, 1]$.
- CDF (Cumulative Distribution Function) $F(x) = P(X \leq x)$: en `scipy .cdf()`. Su inversa es `.ppf()` (quantile function), útil para construir intervalos.
- Normal $N(\mu, \sigma^2)$: simétrica, soporte en $\mathbb{R}$. Regla 68-95-99.7 ($1\sigma, 2\sigma, 3\sigma$). Es la única distribución cuya suma de variables independientes sigue siendo de la misma familia exacta.
- Binomial $\text{Bin}(n, p)$: suma de n ensayos Bernoulli independientes con éxito p . $E[X]=np$, $\text{Var}[X]=np(1-p)$. Para n grande y p no extrema, $\approx N(np, np(1-p))$ — esto es lo que justifica los z-tests de proporciones.
- Poisson $\text{Poi}(\lambda)$: conteo de eventos raros independientes en un intervalo fijo. $E[X]=\text{Var}[X]=\lambda$ (¡equidispersión!). Si tus datos tienen $\text{Var}/\text{Mean} > 1$, hay sobre-dispersión y Poisson no aplica — considerar binomial negativa.
- Exponencial $\text{Exp}(\lambda)$: tiempo entre eventos Poisson. Memoryless: $P(X > s+t \mid X > s) = P(X > t)$. Por eso modela mal cosas con desgaste (un motor de 10 años no falla igual que uno nuevo).
- TCL: si $X_1, \dots, X_n$ son i.i.d. con media μ y varianza finita σ^2 , entonces $(\bar{X} - \mu) / (\sigma/\sqrt{n}) \rightarrow N(0, 1)$ cuando $n \rightarrow \infty$. Regla práctica: $n \geq 30$ alcanza, salvo distribuciones muy asimétricas.
- Q-Q plot: scatter de cuantiles muestrales vs cuantiles teóricos. Si los puntos caen sobre la diagonal, los datos siguen la distribución.

Dataset / recursos

- Conteo de llamados a un call center por hora (sintético): `rng.poisson(lam=4.2, size=10_000)` $\rightarrow$ Poisson.
- Datos reales: `seaborn.load_dataset("tips")` para chequear normalidad de `total_bill` (no es normal, asimétrico positivo — buen contraejemplo).
- Librerías: `numpy`, `scipy.stats`, `matplotlib`, `seaborn`.

Ejercicios

1. Simulación y PDF/PMF: con `rng = np.random.default_rng(42)`, generá 10 000 muestras de cada una de las 4 distribuciones con parámetros razonables. Para cada una: histograma con `density=True` superpuesto con la PDF/PMF teórica de `scipy.stats`.
2. Cuantiles: calculá `scipy.stats.norm(loc=100, scale=15).ppf([0.025, 0.5, 0.975])` (IQ test $\rightarrow$ IC 95 % poblacional) y verificá que el 2.5 % y 97.5 % muestrales de una simulación con `n=100_000` se acerquen.
3. TCL en acción: tomá $\text{Exp}(\lambda=1)$ (claramente no normal). Generá 5 000 promedios de tamaños $n \in \{1, 5, 30, 100\}$ y graficá los 4 histogramas lado a lado. Verificá cómo se va volviendo simétrico y campaniforme.
4. Q-Q plot: `scipy.stats.probplot(tips.total_bill, dist='norm', plot=plt)`. Anotá qué muestra el extremo derecho (asimetría positiva $\rightarrow$ cola larga arriba de la diagonal).
5. ¿Poisson o no?: con los conteos por hora del dataset sintético, calculá `mean()` y `var()`. Si `var/mean` $[0.8, 1.2]$, equidispersión $\rightarrow$ Poisson plausible. Probá con `lam=4.2` (deberías ver `ratio \approx 1`) y con datos contaminados (mezclá con `rng.poisson(20, size=200)` para ver overdispersión).

Homework verificable

Notebook que:

1. Carga `tips.total_bill` de `seaborn`.
2. Genera un Q-Q plot contra 'norm' y otro contra 'lognorm'.
3. Aplica `scipy.stats.kstest` contra ambas (estandarizando los datos).
4. Concluye por escrito qué distribución modela mejor `total_bill` y por qué (≤ 3 líneas).

Criterio de aceptación: el p-value del KS contra lognormal debe ser mayor que contra normal, y la conclusión debe mencionar que `total_bill` está acotado por abajo en 0 y tiene cola derecha — incompatible con normal.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>kstest</code> da <code>p=0.0</code> aunque los datos "se ven n	Probablemente no estandarizaste. KS contra
Histograma no coincide con la PDF teórica	Olvidaste <code>density=True</code> en <code>plt.hist</code> . Sin es
Aplico Poisson y la varianza es mucho mayo	Sobre-dispersión: hay heterogeneidad oculta
<code>np.random.seed(42)</code> no me da resultados rep	El estado global está deprecado para análi
Asumo normalidad con <code>n=8</code> porque "el TCL lo	El TCL es asintótico. Con <code>n</code> chico y distri

Preguntas frecuentes

¿Cuándo uso `scipy.stats.norm(loc, scale)` vs `np.random.normal(mu, sigma)`?

Para simular muestras: ambos funcionan, pero `rng = np.random.default_rng(seed)`; `rng.normal(...)` es el patrón moderno reproducible. Para calcular PDF, CDF, cuantiles: `scipy.stats.norm`, que es un objeto distribución con todos los métodos.

¿Poisson y binomial con `n` grande y `p` chica se parecen?

Sí: si $n \rightarrow \infty$ y $p \rightarrow 0$ con $np = \lambda$ constante, $\text{Bin}(n, p) \rightarrow \text{Poi}(\lambda)$. Por eso "1 cada 1000" se modela igual con cualquiera de las dos. En la práctica, si $n \geq 100$ y $p \leq 0.05$, son intercambiables.

¿La distribución t-Student es lo mismo que la normal?

No, pero converge. $t(v)$ con $v \rightarrow \infty$ tiende a $N(0,1)$. Para $v \geq 30$ son visualmente idénticas. La diferencia importa en muestras chicas (la t tiene colas más pesadas, lo que produce intervalos de confianza más anchos — correcto cuando estimas σ).

¿Mis datos tienen que ser normales para hacer un t-test?

No exactamente. Lo que tiene que ser $\approx$ normal es la distribución muestral de la media, y por TCL eso pasa con $n \geq 30$ aunque los datos crudos no lo sean. Si $n < 30$ y los datos están sesgados, usá bootstrap (Clase 153) o Mann-Whitney (Clase 150).

¿Por qué exponencial es "sin memoria"?

Porque $P(X > s + t \mid X > s) = P(X > t)$. Aplicado a un servidor que lleva 3 h sin caer: la probabilidad de aguantar otra hora es la misma que la de aguantar una hora desde 0. Para sistemas con desgaste (motores, humanos), usá Weibull o Gamma.

Referencias

- ISLP (James et al.), cap. 2 — Statistical Learning, sección sobre distribuciones.
- Bruce, P. & Bruce, A. Practical Statistics for Data Scientists (2ª ed., O'Reilly), cap. 2 Data and Sampling Distributions.
- `scipy.stats` reference — objetos `norm`, `binom`, `poisson`, `expon`.
- `numpy.random.Generator` — API moderna recomendada desde NumPy 1.17.

- 3Blue1Brown — Why π appears in the normal distribution (intuición visual del TCL).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 176 — Clase 176 — Test t (una muestra, dos muestras, pareado)

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 13 + Bruce & Bruce, cap. 3 Statistical Experiments and Significance Testing. Duración estimada: 80 min.

Objetivo

Que el alumno aplique correctamente las tres variantes del test t —una muestra, dos muestras independientes (Welch por default), pareado—, distinga hipótesis nula y alternativa, lea p-value e intervalo de confianza de la salida de `scipy.stats` y `pingouin`, y aprenda a reportar effect size (Cohen's d, Hedges' g) junto con el p-value para no caer en la trampa de "significativo pero irrelevante".

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Formular H_0 y H_1 (bilateral / unilateral) para un problema concreto y elegir la variante correcta del test t.
- Ejecutar `scipy.stats.ttest_1samp`, `ttest_ind(equal_var=False)` y `ttest_rel`, interpretando el statistic, pvalue y el atributo `.confidence_interval()` (`scipy` ≥ 1.10).
- Verificar supuestos: normalidad por grupo (Shapiro / Q-Q plot) o invocar TCL si $n \geq 30$.
- Decidir entre test bilateral vs unilateral sin caer en p-hacking (la dirección debe estar fijada antes de mirar los datos).
- Reportar effect size: Cohen's d, Hedges' g corregido para muestras chicas, y su interpretación cualitativa (small/medium/large).

Temas

- H_0 / H_1 , errores tipo I (α) y tipo II (β).
- Test t de una muestra: $t = (\bar{x} - \mu) / (s/\sqrt{n})$, $gl = n - 1$.
- Test t de dos muestras independientes: Welch (varianzas distintas, default moderno) vs Student (varianzas iguales — supuesto fuerte, casi nunca correcto).
- Test t pareado (mismo sujeto antes/después).
- p-value: probabilidad bajo H_0 de observar algo al menos tan extremo. NO es $P(H_1 | \text{datos})$.
- Intervalo de confianza al 95 % como complemento del p-value.
- Complemento moderno: effect size (Cohen's d, Hedges' g, Cliff's δ) — la pregunta "¿es relevante?" que el p-value no responde.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 147a — Effect size dedicado: Cohen's d, Hedges' g, Cliff's δ con `pingouin`

Definiciones y características

- Hipótesis nula H_0 : la afirmación "conservadora" — no hay diferencia, o la diferencia es 0. Se busca

rechazarla, no probarla.

- Hipótesis alternativa H: lo que querés demostrar. Bilateral ($\neq$) o unilateral ($>$, $<$).
- p-value: P(estadístico al menos tan extremo como el observado | H verdadera). Si $p < \alpha$, rechazo H. NO es la probabilidad de que H sea verdadera.
- α (alpha): probabilidad máxima de error tipo I (rechazar H siendo verdadera). Convencional: 0.05. Para problemas críticos: 0.01 o 0.001.
- β (beta): probabilidad de error tipo II (no rechazar H siendo falsa). Poder = $1 - \beta$ (Clase 154).
- Welch's t-test: variante que no asume varianzas iguales. Es el default razonable. `equal_var=True` (Student clásico) es legacy.
- Test pareado: cuando cada observación de un grupo tiene su "par" en el otro (mismo paciente antes/después, misma página web con/sin cambio). Reduce varianza al diferenciar dentro del par.
- Cohen's d: magnitud estandarizada de la diferencia. $d = 0.2/0.5/0.8 \rightarrow \text{small/medium/large}$.
- `alternative='greater'`: unilateral derecho. Solo legítimo si la dirección la fijaste antes de ver los datos.

Dataset / recursos

- `seaborn.load_dataset('tips')`: comparar tip entre `sex='Male'` vs `'Female'`, o `time='Lunch'` vs `'Dinner'`.
- Para pareado: simular un dataset antes/después con `numpy.random.default_rng` (presión arterial pre/post fármaco).
- Librerías: `scipy.stats`, `pingouin` (pip install pingouin), `seaborn`.

Ejercicios

1. Una muestra: con `tips.total_bill`, testá $H: \mu = 20$ vs $H: \mu \neq 20$ con `scipy.stats.ttest_1samp(tips.total_bill, popmean=20)`. Reportá t, p y el IC95 % (`.confidence_interval()`).
2. Dos muestras (Welch): testá si tip difiere entre `sex='Male'` y `sex='Female'` con `ttest_ind(equal_var=False)`. Calculá Cohen's d a mano y verificá contra `pingouin.ttest`.
3. Pareado: simulá presión arterial antes/después de un fármaco con `rng = np.random.default_rng(0)`: `antes = rng.normal(140, 12, 30)`, `despues = antes - rng.normal(5, 3, 30)`. Aplicá `ttest_rel(antes, despues)` y comparalo contra hacer `ttest_ind` mal (verás cómo el pareado tiene mucho más poder).
4. Bilateral vs unilateral: para el ejercicio 2, repetí con `alternative='greater'` y `'less'`. Observá cómo el p-value se divide ≈ 2 .
5. Significativo vs relevante: generá `grupo_a = rng.normal(100, 15, 10_000)` y `grupo_b = rng.normal(100.5, 15, 10_000)`. El test va a dar $p < 0.001$; calculá Cohen's d y discutí en 2 líneas por qué el resultado "no importa".

Homework verificable

Sobre tips:

1. Hipótesis: la propina promedio es distinta para `time='Lunch'` y `time='Dinner'`.
2. Verificar normalidad de cada grupo con `pingouin.normality` (Shapiro).
3. Ejecutar `pingouin.ttest` y reportar: T, gl, p-value, IC95 %, Cohen's d, power.
4. Una conclusión de 3 líneas que mencione (a) si rechazás H, (b) la magnitud del efecto en palabras (small/medium/large), (c) si recomendarías ese hallazgo a un dueño de restaurante.

Criterio de aceptación: el reporte debe incluir effect size y la conclusión no puede limitarse a " $p < 0.05$ ". Si Cohen's d es < 0.2 , la respuesta a (c) debería ser "no" aunque el p sea bajo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Aplico <code>ttest_ind</code> con <code>equal_var=True</code> por co	Student clásico con varianzas desiguales y

p < 0.05 con n = 10 ⁶ y declaro un hallazgo	"Significancia estadística" con muestra gi
Cambio alternative='greater' después de ve	Eso es p-hacking. La dirección se fija con
Aplico ttest_ind a observaciones pareadas	Pierde poder enormemente y puede dar p > 0
El test asume normalidad y mis datos son m	Welch es robusto pero no mágico. Fix: scip

Preguntas frecuentes

¿Welch o Student por default?

Welch siempre. No cuesta nada en poder cuando las varianzas son iguales, y protege contra el caso (muy común) de varianzas distintas. La literatura moderna (Delacre, Lakens & Leys 2017) recomienda abandonar el Student t-test.

¿Tengo que testear normalidad antes del t-test?

No con $n \geq 30$ por grupo (TCL). Con n chico y datos visiblemente asimétricos, sí — y si Shapiro rechaza, mejor pasar a bootstrap o Mann-Whitney. Cuidado: con n enorme, Shapiro rechaza siempre por desviaciones triviales; mirá Q-Q plot como complemento.

¿Qué es BF10 que reporta pingouin?

Factor de Bayes contra H. $BF_{10} > 3$ es evidencia moderada a favor de H; > 10 fuerte. Es la versión bayesiana del p-value y no tiene los problemas del NHST (la testeás directamente en la Clase 158).

¿Por qué el IC95 % de la diferencia y el p-value siempre concuerdan?

Porque son la misma información presentada de otra forma. Si el IC95 % de $\mu_a - \mu_b$ no incluye 0, entonces $p < 0.05$ (bilateral). Reportar el IC es más informativo: muestra magnitud y dirección.

Cohen's d me da 0.3, ¿cómo lo explico al cliente?

"Un efecto pequeño-mediano". Operacionalmente: si dibujás las dos distribuciones, el $\approx 55\%$ de los individuos del grupo tratamiento superan a la mediana del grupo control (vs 50% bajo H). Para una conversión, tradúcelo a "incremento absoluto de X puntos porcentuales".

Referencias

- ISLP, cap. 13 — Multiple Testing, intro al p-value.
- Bruce & Bruce, cap. 3 — Statistical Experiments and Significance Testing.
- Delacre, Lakens & Leys (2017), Why Psychologists Should by Default Use Welch's t-test, IRSP.
- Cohen, J. (1988), Statistical Power Analysis for the Behavioral Sciences — referencia canónica de effect size.
- pingouin docs — Vallat, R. (2018), Pingouin: statistics in Python, JOSS.
- scipy.stats.ttest_ind — atención a equal_var y alternative.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 177 — Clase 177 — Effect size dedicado: Cohen's d, Hedges' g, Cliff's δ con pingouin

Parte: 3 — Estadística Inferencial y Causal · Fuente: Cohen (1988) + Lakens (2013) + Vallat (2018) pingouin. Duración estimada: 75 min.

Objetivo

Dominar effect size —la métrica que el p-value no responde: "cuán grande es la diferencia"—. Cubrir 6 medidas: Cohen's d (means, varianzas similares), Hedges' g (bias-corrected para n chico), Glass's Δ (varianza del control como denominador), Cliff's δ (no paramétrico), r de correlación, odds ratio. Aplicar con pingouin en una sola llamada. Reportar correctamente: APA 7 lo exige.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Calcular Cohen's d a mano y con pingouin.compute_effsize.
- Aplicar la corrección de Hedges (recomendada cuando $n < 50$ /grupo).
- Interpretar magnitudes (Cohen 1988): 0.2 / 0.5 / 0.8 = small / medium / large.
- Calcular Cliff's δ para datos ordinales / muy asimétricos.
- Reportar effect size con IC95 % bootstrap.
- Diseñar tabla APA-7 con mean $\pm$ SD, Cohen's d [95% CI], t, p.

Temas

- Cohen's d: $(\bar{x} - \bar{x}) / s_{\text{pooled}}$.
- Hedges' g: $d \cdot (1 - 3/(4 \cdot gl - 1))$.
- Glass's Δ : usar s_{control} como denominator. Útil cuando control y treatment tienen varianza distinta.
- Cliff's δ : probabilistic dominance.
- Effect size para correlación: r (= Pearson) o R^2 .
- Effect size para chi-cuadrado: Cramér's V, phi.
- IC95 % de effect size: bootstrap o fórmulas paramétricas.

Definiciones y características

- Cohen's d: estándar para 2 grupos independientes.
- s_{pooled} : $\sqrt{((n1-1) \cdot s1^2 + (n2-1) \cdot s2^2) / (n1+n2-2)}$.
- Hedges' g: corrección para muestras chicas; siempre menor que d.
- Cliff's δ : en [-1, 1]. Interpretación Romano (2006): < 0.147 negligible, < 0.33 small, < 0.474 medium, ≥ 0.474 large.
- CLES (Common Language Effect Size): $P(\text{rand } X > \text{rand } Y)$. Más interpretable para no-técnicos.

Dataset / recursos

- seaborn.load_dataset('tips').
- Librerías: pingouin, scipy.stats, numpy.

Ejercicios

1. Cohen's d a mano: para tip por sex, calcular manualmente con s_{pooled} .
2. pingouin.compute_effsize: verificar contra cálculo manual. Probar eftype='cohen' | 'hedges' | 'glass' | 'CLES'.
3. Hedges' g: con n=10 por grupo, ver diferencia entre d y g (Hedges $<$ d).
4. Cliff's δ : para datos Likert ordinales o muy asimétricos, calcular y interpretar.
5. Effect size + IC: bootstrap del Cohen's d $\rightarrow$ IC95 %.

Homework verificable

Análisis estadístico riguroso de tips:

1. Comparar tip por time (Lunch/Dinner) y por day.

2. Reportar Cohen's d (o Hedges' g si $n < 30$) con IC95 % bootstrap.
3. Para day (4 niveles), reportar partial η^2 del ANOVA.
4. Conclusión APA-7 estilo: "M $\pm$ SD, $t(df) = X$, $p = Y$, d [95% CI]".

Criterio de aceptación: tabla bien formada con todas las columnas; conclusión no usa solo p-value.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Reportar solo $p < 0.05$	Mala práctica desde 1999. Fix: agregar eff
Cohen's d sobre datos muy asimétricos	Sesgado. Fix: Cliff's δ .
Mismo Cohen's d en 2 estudios sin contexto	Magnitudes 0.2/0.5/0.8 son orientativas —
Effect size sin IC	Incompleto. Fix: bootstrap CI.
Confundir d con η^2	Distintos rangos y significados. Fix: d pa

Preguntas frecuentes

Cohen's d o Hedges' g?

Hedges siempre si $n < 50$ /grupo. Para n grande son indistinguibles.

Magnitudes 0.2/0.5/0.8 son universales?

No. Son orientación. En medicina pueden ser muy distintas. Reporte tu d, déja que el lector compare con su literatura.

CLES interpretable?

Sí — "65 % chance que una persona random del grupo A tenga mayor valor que una random del B". Comunicable a no-técnicos.

Effect size para A/B testing?

Sí — Cohen's h para proporciones, Cohen's d para continuous. Ver clase 154.

Effect size negativo?

Solo indica dirección. La magnitud $|d|$ es lo que se interpreta.

Referencias

- Cohen (1988), Statistical Power Analysis for the Behavioral Sciences.
- Lakens (2013), Calculating and reporting effect sizes to facilitate cumulative science, *Frontiers in Psychology*.
- Romano et al. (2006), Cliff's δ interpretation.
- pingouin docs.
- APA Publication Manual 7th ed.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 178 — Clase 178 — Test chi-cuadrado de independencia y bondad de ajuste

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 4 + Bruce & Bruce, cap. 3 Chi-Square

Test. Duración estimada: 70 min.

Objetivo

Aplicar el test chi-cuadrado de Pearson en sus dos formas: (a) independencia en una tabla de contingencia de dos variables categóricas, y (b) bondad de ajuste entre una distribución observada y una teórica. Reconocer cuándo el test es válido (frecuencias esperadas ≥ 5 por celda) y cuándo hay que recurrir a Fisher exact o a la simulación de Monte Carlo.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir una tabla de contingencia con `pd.crosstab` y aplicar `scipy.stats.chi2_contingency` interpretando `chi2`, `dof`, `pvalue` y `expected`.
- Verificar el supuesto de frecuencias esperadas mínimas (regla de Cochran: ≥ 5 en $\geq 80\%$ de celdas).
- Decidir entre chi-cuadrado, Fisher exact (`scipy.stats.fisher_exact`, tablas 2×2 con conteos chicos) y chi-cuadrado con simulación (`lambda_='log-likelihood'` o `montecarlo`).
- Calcular Cramér's V como effect size para tablas $r \times c$ (análogo al Cohen's d categórico).
- Aplicar bondad de ajuste con `scipy.stats.chisquare` para validar dados, ruedas de roulette o conteos en bins.

Temas

#	Tema	Por qué importa
1	Tablas de contingencia	Estructura natural de datos categóricos cr
2	Estadístico $\chi^2 = \sum (O - E)^2 / E$	Lo que mide el test: distancia entre obser
3	Grados de libertad $(r-1) \cdot (c-1)$	Determinan la distribución de referencia.
4	Supuesto de $E \geq 5$ (Cochran)	Si se viola, el p-value asintótico es inco
5	Fisher exact para 2×2 con n chico	Alternativa exacta cuando χ^2 no sirve.
6	Cramér's V	Effect size — qué tan fuerte es la asociac
7	Bondad de ajuste vs independencia	Misma fórmula, distinto problema.

Definiciones y características

- Tabla de contingencia: matriz $r \times c$ donde $O_{\{ij\}}$ es la frecuencia observada del cruce (fila i, columna j).
- Frecuencia esperada bajo independencia: $E_{\{ij\}} = (\text{row}_i_total \cdot \text{col}_j_total) / n$. Es lo que esperaríamos si las dos variables fueran independientes.
- Estadístico χ^2 : $\sum (O_{\{ij\}} - E_{\{ij\}})^2 / E_{\{ij\}}$. Sigue χ^2 con $(r-1) \cdot (c-1)$ gl si las E son suficientemente grandes.
- Test de independencia: H: las variables categóricas son independientes. Se aplica a una tabla cruzada de dos variables.
- Test de bondad de ajuste: H: la muestra proviene de una distribución teórica especificada. Las E vienen de la distribución teórica, no del producto de marginales.
- Cochran's rule: el test asintótico es válido si todas las $E \geq 1$ y al menos 80% de las celdas tienen $E \geq 5$. Si no, usar Fisher (2×2) o Monte Carlo ($> 2 \times 2$).
- Fisher exact test: calcula el p-value exacto enumerando todas las tablas con las mismas marginales. Computacionalmente caro para n grande.
- Cramér's V: $V = \sqrt{(\chi^2 / (n \cdot \min(r-1, c-1)))}$. Va de 0 a 1. Interpretación cualitativa con $\min(r-1, c-1) = 1$: 0.1 small, 0.3 medium, 0.5 large (Cohen 1988).
- Test de homogeneidad: matemáticamente idéntico al de independencia, pero el diseño muestral es distinto (se fijan los marginales de una variable). El cálculo y la interpretación práctica son los mismos.

Dataset / recursos

- `seaborn.load_dataset('titanic')`: cruzar `survived` × `class` o `survived` × `sex`.
- `seaborn.load_dataset('tips')`: `smoker` × `day`.
- Bondad de ajuste: simular tiradas de un dado posiblemente cargado y testear contra distribución uniforme.
- Librerías: `pandas`, `scipy.stats`, `pingouin` (que tiene `pg.chi2_independence` con Cramér's V incluido).

Ejercicios

1. Tabla cruzada: `pd.crosstab(titanic.survived, titanic['class'])`. Aplicá `chi2`, `p`, `dof`, `expected = scipy.stats.chi2_contingency(tabla)`. Reportá los cuatro valores e interpretá.
2. Effect size: calculá Cramér's V manualmente: $V = \sqrt{\chi^2 / (n * \min(r-1, c-1))}$. Verificá contra `pingouin.chi2_independence(titanic, x='survived', y='class')`.
3. Cochran check: imprimí la matriz `expected` y contá cuántas celdas tienen $E < 5$. Si supera el 20 %, recalculá con `chi2_contingency(tabla, lambda_='log-likelihood')` (G-test, mejor para celdas chicas).
4. Fisher exact (2×2): tomá la subtabla `survived` × `sex` y aplicá `scipy.stats.fisher_exact(tabla_2x2)`. Comparalo con chi-cuadrado.
5. Bondad de ajuste: simulá `rng = np.random.default_rng(7)`; `tiros = rng.choice([1,2,3,4,5,6], size=600, p=[0.18, 0.16, 0.17, 0.17, 0.16, 0.16])`. Hipótesis: el dado es justo (p uniforme). Aplicá `scipy.stats.chisquare(observado, f_exp=esperado)` con `esperado = [100]*6`. ¿Rechazás H?

Homework verificable

Notebook que sobre `titanic`:

1. Cruza `survived` × `class` y `survived` × `sex` por separado.
2. Para cada cruce: χ^2 , gl, p-value, Cramér's V.
3. Identifica cuál de los dos tiene asociación más fuerte (mayor V) y cuál tiene evidencia estadística más fuerte (menor p).
4. En 3 líneas, explica por qué p y V pueden ordenar distinto cuando n cambia entre comparaciones.

Criterio de aceptación: ambos tests deben rechazar H al 5 %; Cramér's V debe ser mayor para `sex` que para `class`; la conclusión debe distinguir tamaño de efecto de evidencia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>expected</code> tiene celdas con valores < 5 y re	El p-value asintótico no es confiable. Fix
Aplico χ^2 sobre una columna numérica con	χ^2 es solo para categóricas / conteos. F
Confundo "asociación" con "causalidad" por	χ^2 detecta dependencia estadística, no r
Reporto solo $p < 0.05$ con $n = 10^6$ y declar	Con n gigante, asociaciones triviales son
Bondad de ajuste con <code>f_exp</code> proporcional pe	<code>chisquare(obs, f_exp)</code> requiere que <code>f_exp.s</code>

Preguntas frecuentes

¿Cuándo Fisher exact y cuándo chi-cuadrado?

Fisher cuando alguna $E < 5$ en una tabla 2×2. Para tablas mayores, Fisher es costoso; mejor usar Monte Carlo simulation (`scipy.stats.chi2_contingency(..., method='monte-carlo')` desde `scipy 1.11`) o G-test.

¿Qué es la corrección de Yates?

Para tablas 2×2, resta 0.5 al $|O - E|$ antes de elevar al cuadrado. Hace el test más conservador.

`scipy.stats.chi2_contingency` la aplica por default en 2x2 (`correction=True`). En la práctica, con n moderado, casi no cambia el p-value.

¿Por qué $\text{dof} = (r-1) \cdot (c-1)$?

Porque al fijar los totales marginales ($r+c-1$ restricciones), solo $(r-1) \cdot (c-1)$ celdas son libres de variar — las otras quedan determinadas por aritmética.

¿G-test es mejor que chi-cuadrado?

Asintóticamente equivalentes, pero G-test tiene mejor comportamiento con celdas chicas y se generaliza mejor (es el log-likelihood ratio test). En `scipy`: `chi2_contingency(tabla, lambda_='log-likelihood')`.

¿Puedo usar chi-cuadrado para validar la salida de un modelo de clasificación?

Sí — `crosstab(y_true, y_pred)` da la confusion matrix, y un χ^2 sobre esa tabla testea si la predicción es independiente de la verdad (H : clasificador trivial). Pero ojo: prefieres métricas específicas (accuracy, F1, Kappa de Cohen) — χ^2 no captura el balance de clases.

Referencias

- ISLP, cap. 4 — Classification, parte sobre datos categóricos.
- Bruce & Bruce, cap. 3 — sección Chi-Square Test.
- Cochran, W.G. (1954), Some Methods for Strengthening the Common χ^2 Tests, Biometrics.
- `scipy.stats.chi2_contingency` y `fisher_exact`.
- `pingouin.chi2_independence` — reporta Cramér's V automáticamente.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 179 — Clase 179 — ANOVA (one-way, two-way)

*Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 13 + Bruce & Bruce, cap. 3 ANOVA.
Duración estimada: 80 min.*

Objetivo

Que el alumno aplique ANOVA de una vía (≥ 3 grupos, una variable categórica) y ANOVA de dos vías (dos factores categóricos + interacción), entienda por qué no se hacen "t-tests todos contra todos" (inflación de α) y sepa hacer post-hoc con Tukey HSD. Reconocer los supuestos (independencia, normalidad por grupo, homogeneidad de varianzas) y cuándo usar la alternativa robusta Welch ANOVA o el no paramétrico Kruskal-Wallis (Clase 150).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Plantear $H: \mu = \mu = \dots = \mu_k$ vs H : al menos uno difiere y aplicar `scipy.stats.f_oneway` o `pingouin.anova`.
- Interpretar $F = MS_{\text{between}} / MS_{\text{within}}$ y su relación con la F-distribution ($F(k-1, n-k)$).
- Aplicar Welch's ANOVA (`pingouin.welch_anova`) cuando se viola la homogeneidad de varianzas (Levene rechaza).
- Hacer Tukey HSD post-hoc con `pingouin.pairwise_tukey` y leer los IC ajustados.

- Distinguir efectos principales de interacción en ANOVA two-way y graficar interaction plots con `seaborn.pointplot`.
- Reportar η^2 o ω^2 como effect size de ANOVA.

Temas

- ¿Por qué no t-tests múltiples? Si hacés 10 t-tests al $\alpha=0.05$, la probabilidad de al menos un falso positivo es $\approx 40\%$.
- Descomposición de varianza: $SS_{total} = SS_{between} + SS_{within}$.
- F-statistic: razón entre varianza explicada por los grupos y varianza residual.
- Supuestos: independencia, normalidad (Shapiro por grupo o residuos), homocedasticidad (Levene/Bartlett).
- Welch ANOVA — análogo a Welch's t-test para ≥ 3 grupos.
- Post-hoc: Tukey HSD (controla family-wise error rate), Bonferroni, Holm.
- Two-way ANOVA: efectos principales A, B, e interacción A×B.
- Effect size: η^2 (eta-squared), ω^2 (omega-squared, menos sesgado).

Definiciones y características

- One-way ANOVA: testea si la media de una variable continua difiere entre los niveles de una variable categórica.
- Two-way ANOVA: dos variables categóricas. Permite testear 3 cosas: efecto principal de A, efecto principal de B, e interacción (¿el efecto de A depende del nivel de B?).
- F-statistic: $MS_{between} / MS_{within}$. Si los grupos tienen la misma media, ambos son estimadores de $\sigma^2 \rightarrow F \approx 1$. Si difieren, $MS_{between}$ crece.
- SS (sum of squares) within: variabilidad residual dentro de cada grupo. $\sum (x_{ij} - \bar{x}_i)^2$.
- SS between: variabilidad de las medias grupales respecto a la gran media. $\sum n_i \cdot (\bar{x}_i - \bar{x})^2$.
- Homocedasticidad: igualdad de varianzas entre grupos. Test: Levene (robusto), Bartlett (sensible a normalidad).
- Welch ANOVA: no asume homocedasticidad. Es el default razonable moderno, igual que Welch's t-test.
- Tukey HSD (Honestly Significant Difference): compara todas las parejas controlando el family-wise error rate al α global.
- Interacción: cuando el efecto de un factor cambia según el nivel del otro. En el plot, las líneas no son paralelas.
- η^2 (eta-squared): $SS_{between} / SS_{total}$. Proporción de varianza explicada. Sesgado hacia arriba.
- ω^2 (omega-squared): corrige el sesgo de η^2 . Recomendado para reportar.

Dataset / recursos

- `seaborn.load_dataset('tips')`: total_bill por day (one-way), o por day × time (two-way).
- `seaborn.load_dataset('penguins')`: body_mass_g por species (one-way claro).
- Librerías: `scipy.stats`, `pingouin`, `statsmodels.api` as `sm`, `statsmodels.formula.api` as `smf`.

Ejercicios

1. One-way: aplicá `scipy.stats.f_oneway(*[grupo for grupo in penguins.groupby('species').body_mass_g])`. Reportá F, p y `dof_between`, `dof_within`.
2. Supuestos: testá normalidad por grupo (`pingouin.normality(penguins, dv='body_mass_g', group='species')`) y homocedasticidad (`pingouin.homoscedasticity`). Si Levene rechaza, repetí con `pingouin.welch_anova`.
3. Post-hoc Tukey: `pingouin.pairwise_tukey(data=penguins, dv='body_mass_g', between='species')`. Identificá qué pares de especies difieren significativamente.

- Two-way con interacción: `pingouin.anova(data=tips, dv='total_bill', between=['day', 'time'])`. Mirá las tres filas de la tabla (day, time, day×time).
- Interaction plot: `sns.pointplot(data=tips, x='day', y='total_bill', hue='time')`. Si las líneas se cruzan o no son paralelas → hay interacción visual; cruzala con el p-value del término `day*time`.

Homework verificable

Sobre penguins (filtrando filas con NA):

- ANOVA one-way de `flipper_length_mm` por species.
- Chequear Levene y Shapiro; decidir entre ANOVA clásico y Welch ANOVA, justificando.
- Tukey HSD post-hoc.
- Reportar ω^2 (`pingouin.anova(... effsize='n2'`, y calcular ω^2 manualmente).
- Conclusión en 3 líneas: qué pares difieren, magnitud del efecto general (η^2/ω^2), si hay alguna comparación dudosa.

Criterio de aceptación: el ANOVA debe rechazar H ($p < 0.001$), Tukey debe mostrar las tres parejas significativas, y η^2 debe ser > 0.5 (efecto grande — las tres especies tienen aletas claramente distintas).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Hago <code>ttest_ind</code> entre cada par de 4 grupos	Con 4 grupos son 6 comparaciones; α efecti
ANOVA da $p < 0.05$ pero ningún Tukey lo da	Puede pasar con varianzas muy distintas. F
Aplico ANOVA con varianzas muy distintas (	El F-test clásico es sensible a esto, espe
Los residuos no son normales y reporto ANO	Si los grupos son grandes ($n \geq 30$ c/u), TC
Interpreto "efecto de A" sin mirar la inte	Si hay interacción, el efecto principal de

Preguntas frecuentes

¿ANOVA o regresión lineal con dummies?

Son matemáticamente equivalentes. ANOVA es la presentación tradicional; OLS con dummies (`statsmodels.formula.api.ols('y ~ C(species)', data).fit()`) da los mismos F y p. La regresión también te da los coeficientes de cada nivel respecto a la categoría de referencia, lo cual es más informativo.

¿Tukey o Bonferroni post-hoc?

Tukey es mejor para todas-las-parejas porque controla el family-wise error rate de manera específica para ANOVA (es uniformemente más poderoso que Bonferroni en ese caso). Bonferroni es más conservador (peor poder). Ver Clase 151 para alternativas modernas (Holm, BH).

¿Qué pasa si los grupos están desbalanceados (n distinto)?

ANOVA aguanta moderadamente, pero con varianzas distintas el F-test se rompe. Welch ANOVA lo maneja bien.

¿Two-way ANOVA cuando alguna celda tiene $n=0$?

Modelo no balanceado: usar Type III SS (`statsmodels` lo soporta vía `sm.stats.anova_lm(model, typ=3)`). Type I (default) depende del orden de los factores en la fórmula.

¿Y si tengo medidas repetidas (mismo sujeto en cada nivel)?

`pingouin.rm_anova` (repeated measures ANOVA). Modela la correlación intra-sujeto, lo cual ANOVA clásico

ignora.

Referencias

- ISLP, cap. 13 — sección sobre ANOVA y multiple testing.
- Bruce & Bruce, cap. 3 — sección ANOVA.
- Welch, B.L. (1951), On the comparison of several mean values: an alternative approach, *Biometrika*.
- `scipy.stats.f_oneway`, `pingouin.welch_anova`, `pingouin.pairwise_tukey`.
- `statsmodels` — ANOVA (para Type II/III SS y modelos mixtos).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 180 — Clase 180 — Tests no paramétricos: Mann-Whitney, Wilcoxon, Kruskal-Wallis

Parte: 3 — Estadística Inferencial y Causal · Fuente: Bruce & Bruce, cap. 3 Resampling and Non-parametric Tests + Conover, Practical Nonparametric Statistics. Duración estimada: 70 min.

Objetivo

Aplicar las tres alternativas no paramétricas más usadas: Mann-Whitney U (= dos muestras independientes, análogo a Welch's t), Wilcoxon signed-rank (= pareado, análogo a `ttest_rel`) y Kruskal-Wallis (= ≥ 3 grupos, análogo a ANOVA one-way). Saber cuándo elegirlos sobre los paramétricos: muestras chicas con datos visiblemente asimétricos, datos ordinales (Likert, ranks), o presencia de outliers extremos.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Reconocer las 3 situaciones en que un test no paramétrico es preferible al paramétrico (n chico + asimetría, ordinal, outliers).
- Aplicar `scipy.stats.mannwhitneyu`, `wilcoxon`, `kruskal` con los argumentos correctos (`alternative`, `method='exact'` vs `'asymptotic'`).
- Interpretar que los no paramétricos testean distribuciones (estocásticamente iguales) o medianas, no medias.
- Reportar effect size no paramétrico: rank-biserial correlation (Mann-Whitney) o ϵ^2 / η^2_H (Kruskal-Wallis).
- Hacer post-hoc no paramétrico tras Kruskal con Dunn's test (`scikit-posthocs`) y corrección por múltiples comparaciones.

Temas

#	Test	Reemplaza a	Para qué
1	Mann-Whitney U (Wilcoxon rank-sum)	<code>ttest_ind</code> Welch	2 grupos independientes
2	Wilcoxon signed-rank	<code>ttest_rel</code>	Pareado / 1 muestra contra mediana
3	Kruskal-Wallis H	ANOVA one-way	≥ 3 grupos independientes
4	Dunn's test (post-hoc)	Tukey HSD	Pares post Kruskal
5	Cliff's δ / rank-biserial r	Cohen's d	Effect size no paramétrico

Definiciones y características

- Test no paramétrico: no asume forma específica de la distribución (no requiere normalidad). Trabaja sobre rangos de los datos.
- Mann-Whitney U: para cada par (x_i, y_j) cuenta cuántas veces $x_i > y_j$. Bajo H de distribuciones iguales, U tiene distribución conocida. $H: P(X > Y) = P(X < Y) = 0.5$.
- Wilcoxon signed-rank: para datos pareados o una muestra contra mediana hipotética. Calcula la diferencia d_i , las rankea por $|d_i|$, suma rangos con signo. Asume simetría de la distribución de diferencias.
- Kruskal-Wallis H : extiende Mann-Whitney a k grupos. $H = (12 / (n(n+1))) \cdot \sum R_i^2 / n_i - 3(n+1)$. Bajo H (todas las distribuciones iguales), $H \sim \chi^2(k-1)$ asintóticamente.
- Rank-biserial correlation $r_{rb} = 1 - 2U / (n \cdot n)$: effect size para Mann-Whitney. Va de -1 a 1.
- Cliff's δ : equivalente, $\delta = (\#(x_i > y_j) - \#(x_i < y_j)) / (n \cdot n)$. Interpretación: < 0.147 small, < 0.33 medium, ≥ 0.474 large (Romano et al. 2006).
- Dunn's test: comparaciones pareadas no paramétricas tras Kruskal-Wallis, basadas en la diferencia promedio de rangos. Se ajusta por múltiples tests (Bonferroni, BH).

Dataset / recursos

- seaborn.load_dataset('tips'): tip por sex o day.
- Datos con outliers: precios de Airbnb (Kaggle) — cola larga a la derecha por mansiones.
- Likert ordinal: simular respuestas 1–5 con rng.choice([1,2,3,4,5], p=...).
- Librerías: scipy.stats, pingouin, scikit-posthocs (pip install scikit-posthocs).

Ejercicios

1. Mann-Whitney: comparar tip entre sex con scipy.stats.mannwhitneyu(a, b, alternative='two-sided'). Compará el p con el del t-test del ejercicio 2 de la Clase 147. Calculá rank-biserial r.
2. Wilcoxon signed-rank: con el dataset simulado de presión arterial antes/después de la Clase 147, aplicá scipy.stats.wilcoxon(antes, despues). Verificá supuesto de simetría con un histograma de las diferencias.
3. Outliers: a un dataset normal rng.normal(50, 5, 100) agregale 3 outliers de valor 200. Compará Welch's t-test vs Mann-Whitney contra otro grupo normal — el Mann-Whitney es mucho más robusto.
4. Kruskal-Wallis: aplicalo a body_mass_g por species en penguins. Comparalo con el ANOVA de la Clase 149.
5. Post-hoc Dunn: con scikit_posthocs.posthoc_dunn(penguins, val_col='body_mass_g', group_col='species', p_adjust='holm') identificá qué pares difieren.

Homework verificable

Tomar el dataset de Airbnb por neighborhood (o un sintético equivalente con cola larga):

1. Verificar normalidad por grupo (Shapiro o KS). Mostrar que se rechaza.
2. Comparar precio entre 4 vecindarios con Kruskal-Wallis.
3. Post-hoc Dunn con corrección Holm.
4. Reportar mediana $\pm$ IQR por grupo (no mean $\pm$ SD, que es engañoso con asimetría).
5. Comparar conclusiones con las que daría un ANOVA clásico ingenuo.

Criterio de aceptación: el reporte debe usar mediana/IQR (no media/SD), identificar al menos un par significativo tras Dunn-Holm, y explicar en 2 líneas por qué ANOVA sería sospechoso aquí (cola larga inflando la varianza del grupo con outliers).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
-------------------	-----------------------

Aplico Mann-Whitney y reporto "la media di	El test no es sobre medias; es sobre la pr
Aplico Wilcoxon a diferencias muy asimétri	El signed-rank asume simetría de las difer
Uso Mann-Whitney con $n=10^6$ y se vuelve len	Es $O(n \log n)$ por el ranking, pero scipy l
Reporto Kruskal-Wallis sin post-hoc y digo	Kruskal solo te dice que al menos uno difi
Aplico no paramétrico "para ir a la segura	El t-test tiene más poder cuando sus supue

Preguntas frecuentes

¿Mann-Whitney testea medianas?

Solo si las distribuciones tienen la misma forma (mismo shape, distinto location). En general testea $P(X > Y) \neq 0.5$, que es una afirmación sobre superioridad estocástica, no sobre la mediana per se.

¿Wilcoxon o test de signos?

Wilcoxon usa magnitud de las diferencias (rangos) → más poderoso. Test de signos solo usa la dirección (positivo/negativo) → más robusto pero menos poderoso. Si dudás de la simetría, signos.

¿Cuánto poder pierdo usando no paramétrico cuando los supuestos se cumplen?

Para Mann-Whitney vs t-test con datos normales, la eficiencia asintótica relativa es $3/\pi \approx 0.955$ — perdés $\approx 5\%$ de poder. Si los datos son no normales, podés ganar mucho. Por eso "no paramétrico por default" no es una mala estrategia para n chico.

¿alternative='greater' significa lo mismo que en t-test?

Sí, pero referido a la dirección de la dominancia estocástica (no a la media). alternative='greater' en Mann-Whitney ↔ "la distribución de X tiende a producir valores mayores que la de Y".

¿Qué hago con datos ordinales (Likert 1–5)?

No paramétrico siempre. Ranks son la operación natural sobre escalas ordinales. Mann-Whitney para 2 grupos, Kruskal-Wallis para ≥ 3 .

Referencias

- Bruce & Bruce, cap. 3 — Resampling and Non-parametric Tests.
- Conover, W.J. (1999), Practical Nonparametric Statistics (3rd ed.) — referencia canónica.
- Romano et al. (2006) — interpretación de Cliff's δ .
- scipy.stats.mannwhitneyu, wilcoxon, kruskal.
- scikit-posthocs — Dunn, Conover, Nemenyi.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 181 — Clase 181 — Corrección de comparaciones múltiples (Bonferroni, FDR)

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 13 Multiple Testing + Benjamini & Hochberg (1995). Duración estimada: 70 min.

Objetivo

Entender por qué hacer 100 tests al $\alpha=0.05$ produce ≈ 5 falsos positivos esperados aunque todas las H

sean verdaderas, y aplicar las dos familias de corrección: family-wise error rate (FWER) con Bonferroni y Holm, y false discovery rate (FDR) con Benjamini-Hochberg (BH). Saber elegir entre ambas según el contexto (medicina/seguridad → FWER; screening exploratorio → FDR).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Cuantificar la inflación de α al hacer k tests independientes: $1 - (1-\alpha)^k$.
- Aplicar Bonferroni: $\alpha_{\text{corregido}} = \alpha / m$. Conservador pero simple.
- Aplicar Holm-Bonferroni (`statsmodels.stats.multitest.multipletests(..., method='holm')`) — uniformemente más poderoso que Bonferroni.
- Aplicar Benjamini-Hochberg (BH/FDR) y entender que controla la proporción esperada de falsos positivos entre los rechazos, no el FWER.
- Distinguir FWER ($P[\text{al menos 1 falso positivo}] \leq \alpha$) de FDR ($E[V/R] \leq q$, donde V son falsos positivos y R rechazos totales).
- Reportar p-values ajustados (q-values) y umbrales claros.

Temas

- El problema: si $m=20$ tests independientes con H verdadera y $\alpha=0.05$, $P(\text{al menos uno rechaza}) = 1 - 0.95^{20} \approx 64\%$.
- FWER: probabilidad de al menos 1 falso positivo en toda la familia.
- FDR: proporción esperada de falsos positivos entre los rechazos (no entre todos los tests).
- Bonferroni: rechazar si $p_i \leq \alpha/m$. Controla FWER exactamente.
- Holm: ordenar p-values y comparar $p_{(i)} \leq \alpha/(m-i+1)$. Uniformemente más poderoso que Bonferroni.
- Benjamini-Hochberg (BH): ordenar $p_{(1)} \leq \dots \leq p_{(m)}$; rechazar todos los $p_{(i)}$ tales que $p_{(i)} \leq (i/m) \cdot q$. Controla FDR a nivel q .
- Cuándo usar cada uno: FWER si un falso positivo es catastrófico (drug approval, security). FDR si esperarás muchos descubrimientos verdaderos y querés tolerar algunos falsos (genómica, A/B testing masivo).

Definiciones y características

- Family-Wise Error Rate (FWER): $P(\text{rechazar al menos una } H \text{ verdadera})$. Sin corrección, crece con m . Bonferroni y Holm lo acotan.
- False Discovery Rate (FDR): $E[V / \max(R, 1)]$ donde V = falsos positivos, R = total de rechazos. Concepto introducido por Benjamini & Hochberg (1995). Mucho menos restrictivo que FWER.
- Bonferroni: divide α por m . Si $m=100$ y $\alpha=0.05$, cada test usa $\alpha_{\text{local}}=0.0005$. Conservador, pierde poder con m grande.
- Šidák: $\alpha_{\text{corregido}} = 1 - (1-\alpha)^{1/m}$. Marginalmente menos conservador que Bonferroni; asume independencia.
- Holm-Bonferroni (1979): step-down. Ordena p-values, compara el más chico contra α/m , el siguiente contra $\alpha/(m-1)$, etc. Uniformemente mejor que Bonferroni.
- BH (Benjamini-Hochberg): step-up. Ordena, encuentra el mayor i tal que $p_{(i)} \leq (i/m) \cdot q$, rechaza todos hasta ese. Controla FDR si los tests son independientes o tienen positive regression dependence (BY si la dependencia es arbitraria).
- q-value: p-value ajustado bajo FDR. Interpretación: "si rechazo todo con $q \leq 0.05$, espero que $\leq 5\%$ de mis rechazos sean falsos".

Dataset / recursos

- Genómica sintética: simular $m=1000$ tests, $m=950$ nulos verdaderos y $m=50$ alternativos. Generar

p-values y mostrar comportamiento de cada método.

- A/B testing real: 20 métricas testeadas a la vez → controlar familia.
- Librerías: statsmodels.stats.multitest, pingouin.multicomp, scipy.stats.

Ejercicios

1. Inflación de α : simulá 10 000 experimentos. En cada uno, hacé 20 tests con H verdadera (scipy.stats.ttest_ind entre dos grupos $N(0,1)$, $n=30$). Contá en qué % al menos 1 da $p < 0.05$. Verificá que ≈ 64 %.
2. Bonferroni: con un vector pvals de 20 p-values, calculá $pvals_adj = np.minimum(pvals * 20, 1)$ y compará contra multipletests(pvals, method='bonferroni').
3. Holm: multipletests(pvals, alpha=0.05, method='holm'). Comparar cuántos rechaza vs Bonferroni con el mismo vector.
4. BH/FDR: genera 1000 p-values, 950 de Uniform(0,1) y 50 de Beta(0.5, 5) (concentrados cerca de 0 — alternativos). Aplicá multipletests(pvals, alpha=0.05, method='fdr_bh'). Contá cuántos rechaza y estimá el FDR empírico (rechazos del primer grupo / total rechazos).
5. Comparación: mismo vector del ej. 4, aplicar Bonferroni, Holm y BH. Tabla con: # rechazos, % de los 50 verdaderos descubiertos (recall), FDR empírico. Verificá que BH descubre mucho más con FDR controlado.

Homework verificable

Sobre un dataset con 30 features y un target binario (ej.: load_breast_cancer):

1. Para cada feature, t-test entre la clase 0 y la clase 1.
2. Aplicar tres correcciones: Bonferroni, Holm, BH ($q=0.05$).
3. Tabla comparativa: # features significativas según cada método.
4. Justificar en 3 líneas qué método elegirías si: (a) vas a publicar los hallazgos en un paper médico, (b) usás esto como screening para una etapa siguiente.

Criterio de aceptación: BH debe descubrir más features que Holm, que descubre $\geq$ que Bonferroni. La justificación debe mencionar que (a) → FWER (Bonferroni/Holm), (b) → FDR (BH).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar	
Hago 50 t-tests sin corrección y reporto 1	Cherry-picking estadístico. Fix: BH como m	
Aplico Bonferroni con $m=1000$ y no rechaza	Demasiado conservador para screening. Fix:	
Aplico BH a tests no independientes (ej.:	BH clásico asume independencia o PRDS. Fi	
Reporto p-value ajustado como "probabilidad	El p-value ajustado sigue siendo un p-valu	datos)". Fix: usar lenguaje preciso ("cont
Decido cuál corrección usar después de ver	También es p-hacking. Fix: pre-especificar	

Preguntas frecuentes

¿Cuándo Bonferroni y cuándo BH?

Regla simple: Bonferroni cuando el costo de un falso positivo es alto (medicina, seguridad, decisiones binarias). BH cuando hacés screening y aceptás cierta proporción de FP entre los hallazgos para no perder verdaderos (genómica, A/B testing masivo, feature selection).

¿Por qué Holm es siempre mejor que Bonferroni?

Porque rechaza al menos los mismos tests y a veces más, sin perder control del FWER. La única razón para usar Bonferroni es simplicidad pedagógica.

¿BH controla FDR exactamente al 5 %?

Controla $FDR \leq q \cdot (m/m)$, donde m es el número de H verdaderas. En la práctica, $m \leq m$, así que $FDR \leq q$. Si esperas muchos nulos, BH es un poco más conservador de lo que parece.

¿Storey's q-value es lo mismo que BH?

Es una versión adaptativa: estima $\pi = m/m$ de los datos y corrige menos cuando hay muchos rechazos esperados. En genómica es estándar; en data science general, BH alcanza.

¿Tengo que corregir si los tests son sobre datasets distintos?

Si forman parte del mismo objetivo de inferencia (la misma "familia"), sí. Si son análisis independientes con conclusiones separadas, no. El límite es subjetivo; pre-registrar la familia ayuda.

Referencias

- ISLP, cap. 13 — Multiple Testing.
- Benjamini, Y. & Hochberg, Y. (1995), Controlling the False Discovery Rate, JRSS Series B.
- Holm, S. (1979), A Simple Sequentially Rejective Multiple Test Procedure, Scandinavian Journal of Statistics.
- statsmodels.stats.multitest.multipletests — todos los métodos en una sola función.
- Efron, B. (2010), Large-Scale Inference: Empirical Bayes Methods for Estimation, Testing, and Prediction.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 182 — Clase 182 — Intervalos de confianza

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 5 + Bruce & Bruce, cap. 2 Confidence Intervals. Duración estimada: 70 min.

Objetivo

Construir e interpretar correctamente intervalos de confianza para media (t-based, z-based, bootstrap) y proporción (Wald, Wilson, Clopper-Pearson), entendiendo que un IC95 % NO significa "95 % de probabilidad de que el parámetro caiga en el intervalo" sino "si repitiéramos el experimento muchas veces, el 95 % de los intervalos construidos contendrían el parámetro". Saber elegir el método según n y la métrica.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un IC para la media usando la distribución t: $\bar{x} \pm t_{\{\alpha/2, n-1\}} \cdot (s/\sqrt{n})$ con `scipy.stats.t.interval`.
- Construir un IC para la proporción con tres métodos y entender cuándo cada uno falla (Wald falla con p cerca de 0/1; Wilson y Clopper-Pearson son robustos).
- Usar `scipy.stats.bootstrap` (≥ 1.7) para IC sin supuestos paramétricos (anticipa Clase 153).
- Interpretar correctamente la frase "intervalo de confianza al 95 %" (es una propiedad del procedimiento, no del intervalo específico).
- Relacionar IC y test de hipótesis: si el IC95 % de la diferencia no incluye 0, el test bilateral al $\alpha=5$ % rechaza H_0 .

Temas

- IC para la media (varianza desconocida): t de Student con n-1 gl.
- IC para la media (varianza conocida o n grande): z.
- IC para proporción: Wald ($\hat{p} \pm z \cdot \sqrt{\hat{p}(1-\hat{p})/n}$) vs Wilson score (recomendado por Agresti & Coull 1998) vs Clopper-Pearson (exacto, conservador).
- IC bootstrap percentil (anticipa Clase 153).
- IC del odds ratio, riesgo relativo (medicina/epidemiología).
- Margen de error ($ME = z \cdot SE$) y cómo determina n.

Definiciones y características

- Intervalo de confianza (IC) al $1-\alpha$: par (L, U) calculado a partir de la muestra tal que, sobre repeticiones del experimento, $P(L \leq \theta \leq U) = 1-\alpha$. El parámetro θ es fijo (no aleatorio); lo aleatorio son L y U.
- Cobertura nominal vs real: la cobertura "nominal" es 95 %; la "real" depende del método y la distribución. Wald infla error con n chico o p extrema.
- Standard error (SE) de la media: $s / \sqrt{n}$. Se reduce con $\sqrt{n}$ — para reducir el SE a la mitad necesitas 4× la muestra.
- t-distribution: para IC de la media cuando estimas σ . Tiene colas más anchas que la normal, lo que infla correctamente el IC con n chico.
- Wilson score interval: usa la fórmula $(\hat{p} + z^2/(2n) \pm z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}) / (1 + z^2/n)$. Mantiene cobertura $\approx$ nominal incluso con $\hat{p}$ cerca de 0 o 1.
- Clopper-Pearson: intervalo exacto basado en la distribución binomial. Garantiza cobertura $\geq$ nominal (puede ser conservador).
- Bootstrap percentile interval: cuantiles $\alpha/2$ y $1-\alpha/2$ de la distribución bootstrap del estadístico. No requiere supuestos paramétricos.
- Margen de error (ME): mitad del ancho del IC. Para n requerido al planificar un estudio: $n = (z \cdot \sigma / ME)^2$.

Dataset / recursos

- `seaborn.load_dataset('tips')`: IC de la propina media.
- Encuesta sintética: simular $n=500$ respuestas binarias con $p=0.03$ (proporción chica $\rightarrow$ Wald falla).
- Librerías: `scipy.stats`, `statsmodels.stats.proportion`, `pingouin`.

Ejercicios

1. IC t para la media: con `tips.total_bill`, calculá el IC95 % con `scipy.stats.t.interval(0.95, n-1, loc=mean, scale=sem)`. Verificá contra `pingouin.compute_bootci`.
2. IC para proporción extrema: con `rng.binomial(1, 0.03, 100)` (proporción de eventos raros), calculá IC con `statsmodels.stats.proportion.proportion_confint(count, n, method='normal')` (Wald), 'wilson' y 'beta' (Clopper-Pearson). Observá cómo Wald da límite inferior negativo (¡imposible!) y los otros dos no.
3. Cobertura empírica: simulá 5 000 muestras de tamaño 30 de $N(50, 10)$. Para cada una, construí IC95 % t. Contá qué % contiene $\mu=50$. Debería ser ≈ 95 %.
4. Bootstrap IC: con `tips.total_bill`, aplicá `scipy.stats.bootstrap((tips.total_bill,), statistic=np.mean, n_resamples=10_000, method='percentile')`. Compará con el IC t.
5. Sample size: querés estimar una proporción con margen de error de ± 2 %, asumiendo $\hat{p} \approx 0.5$ (peor caso). Calculá el n requerido para 95 % de confianza.

Homework verificable

Diseñar un estudio para estimar la proporción de clientes satisfechos en una tienda:

1. Determinar n requerido para margen de error ± 3 % con 95 % de confianza asumiendo $\hat{p} \approx 0.5$.

2. Simular el experimento con esa n y $p_{\text{verdadera}}=0.78$.
3. Construir los 3 IC (Wald, Wilson, Clopper-Pearson) y comparar anchos.
4. En 3 líneas: justificar cuál reportarías y por qué.

Criterio de aceptación: $n \approx 1068$. Los 3 IC contienen 0.78. La justificación debe mencionar que Wilson tiene buen comportamiento general (cobertura $\approx$ nominal con menos ancho que Clopper-Pearson) y es el recomendado actual.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar	
"Hay 95 % de probabilidad de que μ esté en	Interpretación frecuentista incorrecta. Fi	datos) = 0.95").
Wald da IC con límite negativo para propor	Pasa con p' cerca de 0/1 o n chico. Fix: W	
IC del 95 % se interpreta como "el dato ca	No, eso sería un intervalo de predicción (	
Construyo IC asumiendo normalidad con $n=8$	t.interval requiere normalidad o n grande.	
Comparo dos IC: "se solapan, no hay difere	Solapamiento de ICs no implica $p > 0.05$. P	

Preguntas frecuentes

¿Por qué a veces uso z y a veces t ?

z cuando conocés σ poblacional (raro) o $n \geq 30$ (TCL hace que la diferencia sea trivial). t cuando estimás σ con la muestra y n es chico. En la práctica moderna, siempre t (con n grande coincide con z , así que no perdés nada).

¿Wilson o Clopper-Pearson para proporciones?

Wilson por default (Agresti & Coull 1998, Brown et al. 2001 lo recomiendan). Clopper-Pearson si necesitás garantizar cobertura $\geq$ nominal (FDA, ensayos clínicos).

¿Bootstrap siempre es mejor?

No siempre. Si tus supuestos paramétricos se cumplen, t -based es más eficiente (intervalos un poco más cortos). Bootstrap brilla con n chico no normal, estadísticos no estándar (mediana, percentil, R^2), o estimadores complejos donde no hay fórmula cerrada.

¿IC95 % es siempre simétrico?

Para la media t -based, sí. Para proporciones y bootstrap, no — sobre todo cerca de los bordes. Eso es una característica, no un bug: refleja la asimetría real de la distribución muestral.

¿Cómo le explico el IC al cliente sin entrar en frecuentismo?

"Si repitiéramos el experimento muchas veces con muestras del mismo tamaño, el 95 % de los rangos que produciríamos contendrían el valor real. Este rango es uno de esos 95 % en promedio." O directamente: "el valor real está plausiblemente entre L y U ; rangos más angostos requieren más datos".

Referencias

- ISLP, cap. 5 — Resampling Methods.
- Bruce & Bruce, cap. 2 — Confidence Intervals.
- Agresti, A. & Coull, B. (1998), Approximate is Better than 'Exact' for Interval Estimation of Binomial Proportions, American Statistician.
- Brown, Cai & DasGupta (2001), Interval Estimation for a Binomial Proportion, Statistical Science — review de métodos.

- statsmodels.stats.proportion.proportion_confint.
- scipy.stats.bootstrap — API moderna.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 183 — Clase 183 — Bootstrap y permutation tests

Parte: 3 — Estadística Inferencial y Causal · Fuente: ISLP, cap. 5 Resampling Methods + Efron & Tibshirani, An Introduction to the Bootstrap. Duración estimada: 85 min.

Objetivo

Sustituir los supuestos paramétricos (normalidad, homocedasticidad, fórmulas cerradas) por resampling: el bootstrap estima la distribución muestral de cualquier estadístico re-muestreando con reemplazo, y los permutation tests calculan un p-value re-mezclando etiquetas de tratamiento. Aprender a usar las APIs modernas de scipy (bootstrap, permutation_test, ≥ 1.9) y a interpretar las tres variantes de IC bootstrap (percentil, basic, BCa).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar bootstrap a mano: B resamples con reemplazo del mismo tamaño que la muestra original, calcular el estadístico en cada uno, sacar cuantiles $\alpha/2$ y $1-\alpha/2$.
- Usar `scipy.stats.bootstrap(data, statistic, n_resamples=9_999, method='BCa')` y entender por qué BCa corrige sesgo y asimetría.
- Diseñar un permutation test bilateral con `scipy.stats.permutation_test((a, b), statistic, n_resamples=10_000, alternative='two-sided')`.
- Saber cuándo usar bootstrap (IC de estadísticos no estándar: mediana, R^2 , AUC) vs permutación (p-value de comparación entre grupos sin supuestos).
- Reconocer las limitaciones del bootstrap (muestra muy chica $n < 20$, dependencia temporal — usar block bootstrap).

Temas

- Intuición del bootstrap: "tratá la muestra como si fuera la población y resampleá".
- Tres intervalos bootstrap: percentile, basic (reflejado), BCa (bias-corrected + accelerated).
- ¿Cuántos resamples? $B = 10_000$ para IC95 % (los percentiles 2.5 y 97.5 se estabilizan).
- Permutation test: intercambiar etiquetas de tratamiento bajo H_0 de "no diferencia".
- Diferencia conceptual: bootstrap estima variabilidad del estadístico; permutación produce un p-value exacto condicional a los datos.
- Block bootstrap para series temporales (preserva autocorrelación).
- Complemento moderno: APIs `scipy.stats.bootstrap` y `permutation_test` desde scipy 1.9, vectorizadas y con BCa por default.

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 153b — BCa bootstrap y APIs modernas de scipy

Definiciones y características

- Bootstrap (Efron 1979): re-muestreo con reemplazo de la muestra original, del mismo tamaño n . Cada resample produce un valor del estadístico; el conjunto aproxima la distribución muestral.
- B (número de resamples): 1 000 alcanza para SE; 10 000 para IC95 %; 100 000 para colas o p-values pequeños.
- Percentile IC: cuantiles $(\alpha/2, 1-\alpha/2)$ de la distribución bootstrap.
- Basic IC: $(2 \cdot \hat{\theta} - q_{\{1-\alpha/2\}}, 2 \cdot \hat{\theta} - q_{\{\alpha/2\}})$. Refleja respecto al estimador puntual.
- BCa IC: percentile ajustado por sesgo (z) y aceleración (a). Default moderno.
- Permutation test: bajo H_0 de no efecto, las etiquetas son intercambiables. Re-mezclar las etiquetas y recalculando el estadístico genera la distribución exacta bajo H_0 .
- p-value de permutación: $(\# \text{ permutations con } |\text{stat}| \geq |\text{stat}_{\text{obs}}| + 1) / (n_{\text{resamples}} + 1)$. El +1 se llama corrección de continuidad y evita $p=0$.
- Block bootstrap: para series temporales, resampla bloques contiguos en vez de observaciones individuales. Preserva autocorrelación.

Dataset / recursos

- `seaborn.load_dataset('diamonds')`: bootstrap de la mediana del precio por cut.
- Modelos: AUC de un clasificador entrenado — IC bootstrap sobre la AUC en test.
- Librerías: `scipy.stats` (≥ 1.9), `numpy`, `sklearn`.

Ejercicios

1. Bootstrap a mano: para `tips.tip`, hacé $B=10\,000$ resamples con `rng.choice(x, size=len(x), replace=True)`, calculá la media, sacá los cuantiles 2.5 y 97.5. Verificá contra `scipy.stats.bootstrap(..., method='percentile')`.
2. BCa vs percentile: con datos lognormales `rng.lognormal(0, 1, 50)`, calculá IC de la mediana con `method='percentile'` y con `method='BCa'`. Comprubá que BCa es asimétrico hacia la cola derecha (refleja la asimetría real).
3. IC para AUC: entrenar un `LogisticRegression` en breast cancer, calcular AUC en test. Bootstrap $n_{\text{resamples}}=2\,000$ sobre $(y_{\text{true_test}}, y_{\text{proba_test}})$ con un statistic que devuelva `roc_auc_score`. Reportar IC95 % BCa.
4. Permutation test bilateral: con `tips.tip` por sex, ejecutá `scipy.stats.permutation_test` y comparalo con el `mannwhitneyu` de la Clase 150.
5. Cobertura: simulá 1 000 datasets de $\text{Exp}(1)$ con $n=25$. Para cada uno, calculá IC95 % de la mediana con BCa y con percentile. Contá la cobertura empírica. BCa debería estar más cerca de 95 % que percentile.

Homework verificable

Sobre diamonds:

1. Bootstrap BCa ($B=10\,000$) para la mediana de price global.
2. Bootstrap BCa para la diferencia de medianas entre `cut='Ideal'` y `cut='Fair'`.
3. Permutation test sobre la misma diferencia, p-value.
4. Reportar mediana $\pm$ IC95 % BCa por categoría de cut (un gráfico con 5 puntos + bigotes).
5. Conclusión en 4 líneas: relacionar el p-value de permutación con el hecho de que el IC de la diferencia no incluye 0.

Criterio de aceptación: la diferencia de medianas tiene IC95 % BCa positivo (`Ideal < Fair` en precio mediano, contraintuitivo — los diamantes Fair son más grandes), p por permutación < 0.001 , y la conclusión menciona

que IC y p coinciden cualitativamente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Bootstrap con <code>n_resamples=100</code> y conclusion	Demasiado poco para IC. Fix: 10 000 mínimo
Aplico bootstrap a una serie temporal sin	Asume independencia → IC demasiado angosto
Reporto <code>p=0</code> en un permutation test	Significa que ninguna permutación produjo
Uso bootstrap para <code>n=8</code>	Sesga el SE hacia abajo. Fix: bootstrap fu
Bootstrap del R^2 da IC negativo	Pasa cuando hay un mal fit; significa que

Preguntas frecuentes

¿Bootstrap o cross-validation?

Distintos objetivos. CV estima el error de generalización de un modelo. Bootstrap estima la variabilidad de un estimador (cualquiera). Para "qué tan bueno es mi modelo en datos nuevos", CV. Para "qué IC tiene el AUC reportado", bootstrap.

¿Por qué BCa no es siempre el default?

Porque requiere jackknife (n recálculos del estadístico), lo cual es caro para modelos costosos. Para estadísticos baratos (media, mediana), siempre BCa. Para estadísticos caros (AUC de un modelo), percentile o basic puede ser un compromiso aceptable.

¿Permutation test es exacto?

Es exacto condicional a los datos observados: si hicieras todas las permutaciones (no una muestra de 10 000), el p-value sería exacto. Con 10 000 permutaciones, el SE del p-value es pequeño ($\approx \sqrt{p(1-p)/n}$).

¿Bootstrap funciona para cualquier estadístico?

No para todos. Falla con estadísticos no suaves (máximo, percentiles extremos en datasets chicos). En esos casos, subsampling o jackknife son alternativas. Para estadísticos suaves (media, mediana, percentiles centrales, regresión), funciona excelente.

¿Y el out-of-bag de Random Forest es bootstrap?

Sí — Random Forest hace bootstrap sobre filas para cada árbol, y las muestras no incluidas (out-of-bag, $\approx 37\%$ por árbol) se usan para estimar error sin necesidad de CV. Es bootstrap aplicado a ensembles.

Referencias

- ISLP, cap. 5 — Resampling Methods.
- Efron, B. & Tibshirani, R. (1993), An Introduction to the Bootstrap, Chapman & Hall.
- Efron, B. (1987), Better Bootstrap Confidence Intervals, JASA — paper original de BCa.
- DiCiccio & Efron (1996), Bootstrap Confidence Intervals, Statistical Science.
- `scipy.stats.bootstrap` y `permutation_test`.
- `arch.bootstrap` — block bootstrap para series temporales.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 184 — Clase 184 — BCa bootstrap y APIs modernas de scipy

Parte: 3 — Estadística Inferencial y Causal · Fuente: Efron (1987) BCa + DiCiccio & Efron (1996) + `scipy.stats.bootstrap docs`. Duración estimada: 75 min.

Objetivo

Profundizar el BCa (Bias-Corrected and accelerated) bootstrap —el default moderno (Efron 1987)— y las APIs modernas de scipy (`scipy.stats.bootstrap` ≥ 1.9 , `scipy.stats.permutation_test` ≥ 1.8). Cubrir las correcciones que BCa hace sobre percentile clásico: bias correction (z) y acceleration (a) vía jackknife.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar bootstrap percentile vs basic vs BCa.
- Calcular z (bias correction) y a (acceleration) manualmente.
- Aplicar `scipy.stats.bootstrap((data,), statistic, method='BCa', n_resamples=10_000)`.
- Aplicar `permutation_test` para p-value de comparación de 2 grupos sin paramétrica.
- Reconocer cuándo BCa importa: estadísticos no lineales, distribuciones asimétricas.

Temas

- Percentile bootstrap: cuantiles directos. Sub-cubre con asimetría.
- Basic bootstrap: reflexión $2\theta - q_{\{1-\alpha/2\}}$.
- BCa: corrige bias (z) y aceleración (a vía jackknife).
- Studentized bootstrap: estandariza con SE bootstrap del SE.
- `Scipy.stats.bootstrap` API.
- Permutation test exacto.

Definiciones y características

- $z: \Phi^1(P(\theta^*_b \leq \theta))$ — fracción de bootstraps por debajo del estimador.
- a (acceleration): estima cómo SE depende del parámetro. Calculado con jackknife.
- BCa interval: percentiles ajustados α , α función de z, a, $z_{\{\alpha/2\}}$.
- `scipy.stats.bootstrap` desde 1.9: vectorizado, BCa default, IC + SE + bias estimate.
- `permutation_test`: re-mezcla labels bajo H, calcula stat distribution.

Dataset / recursos

- Datos lognormales sintéticos.
- `seaborn.load_dataset('diamonds')` para mediana de price.
- Librerías: `scipy.stats`, `numpy`, `matplotlib`.

Ejercicios

1. Tres ICs: para mediana de $x = \text{rng.lognormal}(0, 1, 100)$, calcular IC con percentile, basic, BCa. Comparar.
2. z a mano: implementar $z = \text{ppf}((B_{\text{below}}\theta) / B)$. Verificar contra scipy.
3. a con jackknife: implementar leave-one-out para cada $\theta_{(i)}$. Calcular a.
4. Cobertura empírica: 1000 datasets $\text{Exp}(1)$, $n=25$; cobertura percentile vs BCa. BCa más cerca de 95 %.
5. `Permutation_test`: comparar dos lognormales con tamaño efecto chico. P-value exacto.

Homework verificable

IC del AUC de un clasificador binario:

1. LogisticRegression en breast cancer. AUC en test.
2. Bootstrap BCa de (y_test, y_proba): 5000 resamples.
3. Reportar AUC [BCa 95% CI].
4. Comparar con percentile bootstrap (más estrecho, sub-cubre).

Criterio de aceptación: BCa CI asimétrico (refleja asimetría de AUC cerca de 1.0); más amplio que percentile.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
BCa con n_resamples=100	Inestable. Fix: ≥ 5000 , idealmente 10000.
Estadístico no vectorizable lento	Bootstrap es $O(B)$. Fix: vectorized=False e
Bootstrap sobre serie temporal	Asume independencia. Fix: block bootstrap
permutation_test n_resamples=999	Resolución del p-value $1/(n+1)$. Fix: 10_00
Reportar percentile vs BCa indistintamente	BCa tiene cobertura nominal. Fix: document

Preguntas frecuentes

Cuándo BCa importa?

Con estadísticos sesgados (mediana en asimetría) o n chico. Para media + n grande, percentile basta.

Studentized bootstrap mejor que BCa?

A veces. Requiere SE del SE $\rightarrow$ bootstrap doble $\rightarrow$ costoso. BCa es el compromiso pragmático.

Para IC de proporciones?

Wilson o Clopper-Pearson son específicos y mejores que bootstrap genérico.

vectorized=True en scipy?

Si tu statistic acepta axis=, sí — 100× más rápido.

Block bootstrap para series?

scipy no lo tiene; arch.bootstrap.MovingBlockBootstrap sí.

Referencias

- Efron (1987), Better Bootstrap Confidence Intervals, JASA.
- DiCiccio & Efron (1996), Bootstrap Confidence Intervals, Statistical Science.
- scipy.stats.bootstrap.
- scipy.stats.permutation_test.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 185 — Clase 185 — A/B testing: tamaño de muestra, poder estadístico

Parte: 3 — Estadística Inferencial y Causal · Fuente: Bruce & Bruce, cap. 3 A/B Testing + Kohavi, Tang & Xu, Trustworthy Online Controlled Experiments (2020). Duración estimada: 90 min.

Objetivo

Diseñar y analizar un A/B test end-to-end: definir hipótesis y métrica primaria, calcular tamaño de muestra con el poder estadístico deseado, randomizar correctamente, analizar resultados sin peeking y reportar con effect size + IC. Conocer tres herramientas modernas que reducen muestra requerida o eliminan el problema de peeking: CUPED, sequential testing (always-valid p-values) y A/B bayesiano.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Calcular n requerido con `statsmodels.stats.power.TTestIndPower` o `NormalIndPower` para una MDE (minimum detectable effect) dada, α y poder.
- Implementar el análisis: t-test (continua) o z-test de proporciones (binaria), con effect size + IC95 %.
- Identificar y evitar 5 errores clásicos: peeking, p-hacking, no estratificación, SRM (sample ratio mismatch), Simpson's paradox.
- Aplicar CUPED para reducir varianza usando una covariable pre-experimento.
- Diseñar un test secuencial con always-valid p-values (Howard et al. 2021) o GST (group sequential testing) que permita parar antes sin inflar α .
- Comparar A/B clásico (frecuentista) con A/B bayesiano (PyMC o bayesab) y entender ventajas (interpretación directa, parar cuando alcance precisión).

Temas

- Hipótesis nula vs alternativa en A/B; métrica primaria, guardrails (no degradar latencia, error rate).
- Poder estadístico: $P(\text{rechazar } H_0 \mid H_0 \text{ verdadera})$. Convención: 80 %.
- Sample size: depende de α (0.05), poder (0.8), σ y MDE.
- Aleatorización a nivel correcto (usuario vs sesión vs request).
- Peeking problem: mirar el resultado intermedio e inflar α .
- SRM (Sample Ratio Mismatch): si el ratio observado A/B se aleja del esperado 50/50, hay bug de asignación.
- Simpson's paradox: la tendencia global se invierte al estratificar.
- Complemento moderno: CUPED, sequential testing, A/B bayesiano.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 154a — CUPED, sequential testing, always-valid p-values

Definiciones y características

- MDE (Minimum Detectable Effect): el efecto más chico que considerarás relevante de detectar. Lo elegís antes del experimento.
- Poder estadístico ($1-\beta$): probabilidad de detectar el MDE si es real. Convención: 0.80.
- α : error tipo I. Para A/B testing, 0.05 es estándar; 0.01 para decisiones críticas.
- SRM (Sample Ratio Mismatch): cuando el ratio de asignación observado se desvía significativamente del esperado. Indica bug en la randomización. Test: χ^2 sobre conteos A vs B.
- Stratificación: balancear covariables (país, plataforma) entre A y B para evitar Simpson's paradox.
- CUPED: reducción de varianza usando covariable pre.
- Always-valid p-value: válido bajo cualquier tiempo de parada; permite peeking sin inflación de α .
- Novelty effect: los usuarios reaccionan al cambio en sí, no al diseño. Confirma con análisis por cohortes/tiempo.

Dataset / recursos

- Simular A/B: $\text{rng.binomial}(1, 0.10, n)$ vs $\text{rng.binomial}(1, 0.12, n) \rightarrow$ MDE de 2 pp absoluto.
- Para CUPED: simular X (pre) y $Y = 0.5 \cdot X + \varepsilon + \delta \cdot \text{tratamiento}$.
- Librerías: statsmodels.stats.power, scipy.stats, confseq, pingouin.

Ejercicios

1. Sample size: querés detectar un uplift de tasa de conversión de 10 % $\rightarrow$ 11 % con poder 0.8 y $\alpha=0.05$. Usá `statsmodels.stats.proportion.samplesize_proportions_2indep_onetail` o `power.NormalIndPower().solve_power`. ¿Cuánto necesitás por grupo?
2. Análisis clásico: simulá el experimento ($n=8\ 000$ por grupo, $p_A=0.10$, $p_B=0.108$), aplicá z-test de proporciones, reportá p, IC95 % de la diferencia y poder post-hoc.
3. CUPED: simulá $X = \text{rng.normal}(50, 10, 2000)$ y $Y = X + \varepsilon + 2 \cdot \text{tratamiento}$ con $\varepsilon \sim N(0,5)$. Calculá n requerido con y sin CUPED para detectar el efecto de 2.
4. Peeking simulado: bajo H verdadera, simulá 1 000 experimentos donde "parás temprano" si $p < 0.05$ mirando cada 100 obs hasta 5 000. Mostrá cómo el α real se infla a ≈ 0.25 .
5. A/B bayesiano: con $A=(1000, 80)$, $B=(1000, 100)$, calculá $P(p_B > p_A)$ con priors $\text{Beta}(1,1)$. Interpretá.

Homework verificable

Diseñar y analizar un A/B test simulado:

1. Calcular n por grupo para MDE=1 pp absoluto, baseline 10 %, $\alpha=0.05$, poder=0.8.
2. Simular el experimento con n calculado y true uplift de 1.2 pp.
3. Reportar: z-test (p, IC), Cohen's h (effect size para proporciones), análisis bayesiano ($P(B > A)$, expected uplift).
4. Repetir simulando peeking cada 1 000 obs sin corrección $\rightarrow$ mostrar inflación de α .
5. Concluir en 4 líneas: recomendación para producción (frecuentista clásico vs bayesiano vs always-valid).

Criterio de aceptación: $n \approx 14\ 800$ por grupo. El z-test debe rechazar H con $p < 0.05$, $P(B > A)$ debe ser > 0.95 en bayesiano, y el ejercicio de peeking debe mostrar α real entre 0.20 y 0.30.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Pareo temprano cuando $\text{vi } p < 0.05$	Peeking $\rightarrow \alpha$ inflado. Fix: pre-registrar fe
El test A/B muestra $p < 0.05$ pero el negoc	Novelty effect, sesgo de selección, no est
Conteos A vs B son 48/52 con $n=10^6$ y "es c	SRM serio. Fix: χ^2 sobre conteos; si $p <$
Resultado positivo global, negativo por ca	Simpson's paradox por estratificación disp
Reporto $p < 0.05$ sin effect size ni IC	Inutilizable para decisión de negocio. Fix

Preguntas frecuentes

¿Cuánto dura un A/B test?

Hasta alcanzar el n calculado y cubrir al menos un ciclo completo del negocio (típicamente 1-2 semanas para capturar weekday/weekend effects, holidays, etc.). Parar antes solo con sequential testing válido.

¿Si mi MDE es muy chico, qué hago?

Necesitás más muestra ($1/\text{MDE}^2$). Si no podés conseguirla: (a) usá CUPED para reducir varianza, (b)

re-evaluá si ese MDE realmente importa para el negocio (un 0.5 % de uplift puede no compensar el costo de desplegar).

¿A/B bayesiano necesita pre-registro?

Conceptualmente menos: la decisión bayesiana ($P(B > A) > \text{threshold}$) es coherente con cualquier tiempo de parada si el prior es honesto. En práctica industrial igual conviene pre-registrar el threshold para evitar racionalizaciones.

¿Aleatorizo a nivel de usuario o de sesión?

A nivel de unidad de tratamiento: si el cambio afecta al usuario (ej.: redesign de homepage), por usuario. Si es un cambio que el usuario puede experimentar múltiples veces sin "aprenderlo" (ej.: ranking de búsqueda), por sesión o request — pero con cuidado por correlación intra-usuario.

¿Qué hago si n calculado es prohibitivo?

Opciones: (a) aumentar MDE (¿es realista el efecto que esperás?), (b) reducir varianza con CUPED, (c) reducir α a 0.10 si el costo de un falso positivo es bajo, (d) hacer un quasi-experiment con synthetic controls (Clase 157).

Referencias

- Bruce & Bruce, cap. 3 — A/B Testing.
- Kohavi, R., Tang, D. & Xu, Y. (2020), Trustworthy Online Controlled Experiments, Cambridge University Press — la biblia industrial.
- Deng, A. et al. (2013), Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data, WSDM (CUPED original).
- Howard, Ramdas, McAuliffe & Sekhon (2021), Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences, Annals of Statistics.
- statsmodels.stats.power.
- confseq — always-valid CIs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 186 — Clase 186 — CUPED, sequential testing, always-valid p-values

Parte: 3 — Estadística Inferencial y Causal · Fuente: Deng et al. (2013) CUPED + Howard et al. (2021) always-valid + Kohavi et al. (2020). Duración estimada: 85 min.

Objetivo

Aplicar las 3 técnicas modernas que la industria (Microsoft, Netflix, Booking, Spotify) usa para hacer A/B testing más eficientemente: CUPED (variance reduction con covariable pre-experiment), Sequential Testing (mirar el resultado durante el experimento sin inflar α), y always-valid p-values / confidence sequences (Howard et al. 2021, decisión correcta en cualquier momento).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar CUPED: ajustar $Y_{\text{cuped}} = Y - \theta \cdot (X - E[X])$ con $\theta = \text{Cov}(Y, X) / \text{Var}(X)$.

- Calcular la reducción de varianza esperada: $1 - \rho^2$.
- Configurar Group Sequential Testing con O'Brien-Fleming boundaries.
- Aplicar always-valid CIs con la librería confseq.
- Decidir entre frequentist clásico, GST, y mSPRT según contexto.

Temas

- CUPED math: θ óptimo minimiza varianza.
- Implementación: con Y_{pre} (mismo usuario en período previo) o X (covariable).
- GST: K looks, boundaries pre-definidas.
- O'Brien-Fleming (gasta poco α al principio) vs Pocock (constante).
- mSPRT: ratio test que provee p-value válido siempre.
- Confidence sequences: IC válido en cualquier t .

Definiciones y características

- CUPED: Controlled-experiment Using Pre-Experiment Data.
- Variance reduction: nuevo $\text{Var}(Y_{cuped}) = \text{Var}(Y) \cdot (1 - \rho^2)$.
- Sequential testing: tomar decisión antes de N planificado.
- GST: predefine K mira-instantes, pasa α budget según schedule.
- Always-valid p-value: válido bajo cualquier optional stopping rule.
- confseq library: implementación de Howard et al.

Dataset / recursos

- Simulación A/B con `numpy.random.default_rng`.
- Librerías: `numpy`, `scipy`, `confseq` (`pip install confseq`).

Ejercicios

1. CUPED implementation: simular X_{pre} , $Y_{post} = \alpha \cdot X_{pre} + \text{tratamiento} + \epsilon$. Calcular θ . Comparar $\text{Var}(Y)$ vs $\text{Var}(Y_{cuped})$.
2. Variance reduction: con $\rho=0.7$, calcular reducción esperada (= 51 %); verificar con simulación.
3. Peeking inflado: bajo H , simular 1000 experimentos con 5 looks naïve, contar % de rejects. Debería ser ≈ 18 %.
4. O'Brien-Fleming: implementar boundaries con `rpy2` + `gsDesign` (o aproximación). Verificar α controlled.
5. Always-valid CI: `confseq.bounds.normal_log_mixture_bound` sobre stream simulado. Plotear CI a lo largo del tiempo.

Homework verificable

Comparar 4 enfoques sobre simulación A/B realista:

1. Frequentist clásico (fixed N).
2. CUPED + frequentist.
3. Naïve peeking cada 1k samples (α inflated).
4. Always-valid (confseq).

Reportar para cada uno: tipo I error (bajo H), poder (bajo H), promedio sample size hasta decisión.

Criterio de aceptación: CUPED reduce sample requerido ~ 30 % con poder igual; naïve peeking infla α a 0.15-0.25; always-valid mantiene $\alpha=0.05$ con +20 % sample.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
CUPED reduce varianza solo si $p > 0$	Si X no correlaciona, no ayuda. Fix: elegí
Peeking sin corrección	α inflado. Fix: GST o always-valid.
GST con boundaries mal calibradas	Engineers re-implementan mal. Fix: librerí
Always-valid muy conservador con pocos sam	Inherente. Fix: aceptarlo o usar GST.
Reportar GST como "p-value normal"	Distinto significado. Fix: documentar que

Preguntas frecuentes

CUPED siempre vale la pena?

Sí, si tenés X correlated y es free de computar. Microsoft reporta 50 % menos samples en muchos casos.

GST o always-valid?

GST: K looks fijos, simple, comunidad estadística. Always-valid: mirá cuando querás, más conservador. Para casos modernos (peeking continuo), always-valid.

Implementations open source?

- GST: gsDesign (R), pyabtest o reimplementación.
- Always-valid: confseq (Python), cpsequential (R).

Bayesian A/B con stopping?

También válido para optional stopping, pero requiere prior honesto. Decisión: $P(B > A \mid \text{data}) > 0.95$.

Industria realmente lo usa?

Sí: Microsoft (CUPED), Netflix (sequential), Optimizely (always-valid). Buscar "Trustworthy Online Controlled Experiments" book.

Referencias

- Deng, Xu, Kohavi & Walker (2013), Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data, WSDM.
- Howard, Ramdas, McAuliffe & Sekhon (2021), Time-uniform, nonparametric, nonasymptotic confidence sequences, Annals of Statistics.
- Kohavi, Tang & Xu (2020), Trustworthy Online Controlled Experiments, Cambridge.
- confseq.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 187 — Clase 187 — Diseño experimental

Parte: 3 — Estadística Inferencial y Causal · Fuente: Montgomery, Design and Analysis of Experiments (8ª ed.) + Kohavi, Tang & Xu (cap. 4-5). Duración estimada: 80 min.

Objetivo

Pasar del A/B simple a diseños más ricos: bloques aleatorizados, factorial completo / fraccional, diseños cruzados (cross-over), switchback para experimentos con interferencia, y cluster randomization cuando la

unidad de análisis no coincide con la unidad de tratamiento. Saber qué problema resuelve cada diseño y leer las consideraciones de SUTVA (Stable Unit Treatment Value Assumption).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Distinguir diseño completamente aleatorizado (CRD), bloques aleatorizados (RBD), factorial y fraccional 2^{k-p} .
- Detectar cuándo SUTVA se viola (efectos de red, interferencia entre usuarios, fila/competencia) y aplicar el diseño correcto: cluster randomization, switchback, marketplace experiments.
- Diseñar un factorial 2^2 o 2^3 con pyDOE2 / statsmodels y descomponer efectos principales + interacciones.
- Saber cuándo usar fraccional (2^{k-p}) para reducir corridas y qué se sacrifica (confounding de interacciones de alto orden).
- Aplicar cross-over para experimentos pareados dentro de sujeto, con análisis vía test pareado o modelo mixto.

Temas

- CRD: el A/B clásico. Asume SUTVA (no interferencia entre unidades).
- RBD (bloques): bloquear por variable nuisance (ej.: día de semana, país) para reducir varianza dentro del bloque.
- Factorial 2^k : testear k factores simultáneamente. Captura interacciones; mucho más eficiente que A/B por factor.
- Fraccional 2^{k-p} : corridas reducidas. Se confunden ("aliasing") efectos de alto orden con principales.
- Cross-over: cada sujeto recibe ambos tratamientos en períodos distintos. Análisis pareado, controla variabilidad inter-sujeto. Riesgo: carry-over effect.
- Cluster randomization: aleatorizar grupos (clases, ciudades) en lugar de individuos cuando hay contaminación social.
- Switchback: alternar tratamiento global en bloques de tiempo (típico de marketplaces de dos lados — Uber, DoorDash).
- SUTVA: cada unidad solo recibe una versión del tratamiento; los efectos no se propagan entre unidades.

Definiciones y características

- Aleatorización: asignación al azar a tratamiento; protege contra confounders observados y no observados.
- Bloqueo: agrupar unidades similares en bloques homogéneos; aleatorizar dentro del bloque. Reduce varianza si la variable de bloqueo importa.
- Efecto principal: efecto promedio de un factor sobre la respuesta.
- Interacción: efecto que un factor tiene sobre el efecto de otro (no aditividad).
- Confounding (en fraccional): la imposibilidad estructural de distinguir un efecto de otro debido al diseño. Se acepta confundir interacciones triples con efectos principales (asumimos triples ≈ 0).
- SUTVA: no interference + no hidden variations of treatment. Es la asunción callada de todo A/B clásico.
- Carry-over: efecto residual de un tratamiento previo en cross-over. Se mitiga con washout periods y diseños equilibrados (Latin square).
- ICC (Intra-Cluster Correlation): correlación dentro del cluster. Si $\rho > 0$, el n efectivo es menor; corregir con $n_{\text{effective}} = n / (1 + (m-1) \cdot \rho)$ donde m es tamaño del cluster.

Dataset / recursos

- Sintéticos para factorial 2^2 (e.g., A=color botón, B=texto botón → CTR).
- Iris / penguins para análisis ANOVA tipo factorial.
- Librerías: pyDOE2 (pip install pyDOE2), statsmodels.formula.api, pingouin.

Ejercicios

1. Factorial 2^2 : simulá CTR con $\text{ctr} = 0.10 + 0.02 \cdot A + 0.015 \cdot B + 0.005 \cdot A \cdot B + \epsilon$. Hacé el experimento con 1 000 obs por celda. Ajustá $\text{ols}(\text{'ctr} \sim A * B', \text{data}).\text{fit}()$ y reportá los 4 coeficientes (intercepto, A, B, A:B). Verificá contra el verdadero.
2. Bloqueo: simulá un experimento de uplift en tasa de retención por país con $\text{países} = [\text{'AR'}, \text{'BR'}, \text{'MX'}]$ con baselines distintos ($p_0 \in \{0.5, 0.3, 0.4\}$). Compará: A/B sin estratificar vs bloqueado por país. Mostrá cómo el SE de δ cae con bloqueo.
3. Fraccional $2^{(4-1)}$: usá $\text{pyDOE2.fracfact}(\text{'a b c d'})$ y discutí qué interacciones quedan aliased. ¿Cuántas corridas vs full factorial?
4. Cluster randomization: simulá 50 escuelas con 30 alumnos c/u, ICC=0.10, efecto verdadero 0.3. Compará t-test ingenuo ($n=1500$) vs análisis correcto a nivel cluster ($n=50$). El primero infla α ; el segundo es correcto.
5. Switchback: simulá precio dinámico en una ciudad con bloques de 1 h alternando A y B durante 7 días. Análisis: comparar bloques A vs B con tests pareados por hora-del-día.

Homework verificable

Sobre un dataset simulado de factorial 2^3 (3 factores binarios, ej.: color × texto × posición sobre conversion):

1. Generar datos con efectos principales no nulos y una interacción A:B.
2. Ajustar OLS con todos los términos hasta triple.
3. Reportar la tabla ANOVA y identificar los términos significativos.
4. Verificar gráficamente la interacción A:B con $\text{sns.pointplot}(x=\text{'A'}, y=\text{'conv'}, \text{hue}=\text{'B'})$.
5. Discutir en 3 líneas qué pasaría si hubieras hecho 3 A/B tests separados en vez del factorial.

Criterio de aceptación: el OLS recupera los efectos verdaderos; la ANOVA marca A, B y A:B como significativos; el análisis muestra que el factorial requiere menos corridas totales que 3 A/B independientes (típicamente la mitad) y además detecta la interacción.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Hago A/B en una red social y "el grupo con	SUTVA violada por contaminación (el contro
Análisis a nivel individual de experimento	α inflado por correlación intra-cluster. F
Asumo interacción nula y la interpretación	Si A solo funciona cuando B está activo, l
Cross-over sin washout y carry-over inflad	El efecto del tratamiento 1 contamina la m
Hago RBD pero no bloqueó lo que importa	Si bloqueás por país pero el efecto import

Preguntas frecuentes

¿Cuándo factorial vs A/B múltiple?

Casi siempre factorial. A/B múltiple (1 factor a la vez) requiere más muestra y no detecta interacciones. Factorial 2^2 con n por celda tiene la misma precisión que dos A/B independientes con $\sim n$ total.

¿Cuántos factores antes de necesitar fraccional?

Con $2^k = 16$ ($k=4$) ya tenés 16 celdas → si querés $\sim 1\,000$ por celda son 16 000 obs. Manejable. A partir de $k=5$ (32 celdas) considerar fraccional, sobre todo si esperás interacciones de alto orden insignificantes.

¿Cuándo cluster randomization?

Cuando hay contaminación natural: educación (alumnos en la misma aula), salud pública (familias), marketplaces (oferta + demanda compartidas). Costo: n efectivo cae con ICC; necesitás muchos clusters.

¿Switchback es válido si hay tendencia temporal?

Sí pero requiere modelar la tendencia (ej.: incluir hora del día como covariable). Si tu métrica tiene fuerte estacionalidad y bloques son largos, mejor diseño Latin square en el tiempo.

¿Diseño antes o análisis primero?

Diseño primero, siempre. El análisis sin diseño correcto produce conclusiones sospechosas (Simpson, confounders, peeking). "You can't analyze your way out of a bad design" (Tukey).

Referencias

- Montgomery, D.C. (2017), Design and Analysis of Experiments (8ª ed.) — referencia clásica completa.
- Kohavi, R., Tang, D. & Xu, Y. (2020), Trustworthy Online Controlled Experiments, caps. 4-5 (advanced designs, interference).
- Imbens & Rubin (2015), Causal Inference for Statistics, Social, and Biomedical Sciences.
- pyDOE2 — generación de matrices de diseño.
- Athey, Eckles & Imbens (2018), Exact p-Values for Network Interference, JASA.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 188 — Clase 188 — Inferencia causal: DAGs, confounders, instrumentos

Parte: 3 — Estadística Inferencial y Causal · Fuente: Pearl, The Book of Why + Hernán & Robins, Causal Inference: What If (libro gratuito, 2024) + Imbens & Rubin. Duración estimada: 95 min.

Objetivo

Distinguir correlación de causalidad con rigor: dibujar DAGs (Directed Acyclic Graphs), identificar confounders, colliders y mediators, aplicar el backdoor criterion para decidir qué variables controlar, y usar variables instrumentales (IV) cuando la randomización no es posible. Conocer la herramienta moderna Double Machine Learning (DoubleML / EconML) para estimar ATE/CATE con ML como nuisance estimator.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Dibujar un DAG para un problema de negocio e identificar los tres tipos de estructura: chain ($X \rightarrow M \rightarrow Y$), fork ($X \leftarrow Z \rightarrow Y$, confounder), collider ($X \rightarrow C \leftarrow Y$).
- Aplicar el backdoor criterion de Pearl: encontrar el conjunto mínimo de variables a controlar para identificar el efecto causal.
- Reconocer que controlar por un collider o un mediator introduce sesgo, NO lo elimina.
- Estimar ATE (Average Treatment Effect) con regresión + controles, IPW (Inverse Probability Weighting) y matching.
- Usar 2SLS (Two-Stage Least Squares) con linearmodels.iv cuando hay un instrumento válido.
- Aplicar Double Machine Learning con doubleml / econml para estimar ATE/CATE con ML como

nuisance (sin asumir linealidad).

Temas

- Correlación $\neq$ causalidad: el clásico ejemplo "helado y ahogamientos" — confounder: temperatura.
- DAGs: nodos = variables, flechas = relación causal.
- 3 estructuras canónicas: chain, fork, collider.
- Backdoor criterion: bloquear todos los caminos no causales de X a Y; no abrir colliders.
- ATE = $E[Y | \text{do}(T=1)] - E[Y | \text{do}(T=0)]$. El "do" indica intervención, no observación.
- Identificación: ¿se puede expresar $E[Y | \text{do}(T)]$ con datos observacionales? Si sí $\rightarrow$ estimar.
- IV: variable Z que afecta T pero NO a Y excepto vía T. Permite identificar el efecto cuando hay confounders no observados.
- Complemento moderno: Double Machine Learning (Chernozhukov et al. 2018) — usa ML para estimar las "nuisance functions" y separa la inferencia causal de la complejidad del fit.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 156a — DoubleML / EconML: Machine Learning para causalidad

Definiciones y características

- DAG: grafo dirigido acíclico que representa relaciones causales. Cada flecha es una hipótesis causal.
- Confounder (fork): $T \leftarrow Z \rightarrow Y$. Z causa tanto T como Y; controlando Z se identifica el efecto.
- Collider: $T \rightarrow C \leftarrow Y$. Controlar C introduce asociación espuria (sesgo de selección). Ejemplo clásico: Berkson's bias.
- Mediator (chain): $T \rightarrow M \rightarrow Y$. Controlar M bloquea el efecto causal indirecto y solo deja el directo (a veces útil, a veces no).
- Backdoor criterion (Pearl): un set Z de variables identifica el efecto causal de T sobre Y si: (a) bloquea todos los caminos backdoor (que empiezan con flecha entrando a T), y (b) no contiene descendientes de T.
- ATE (Average Treatment Effect): $E[Y(1) - Y(0)]$ — efecto promedio si todos vs nadie recibiera tratamiento.
- CATE (Conditional ATE): $E[Y(1) - Y(0) | X=x]$ — efecto para individuos con características $X=x$.
- Instrumento (IV): variable Z que satisface (a) relevancia (afecta a T), (b) exclusión (no afecta Y excepto vía T), (c) independencia (no comparte confounders con Y).
- 2SLS: estima IV ajustando $T = \alpha Z + \varepsilon$ (1ª etapa), reemplazando T en $Y = \theta T + \varepsilon$ (2ª etapa).
- DML: ML para nuisance + Neyman-orthogonal score + cross-fitting $\rightarrow$ inferencia causal robusta.

Dataset / recursos

- Ejemplos simulados de Pearl: smoking-cancer con tar como mediator.
- econml.tests.dgps para datos sintéticos con efectos heterogéneos conocidos.
- Lalonde 1986 (NSW job training program) — clásico de causal inference.
- Librerías: doubleml, econml, linearmodels, pgmpy (DAG inference), dowhy (Microsoft, framework completo).

Ejercicios

1. DAG en código: con pgmpy (o networkx), definí un DAG con T, Y, Z (confounder), C (collider). Identificá visualmente paths y aplicá dowhy para encontrar el adjustment set.
2. Sesgo del collider: simulá $T \sim N(0,1)$, $Y \sim N(T, 1)$, $C = T + Y + \varepsilon$. Estimá $Y \sim T$ sin controlar C y

controlando C. Mostrá que controlar el collider destruye la relación causal.

3. Backdoor ajustando confounder: simulá Z , $T = f(Z) + \varepsilon$, $Y = 2T + 3Z + \delta$. OLS $Y \sim T$ sesgado. OLS $Y \sim T + Z$ recupera el 2.
4. 2SLS: simular un IV $Z \rightarrow T \rightarrow Y$ con confounder no observado entre T y Y . Aplicar `linearmodels.iv.IV2SLS.from_formula('Y ~ 1 + [T ~ Z]', data).fit()`. Recuperar el efecto verdadero.
5. DML con random forest: dataset sintético con confounders no lineales. Comparar OLS ingenuo vs OLS con polinomios vs DoubleMLPLR(`ml_g=RF`, `ml_m=RF`). Verificar que DML es el menos sesgado.

Homework verificable

Sobre un dataset simulado de "efecto de un programa de capacitación sobre salario":

1. Generar X (edad, educación, experiencia) como confounders. Generar T (participa) con $P(T|X)$ no trivial. Generar Y con efecto causal $\theta_{\text{true}} = 2_{000}$.
2. Estimar el efecto con: (a) diferencia ingenua de medias, (b) OLS con controles lineales, (c) DoubleML con RF.
3. Comparar contra θ_{true} . Reportar bias y IC95 %.
4. Dibujar el DAG (puede ser un comentario con notación o un grafo simple).
5. Discutir en 3 líneas qué pasaría si hubieras controlado por un mediator (ej.: "horas trabajadas").

Criterio de aceptación: DML debe recuperar $\theta_{\text{true}} \pm 200$. OLS lineal puede estar sesgado si la relación $X \rightarrow Y$ no es lineal. La diferencia ingenua debe estar fuertemente sesgada. La discusión debe mencionar que controlar mediators sesga hacia 0.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Controlo "todo lo que tengo" en la regresión	Si entre esos hay colliders o mediators, ¡
Asumo que el coeficiente OLS es causal	No lo es. Fix: IV (si tenés instrumento),
Uso un instrumento débil (correlación con	2SLS con IV débil tiene sesgo y SE inflado
doubleml con <code>n_folds=2</code> y muestra chica	Cross-fitting con pocos folds no estabiliz
Interpreto un coeficiente OLS como ATE sin	OLS = ATE solo bajo unconfoundedness. Fix:

Preguntas frecuentes

¿Cómo sé si dibujé bien el DAG?

No hay método estadístico para validarlo completamente — el DAG codifica supuestos sustantivos (de dominio). Lo que sí podés hacer: falsificación condicional — el DAG implica ciertas independencias condicionales; testealas con los datos y si fallan, el DAG está mal. `dowhy.refute_estimate` automatiza muchos refutation tests.

¿IV o DML cuando tengo ambos?

Si tenés un IV válido y confiable, IV es identificación más fuerte (resiste confounders no observados). DML solo aguanta confounders observados. Lo ideal: triangular con ambos.

¿Causal forest vs random forest clásico?

Causal forest no minimiza error de predicción; minimiza heterogeneidad del efecto causal entre hojas. Cada hoja contiene unidades con efecto causal similar.

¿Qué tan robusto es DML a especificación errónea?

DML es doubly robust: si o el modelo de g o el de m está bien especificado, el estimador del efecto es consistente. Cero modelos correctos $\rightarrow$ sesgo. Es la mejor garantía actual sin randomización.

¿Inferencia causal con datos observacionales puede reemplazar un RCT?

No completamente. RCT randomiza el tratamiento $\rightarrow$ corta todas las flechas backdoor por diseño. Observacional siempre depende de supuestos no testables (unconfoundedness, IV exclusion). El estándar es: RCT cuando se puede; cuasi-experimental (DiD, IV, RDD) cuando no; y reportar sensitivity analyses.

Referencias

- Pearl, J. (2018), The Book of Why — intuición sin matemática pesada.
- Pearl, J., Glymour, M. & Jewell, N. (2016), Causal Inference in Statistics: A Primer — el libro técnico corto.
- Hernán, M. & Robins, J. (2024), Causal Inference: What If. Libro gratuito.
- Chernozhukov et al. (2018), Double/Debiased Machine Learning for Treatment and Structural Parameters, Econometrics Journal.
- doubleml docs (Python + R).
- EconML docs — Microsoft Research.
- DoWhy — framework de identificación + estimación + refutación.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 189 — Clase 189 — DoubleML / EconML: Machine Learning para causalidad

Parte: 3 — Estadística Inferencial y Causal · Fuente: Chernozhukov et al. (2018) DML + docs DoubleML + EconML. Duración estimada: 95 min.

Objetivo

Aplicar Double Machine Learning (Chernozhukov 2018) y EconML (Microsoft Research) para estimar ATE y CATE (Conditional ATE — heterogeneidad del efecto) usando cualquier ML como nuisance estimator (Random Forest, XGBoost, neural net). Inference válida (CI, p-value) sin asumir linealidad de los confounders.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar Neyman-orthogonal score y por qué DML es doubly robust.
- Aplicar DoubleMLPLR para ATE con ML nuisance.
- Usar CausalForestDML de EconML para CATE personalizado.
- Cross-fitting: K-fold para evitar overfitting del nuisance.
- Inspeccionar policy óptimo: policy_tree para árbol de decisión de tratamiento.

Temas

- Marco PLR (Partially Linear Regression): $Y = \theta T + g(X) + \epsilon$, $T = m(X) + v$.
- Score orthogonal: derivada respecto a nuisances = 0 en expectation.
- Cross-fitting: estimar nuisances en fold A, evaluar en B.
- CATE: efecto por subgroup.

- Heterogeneity tests.
- Policy learning: decidir a quién tratar.

Definiciones y características

- ATE (Average Treatment Effect): $E[Y(1) - Y(0)]$.
- CATE: $E[Y(1) - Y(0) | X=x]$.
- Nuisance functions: $g(X) = E[Y|X]$, $m(X) = E[T|X]$.
- Doubly robust: consistente si o g o m es correctamente especificado.
- Cross-fitting: K folds, estimar nuisance en train fold, evaluar score en test fold.
- CausalForestDML: random forest entrenado para minimizar heterogeneidad del effect (no error de predicción).

Dataset / recursos

- Sintético con efectos conocidos (de `econml.tests.dgps`).
- Lalonde 1986 (NSW training program) — clásico.
- IHDP (Infant Health and Development).
- Librerías: `doubleml`, `econml`, `scikit-learn`, `xgboost`.

Ejercicios

1. Sintético: simular $Y = 2 \cdot T + 3 \cdot X_1 + X_2^2 + \epsilon$, $T = P(\dots|X)$. ATE verdadero = 2.
2. DML básico: `DoubleMLPLR(data, ml_g=RF, ml_m=RF, n_folds=5)`. Reportar $\hat{\theta}$.
3. Comparar OLS vs DML: OLS lineal con $Y \sim T + X_1 + X_2$ sesgado por X_2 no lineal. DML lo recupera.
4. CausalForestDML: con dataset heterogéneo, estimar CATE. Mapa por X_1 .
5. Policy tree: `econml.policy.PolicyTree` para decidir a quién tratar.

Homework verificable

Estimar efecto causal sobre dataset con confounders no lineales:

1. Generar dataset sintético (`econml.dgps.ihdpsurface_B` o custom).
2. Estimar ATE con: (a) diferencia ingenua, (b) OLS lineal, (c) DML RF, (d) DML XGBoost.
3. Reportar bias contra ground truth.
4. CATE con CausalForestDML; visualizar heterogeneidad.

Criterio de aceptación: DML recupera ATE con bias < 5 %; OLS lineal sesgado; CATE muestra variación por subgrupo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
DML con <code>n_folds=2</code> inestable	Fix: 5-10 folds.
Confounders no observados → DML sesgado	DML asume unconfoundedness. Fix: si hay IV
<code>ml_g</code> muy expresivo overfittea	Cross-fitting ayuda pero no inmune. Fix: <code>r</code>
Reportar CATE sin IC	Necesario para inference. Fix: <code>est.effect_</code>
Asumir SUTVA cuando hay interferencia	Fix: ajustar diseño experimental.

Preguntas frecuentes

DML o regresión clásica?

Si confounders few + relación cerca-lineal: regresión OK. Si muchos / no lineales / interacciones: DML.

DoubleML vs EconML?

DoubleML: más académico, claro framework. EconML: más features (instrumental variables, dynamic treatment, policy). En la práctica complementarios.

Random Forest mejor que XGBoost como nuisance?

Suele dar similar. Probar ambos y elegir el mejor en out-of-fold prediction.

Causal Forest = Random Forest?

NO. Causal Forest divide para minimizar heterogeneidad del efecto causal, no error de Y.

Cuántos samples?

Mínimo 1000-5000 para CATE confiable. Para ATE, menos.

Referencias

- Chernozhukov, Chetverikov, Demirer et al. (2018), Double/Debiased Machine Learning, Econometrics Journal.
- Athey & Wager (2019), Estimating Treatment Effects with Causal Forests.
- Imbens (2020), Potential Outcome and Directed Acyclic Graph Approaches to Causality.
- DoubleML docs, EconML docs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 190 — Clase 190 — Uplift modeling, DiD (difference-in-differences)

Parte: 3 — Estadística Inferencial y Causal · Fuente: Gutierrez & Gerardy (2017), Causal Inference and Uplift Modeling + Abadie, Diamond & Hainmueller (2010), Synthetic Control Methods. Duración estimada: 90 min.

Objetivo

Dominar las dos técnicas causales más usadas en industria cuando hay datos panel/observacionales: DiD (Difference-in-Differences) —comparar la evolución antes/después en grupo tratado vs control— y uplift modeling —predecir a quién conviene tratar (heterogeneidad del efecto causal a nivel individuo). Conocer el complemento moderno Synthetic Control Method para cuando no hay grupo de control natural y solo se trata una unidad (una ciudad, un país).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Aplicar DiD clásico con OLS: $Y = \beta + \beta \cdot \text{tratado} + \beta \cdot \text{post} + \beta \cdot (\text{tratado} \times \text{post}) + \epsilon$. El coeficiente β es el efecto causal bajo parallel trends.
- Diagnosticar la asunción de parallel trends con un gráfico antes/después y un placebo test.
- Construir modelos de uplift: T-learner, S-learner, X-learner (Künzel et al. 2019), Causal Forest (econml).
- Evaluar uplift con Qini curve y uplift@k (no con AUC clásico — uplift es individual, no global).
- Aplicar Synthetic Control con pysyncon o SparseSC cuando una sola unidad recibe tratamiento (estudio de caso).

Temas

- DiD: comparación dos-por-dos en panel (2 grupos × 2 tiempos).
- Asunción crítica: parallel trends — sin tratamiento, ambos grupos hubieran evolucionado paralelos.
- Generalización: DiD con muchos tiempos, two-way fixed effects (TWFE), event study designs.
- Uplift = CATE individual = $E[Y(1) - Y(0) | X=x]$.
- 4 cuadrantes de uplift: persuadables, sure things, lost causes, do-not-disturb (no tocarlos).
- Métricas: Qini, uplift@k, AUUC (area under uplift curve).
- Complemento moderno: Synthetic Control Method (Abadie et al.) — construye un "país sintético" como combinación convexa de unidades no tratadas que replica la trayectoria pre-tratamiento.

Versión profundizada — 2026

El tema moderno que antes vivía como complemento dentro de esta clase ahora tiene su(s) clase(s) propia(s) con patrón completo, ejercicios y homework:

- Clase 157a — Synthetic Control Method dedicado (pysyncon, SparseSC)

Definiciones y características

- DiD: estima $(Y_{\text{tratado,post}} - Y_{\text{tratado,pre}}) - (Y_{\text{control,post}} - Y_{\text{control,pre}})$. Equivale al coeficiente de la interacción tratado × post en OLS.
- Parallel trends: sin el tratamiento, ambos grupos hubieran tenido la misma trayectoria. NO testeable directamente; sí en el pre.
- TWFE (Two-Way Fixed Effects): regresión con dummies de unidad + dummies de tiempo + tratamiento. Generaliza DiD a panel.
- Event study: grafica los coeficientes de dummy × $(t - t_{\text{tratamiento}})$ para cada t . Permite ver dinámica del efecto y testear pre-tendencias.
- Uplift / CATE: $\tau(x) = E[Y(1) - Y(0) | X=x]$. Diferencia entre lo que pasa con tratamiento vs sin.
- T-learner: ajustar dos modelos separados, uno por grupo ($\mu(x) = E[Y|X, T=1]$, $\mu(x) = E[Y|X, T=0]$); restar.
- S-learner: un solo modelo con T como feature; predecir con $T=1$ y $T=0$ y restar.
- X-learner (Künzel et al. 2019): combina T y S, usando propensity para ponderar; mejor con grupos desbalanceados.
- Causal Forest (econml.CausalForestDML): random forest entrenado para minimizar heterogeneidad del efecto, con IC válidos.
- Qini curve: ranking individuos por uplift predicho; eje x = % tratado; eje y = ganancia incremental acumulada vs random.

Dataset / recursos

- DiD clásico: Card & Krueger 1994 (mínimum wage en NJ vs PA).
- Uplift: Hillstrom email dataset (criteo), Lenta uplift dataset.
- Synthetic Control: California Prop 99 (smoking) — clásico de Abadie.
- Librerías: linearmodels (PanelOLS), econml, causalml (Uber), pysyncon, SparseSC.

Ejercicios

1. DiD ingenuo: simulá panel con 2 grupos y 2 períodos. Aplicá DiD con OLS y verificá que β recupera el efecto verdadero. Probá violar parallel trends y ver el sesgo.
2. Event study: con panel 10 períodos (5 pre, 5 post), graficá coeficientes por período. Si los pre son ≈ 0 → parallel trends plausible.
3. T-learner: en Hillstrom (binario tratamiento email), entrená dos RandomForestClassifier y predecí

$\text{uplift} = p(x) - p(x)$.

4. Qini curve: con las predicciones del ej. 3, calculá Qini con `sklift.metrics.qini_score` o a mano. Compará contra "tratar al azar".
5. Synthetic Control: con un dataset panel simulado (10 estados $\times$ 20 años, tratamiento en California año 11), ajustá pesos con `pysyncon` y graficá `path_plot` + `gaps_plot`. Aplicá `placebo_test`.

Homework verificable

Sobre el dataset Card & Krueger (mínimum wage NJ vs PA, 1992):

1. Cargar datos, calcular promedio de empleo por estado en pre (Feb 1992) y post (Nov 1992).
2. Aplicar DiD con OLS y reportar β con IC95 %.
3. Hacer un event study si hay más de 2 períodos disponibles, o el gráfico 2×2 si no.
4. Discutir en 4 líneas: ¿se cumple parallel trends? ¿Cómo se relaciona el β con el debate clásico (Card vs Neumark)?

Criterio de aceptación: el coeficiente DiD debe ser positivo y $\approx +2.7$ empleados (consistente con el paper original); la discusión menciona parallel trends y un riesgo concreto (sesgo de selección de stores, attrition).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar	
Aplico DiD sin verificar parallel trends	Si los grupos tenían trayectorias divergen	
Uplift evaluado con AUC	AUC mide clasificación, no uplift. Un mode	X) puede ser pésimo para uplift. Fix: Qin
T-learner con un grupo mucho más chico que	El modelo del grupo chico overfittea. Fix:	
Synthetic control con pre-período de 2 año	Pesos no convergen a algo confiable. Fix:	
Interpreto el sintético como "lo que hubie	Sin placebo, no podés saber si tu "efecto"	

Preguntas frecuentes

¿DiD funciona con un solo período pre y un solo post?

Sí (es el " 2×2 DiD"), pero no podés testear parallel trends — solo asumirlo. Con más períodos, podés hacer event study y verificarlo.

¿Uplift modeling vs causal forest?

Causal forest es uplift modeling — produce CATE por instancia con IC. Las otras (T/S/X-learner) son métodos clásicos. En la práctica, X-learner y causal forest tienen el mejor performance en benchmarks.

¿Cuántas unidades de control mínimas para synthetic control?

10-20 idealmente; con menos los pesos saturan en 1 o 2 unidades y pierde robustez. Si pocas unidades pero muchos períodos: synthetic DiD o factor models.

¿Negative weights en synthetic control?

El método clásico fuerza $w \geq 0$ (combinación convexa). Variantes modernas (Doudchenko & Imbens 2016, SparseSC) relajan esto y permiten negativos con regularización — más flexible pero menos interpretable.

¿DiD con TWFE es siempre correcto?

No con tratamientos escalonados (unidades tratadas en momentos distintos). TWFE puede pesar con signo negativo períodos donde unidades "ya tratadas" sirven de control de "recién tratadas". Fix: Callaway & Sant'Anna 2021, did package en R o differences en Python.

Referencias

- Angrist & Pischke (2009), Mostly Harmless Econometrics, cap. 5 (DiD).
- Künzel, Sekhon, Bickel & Yu (2019), Metalearners for estimating heterogeneous treatment effects using ML, PNAS.
- Abadie, Diamond & Hainmueller (2010), Synthetic Control Methods, JASA.
- Callaway & Sant'Anna (2021), Difference-in-Differences with Multiple Time Periods, J. of Econometrics.
- Gutierrez & Gerardy (2017), Causal Inference and Uplift Modeling, JMLR Workshop.
- econml, causalml (Uber), pysyncon, SparseSC.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 191 — Clase 191 — Synthetic Control Method dedicado (pysyncon, SparseSC)

Parte: 3 — Estadística Inferencial y Causal · Fuente: Abadie, Diamond & Hainmueller (2010) + Doudchenko & Imbens (2016) + Arkhangelsky et al. (2021) Synthetic DiD. Duración estimada: 80 min.

Objetivo

Aplicar Synthetic Control Method (Abadie et al. 2010) — el estándar para evaluar políticas o intervenciones aplicadas a una única unidad (un país, un estado, una ciudad) sin grupo control natural. Construir un "control sintético" como combinación ponderada de donors. Conocer variantes modernas: Synthetic DiD (Arkhangelsky 2021), Generalized SC (Xu 2017), SparseSC (Microsoft Research).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un Synthetic Control con pysyncon: tratado, donors, predictors, períodos pre/post.
- Interpretar pesos W (combinación convexa) y path plot.
- Aplicar placebo test (in-time y in-space) como inference informal.
- Conocer Synthetic DiD que combina lo mejor de DiD y SCM.
- Reconocer cuándo SCM no aplica (pocos donors, fit pre malo).

Temas

- Setup: 1 tratado + N donors + features predictoras + período pre/post.
- Optimización: pesos minimizan $\|Y_{\text{treat_pre}} - W \cdot Y_{\text{donors_pre}}\|^2$.
- Constraint: $w_i \geq 0$, $\sum w_i = 1$ (combinación convexa) — clásico.
- Placebo test in-space: aplicar SCM a cada donor; comparar effect real vs distribución de placebos.
- Placebo in-time: aplicar antes del tratamiento real → debería dar 0.
- Synthetic DiD: relax constraints + agregar pesos temporales.

Definiciones y características

- Tratado: la única unidad que recibió intervención.
- Donors: pool de unidades no tratadas, idealmente similares pre.
- Predictores: covariables usadas para hacer matching pre.
- Dataprep: clase pysyncon que estructura input.
- Synth: optimizador que encuentra pesos.
- Pre-RMSPE: error de matching pre-período. Si alto, malo.

- Post-RMSPE / Pre-RMSPE ratio: indicador informal de magnitud del efecto.

Dataset / recursos

- California Prop 99 smoking (Abadie's dataset clásico).
- Cualquier panel de países × años con una intervención.
- Librerías: pysyncon, SparseSC (Microsoft), numpy, pandas.

Ejercicios

1. California Prop 99: cargar dataset, definir tratado California, donors otros estados, pre 1970-1988, post 1989-2000.
2. Path plot: synth.path_plot() — California real vs sintética. Visualizar gap post-1989.
3. Pesos: imprimir synth.weights. Verificar que solo few estados tienen peso > 0.
4. Placebo in-space: aplicar a cada otro estado; plot de gaps. California debe destacar.
5. Placebo in-time: tratamiento artificial en 1980 (5 años antes del real). Gap debería ser ≈ 0.

Homework verificable

Estudio de caso: efecto de una política aplicada a 1 unidad (ej.: Brexit en UK, COVID lockdowns en una ciudad):

1. Dataset panel, 10+ donors, ≥ 5 años pre.
2. SCM con pysyncon.
3. Path + gap plots.
4. Placebo in-space.
5. Interpretación: ¿el efecto observado está en el extremo de la distribución placebo?

Criterio de aceptación: pre-RMSPE bajo (good fit); diagnóstico de placebo informativo; conclusión justificada.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Pre-RMSPE alto	Donors no comparables. Fix: filtrar donors
Pocos donors (<10)	Pesos no robustos. Fix: ampliar donor pool
Predictores correlated → pesos inestables	Fix: SparseSC con regularización.
Reportar p-value clásico	No existe en SCM. Fix: placebo test ranks
Aplicar SCM a tratamiento gradual	SCM asume tratamiento discreto. Fix: Synth

Preguntas frecuentes

SCM vs DiD?

DiD necesita "parallel trends"; SCM construye control sintético. SCM mejor cuando 1 tratado; DiD mejor con many.

Synthetic DiD cuándo?

Combina virtudes: usable con many tratados + permite no-parallel trends. Default 2024+ en muchos casos.

Cuántos períodos pre necesarios?

5-10 mínimo. Más es mejor para fit confiable.

SparseSC?

Microsoft Research lib que extiende SCM a paneles grandes con regularización L2.

Bayesian SCM?

Existe (Bayesian Structural Time Series — CausallImpact de Google). Otra alternativa.

Referencias

- Abadie, Diamond & Hainmueller (2010), Synthetic Control Methods for Comparative Case Studies, JASA.
- Doudchenko & Imbens (2016), Balancing, Regression, Difference-in-Differences and Synthetic Control Methods.
- Arkhangelsky et al. (2021), Synthetic Difference-in-Differences, AER.
- pysyncon.
- SparseSC (Microsoft).
- Google CausallImpact (R / Python ports).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 192 — Clase 192 — Bayes intro: priors, posterior, MCMC con PyMC

Parte: 3 — Estadística Inferencial y Causal · Fuente: McElreath, Statistical Rethinking (2ª ed.) + Gelman et al., Bayesian Data Analysis (3ª ed.) + Martin, Bayesian Modeling and Computation in Python.
Duración estimada: 95 min.

Objetivo

Entender la lógica bayesiana —prior + likelihood → posterior vía teorema de Bayes— y construir un modelo simple end-to-end con PyMC v5 (regresión bayesiana sobre datos reales), interpretar el posterior con ArviZ (trace plots, posterior intervals, posterior predictive checks), y conocer el stack moderno: PyMC v5 (post-Theano, sobre PyTensor), NumPyro (sobre JAX, GPU-friendly) y ArviZ (visualización + diagnóstico backend-agnóstico).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir posterior likelihood × prior y aplicarlo a un caso conjugado (Beta-Binomial para una tasa de conversión).
- Construir un modelo lineal bayesiano con `pymc.Model()` as `m`: ... y muestrearlo con `pm.sample()` (NUTS).
- Inspeccionar trace con `arviz.plot_trace`, diagnosticar convergencia ($\hat{r} \leq 1.01$, $\text{ess_bulk} \geq 400$).
- Interpretar HDI (Highest Density Interval) como reemplazo del IC clásico — con interpretación directa de probabilidad.
- Hacer posterior predictive check (`pm.sample_posterior_predictive`) y entender por qué es la validación bayesiana fundamental.
- Conocer NumPyro y cuándo elegirlo sobre PyMC (modelos grandes, GPU/JAX, optimización estocástica via SVI).

Temas

- Teorema de Bayes: $P(\theta|D) = P(D|\theta) \cdot P(\theta) / P(D)$.
- Conjugados: Beta–Binomial, Gamma–Poisson, Normal–Normal (intuición sin MCMC).

- MCMC: idea — muestrear de una distribución sin computarla analíticamente. NUTS (No U-Turn Sampler).
- HDI vs IC frecuentista: el HDI es interpretado directamente como $P(\theta \text{ HDI} | \text{datos}) = 0.94$.
- Posterior predictive: la distribución de datos futuros simulados desde el posterior. Test de modelo.
- Priors: no informativos (Uniform, HalfNormal con scale grande), débilmente informativos (recomendado), informativos (cuando hay expertise).
- Complemento moderno: PyMC v5 (PyTensor backend, ya estable post-Theano), NumPyro (JAX, GPU), ArviZ (diagnóstico estándar).

Versión profundizada — 2026

El tema moderno que vivía como complemento dentro de esta clase ahora tiene clase propia dedicada con patrón completo, ejercicios y homework:

- Clase 158b — Stack bayesiano moderno: PyMC v5, NumPyro, ArviZ

Definiciones y características

- Prior $P(\theta)$: creencia sobre el parámetro antes de ver los datos.
- Likelihood $P(D|\theta)$: probabilidad de los datos dado el parámetro (misma que en MLE/frecuentista).
- Posterior $P(\theta|D)$: creencia sobre el parámetro después de los datos. Combina prior + likelihood vía Bayes.
- MCMC (Markov Chain Monte Carlo): técnica de muestreo de distribuciones complejas. NUTS es el algoritmo moderno default (extiende HMC).
- HDI (Highest Density Interval): intervalo de la distribución posterior que contiene la masa de probabilidad especificada Y tiene la mayor densidad. No confundir con IC — el HDI sí tiene interpretación de probabilidad directa.
- Posterior predictive distribution: $P(\tilde{y} | D) = \int P(\tilde{y}|\theta) \cdot P(\theta|D) d\theta$. Distribución de datos nuevos integrando sobre la incertidumbre del parámetro.
- r_{hat} : Gelman-Rubin convergence diagnostic. Compara varianza intra-cadena vs entre-cadenas.
- ESS (effective sample size): número de muestras independientes equivalentes. MCMC produce muestras correlacionadas, así que $\text{ESS} < N \text{ total}$.
- Conjugate prior: prior cuya combinación con un likelihood específico da posterior de la misma familia. Beta-Binomial, Gamma-Poisson, Normal-Normal.
- SVI (Stochastic Variational Inference): aproxima el posterior con una familia paramétrica más simple (ej.: Normal) optimizando ELBO. Muchísimo más rápido que MCMC; menos exacto.

Dataset / recursos

- tips (regresión bayesiana).
- Tasa de conversión sintética (Beta-Binomial conjugado).
- McElreath's Howell1 (estatura vs peso) — el ejemplo canónico del libro.
- Librerías: pymc (≥ 5), arviz, numpyro, seaborn, matplotlib.

Ejercicios

1. Conjugado a mano: 100 visitas, 8 conversiones. Prior Beta(1,1). Posterior = Beta(9, 93). Graficá prior y posterior; calculá HDI 94 % con `scipy.stats.beta.ppf`.
2. Regresión bayesiana: ajustá el modelo PyMC del ejemplo sobre tips. Reportá summary y plot_trace. Verificá $r_{\text{hat}} \leq 1.01$.
3. Comparación: compará los coeficientes bayesianos (mean del posterior) con OLS de statsmodels sobre el mismo dataset. Con priors débiles, deberían ser casi idénticos.
4. Posterior predictive check: ejecutá `sample_posterior_predictive` y `az.plot_ppc`. Discutí si el modelo

captura la asimetría de tips (probablemente no — sugerir cambiar a likelihood Gamma o lognormal).

5. NumPyro: traducí el modelo a NumPyro, ajustá, convertí con `az.from_numpyro` y verificá que los resultados son equivalentes. Comparar tiempo de ejecución.

Homework verificable

Modelo bayesiano jerárquico simple sobre tips:

1. Modelar la propina como $\text{tip} \sim \text{Normal}(\alpha_{\text{día}} + \beta \cdot \text{total_bill}, \sigma)$ donde $\alpha_{\text{día}}$ varía por día (efecto aleatorio jerárquico): $\alpha_{\text{día}} \sim \text{Normal}(\mu_{\alpha}, \sigma_{\alpha})$.
2. Ajustar con PyMC v5, 4 cadenas, 2 000 muestras.
3. Diagnosticar: `r_hat`, `ess_bulk`, trace plots, divergencias.
4. Reportar HDI 94 % de β (efecto del bill sobre tip) y de los 4 $\alpha_{\text{día}}$.
5. Conclusión en 4 líneas: ¿qué día tiene mayor intercepto? ¿Cuán incierto es β ?

Criterio de aceptación: el modelo converge (`r_hat` ≤ 1.01 para todos los parámetros), β HDI no incluye 0 (efecto positivo claro del bill sobre tip), y la conclusión menciona la jerarquía como ventaja sobre 4 regresiones separadas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>r_hat = 1.3</code> y reporto el resultado igual	No convergió. Fix: más tune, <code>target_accept</code>
Prior Uniform(-10^6 , 10^6) "para ser objetiv	Priors muy planos hacen MCMC inestable y n
Interpreto el HDI como "intervalo de predi	El HDI es del parámetro, no de futuras obs
MCMC se queda atascado con divergences > 1	Posterior con geometría difícil (embudos).
Comparar modelos por DIC	DIC tiene problemas conocidos. Fix: usar L

Preguntas frecuentes

¿Bayes o frecuentista?

No son enemigos — son herramientas distintas. Bayes brilla con n chico, modelos jerárquicos, interpretación directa de probabilidades. Frecuentista brilla con datasets enormes y modelos simples. Industria moderna: ambos, eligiendo por problema.

¿Cuánto tarda MCMC?

Para regresión lineal con 1 000 obs y 3 parámetros: segundos en PyMC v5. Para modelos jerárquicos con 100 grupos: 1-10 min. Para modelos con miles de parámetros: minutos a horas. NumPyro/JAX puede acelerar 10-50×.

¿Qué pasa si elijo un prior "malo"?

Con datos abundantes, el likelihood domina y el posterior es casi insensible al prior. Con datos escasos, el prior importa — y eso es bueno, refleja la realidad de que con $n=10$ no se puede aprender nada sin asumir algo. Hacé prior predictive check (`pm.sample_prior_predictive`) para verificar que los priors no generan datos absurdos.

¿Variational inference (VI) o MCMC?

MCMC es más exacto asintóticamente; VI es mucho más rápido. Para producción con modelos grandes, VI (en NumPyro o Pyro) es la opción. Para inferencia rigurosa, MCMC.

¿Cuál es la diferencia con regresión lineal "normal"?

OLS te da β puntual + IC frecuentista. Bayes te da distribución completa de β , lo cual permite responder preguntas como $P(\beta > 0.5 \mid \text{datos})$, $P(\beta \in [0.3, 0.7])$ directamente — sin pirueta interpretativa.

Referencias

- McElreath, R. (2020), Statistical Rethinking (2ª ed.), CRC Press — el mejor libro para empezar.
- Gelman et al. (2013), Bayesian Data Analysis (3ª ed.) — la referencia técnica completa.
- Martin, O. (2024), Bayesian Modeling and Computation in Python — Python-first, con PyMC v5.
- PyMC v5 docs — sección Introductory Overview.
- NumPyro docs — JAX-based.
- ArviZ docs — diagnóstico backend-agnóstico.
- Hoffman & Gelman (2014), The No-U-Turn Sampler, JMLR — paper de NUTS.

Siguiente parte

Clase 193 — Stack bayesiano moderno: PyMC v5, NumPyro, ArviZ

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 193 — Clase 193 — Stack bayesiano moderno: PyMC v5, NumPyro, ArviZ

Parte: 3 — Estadística Inferencial y Causal · Fuente: PyMC v5 docs + NumPyro docs + ArviZ docs.

Duración estimada: 90 min.

Objetivo

Aprender el stack bayesiano moderno post-Theano —PyMC v5 (PyTensor), NumPyro (JAX), ArviZ (visualización backend-agnóstica)— a nivel de poder construir modelos jerárquicos serios, diagnosticar convergencia ($\hat{r}$, ess_bulk , divergences), comparar modelos con LOO-CV y WAIC, y elegir backend según escala.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir modelo jerárquico no-centered en PyMC v5.
- Diagnosticar: $\hat{r} \leq 1.01$, $\text{ess_bulk} \geq 400$, divergences = 0.
- Aplicar non-centered parameterization para evitar funnel posteriors.
- Comparar modelos con `az.compare([m1, m2], ic='loo')`.
- Migrar modelo de PyMC a NumPyro para 10-50× speedup en CPU.
- Aplicar SVI (Stochastic Variational Inference) en NumPyro como alternativa rápida a MCMC.

Temas

- PyMC v5: PyTensor backend, sintaxis estable.
- NumPyro: JAX backend, JIT + autograd + GPU/TPU.
- Centered vs non-centered parametrization.
- Posterior predictive check con ArviZ.
- LOO-CV y WAIC para comparación.
- SVI con AutoNormal / AutoMultivariateNormal.

Definiciones y características

- PyTensor: sucesor de Theano (archivado 2017). Backend nativo de PyMC.
- NumPyro: probabilistic programming en JAX. 5-50× más rápido en CPU, GPU/TPU nativo.
- $\hat{r}$: Gelman-Rubin convergence. $\leq 1.01 \rightarrow \text{OK}$.
- $\text{ess_bulk} / \text{ess_tail}$: effective sample size. ≥ 400 bulk para inferencia.
- Non-centered: $x = \mu + \sigma \cdot z$, $z \sim N(0,1)$ en vez de $x \sim N(\mu, \sigma)$. Evita funnel.
- LOO-CV (Vehtari): leave-one-out con Pareto smoothed importance sampling.
- SVI: aproximación variational; cierra el gap MCMC con velocidad.

Dataset / recursos

- tips (regresión jerárquica por day).
- McElreath's Howell1 o chimpanzees (modelos clásicos).
- Librerías: pymc (≥ 5), numpyro, arviz, jax.

Ejercicios

1. PyMC v5 hierarchical: $\text{tip} \sim \text{Normal}(\alpha_{\text{day}} + \beta \cdot \text{bill}, \sigma)$; $\alpha_{\text{day}} \sim \text{Normal}(\mu\alpha, \sigma\alpha)$.
2. Non-centered: re-parametrizar el modelo con $\alpha_{\text{day}} = \mu\alpha + \sigma\alpha \cdot z_{\text{day}}$, $z_{\text{day}} \sim N(0,1)$. Comparar divergences.
3. PPC: `pm.sample_posterior_predictive + az.plot_ppc`. Decidir si modelo razonable.
4. NumPyro version: traducir, comparar tiempo.
5. LOO compare: 3 modelos (intercepto solo, + slope, + jerárquico). `az.compare`.

Homework verificable

Modelo bayesiano completo sobre tips:

1. PyMC v5 jerárquico con day como nivel.
2. NumPyro mismo modelo.
3. SVI en NumPyro como alternativa rápida.
4. Comparar 3 enfoques: MCMC PyMC, MCMC NumPyro, SVI NumPyro.
5. `az.plot_trace`, `az.summary`, `az.compare` para 2 variantes del modelo.

Criterio de aceptación: convergencia ($\hat{r} \leq 1.01$); SVI cierra gap con MCMC en < 30 s; ArviZ plots producidos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Divergences > 100	Funnel posterior. Fix: non-centered parame
$\hat{r} = 1.3$	No convergió. Fix: más tune, <code>target_accept</code>
Prior Uniform(-1e6, 1e6)	Plano no es "no informativo". Fix: weakly
SVI con resultados raros	Posterior multimodal o AutoNormal no aplic
Comparar modelos con DIC	Deprecated. Fix: LOO o WAIC.

Preguntas frecuentes

PyMC v5 o NumPyro?

PyMC v5 si tu modelo ya está en PyMC3/v4 (migración fácil). NumPyro si necesitas velocidad o GPU/TPU.

SVI o MCMC?

MCMC para inferencia rigurosa. SVI para producción donde latencia importa.

Stan?

Sí, alternativa madura. cmdstanpy. Sintaxis Stan propia. ArviZ integra.

Edward2 / TFP?

TensorFlow Probability. Menos comunidad que PyMC/NumPyro. Bueno si ya en TF.

Modelos jerárquicos cuándo non-centered?

Casi siempre. Centered solo si hay mucho data por nivel.

Referencias

- Salvatier, Wiecki & Fonnesbeck (2016), PyMC3 — original paper.
- Phan, Pradhan & Jankowiak (2019), NumPyro.
- Vehtari et al. (2017), Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC.
- McElreath (2020), Statistical Rethinking — best book intro bayesiano.

Siguiente parte

Clase 194 — Versionado de datos con DVC

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Parte 4 — Parte 4 — MLOps — Modelos en Producción

14 clases en esta parte.

Clase 194 — Clase 194 — Versionado de datos con DVC

Parte: 4 — MLOps · Fuente: Huyen, *Designing Machine Learning Systems* cap. 6 + docs oficiales DVC 3.x. Duración estimada: 75 min.

Objetivo

Versionar datasets pesados (>100 MB, que git rechaza) con DVC 3.x, separando qué dato se usó (puntero en git, ~200 bytes) de dónde vive el blob real (S3, GCS, Azure, disco local). Reproducir un entrenamiento de hace 3 meses con git checkout <sha> && dvc pull y entender por qué dvc.lock es a los datos lo que package-lock.json es a npm.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Inicializar un repo con dvc init y rastrear un dataset con dvc add data/raw.csv (entiende que git solo guarda el .dvc pointer).
- Configurar un remote (dvc remote add -d origin s3://bucket/path) y sincronizar con dvc push / dvc pull.
- Construir un pipeline reproducible declarando stages en dvc.yaml (deps, outs, params, metrics) y ejecutarlo con dvc repro.

- Comparar experimentos con `dvc exp run`, `dvc exp show`, `dvc exp diff` — alternativa ligera a MLflow para casos simples.
- Diagnosticar ERROR: output 'X' is already tracked by SCM y otros choques DVC ↔ git.

Temas

#	Tema	Por qué importa
1	El problema: git LFS no escala a TB	Costo por GB, throughput, y bloqueo en git
2	Modelo mental DVC: pointer en git + blob e	Git versiona el hash MD5/SHA-256; el dato
3	<code>dvc add</code> vs <code>dvc.yaml</code> stages	Trackeo manual vs pipeline declarativo.
4	Remotes (S3, GCS, Azure, SSH, local)	Donde realmente está la data; <code>dvc remote m</code>
5	<code>dvc.lock</code> — el "lockfile" de tu pipeline	Hashes congelados de inputs/outputs por st
6	<code>dvc exp</code> — branching-less experiments	Ejecutar 20 corridas sin ensuciar git log.

Definiciones y características

- DVC (Data Version Control): herramienta open-source que extiende git para datos/modelos. Versión actual: 3.x (rompió compat con 2.x — `dvc.yaml` mismo, pero comandos `exp` cambiaron).
- `.dvc` file: archivo YAML pequeño (~200 B) con md5, size, path del blob. Va a git. El blob va al remote.
- Cache local (`.dvc/cache/`): copia local de los blobs. `dvc add` mueve el archivo al cache y deja un link (reflink/hardlink/symlink) en el working tree. Por eso `dvc add` no duplica espacio en disco (en filesystems que soportan reflinks: APFS, XFS, btrfs, ReFS).
- Remote: backend remoto (S3/GCS/Azure/SSH/HTTP/local-path). `dvc push` sube cache local → remote; `dvc pull` baja remote → cache → working tree.
- Stage (en `dvc.yaml`): unidad reproducible. Tiene `cmd`, `deps` (inputs), `outs` (outputs), opcionalmente `params` (de `params.yaml`) y `metrics`. DVC re-ejecuta solo los stages cuyos hashes de `deps` cambiaron — como `make`, pero por contenido en vez de timestamp.
- `dvc.lock`: hashes congelados de `deps/outs` después de la última `dvc repro` exitosa. Va a git. Es lo que hace reproducible el pipeline.
- `dvc exp run`: ejecuta el pipeline con (opcionalmente) parámetros distintos y guarda el resultado como "experimento" — un commit interno que no contamina git log hasta que hagás `dvc exp apply` o `dvc exp branch`.

Dataset / recursos

- Dataset: `seaborn.load_dataset('titanic')` exportado a `data/raw/titanic.csv` (~60 KB — pequeño a propósito para que la clase corra sin S3 real).
- Remote demo: directorio local `/tmp/dvc-remote-demo` (simula S3 sin credenciales). El comando para S3 real se muestra pero no se ejecuta.
- Librerías: `dvc[s3]` (o `dvc` pelado si remote local), `pandas`, `scikit-learn`.

Ejercicios

1. Setup mínimo: inicializá un repo git + DVC (`git init && dvc init`). Generá `data/raw/titanic.csv`, hacelo trackear con `dvc add data/raw/titanic.csv`. Inspeccioná el `.dvc` resultante con `cat data/raw/titanic.csv.dvc` y entendé los campos `md5`, `size`, `path`.
2. Remote local: configurá un remote en `/tmp/dvc-remote-demo` con `dvc remote add -d local /tmp/dvc-remote-demo`. Hacé `dvc push` y verificá con `ls /tmp/dvc-remote-demo/files/md5/` que el blob aparece como `<2-char-prefix>/<resto-del-hash>`.
3. Pipeline declarativo: creá `dvc.yaml` con dos stages — `prepare` (lee `raw/titanic.csv`, elimina nulos, escribe `data/processed/clean.csv`) y `train` (entrena un `LogisticRegression`, escribe `model.pkl` y

metrics.json). Corré dvc repro y observá dvc.lock.

4. Reproducción: tocá un parámetro en params.yaml (test_size: 0.2 → 0.3). Volvé a correr dvc repro y verificá que ambos stages se re-ejecutan (porque prepare no depende de params, pero train sí — y el output de train cambió). Compará con dvc repro --dry.
5. Experimentos sin branching: dvc exp run -S 'train.C=0.1' tres veces con valores distintos de C. Listalos con dvc exp show. Aplicá el mejor con dvc exp apply <hash>.

Homework verificable

Pipeline DVC en un repo nuevo con:

1. dvc.yaml con stages prepare → train → evaluate.
2. params.yaml con al menos test_size, random_state, model.C.
3. metrics.json con accuracy y f1 que se reporten con dvc metrics show.
4. Tres corridas dvc exp run variando model.C {0.01, 1, 100}.
5. Output de dvc exp show --no-pager adjunto como experiments.txt.

Criterio de aceptación: dvc repro corre limpio desde cero (rm -rf .dvc/cache data/processed model.pkl metrics.json && dvc pull && dvc repro) y produce los mismos hashes en dvc.lock.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar				
ERROR: output 'data/'	El archivo ya está en g				
dvc push no hace nad	El cache local y el rem				
dvc repro re-ejecuta to	Modificaste un archivo				
Pierdo el cache y dvc	Probablemente el rem				
Git push lentísimo des	Alguien hizo git add da	sort -u \	xargs -l{} du -h {} 2>/d	sort -h \	tail. Después git filter-repo` o BFG pa

Preguntas frecuentes

¿DVC vs git LFS?

LFS es más simple (un binario, git lfs track "*.csv") pero (1) cobra por GB en GitHub Enterprise/GitLab, (2) bloquea git push hasta que el blob suba, (3) no tiene noción de pipeline reproducible ni de experimentos. DVC desacopla storage del provider git y agrega dvc.yaml/dvc.lock/dvc exp. Para datasets <1 GB y casos triviales, LFS alcanza; para ML serio, DVC.

¿DVC vs MLflow?

Son complementarios, no competidores. DVC versiona datos + pipeline (lado git). MLflow trackea runs (params, metrics, artifacts) y registra modelos en un model registry (Clase 195). En la práctica: DVC para reproducibilidad determinística del pipeline; MLflow para comparar 500 experimentos en un dashboard.

¿Tengo que usar S3? ¿Funciona en una notebook personal?

No, funciona con remote local (dvc remote add -d local /path/a/carpeta). Útil para aprender, o para equipos chicos con un NAS compartido. Para producción real con varios colaboradores: S3/GCS/Azure.

¿Qué pasa con datos sensibles (PII)?

DVC no encripta por sí mismo. Si el remote es S3 con SSE-KMS, ya está encriptado en reposo. Para PII fuerte: bucket privado + IAM rol por equipo + auditoría con CloudTrail. No subas datos con PII a un remote público (S3 público, GCS público) — DVC no te avisa.

¿dvc.lock lo edito a mano?

No. Lo regenera dvc repro. Editarlo manualmente rompe la garantía de reproducibilidad. Si necesitas "forzar" un hash, usá dvc commit (con cuidado — saltea la ejecución).

¿Funciona con Jupyter notebooks?

Sí, pero el notebook en sí sigue versionado por git. DVC entra cuando un cell genera/lee artefactos pesados. Patrón común: notebook orquesta, pero las funciones heavy van a un src/, y los datasets están bajo DVC. (Para diff limpio de notebooks: nbdime o jupyterx — fuera del scope de esta clase.)

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 6 — Model Development and Offline Evaluation, sección sobre data versioning.
- DVC 3.x documentation — empezar por Get Started.
- DVC dvc.yaml reference — todos los campos de stage.
- Iterative Studio — UI hosted opcional para visualizar pipelines DVC (no requerida, gratis para repos públicos).
- Comparativa práctica DVC vs LFS vs LakeFS: Data versioning landscape 2024.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abirlo desde el laboratorio del programa o desde Jupyter.

Clase 195 — Clase 195 — Versionado de modelos y experimentos con MLflow

Parte: 4 — MLOps · Fuente: Huyen, Designing Machine Learning Systems cap. 6 + 11 + docs MLflow 2.x. Duración estimada: 75 min.

Objetivo

Trackear experimentos de ML (parámetros, métricas, artefactos, código) con MLflow Tracking, registrar modelos en el Model Registry con stages (None → Staging → Production → Archived), y entender la diferencia conceptual con DVC: DVC versiona el pipeline; MLflow versiona los runs.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Levantar un servidor MLflow local (mlflow ui o mlflow server) y logear runs con mlflow.start_run() + log_param/log_metric/log_artifact.
- Usar autolog (mlflow.sklearn.autolog()) para que params/metrics se capturen sin código boilerplate.
- Registrar un modelo en el Model Registry (mlflow.register_model) y transicionarlo de Staging a Production con la API o la UI.
- Cargar un modelo registrado en producción con mlflow.pyfunc.load_model("models:/MiModelo/Production").
- Configurar un backend store (PostgreSQL) y un artifact store (S3) para uso en equipo.

Temas

#	Tema	Por qué importa
1	Tracking server: backend store + artifact	Dónde van los metadatos vs los blobs (mode

2	Runs, experiments, tags	Unidad atómica + agrupación + búsqueda.
3	log_param, log_metric, log_artifact, log_m	Las 4 primitivas.
4	Autolog por framework (sklearn, PyTorch, X	Cero boilerplate cuando funciona — y sus l
5	Model Registry + stages	El camino entre "entrené esto" y "esto sir
6	MLflow Models flavor (pyfunc, sklearn, pyt	Un mismo modelo se puede cargar en N runti

Definiciones y características

- MLflow Tracking: API + UI para registrar runs (params, metrics, artifacts, código source). Cada run pertenece a un experiment (agrupación lógica, ej. "fraud-detection-v2").
- Backend store: dónde se guardan los metadatos del run (params, metrics, tags). Opciones: filesystem (default ./mlruns), SQLite, PostgreSQL, MySQL. Para equipo: Postgres.
- Artifact store: dónde se guardan los blobs (modelos, plots, datasets exportados). Opciones: filesystem local, S3, GCS, Azure Blob, HDFS. Para equipo: S3/GCS.
- Run: ejecución atómica de un experimento. Tiene un run_id (UUID), start_time, end_time, status, y N params/metrics/tags/artifacts.
- Autolog: hook que intercepta llamadas al framework (sklearn, XGBoost, PyTorch Lightning, ...) y registra params/metrics sin código. Activar con mlflow.<framework>.autolog() antes del .fit().
- Model Registry: catálogo central de modelos versionados. Cada modelo registrado tiene versiones (v1, v2, ...), cada versión tiene un stage (None/Staging/Production/Archived) y aliases (desde MLflow 2.5+: @champion, @challenger).
- MLflow Models: formato estándar para guardar un modelo. Define un MLmodel YAML con uno o más flavors (sklearn, pyfunc, pytorch). pyfunc es el flavor universal — cualquier modelo se sirve como predict(input_df).

Dataset / recursos

- Dataset: California Housing (sklearn.datasets.fetch_california_housing) — 20 640 filas, regresión, sin problemas de PII.
- Backend local: SQLite (mlflow server --backend-store-uri sqlite:///mlflow.db).
- Artifact local: ./mlruns/artifacts.
- Librerías: mlflow>=2.10, scikit-learn, xgboost.

Ejercicios

1. Tracking manual: entrená un LinearRegression y un RandomForestRegressor sobre California Housing. Para cada uno, abrí un run y logueá n_estimators (si aplica), max_depth, rmse_train, rmse_test. Comparalos en la UI (mlflow ui --port 5000).
2. Autolog: repetí el ejercicio anterior con mlflow.sklearn.autolog() y verificá que params + metrics + el modelo entero quedaron registrados sin código extra.
3. Sweep de hiperparámetros: corré 10 runs variando max_depth {3, 5, 10, 15, 20} y n_estimators {50, 200}. Usá mlflow.search_runs() para encontrar el mejor por rmse_test.
4. Registry: registrá el mejor modelo (mlflow.register_model(model_uri, "housing-rf")). Transicionalo a Staging, después a Production desde la API (MlflowClient.transition_model_version_stage).
5. Carga en "producción": en una celda nueva, simulá un servicio que carga el modelo Production y predice una fila: model = mlflow.pyfunc.load_model("models:/housing-rf/Production"); model.predict(X_one).

Homework verificable

Notebook + carpeta mlruns/ que contenga:

1. ≥ 10 runs en un experimento llamado homework-195.
2. Cada run con al menos `model_type`, `rmse_test`, `r2_test`, y el modelo logueado (`mlflow.sklearn.log_model`).
3. Un modelo registrado como `housing-best` con al menos una versión en stage `Production`.
4. Una celda final que carga `models:/housing-best/Production` y predice sobre 5 filas de test.

Criterio de aceptación: `mlflow.search_runs(experiment_names=["homework-195"], order_by=["metrics.rmse_test ASC"])` devuelve la fila top, y su `run_id` coincide con la versión `Production` del modelo registrado.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Los runs van a <code>.mlruns</code> y no aparecen en la UI	La UI está leyendo otro <code>--backend-store-uri</code>
<code>mlflow.sklearn.autolog()</code> no registra el modelo	El modelo se loguea solo si el <code>.fit()</code> ocurre
<code>ConnectionRefusedError</code> contra el tracking	El <code>MLFLOW_TRACKING_URI</code> apunta a un server
Artefactos vacíos en la UI cuando uso S3	El cliente no tiene credenciales AWS. Fix:
<code>RestException: RESOURCE_ALREADY_EXISTS</code> al crear un modelo	Ya existe un modelo con ese nombre. Fix: <code>u</code>
El stage <code>Production</code> está deprecated en log	Desde MLflow 2.9+ se recomienda aliases (<code>@</code>)

Preguntas frecuentes

¿MLflow vs Weights & Biases vs Neptune?

MLflow es open-source, self-hosted, y el estándar de facto en empresas que no quieren un SaaS externo. W&B y Neptune tienen mejores UIs y features colaborativos (reports, sweeps integrados), pero son SaaS con costo por usuario. Para aprender y para deploys self-hosted: MLflow. Para equipos de research grandes con presupuesto: W&B.

¿MLflow vs DVC?

Resuelven cosas distintas. DVC (Clase 194): versionado de datos + pipeline reproducible (vive en el repo git). MLflow: tracking de runs + model registry (vive en un server). Se usan juntos: el `dvc.yaml` define cómo correr el pipeline; dentro del script, MLflow logea cada run.

¿Debo usar mlflow server o mlflow ui?

`mlflow ui` es la UI sobre un backend filesystem (local). `mlflow server` levanta también una REST API para que otros procesos/máquinas registren runs remotamente. Para equipo: server. Para vos solo: ui alcanza.

¿pyfunc o el flavor nativo (sklearn, pytorch)?

Si vas a cargar el modelo desde Python y usar features específicas (acceso a `.coef_`, `.feature_importances_`): flavor nativo. Si vas a servirlo detrás de una API REST o desde otro lenguaje: `pyfunc` — abstrae todo a `predict(DataFrame) → array`.

¿Qué pasa si pierdo mlflow.db?

Perdés metadatos (params, metrics, run history). Los artefactos sobreviven si están en S3. Backup: `pg_dump` si usás Postgres, o copiar `mlflow.db` si SQLite. En producción: backend Postgres con backups gestionados por la nube.

¿Cómo versionar el código junto al run?

MLflow registra automáticamente el commit git (`mlflow.source.git.commit` tag) si corrés desde un repo git

limpio. Si tenés cambios sin commitear, el tag dice dirty. Política sana: el CI registra runs solo desde commits firmados.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 6 (experimentos) y cap. 11 (model serving).
- MLflow Docs — Tracking + Model Registry + Models.
- mlflow.sklearn.autolog — qué se captura automáticamente.
- MLflow Model Registry — stages, aliases (2.5+), webhooks.
- Comparativa MLflow vs W&B vs Neptune (2024) — tomar con grano de sal (parte interesada).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 196 — Clase 196 — Feature stores (Feast)

Parte: 4 — MLOps · Fuente: Huyen cap. 6 + Feast docs 0.40+. Duración estimada: 70 min.

Objetivo

Resolver el problema más caro de ML en producción —training/serving skew: que las features que ve el modelo en producción sean distintas a las que vio en training— centralizando definiciones de features en un feature store. Usar Feast para definir entidades + feature views, materializar al online store (Redis/SQLite), y servir features con `get_online_features` en <10 ms.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la diferencia entre offline store (parquet/BigQuery, para training) y online store (Redis/DynamoDB, para serving low-latency).
- Definir Entity, FeatureView, FileSource y registrar el repo con feast apply.
- Generar un training dataset point-in-time correct con `get_historical_features` (evita data leakage temporal).
- Materializar features (feast materialize-incremental) y consumirlas en serving con `get_online_features`.
- Reconocer cuándo NO usar feature store (ML con <5 features estables, equipo de 1, datos batch puro).

Temas

#	Tema	Por qué importa
1	Training/serving skew — el bug más caro de	Modelo bueno en offline, malo en producció
2	Offline vs online store	Latencia distinta, datos idénticos (en teo
3	Entity + FeatureView + FileSource	Modelo de datos de Feast.
4	Point-in-time joins	Mismo timestamp para feature y label → sin
5	materialize / materialize-incremental	Offline → online en batch programado.
6	Feast vs construir uno propio	Cuándo justifica la dependencia.

Definiciones y características

- Feature store: sistema que (1) define features una vez y las sirve en training + producción con garantía

de equivalencia, (2) provee point-in-time correctness para evitar leakage, (3) baja latencia en serving.

- Training/serving skew: la causa #1 de modelos que rinden bien en CV y mal en producción. Surge cuando el pipeline de feature engineering offline y online divergen (lenguajes distintos, librerías distintas, bugs distintos).
- Offline store: dónde viven los datos históricos para training. Feast soporta File (parquet local), BigQuery, Snowflake, Redshift, Spark.
- Online store: dónde viven los valores actuales para serving low-latency. Soporta SQLite (dev), Redis, DynamoDB, Bigtable, Postgres.
- Entity: dimensión de negocio sobre la que se computan features (ej. user_id, driver_id, product_sku). Tiene un nombre y un tipo (String, Int64).
- FeatureView: agrupa features que (1) vienen de la misma fuente y (2) se computan para la misma entidad. Define ttl (cuánto vive el feature antes de ser stale).
- Point-in-time join: dado (entity_id, event_timestamp) para cada fila del training set, Feast devuelve el valor del feature vigente en ese timestamp — no el más reciente. Evita meter información del futuro en training.
- Materialize: copia features de offline → online. materialize-incremental solo lo que cambió desde la última corrida.

Dataset / recursos

- Dataset: drivers de un servicio tipo Uber — driver_id, event_timestamp, conv_rate, acc_rate, avg_daily_trips. Generado sintéticamente en el notebook.
- Offline store: parquet local en data/.
- Online store: SQLite local en data/online_store.db.
- Librerías: feast>=0.40, pandas, pyarrow.

Ejercicios

1. Setup mínimo: feast init driver_repo. Inspeccioná feature_store.yaml, example_repo.py. Corré feast apply y verificá feast feature-views list.
2. Training dataset histórico: armá un entity_df con driver_id y event_timestamp para 5 momentos distintos del día. Pedile a Feast el feature driver_hourly_stats:conv_rate con get_historical_features. Confirmá manualmente que el valor devuelto es el último anterior al timestamp pedido (no el futuro).
3. Materialización + serving: feast materialize-incremental \$(date +%Y-%m-%d). Después: store.get_online_features(features=['driver_hourly_stats:conv_rate'], entity_rows=[{'driver_id': 1001}]).to_dict(). Medí latencia con %timeit (debería ser <2 ms).
4. TTL en acción: configurá ttl=timedelta(days=1). Materializá datos viejos de 3 días atrás. Pedí features online → debería devolver None (porque expiró). Cambiá ttl=timedelta(days=7) y reintentá.
5. Skew check: comparé el feature offline (parquet) y el online (SQLite) para el mismo driver_id. Si difieren después de materialize, hay un bug.

Homework verificable

Repo Feast con:

1. Una Entity customer_id y una segunda merchant_id.
2. Tres FeatureView: customer_stats (avg_purchase, n_purchases_7d), merchant_stats (avg_rating, n_disputes_30d), transaction_features (amount, hour_of_day).
3. Un script build_training.py que arma el training set point-in-time correct para una tarea de fraud detection.
4. Un script serve.py que, dado un (customer_id, merchant_id), devuelve el vector de features listo para el modelo.

5. README del repo Feast con instrucciones para feast apply + materialize + serve.

Criterio de aceptación: el vector de features que devuelve `serve.py` para un (`customer_id`, `merchant_id`, `timestamp`) coincide exactamente con la fila correspondiente del training set generado por `build_training.py` (mismo `customer/merchant/timestamp`).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>get_historical_features</code> devuelve NaN para	El <code>event_timestamp</code> del <code>entity_df</code> está fuera
<code>get_online_features</code> devuelve None aunque a	TTL expirado, o materializaste hasta <code>now()</code>
feast apply falla con No module named exam	El cwd no es el <code>feature_repo/</code> . Feast carga
Training tarda muchísimo con <code>offline=BigQu</code>	Feast no agrega filtros por timestamp en e
Mi modelo en producción usa features disti	Probablemente tu pipeline de transformació

Preguntas frecuentes

¿Necesito un feature store?

No siempre. Si: ≥ 3 modelos comparten features, equipo ≥ 5 personas, requerís `serving < 50 ms`, o tenés bug recurrente de `training/serving skew` → sí. Si: 1 modelo, 1 persona, `batch scoring` nocturno → es overkill. Empezá sin feature store y migrá cuando duela.

¿Feast vs Tecton vs Hopsworks?

Feast es open-source, self-hosted, no cobra. No hace compute (vos generás los `parquets/tablas`). Tecton y Hopsworks son SaaS managed que incluyen compute (definís transformaciones SQL/Python y ellos las corren). Para empezar: Feast. Para empresas grandes con presupuesto y muchas features: Tecton.

¿Feast hace feature engineering?

No por default. Feast sirve features — no las computa. Las `OnDemandFeatureView` (`request-time`) y las `Stream Feature Views` (con Spark) le agregan compute, pero el patrón principal es: vos generás `parquet`, Feast lo sirve.

¿Por qué point-in-time joins son tan importantes?

Porque la alternativa naive (`LEFT JOIN ... ON entity_id`) trae el valor más reciente del feature — que pudo ocurrir después del label. Ejemplo: predecís `churn` del usuario X el 1 de marzo, pero la feature `n_logins_7d` la tomás de hoy → metés información del futuro en training, y el modelo "predice" perfecto en CV y rompe en prod.

¿SQLite online store sirve para producción?

No: 1 escritor, no es multi-host. Para producción usá Redis (más común), DynamoDB (si ya estás en AWS), o Bigtable (si estás en GCP). SQLite es para desarrollo local y tests.

¿Cómo testear que no hay skew?

Job de comparación periódica: tomar N `entities` random, computar features offline (mismo código que el training) y online (vía Feast), `assert`ear igualdad con tolerancia. Si falla → alerta. Es el equivalente de "smoke test" para feature stores.

Referencias

- Huyen, Chip. *Designing Machine Learning Systems* (O'Reilly, 2022), cap. 6 — sección Feature

Engineering y Feature Store.

- Feast docs — empezar por Quickstart.
- Point-in-time joins explained (Feast blog) — el por qué con ejemplos.
- Tecton vs Feast (2024 comparison) — vendor-biased, leer con criterio.
- MLOps at Reasonable Scale (Tagliabue, 2022) — cuándo NO usar feature store.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrirlo desde el laboratorio del programa o desde Jupyter.

Clase 197 — Clase 197 — CI/CD para ML con GitHub Actions

Parte: 4 — MLOps · Fuente: Huyen cap. 10 + docs GitHub Actions + CML.dev. Duración estimada: 80 min.

Objetivo

Automatizar el ciclo lint → test → entrenar → evaluar → comparar → desplegar con GitHub Actions. Cada PR dispara entrenamiento sobre el slice actual de datos, postea métricas como comentario, y bloquea merge si la métrica empeoró >X%. Sin CI/CD, "el modelo nuevo es mejor" es opinión; con CI/CD, es un workflow check o .

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un workflow .github/workflows/ml.yml con jobs lint, test, train, evaluate.
- Usar CML (Continuous Machine Learning) para postear plots/metrics en el comentario del PR.
- Cachear dependencias (actions/setup-python con cache: pip) y pesos de modelos (actions/cache) para que el CI no tarde 20 min.
- Usar secrets y OIDC para deploy seguro a AWS/GCP sin claves long-lived.
- Configurar branch protection + required checks que bloqueen merge si las métricas degradan.

Temas

#	Tema	Por qué importa
1	Anatomía de un workflow: triggers, jobs, s	Vocabulario base.
2	Matrix builds (strategy.matrix)	Probar 3 versiones de Python × 2 de PyTorch
3	Caché de pip + datasets pesados	CI <5 min vs CI >30 min.
4	CML: reportar métricas en PR	Trae al code review los números, no solo e
5	OIDC para deploy (sin secret long-lived)	Federation con AWS/GCP/Azure.
6	Branch protection + required status checks	Convierte el lint/test/eval en gate, no en

Definiciones y características

- Workflow: archivo YAML en .github/workflows/*.yml. Disparado por on: (push, pull_request, schedule, workflow_dispatch).
- Job: agrupación de steps que corren en el mismo runner (VM/container). Jobs corren en paralelo por default; usar needs: [job1] para serializar.
- Step: comando individual. Puede ser shell (run:) o una action reutilizable (uses: actions/setup-python@v5).

- Runner: máquina donde corren los jobs. ubuntu-latest (gratis para repos públicos, 2 vCPU/7 GB RAM), windows-latest, macos-latest, o self-hosted (GPU, recursos custom).
- CML (Continuous Machine Learning): CLI de Iterative.ai que postea comentarios en PR con tablas/plots desde el workflow. Hace que las métricas sean visibles al reviewer sin abrir Actions tab.
- OIDC (OpenID Connect): GitHub emite un token efímero firmado que AWS/GCP/Azure aceptan para asumir un rol. Reemplaza guardar AWS_ACCESS_KEY_ID como secret. Una vulnerabilidad menos.
- Required status checks: configurable en Settings → Branches → Branch protection. Lista de jobs cuyo nombre debe terminar en para permitir merge.

Dataset / recursos

- Modelo del ejemplo: RandomForestClassifier sobre Iris (corre en <5 s — ideal para CI).
- Repo template: estructura src/, tests/, .github/workflows/, params.yaml, metrics.json.
- Librerías: scikit-learn, pytest, ruff (lint), cml (npm package).

Ejercicios

1. Workflow mínimo (lint + test): creá .github/workflows/ci.yml con dos jobs en paralelo: lint (corre ruff check) y test (corre pytest). Push y verificá que aparecen los dos checks en el PR.
2. Job de training: agregá un job train que corre python src/train.py, sube model.pkl y metrics.json como actions/upload-artifact. Solo en PRs (if: github.event_name == 'pull_request').
3. Reporte CML: agregá un step que use iterative/setup-cml@v2, escribe report.md con cat metrics.json como tabla, y postea con cml comment create report.md. El comentario aparece en el PR.
4. Comparación contra main: en el mismo job, git fetch origin main && python src/train.py desde main, guardás metrics_main.json, y agregás al reporte una tabla con Δ accuracy, Δ f1.
5. Branch protection: en Settings → Branches → Add rule → main, marcá Require status checks to pass before merging y seleccioná lint, test, train. PR con tests rotos = no podés merge.

Homework verificable

Repo público en GitHub con:

1. Workflow .github/workflows/ml.yml con jobs lint, test, train, report.
2. CML comment funcionando: PR muestra tabla con accuracy, f1, Δ accuracy_vs_main.
3. Cache de pip configurado (actions/setup-python con cache: pip).
4. Un PR abierto donde el modelo degrada accuracy en >3% — el job report debe fallar (exit 1) por umbral configurable en params.yaml.
5. Captura del PR mostrando el comentario CML y los checks rojos.

Criterio de aceptación: el reviewer puede decidir merge/no-merge mirando solo el PR (no Actions tab), y el merge está bloqueado por branch protection cuando algún check falla.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Resource not accessible by integration al	El job no tiene permiso pull-requests: wri
El cache de pip nunca se reutiliza	El cache-key cambia en cada run (probablem
Workflow no se dispara en PRs de forks	Es comportamiento de seguridad: PRs de for
Process completed with exit code 137	OOM (out of memory). El runner gratis tien
OIDC token unavailable al asumir rol AWS	Falta permissions: { id-token: write } en
El job train tarda 25 min en cada push	Falta caché de datasets y modelos pre-entr

Preguntas frecuentes

¿Conviene entrenar el modelo real en CI?

Depende del tamaño. Si el training corre en <10 min en CPU: sí, en GitHub Actions. Si requiere GPU o tarda horas: usá CI solo para correr smoke tests (entrenar 1 epoch en data sample) y dispará el training real en un runner self-hosted con GPU, o en SageMaker/Vertex AI con workflow_dispatch.

¿GitHub Actions vs GitLab CI vs Jenkins?

GH Actions: integrado con GitHub, marketplace enorme de actions, gratis hasta cierto límite para repos públicos. GitLab CI: equivalente para GitLab, con auto-devops más opinado. Jenkins: self-hosted, máxima flexibilidad, mucho mantenimiento. Para equipos en GitHub: Actions sin pensar.

¿Qué hago con secrets de larga vida (API keys, DB passwords)?

(1) Si podés: OIDC + IAM role (zero secrets). (2) Si no: secrets de repo o de organization, rotados cada 90 días vía secrets-manager. (3) Nunca: hardcoded en el YAML ni en el código.

¿Cómo evito que un PR malicioso fugue secrets via CI?

PRs de forks NO ven secrets por default (correcto). Para repos privados con muchos contributors: Settings → Actions → General → Fork pull request workflows from outside collaborators: Require approval for all outside collaborators. Revisar el diff del workflow antes de approve.

¿CML es necesario o uso comentarios manuales con gh pr comment?

CML está diseñado para ML (sabe armar tablas de metrics, embedded plots como PNG base64). gh pr comment es genérico. Para reportes simples: cualquiera. Para comparación de runs con plots: CML.

¿Cómo manejo CI cuando uso self-hosted runners con GPU?

Etiquetá runners (runs-on: [self-hosted, gpu]). Para jobs cortos sin GPU, dejá runs-on: ubuntu-latest. Atención a seguridad: PRs de forks NO deberían correr en self-hosted (pueden ejecutar arbitrary code en tu hardware). Por default GH bloquea esto — no lo deshabilites.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 10 — Infrastructure and Tooling.
- GitHub Actions docs — empezar por Quickstart y Workflow syntax.
- CML.dev — Continuous Machine Learning de Iterative.ai.
- Configuring OIDC with AWS — deploy sin claves.
- awesome-actions — actions útiles del marketplace.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 198 — Clase 198 — Docker para empaquetar modelos

Parte: 4 — MLOps · Fuente: Huyen cap. 11 + docs Docker + Nilsson, Docker Deep Dive. Duración estimada: 75 min.

Objetivo

Empaquetar un modelo entrenado + su runtime (Python, deps, código) en una imagen Docker reproducible,

optimizada (multi-stage build, capas cacheadas, imagen <500 MB), y segura (non-root user, no secrets baked in). El resultado se corre idéntico en tu laptop, en CI, y en producción.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un Dockerfile correcto para un servicio ML: base slim, multi-stage, layer caching, non-root.
- Diferenciar COPY de ADD, RUN de CMD de ENTRYPOINT, y cuándo usar cada uno.
- Reducir tamaño de imagen (de 2 GB a <500 MB) con python:3.12-slim, .dockerignore, y multi-stage builds.
- Versionar imágenes con tags semánticos (mymodel:1.2.3) y digests (@sha256:...) — y por qué :latest es trampa en producción.
- Diagnosticar image not building, image too big, slow rebuild con docker history, dive, docker scout.

Temas

#	Tema	Por qué importa
1	Imagen, capa, container, registry	Vocabulario base.
2	Dockerfile: FROM, COPY, RUN, CMD, ENTR	Las instrucciones que importan.
3	Layer caching: orden de instrucciones	Diferencia entre build de 2 min vs 30 s.
4	Multi-stage build	Separar build-deps de runtime-deps.
5	Imágenes base: python:slim vs distroless v	Trade-off tamaño / compat / debug.
6	Security: non-root user, secrets via env/m	No bakear API keys en la imagen.

Definiciones y características

- Imagen: filesystem inmutable + metadata (entrypoint, env, ports). Compuesta por capas read-only apiladas (cada RUN/COPY es una capa).
- Container: instancia ejecutándose de una imagen + capa writable encima. Efímero por default — al borrarlo, se pierde lo escrito (salvo volúmenes).
- Layer caching: si la instrucción N de tu Dockerfile no cambió ni cambiaron las anteriores, Docker reusa la capa cacheada. Regla de oro: lo que cambia poco va primero, lo que cambia mucho va al final.
- Multi-stage build: usar varias secciones FROM ... AS stage y copiar artefactos de una a otra con COPY --from=stage. Permite usar imagen grande para compilar y imagen pequeña para correr.
- .dockerignore: como .gitignore pero para docker build. Crítico — sin él, mandás todo el repo (incluido .git/, __pycache__/, datasets) como build context.
- Tag semántico (myapp:1.2.3): fija la versión exacta. Digest (myapp@sha256:abc...): fija el contenido exacto (inmutable). Producción usa digest. :latest no tiene garantía — puede cambiar entre docker pull y docker run.
- Distroless (gcr.io/distroless/python3): imagen sin shell, sin package manager, casi sin nada. Reduce attack surface; complica debug (no podés exec un shell).

Dataset / recursos

- Modelo: RandomForestClassifier entrenado en Iris, serializado con joblib.
- API: FastAPI sirviendo POST /predict.
- Herramientas: docker>=24, opcional dive, docker scout.

Ejercicios

1. Dockerfile básico: empaquetá un script predict.py que carga model.pkl y predice una fila random. FROM python:3.12-slim, instalá deps de requirements.txt, COPY . /app, CMD ["python", "predict.py"].

Build con `docker build -t miml:v1 .` y corré.

2. Layer caching: cambiá una línea en `predict.py` sin tocar `requirements.txt`. Rebuildá. Confirmá que la capa de pip install se reusa (mensaje "CACHED").
3. Multi-stage: separá en dos stages: builder (FROM `python:3.12` AS builder, instalá deps con compiladores) y runtime (FROM `python:3.12-slim`, copiá solo site-packages del builder). Compará tamaño con docker images.
4. Non-root: agregá `RUN useradd -m app && USER app` antes del CMD. Verificá con `docker run --rm miml:v3 whoami`.
5. Tags y digest: hacé `docker push miml:v1` a Docker Hub. Obtené el digest con `docker inspect miml:v1 --format '{{index .RepoDigests 0}}'`. Discutí por qué deploys de producción referencian el digest.

Homework verificable

Repo con:

1. Dockerfile multi-stage que produce imagen <500 MB para un modelo sklearn + FastAPI.
2. `.dockerignore` que excluye `.git`, `__pycache__`, `*.ipynb`, `data/raw`, `mlruns/`.
3. `docker-compose.yml` que levanta el servicio en `:8000` con healthcheck.
4. README del repo con comandos build, run, push, y la URL/digest de la imagen pushed.
5. Output de `docker history` (o `docker logs`) mostrando que ninguna capa única pesa más de 200 MB.

Criterio de aceptación: `docker run --rm -p 8000:8000 miml:v1` levanta y responde a `curl localhost:8000/predict -X POST -d '{"features":[5.1,3.5,1.4,0.2]}' -H 'content-type: application/json'`. El contenedor corre como app, no como root (`docker exec ... whoami` devuelve app).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Imagen pesa 2.5 GB con un modelo de 50 MB	Estás usando <code>python:3.12</code> (no slim) y/o tra
Cada build rebuildea todo aunque no cambié	Tenés <code>COPY . /app</code> antes de <code>RUN pip install</code>
<code>docker build</code> manda 3 GB de context	Falta <code>.dockerignore</code> . Fix: crearlo con <code>al m</code>
Error <code>ModuleNotFoundError</code> en el container	El pip install corrió en otra versión de P
Container corre como root	No agregaste <code>USER</code> al Dockerfile. Fix: <code>RUN</code>
<code>:latest</code> apunta a algo distinto que ayer	<code>:latest</code> es mutable. Otro <code>docker push myimg</code>
<code>COPY</code> no encuentra <code>model.pkl</code>	El path es relativo al build context (el <code>.</code>

Preguntas frecuentes

¿slim, alpine, o distroless?

slim (Debian-based, sin doc/locales): default razonable, ~120 MB, glibc, compatible con wheels precompilados. alpine (musl): 50 MB pero usa musl, no glibc — wheels precompilados de muchas libs Python no funcionan. distroless: la imagen runtime más chica y segura; complica debugging. Para ML: slim salvo razón fuerte.

¿Docker o Podman?

Podman es API-compatible con Docker (alias `podman ↔ docker`), rootless por default, sin daemon. Para individuales: indistinto. Para producción Kubernetes: el OCI runtime que use el cluster (containerd, CRI-O). Las imágenes son OCI, no "Docker images" — funcionan en cualquier OCI runtime.

¿Cómo paso credenciales al container?

(1) Variables de entorno (docker run -e API_KEY=...) — visible en docker inspect. (2) Secrets mount (--secret, Docker Swarm/Kubernetes Secret) — preferido. (3) IAM role (en EC2/EKS) — el container hereda credenciales sin que las escribas. Nunca: ENV API_KEY=xxx en el Dockerfile, queda en una capa para siempre.

¿Tamaño del modelo dentro o fuera de la imagen?

Modelos pequeños (<200 MB): adentro, simple. Modelos grandes o que cambian seguido: afuera, en S3/MLflow, bajados en entripoint. Trade-off: adentro = inmutable + lento de pull; afuera = imagen liviana pero acoplamiento con storage.

¿CMD vs ENTRYPOINT?

ENTRYPOINT es el "binario" fijo del container; CMD son sus "args default". Combinados: ENTRYPOINT ["python"] + CMD ["app.py"] → docker run img otro.py ejecuta python otro.py. Para imagen de modelo: ambos juntos, o solo CMD ["python", "app.py"] si querés que sea fácilmente overrideable.

¿Por qué docker scout o trivy?

Escanean la imagen contra CVE conocidas. Crítico antes de subir a producción — una python:3.12-slim de hace 6 meses tiene 200 CVE conocidas; rebuildear con la base actual elimina la mayoría. CI debería tener un step trivy image myimg:tag --severity HIGH,CRITICAL --exit-code 1.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 11 — Model Deployment and Prediction Service.
- Docker official docs — Best practices for writing Dockerfiles.
- Dive — TUI para inspeccionar capas y eficiencia.
- Docker Scout / Trivy — vulnerability scanning.
- distroless — imágenes mínimas de Google.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 199 — Clase 199 — APIs con FastAPI sirviendo modelos

Parte: 4 — MLOps · Fuente: Huyen cap. 11 + docs FastAPI + Ramalho cap. 5. Duración estimada: 80 min.

Objetivo

Exponer un modelo entrenado como REST API con FastAPI: validación de input con Pydantic, batching, async, healthcheck, métricas Prometheus, OpenAPI auto-generado. Target: p99 latency <100 ms y throughput >500 req/s en un solo nodo CPU.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un servicio FastAPI con endpoints POST /predict, POST /predict-batch, GET /health, GET /metrics.
- Definir schemas de input/output con pydantic.BaseModel y obtener validación + docs gratis.

- Usar lifespan events para cargar el modelo una sola vez (no por request).
- Loadtestear con locust y medir latency p50/p95/p99 + throughput.
- Decidir entre sync def y async def según si el predict es CPU-bound o I/O-bound.

Temas

#	Tema	Por qué importa
1	ASGI vs WSGI: por qué FastAPI no es Flask	Async nativo, perf en I/O-bound.
2	Pydantic v2: validación + serialización	Schema = contrato; cambios rompen tests.
3	Lifespan: cargar modelo 1 vez	Sin esto, cada request reabre joblib.load.
4	Sync vs async para predict	CPU-bound → sync (deja al thread pool); I/
5	Batching: /predict-batch	100 predicciones en 1 request, no en 100.
6	Observabilidad: /health, /metrics, logs es	Lo que pide el oncall a las 3 AM.

Definiciones y características

- FastAPI: framework ASGI (no WSGI). Web server: uvicorn (single-process) o gunicorn -w N -k uvicorn.workers.UvicornWorker (multi-worker).
- ASGI (Asynchronous Server Gateway Interface): protocolo Python para apps async. Permite async def endpoints, websockets, streaming.
- Pydantic v2: validación + serialización con modelos tipados. ~20× más rápido que v1 (re-escrito en Rust).
- lifespan event (@asynccontextmanager): hook para inicializar recursos (cargar modelo, abrir DB) cuando el server arranca y limpiarlos al apagar. Reemplaza el deprecado @app.on_event("startup").
- Sync vs async endpoint: en FastAPI, def predict(...) (sync) corre en un thread pool — bueno para CPU-bound o libs que no son async. async def predict(...) corre en el event loop — bueno para I/O-bound (DB, HTTP a otro servicio). Si tu predict es solo model.predict(X) (CPU): sync.
- Batching: agrupar N predicciones en un solo request. Reduce overhead de HTTP (~5 ms por request) y permite vectorizar (model.predict(matrix) es ~10× más rápido que 100 model.predict(vector)).
- Healthcheck: endpoint que devuelve 200 si el servicio puede servir tráfico. K8s lo usa para livenessProbe (reiniciar pod si falla) y readinessProbe (sacar del LB si falla).

Dataset / recursos

- Modelo: cualquiera entrenado en clases previas — usamos sklearn por simplicidad.
- Librerías: fastapi, uvicorn[standard], pydantic>=2, prometheus-fastapi-instrumentator, locust (loadtest).

Ejercicios

1. API mínima: POST /predict con IrisInput(features: list[float]) → IrisOutput(class: int, proba: list[float]). Levantá con uvicorn app:app --reload. Abrió /docs (OpenAPI Swagger UI). Confirmá que probar desde la UI funciona.
2. Lifespan: cargá el modelo en un lifespan y guardalo en app.state.model. Verificá que el modelo se carga UNA vez (print al inicio) aunque hagas 100 requests.
3. Batching: agregá POST /predict-batch con BatchInput(rows: list[list[float]]). Medí latencia de 100 predicciones individuales vs 1 batch de 100.
4. Async vs sync: simulá un predict que llama a una API externa (await httpx.AsyncClient().get(...)). Compará def (bloquea thread pool) vs async def (libera event loop). Loadtest con 200 concurrent users.
5. Observabilidad: agregá prometheus-fastapi-instrumentator → /metrics. Healthcheck /health que devuelve {"status": "ok", "model_loaded": bool}. Loggeá cada request con logger.info (JSON).

Homework verificable

Servicio FastAPI + Dockerfile (Clase 198) con:

1. Endpoints POST /predict, POST /predict-batch, GET /health, GET /metrics, GET /docs.
2. Pydantic models con validación: features debe tener exactamente 4 floats positivos.
3. lifespan que carga model.pkl una vez al startup.
4. locustfile.py que simula 100 users concurrentes durante 60 s.
5. Reporte de loadtest con p50/p95/p99 latency y RPS, justificando si el target (<100 ms p99, >500 RPS) se cumple.

Criterio de aceptación: `curl -X POST localhost:8000/predict -d '{"features":[-1,2,3,4]}' -H 'content-type:application/json'` devuelve HTTP 422 (validación rechaza valor negativo). El loadtest cumple p99 <100 ms en al menos un workload.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Latency altísima (p99 >500 ms) con un mode	Estás cargando <code>joblib.load("model.pkl")</code> de
Error RuntimeError: This event loop is alr	Mezclaste <code>asyncio.run(...)</code> dentro de un en
422 Unprocessable Entity en requests que p	Pydantic v2 es estricto: <code>list[float]</code> recha
Single worker satura 1 CPU	uvicorn app:app corre 1 worker. Fix: gunic
/docs está activo en producción	Por default FastAPI expone /docs, /redoc,
Healthcheck devuelve 200 aunque el modelo	Healthcheck simplista. Fix: chequear app.s

Preguntas frecuentes

¿FastAPI o Flask?

Para servir modelos hoy: FastAPI. Pros: async nativo, validación con Pydantic (no request.json + checks manuales), OpenAPI gratis, performance superior en benchmarks (~3× Flask). Flask sigue siendo elegible para apps simples o equipos con experiencia previa.

¿Cuándo async def y cuándo def?

Regla: si el endpoint solo hace CPU-bound (cargar modelo, predecir): def (FastAPI lo manda al thread pool, no bloquea el event loop). Si hace I/O-bound (await DB, await HTTP): async def. Si mezclás requests (sync HTTP) en un async def: bloqueás el loop. Cambialo a `httpx.AsyncClient`.

¿Servir con uvicorn o gunicorn?

Dev: uvicorn --reload. Prod: gunicorn -w N -k uvicorn.workers.UvicornWorker (N = 2 * cores + 1). Razón: gunicorn maneja restarts, multi-proceso, signals. En K8s con HPA + 1 worker/pod: uvicorn solo está bien.

¿Cómo manejo modelos grandes (>1 GB) que tardan en cargar?

(1) lifespan (no re-cargar). (2) readinessProbe con initialDelaySeconds: 60 — pod no recibe tráfico hasta cargar. (3) `model.eval()` + `torch.jit.script` (PyTorch) o `onnxruntime` (cualquier framework) — más rápido en inferencia que el framework original. (4) Para LLMs: vLLM/TGI con paged attention (Clase 165).

¿Validación strict de Pydantic me rompe clientes legacy?

Sí, si pasaban tipos laxos. Pydantic v2 puede ser permisivo con `model_config = {"strict": False}` o usar `validators` (`field_validator`). Lo correcto a mediano plazo: docs claras + versioning de API (/v1/predict vs /v2/predict).

¿FastAPI sirve modelos PyTorch/TF directamente?

Sí, pero para producción seria conviene un inference server dedicado: TorchServe, TensorFlow Serving, NVIDIA Triton (multi-framework, batching dinámico, GPU optimal). FastAPI queda como gateway/proxy con auth y reglas de negocio. Para casos simples sklearn/XGBoost: FastAPI alcanza.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 11.
- FastAPI docs — Tutorial y Advanced User Guide.
- Pydantic v2 migration guide — qué cambió desde v1.
- Locust — loadtest distribuido en Python.
- NVIDIA Triton — alternativa para producción seria.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 200 — Clase 200 — Kubernetes para servir modelos a escala

Parte: 4 — MLOps · Fuente: Huyen cap. 11 + Burns et al., Kubernetes: Up and Running (3ª ed.) + docs k8s. Duración estimada: 90 min.

Objetivo

Desplegar el contenedor de Clase 198–199 en Kubernetes con Deployment + Service + Ingress, autoescalar con HPA (CPU + custom metrics), hacer rolling updates seguros, y configurar livenessProbe/readinessProbe/resources correctamente. Aprender los 5 manifests mínimos que necesita cualquier servicio de inferencia.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir manifests YAML para Deployment, Service, ConfigMap, Secret, HPA, Ingress.
- Diferenciar livenessProbe (reinicia pod) de readinessProbe (saca del LB) de startupProbe (gracia inicial para modelos lentos).
- Configurar resources.requests/limits y entender por qué un pod sin requests es OOM-killable y sin limits es noisy-neighbor.
- Hacer kubectl rollout/rollback y entender los maxSurge/maxUnavailable del rolling update.
- Diagnosticar CrashLoopBackOff, ImagePullBackOff, Pending, Evicted con kubectl describe y kubectl logs.

Temas

#	Tema	Por qué importa
1	Pod, Deployment, ReplicaSet, Service	Las abstracciones core.
2	Probes (liveness, readiness, startup)	Diferencia entre "el pod murió" y "el pod
3	Resources: requests vs limits	Scheduling + OOMkill control.
4	HPA: CPU + custom metrics (latency, queue	Autoescalado más allá de "CPU alta".
5	Rolling update + rollback	Deploy sin downtime y vuelta atrás.
6	Ingress + service mesh (mention)	Cómo expone tráfico externo.

Definiciones y características

- Pod: unidad de scheduling. 1+ containers que comparten network + storage. Para ML: típicamente 1 container por pod.
- Deployment: declara "quiero N réplicas de este pod con esta imagen". Gestiona un ReplicaSet que gestiona los pods. Rolling updates son responsabilidad del Deployment.
- Service: load balancer estable interno. Tipo ClusterIP (default, interno), NodePort (expone puerto en cada node), LoadBalancer (cloud LB).
- livenessProbe: si falla, K8s reinicia el container. Apuntar a /health con timeouts generosos.
- readinessProbe: si falla, K8s saca el pod del Service (no recibe tráfico) pero NO lo reinicia. Apuntar a /ready que chequee modelo cargado + deps disponibles.
- startupProbe: ventana de gracia para containers lentos en arrancar (modelos grandes). Mientras corre, liveness/readiness no se ejecutan. Una vez OK, pasan a evaluarse normalmente.
- resources.requests: lo que el scheduler reserva para el pod. Sin requests, el pod va a "Best Effort" → primero en ser killed con presión de memoria.
- resources.limits: tope duro. Si memoria > limit: OOMKill. Si CPU > limit: throttling (no kill).
- HPA (Horizontal Pod Autoscaler): escala réplicas entre minReplicas y maxReplicas según métrica (default: CPU%). Custom metrics (latency p99, queue depth) requieren metrics-server + adapter (Prometheus Adapter).
- Rolling update: estrategia default. Crea pods nuevos (maxSurge), espera readiness, mata viejos (maxUnavailable). Garantiza disponibilidad continua.
- Rollback: kubectl rollout undo deployment/<name>. Vuelve al último ReplicaSet exitoso. Casi instantáneo si los pods viejos siguen referenciados.

Dataset / recursos

- Cluster: minikube, kind, o k3d para local. Equivalente en cloud: GKE/EKS/AKS.
- Imagen: la de Clase 198 (iris-api:v1).
- Herramientas: kubectl, kustomize (built-in) o helm.

Ejercicios

1. Cluster local: kind create cluster --name ml. Pusheá la imagen local con kind load docker-image iris-api:v1 --name ml. Verificá con kubectl get nodes.
2. Deployment + Service: aplicá los YAML del notebook. kubectl get pods -w mientras los 3 pods arrancan. kubectl port-forward svc/iris-api 8000:80 y pegale con curl localhost:8000/predict.
3. Probes: cambiá livenessProbe a apuntar a /wrong-endpoint. Observá con kubectl get pods -w cómo entra en CrashLoopBackOff. Revertí.
4. HPA: kubectl apply -f hpa.yaml con targetCPUUtilizationPercentage: 50. Generá carga con kubectl run loadtester --image=busybox -it --rm -- /bin/sh -c "while true; do wget -q -O- iris-api/predict; done". Observá kubectl get hpa -w escalar de 3 → 10.
5. Rolling update + rollback: cambiá la imagen a iris-api:v2 (versión rota a propósito). kubectl rollout status deployment/iris-api debería timeoutear. kubectl rollout undo deployment/iris-api y verificá recuperación.

Homework verificable

Cluster (local o cloud) con:

1. Manifests YAML para Deployment (3 réplicas, probes, resources), Service (ClusterIP), HPA (CPU 50%, min 3 max 10), Ingress.
2. Imagen pusheada a un registry (Docker Hub o ECR).

3. Loadtest que dispara HPA y demostrar escalado en logs/screenshots.
4. Rolling update a una versión v2 exitosa, después rollback con `kubectl rollout undo`.
5. README con comandos `kubectl apply`, `port-forward`, troubleshooting con `describe`.

Criterio de aceptación: `kubectl get pods -l app=iris-api` muestra 3-10 pods escalando con la carga, `kubectl get hpa` reporta target CPU, y `curl <ingress-host>/predict` funciona desde fuera del cluster.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Pods en Pending infinito	Cluster sin recursos para requests. Fix: k
CrashLoopBackOff	Container falla al iniciar y K8s reintentará
ImagePullBackOff	No puede bajar la imagen. Fix: chequear no
Liveness mata el pod durante startup del m	livenessProbe arranca antes que el modelo
OOMKilled silencioso	<code>kubectl describe pod</code> muestra "OOMKilled" e
HPA no escala aunque CPU está alta	metrics-server no está instalado o el pod

Preguntas frecuentes

¿K8s vs Cloud Run vs ECS Fargate vs Lambda?

K8s es el más flexible y portable; el más pesado en mantenimiento. Cloud Run / Fargate: contenedor managed, escalan a 0, sin K8s machinery — buen punto intermedio. Lambda (Clase 201): serverless puro, frío, máx 15 min, ideal para batch chico. Para ML serving 24/7: K8s o Cloud Run.

¿1 worker por pod o N workers por pod?

Recomendación K8s: 1 worker por pod. K8s se encarga del fleet (HPA, rolling update, restart). Múltiples workers complican observabilidad por pod, hacen restart costoso, y suelen reproducir lo que ya hace K8s.

¿GPU pods?

Necesitas un node pool con GPUs y el NVIDIA device plugin instalado. En el pod: `resources.limits["nvidia.com/gpu"]`: 1. La imagen base debe ser CUDA-compatible (`nvidia/cuda:...` o `pytorch/pytorch:cuda`).

¿Helm o Kustomize?

Kustomize (built-in en `kubectl`): overlays por entorno (dev/staging/prod) sin templating string-based. Helm: package manager con templates Go, más expresivo pero también más complejo. Para empezar: Kustomize. Para distribuir charts (ej. instalar Prometheus): Helm.

¿Cómo manejo secrets?

Secret resource (base64, NO encripta por default). Para producción: enable encryption-at-rest en etcd, o mejor: External Secrets Operator que sincroniza de AWS Secrets Manager / Vault → K8s Secrets. Nunca commitar secrets a git, ni siquiera "fake" — usá `kubectl create secret` o secrets externos.

¿Service mesh (Istio/Linkerd) es necesario?

No para empezar. Resuelve: mTLS automático entre pods, retries/timeouts/circuit breaking, canary releases (Clase 204) sin tocar código, observabilidad fina. Agrega complejidad (sidecar por pod, control plane). Vale la pena cuando: ≥10 microservicios, requerimiento mTLS interno, o canary serio.

Referencias

- Burns et al. Kubernetes: Up and Running (3ª ed., O'Reilly, 2022).
- Kubernetes docs — Workloads, Services, Scheduling, Preemption and Eviction.
- Probes guide — patrones correctos.
- Horizontal Pod Autoscaler walkthrough.
- Kind quickstart — cluster local en segundos.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 201 — Clase 201 — Serverless ML: AWS Lambda, GCP Cloud Functions

Parte: 4 — MLOps · Fuente: Huyen cap. 11 + docs Lambda container images + Cloud Functions 2nd gen. Duración estimada: 75 min.

Objetivo

Desplegar un modelo como función serverless que escala de 0 a N sin gestionar servidores. Decidir cuándo serverless gana (tráfico bursty, batch chico, no podés mantener infra) y cuándo pierde (cold start crítico, modelo >1 GB, latencia <50 ms requerida).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Empaquetar un modelo sklearn/XGBoost como Lambda Container Image (hasta 10 GB) o Cloud Function 2nd gen (build automático con Buildpacks).
- Mitigar cold starts con: provisioned concurrency (Lambda), min-instances=1 (Cloud Functions), o snapshot-based init (SnapStart).
- Configurar API Gateway o HTTP trigger para exponer la función como REST.
- Calcular costo: \$/M invocations × duration × memory, vs un pod K8s 24/7.
- Reconocer límites: timeout 15 min (Lambda), payload 6 MB sync / 256 KB async, disco efímero 512 MB /tmp (10 GB con ephemeral-storage).

Temas

#	Tema	Por qué importa
1	Modelo de ejecución: scale-to-zero, reques	Diferente a K8s/EC2 24/7.
2	Cold start vs warm	El bug que arruina la UX.
3	Package formats: ZIP (≤250 MB) vs Containe	Para modelos ML: container.
4	Cost model: invocations + GB-seconds	Cruce con K8s al ~1M req/día sostenido.
5	Provisioned concurrency / min-instances	Cambia la economía y la latencia.
6	Cuándo NO usar serverless	Modelos grandes, latencia <50 ms, stateful

Definiciones y características

- Cold start: tiempo entre llegada del request y arranque del runtime + carga del modelo en una instancia nueva. Lambda Python: ~300 ms (runtime) + tu init code. Con modelo grande: 5-15 s.
- Warm: instancia ya inicializada que recibe request → latencia = solo predict. Lambda mantiene warm ~5-15 min sin invocations (no garantizado).
- Provisioned Concurrency (Lambda): pagás por mantener N instancias warm permanentemente. Elimina

cold starts a cambio de costo fijo.

- SnapStart (Lambda, Java/Python 3.12+): toma snapshot del runtime después de init y lo restaura — cold start <500 ms aún con modelos grandes.
- Cloud Functions 2nd gen (basado en Cloud Run): timeouts hasta 60 min, hasta 32 GB RAM, mejor cold start que 1st gen, y la opción `--min-instances=1` para mantener warm.
- API Gateway: pone HTTP delante de Lambda. Maneja auth, throttling, CORS. Cobra extra por request — para tráfico volumétrico, Lambda Function URL (gratis) es alternativa.
- Provisioned vs On-Demand: provisioned escala a 0 manteniendo N warm. On-demand escala a 0 puro. Trade-off costo vs latency tail.

Dataset / recursos

- Modelo: LogisticRegression sobre Iris (chico — buen fit serverless).
- Tools: AWS CLI + SAM (Serverless Application Model), o `gcloud functions deploy`.
- Imagen base Lambda: `public.ecr.aws/lambda/python:3.12`.

Ejercicios

1. Lambda con container: buildéa un Dockerfile basado en `public.ecr.aws/lambda/python:3.12`, copiá `app.py` con `lambda_handler(event, context)`, y `model.pkl`. Push a ECR. `aws lambda create-function` con `--package-type Image`.
2. API Gateway: creá una API HTTP y conectala a la Lambda. `curl <api-url>/predict -d '{"features":[5.1,3.5,1.4,0.2]}'`. Medí latencia primera vs segunda llamada (cold vs warm).
3. Provisioned Concurrency: configurá `provisioned-concurrency=1` en la Lambda. Vuelve a medir — debería eliminar el cold start.
4. Cloud Functions equivalent: `gcloud functions deploy iris --gen2 --runtime=python312 --trigger-http --source=. --entry-point=predict --memory=512Mi --min-instances=1`. Compará cold start con Lambda.
5. Cost calc: asumí 100 req/s sostenido, modelo en RAM 512 MB, 80 ms p99. Calculá \$/mes en Lambda vs un pod K8s con 4 vCPU 24/7. Encontrá el punto de cruce.

Homework verificable

1. Lambda function (container image) que sirve predict con cold start <3 s y warm <80 ms.
2. API Gateway HTTP exponiendo la Lambda.
3. CloudWatch alarm que dispara si Errors > 5 en 5 min, o Duration P99 > 1000 ms.
4. Script python `cost_analysis.py` que dado `req_per_second_mean`, `req_size`, `mem_mb`, `duration_ms` calcula \$/mes en (a) Lambda on-demand, (b) Lambda con PC=2, (c) ECS Fargate equivalente, (d) K8s con 3 pods t3.medium.
5. Recomendación escrita: para tu workload, qué opción elegir y por qué.

Criterio de aceptación: `curl <api>/predict` responde en <100 ms (warm) y <3 s (cold), el cost analysis identifica el punto de cruce correcto (típicamente serverless gana <~500k requests/día sostenido).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Cold start de 30 s	Carga del modelo dentro del handler. Fix:
Task timed out after 3.00 seconds	Default Lambda timeout es 3 s. Fix: <code>aws la</code>
Imagen ZIP excede 250 MB descomprimido	Probablemente trayendo <code>numpy+scipy+pandas</code> .
Pérdida de estado entre invocations	Esperado: cada instancia es efímera. Fix:
Costos explotaron	(1) Provisioned concurrency olvidada. (2)
<code>errorMessage: Unable to import module</code>	Falta una dep o estructura incorrecta. Fix

Preguntas frecuentes

¿Cuándo serverless conviene para ML?

(1) Tráfico bursty con valles largos (1000 req/min picos, 0 req/min noche). (2) Modelo chico (<1 GB) y latencia tolerante (>200 ms p99 OK). (3) Equipo sin DevOps. (4) Predicciones batch ad-hoc (S3 trigger). Suma ~70% de los casos típicos en startups.

¿Cuándo NO?

(1) Latencia estricta <50 ms — cold start es matador. (2) Modelo grande (>3 GB) — init carísimo. (3) Throughput sostenido >1000 req/s 24/7 — sale más caro que K8s. (4) Stateful: caches calientes, sessions, websockets persistentes.

¿Lambda o SageMaker Serverless Inference?

SageMaker Serverless: built-in para ML (deploy con un click desde model registry, batch transform). Misma idea de scale-to-zero. Latencia similar. Pros: integración con SageMaker; cons: vendor-locked y más caro por request. Para ML puro en AWS: SageMaker. Para mezcla ML + lógica de negocio: Lambda.

¿GPU en serverless?

Lambda no tiene GPU. Cloud Run/Functions 2nd gen tampoco (al momento de escribir). Para inferencia GPU serverless: Modal, Banana, Replicate (3rd party SaaS). Sino: K8s con GPU nodes.

¿Cómo monitoreo cold start vs warm?

CloudWatch: métrica InitDuration (solo en cold), Duration (siempre). Lambda Insights agrega CPU/mem/network. Para correlacionar con latencia user-facing: X-Ray tracing.

¿Container Image vs Layer?

Lambda Layers: hasta 5 layers de 250 MB cada uno (compartidos entre funciones). Container Image: hasta 10 GB, todo en uno. Para ML moderno: Container Image siempre — más simple, sin límite efectivo.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 11.
- AWS Lambda container images for Python.
- Lambda SnapStart for Python.
- Cloud Functions 2nd gen.
- Modal — serverless con GPU para ML.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 202 — Clase 202 — Monitoreo: data drift, model drift, alertas

Parte: 4 — MLOps · Fuente: Huyen cap. 8 + Evidently AI docs + NannyML + Gama et al., A Survey on Concept Drift Adaptation (ACM, 2014). Duración estimada: 80 min.

Objetivo

Detectar antes que el negocio: (a) data drift (la distribución de features cambió), (b) prediction drift (la

distribución de predicciones cambió), (c) concept drift (la relación $X \rightarrow y$ cambió). Configurar alertas que avisen antes de que la métrica de negocio caiga, no después.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Distinguir data drift ($P(X)$ cambia), prediction drift ($P(\hat{y})$ cambia), concept drift ($P(y|X)$ cambia) y elegir el test correcto para cada uno.
- Detectar drift con tests estadísticos: PSI (Population Stability Index), K-S (continuas), chi-cuadrado (categóricas), Wasserstein (más sensible que K-S en colas).
- Usar Evidently AI para generar reportes HTML con todos los tests + visualizaciones, y NannyML para estimar performance sin labels en producción (CBPE).
- Configurar alertas en Grafana / CloudWatch / Slack vía webhook cuando el drift score supera el umbral.
- Reconocer cuándo el "drift" es ruido (re-test con bonferroni) vs señal (acción).

Temas

#	Tema	Por qué importa
1	3 tipos de drift y por qué importan distin	Confundirlos lleva a "rentrenar" sin neces
2	PSI: umbral 0.1 / 0.2	El estándar de la industria; simple, inter
3	K-S, chi-cuadrado, Wasserstein	Tests con propiedades distintas — elegir s
4	Performance estimation sin labels (CBPE)	Cómo saber si el modelo empeora antes de t
5	Reference window vs current window	El qué se compara con qué.
6	Alertas: umbral + cooldown + canal	Sin diseño → alert fatigue.

Definiciones y características

- Data drift (covariate shift): $P_{\text{train}}(X) \neq P_{\text{prod}}(X)$. Ej: en training los users tenían 18-65 años, en prod aparecen muchos >70. El modelo todavía es válido si la relación $X \rightarrow y$ no cambió, pero las predicciones serán menos confiables en zonas no vistas.
- Prediction drift: $P_{\text{train}}(\hat{y}) \neq P_{\text{prod}}(\hat{y})$. La distribución de salidas cambió. A veces es síntoma de data drift, a veces es legítimo (estacionalidad). No siempre malo.
- Concept drift (real drift): $P(y|X)$ cambió — la relación entre features y target. Ej: durante COVID, "salir a la calle" pasó de actividad normal a anómala. Aún sin data drift, el modelo deja de servir. Este es el más grave, y el más difícil de detectar sin labels.
- PSI (Population Stability Index): $\sum (p_i - q_i) * \ln(p_i / q_i)$ sobre bins. Umbrales clásicos: <0.1 estable, 0.1-0.2 cambio menor, >0.2 shift significativo (acción).
- K-S test (Kolmogorov-Smirnov): estadístico = $\sup |F_{\text{ref}}(x) - F_{\text{cur}}(x)|$. p-value bajo → distros distintas. Sensible al centro de la distribución.
- Chi-cuadrado: para variables categóricas. Compara frecuencias observadas vs esperadas.
- Wasserstein distance (Earth Mover's Distance): "cuánta tierra hay que mover" para transformar una distro en otra. Más sensible que K-S a diferencias en cola.
- Reference window: muestra del periodo de training (o un slice estable previo).
- Current window: muestra del periodo a evaluar (último día, última semana). Tamaño del window afecta sensibilidad (chico → ruidoso, grande → lento).
- CBPE (Confidence-Based Performance Estimation, NannyML): estima accuracy o AUC de producción usando solo las probabilidades del modelo (no labels). Asume bien-calibrado.

Dataset / recursos

- Dataset training: California Housing — usado como reference.
- Dataset "producción": California Housing con shift sintético (escalar MedInc $\times 1.5$ para simular inflación, o filtrar HouseAge > 30 para simular nuevo segmento).
- Librerías: evidently, scipy, nannyml, scikit-learn.

Ejercicios

1. PSI manual: bineá MedInc en 10 deciles (con bordes del reference). Calculá p_{ref} , p_{cur} y aplicá la fórmula PSI. Verificá que sin shift $PSI < 0.05$; con shift $\times 1.5$ $PSI > 0.5$.
2. K-S vs Wasserstein: agregale outliers a 5% de la muestra de producción (multiplicá esos por 100). K-S podría no detectar (la mediana no cambió mucho); Wasserstein sí. Reproducí ambos casos.
3. Reporte Evidently: Report(metrics=[DataDriftPreset()]) sobre reference vs current. Guardalo como HTML, abrilo, identificá qué features drifearon y con qué test.
4. Concept drift sin labels (CBPE): con NannyML, fit el estimador sobre reference + predicciones. Aplicalo a current. Compará estimated_accuracy vs actual_accuracy (calculable porque tenés y en este ejercicio).
5. Alerta: escribí un script que (a) calcule PSI por feature, (b) si alguna > 0.2 emite un POST a un webhook Slack/Discord, (c) usá cooldown de 4 h con un archivo last_alert.txt para evitar spamming.

Homework verificable

Sistema de monitoreo que produce:

1. Job diario (Cron / Airflow / GH Actions schedule) que toma el snapshot de producción de las últimas 24 h y lo compara con el reference window.
2. Reporte Evidently HTML guardado en S3 / GCS (timestamped).
3. PSI dashboard en Grafana o Streamlit que muestra evolución por feature en los últimos 30 días.
4. Alerta a Slack si: $PSI > 0.2$ en cualquier feature, O drift score global $>$ umbral, O CBPE estimated_accuracy cayó > 5 puntos.
5. Runbook (runbook.md) con 3 escenarios: (a) drift de feature legítimo (cambio negocio) $\rightarrow$ retrainings, (b) drift por bug en pipeline $\rightarrow$ fix upstream, (c) concept drift $\rightarrow$ A/B test modelo nuevo.

Criterio de aceptación: simular un shift introducido a propósito (multiplicar feature $\times 2$) dispara la alerta en < 24 h. El runbook tiene pasos accionables, no descripciones genéricas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
PSI = 0.3 pero el modelo sigue bien	Es data drift sin concept drift — la distr
Tests dan "drift" en TODO siempre	n muy grande $\rightarrow$ cualquier diferencia es "si
Alerta dispara 50 veces / día	Falta cooldown y/o el threshold es muy agr
Reference window se actualiza cada deploy	Si el reference cambia con cada release, p
Detecto drift pero no sé qué hacer	Falta runbook. Fix: para cada feature crít
CBPE da accuracy muy distinta a la real	El modelo está mal calibrado — CBPE asume

Preguntas frecuentes

¿Cada cuánto retrainear?

Hay 3 estrategias: (1) schedule fijo (cada lunes) — simple pero ciego. (2) trigger por drift ($PSI > 0.3$) — reactivo. (3) trigger por performance degradation (CBPE estima accuracy caída $> X\%$) — mejor. La industria converge a (3) con fallback a (2). Ver Clase 203.

¿PSI o KL divergence?

PSI es simétrica ($PSI(A,B) = PSI(B,A)$) y robusta; KL no. PSI domina en la industria financiera; ML moderno usa también Wasserstein o Earth Mover's Distance. Para empezar: PSI.

¿Mido drift por feature o multivariado?

Por feature es interpretable ("MedInc driftó"). Multivariado (PCA + drift sobre componentes, o Maximum Mean Discrepancy) capta correlaciones nuevas. Ideal: ambos, pero un drift de feature suele ser suficiente para disparar investigación.

¿Cómo evito alertas en feriados/temporadas?

(1) Reference window por época: comparar diciembre 2026 vs diciembre 2025, no vs julio. (2) Modelar estacionalidad explícitamente (incluir month/day_of_week como feature). (3) Suprimir alertas en feriados conocidos.

¿Evidently vs NannyML vs Whylogs vs Arize?

Evidently: open-source, reportes HTML excelentes. NannyML: open-source, especialista en CBPE (estimar performance sin labels). Whylogs: librería de profiling minimalista de WhyLabs. Arize: SaaS comercial, dashboards e integraciones empresariales. Para empezar: Evidently + NannyML.

¿Y si no tengo labels en producción?

Es lo normal en muchos casos. Estrategias: (1) CBPE (NannyML) — estima con probas. (2) Proxy labels — métricas de negocio downstream (CTR, conversión). (3) Active learning — etiquetar un subset random para validar. (4) Shadow scoring — guardas predicciones + features y cuando llegan labels (días después), evaluas.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 8 — Monitoring.
- Evidently AI — reportes y tests.
- NannyML — CBPE y DLE.
- Gama et al., A Survey on Concept Drift Adaptation (ACM Computing Surveys, 2014) — taxonomía clásica.
- Monitoring ML Models: Theory (Tagliabue) — visión práctica.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 203 — Clase 203 — Reentrenamiento programado

Parte: 4 — MLOps · Fuente: Huyen cap. 9 + Airflow docs + Prefect docs. Duración estimada: 75 min.

Objetivo

Convertir el reentrenamiento en un proceso programado, auditado y reversible: DAG que cada N horas/días corre pull-data → validate → train → evaluate → promote-if-better → notify, con shadow/canary (Clase 204) antes del rollout pleno. Decidir entre schedule fijo, trigger por drift y trigger por degradación según el caso.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diseñar un DAG (Airflow/Prefect/Dagster) que orqueste el reentrenamiento completo.
- Implementar el patrón champion-challenger: el modelo Production sigue sirviendo hasta que el challenger demuestra ser mejor en métricas de validación + en shadow.
- Configurar tres estrategias de trigger: cron(0 2 MON), on_drift > threshold, on_performance_degraded.
- Garantizar idempotencia (re-runs no duplican datos) y observabilidad (logs estructurados, alertas en fallas).
- Diferenciar online learning (modelo actualiza con cada nueva muestra) de continual training (re-trainings periódicos).

Temas

#	Tema	Por qué importa
1	Triggers: schedule vs drift vs degradation	Sin trigger correcto, retrainings caros si
2	DAG = grafo de tareas con deps	Airflow/Prefect/Dagster son variaciones de
3	Champion-challenger	Cómo evitar regresiones en producción.
4	Idempotencia	Re-runs no deben duplicar/destruir state.
5	Online vs continual training	Trade-off frescura vs estabilidad.
6	Catastrophic forgetting	Retraining puede olvidar lo aprendido si d

Definiciones y características

- DAG (Directed Acyclic Graph): grafo de tareas con dependencias, ejecutable por un scheduler. Airflow inventó la categoría; Prefect/Dagster/Mage son alternativas modernas.
- Champion: modelo activo en producción (alias @champion en MLflow, Clase 195).
- Challenger: candidato nuevo entrenado. Promueve a champion solo si pasa todos los gates (validation metric + shadow + canary).
- Promotion gate: criterio que debe cumplirse para que el challenger reemplace al champion. Ej: $f1_test > f1_champion + 0.005$ AND no_regression_in_protected_slices.
- Idempotencia: re-ejecutar la misma tarea con los mismos inputs produce el mismo output, sin efectos colaterales acumulativos. Implementación típica: usar execution_date como key, sobrescribir destino, evitar INSERT INTO sin WHERE NOT EXISTS.
- Backfill: re-ejecutar el DAG para fechas pasadas (Airflow soporta nativo). Útil para corregir bugs o llenar gaps.
- Online learning: el modelo actualiza con cada muestra/mini-batch que llega. Funciona bien con SGD-based (LR, RF online, RL). No funciona con XGBoost/RF batch.
- Continual training: re-trainings periódicos (cada hora/día/semana) sobre data acumulada. Más común y más simple que online learning.
- Catastrophic forgetting: cuando retraining sobre data reciente "olvida" patrones que aprendió antes. Mitigación: incluir data histórica en cada retraining, regularización (EWC en deep learning).

Dataset / recursos

- Pipeline target: Airflow local (Docker), o Prefect Cloud free tier, o GitHub Actions schedule.
- Librerías: apache-airflow>=2.9, prefect>=3, mlflow, evidently.

Ejercicios

1. DAG mínimo (Airflow o Prefect): 5 tareas: pull_data → validate_data → train → evaluate → promote. Cada tarea es una función Python. Schedule diario.
2. Champion-challenger: en promote, comparar challenger.f1 vs champion.f1 (leídos de MLflow).

Promover solo si $\text{challenger.f1} > \text{champion.f1} + 0.005$. Sino, logear [skipped] challenger no es significativamente mejor y dejar champion intacto.

3. Idempotencia: ejecutá el DAG dos veces seguidas para la misma fecha. Verificá que no se duplican filas en data/processed/, ni se crean dos registros en MLflow con el mismo execution_date.
4. Trigger por drift: agregá una tarea inicial check_drift que (a) si $\text{PSI} > 0.2$ continúa el DAG, (b) si no, hace raise AirflowSkipException (skipea el resto). Resultado: solo se entrena si hay drift.
5. Backfill: simulá un bug en la tarea validate_data que ya corrió bien por una semana. Fixeá el bug, airflow dags backfill --start-date X --end-date Y. Verificá que se re-procesan los días afectados sin duplicar registros downstream.

Homework verificable

DAG en producción (o local con docker-compose) que:

1. Corre diariamente a las 02:00 UTC.
2. Pull últimos 30 días de data, valida con Great Expectations (Clase 206) — falla con alert si no pasa.
3. Re-entrena, evalúa contra hold-out + slices de fairness, registra en MLflow.
4. Compara contra champion: promueve a @challenger (alias) si métrica supera umbral.
5. Tarea shadow_test que durante 24 h compara predicciones challenger vs champion sobre tráfico real (Clase 204).
6. Tarea promote_champion (manual o auto) que reemplaza alias @champion solo si shadow OK.
7. Alertas Slack en cada falla del DAG + un resumen diario [OK] o [NO TRAIN, no drift].

Criterio de aceptación: en una semana de operación, el DAG corrió 7 veces, ≥ 1 promoción real ocurrió, ≥ 1 skip por no-mejora ocurrió, y ningún re-run duplicó datos.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
DAG corre OK pero modelo no mejora	Probablemente estás re-entrenando solo con
Re-run del DAG duplica filas en la DB de f	Insert sin idempotencia. Fix: INSERT ... O
Catastrophic forgetting visible: f1 sobre	El nuevo retraining sobre data fresca borrar
Champion-challenger nunca promueve	Umbral muy estricto, o el modelo nuevo no
DAG falla pero nadie se entera	Sin alerta en on_failure_callback. Fix: co
TaskGroup con N tareas paralelas satura el	Sin paralelismo control. Fix: en Airflow,
MLflow runs sin tag de dag_run_id	Cuando algo falla, no podés trazar qué cor

Preguntas frecuentes

¿Airflow, Prefect, Dagster, Mage, Kubeflow Pipelines?

Airflow: el estándar, máxima madurez, syntax verbosa, multi-cloud. Prefect 3: API Python moderna, hybrid execution (workers locales + cloud). Dagster: orientado a software-defined assets (data-aware). Mage: notebooks-first, UI fuerte. Kubeflow Pipelines: K8s-native, orientado a ML específicamente. Para empezar en ML: Prefect o Dagster suelen ser más rápidos que Airflow.

¿Cada cuánto retrainear?

Depende del drift rate del dominio: NLP general (cambios estacionales): mensual. Fraud (atacantes adaptativos): diario u online. Recommendation (catalog drift): horario. Empezar conservador y acelerar si las métricas justifican.

¿Cuándo NO promover automáticamente?

(1) Decisiones de alto impacto (préstamos, salud): siempre human-in-the-loop. (2) Cuando el A/B test online no es factible: requerir aprobación manual. (3) Cuando el dataset tiene mucho ruido reciente: filtrar antes de retrainar.

¿schedule_interval o trigger por evento?

Para producción crítica: ambos. Schedule fijo como red de seguridad ("al menos 1 vez/día") + trigger por drift/degradación para ser reactivo. Falla del trigger no deja al sistema sin retraining.

¿Cómo manejo training caro (días) en un DAG?

Tarea train no corre en el worker Airflow — manda job a Vertex AI / SageMaker / Ray cluster vía API. La tarea Airflow es un proxy que polea hasta completion (SageMakerTrainingOperator lo hace).

¿Y si el retraining empeora producción?

Por eso existen los gates: shadow → canary → full rollout. Si la métrica online cae después de canary 10%: rollback automático del alias @champion al modelo previo, mantenido en el registry.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 9 — Continual Learning and Test in Production.
- Airflow docs — DAGs y operators.
- Prefect docs — flows modernos.
- Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks (PNAS, 2017) — EWC.
- SageMaker Pipelines — equivalente managed AWS.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 204 — Clase 204 — Shadow deployment y canary releases

Parte: 4 — MLOps · Fuente: Huyen cap. 11 + Fowler, Continuous Delivery + Istio traffic management.
Duración estimada: 75 min.

Objetivo

Desplegar un modelo nuevo sin arriesgar producción: primero en shadow (recibe tráfico real pero sus predicciones no se devuelven al usuario), después canary (1% → 5% → 25% → 100% del tráfico). Convertir "creo que esto es mejor" en "lo medí en producción real".

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar shadow mode: doble call (champion + challenger), respuesta del champion al user, log de ambas para análisis offline.
- Configurar canary release progresivo (1% → 5% → 25% → 100%) con K8s + Istio, o con feature flag a nivel app (LaunchDarkly, Unleash).
- Definir rollback automático: si la métrica del canary degrada >X%, volver al 100% champion en <5 min.
- Implementar A/B test correcto: muestra del mismo segmento, métrica primaria pre-registrada, poder estadístico calculado.

- Diferenciar shadow (sin riesgo, costo doble) de canary (riesgo limitado, costo nuevo solo).

Temas

#	Tema	Por qué importa
1	Shadow vs canary vs blue-green vs A/B	4 estrategias, propósitos distintos.
2	Implementación: app-level vs infra-level	Feature flag (LaunchDarkly) vs Istio/AWS A
3	Métrica de "salud" del canary	Latency, error rate, business KPI proxy.
4	Rollback automático	Sin esto, canary es solo "esperar a que al
5	Análisis de shadow data	Comparación distribucional (KS) + métricas
6	A/B test riguroso	Effect size + poder + duración mínima.

Definiciones y características

- Shadow deployment: el modelo nuevo recibe el mismo tráfico que el viejo, predice, pero su predicción NO se devuelve al usuario. Se loggea para análisis. Ventaja: cero riesgo. Costo: 2× compute.
- Canary release: % del tráfico real es servido por el modelo nuevo. Empezar 1%, escalar gradual. Si métricas OK al 100%, el canary se convierte en champion. Si no, rollback.
- Blue-green: dos versiones desplegadas simultáneamente (blue=actual, green=nueva). Switch del LB de blue→green en un momento puntual. Rollback = switch back. Sin gradualidad, pero rollback instantáneo.
- A/B test: comparación estadística entre dos variantes con asignación random por usuario (sticky). Mide impacto en KPI de negocio (no solo en accuracy). Requiere poder estadístico calculado.
- Sticky assignment: el mismo usuario siempre cae en la misma variante (hash por user_id). Evita que un usuario vea ambos modelos y "engañe" la medición.
- Rollback automático: cuando alguna métrica clave del canary cae bajo un umbral (latency p99 > X, error rate > Y%, conversion rate cae > Z%), el sistema vuelve el peso del canary a 0 sin intervención humana.
- Guardrail metric: métrica que NO querés que empeore aunque la primaria mejore. Ej: latency, errors. Si guardrail rompe → rollback aunque "el modelo prediga mejor".

Dataset / recursos

- Stack ejemplo: FastAPI + Istio (canary) o feature flag in-process (shadow simple).
- Librerías: scipy.stats (A/B test), numpy.

Ejercicios

1. Shadow en proceso: en el FastAPI de Clase 199, agregá la lógica: cargá model_champion y model_challenger. En /predict, predecí con ambos, devolvé solo champion, log los dos en JSON. Después de 1000 requests, analizá la distribución de diferencias.
2. Canary con feature flag: implementá un toggle CANARY_PERCENT (env var). Si random.random() < CANARY_PERCENT/100, usá challenger. Sticky: hash por user_id para que el mismo user vea siempre lo mismo dentro del test.
3. Canary con Istio: VirtualService con weight: 95 / 5. kubectl apply y verificá distribución de tráfico con kubectl logs.
4. Rollback automático: agregá un sidecar (Python script) que cada 60 s consulta Prometheus: si latency_p99{model=challenger} > 200ms, cambia el weight a 100 / 0 automáticamente.
5. A/B test riguroso: calculá tamaño de muestra para detectar $\delta = 0.02$ en accuracy con $\alpha=0.05$, power=0.8. Corré el A/B test el tiempo necesario para acumular ese N. Reportá p-value + CI de la diferencia.

Homework verificable

Sistema de deployment con:

1. Modelo champion en producción sirviendo 100% del tráfico.
2. Endpoint /admin/canary que setea peso del challenger (sticky por user_id hash).
3. Métricas en Prometheus: predictions_total{model, class}, latency_seconds{model}, error_rate{model}.
4. Alerta + auto-rollback si: $\text{latency_p99}\{\text{model}=\text{challenger}\} > 1.2 * \text{latency_p99}\{\text{model}=\text{champion}\}$ durante 5 min seguidos.
5. Dashboard Grafana con: distribución de tráfico, latency por modelo, agreement rate champion-challenger, business KPI proxy.
6. Runbook documentando los pasos: shadow 7 días → canary 1% 24 h → 5% 24 h → 25% 48 h → 100%.

Criterio de aceptación: simular un challenger con `time.sleep(0.5)` artificial (alta latencia). El rollback automático debe activarse en <10 min, sin pérdida de servicio para usuarios fuera del canary.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Canary muestra mejor accuracy pero peor bu	Métrica offline (accuracy) ≠ métrica onlin
El A/B test "ganó" pero la decisión no se	Regression to the mean + tamaño de muestra
El user ve modelos distintos en distintos	Asignación no es sticky. Fix: hash por use
Shadow no es "free" como prometieron — el	2× compute requiere 2× capacidad. Fix: dim
Canary a 5% no muestra estadística signifi	El N es chico — $5\% \times N$ usuarios típicos re
Rollback automático se dispara por ruido	Umbral muy estricto o ventana muy chica. F

Preguntas frecuentes

¿Shadow obligatorio antes de canary?

Para modelos críticos: sí. Shadow detecta bugs evidentes (modelo cae, output con NaN) sin afectar usuarios. Canary asume que el modelo funciona y mide impacto. Si saltás shadow, podés desplegar un modelo roto al 1% real.

¿Canary o blue-green?

Canary: gradual, detección temprana, recovery limitado. Blue-green: rollback instantáneo (un switch), sin gradualidad — el 100% se va o no se va. Para ML: canary es más común (cambios en distribución de tráfico revelan problemas que blue-green no).

¿Tengo que usar Istio?

No. Alternativas:

- AWS ALB/NLB con weighted target groups
- GCP Cloud Load Balancing weighted backends
- App-level feature flag (LaunchDarkly, Unleash) — más simple, opera fuera del infra
- Nginx con split_clients

Istio agrega valor cuando ya tenés service mesh.

¿Cómo elijo la métrica primaria del A/B?

Debe ser: (1) directly tied to business value (revenue, retention, conversion), (2) medible rápido (mejor:

detectable en días, no meses), (3) una sola (multi-metric = comparison-shopping, mala práctica). Métricas técnicas (accuracy, AUC) son proxies — útiles pero no la métrica final.

¿Y si el canary mejora un segmento y empeora otro?

Análisis por segmento siempre. Si segmento A mejora 10% y segmento B empeora 5%: discutir trade-off con producto. Algunas opciones: (a) modelo distinto por segmento, (b) re-entrenar con foco en B, (c) decidir explícitamente que A pesa más.

¿Stat test correcto para A/B con accuracy?

Para proporciones (CTR, conversion): test de proporciones con corrección de continuidad. Para medias (latencia, revenue/user): t-test de Welch (no asume varianzas iguales). Para counts (clicks/usuario): Mann-Whitney o bootstrap (típicamente no-normal). Ver Partes 3 (clases 176-178) para más detalle.

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 11 — Model Deployment y Continuous Delivery.
- Fowler, M. — CanaryRelease y BlueGreenDeployment.
- Istio Traffic Splitting tutorial.
- Kohavi, R. et al. Trustworthy Online Controlled Experiments (Cambridge, 2020) — la biblia del A/B test.
- LaunchDarkly / Unleash — feature flags as a service.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 205 — Clase 205 — Interpretabilidad: SHAP, LIME, PDP, ICE

Parte: 4 — MLOps · Fuente: Molnar, Interpretable ML (2ª ed.) + Lundberg & Lee (2017, NIPS, SHAP paper) + Ribeiro et al. (2016, KDD, LIME paper). Duración estimada: 80 min.

Esta clase es complementaria a la Clase 079 (SHAP profundo, Parte 1). Acá el foco es producción: cómo integrar interpretabilidad al servicio (/explain endpoint), cómo escalar SHAP a grandes datasets, y cómo presentar explicaciones a stakeholders no técnicos.

Objetivo

Hacer interpretabilidad deployable: exponer un endpoint /explain que devuelva la atribución por feature de una predicción individual (SHAP/LIME), generar reportes globales (PDP/ICE) al promover un modelo, y entender los tres trade-offs: fidelidad vs simplicidad, global vs local, exacto vs aproximado.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar local (esta predicción) vs global (modelo en general) y model-agnostic vs model-specific.
- Usar SHAP con TreeExplainer (rápido, exacto, árboles), KernelExplainer (lento, model-agnostic), DeepExplainer (redes neuronales) y Partition (recientes).
- Usar LIME para explicaciones model-agnostic en texto/imágenes/tabular.
- Generar PDP (Partial Dependence Plot) e ICE (Individual Conditional Expectation) con sklearn.inspection.

- Decidir cuándo SHAP es lo correcto y cuándo es overkill (ej. modelo lineal: usar coeficientes directamente).

Temas

#	Tema	Por qué importa
1	Local vs global, model-agnostic vs specifi	Vocabulario antes de elegir herramienta.
2	SHAP: TreeExplainer vs KernelExplainer	Exacto y rápido vs aproximado y lento.
3	LIME para texto/imágenes	Cuando SHAP no aplica.
4	PDP + ICE: efecto marginal vs heterogeneid	Lo que cambia con cada feature.
5	Endpoint /explain en producción	Latencia, caching, on-demand vs pre-comput
6	Comunicar a stakeholders no técnicos	Bar chart con top 5 features ≠ paper acade

Definiciones y características

- Interpretabilidad local: explica una predicción individual. "¿Por qué a este cliente lo categorizaste como riesgo?".
- Interpretabilidad global: explica el modelo en agregado. "¿Qué features pesan más en general?".
- Model-agnostic: funciona sin importar el modelo (LIME, KernelSHAP, PDP). Pro: portable. Con: más lento, aproximaciones.
- Model-specific: usa estructura del modelo (TreeSHAP usa árboles). Pro: rápido, exacto. Con: solo para esa familia.
- SHAP values: contribución de cada feature al cambio respecto a la predicción base, basado en teoría de juegos (valores de Shapley). Propiedades garantizadas: efficiency (suman al output), symmetry, dummy, additivity.
- TreeExplainer: para tree-based (XGBoost, LightGBM, CatBoost, sklearn RF). Polynomial time, exacto.
- KernelExplainer: agnostic. Aproxima Shapley values con regresión lineal local con kernel. $O(2^n)$ para exacto, K samples para aprox.
- PDP (Partial Dependence Plot): efecto promedio de una feature j sobre la predicción, marginalizando las demás. Asume independencia entre features (riesgo: si correladas, PDP miente).
- ICE (Individual Conditional Expectation): un PDP por instancia, sin promediar. Permite ver heterogeneidad (si el efecto de la feature varía por sub-grupo).
- LIME: aprende un modelo lineal/árbol simple alrededor de una instancia perturbando los inputs y observando outputs.

Dataset / recursos

- Dataset: California Housing.
- Modelos: XGBRegressor (TreeSHAP) y MLPRegressor (KernelSHAP/DeepSHAP).
- Librerías: shap>=0.45, lime, sklearn>=1.5, matplotlib.

Ejercicios

1. TreeSHAP: entrená XGB sobre California. `explainer = shap.TreeExplainer(model)`. `shap_values = explainer(X_test[:100])`. `shap.plots.waterfall(shap_values[0])` (local) y `shap.plots.beeswarm(shap_values)` (global).
2. KernelSHAP vs TreeSHAP: corré KernelExplainer con 100 background samples sobre el mismo XGB. Compará SHAP values con TreeSHAP. ¿Son iguales? ¿Cuánto tardó cada uno?
3. LIME tabular: LimeTabularExplainer sobre el mismo modelo. Explicá la misma instancia que en SHAP. Compará las features importantes.
4. PDP + ICE: `sklearn.inspection.PartialDependenceDisplay.from_estimator(model, X,`

features=['MedInc', 'HouseAge'], kind='both'). Identificá si hay no-linealidad y/o heterogeneidad.

- Endpoint /explain: extendé el FastAPI (Clase 199) con POST /explain que devuelve top-5 features + SHAP values + base value, para una instancia. Medí latencia — TreeSHAP debería ser <50 ms.

Homework verificable

Servicio FastAPI con:

- POST /predict (de Clase 199).
- POST /explain que devuelve {"prediction": float, "base_value": float, "top_features": [{"name": "MedInc", "shap": 0.23}, ...]}.
- Endpoint GET /global-explanation que devuelve PDP + global SHAP precomputados (cacheados al startup).
- UI HTML básica que muestra waterfall plot (SHAP) para inputs interactivos.
- README que justifica: por qué TreeSHAP y no KernelSHAP, por qué top-5 y no top-20, por qué pre-computar global.

Criterio de aceptación: curl /explain -d '{"features": [...]}' devuelve top-5 features con SHAP, latencia <100 ms p99, y la UI muestra el waterfall correctamente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
KernelExplainer tarda 30 s por instancia	Esperado — es $O(N \text{ samples} \times N \text{ features})$. F
SHAP values no suman a la predicción	Probablemente comparás shap_values + base_
PDP miente: la curva sugiere feature j no	feature j está correlada con otra feature;
Explicaciones distintas en cada corrida	LIME usa sampling random. Fix: fijar seed;
Top features no coinciden entre SHAP y LIM	Es esperado — son métodos distintos con as
Bar chart con SHAP global muestra "MedInc"	Comunicación. Fix: traducir nombres ("MedI

Preguntas frecuentes

¿SHAP, LIME, Permutation Importance — cuál uso?

- Modelo árbol (XGBoost, LightGBM, RF): TreeSHAP sin pensar.
- Modelo agnóstico, dataset grande: Permutation Importance (sklearn) para global; LIME para local si SHAP es muy lento.
- Red neuronal: DeepSHAP o Integrated Gradients (Captum para PyTorch).
- Modelo lineal: coeficientes $\times$ valor de feature.

¿Por qué SHAP es "el estándar"?

Por las propiedades teóricas (axiomas de Shapley: efficiency, symmetry, dummy, additivity) y porque tiene implementaciones rápidas (TreeSHAP) para modelos populares. LIME es más viejo y más heurístico.

¿On-demand /explain o pre-computado?

- Pocas inferencias/día: on-demand está bien.
- Muchas inferencias, mismo input recurrente: cachear.
- Análisis batch (auditoría mensual): pre-computar en bulk con TreeSHAP — sklearn pipelines + Spark/Dask.

¿Cómo manejo SHAP para LLMs?

shap.Explainer(model) con Partition masker funciona para text. Para LLMs grandes: caro y lento. Alternativas:

attention attribution, gradient-based (Captum), o prompt-level ("¿qué part del prompt fue importante?"). Área activa de research.

¿Y si el regulador me pide explicabilidad pero el modelo es Black Box?

Tres opciones: (1) Cambiá a un modelo intrínsecamente interpretable (Logistic Regression, GAM, EBM de interpret); (2) Generá SHAP por cada decisión + archivá; (3) Para fairness/compliance, también probá aequitas o fairlearn (Clase 161).

¿PDP o ALE?

PDP es más popular y simple, pero mentira con features correladas. ALE corrige eso, ligeramente más complejo. Si tus features son correladas (lo usual): ALE. Si independientes (raro): PDP alcanza.

Referencias

- Molnar, C. Interpretable Machine Learning (2ª ed., libro online gratis): <https://christophm.github.io/interpretable-ml-book/>
- Lundberg, S. & Lee, S. A Unified Approach to Interpreting Model Predictions (NIPS 2017) — SHAP.
- Ribeiro, M. et al. "Why Should I Trust You?": Explaining the Predictions of Any Classifier (KDD 2016) — LIME.
- SHAP docs — TreeExplainer, KernelExplainer, plots.
- InterpretML / EBM — modelos intrínsecamente interpretables (Microsoft Research).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 206 — Clase 206 — Testing de datos: Great Expectations, Deequ

Parte: 4 — MLOps · Fuente: Huyen cap. 4 + Great Expectations docs (1.x) + Deequ paper. Duración estimada: 75 min.

Objetivo

Aplicar testing como código a los datos: definir "expectations" (assertions sobre el dataset) y validarlas en cada corrida del pipeline. Detectar schema drift, outliers extremos, nulos inesperados antes de que lleguen al entrenamiento o a la predicción. Bug típico: no es que el modelo se rompa — es que la data está rota y nadie se entera.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Definir un Expectation Suite con Great Expectations 1.x: schema, ranges, uniqueness, null rates, regex.
- Generar un Data Docs HTML que documenta el dataset + resultados de validación (auditoría para regulador).
- Integrar GE en un pipeline DVC/Airflow: si validación falla, abortar el pipeline.
- Usar Deequ (Scala/Python via PyDeequ) para datasets grandes en Spark.
- Diferenciar data tests (sobre el dataset) de unit tests (sobre el código) — son ortogonales.

Temas

#	Tema	Por qué importa
---	------	-----------------

1	Por qué unit tests no alcanzan en data pip	El código está OK; la data llega rota.
2	Expectation Suite: schema + business rules	Catalogar lo que es "data correcta" en est
3	Profiling automático	GE puede inferir un suite inicial del data
4	Data Docs: docs ejecutables	Auditoría + onboarding sin esfuerzo extra.
5	Checkpoints e integración con pipelines	El gate que aborta el flow si la data está
6	Deequ para Spark scale	Cuando el dataset no entra en pandas.

Definiciones y características

- Expectation: assertion sobre los datos. Ej: `expect_column_values_to_not_be_null('user_id')`, `expect_column_values_to_be_between('age', 0, 120)`, `expect_column_distinct_values_to_be_in_set('country', ['AR', 'BR', 'CL'])`.
- Expectation Suite: colección de expectations versionada (YAML/JSON).
- Validation: ejecutar el suite contra un batch real → result con success: true/false por expectation.
- Checkpoint: configuración que une data + suite + acción (postear a Slack, abortar pipeline, generar docs).
- Data Docs: HTML auto-generado que muestra: schema, ejemplos, resultados de validación, evolución.
- Profiling: GE puede generar un suite inicial inspeccionando el dataset (UserConfigurableProfiler). Punto de partida — siempre revisar manualmente.
- Deequ: librería de Amazon (Scala, también PyDeequ para Python) que hace lo mismo pero sobre Spark. Diseñada para TB.
- Pandera: alternativa Python-first, integrada con pandas/polars DataFrames con sintaxis decorator.

Dataset / recursos

- Dataset: California Housing (sin shift) como reference, después con shift sintético para ver fallas.
- Librerías: great-expectations>=1.0, opcional pandera, pydeequ.

Ejercicios

1. Bootstrap de un suite: `gx init` para crear el proyecto. `gx datasource new` apuntando a CSV. Profilea el dataset para suite inicial. Revisalo a mano: descomentá las expectations razonables, borra las absurdas.
2. Custom expectations: agregá: `expect_column_values_to_be_between('MedInc', 0, 20)`, `expect_table_row_count_to_be_between(1000, 100000)`, `expect_column_pair_values_A_to_be_greater_than_B('AveRooms', 'AveBedrms')` (más rooms que bedrooms).
3. Checkpoint: configurá un checkpoint que (a) corre el suite, (b) si falla, abre un Slack alert. Corré con `gx checkpoint run my_checkpoint`.
4. Data Docs: `gx docs build`. Abrió el HTML. Mostrá a un PM/regulador hipotético: la suite + el último validation result.
5. Pandera alternative: el mismo schema con pandera: `class HousingSchema(pa.DataFrameModel):`
`MedInc: Series[float] = pa.Field(in_range={"min_value": 0, "max_value": 20})`. Compará verbosidad y velocidad.

Homework verifiable

Pipeline ML con:

1. Expectation Suite Great Expectations versionada en git (YAML/JSON).
2. Checkpoint que se corre como step del pipeline DVC o Airflow (Clase 203) antes del training.
3. Data Docs HTML buildeado en CI y publicado a GitHub Pages.

4. Slack alert (con webhook stub si no tenés real) cuando una expectation falla.
5. Demostración de un dataset alterado (NaN inyectados, outliers, schema cambiado) que el pipeline detecta y aborta, sin que el modelo se entrene con data sucia.

Criterio de aceptación: el pipeline corre clean con data buena. Inyectando NaN en MedInc: el checkpoint falla, el pipeline aborta, el Slack alerta dispara, el modelo NO se re-entrena con data corrupta.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
gx init no encuentra Python venv correcto	GE 1.x quiere su propio gx CLI. Fix: activ
expect_column_values_to_be_in_set falla po	Tu suite quedó vieja. Fix: actualizar el s
Validation pasa pero el modelo igual rinde	Validación de forma ≠ validación de distri
Profiler genera 200 expectations, todas se	El profiler es muy permisivo. Fix: empezar
El Data Docs no muestra el último validati	data_docs no se rebuildió. Fix: agregar up
Performance lento en 10M filas	GE sobre pandas escanea todo. Fix: para vo

Preguntas frecuentes

¿Great Expectations, Pandera o Deequ?

- GE: más completo (Data Docs, suite versioning, plugin ecosystem), pero verboso.
- Pandera: Python-first, sintaxis decorator, mejor DX para devs. No tiene Data Docs.
- Deequ / PyDeequ: para Spark, datasets grandes.

Para equipos chicos en pandas: Pandera. Para empresas con auditoría: GE. Para Spark: Deequ.

¿Cuándo data testing vs schema validation (Pydantic)?

- Pydantic / dataclasses: validación por fila / record (en API requests). Rápido, in-process.
- GE / Pandera: validación por dataset (batch). Tests sobre agregados (medias, conteos), no solo schema.

Son complementarios: API valida cada request con Pydantic; pipeline batch valida cada slice con GE.

¿Cómo manejo expectations que cambian con el tiempo (estacionalidad)?

- (1) Expectations relativas: "row count entre last_week × 0.8 y last_week × 1.2".
- (2) Múltiples suites por época.
- (3) Marcar la expectation como warning (no aborta) y revisar manualmente.

¿GE rompe mi pipeline cada deploy?

Sí, si tu suite es muy estricto y la data cambia legítimamente. Fix: separar expectations críticas (abortan: PII no debe ser null, schema no debe cambiar) de alertas (warning, no abortan: distribución shift).

¿Versionar el Expectation Suite en git?

Sí siempre. Cambios al suite van por PR como cambios de código — review, CI, merge. El suite es contrato de datos = parte del contrato del producto.

¿Y datos de producción real-time (streaming)?

GE no es ideal para streaming. Alternativas: Apache Griffin, validations custom en el consumer Kafka, o aceptar que el testing es batch (validar cada 1 h sobre el último window).

Referencias

- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 4 — Training Data.
- Great Expectations docs — 1.x es la versión actual (rompió compat con 0.x).
- Pandera docs — schema validation for pandas/polars.
- PyDeequ — Spark-scale data testing.
- Schelter et al. Automating large-scale data quality verification (VLDB 2018) — paper de Deequ.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 207 — Clase 207 — Testing de modelos: invariance + behavioral tests

Parte: 4 — MLOps · Fuente: Ribeiro et al., Beyond Accuracy: Behavioral Testing of NLP Models with CheckList (ACL 2020, best paper) + Huyen cap. 9 + Deepchecks docs. Duración estimada: 80 min.

Objetivo

Ir más allá de "accuracy en hold-out": tests que verifican que el modelo se comporta como debería en casos específicos. Tres familias: invariance tests ("misma predicción si reemplazo Juan por María"), directional tests ("si subo el ingreso, la proba de aprobar el préstamo no debe bajar"), minimum functionality tests ("predicción correcta sobre casos canónicos hand-crafted").

Cierra la Parte 4: con esto, el modelo en producción tiene 6 capas de protección (data tests, model tests, monitoring, shadow, canary, rollback).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Implementar invariance tests (perturbaciones que NO deben cambiar la predicción): swap de nombres protegidos, sinónimos, ruido pequeño.
- Implementar directional tests (perturbaciones que deben cambiar la predicción en una dirección esperada): subir ingreso → menos riesgo crediticio.
- Crear minimum functionality test sets (MFT): casos hand-crafted que cubren cada feature/segmento crítico.
- Integrar tests de modelo en pytest y correrlos en CI antes de promover (gate de Clase 197).
- Usar Deepchecks o CheckList para suites pre-armados (especialmente NLP).

Temas

#	Tema	Por qué importa
1	Accuracy ≠ corrección — los 3 tipos de tes	Modelo "bueno" puede tener bugs sistemáticos
2	Invariance tests	Detecta sesgos (mismo CV con nombre cambiado)
3	Directional tests	Detecta inconsistencias (más experiencia →
4	MFT (Minimum Functionality)	Casos triviales que un humano resuelve sin
5	Slice-based testing	Performance por subgrupo, no solo overall.
6	Integración con CI	Tests verdes ≠ gate, requerir explícitamente

Definiciones y características

- Invariance test (INV): `assert model(x) == model(perturb(x))`. La perturbación es una transformación que

no debería cambiar la predicción. Ej: cambiar nombre, agregar puntuación, sinónimos.

- Directional Expectation test (DIR): `assert sign(model(x_modified) - model(x)) == expected_direction`. Ej: si aumento income, `P(default)` debe bajar o quedar igual — nunca subir.
- Minimum Functionality Test (MFT): un set chico de ejemplos hand-crafted etiquetados manualmente. El modelo DEBE acertar todos. Ej: "Pésimo servicio" → negativo en un sentiment classifier.
- Slice-based testing: medir métrica por sub-grupo (gender, age, country) y reportar el peor caso, no el promedio. El promedio esconde regresiones en minorías.
- CheckList (Ribeiro et al.): framework + paper que estandarizó los 3 tipos de test (INV/DIR/MFT) para NLP. Acabó ganando best paper en ACL 2020.
- Deepchecks: librería Python con suites pre-armados para tabular, vision, NLP. Detecta data leakage, train-test drift, performance bias.
- Adversarial testing: variante de invariance — perturbaciones pequeñas adversarialmente diseñadas que rompen el modelo. cleverhans, foolbox para deep learning.

Dataset / recursos

- Tabular: Adult Income (UCI) con gender como atributo sensible.
- NLP: subset de IMDb reviews para sentiment.
- Librerías: deepchecks $\geq$ 0.18, checklist, pytest, pandas.

Ejercicios

1. MFT tabular: para un modelo de crédito sobre Adult, hand-craft 20 casos: 10 obvios "should approve" (alto ingreso, educación alta, sin debts) y 10 obvios "should reject" (ingreso bajo, edad joven, sin historial). Asseré con pytest que el modelo acierta los 20.
2. Invariance — gender swap: tomá 500 registros, cambiá gender $M \leftrightarrow F$, dejá el resto igual. Mediá `accuracy(predictions_original == predictions_swapped)`. Si $<99\%$: el modelo está usando gender (probablemente vía proxy).
3. Directional — income up: para los mismos 500, multiplicá `income` $\times 1.5$. La proba predicha de $>50K$ debería subir (o quedar igual) en $>95\%$ de los casos. Si baja en muchos: bug.
4. Slice-based: calculá `accuracy` por slice (gender $\times$ race $\times$ age_bucket). Reportá top 5 worst slices. Si el peor slice tiene `accuracy` 0.55 y el promedio 0.85: tenés un problema de fairness invisible en métricas agregadas.
5. Pytest gate: empaquetá los 4 tests anteriores en `tests/test_model_behavior.py`. Hacelo correr en GitHub Actions (Clase 197) como required check. Si tests rojos: PR no se mergea.

Homework verificable

Repo con:

1. `tests/test_model_behavior.py` con ≥ 4 tests: 1 MFT, 1 INV, 1 DIR, 1 slice-based.
2. CI workflow que corre `pytest tests/test_model_behavior.py` después de cada training.
3. deepchecks full suite ejecutado, reporte HTML guardado.
4. Reporte por slice (markdown) con métricas en gender, race, age_bucket y bottom-5 worst slices identificados.
5. Documentación de 3 bugs reales encontrados por estos tests durante el desarrollo (puede ser bug fixed; lo importante es demostrar que los tests sirven).

Criterio de aceptación: si en una iteración futuro alguien cambia el modelo y un test rompe (ej. gender swap `accuracy` cae de 0.99 a 0.85), el CI falla y el PR no se mergea sin discusión explícita.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Gender swap test "falla" pero el modelo no	Está usando un proxy (ocupación, código po
MFT pasa pero el modelo rinde mal en produ	El MFT está sesgado al subset que vos imag
Slice-based: el worst slice tiene n=12 y m	Sample size insuficiente para sacar conclu
Tests behavioral pasan todos siempre	Probablemente las perturbaciones son trivi
Directional test falla "raro"	Quizás la relación NO es monótona — ej: in
CI lento por tests	Si pytest tarda 20 min, nadie los corre lo

Preguntas frecuentes

¿Esto reemplaza al test set normal?

No, lo complementa. Hold-out test set mide performance estadística sobre distribución similar a training. Behavioral tests miden correctness sobre casos importantes hand-crafted. Necesitás ambos.

¿Cuántos behavioral tests son suficientes?

CheckList paper sugiere 1 MFT + 1 INV + 1 DIR por capability del modelo. En tabular: por cada feature crítica. En NLP: por cada fenómeno lingüístico. Empezá con 10-20 tests bien diseñados, no con 1000 mediocres.

¿Esto es lo mismo que fairness testing?

Hay overlap. Fairness suele ser un subset de invariance tests (perturbar atributos protegidos) + slice-based (medir disparidad). Pero behavioral tests también cubren directional/MFT que no son sobre fairness directamente.

¿Deepchecks o escribir tests propios?

Deepchecks ofrece checks pre-armados (data leakage, integrity, performance bias) que cubren casos genéricos. Para tu dominio específico (correctness de negocio), escribís tests custom. Usar ambos.

¿Cómo manejo tests probabilísticos (no determinísticos)?

(1) Fijar seeds. (2) Para tests aproximados: `assert metric > 0.99` (no `== 1.0`). (3) Bootstrap CI para `asseré lower_bound > threshold`.

¿Y modelos LLM?

Aplican igual los 3 tipos:

- MFT: prompts canónicos con respuestas esperadas exactas/reguladas.
- INV: paráfrasis del mismo prompt no debería cambiar el output dramáticamente.
- DIR: agregar "be brief" al prompt debería acortar output.

Frameworks: promptfoo, LangSmith, Phoenix (Arize).

Referencias

- Ribeiro, M. et al. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList (ACL 2020) — el paper que estableció el vocabulario.
- Huyen, Chip. Designing Machine Learning Systems (O'Reilly, 2022), cap. 9 — Continual Learning and Test in Production.
- Deepchecks docs — suites para tabular/vision/NLP.
- CheckList GitHub — framework de Ribeiro.
- promptfoo / LangSmith — equivalente para LLMs.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Parte 5 — Parte 5 — Ingeniería de Datos

8 clases en esta parte.

Clase 208 — Clase 208 — Pipelines ETL/ELT con Airflow

Parte: 5 — Ingeniería de Datos · Fuente: Reis & Housley *Fundamentals of Data Engineering* (O'Reilly, 2022) cap. 8 + Airflow docs 2.10+. Duración estimada: 80 min.

Objetivo

Escribir DAGs de Airflow 2.x con la API moderna (TaskFlow + @dag/@task decorators), entender la diferencia entre ETL (transform antes de cargar) y ELT (cargar al warehouse y transformar ahí), y orquestar un pipeline extract → load → transform → notify con retries, SLAs y backfill.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un DAG con TaskFlow API (@dag, @task) y entender el grafo resultante.
- Diferenciar ETL (clásico) de ELT (moderno, warehouse-first) y elegir según contexto.
- Configurar retries, SLAs, trigger rules (all_success, one_failed, none_failed), y XComs.
- Hacer backfill (airflow dags backfill) para reprocesar fechas históricas sin duplicar.
- Diagnosticar Task stuck in queued, Worker died, DAG not appearing con airflow dags list, logs, y airflow.cfg.

Temas

#	Tema	Por qué importa
1	ETL vs ELT — cuándo cada uno	El warehouse moderno cambió el default.
2	DAG = grafo dirigido acíclico	Vocabulario fundacional.
3	TaskFlow API vs Operators clásicos	Código más limpio en Airflow 2.x.
4	XComs — pasar data entre tasks	Su uso correcto y sus límites (no para GBs)
5	Schedule + catchup + backfill	Reprocesar histórico sin duplicar.
6	Sensors, hooks, providers	Esperar eventos externos, conectar a siste

Definiciones y características

- ETL (Extract-Transform-Load): transformás en Python/Spark antes de cargar al warehouse. Patrón clásico cuando el warehouse era caro.
- ELT (Extract-Load-Transform): cargás raw al warehouse y transformás con SQL ahí. Patrón moderno (BigQuery/Snowflake/DuckDB son baratos para compute SQL).
- DAG: grafo de tareas con dependencias. En Airflow se define con @dag decorator + @task per tarea.
- TaskFlow API: introducida en Airflow 2.0. Devuelve valores de @task que se vuelven inputs de otras

@task — Airflow iniere XComs automático.

- XCom (Cross-Communication): mecanismo para pasar pequeños valores entre tasks. Default backend: metadata DB. No usar para GBs — pasá referencias (paths S3) en su lugar.
- Catchup: si un DAG con schedule='@daily' se prende un lunes, ¿corre todos los días desde su start_date? Si catchup=True: sí (reprocesa retroactivo). Si False: arranca desde hoy. Default True — fuente de sorpresas caras.
- Backfill: reprocesar fechas específicas. `airflow dags backfill --start-date 2026-06-01 --end-date 2026-06-15 my_dag`.
- Sensor: tarea que espera un evento externo (archivo en S3, fila en DB) con polling. reschedule mode evita ocupar worker slot mientras espera.
- Hook: wrapper sobre un cliente externo (S3Hook, PostgresHook). Lee credenciales desde Connections UI.
- Provider: paquete con operators/hooks/sensors para una integración (apache-airflow-providers-amazon, apache-airflow-providers-google).

Dataset / recursos

- Stack ejemplo: Airflow 2.10+ con docker-compose oficial.
- Pipeline target: extrae CSV → carga a DuckDB → transforma con SQL → publica métricas a Slack.
- Librerías: `apache-airflow>=2.10`, `duckdb`, `pandas`.

Ejercicios

1. DAG mínimo TaskFlow: 3 tasks: extract (descarga CSV), transform (limpia con pandas), load (escribe a DuckDB). Ver el grafo en /graph.
2. Schedule + catchup: `schedule='@daily'`, `start_date=days_ago(7)`, `catchup=True`. Verificá que Airflow crea 7 runs históricos. Cambiar a `catchup=False` y observar diferencia.
3. Retries + SLA: agregá `retries=3`, `retry_delay=timedelta(minutes=2)`, `sla=timedelta(minutes=10)` al transform. Simulá falla con `raise Exception("flaky")` y observá reintento.
4. XCom: extract devuelve un dict pequeño (filename + row count). transform lo recibe como argumento (TaskFlow autoinjecta). Verificá en la UI tab "XCom".
5. Backfill: `airflow dags backfill --start-date 2026-06-01 --end-date 2026-06-05 my_dag`. Confirmá que se ejecutan los 5 días sin duplicar (gracias a idempotencia con `execution_date` como key).

Homework verificable

Repo con docker-compose Airflow + DAG que:

1. Corre cada hora (`schedule='@hourly'`).
2. Extrae data de una API pública (ej. CoinGecko BTC price), la carga a DuckDB.
3. Transforma con SQL (`run_sql` task usando DuckDBHook).
4. Calcula métrica (avg price 24h) y la postea a Slack via `SlackWebhookOperator`.
5. SLA de 5 min en extract, `retries=3`.
6. README con docker-compose up, comandos kubectl, captura de la UI.

Criterio de aceptación: el DAG corre 24 veces seguidas (1 día) sin fallar, los datos en DuckDB son idempotentes (re-run para misma hora no duplica), Slack recibe el mensaje cada hora.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
DAG no aparece en la UI	Sintaxis error en el archivo. Fix: python
Catchup explota: 365 runs creados al arran	<code>catchup=True</code> (default) con <code>start_date</code> de h

Task stuck in queued forever	Worker no levantó, o pool agotado. Fix: ai
XCom serialization error con DataFrame	XCom default es JSON; un DataFrame no es J
start_date en el futuro hace que no corra	El scheduler ignora DAGs con start_date >
Variables de entorno no llegan al worker	airflow.cfg o Docker compose con env vars

Preguntas frecuentes

¿Airflow, Prefect, Dagster, Mage — cuál elijo en 2026?

- Airflow: estándar de la industria, max madurez, syntax verbosa.
- Prefect 3: API Python moderna, hybrid execution (Clase 209).
- Dagster: data-aware (assets), mejor lineage.
- Mage: notebook-first.

Para empresas grandes / equipos data engineering serios: Airflow. Para equipos chicos / ML-focused: Prefect o Dagster son más rápidos.

¿ETL o ELT?

ELT cuando: warehouse moderno (BigQuery/Snowflake), data analyst con SQL, transformaciones cambian seguido. ETL cuando: data muy sensible (no podés cargar raw al warehouse), transformaciones complejas (geo, ML inference), warehouse caro. La industria converge a ELT con dbt para SQL transforms post-load.

¿XCom para pasar un DataFrame de 500 MB?

No. XCom default está en la metadata DB (Postgres/MySQL) — vas a saturarla. Pasá un path S3 / GCS por XCom, y cada task lee/escribe del storage. O configurá Custom XCom Backend con S3.

¿Tengo que aprender los Operators clásicos (PythonOperator, BashOperator)?

Para mantener DAGs viejos: sí. Para nuevos: TaskFlow API es mejor. Pero algunos casos (operators provider-specific como BigQueryInsertJobOperator) siguen siendo más limpios sin TaskFlow.

¿Cómo testeo un DAG?

Tres niveles: (1) import test: python my_dag.py no rompe. (2) unit test de tasks: extraer la lógica a funciones puras + tests. (3) integration: airflow dags test my_dag 2026-06-01 corre el DAG sin scheduler.

¿Airflow corre el Python de mi DAG en cada heartbeat?

Sí — el scheduler parsea todos los DAGs cada min_file_process_interval segundos. No pongas código pesado al top level (requests.get(...) en import time es un anti-pattern). Todo lo costoso va dentro de tasks.

Referencias

- Reis, J. & Housley, M. Fundamentals of Data Engineering (O'Reilly, 2022), cap. 8 — Queries, Modeling, and Transformation.
- Airflow docs 2.x — TaskFlow API.
- Awesome Apache Airflow.
- dbt-core — el complemento natural para ELT.
- Designing Data-Intensive Applications (Kleppmann, 2017) — fundamentos.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 209 — Clase 209 — Pipelines con Prefect o Dagster

Parte: 5 — Ingeniería de Datos · Fuente: Prefect 3 docs + Dagster docs + Reis & Housley cap. 8.
Duración estimada: 75 min.

Objetivo

Construir el mismo pipeline de Clase 208 con Prefect 3 (API Python moderna, hybrid execution) y con Dagster (asset-oriented, mejor lineage). Entender qué problemas resuelven mejor que Airflow y cuándo elegir cada uno.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Escribir un flow Prefect con `@flow/@task`, deployments, work pools y workers.
- Definir assets Dagster (`@asset`) y entender la diferencia entre "task-oriented" (Airflow/Prefect) y "asset-oriented" (Dagster).
- Configurar hybrid execution Prefect: control plane en cloud, workers en tu infra (sin enviar data sensible).
- Usar Dagster's UI para ver lineage automático: qué asset depende de qué, cuándo se materializó cada uno.
- Decidir Airflow vs Prefect vs Dagster según contexto (equipo, escala, tipo de pipeline).

Temas

#	Tema	Por qué importa
1	Prefect 3: flows, tasks, deployments	Reemplaza DAGs con código Python idiomático
2	Work pools + workers	Hybrid execution: control en cloud, comput
3	Dagster: asset-oriented vs task-oriented	Lineage automático, mejor para data produc
4	Software-defined assets (SDA)	Cada asset es código + metadata + checks.
5	Scheduling: cron, interval, event-driven	Las 3 formas de disparar.
6	Cuándo migrar de Airflow	Costo de migración vs beneficio.

Definiciones y características

- Prefect Flow: equivalente al DAG. Decorador `@flow`. Puede invocar tasks (`@task`) y sub-flows.
- Prefect Task: unidad de trabajo. Tiene retries, cache, timeout configurables. Diferente a Airflow: las tasks pueden tener loops/condicionales sin XCom hackery.
- Deployment: la receta de "cómo y cuándo correr este flow" (schedule, parameters, infra). Prefect 3 los empaqueta como código (`flow.deploy(...)`).
- Work pool: cola lógica donde los deployments mandan runs. Los workers la consumen.
- Worker: proceso que corre los runs. Puede vivir en tu laptop, K8s, ECS, etc. Cloud variant: control plane managed; workers tuyos (hybrid).
- Dagster Asset: objeto persistente versionado (una tabla, un modelo, un dashboard). Definido con `@asset`. Las dependencias entre assets se infieren del código.
- Op (Dagster): equivalente más cercano a "task" — unidad atómica. Los assets generalmente componen ops.
- Materializar: ejecutar el código que produce el asset (re-genera el output). Dagster trackea cuándo se materializó cada asset por última vez.
- Sensor (Dagster/Prefect): trigger basado en eventos (nuevo archivo, mensaje en queue) en vez de schedule.

Dataset / recursos

- Mismo pipeline de Clase 208 (BTC price), implementado dos veces.
- Librerías: prefect>=3, dagster>=1.7, duckdb, pandas, requests.

Ejercicios

1. Prefect flow: copió la lógica del DAG Airflow al patrón Prefect: `@flow def btc_pipeline(): notify(transform(load(extract())))`. Corré python btc.py directo (no necesita scheduler).
2. Deployment Prefect: `flow.serve(name="btc-hourly", cron="0 * * * *")`. Dejá corriendo, observá ejecuciones programadas en localhost:4200.
3. Dagster assets: convertí las funciones a `@asset def btc_price()`, `@asset def daily_avg(btc_price)`. Dagster infiere `daily_avg` depende de `btc_price`. UI muestra el grafo.
4. Materializar: en Dagster UI, click "Materialize" sobre `btc_price`. Solo se ejecuta ese asset; `daily_avg` queda "stale" hasta que se materialice también.
5. Comparativa: mismo pipeline en Airflow + Prefect + Dagster. Compará LOC, claridad, UI, velocidad de feedback dev.

Homework verificable

Repo con el mismo pipeline implementado en los 3 frameworks:

1. airflow/dags/btc.py (de Clase 208).
2. prefect/btc.py con `@flow/@task` y deployment programado.
3. dagster/btc.py con `@asset` definitions y un Definitions object.
4. README comparativo: LOC, complejidad de setup, calidad de UI, lineage support, cuándo elegir cada uno.
5. Bonus: GitHub Actions que corre los 3 en CI y verifica que producen el mismo output.

Criterio de aceptación: los 3 pipelines producen idénticos resultados sobre el mismo input; el README compara honestamente fortalezas/debilidades.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
prefect server start falla	Otro proceso en :4200. Fix: <code>--port 4201</code> .
Deployment no se ejecuta automáticamente	Workers no están corriendo o pool mal conf
Dagster asset materialization "stale" pero	Auto-materialization no está habilitada. F
Re-import circular entre <code>@assets</code>	Dagster intenta resolver dependencias en i
Performance lento en Prefect con muchas ta	El backend default está en SQLite. Fix: pa
Mixed Airflow + Prefect + Dagster en el mi	Cada uno tiene su propio ecosistema. Fix:

Preguntas frecuentes

Airflow vs Prefect vs Dagster en una frase

- Airflow: el camión de carga industrial — feo pero llega.
- Prefect: el Tesla — UI moderna, código limpio, hybrid execution.
- Dagster: la BMW — lineage hermoso, ideal cuando pensás en data products.

¿Vale la pena migrar desde Airflow?

Calcular: (costo de migrar N DAGs) vs (ahorro mensual en mantenimiento + horas dev). Si Airflow funciona y nadie está sufriendo: no. Si nuevos pipelines: empezar con Prefect/Dagster, dejar los viejos donde están.

¿Prefect Cloud o self-hosted?

Cloud: control plane managed, free tier generoso, hybrid execution (workers tuyos). Self-hosted: prefect server start corre todo local — para dev. Para prod: Cloud es mucho más práctico.

¿Dagster's asset model es overkill para pipelines simples?

Para 3-5 tareas en cascada: sí, Prefect es más simple. Para data warehouse con 100+ tablas modeladas: Dagster brilla (lineage, freshness, partition awareness).

¿Cómo manejo secrets en Prefect/Dagster?

- Prefect: `Secret.load("my-key").get()` desde blocks (cifrados en backend).
- Dagster: `EnvVar("MY_SECRET")` o resources con `ConfigurableResource`.

Ambos integran con AWS Secrets Manager / GCP Secret Manager / Vault.

¿Puedo usar Dagster con dbt?

Sí — dagster-dbt carga modelos dbt como assets Dagster automático. Lineage atraviesa Python → dbt → SQL → tablas. Es el sweet spot del stack moderno.

Referencias

- Prefect 3 docs — empezar por Quickstart.
- Dagster docs — Concepts → Assets.
- Reis & Housley Fundamentals of Data Engineering (O'Reilly, 2022) cap. 8.
- Prefect vs Airflow (Prefect, 2024) — sesgado pero útil.
- dagster-dbt integration — el combo más usado.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 210 — Clase 210 — PySpark para datasets grandes

Parte: 5 — Ingeniería de Datos · Fuente: Chambers & Zaharia Spark: The Definitive Guide (O'Reilly, 2018) + PySpark docs 3.5+. Duración estimada: 90 min.

Objetivo

Procesar datasets que no entran en RAM con PySpark 3.5+: DataFrames con lazy evaluation, Spark SQL, particionado, joins eficientes (broadcast vs shuffle), y entender cuándo Spark gana vs pandas/Polars (>10 GB) y cuándo pierde (<1 GB, dev local).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Crear una `SparkSession` (local o cluster) y leer Parquet/CSV/JSON con schema inference o explícito.
- Diferenciar transformations (lazy: select, filter, groupBy) de actions (eager: count, collect, show, write).
- Optimizar joins: broadcast join (tabla chica × tabla grande) vs sort-merge join (dos tablas grandes).
- Particionar correctamente (`partitionBy("date")`) al escribir, evitar `partitionBy` con cardinalidad alta).
- Diagnosticar performance con Spark UI: stages, shuffle data, skew.

Temas

#	Tema	Por qué importa
1	RDD vs DataFrame vs SQL — 3 APIs	DataFrame es el default; SQL si tu equipo
2	Lazy evaluation + DAG de ejecución	Optimización que pandas no tiene.
3	Particionado: al leer, al escribir, en mem	Donde se gana o pierde 10× perf.
4	Joins: broadcast vs shuffle vs sort-merge	El bottleneck más común.
5	Caching / persist	Cuándo materializar; cuándo NO.
6	Spark UI: stages, shuffle, skew	Diagnosis sin esto = ciego.

Definiciones y características

- SparkSession: entrypoint a Spark. `SparkSession.builder.appName("x").getOrCreate()`.
- DataFrame: tabla distribuida (RDD de Rows con schema). API similar a pandas pero lazy.
- Transformation: operación que devuelve otro DataFrame sin ejecutar (`df.filter(...)`, `df.select(...)`). Construye el DAG.
- Action: operación que dispara ejecución (`df.count()`, `df.show()`, `df.write(...)`). Spark optimiza el DAG entero y luego ejecuta.
- Partition: chunk del DataFrame que vive en un task. Por default ~200 (config `spark.sql.shuffle.partitions`). Demasiadas → overhead; pocas → no paraleliza.
- Broadcast join: si una tabla es chica (<10 MB default), Spark la copia a todos los executors → join sin shuffle. Usar `broadcast()` hint para forzar.
- Sort-merge join: dos tablas grandes → shuffle ambas por la key + sort + merge. Costoso pero general.
- Skew: cuando una key tiene 10× más filas que el promedio → un task tarda 10×. Mitigación: salting, AQE (Adaptive Query Execution).
- AQE (Adaptive Query Execution, Spark 3+): runtime optimization — re-particiona, coalesce shuffle, handle skew. Habilitado por default desde 3.2.
- Catalyst: optimizer de queries SQL/DataFrame. Reescribe el plan lógico (predicate pushdown, column pruning) antes de ejecutar.

Dataset / recursos

- Local mode: `pyspark.SparkSession.builder.master("local[*]")` — usa todos los cores.
- Dataset: NYC TLC Yellow Taxi 2024 (parquet, ~10 GB) — clásico para Spark demos.
- Librerías: `pyspark>=3.5`, `pyarrow`.

Ejercicios

1. Spark session local: `spark = SparkSession.builder.master("local[4]").appName("demo").getOrCreate()`. Cargá un parquet, mostrá schema con `df.printSchema()`.
2. Lazy vs eager: `df2 = df.filter(...).select(...)` (rápido, no ejecuta). `df2.count()` (lento, ejecuta). Mirá en Spark UI (localhost:4040) los stages.
3. Broadcast join: cargá taxi (10 GB) y zones (1 KB). Hacé `taxi.join(broadcast(zones), "zone_id")`. Compará con `taxi.join(zones, "zone_id")` sin hint — debería ser igual gracias a AQE auto-broadcast.
4. Particionado al escribir: `df.write.partitionBy("date").parquet("out/")`. Verificá estructura `out/date=2024-01-01/part-*.parquet`. Lecturas con filtro `WHERE date='2024-01-01'` solo leen ese subdirectorio.
5. Skew: simulá una key skewed (90% rows con `user_id=1`). Hacé `groupBy` → observá UI: 1 task tarda 90% del tiempo. Mitigá con salting: agregar columna `random salt = (rand() * 10).cast("int")`, group por (`user_id`, `salt`), después sumar.

Homework verificable

Notebook + reporte:

1. Pipeline PySpark que: lee NYC Taxi (parquet), filtra outliers, agrega por pickup_zone y hour, escribe parquet particionado por pickup_date.
2. Comparación de performance: misma agregación en (a) pandas (si entra), (b) Polars, (c) PySpark. Reportar tiempo y RAM peak.
3. Identificar 1 stage skewed en Spark UI y aplicar salting para mitigar — mostrar antes/después.
4. Output final con `.write.bucketBy(20, "zone_id").parquet(...)` para acelerar joins futuros.
5. README con cuándo elegir cada uno: criterios objetivos (tamaño, latencia, equipo).

Criterio de aceptación: el alumno justifica con números por qué Spark gana en su dataset y muestra una optimización (broadcast/salting/partitioning) con impacto medido.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
OutOfMemoryError: Java heap space	Executor sin RAM. Fix: <code>--driver-memory 4g</code>
<code>df.show()</code> tarda 5 minutos	Estás computando todo el DataFrame para mo
Job tarda 30 min en una agregación de 1 GB	Sospechosos: shuffle excesivo, default par
Caused by: BindException: Address already	Spark UI en :4040 conflicto. Fix: <code>.config(</code>
<code>to_pandas()</code> falla con OOM	Estás trayendo TODO el DataFrame al driver
Resultados distintos entre runs	UDFs no-determinísticos, o monotonically_i
<code>partitionBy("user_id")</code> con 1M users crea 1	High-cardinality partitioning es anti-patt

Preguntas frecuentes

¿PySpark, pandas, Polars o DuckDB?

Decisión por tamaño + uso:

- <1 GB local: pandas o Polars (Polars 5-10× más rápido).
- 1-50 GB single machine: Polars o DuckDB.
- >50 GB single machine: Polars streaming, DuckDB out-of-core, o PySpark local.
- >500 GB y/o cluster: PySpark.

¿PySpark en local o necesito cluster?

`master("local[*]")` corre Spark en tu laptop usando todos los cores — perfecto para dev/test con datasets hasta unos GB. Para producción TB: Databricks, EMR, GCP Dataproc, Azure Synapse, o K8s con Spark Operator.

¿UDF Python vs Spark SQL functions?

SQL functions (`F.col`, `F.when`, `F.regexp_extract`) son mucho más rápidas — corren en JVM. UDFs Python serializan a Python por row (lento). Si no podés evitarlo: Pandas UDFs (vectorizadas, ~10× UDF normal).

¿Por qué `df.cache()` no aceleró?

Cache es lazy también. Tenés que disparar una action (`df.count()`) para materializarlo. Después las siguientes actions reutilizan el cache. Y: si tu dataset no entra en memoria, cache no ayuda — usá `df.persist(StorageLevel.DISK_ONLY)`.

¿Spark 3 vs 4 vs Databricks Runtime?

Spark 3.5 es el estándar open-source en 2026. Spark 4 trae Spark Connect (cliente liviano vs server JVM),

Variant type, mejoras en streaming. Databricks Runtime es Spark + extensiones propietarias (Photon engine 2-5× más rápido). Para empezar: Spark 3.5 open-source.

¿Cuándo usar Spark SQL vs DataFrame API?

Equivalentes en performance (mismo Catalyst). DataFrame API: refactor-friendly, type hints. SQL: mejor si tu equipo es SQL-first o querés migrar de un warehouse. Mezclables: `spark.sql("SELECT * FROM tbl").filter(...)`.

Referencias

- Chambers, B. & Zaharia, M. Spark: The Definitive Guide (O'Reilly, 2018) — algo viejo pero los conceptos siguen.
- PySpark docs 3.5+ — empezar por Quickstart: DataFrame.
- Adaptive Query Execution deep dive.
- Spark Performance Tuning (official).
- pyspark-stubs — type hints (incluidos en 3.4+).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 211 — Clase 211 — Polars como alternativa moderna

Parte: 5 — Ingeniería de Datos · Fuente: docs Polars + Polars vs pandas benchmark. Duración estimada: 75 min.

Esta clase es complementaria a la Clase 008 (Polars básico, Parte 0). Acá el foco es producción: lazy API, streaming engine, Arrow zero-copy, integración con DuckDB/Parquet.

Objetivo

Reemplazar pandas en pipelines productivos por Polars 1.x: 5-30× más rápido, multi-threaded por default, lazy API que optimiza la query antes de ejecutar, y streaming engine que procesa datasets mayores que RAM. Identificar los pocos casos donde pandas sigue ganando (ecosistema, statsmodels, sklearn pre-Arrow).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Convertir scripts pandas a Polars (eager API: `pl.DataFrame`) y medir speedup.
- Usar lazy API (`pl.scan_parquet + .collect()`) para que el optimizador haga predicate pushdown + column pruning.
- Procesar archivos mayores que RAM con `.collect(engine="streaming")` (1.x rename de `streaming=True`).
- Hacer zero-copy interop con Arrow/DuckDB/pandas.
- Decidir Polars vs DuckDB vs pandas vs PySpark según tamaño + caso de uso.

Temas

#	Tema	Por qué importa
1	Eager vs Lazy API	El optimizador de queries es lo que hace g
2	Expressions: paralelización implícita	<code>pl.col('x').mean()</code> corre en todos los core
3	<code>scan_parquet/scan_csv</code> + predicate pushdown	Lee solo lo necesario del disco.
4	Streaming engine para datasets > RAM	Out-of-core sin Spark.

5	Arrow interop con DuckDB/pandas	Zero-copy → cero overhead.
6	when().then().otherwise() y over()	Window functions sin SQL.

Definiciones y características

- Polars: DataFrame library escrita en Rust, con Apache Arrow como memory model. Diseñada para columnar + paralelo + lazy.
- Eager API: `pl.DataFrame(...)`. Similar a pandas — ejecuta cada operación.
- Lazy API: `pl.scan_parquet("x.parquet").filter(...).select(...).collect()`. Construye un plan; lo optimiza; ejecuta una vez. El default que querés en producción.
- Expression: una operación columnar (`pl.col('x') * 2 + pl.col('y')`). Reutilizable, componible, paralelizable.
- Streaming engine (1.x): ejecuta lazy queries en chunks, permitiendo procesar archivos mayores que RAM. `.collect(engine="streaming")`. Apto para la mayoría de las queries, no para todas (joins arbitrarios pueden no soportarse — chequear plan).
- Predicate pushdown: el optimizer empuja los WHERE lo más temprano posible. Si filtrás por `date='2024-01-15'` después de un join, Polars filtra antes y reduce data.
- Column pruning: si solo select-ás 3 columnas, Polars no lee las otras del Parquet. pandas no hace esto.
- Arrow zero-copy: `pl.from_pandas(df, rechunk=False)` o `df.to_arrow()` → no se copia memoria, solo se cambia el "label" del buffer.

Dataset / recursos

- Mismo NYC Yellow Taxi de Clase 210 (parquet, ~150 MB/mes, ~10 GB/año).
- Librerías: `polars>=1.5`, `pyarrow`, opcional `duckdb` para integración.

Ejercicios

1. Eager vs Lazy benchmark: misma agregación con `pl.read_parquet(...)` (eager) y `pl.scan_parquet(...).collect()` (lazy). Compará tiempos. La diferencia es chica con dataset chico; aumenta dramáticamente con datasets >GB.
2. Pandas → Polars: tomá un script pandas existente, traducí a Polars. Medí speedup. Casos comunes: `groupby().agg()` → `group_by().agg()`, `.apply` → expressions.
3. Streaming: con un parquet de 5 GB (descargar 12 meses NYC Taxi), correr una agregación con `.collect()` y luego con `.collect(engine="streaming")`. Comparar RAM peak (`memory_profiler`).
4. Predicate pushdown explícito: `pl.scan_parquet("data/").filter(pl.col("date") == "2024-01-15").select("fare").collect()` vs `pl.read_parquet("data/").filter(...).select(...)`. Mirá el plan con `.explain()`.
5. Polars + DuckDB: hacer la query principal en Polars; pasar el resultado a DuckDB con `con.from_arrow(df.to_arrow())` para hacer una consulta SQL compleja.

Homework verificable

1. Pipeline en Polars que: lee 12 meses NYC Taxi, filtra outliers, calcula avg fare por borough × hour, escribe parquet particionado.
2. Benchmark contra: pandas (si el dataset entra), DuckDB (`con.execute(query).pl()`), PySpark (Clase 210). Reportar tiempo + RAM peak.
3. Una query usando streaming engine que procesa >RAM (forzar bajando `psutil.virtual_memory().available / 4` con `docker-compose limit`).
4. Una query con window functions (`pl.col("fare").rolling_mean(window_size=7).over("borough")`) que en pandas requeriría 20 LOC.
5. README con cuándo Polars vs cuándo otra cosa.

Criterio de aceptación: el alumno justifica con números (tiempo + RAM) por qué Polars es el default para su

workload, y muestra al menos 1 caso donde DuckDB o pandas ganan.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Polars not faster than pandas en mi caso	Probablemente: (1) dataset muy chico (<100
to_pandas() falla con pyarrow	Conflicto versiones pyarrow. Fix: pip inst
Lazy query no ejecuta nada	Olvidaste .collect(). Fix: cualquier query
MemoryError con streaming	Algunos operadores no soportan streaming (
API diferente a pandas confunde	groupby → group_by, value_counts → .value_
Date parsing devuelve Object o String	Schema inference falló. Fix: pl.scan_csv("

Preguntas frecuentes

¿Pandas se queda? ¿Migrar todo a Polars?

Pandas tiene 15 años de ecosistema: sklearn, statsmodels, plotly aceptan DataFrames nativos. Polars 1.x se integra (vía Arrow) pero algunas libs requieren conversión. Para pipelines de ETL/feature engineering: migrar a Polars. Para modeling/análisis exploratorio: pandas sigue cómodo.

¿Polars vs DuckDB?

Solapan, son complementarios.

- Polars: DataFrame API, más natural si pensás en código Python. Pipelines de transformación.
- DuckDB: SQL-first, OLAP optimized, ideal para análisis ad-hoc, queries complejas, joins masivos.

Combinables: Polars para transformación, DuckDB para queries finales analíticas.

¿Lazy o eager por default?

En producción: lazy siempre. En notebooks exploratorios: eager está bien (es como pandas). El mantra: "si vas a hacer más de 2 ops sobre el DataFrame, usá lazy".

¿Streaming engine es estable?

Polars 1.0 (julio 2024) lo marcó estable. Algunos operadores aún no lo soportan; el plan con .explain() dice qué nodos van streaming y cuáles in-memory.

¿Cómo manejo NaN/null en Polars vs pandas?

Polars distingue null (missing) de NaN (float). pandas los mezcla (problema histórico). En Polars: pl.col('x').is_null() vs pl.col('x').is_nan(). Más limpio pero requiere ajuste mental.

¿Multi-process o multi-thread?

Polars usa multi-thread (Rust + Rayon) — sin GIL. Mejor que pandas + multiprocessing. Para escalar a multi-machine: PySpark o Dask.

Referencias

- Polars docs — empezar por Getting started.
- Polars vs pandas migration guide.
- DB benchmark (DuckDB Labs) — Polars vs DuckDB vs pandas vs Spark en 5/50/500 GB.
- Modern Polars (book): <https://kevinheavey.github.io/modern-polars/>
- hyperscan Arrow ecosystem — zero-copy interop.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 212 — Clase 212 — Data warehouses: BigQuery, Snowflake, DuckDB

Parte: 5 — Ingeniería de Datos · Fuente: Reis & Housley Fundamentals of Data Engineering (O'Reilly, 2022) cap. 6 + 9 + docs oficiales. Duración estimada: 80 min.

Objetivo

Consultar y operar 3 data warehouses modernos desde Python: BigQuery (GCP, serverless, separación compute/storage), Snowflake (multi-cloud, virtual warehouses, time travel), DuckDB (embedded, OLAP local, no requiere server). Decidir cuál usar según escala, presupuesto y latencia.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Conectar a BigQuery (google-cloud-bigquery), Snowflake (snowflake-connector-python), DuckDB (duckdb) y ejecutar queries.
- Diseñar tablas con particionado (BigQuery: PARTITION BY date_field) y clustering (BigQuery, Snowflake) para reducir costos y latencia.
- Usar COPY INTO (Snowflake) y LOAD DATA (BigQuery) para bulk ingest desde S3/GCS.
- Aprovechar time travel (SELECT ... AT(TIMESTAMP => ...) Snowflake) para auditar / recuperar.
- Decidir DW: BigQuery (serverless, GCP-first), Snowflake (multi-cloud, separation, sharing), DuckDB (local/embedded), Redshift (legacy AWS).

Temas

#	Tema	Por qué importa
1	Compute/storage separation	Por qué los DW modernos son baratos: solo
2	Particionado vs clustering	Reducir bytes leídos.
3	BigQuery: SQL standard + UDF + ML	El más simple para empezar.
4	Snowflake: virtual warehouses, time travel	Features únicos.
5	DuckDB: el DW que entra en pip install	Análisis local, ETL liviano, embed en app.
6	Costo: cómo NO gastar miles	SELECT * sin partition filter = bankruptcy

Definiciones y características

- Data warehouse: DB OLAP optimizada para queries analíticas (agregados sobre millones/billones de filas). Diferente de OLTP (Postgres, MySQL) optimizada para transacciones.
- Compute/storage separation: storage en blob (S3/GCS/Azure) — barato y elástico. Compute (virtual warehouse / slot) se enciende para queries. Pagás storage barato + compute por uso.
- Particionado: la tabla está dividida por una columna (típicamente date). Query con WHERE date='2024-01-15' lee solo esa partición → menos bytes → menos \$.
- Clustering (BigQuery) / Cluster keys (Snowflake): orden físico dentro de cada partición según N columnas. Mejora queries con WHERE col_clustered.
- Slot / Virtual Warehouse: unidad de compute que paralela tu query. BigQuery: slots compartidos o reservados. Snowflake: tamaño de VW (XS/S/M/L/XL...) en \$/hora.

- Time travel (Snowflake): consultar el estado pasado de una tabla (SELECT ... AT(TIMESTAMP => '2026-06-15 00:00:00')). Retención 1-90 días según tier.
- Data sharing (Snowflake): compartir tablas con otra cuenta Snowflake sin copiar data (live read).
- DuckDB: DB OLAP embedded (como SQLite pero columnar + vectorizada). No requiere server. Lee/escribe Parquet/CSV/Arrow nativo. Ideal para dev local + ETL liviano + análisis ad-hoc.

Dataset / recursos

- BigQuery: bigquery-public-data.new_york_taxi_trips.tlc_yellow_trips_2018 (GB-scale, gratis para query con free tier).
- DuckDB: local con parquets de NYC Taxi.
- Snowflake: trial 30 días con \$400 de crédito.
- Librerías: duckdb>=1.0, google-cloud-bigquery, snowflake-connector-python.

Ejercicios

1. DuckDB local: con = duckdb.connect("warehouse.duckdb"). Cargá un parquet con CREATE TABLE trips AS SELECT FROM 'trips.parquet'. Hacé SELECT COUNT(), AVG(fare) FROM trips. Compará tiempo vs Polars (Clase 211).
2. BigQuery query: from google.cloud import bigquery; client = bigquery.Client(project="..."). client.query("SELECT borough, COUNT(*) FROM bigquery-public-data.new_york_taxi_trips.tlc_yellow_trips_2018 GROUP BY borough"). Crítico: usar LIMIT en exploración, no escanear 100 GB sin querer.
3. Particionado en BQ: crear tabla mi_proyecto.dataset.trips_partitioned con PARTITION BY DATE(pickup_datetime). Query con WHERE DATE(pickup_datetime) = '2018-01-15' → mostrá "bytes processed" con/sin partition filter.
4. Snowflake time travel: CREATE TABLE x AS SELECT Insertá data. DELETE FROM x WHERE SELECT * FROM x AT(OFFSET => -60) (60s atrás) — los datos vuelven.
5. DuckDB queryando S3 directo: con.execute("SELECT FROM 's3://bucket/path/.parquet' LIMIT 10") sin descargar nada — DuckDB lee remoto con HTTPFS.

Homework verificable

Proyecto con:

1. Pipeline que extrae datos de una API → carga a un DW (elegir 1: BQ free tier, DuckDB, Snowflake trial) → corre 5 queries analíticas.
2. Comparativa de costo: misma query escaneando (a) tabla sin particionar (5 GB), (b) tabla particionada con filter (50 MB). Reportar \$ estimado.
3. Tabla con clustering / cluster keys sobre 2 columnas que mejoran una query frecuente. Mostrar mejora de performance.
4. Setup de service account con permisos mínimos (BQ Data Viewer, no Admin).
5. README explicando cuál DW elegirías para 3 escenarios: startup 5 dev, empresa multi-cloud, análisis personal local.

Criterio de aceptación: el alumno demuestra entender el modelo de costo (bytes scanned × \$/TB) y propone arquitectura razonable para cada escenario.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Factura BigQuery \$500 inesperados	Alguien hizo SELECT sin partition filter
Query Snowflake lenta — escalar el warehou	Posible, pero antes: revisar el plan (EXPL

DuckDB OOM en query agregada	Default memory limit (80% RAM). Fix: SET m
Permission denied BigQuery	El service account no tiene rol. Fix: en G
Snowflake Statement timeout	Tu VW es chico. Fix: subir tamaño temporal
JOIN lento entre dos tablas grandes	Falta cluster key compartida. Fix: cluster
Particionado por columna high-cardinality	BQ permite max 4000 particiones. Fix: part

Preguntas frecuentes

¿BQ vs Snowflake vs Redshift vs Databricks SQL?

- BigQuery: serverless puro, ideal en GCP, modelo "pay per query".
- Snowflake: separación compute/storage explícita, time travel, data sharing nativo. Multi-cloud.
- Redshift: el viejo (2012). AWS-native. RA3 instances son competitivos pero menos elegantes.
- Databricks SQL (sobre lakehouse): si ya tenés Spark en Databricks, integrado.

Para empezar: BigQuery (GCP) o Snowflake (multi-cloud).

¿DuckDB como warehouse de producción?

Para una sola máquina y team chico: sí (hasta cientos de GB). Para múltiples usuarios escribiendo concurrente y escala TB+: no — usá Snowflake/BQ. MotherDuck ofrece DuckDB managed cloud con escalado.

¿Cómo evito quemar \$500 sin querer en BQ?

(1) Project Settings → Quotas → Query usage limit per day. (2) Linter pre-commit que rechaza SELECT * sin partition. (3) Crear materialized views para queries repetidas — pagás 1 vez. (4) dry_run=True antes de ejecutar.

¿DELETE en un DW columnar es caro?

Sí. Las tablas columnares no están optimizadas para DELETE individual. Patrón: marcar como deleted_at (soft delete) o reescribir partition (INSERT OVERWRITE). Snowflake/BQ lo manejan razonablemente; Redshift sufre.

¿Cómo cargo 1 TB a un warehouse?

(1) Subí archivos a blob storage (S3/GCS). (2) Usá comando bulk del DW: LOAD DATA (BQ), COPY INTO (Snowflake), COPY (Redshift). NUNCA INSERT ... VALUES por row.

¿Costo storage es relevante?

En 2026: ~\$23/TB/mes en BQ/Snowflake. Para 10 TB = \$230/mes. Para 1 PB: \$23K/mes — empieza a importar. Para datasets enormes: lifecycle policies (mover a Glacier/Coldline después de N días).

Referencias

- Reis & Housley Fundamentals of Data Engineering (O'Reilly, 2022) cap. 6 (storage) y 9 (queries).
- BigQuery docs — empezar por Quickstarts.
- Snowflake docs — Quickstarts + Cost optimization.
- DuckDB docs + MotherDuck (DuckDB managed).
- Designing Data-Intensive Applications (Kleppmann, 2017) — fundamentos OLAP.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.

- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 213 — Clase 213 — Streaming intro: Kafka, Kinesis

Parte: 5 — Ingeniería de Datos · Fuente: Kreps, Building a Real-Time Data Pipeline + Narkhede, Shapira, Palino Kafka: The Definitive Guide (O'Reilly, 2ª ed., 2021) + Reis & Housley cap. 7. Duración estimada: 85 min.

Objetivo

Entender el modelo streaming vs batch (Clase 208), producir y consumir mensajes en Kafka (con confluent-kafka o kafka-python), comparar contra AWS Kinesis Data Streams (managed equivalent), y reconocer los 3 problemas clásicos de streaming: exactly-once, out-of-order events, backpressure.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar batch (datos llegan en bloques) de streaming (datos llegan continuos, baja latencia).
- Producir mensajes a un topic Kafka con keys (garantiza orden por key, distribuye load).
- Consumir con consumer groups: partitions distribuidas entre consumers, offset management.
- Diseñar para at-least-once (default razonable) y entender qué requiere exactly-once (transactional API).
- Decidir Kafka (self-hosted o Confluent Cloud) vs Kinesis (AWS) vs Pub/Sub (GCP) vs Event Hubs (Azure).

Temas

#	Tema	Por qué importa
1	Batch vs streaming — el espectro real	Spectrum: batch → micro-batch → streaming
2	Kafka model: topic, partition, offset	Vocabulario obligatorio.
3	Producer: keys, acks, idempotence	Garantías que querés desde el día 1.
4	Consumer groups + rebalancing	Cómo escalan los consumers.
5	Delivery semantics: at-most/at-least/exact	Trade-offs reales con código.
6	Kinesis comparison	Mismo modelo, vendor-specific.

Definiciones y características

- Topic: log durable, particionado, append-only. Análogo a una "tabla" en streaming.
- Partition: subset ordenado del topic. Cada partition tiene un único consumer dentro de un consumer group. Más partitions → más paralelismo.
- Offset: posición del consumer dentro de una partition. Persistido en Kafka (__consumer_offsets topic) o externamente.
- Producer: escribe mensajes a un topic. Puede especificar key (decide partition vía hash → mensajes con misma key van a la misma partition → orden garantizado por key).
- Consumer: lee mensajes de un topic en un consumer group. Si un consumer muere, otro toma sus partitions (rebalancing).
- Consumer group: lógicamente "un consumidor". 3 consumers en el mismo grupo se dividen las partitions; 3 grupos distintos cada uno ve todos los mensajes.
- At-most-once: mensaje se pierde si falla algo. acks=0.
- At-least-once: mensaje puede llegar duplicado. acks=all + enable_auto_commit=False + commit después de procesar. Default razonable.

- Exactly-once: requiere idempotent producer (enable.idempotence=true) + transactional API (Kafka Streams o Flink). Caro, complejo, no siempre necesario.
- Backpressure: cuando consumer no procesa tan rápido como producer escribe → lag crece. Acción: escalar consumers, o aplicar throttling al producer.
- Kinesis Data Stream: AWS-managed. "Shard" ≈ partition, "iterator" ≈ offset, KCL (Kinesis Client Library) ≈ consumer group. Mismo modelo conceptual, distintos nombres.

Dataset / recursos

- Kafka local: docker-compose con Kafka (KRaft mode, sin Zookeeper) + Kafka UI.
- Stream sintético: producer genera "click events" cada 100 ms.
- Librerías: confluent-kafka>=2.5 (más robusta) o kafka-python>=2.0 (más simple), faker para data sintética.

Ejercicios

1. Setup local: docker-compose con Kafka + Kafka UI. Crear topic clicks con 4 partitions. Verificar con docker exec kafka kafka-topics --list
2. Producer: script Python que produce 1000 mensajes con key=user_id, value={"page":"/foo","ts":...}. Verificar en Kafka UI que mensajes con mismo user_id caen en la misma partition.
3. Consumer 1 instancia: consumer.subscribe(['clicks']) + loop for msg in consumer. Procesar = print(msg.value()). Committear offset cada 100 mensajes.
4. Consumer group, 2 instancias: levantar 2 consumers con mismo group.id. Confirmar que se reparten las 4 partitions (2-2). Matar uno, observar rebalancing — el otro toma las 4.
5. At-least-once explícito: enable.auto.commit=False, procesar mensaje, consumer.commit(). Si crash entre procesar y commit → duplicate al reiniciar.

Homework verificable

Sistema con:

1. Producer Python que simula 100 eventos/s durante 1 min (sintéticos con faker).
2. 3 consumers en el mismo group consumiendo de un topic con 6 partitions.
3. Cada consumer procesa y escribe a una tabla DuckDB (idempotente: PK = (user_id, event_ts)).
4. Métrica de consumer lag monitoreada (chequear kafka-consumer-groups --describe).
5. Demostración: matar 1 consumer durante la corrida y verificar rebalancing + cero pérdida de mensajes.
6. README comparando con un equivalente en Kinesis (snippet de código sin necesidad de cuenta AWS).

Criterio de aceptación: 6000 mensajes producidos → 6000 mensajes en DuckDB (sin duplicados gracias a PK), consumer lag estabiliza <1000, rebalancing funcionó.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Topic does not exist aunque acabás de crea	Auto-create topics está deshabilitado y cr
Consumer lag crece infinito	Consumer no procesa rápido suficiente. Fix
Mensajes duplicados al reiniciar consumer	Falta commit del último offset procesado.
UnknownTopicOrPartitionError intermitente	Rebalancing en curso. Fix: catch y reinten
Performance horrible con enable.idempotenc	Idempotente requiere ordering por key, baj
Producer.flush() se cuelga	Network unreachable. Fix: agregar timeout,

Kafka UI no muestra topics

UI conectada a broker antes que esté ready

Preguntas frecuentes

¿Cuándo necesito streaming en vez de batch?

Cuando: (1) latencia importa (fraud detection: minutos), (2) volume es alto y constante (logs, telemetría), (3) data es continuamente generada (clickstream, IoT). Si tus datos llegan en cron diario y no urge: batch. La regla: batch primero, streaming cuando duela.

Kafka self-hosted o Confluent Cloud?

Self-hosted: control total, costo en EC2/operaciones. Confluent Cloud (managed): no operas brokers, pagas más. Para empezar: Confluent Cloud free tier. Para escala >100 MB/s sostenido: el TCO favorece self-hosted o Confluent Cloud Enterprise.

Kafka vs Kinesis vs Pub/Sub?

- Kafka: open-source, multi-cloud, ecosistema enorme (Kafka Connect, Kafka Streams, ksqlDB).
- Kinesis: AWS-native, integrado con Lambda/Firehose. Más limitado (shards manualmente escalados; Kinesis On-Demand mejora esto).
- Pub/Sub: GCP-native, simpler API (pull/push, no shards expuestos), gestionado al 100%.

Para multi-cloud o vendor-neutral: Kafka. Para AWS/GCP-native simple: Kinesis/Pub/Sub.

¿Cuándo necesito exactly-once?

Casi nunca. At-least-once + idempotent consumer (idempotency via PK en DB) resuelve el 95% de los casos. Exactly-once con transactional API tiene 20-30% overhead. Reservarlo para: pagos, billing.

¿Cómo manejo schemas evolucionando?

Schema Registry (Confluent, AWS Glue, Apicurio): producer registra schema Avro/Protobuf, consumer lo lee. Compatibilidad checks evita producer rompiendo consumers downstream.

¿Streaming SQL? ¿Flink, Spark Streaming, ksqlDB?

Para queries continuas (windowed aggregates, joins de streams):

- Kafka Streams: lib Java/Scala, embed en tu app.
- ksqlDB: SQL sobre Kafka topics.
- Apache Flink: streaming-first, stateful, exactly-once.
- Spark Structured Streaming: micro-batch (latencia ~seconds), reusa skills de Spark.

Para Python: PyFlink y Spark Structured Streaming son los más usados.

Referencias

- Narkhede, Shapira, Palino Kafka: The Definitive Guide (O'Reilly, 2ª ed., 2021).
- Kafka docs — Quickstart, Producer/Consumer configs.
- Confluent Cloud free tier.
- Reis & Housley Fundamentals of Data Engineering (O'Reilly, 2022) cap. 7.
- Awesome Kafka.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.

- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 214 — Clase 214 — Formatos columnares: Parquet, Avro

Parte: 5 — Ingeniería de Datos · Fuente: docs Parquet 2.x + Avro spec + Reis & Housley cap. 6.
Duración estimada: 70 min.

Objetivo

Elegir formato de almacenamiento según el patrón de lectura: Parquet (columnar, OLAP, queries analíticas), Avro (row-based, OLTP/streaming, schema evolution), ORC (columnar, Hive ecosystem). Entender por qué Parquet es 5-100× más rápido que CSV para queries analíticas, y por qué Avro domina en Kafka.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Convertir CSV → Parquet con `pandas.to_parquet` / `polars.write_parquet` / `pyarrow`.
- Diferenciar row-based (CSV, JSON, Avro) de columnar (Parquet, ORC) y elegir según query pattern.
- Aprovechar column pruning + predicate pushdown + row group pruning de Parquet (Polars/DuckDB/Spark lo hacen automático).
- Aplicar compresión (snappy default, zstd mejor ratio, gzip mejor compat) y entender trade-off CPU vs tamaño.
- Definir un schema Avro y usarlo en Kafka con Schema Registry (concept).

Temas

#	Tema	Por qué importa
1	Row vs columnar	OLTP vs OLAP en el storage.
2	Parquet anatomy: file > row group > column	Lo que permite predicate pushdown.
3	Compresión: snappy, zstd, gzip, lz4	Trade-off CPU vs tamaño.
4	Dictionary encoding, RLE	Por qué Parquet con strings repetidos pesa
5	Avro: schema-first, row-based, compact bin	Por qué domina en Kafka.
6	Schema evolution: forward/backward/full co	Cuándo se rompe; cómo manejarlo.

Definiciones y características

- Row-based (CSV, JSON, Avro): cada fila es un bloque contiguo. Eficiente para `SELECT *` de pocas filas; ineficiente para agregados sobre 1 columna de 1M filas.
- Columnar (Parquet, ORC): cada columna es un bloque contiguo. Eficiente para `SELECT col1, col2 FROM tbl WHERE col1 > X` — lee solo col1 y col2.
- Row group (Parquet): conjunto de filas (default ~128 MB). Stats (min/max/null_count) por columna por row group — habilitan predicate pushdown sin abrir el row group.
- Column chunk: la parte de una columna dentro de un row group. Físicamente contigua → vectorizable.
- Page: subdivisión de column chunk (~1 MB). Unidad de compresión + read.
- Predicate pushdown: si filtrás `WHERE col > 100` y el row group tiene `max(col)=50`, se skipa sin leer.
- Dictionary encoding: si una columna tiene $\leq 2^{16}$ valores distintos, Parquet guarda un diccionario + índices. Strings repetidos: huge win.
- RLE (Run-Length Encoding): si los valores se repiten (AAAA BBBB), guarda (A,4)(B,4). Bueno para columnas con poca varianza.
- Avro: formato row-based binario con schema embedded o en Schema Registry. Diseñado para streaming + schema evolution.

- Schema evolution: agregar/quitar campos sin romper consumers viejos. Avro lo hace bien (default values, alias). Parquet también soporta pero limitado.

Dataset / recursos

- Dataset: NYC Taxi (CSV ~2 GB/mes) — convertir a Parquet (~150 MB/mes).
- Librerías: pyarrow>=15, polars, duckdb, fastavro (Avro).

Ejercicios

1. CSV → Parquet: descargá NYC Taxi 1 mes en CSV. Cargá con pandas, escribí Parquet snappy. Comparar tamaños (CSV vs Parquet) y tiempo de query (COUNT WHERE borough='Manhattan').
2. Compresión benchmark: mismo dataset, escribir con snappy, zstd, gzip, lz4. Reportar tamaño + tiempo de lectura. (Hint: zstd suele ganar en ratio; snappy en speed).
3. Predicate pushdown: en DuckDB, EXPLAIN ANALYZE SELECT FROM parquet WHERE pickup_date='2024-01-15'. Compará "rows read" vs SELECT FROM parquet sin WHERE.
4. Row groups y stats: con pyarrow.parquet.ParquetFile(path).metadata, inspeccioná min/max/null_count por columna por row group.
5. Avro schema + roundtrip: definí schema Avro para evento de click. Serializá 1000 eventos con fastavro, deserializá. Compará tamaño con JSON puro.

Homework verificable

Notebook + reporte:

1. Benchmark sobre 1 año de NYC Taxi:
 - CSV (raw), Parquet snappy, Parquet zstd, Avro snappy, JSON gzipped.
 - Reportar para cada uno: tamaño en disco, tiempo de query COUNT *, tiempo de SELECT col WHERE filter.
1. Demostración de schema evolution: tabla Avro con schema v1 (5 campos), agregar campo opcional → v2. Consumer viejo (con v1) lee data de v2 sin romperse.
2. Justificación de elección: para 3 casos de uso (analytics warehouse, Kafka stream, archive a largo plazo), recomendar formato y compresión.
3. Cálculo de costos: pasar de CSV a Parquet en S3, asumir 100 GB/mes generados. ¿\$ ahorrados en S3 + transferencia?

Criterio de aceptación: el alumno demuestra con números (no opinión) por qué Parquet es ~5-50× más rápido para queries analíticas, y produce esquema Avro funcional con evolución backward-compatible.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Parquet más grande que CSV	(1) Schema con muchas columnas vacías. (2)
Lectura Parquet lenta — más lenta que CSV	Estás haciendo SELECT * (no aprovecha colu
ArrowInvalid: Schema mismatch al concatena	Dos parquets con types ligeramente distint
Avro consumer rompe al evolucionar schema	Cambio no-compatible (renombrar campo sin
partitionBy('user_id') en Parquet crea 1M	Particionado high-cardinality. Fix: partic
zstd no se lee en sistema legacy	Algunos viejos solo soportan snappy/gzip.

Preguntas frecuentes

¿Parquet o ORC?

Parquet: dominante fuera del ecosistema Hadoop puro. ORC: nativo del ecosistema Hive/Hortonworks (algo

declining). Para 2026 greenfield: Parquet sin pensar.

¿Parquet o Delta Lake / Iceberg / Hudi?

Parquet es solo formato de archivo. Delta Lake, Iceberg, Hudi son "table formats" sobre Parquet que agregan: ACID transactions, time travel, schema evolution rica, MERGE/UPSERT. Para data lakes serios: una de estos sobre Parquet. Para análisis ad-hoc: Parquet pelado alcanza.

¿CSV alguna vez tiene sentido?

(1) Interoperabilidad humana (Excel, miras con less). (2) Tamaño chico (<1 MB) — no vale la pena la complejidad. (3) Output de un proceso para humano. Para todo lo demás: Parquet o JSON.

¿Avro o Protobuf en Kafka?

Ambos serializan binario con schema. Avro: schema embeddable o Registry, mejor evolution semantics. Protobuf: más rápido, mejor ecosystem gRPC. Confluent Kafka favorece Avro (Schema Registry built-in); gRPC users favorecen Protobuf.

¿Cuánto comprime zstd vs snappy?

Aprox: snappy reduce 50-60%, zstd reduce 65-75%. Zstd CPU cost ~2-3× snappy en compresión, ~1× en decompresión. Para almacenamiento long-term: zstd. Para alta throughput: snappy.

¿Cómo maneja Parquet con timezone-aware datetimes?

Parquet 2.x soporta timestamp(unit=us, tz='UTC'). Polars y PyArrow lo respetan. Cuidado: pandas <2.0 a veces hace conversión silenciosa. Estandarizar en UTC siempre.

Referencias

- Parquet docs — file format spec.
- Avro spec — schema language.
- PyArrow Parquet API.
- Apache Iceberg — table format moderno.
- CSV is dead, long live Parquet (mejor para 2026).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 215 — Clase 215 — Modelado dimensional: star/snowflake schemas

Parte: 5 — Ingeniería de Datos · Fuente: Kimball & Ross The Data Warehouse Toolkit (Wiley, 3ª ed.) + Reis & Housley cap. 8. Duración estimada: 75 min.

Objetivo

Diseñar el esquema de un data warehouse usando modelado dimensional (Kimball): una fact table central + dimension tables alrededor (star schema), o dimensiones normalizadas (snowflake schema). Es el modelo que han usado los data warehouses serios desde los 90s y sigue vigente en BigQuery/Snowflake/dbt en 2026.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Identificar fact tables (eventos medibles: sale, click, payment) vs dimension tables (entidades descriptivas: customer, product, date).
- Diseñar un star schema con fact + dimensions desnormalizadas.
- Manejar slowly changing dimensions (SCD): Tipo 1 (overwrite), Tipo 2 (history con valid_from/valid_to), Tipo 3 (current + previous).
- Crear una date dimension completa (con feriados, fiscal year, etc.) — la dimensión más reutilizada.
- Diferenciar OLAP (modelado dimensional, query analíticas) de 3NF (modelado normalizado, OLTP).

Temas

#	Tema	Por qué importa
1	Star schema: fact + dims	El patrón fundamental.
2	Snowflake schema: dims normalizadas	Cuándo agregar el normalizado.
3	Surrogate keys vs natural keys	Por qué usar IDs autoincrement en DW.
4	Slowly Changing Dimensions (SCD 1/2/3)	Cómo manejar history.
5	Date dimension	La que casi siempre olvidás generar.
6	Granularidad de la fact	"Una fila por click" vs "una fila por sesi

Definiciones y características

- Fact table: tabla con métricas numéricas + foreign keys a dimensions. Ej: fact_sales(date_key, product_key, customer_key, store_key, qty, revenue). Filas chicas, MUCHAS (millones-billones).
- Dimension table: descripción de las entidades. Ej: dim_product(product_key, sku, name, category, brand). Filas grandes (muchas columnas), POCAS (miles-millones).
- Star schema: 1 fact + N dims, todas conectadas directo a la fact. Cada dim es plana (denormalizada). El default — mejor performance, joins simples.
- Snowflake schema: las dims están normalizadas (dim_product → dim_category → dim_segment). Ahorra storage en dims gigantes; complica queries.
- Surrogate key: ID autoincrement único para la dimensión (product_key=42), distinto del natural key del sistema source (sku="ABC-123"). Permite SCD Tipo 2 sin colisión.
- SCD Tipo 1 (overwrite): cambia el valor in-place. No mantiene historia. Útil para correcciones tipo "fix typo en nombre".
- SCD Tipo 2 (history): inserta nueva fila con valid_from, valid_to, is_current. La fact apunta al product_key vigente al momento del evento.
- SCD Tipo 3 (current + previous): columnas current_value + previous_value. Maneja solo 1 cambio histórico. Raro en práctica.
- Date dimension: tabla precalculada con 1 row por día (date_key, year, quarter, month, day_of_week, is_weekend, is_holiday, fiscal_year, ...). Evita derivar en cada query.
- Granularidad (grain): el nivel de detalle de la fact. "1 row per transaction" vs "1 row per day per customer". Definir el grain ES la decisión de diseño más importante.

Dataset / recursos

- Caso ejemplo: e-commerce con tablas operacionales orders, order_items, products, customers. Transformar a star schema.
- Librerías: DuckDB/SQL puro (las tablas son SQL, no Python).

Ejercicios

1. Identificar grain: dado un dataset de pedidos de e-commerce, decidí el grain de tu fact. ¿"1 row per

order"? ¿"1 row per order line"? Elegí, justificá.

2. Date dimension: SQL que genera dim_date con 5 años de días, columnas year, quarter, month, day_of_week_iso, is_weekend, is_holiday_us, fiscal_year. (generate_series de Postgres/DuckDB).
3. Star schema: del e-commerce, diseñá fact_sales(date_key, product_key, customer_key, store_key, qty, revenue, discount), dim_product, dim_customer, dim_store, dim_date. SQL completo.
4. SCD Tipo 2 en dim_customer: cliente cambia de ciudad. Tu pipeline detecta el cambio → UPDATE la fila vigente con valid_to=NOW(), is_current=FALSE → INSERT nueva fila con valid_from=NOW(), is_current=TRUE. Toda venta del cliente queda asociada a su ciudad al momento de la compra.
5. Query típica: "Revenue por brand × month × is_weekend" — escribila con JOINS entre fact + dims + dim_date. Comparala con la equivalente en tablas no-modeladas (más JOINS, más subqueries).

Homework verificable

Repo con:

1. SQL completo para crear: dim_date, dim_customer (SCD 2), dim_product, dim_store, fact_sales.
2. Script de carga: lee tablas operacionales (Postgres/CSV) → transforma → carga al DW (DuckDB local).
3. Demostración de SCD 2: un cliente que cambia de ciudad; venta antes y después del cambio quedan correctamente asociadas a la ciudad de ese momento.
4. 5 queries analíticas representativas: revenue por segment × quarter, top 10 products, cohort retention, etc.
5. README explicando: grain elegido, decisiones de denormalización, qué dim usaría snowflake (si alguna).

Criterio de aceptación: las 5 queries corren limpio; SCD 2 está correctamente implementada (verificable con un test que valida revenue agregado por ciudad-en-momento).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Star schema con grain mixto (algunas filas)	Confusión sobre el grain. Fix: definir UNA
Customer cambia de ciudad y todas las vent	Usaste SCD Tipo 1. Fix: SCD Tipo 2 con sur
Date queries lentas: WHERE EXTRACT(month F	Index no se usa con función. Fix: usar dim
Fact table tiene 200 columnas	Estás metiendo dimensiones como columnas.
Snowflake schema con 10 niveles de jerarqu	Over-engineering. Fix: empezar con star (p
dbt jobs explotan: full refresh diario de	Falta incremental loading. Fix: dbt increm

Preguntas frecuentes

¿Modelado dimensional sigue vigente con BigQuery/Snowflake?

Sí. Aunque el compute moderno es elástico, queries siguen siendo más rápidas y baratas con star schema. dbt promueve el patrón. La diferencia con los 90s: no necesitás crear índices manualmente (los DW modernos lo manejan).

¿Star o snowflake?

Star por default. Snowflake solo si tu dim tiene jerarquía profunda + millones de rows + repetición de strings que duele en storage. Para dim_product con category repetido 10K veces: snowflake (sacar dim_category). Para customer.country repetido 1M veces: dejar en star (no vale la pena el JOIN extra).

¿Qué grain elijo?

El más fino posible que tenga sentido. "1 row per transaction line" mejor que "1 row per order" — siempre podés agregar al sumar, no podés desagregar después. Excepción: si la fact crece en TBs por día, agregar a daily_sales_by_product puede ser necesario.

¿dim_date o calcular en query?

dim_date siempre. Razones: (1) feriados, fiscal year, business days requieren lookup, no aritmética. (2) Performance: JOIN a dim_date es más rápido que EXTRACT/CASE. (3) Consistencia: un único lugar define "qué es weekend".

¿Data Vault, OBT (One Big Table), Activity Schema, Anchor model — y modelos alternativos?

- Data Vault: para enterprise con many sources, schema cambiando seguido. Más complejo.
- OBT: una tabla con todo desnormalizado (extremo de star). Bueno para una sola dashboard; explota con más casos.
- Activity Schema: 1 fact universal de "actividades" + 1 dim por entidad. Patrón moderno (Narrator).
- Anchor model: 6NF extremo, raro fuera de academia.

Para 90% de los casos: star schema sigue ganando.

¿dbt es necesario?

No estrictamente, pero es el estándar. dbt convierte SQL transforms en código (versionado, testado, lineage). Si vas a hacer >5 modelos: usá dbt.

Referencias

- Kimball, R. & Ross, M. The Data Warehouse Toolkit (Wiley, 3ª ed., 2013) — la biblia del modelado dimensional.
- Reis & Housley Fundamentals of Data Engineering (O'Reilly, 2022) cap. 8.
- dbt docs — el ecosistema moderno.
- The Open Source Data Stack Conference — talks de Kimball moderno con DuckDB/dbt.
- dim_date SQL generator — dbt package listo para usar.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Parte 6 — Parte 6 — Sistemas de Recomendación

7 clases en esta parte.

Clase 216 — Clase 216 — Filtrado colaborativo: user-based e item-based

Parte: 6 — Sistemas de Recomendación · Fuente: Aggarwal, Recommender Systems: The Textbook (Springer, 2016) cap. 2 + Linden, Smith, York (Amazon, 2003) Item-to-Item Collaborative Filtering. Duración estimada: 75 min.

Objetivo

Construir el recomendador más antiguo y todavía usado: filtrado colaborativo basado en vecinos (kNN).

Calcular similitudes user-user e item-item sobre una matriz usuario-item dispersa, generar top-N recomendaciones, y entender por qué Amazon publicó en 2003 que item-based gana a user-based en escala (computar similitudes item-item es offline y estable).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Representar las interacciones usuario-item en una `scipy.sparse.csr_matrix` (típicamente >99% sparse).
- Calcular similitud coseno, Pearson y Jaccard entre filas (users) o columnas (items).
- Generar top-N recomendaciones user-based: $\text{predicted_rating} = \sum \text{sim}(u, v) * \text{rating}(v, i) / \sum \text{sim}(u, v)$.
- Generar top-N recomendaciones item-based: $\text{score}(u, i) = \sum \text{sim}(i, j) * \text{interaction}(u, j)$.
- Reconocer límites: sparsity $\rightarrow$ similitudes ruidosas; cold-start $\rightarrow$ items/users nuevos sin recomendaciones (Clase 221).

Temas

#	Tema	Por qué importa
1	Matriz usuario-item: dense vs sparse	1M users $\times$ 100K items $\rightarrow$ 100B celdas; 99.9%
2	Similitud coseno, Pearson, Jaccard	La elección define qué entiende el modelo
3	User-based kNN	Intuitivo; no escala (similitudes user-user)
4	Item-based kNN	El paper de Amazon: similitudes item-item
5	Implicit vs explicit feedback	Rating 1-5 vs "vio/no vio". Cambia loss y
6	Mean centering	Restar <code>user_mean</code> antes de calcular similit

Definiciones y características

- Matriz usuario-item $R[u, i]$: rating del user u sobre item i . Para implicit: 1 si interactuó, 0 si no.
- Sparse matrix: solo almacenan los valores no-cero. `csr_matrix` (Compressed Sparse Row) — eficiente para acceso por fila. `csc_matrix` por columna.
- Similitud coseno: $\cos(u, v) = (u \cdot v) / (|u| \times |v|)$. Insensible a la magnitud — un usuario que rateó muchas películas $\approx$ uno que rateó pocas si los patrones coinciden.
- Correlación Pearson: similar a coseno pero resta la media antes — controla el sesgo del usuario (algunos califican 5 todo, otros 3 todo).
- Jaccard: para datos binarios. $|A \cap B| / |A \cup B|$. Útil con implicit feedback.
- User-based prediction: $\hat{r}(u, i) = \sum_v \text{sim}(u, v) \times r(v, i) / \sum_v |\text{sim}(u, v)|$ con v vecinos del top-K más similares a u que rateó i .
- Item-based prediction: $\hat{r}(u, i) = \sum_j \text{sim}(i, j) \times r(u, j) / \sum_j |\text{sim}(i, j)|$ con j items más similares a i que u rateó.
- Implicit feedback: el dato es "interactuó/no interactuó" (vio, clickeó, compró). No hay rating explícito; el modelo no debe asumir 0 = dislike.

Dataset / recursos

- Dataset: MovieLens 100K (ml-100k, ~1 MB, ~100K ratings de 943 users $\times$ 1682 movies). El clásico para empezar.
- Librerías: `scipy.sparse`, `numpy`, `pandas`, `scikit-learn` (para `cosine_similarity`).

Ejercicios

1. Sparse matrix: cargar MovieLens 100K. Construir R como `csr_matrix` shape (n_users, n_items) . Verificar sparsity: $(1 - R.nnz / \text{np.prod}(R.\text{shape}))$.
2. User similarity: `sim_users = cosine_similarity(R)` — devuelve (n_users, n_users) . Encontrar los 5 más

similares a `user_id=42`.

3. User-based top-10: para `user_id=42`, predecir score para items no vistos como `R.T @ sim_users[42]` (broadcasting). Recomendar top-10 que aún no vio.
4. Item-based top-10: `sim_items = cosine_similarity(R.T)` (ahora `(n_items, n_items)`). Para `user_id=42`, `score = R[42] @ sim_items`. Mostrar top-10.
5. Pearson + mean centering: `R_centered = R - user_means.reshape(-1, 1)` (cuidado con sparse — usar sklearn). Comparar recomendaciones con coseno vanilla.

Homework verificable

Notebook con:

1. Cargar MovieLens 1M (10× más grande que 100K).
2. Implementar 2 recomendadores: user-based kNN ($k=30$) e item-based kNN ($k=30$).
3. Split train/test con leave-one-out por user (el último rating de cada user va a test).
4. Reportar MAP@10, NDCG@10, recall@10 para ambos sobre test (Clase 220).
5. Comparativa: tiempo de entrenamiento, tiempo de predicción, calidad. Discutir por qué item-based suele ganar en producción.

Criterio de aceptación: ambos recomendadores entrenan en <5 min sobre 1M, `recall@10 > 0.05`, y el estudiante justifica la elección item-based para producción.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MemoryError al construir <code>n_users × n_users</code>	Estás densificando. Fix: usar <code>sklearn.metrics</code>
Recomendaciones son siempre los items popu	Sin top-K filter de neighbors, dominan ite
Recall=0.0 sobre test	Estás recomendando items que el user ya vi
Pearson sobre rows con poca overlap da nan	Si 2 users tienen <3 items en común, Pears
Item-based <code>sim_matrix</code> de 100K × 100K	Densa = 80 GB. Fix: top-K nearest neighbor
Implicit feedback con coseno se sesga a it	Coseno normaliza, da peso a items raros. F

Preguntas frecuentes

¿User-based o item-based?

Item-based en producción (Amazon, 2003). Razones:

- Similitudes item-item cambian lento (un item es estable; un nuevo rating no mueve mucho). Pre-computable offline.
- Similitudes user-user cambian con cada rating nuevo del user.
- $N \text{ items} \ll N \text{ users}$ en muchos casos (productos vs millones de clientes).

¿cosine_similarity o pearson_correlation?

Coseno por default. Pearson cuando el sesgo absoluto del usuario importa (algunos califican alto todo, otros bajo todo) — frecuente en sistemas de rating 1-5 explícito.

¿Implicit feedback se trata igual?

No. Con implicit, $R[u, i] = 0$ NO significa "dislike" — puede significar "no lo vio aún". Estrategias: BPR (Bayesian Personalized Ranking), ALS implicit con confidence weighting (Clase 217), o métricas top-N (NDCG@k) en vez de RMSE.

¿Cuándo CF se rompe?

(1) Cold-start total: user/item nuevo con 0 historia (Clase 221). (2) Sparsity extremo (>99.99%): similitudes son ruidosas. (3) Long-tail dominante: 1% de items reciben 80% del tráfico, recomendador converge a "los populares".

¿Surprise/LightFM/Implicit hacen esto por mí?

Sí (Clase 222). Surprise tiene KNNBasic/KNNWithMeans. Implicit es especialista en implicit feedback. LightFM combina CF + content (Clase 219).

¿Y si tengo 100M users × 10M items?

kNN no escala. Saltá directo a factorización de matrices con ALS distribuido (Spark ALS o Implicit als.AlternatingLeastSquares), o a embeddings deep (TwoTowers, BERT4Rec).

Referencias

- Aggarwal, C. Recommender Systems: The Textbook (Springer, 2016), cap. 2 — Neighborhood-Based Collaborative Filtering.
- Linden, G., Smith, B., York, J. Amazon.com Recommendations: Item-to-Item Collaborative Filtering (IEEE Internet Computing, 2003) — el paper fundacional.
- MovieLens datasets.
- sklearn.metrics.pairwise.cosine_similarity — con dense_output=False para sparse.
- Programming Collective Intelligence (Segaran, 2007) — introducción accesible.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 217 — Clase 217 — Factorización de matrices: SVD, ALS

Parte: 6 — Sistemas de Recomendación · Fuente: Koren, Bell, Volinsky Matrix Factorization Techniques for Recommender Systems (IEEE Computer, 2009) + Hu, Koren, Volinsky Collaborative Filtering for Implicit Feedback Datasets (ICDM 2008). Duración estimada: 80 min.

Objetivo

Reemplazar la matriz usuario-item dispersa por dos matrices densas de baja dimensión: $R \approx P \times Q^T$ donde P ($n_users \times k$) y Q ($n_items \times k$). Aprender los embeddings k-dimensionales que capturan los factores latentes (género de película, gusto del usuario). Usar SVD (cuando hay datos completos) y ALS (Alternating Least Squares — robusto a sparse). Implicit-feedback ALS (Hu et al. 2008) es el algoritmo que ganó la mayoría de los Netflix Prize spin-offs en producción.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar el modelo $\hat{r}(u, i) = p_u \cdot q_i + b_u + b_i + \mu$ (con biases).
- Aplicar SVD truncado sobre matriz densa (poco realista, pero base teórica).
- Implementar ALS explicit (rating predicho) y ALS implicit (con confidence weighting $c_{ui} = 1 + \alpha \times r_{ui}$).
- Elegir hiperparámetros: factors (típicamente 20-200), regularization (λ para evitar overfitting), iterations (10-30).
- Usar implicit.AlternatingLeastSquares (Cython, multi-thread, rápido).

Temas

#	Tema	Por qué importa				
1	Modelo latente: fac	"Acción", "drama",				
2	SVD vs SVD trunc	SVD asume matriz				
3	Biases: μ , b_u , b_i	"Usuarios duros" y				
4	Implicit feedback +	0 no es "dislike"; p				
5	Regularización L2	$\lambda \times ($	p	$^2 +$	q	$^2)$ para evitar overfitting con k' alto
6	Cold-start parcial:	Para user nuevo: $\hat{r}$				

Definiciones y características

- Factor latente: dimensión del embedding que captura una variable no observada (género, mood, demographic).
- SVD truncado: descomposición $R = U \Sigma V^T$ quedándose con top-k singular values. Asume R completa (no es nuestro caso).
- ALS (Alternating Least Squares): fija Q, resuelve P por LSQ $\rightarrow$ fija P, resuelve Q $\rightarrow$ itera. Cada subproblema es lineal y paralelizable.
- Modelo con biases: $\hat{r}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$. μ es la media global; b_u/b_i capturan el sesgo del usuario/item.
- Implicit feedback ALS: en vez de minimizar error sobre ratings observados, minimiza sobre TODA la matriz, ponderando por confidence $c_{ui} = 1 + \alpha \times r_{ui}$. Items con interacción tienen alta confianza de "1" (preferencia); sin interacción tienen confianza baja de "0".
- Regularización: $L = \sum (r_{ui} - p_u \cdot q_i)^2 + \lambda \times (|p_u|^2 + |q_i|^2)$. Sin esto, embeddings explotan a memorizar el train.
- Factors k: típicamente 20-200. Más $\rightarrow$ más capacidad y más overfitting. Tunear con validación.
- Funk SVD (Simon Funk, Netflix Prize 2006): el "SVD" popular del Netflix Prize NO es SVD sensu stricto — es factorización por SGD con regularización. El nombre quedó.

Dataset / recursos

- Explicit: MovieLens 1M con ratings 1-5.
- Implicit: Last.fm "lastfm-360K" o convertir MovieLens (rating ≥ 4 = "le gustó" = 1).
- Librerías: implicit ≥ 0.7 (ALS rápido en Cython), scipy.sparse.linalg.svds, opcional surprise.

Ejercicios

1. SVD truncado: tomar matriz R densa imputando 0. $U, \sigma, V_t = \text{svds}(R, k=20)$. Reconstruir $R' = U \times \text{diag}(\sigma) \times V_t$. Verificar que R' predice valores no-cero similares y "rellena" los ceros con guesses.
2. ALS explicit con Surprise: `from surprise import SVD; algo = SVD(n_factors=50, n_epochs=20). algo.fit(trainset). Predict algo.predict(user, item). RMSE sobre test split.`
3. ALS implicit con implicit: `model = implicit.als.AlternatingLeastSquares(factors=64, regularization=0.05, iterations=15). model.fit(R_train * 40) (multiplicar por  $\alpha=40$ ). model.recommend(user_id, R_train[user_id], N=10).`
4. Inspeccionar embeddings: PCA de `model.item_factors` a 2D y plot. Items "parecidos" deben caer cerca.
5. Biases analysis: imprimir top 10 movies por b_i (las que todos aman / odian) y top 10 usuarios por b_u .

Homework verificable

Notebook con:

1. Comparar 3 modelos sobre MovieLens 1M con leave-one-out por user:

- kNN item-based (Clase 216).
 - Funk SVD (Surprise).
 - ALS implicit (binarizando rating ≥ 4).
1. Reportar NDCG@10, recall@10, tiempo de entrenamiento.
 2. Plot 2D (UMAP/t-SNE) de item_factors. Colorear por género — los géneros deberían formar clusters visibles.
 3. Estudio de sensibilidad: $k \in \{10, 50, 100, 200\} \times \lambda \in \{0.001, 0.01, 0.1\}$ con cross-validation.
 4. Recomendar para 3 users diferentes (un cinéfilo, un casual, un nuevo) — mostrar diferencia cualitativa.

Criterio de aceptación: ALS implicit gana en NDCG@10 sobre kNN; el grid de (k, λ) revela el sweet spot; los embeddings de items muestran estructura por género al PCAear.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
svds da componentes con signo arbitrario	SVD no fija el signo de los singular vecto
ALS converge a embeddings constantes	λ muy alto. Fix: bajar regularización (0.0
MemoryError con ALS sobre 10M users $\times$ 1M i	Implementación naive. Fix: implicit librar
Implicit-ALS overfit: recomienda solo lo q	Sin confidence weighting o con α muy alto.
Embeddings no muestran estructura semántic	k muy chico ($k=5$ no capta nada) o train se
Predicciones constantes para todos los use	Modelo bajó a solo predecir biases. Fix: c
Cold-start total: nuevo user $\rightarrow$ score=0	p_u no existe para user nuevo. Fix: fallback

Preguntas frecuentes

¿SVD del Netflix Prize es SVD matemático?

No. Lo que ganó Netflix era factorización por SGD con regularización ("Funk SVD"), no SVD-decomposition real. El nombre pegó por marketing.

¿implicit o lightfm o tensorflow recommenders?

- implicit: ALS implicit + BPR. Rápido (Cython), implicit-only. Default para CF moderno.
- lightfm: hybrid (CF + content features). Bueno si tenés metadata (Clase 219).
- tensorflow recommenders / recsim: deep learning, two-towers, sequential. Para escala grande + features ricas.

¿Qué k (factors) elijo?

Típicamente 50-200. Más k requiere más data + más regularización. Si tu dataset es chico ($<100K$ interactions): $k=20-50$. Si tenés millones de interactions: $k=100-200$. Cross-validation es la única respuesta correcta.

¿ALS vs SGD para optimizar?

- ALS: cada subproblema es LSQ lineal $\rightarrow$ paralelizable, converge en pocas iteraciones. Bueno cuando matrix cabe en memoria y querés multi-core.
- SGD: por minibatch, on-line, escala a streaming. Más sensible a learning rate.

implicit usa ALS; PyTorch implementations típicamente usan SGD.

¿Embeddings se pueden usar para más cosas?

Sí. Aplicaciones: (1) similar items ($\cos(q_i, q_j)$), (2) clustering de items, (3) features para modelos downstream (CTR prediction), (4) two-tower retrieval (precomputar q_i , buscar nearest neighbor de p_u en producción con FAISS).

¿Y deep learning recommenders (NCF, deep CF)?

Two-towers + transformers (BERT4Rec, SASRec) son SOTA para sequential recommendation. Pero: ALS implicit sigue compitiendo en producción por simplicidad, latencia y costo. Empezá con ALS; pasá a DL si la métrica lo justifica.

Referencias

- Koren, Y., Bell, R., Volinsky, C. Matrix Factorization Techniques for Recommender Systems (IEEE Computer, 2009) — el clásico Netflix Prize.
- Hu, Y., Koren, Y., Volinsky, C. Collaborative Filtering for Implicit Feedback Datasets (ICDM 2008) — implicit ALS.
- implicit library — ALS + BPR en Cython.
- Funk SVD blog (Simon Funk, 2006) — el blog que cambió la historia.
- Recommender Systems: An Introduction (Jannach et al., 2010).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 218 — Clase 218 — Content-based filtering

Parte: 6 — Sistemas de Recomendación · Fuente: Aggarwal Recommender Systems: The Textbook cap. 4 + docs scikit-learn TfidfVectorizer. Duración estimada: 70 min.

Objetivo

Recomendar items basándose en sus atributos (texto, género, categoría) en vez de interacciones — útil cuando hay cold-start de items (Clase 221) o cuando los items tienen rica metadata. Combinar TF-IDF / embeddings sobre descripciones + scoring por similitud al perfil del usuario.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Construir un item profile desde texto + features categóricas con TF-IDF / one-hot.
- Construir un user profile como agregado ponderado de items que le gustaron.
- Calcular $\text{score}(u, i) = \cos(\text{user_profile}, \text{item_profile})$ y rankear.
- Reemplazar TF-IDF por embeddings modernos (sentence-transformers) para entender semántica.
- Reconocer límites: serendipia baja (recomienda variantes de lo conocido); sobreespecialización.

Temas

#	Tema	Por qué importa
1	TF-IDF + cosine similarity	El baseline desde 1990.
2	One-hot vs multi-hot features categóricas	Géneros, tags, autores.
3	User profile como media ponderada	El "embedding del gusto".
4	Embeddings semánticos (sentence-transformers)	all-MiniLM-L6-v2 reemplaza TF-IDF con much

5	FAISS para top-N rápido	Cuando items son millones.
6	Hybrid con CF (Clase 219)	Content-only sufre serendipia.

Definiciones y características

- Content-based: usa atributos del item (texto, género, autor, año, precio) para recomendar. NO usa interacciones de otros usuarios (a diferencia de CF).
- TF-IDF (Term Frequency × Inverse Document Frequency): convierte texto en vector. Frecuencia de la palabra en el item × $\log(N / \text{docs que contienen la palabra})$.
- Item profile: vector que representa al item. Para texto: TF-IDF. Para géneros: multi-hot. Para mix: concatenación.
- User profile: vector que representa el "gusto" del user. Típicamente: media (ponderada por rating) de los item profiles que consumió.
- Cosine similarity: ángulo entre vectores, robusto a magnitud. Default para content-based.
- Embedding semántico: vector denso (e.g. 384-dim) de un modelo pre-entrenado (BERT, MiniLM) que captura significado, no solo palabras exactas. "Auto" y "carro" → similar.
- Serendipia: probabilidad de descubrir algo inesperado. Content-based la mata (recomienda variantes de lo conocido). CF sufre menos.

Dataset / recursos

- Dataset: MovieLens + sinopsis (de TMDb API) o kaggle/the-movies-dataset.
- Librerías: scikit-learn (TfidfVectorizer), sentence-transformers, opcional faiss-cpu.

Ejercicios

1. TF-IDF base: cargar movies con title + overview + genres. TfidfVectorizer(max_features=10000, ngram_range=(1,2)). Matrix shape (n_items, 10000).
2. Item-item similarity: cosine_similarity(tfidf_matrix). Para Toy Story, mostrar top-10 más similares — deberían ser otras animaciones.
3. User profile: para user_id=42, tomar items rateados ≥ 4 . user_profile = mean(item_profiles) (ponderado por rating). Recomendar top-10.
4. Embeddings modernos: model = SentenceTransformer('all-MiniLM-L6-v2'). Generar embeddings de cada movie. Comparar top-10 de TF-IDF vs embeddings — los embeddings entienden semántica.
5. FAISS rápido: index = faiss.IndexFlatIP(384); index.add(embeddings). index.search(user_profile, k=10) → top-10 en <1 ms aunque haya 1M items.

Homework verificable

Notebook con:

1. Recomendador content-based con sentence-transformers sobre MovieLens + sinopsis (10K movies).
2. FAISS index para retrieval <10 ms p99.
3. Comparativa con CF (Clase 216-217): NDCG@10, coverage (% items recomendados al menos una vez), diversidad (1 - avg intra-list similarity).
4. Demostración cold-start: agregar una "movie nueva" sin ratings; verificar que content-based la recomienda; CF no.
5. README discutiendo: cuándo content-based gana (cold-start, niche items), cuándo pierde (filter bubble, serendipia baja).

Criterio de aceptación: content-based recomienda la movie nueva sin ratings (CF la ignora); coverage de content-based es $\geq$ CF; el estudiante justifica un hybrid (Clase 219).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
TF-IDF gigante: shape (10K items, 100K fea	Sin max_features o stopwords. Fix: max_fea
Recomendaciones todas iguales: variantes d	Sobreespecialización (filter bubble). Fix:
User profile vacío para new user	No tiene interacciones todavía. Fix: onboar
Embeddings de 100K items en RAM = 150 MB	Es OK pero el cosine_similarity 100K × 100
Multi-hot de géneros domina el TF-IDF	Concatenaste sin normalizar. Fix: normaliz
sentence-transformers tarda mucho al carga	Es lento la primera vez (download del chec

Preguntas frecuentes

¿Content-based vs CF?

Resuelven cosas distintas, y la mejor solución suele ser hybrid (Clase 219).

- Content-based: maneja item cold-start, explica por qué ("te recomendamos X porque te gustó Y, ambos de género acción"), pero filter bubble.
- CF: capta señales sociales (a gente como vos le gustó), serendipia, pero no funciona con items nuevos.

¿TF-IDF o embeddings?

Embeddings (sentence-transformers) ganan en calidad semántica. TF-IDF gana en velocidad de inferencia y simplicidad. Para 2026 greenfield: empezá con embeddings, agregá FAISS para retrieval.

¿Cómo combino texto + features categóricas?

Tres opciones: (1) concatenar TF-IDF + one-hot (escalado), (2) entrenar un modelo de feature embedding (fastText, USE) sobre todo, (3) usar metadata como features adicionales en LightFM (Clase 222).

¿sentence-transformers modelo elijo?

- all-MiniLM-L6-v2: 384-dim, rápido, buena calidad. Default razonable.
- all-mpnet-base-v2: 768-dim, mejor calidad, ~3× más lento.
- multilingual: paraphrase-multilingual-MiniLM-L12-v2.

¿FAISS es necesario?

Si tenés <10K items: cosine_similarity con sklearn alcanza. Si tenés >100K items y querés top-N en <50 ms: FAISS obligatorio. Para 1M+ items y escala: FAISS con índices avanzados (HNSW, IVF) o Milvus/Pinecone (managed vector DBs).

¿Y los embeddings de OpenAI?

text-embedding-3-small/large son fuertes; pagás por inference + lock-in. Para empezar/offline: sentence-transformers gratis. Para producción con presupuesto: OpenAI / Cohere / Voyage.

Referencias

- Aggarwal, C. Recommender Systems: The Textbook (Springer, 2016), cap. 4 — Content-Based Recommender Systems.
- sklearn TfidfVectorizer.
- sentence-transformers — modelo pre-entrenados.
- FAISS — similarity search.
- Carbonell & Goldstein, The Use of MMR, Diversity-Based Reranking for Reordering Documents (1998) — para combatir filter bubble.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 219 — Clase 219 — Recomendadores híbridos

Parte: 6 — Sistemas de Recomendación · Fuente: Burke, Hybrid Recommender Systems: Survey and Experiments (UMUAI 2002) + Kula, Metadata Embeddings for User and Item Cold-start Recommendations (LightFM, 2015). Duración estimada: 75 min.

Objetivo

Combinar CF (filtrado colaborativo, Clase 216-217) + content-based (Clase 218) para conseguir lo mejor de ambos: serendipia + cold-start + explicabilidad. Aplicar los 7 patrones de hybrid de Burke (2002) y usar LightFM (que aprende un modelo único con CF + features).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar los 7 patrones de hybrid: weighted, switching, mixed, feature combination, cascade, feature augmentation, meta-level.
- Implementar un hybrid weighted: $\text{score} = \alpha \times \text{score_cf} + (1-\alpha) \times \text{score_content}$.
- Implementar un hybrid switching: usar content para users con $<N$ interactions, CF para el resto.
- Usar LightFM como hybrid built-in: el modelo aprende embeddings que combinan CF + content features.
- Tunear α con validation y entender por qué α óptimo varía por user/item segment.

Temas

#	Tema	Por qué importa
1	7 patrones de Burke (2002)	Vocabulario para discutir arquitecturas.
2	Weighted hybrid: $\alpha \times \text{CF} + (1-\alpha) \times \text{CB}$	El más simple y muy efectivo.
3	Switching: cold-start triage	"Si user tiene <5 ratings, usá content".
4	LightFM: hybrid aprendido	Embeddings que combinan ambas señales.
5	Tuning α por segmento	Cold-start: peso a content; long-tail: pes
6	Two-tower modelo (concept)	El sucesor moderno de LightFM.

Definiciones y características

- Weighted: combinar scores por suma ponderada. $\text{score} = \alpha \times \text{CF} + (1-\alpha) \times \text{CB}$. α se tunea con validación.
- Switching: elegir uno u otro según contexto. Ej: cold-start (user nuevo) → content; usuario maduro → CF.
- Mixed: presentar resultados de ambos en la misma página (ej: carousel "porque te gustó X" + carousel "popular ahora").
- Feature combination: agregar señales CF (ratings) como features a un modelo content-based, o viceversa.
- Cascade: primero filtra con un modelo (e.g. CF top-100), después re-rankea con otro (e.g. content-based fine-grained).
- Feature augmentation: usar el output de un modelo (e.g. cluster CF del user) como feature de otro.
- Meta-level: el modelo aprende cuándo usar cada uno (gating network, mixture of experts).

- LightFM: librería que aprende un único embedding por user/item combinando CF + content features. Funciona en pure CF, pure content, o hybrid.
- Two-tower architecture: red neuronal con dos encoders (user tower + item tower), ambos producen embeddings que se combinan via dot product. SOTA moderno para retrieval.

Dataset / recursos

- Dataset: MovieLens 100K con u.item que tiene géneros (19 binarios).
- Librerías: lightfm>=1.17, scipy.sparse, pandas.

Ejercicios

1. Weighted hybrid manual: tomar scores de Clase 217 (ALS) y Clase 218 (content-based). Combinar $\text{score} = \alpha \times \text{cf} + (1-\alpha) \times \text{cb}$ para $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. Reportar NDCG@10 para cada α .
2. Switching por user: si $\text{interactions}(u) < 5$: usar content; si no: usar CF. Comparar contra weighted para users en distintos segmentos (new/mature).
3. LightFM hybrid: entrenar LightFM(loss='warp') con item_features (géneros) y user_features (demographics). Comparar NDCG vs pure CF (sin features).
4. Cold-start eval: held-out incluye items nuevos (no en train) y users nuevos (sin ratings). Comparar pure CF (~0%), pure content (~OK), LightFM hybrid (~mejor).
5. Cascade: top-100 con CF, re-ranear top-10 con content (boost a items con descripción similar al historial del user).

Homework verificable

Notebook con:

1. 4 modelos sobre MovieLens 100K:
 - Pure CF (implicit ALS).
 - Pure content-based (sentence-transformers).
 - Weighted hybrid (α tunado).
 - LightFM hybrid (con item features).
1. Tabla comparativa: NDCG@10, recall@10, coverage, diversity, tiempo train, tiempo predict.
2. Análisis por segmento: cold-start users (≤ 3 ratings), warm users (≥ 20). ¿Cuál gana en cada?
3. Curva α vs NDCG@10 para weighted hybrid.
4. Documentación: arquitectura recomendada para 3 casos: e-commerce, streaming, news.

Criterio de aceptación: weighted hybrid o LightFM gana sobre pure CF en al menos uno de los segmentos; el α óptimo está justificado con números; las recomendaciones por arquitectura son sensatas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Weighted hybrid no mejora sobre CF	α no tuneado, o scores en escalas distinta
LightFM peor que pure CF	Item features con poca info (e.g. 19 géner)
Switching tiene "jump" entre user con 4 vs	Discontinuidad. Fix: weighted con $\alpha(n) = s$
Cold-start eval reporta ~0 para CF	Esperado. Fix: ese es el punto del ejercic
LightFM WARP loss muy lento	WARP es $O(n_{\text{items}} \times \text{negatives})$ por ejemplo
Mismas recomendaciones para todos los user	Probablemente $\alpha=0$ o $\alpha=1$ colapsa todo. Fix:

Preguntas frecuentes

¿Qué hybrid pattern elegir?

Empezá con weighted o switching — son los más simples e interpretables. LightFM si tenés features ricos. Two-tower deep solo si tenés equipo ML + escala que lo justifique.

¿LightFM sigue siendo relevante en 2026?

Sí para datasets pequeños/medianos con features. Para grandes escalas: two-tower (TF Recommenders, recsys-pytorch). Pero LightFM es el sweet spot "complejidad-calidad" para casos chicos-medianos.

¿Cómo tuneo α ?

(1) Grid search con validation NDCG@10. (2) Bayesian opt si tenés más hiperparámetros. (3) Por segmento (cold-start users $\rightarrow \alpha$ bajo, mature $\rightarrow \alpha$ alto). (4) Online via contextual bandit (avanzado).

¿Mixed (carousels) cuenta como hybrid?

Sí — es el patrón más usado en producción real. Spotify/Netflix muestran múltiples carousels con distintas estrategias ("porque viste X", "popular en tu país", "nuevos lanzamientos"). Cada carousel es un recomendador distinto; el "hybrid" es la página final.

¿Cuándo NO usar hybrid?

Si: (1) tu dataset es 100% interactions sin features útiles $\rightarrow$ pure CF. (2) Tus items no tienen historial pero tenés metadata rica $\rightarrow$ pure content. (3) Sos un startup con 1 ML eng $\rightarrow$ empezá simple, sumá hybrid cuando duela.

¿Two-tower vs LightFM?

Two-tower (TF Recommenders, PyTorch) escala mejor + permite features más complejos (text embeddings, image embeddings). LightFM es matriz factorization + lineal sobre features. Para 2026 SOTA: two-tower. Para empezar: LightFM.

Referencias

- Burke, R. Hybrid Recommender Systems: Survey and Experiments (UMUAI 2002) — los 7 patrones canónicos.
- Kula, M. Metadata Embeddings for User and Item Cold-start Recommendations (DLRS 2015) — LightFM paper.
- LightFM docs.
- TensorFlow Recommenders — two-tower moderno.
- Aggarwal cap. 6 — Ensemble-Based and Hybrid Recommender Systems.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 220 — Clase 220 — Métricas: MAP@k, NDCG, recall@k

Parte: 6 — Sistemas de Recomendación · Fuente: Aggarwal cap. 7 + Järvelin & Kekäläinen, Cumulated Gain-Based Evaluation of IR Techniques (TOIS 2002 — NDCG paper). Duración estimada: 70 min.

Objetivo

Evaluar recomendadores con las métricas correctas: NO accuracy ni RMSE de rating (irrelevante para top-N). Sí recall@k (¿cuántos relevantes recuperaste en top-k?), precision@k (¿qué % del top-k es relevante?),

MAP@k (precision promedio sensible al ranking), NDCG@k (gain descontado por posición). Decidir qué métrica reportar según objetivo de negocio.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Calcular precision@k, recall@k, F1@k, hit rate desde cero.
- Calcular MAP@k (Mean Average Precision) y entender por qué sensible al ranking dentro del top-k.
- Calcular NDCG@k (Normalized Discounted Cumulative Gain) y entender el descuento logarítmico.
- Diseñar el split correcto: leave-one-out por user vs temporal split vs random.
- Decidir métrica según objetivo: recall (catálogo grande, descubrimiento) vs NDCG (orden importa) vs MAP (búsqueda).

Temas

#	Tema	Por qué importa
1	Por qué NO usar RMSE de rating	Top-N no usa el rating predicho — usa el r
2	Precision@k vs recall@k	Trade-off; recall importa más con catálogo
3	MAP@k: precision promediada por posición	Sensible a poner relevantes ARRIBA del top
4	NDCG@k con descuento $\log_2(i+1)$	El relevante en posición 1 vale más que en
5	Leave-one-out vs temporal split	Temporal evita data leakage en RS de tiempo
6	Coverage + diversity (beyond accuracy)	Métricas de "salud" del catálogo.

Definiciones y características

- Recall@k: $|\text{relevantes} \cap \text{top-k}| / |\text{relevantes totales}|$. Pregunta: "¿qué % de lo que el user quiere mostré?". Ideal con catálogo grande.
- Precision@k: $|\text{relevantes} \cap \text{top-k}| / k$. Pregunta: "¿qué % del top-k es relevante?". Ideal cuando el usuario consume pocos (top-5).
- F1@k: armónico de precision y recall.
- Hit rate@k: 1 si al menos 1 relevante en top-k, 0 si no. Más simple, popular en deep recsys research.
- AP@k (Average Precision): $\sum_i (\text{precision}@i \times \text{rel}(i)) / |\text{relevantes}|$ para $i \in [1, k]$. Penaliza relevantes en posiciones bajas.
- MAP@k (Mean AP): promedio de AP@k sobre todos los users.
- DCG@k (Discounted Cumulative Gain): $\sum_i \text{rel}(i) / \log_2(i+1)$. Posiciones bajas valen menos.
- NDCG@k: $\text{DCG}@k / \text{iDCG}@k$ donde iDCG es el máximo posible. Normaliza a $[0, 1]$.
- Coverage (catalog coverage): $|\text{items recomendados al menos 1 vez en N users}| / |\text{items totales}|$.
- Diversity (intra-list): $1 - \text{avg}(\text{sim}(i, j))$ entre pares en la lista. Alto = recomendaciones diversas.

Dataset / recursos

- Dataset: MovieLens 100K.
- Librerías: numpy puro (las métricas son fáciles), opcional recmetrics.

Ejercicios

1. Implementar precision@k, recall@k: dadas listas $[1, 0, 0, 1, 0]$ (relevancia) y $k=5$, calcular. Verificar contra recmetrics u otra librería.
2. MAP@k: implementar AP@k. Mostrar diferencia con precision@k: AP penaliza tener los relevantes al final.
3. NDCG@k: implementar DCG y iDCG. Mostrar caso donde recall@k es igual pero NDCG distinto porque el orden cambia.

4. Leave-one-out por user: para cada user con ≥ 5 ratings, dejar el último (cronológico o random) para test, resto para train. Evaluar.
5. Coverage y diversity: para 1000 users, ¿qué % del catálogo fue recomendado? Diversidad media intra-list (cosine entre items recomendados).

Homework verificable

Notebook con:

1. Implementación de 6 métricas: recall@10, precision@10, F1@10, MAP@10, NDCG@10, hit rate@10. Validar contra librería.
2. Aplicar a 4 modelos (kNN, ALS, content, hybrid de Clases 216-219). Tabla comparativa.
3. Análisis: ¿qué métrica decidiría cuál modelo gana? Discutir.
4. Beyond accuracy: coverage, diversity, novelty (1 - popularity rank promedio). ¿Algún modelo gana en accuracy pero pierde en diversity?
5. Temporal split: si tu dataset tiene timestamps, hacer split por fecha (no por ratio). Comparar con random split. Discutir por qué temporal es más realista.

Criterio de aceptación: las 6 métricas coinciden con recmetrics $\pm 1e-6$; la elección de "mejor modelo" depende de la métrica; el estudiante justifica una arquitectura con métricas de negocio.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
MAP@k diferente a la librería	Diferencias de convención: ¿dividís por mi
NDCG@k > 1	Bug: estás computando DCG sin normalizar p
Recall=0.0 cuando esperaba algo	Probablemente recomendaste items que ya es
RMSE alto pero recall@10 alto	Esperado en implicit. RMSE no importa para
Random split en dataset temporal → métrica	Data leakage: train tiene futuro vs test.
Coverage 100% → "óptimo"	Coverage 100% probablemente significa reco
Diversity alta pero NDCG bajo	El modelo está recomendando cosas random.

Preguntas frecuentes

¿Qué métrica reportar?

Casi siempre: NDCG@k + recall@k + coverage. NDCG mide calidad del ranking, recall mide cobertura del relevante, coverage mide salud del catálogo. Si tu UI muestra top-5: k=5. Top-20: k=20.

¿MAP o NDCG?

NDCG es más expresivo (acepta relevancia graduada: rating 5 > rating 3). MAP solo binario. En implicit feedback (binary): equivalentes en práctica. Default: NDCG.

¿RMSE alguna vez sirve?

Sí, en sistemas de rating prediction donde el rating predicho se muestra al usuario (ej. Netflix mostraba "92% match"). Si solo mostrás top-N: NO usar RMSE.

¿Leave-one-out o random split?

LOO por user es estándar en RS papers — emula la situación "predecir el próximo item". Random split puede sesgar (data leakage si hay temporalidad). Para producción real: temporal split (train con datos hasta T-7d, test con T-7d a T-0d).

¿Cómo manejo cold-start en evaluación?

(1) Reportar métricas por segmento (cold vs warm users; new vs popular items). (2) En "all", incluir cold-start (es honesto, aunque baje promedio). (3) NO excluir cold-start users — el sistema real los tiene.

¿Online metrics vs offline?

Las offline (NDCG, recall) son proxies. Online (CTR, conversion, watch time, retention) son la verdad. La correlación offline-online suele ser positiva pero no perfecta. En producción: A/B test (Clase 204).

Referencias

- Aggarwal Recommender Systems: The Textbook (Springer, 2016), cap. 7 — Evaluating Recommender Systems.
- Järvelin, K. & Kekäläinen, J. Cumulated Gain-Based Evaluation of IR Techniques (ACM TOIS 2002) — paper original de NDCG.
- recmetrics — librería con todas implementadas.
- Beyond accuracy: evaluating recommender systems by coverage and serendipity (Ge et al., RecSys 2010).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 221 — Clase 221 — Cold-start problem

Parte: 6 — Sistemas de Recomendación · Fuente: Schein, Popescul, Ungar, Pennock, Methods and Metrics for Cold-Start Recommendations (SIGIR 2002) + Aggarwal cap. 13. Duración estimada: 70 min.

Objetivo

Cuándo un user o item es "nuevo" (0 interacciones), CF (Clase 216-217) no funciona. Estrategias concretas para los 3 tipos de cold-start: user cold-start (onboarding), item cold-start (catalog launch), system cold-start (lanzamiento del producto). Las estrategias correctas son la diferencia entre un recomendador útil desde el día 1 vs uno inútil hasta el mes 6.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar user cold-start (usuario nuevo), item cold-start (item nuevo), system cold-start (todo nuevo).
- Aplicar popularity fallback con shrinkage Bayesiano $((c + m \times C) / (n + m))$ para users sin historia.
- Diseñar onboarding explícito (pedir N preferencias antes de personalizar).
- Usar content features (Clase 218) para items nuevos.
- Aplicar exploration-exploitation con bandits (epsilon-greedy, Thompson sampling) para users mid-cold-start.

Temas

#	Tema	Por qué importa
1	3 tipos de cold-start	Cada uno necesita estrategia distinta.
2	Popularity fallback	Default sano cuando no sabés nada.
3	Bayesian shrinkage para popularity	Evita que items con 2 ratings 5/5 dominen.

4	Onboarding: preguntar al user	"Elegí 3 géneros" cambia el juego.
5	Content-based para item cold-start	Embeddings de texto/imágenes.
6	Bandits para explorar	Cuando offline metrics no alcanzan.

Definiciones y características

- User cold-start: usuario sin historial (recién registrado). El modelo no tiene p_u . Estrategias: popularity, onboarding, demographics.
- Item cold-start: item nuevo sin ratings. El modelo no tiene q_i . Estrategias: content features (descripción, categoría), boost temporal "nuevos".
- System cold-start: lanzamiento del producto, ni users ni items con historia. Estrategias: editorial picks, externos (importar gustos desde Facebook/Google), popularity por geo/demographic.
- Popularity shrinkage Bayesiano: $\text{score} = (\sum \text{ratings} + m \times C) / (n + m)$ donde C es la media global, m es el "weight" del prior. Items con $n=2$ ratings 5/5 no superan items con $n=1000$ ratings 4.2/5.
- Onboarding explícito: pedir al user N preferencias (géneros, intereses) antes de personalizar. Estilo Spotify/Netflix.
- Bandits: framework de RL para exploration vs exploitation. Epsilon-greedy: con prob ϵ explora random, con prob $1-\epsilon$ recomienda el "best". Thompson sampling: muestra de la posterior de cada arm.
- Contextual bandits: el contexto (user features) influye en la decisión. Más sofisticado que vanilla bandits.

Dataset / recursos

- Dataset: MovieLens 100K + new users / new items simulados.
- Librerías: numpy, opcional vowpalwabbit (bandits), scikit-learn.

Ejercicios

1. Popularity baseline: rankear items por n_{ratings} . Top-10 son siempre los mismos. Evaluar $\text{recall}@10$ para users cold-start (con 0 interactions).
2. Bayesian shrinkage: implementar $(\text{sum} + m \times C) / (n + m)$ con $m=10$, $C=\text{mean_rating}$. Comparar top-10 con popularity vanilla. Items con pocos ratings caen al promedio.
3. Onboarding 3 géneros: simular que user nuevo elige ["Action", "Sci-Fi", "Comedy"]. Recomendar top-10 movies de esos géneros (content-based + popularity tiebreaker).
4. Item cold-start: agregar 10 movies nuevas con descripción pero 0 ratings. Content-based (Clase 218) las puede rankear; CF no. Demostrar.
5. Epsilon-greedy: en cada slot del top-10, con prob $\epsilon=0.1$ recomendar item random (explore), con prob 0.9 recomendar el "best" del modelo (exploit). Medir coverage en N usuarios.

Homework verificable

Repo con:

1. Recomendador full con manejo explícito de cold-start:
 - User nuevo: onboarding (elegir géneros) $\rightarrow$ content-based.
 - User con ≤ 5 interactions: weighted hybrid con α bajo (más content).
 - User maduro: CF puro.
1. Item nuevo: content-based hasta acumular 10 ratings, después ALS.
2. Bayesian shrinkage en todos los rankings "popularity-based".
3. Bandit epsilon-greedy en producción (simulado): tasa $\epsilon=0.05$, decay a 0.01 después de 30 días.
4. Evaluación por segmento (cold-start, warm) con $\text{NDCG}@10$. Mostrar que estrategia adaptativa supera a CF puro para cold.

Criterio de aceptación: cold-start users tienen $NDCG@10 > 0.05$ (vs ~ 0 con CF puro); ítems nuevos aparecen en recomendaciones dentro de las primeras 48h.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Popularity baseline domina todo	Tu CF no está aprovechando — probable bug
Items nuevos nunca son recomendados ("rece	Pure content sin boost por novelty. Fix: a
Onboarding pide 20 géneros — los users se	UX trade-off. Fix: pedir 3-5 max; mejor 1
ϵ -greedy estanca: nunca aprende	ϵ muy bajo desde día 1. Fix: arrancar con
Bayesian shrinkage me da scores casi igual	m muy alto. Fix: tunear m por validación —
Geographic popularity sesga a country mayo	Sin segmentación. Fix: popularity por (cou

Preguntas frecuentes

¿Onboarding pregunta directa o implícita?

- Directa (elegir 3 géneros): rápida, controlable, pero rompe el flujo. Spotify la usa al sign-up.
- Implícita (observar primeras N interacciones, no recomendar hasta tener señal): seamless, pero los primeros 5 clicks son perdidos.

Híbrido: pregunta directa básica (1-2 ítems) + observar primeras N.

¿Mostrar ítems "Nuevo" boost por cuánto tiempo?

Depende del catálogo. News: minutos-horas. Movies: días-semanas. Productos e-commerce: días. Decay logarítmico: $\text{boost} = 1 / (1 + \text{days})$.

¿Bandits valen la pena vs A/B test?

A/B test mide impacto de una variante vs otra. Bandits explotan automáticamente la mejor opción mientras siguen explorando. Para múltiples variantes (10+ tipos de recomendación) o cuando el costo de no explorar es alto: bandits. Para 2-3 variantes y experimentación controlada: A/B (Clase 204).

¿Cold-start de sistema cómo arranco?

(1) Importar gustos desde otra plataforma (Facebook Connect, Google sign-in). (2) Editorial picks curados. (3) Promociones para incentivar primer rating. (4) Partner con plataforma que ya tiene data.

¿Cuándo termina el cold-start?

Heurística: cuando $NDCG@10$ personal supera consistente a $NDCG@10$ de popularity baseline. Típicamente: 5-20 interactions. Depende del catálogo.

¿Hay solo CF + content para resolverlo?

No. Cross-domain transfer (gustos en Spotify → recomendaciones en Audible), demographic CF (gente como vos suele consumir X), conversational ("¿qué te interesa hoy?") son alternativas avanzadas.

Referencias

- Schein, A. et al. Methods and Metrics for Cold-Start Recommendations (SIGIR 2002).
- Aggarwal Recommender Systems: The Textbook (Springer, 2016), cap. 13 — Advanced Topics (incluye cold-start).
- Sutton & Barto, Reinforcement Learning: An Introduction (2nd ed., 2018), cap. 2 — Multi-armed bandits.
- Vowpal Wabbit contextual bandits.

- Two Decades of Recommender Systems at Amazon (Smith & Linden, IEEE, 2017) — perspectiva histórica.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 222 — Clase 222 — Librerías: LightFM, Implicit, Surprise

Parte: 6 — Sistemas de Recomendación · Fuente: docs oficiales Surprise + LightFM + Implicit + TensorFlow Recommenders. Duración estimada: 70 min.

Objetivo

Conocer las librerías de recomendación que vas a usar en la vida real sin tener que reimplementar lo de Clases 216-221. Decidir cuál usar según: tipo de feedback (explicit/implicit), tamaño del dataset, features disponibles, integración con stack.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Usar Surprise para algoritmos clásicos explicit (KNNBasic, SVD, SVD++, NMF, Co-Clustering) con API tipo sklearn.
- Usar Implicit para ALS implicit, BPR, Logistic MF — la opción más rápida en Python para datasets grandes.
- Usar LightFM cuando tenés features (Clase 219 hybrid).
- Conocer TensorFlow Recommenders y Spotlight (PyTorch) para arquitecturas deep (two-tower, sequential).
- Decidir librería según: explicit vs implicit, escala, features, deployment target.

Temas

#	Tema	Por qué importa
1	Surprise: explicit, didáctica	Empezar a aprender RS.
2	Implicit: ALS/BPR Cython	El caballo de batalla en producción.
3	LightFM: CF + features	Hybrid built-in.
4	TF Recommenders + Spotlight	Deep RS (two-tower, BERT4Rec).
5	Spark pyspark.ml.recommendation.ALS	Cuando el dataset es TB.
6	Vector DBs (FAISS, Milvus, Pinecone)	Serving de embeddings a escala.

Definiciones y características

- Surprise: librería didáctica, API tipo sklearn. Maneja explicit feedback con KNN, SVD, NMF, etc. Bueno para aprender, lento para grandes datasets.
- Implicit (Ben Frederickson): Cython + multi-thread. ALS implicit (Hu et al.), BPR (Bayesian Personalized Ranking), Logistic MF. Default para CF en producción Python.
- LightFM (Lyst): factorización con features. Loss WARP / BPR / Logistic. El sweet spot para hybrid con metadata.
- TensorFlow Recommenders (TFRS): API alto-nivel sobre TF/Keras para two-tower, ranking, retrieval. SOTA para deep RS, requiere setup TF.

- Spotlight (Maciej Kula): PyTorch library para sequential + factorización. Para experimentos research.
- Spark ML ALS: distribuido, escala a billones de interactions. Cuando el dataset no entra en single machine.
- Vector DB: para servir embeddings. FAISS (in-process), Milvus (open-source server), Pinecone/Qdrant/Weaviate (managed). Permiten `recommend(user_emb, top_k=10)` en <10 ms con M items.

Dataset / recursos

- Dataset: MovieLens 100K y 1M.
- Librerías: `scikit-surprise`, `implicit` ≥ 0.7 , `lightfm` ≥ 1.17 , opcional `tensorflow-recommenders`.

Ejercicios

1. Surprise SVD: `from surprise import SVD, Dataset; data = Dataset.load_builtin('ml-100k'); algo = SVD(); cross_validate(algo, data, measures=['RMSE','MAE'], cv=5)`. Reportar RMSE.
2. Implicit ALS: `model = implicit.als.AlternatingLeastSquares(factors=64, regularization=0.05, iterations=20)`. `model.fit(R_sparse * 40)`. `recommend(user_id, R[user_id], N=10)`.
3. Implicit BPR: mismo modelo pero `implicit.bpr.BayesianPersonalizedRanking(factors=64)`. Comparar NDCG vs ALS.
4. LightFM: `LightFM(loss='warp', no_components=32)`. Con item features (géneros). Comparar pure CF vs hybrid.
5. Benchmarking: mismo dataset, mismo split, las 4 librerías. Tabla con NDCG@10, recall@10, tiempo entrenamiento, tiempo predict.

Homework verificable

Notebook con:

1. Comparativa rigurosa sobre MovieLens 1M:
 - Surprise SVD
 - Implicit ALS
 - Implicit BPR
 - LightFM WARP (con features)
1. Mismo train/test split temporal. Mismas métricas (Clase 220).
2. Tabla: NDCG@10, recall@10, coverage, tiempo train (s), tiempo predict (ms/user), RAM peak.
3. Recomendaciones por caso de uso (decisión justificada):
 - "Quiero algo simple para aprender" → Surprise.
 - "Quiero performance en producción Python para 10M users" → Implicit.
 - "Tengo features ricos y dataset chico-medio" → LightFM.
 - "Tengo equipo ML serio y quiero SOTA" → TF Recommenders / two-tower.
1. Bonus: deploy del mejor modelo como servicio FastAPI (Clase 199) con FAISS para retrieval rápido.

Criterio de aceptación: 4 modelos entrenan limpio, NDCG@10 reportado correctamente, la tabla permite decidir cuál usar por caso.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Surprise lentísimo sobre 1M+	Surprise está pensado para didáctico. Fix:
implicit warning OpenBLAS threads	Conflict entre BLAS threading y OpenMP thr
LightFM WARP muy lento	WARP es $O(\text{samples})$ por epoch. Fix: <code>loss='b'</code>
TFRS requiere graph mode + estructura comp	Curva de aprendizaje fuerte. Fix: empezar

FAISS index demasiado grande para RAM	IndexFlatIP guarda todo en RAM. Fix: Index
Score implicit ALS no es probabilidad	Es un score, no proba. Fix: para servir co

Preguntas frecuentes

¿Cuál instalo primero?

Implicit para CF moderno (90% de los casos). LightFM si tenés features. Surprise solo para aprender el ABC.

¿Implicit y LightFM compiten o se complementan?

Compiten. Implicit es CF puro (más rápido, más simple). LightFM es CF + features (más flexible). Si no tenés features útiles: Implicit gana. Si tenés metadata rica: LightFM.

¿TF Recommenders vale la pena?

Sí cuando: (1) Dataset >100M interactions. (2) Necesitás features complejos (text embeddings, image embeddings). (3) Tu stack ya es TF. (4) Querés sequential / session-based RS (BERT4Rec, SASRec). Para casos chicos: overkill.

¿Cómo sirvo embeddings en producción?

(1) Pre-computar item_factors offline. (2) Indexar con FAISS (in-process) o Milvus/Pinecone (managed). (3) En request: computar user_emb (rápido o desde cache) → index.search(user_emb, k=10) → top-N items. Latencia: <10 ms para millones de items con HNSW.

¿Reentrenamiento — cada cuánto?

Para CF: diario o semanal típicamente. Para deep RS con features estáticos: semanal-mensual. Para sequential: continuous training. Ver Clase 203.

¿RecBole, Cornac, Spotlight?

Frameworks de research, mucho catálogo de algoritmos. Útiles para comparar 20 modelos en paper. Para producción: empezás con uno (Implicit/LightFM) y lo customizás.

Referencias

- Surprise docs.
- Implicit docs + GitHub.
- LightFM docs.
- TensorFlow Recommenders — two-tower + ranking.
- FAISS docs — vector similarity search.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrido desde el laboratorio del programa o desde Jupyter.

Parte 7 — Parte 7 — Ética, Fairness y Privacidad

6 clases en esta parte.

Clase 223 — Clase 223 — Tipos de sesgo algorítmico y orígenes

Parte: 7 — Ética, Fairness y Privacidad · Fuente: Suresh & Gutttag, *A Framework for Understanding Sources of Harm throughout the ML Life Cycle* (EAAMO 2021) + Barocas, Hardt, Narayanan, *Fairness and Machine Learning* (2023), caps. 1-2. Duración estimada: 75 min.

Objetivo

Aprender a nombrar y diagnosticar el origen del sesgo en un sistema ML antes de intentar mitigarlo. Un modelo "sesgado" no es un bug: es el resultado de decisiones tomadas en cada fase del life cycle (recolección, medición, modelado, evaluación, despliegue). Si no sabemos dónde entró el sesgo, no podemos elegir la mitigación correcta (Clases 225-227).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Distinguir los 6 tipos del framework Suresh-Gutttag: histórico, representación, medición, agregación, evaluación, despliegue.
- Identificar el origen probable de un sesgo dado evidencia empírica (gap de accuracy entre subgrupos, drift, proxy-target gap).
- Reproducir el patrón de Gender Shades (Buolamwini & Gebru 2018): accuracy alta global, accuracy baja en subgrupo minoritario.
- Reconocer Simpson's paradox y por qué un modelo único puede ser peor que un modelo por subgrupo.
- Justificar por qué fairness no es solo un problema del modelo — empieza en la definición de la tarea.

Temas

#	Tema	Por qué importa
1	Sesgo histórico	El dato es "correcto" pero refleja un mund
2	Sesgo de representación	Subgrupos sub-muestreados → modelo aprende
3	Sesgo de medición	El proxy ≠ el target real (re-arresto ≠ re
4	Sesgo de agregación	Un modelo único asume que todos los subgru
5	Sesgo de evaluación	El benchmark de test no representa la pobl
6	Sesgo de despliegue	El modelo se usa fuera del contexto en que

Definiciones y características

- Sesgo histórico: la realidad social que se midió ya era injusta. El dato es preciso, pero codifica desigualdad. Ej.: Amazon entrenó su hiring tool con 10 años de CVs (~mayoría hombres en tech) → penalizaba la palabra "women's" en CVs.
- Sesgo de representación: el dataset bajo-representa un subgrupo. No es que el modelo sea injusto: nunca vio suficientes ejemplos. Ej.: IJB-A face dataset era 79% piel clara → Gender Shades mostró error 34.7% en mujeres de piel oscura vs 0.8% en hombres de piel clara (Buolamwini & Gebru, 2018).
- Sesgo de medición (proxy bias): el y que medimos no es el y que queremos. Recidiva (queremos predecir) ≠ re-arresto (lo que medimos) — el arresto depende del policing, que ya está sesgado.
- Sesgo de agregación: un modelo único asume una única función óptima $f(x)$ → y para toda la población. Si los subgrupos tienen distintas $P(y | x)$, el modelo único es peor que k modelos por subgrupo (instancia del Simpson's paradox).
- Sesgo de evaluación: el test set no representa la población objetivo. Ej.: evaluar un detector de melanoma sobre un benchmark 90% piel clara y reportar "97% accuracy".
- Sesgo de despliegue: el modelo se aplica a una distribución distinta a la de entrenamiento (covariate

shift, label shift) — o a un contexto socio-técnico donde su salida significa otra cosa. Un score que era "una sugerencia para el juez" se vuelve "la decisión".

- Framework Suresh-Guttag (2021): mapea los 6 sesgos a las fases del ML life cycle (data collection → preparation → modeling → evaluation → deployment). Cada fase introduce su propio harm; mitigar en la fase equivocada no funciona.
- Harm allocacional vs representacional (Barocas et al., cap. 1): el sistema deniega recursos (préstamo, libertad bajo fianza) vs refuerza estereotipos (autocomplete de Google asocia "CEO" con foto de hombre).

Dataset / recursos

- Dataset: sintético (préstamos con sesgo histórico inyectado). Auto-generado en el notebook con numpy seed 42.
- Librerías: numpy, pandas, scikit-learn. Sin descargas externas.

Ejercicios

1. Sesgo histórico: generar un dataset de préstamos donde $P(\text{aprobado} \mid \text{grupo}=A) = 0.70$ y $P(\text{aprobado} \mid \text{grupo}=B) = 0.30$ por razones históricas (no por capacidad de pago). Entrenar LogisticRegression sin la feature grupo y mostrar que el modelo igual reproduce el gap vía proxies (código postal, ingreso, etc.).
2. Selection rate disparity: calcular $P(\hat{y}=1 \mid \text{grupo}=A)$ vs $P(\hat{y}=1 \mid \text{grupo}=B)$ — la métrica más simple de demographic parity (Clase 224).
3. Sesgo de representación (Gender Shades): re-muestrear el dataset al 10% del grupo B. Reportar accuracy global vs accuracy por subgrupo. Mostrar el patrón "97% global, 60% en B".
4. Sesgo de medición: definir $y_{\text{proxy}} = y_{\text{true}} \text{ XOR ruido_correlacionado_con_grupo}$. Entrenar sobre y_{proxy} . Mostrar que el modelo aprende el patrón del ruido, no del target real.
5. Sesgo de agregación (Simpson): comparar AUC de un modelo único vs un modelo por subgrupo. Mostrar que el modelo único es subóptimo en ambos subgrupos.

Homework verificable

Notebook con:

1. Reproducir el patrón Gender Shades sobre un dataset tabular (sintético o UCI Adult con sex y race).
2. Calcular gap de accuracy entre subgrupos para 3 niveles de sub-muestreo del grupo minoritario (50%, 20%, 5%).
3. Aplicar el framework Suresh-Guttag a un caso real (COMPAS, Amazon Hiring, o un modelo del trabajo) — escribir 1 párrafo por cada uno de los 6 tipos: ¿está presente? evidencia.
4. Implementar Simpson's paradox: dataset donde la correlación global es opuesta a la correlación intra-grupo.
5. Discutir: ¿qué mitigación corresponde a cada tipo? (representación → re-sampling; medición → re-definir target; agregación → modelo por subgrupo).

Criterio de aceptación: el alumno identifica los 6 tipos en al menos un caso real con evidencia cuantitativa, y propone una mitigación específica por tipo (no genérica "más datos").

Errores comunes

Síntoma	Causa y cómo arreglar
"Saqué la variable sensible y el modelo si	Fairness through unawareness no funciona —
Accuracy global alta pero el cliente repor	Sesgo de representación o evaluación. Fix:
El modelo es "objetivo, solo aprendió del	El dato no es neutro — refleja decisiones

"Agregamos más datos del grupo minoritario"	Probablemente sesgo de medición — el y par
Modelo único performa peor que k modelos p	Sesgo de agregación. Fix: modelo por subgr
Modelo cae en producción pero pasó test	Sesgo de evaluación/despliegue. Fix: test

Preguntas frecuentes

¿Por qué no simplemente sacar la variable sensible (sex, race)?

Eso es fairness through unawareness y casi nunca funciona. Las features que sí usás (ZIP, nombre, historial crediticio, escuela) son proxies correlacionados. Quitar la variable sensible te impide medir el sesgo pero no lo elimina. Lo que sí se hace: usar la variable sensible para auditar disparidades, no como input del modelo.

¿Sesgo histórico vs sesgo de representación — cuál es la diferencia?

Histórico = el dato es preciso pero el mundo medido era injusto (mujeres ganan menos → modelo de salarios predice salarios más bajos para mujeres). Representación = el dato no representa bien a un subgrupo (pocas caras oscuras en ImageNet → mala accuracy ahí). El primero requiere reescribir el target o el objetivo; el segundo, recolectar/re-muestrear.

¿Simpson's paradox es realmente común en ML?

Sí, especialmente en datasets con subgrupos heterogéneos (Berkeley admissions 1973 es el clásico). Si tu accuracy global mejora pero la accuracy por subgrupo empeora, es Simpson. Por eso siempre se reporta métrica stratificada.

¿Esto se resuelve con un algoritmo de fairness?

No solo. Los algoritmos (re-weighting, adversarial debiasing, Clase 226) atacan principalmente representación y agregación. El sesgo histórico y de medición requieren decisiones humanas: ¿qué estamos prediciendo? ¿es el target correcto? Ningún optimizador resuelve "el target estaba mal definido".

¿Cómo encaja todo esto con Clases 224 (métricas) y 225-227 (mitigación)?

223 = diagnóstico (qué tipo de sesgo y dónde nace). 224 = medición cuantitativa (demographic parity, equalized odds, calibration). 225-227 = mitigación (pre-procesamiento, in-procesamiento, post-procesamiento). El orden importa: sin diagnóstico, la mitigación es a ciegas.

Referencias

- Suresh, H., Gutttag, J. A Framework for Understanding Sources of Harm throughout the ML Life Cycle (EAAMO 2021) — el framework canónico de los 6 tipos.
- Buolamwini, J., Gebru, T. Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification (FAT* 2018) — el paper que disparó la auditoría algorítmica.
- Barocas, S., Hardt, M., Narayanan, A. Fairness and Machine Learning: Limitations and Opportunities (MIT Press, 2023) — caps. 1-2 (libre online).
- Mehrabi, N. et al. A Survey on Bias and Fairness in Machine Learning (ACM Computing Surveys, 2021).
- Angwin, J. et al. Machine Bias (ProPublica, 2016) — la investigación sobre COMPAS.
- Dastin, J. Amazon scraps secret AI recruiting tool that showed bias against women (Reuters, 2018).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 224 — Clase 224 — Métricas de fairness: demographic parity, equalized odds, calibration

Parte: 7 — Ética, Fairness y Privacidad · Fuente: Barocas, Hardt, Narayanan — *Fairness and Machine Learning* cap. 3 + Hardt, Price, Srebro (NeurIPS 2016) *Equality of Opportunity in Supervised Learning*.
Duración estimada: 75 min.

Objetivo

Pasar de "el modelo es injusto" a medirlo con un número. Implementar las tres familias de métricas grupales que dominan la literatura — demographic parity, equalized odds, calibration — sobre un dataset binario con atributo protegido, y demostrar numéricamente el teorema de imposibilidad de Kleinberg-Mullainathan-Raghavan / Chouldechova (2017): salvo casos triviales, no se pueden satisfacer las tres a la vez.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Calcular demographic parity gap = $|P(\hat{Y}=1|A=0) - P(\hat{Y}=1|A=1)|$ sobre predicciones de cualquier clasificador binario.
- Calcular equal opportunity (TPR por grupo) y equalized odds (TPR y FPR por grupo) — Hardt, Price, Srebro 2016.
- Verificar calibración por grupo con reliability curves: $P(Y=1|\hat{S}=s, A=a)$ debe ser igual entre grupos para un mismo score s .
- Demostrar el teorema de imposibilidad: ajustar threshold por grupo para forzar demographic parity rompe calibración.
- Aplicar mitigación post-processing con thresholds por grupo (Hardt 2016) y reportar el trade-off accuracy vs fairness gap.

Temas

#	Tema	Por qué importa
1	Atributo protegido A y notación $Y / \hat{Y} / \hat{S}$	Sin notación común no se discute fairness;
2	Demographic parity (statistical parity)	La métrica más antigua (regla del 80%, US
3	Equal opportunity y equalized odds (Hardt	Condicionan en Y — corrigen el defecto de
4	Calibration por grupo (Chouldechova 2017)	El score debe significar lo mismo en cada
5	Teorema de imposibilidad (KMR / Chouldechova	Si las base-rates difieren, DP + equalized
6	Post-processing: threshold por grupo	Mitigación más simple; revela explícitamen

Definiciones y características

- Atributo protegido A: variable demográfica sensible (sexo, raza, edad). En la práctica nunca está sola — hay proxies (zip code, nombre, historial).
- Demographic parity (DP): $P(\hat{Y}=1 | A=0) = P(\hat{Y}=1 | A=1)$. Métrica: $DP_gap = |selection_rate(A=0) - selection_rate(A=1)|$. Regla del 80%: $ratio \geq 0.8$ para que la US EEOC no lo considere discriminación.
- Equal opportunity: $P(\hat{Y}=1 | Y=1, A=0) = P(\hat{Y}=1 | Y=1, A=1)$. O sea, TPR igual. Solo a los positivos reales se les exige misma tasa de aceptación.
- Equalized odds (Hardt-Price-Srebro 2016): equal opportunity + FPR igual. $EO_gap = \max(|TPR_diff|, |FPR_diff|)$.
- Calibration (predictive parity, Chouldechova 2017): $P(Y=1 | \hat{S}=s, A=a)$ es igual entre grupos para todo score s . Un score 0.7 significa "70% chance" tanto para $A=0$ como para $A=1$.
- Impossibility theorem (Kleinberg-Mullainathan-Raghavan 2017 + Chouldechova 2017): si $P(Y=1|A=0) \neq$

$P(Y=1|A=1)$ (base rates diferentes) y el clasificador no es perfecto, no existe clasificador que sea simultáneamente calibrado por grupo y con equalized odds. Es matemático, no técnico — no se resuelve con más datos.

- Accuracy-fairness trade-off: forzar cualquier paridad sobre un clasificador óptimo de Bayes baja accuracy. La pregunta política es cuánta accuracy estamos dispuestos a sacrificar.
- Tooling: fairlearn (Microsoft) — MetricFrame, ThresholdOptimizer. aif360 (IBM) — 70+ métricas y mitigadores. Ambas open source.

Dataset / recursos

- Dataset notebook: sintético binario con un atributo protegido $A\{0,1\}$ y base rates diferentes (60% vs 40%) — necesario para que el teorema de imposibilidad se active.
- Dataset real recomendado para tarea: Adult / Census Income (UCI) con sex como atributo protegido, o COMPAS (ProPublica) con race.
- Librerías: numpy, pandas, scikit-learn. Opcional: fairlearn.

Ejercicios

1. Selection rate por grupo: entrenar LogisticRegression baseline. Calcular $P(\hat{Y}=1|A=0)$ y $P(\hat{Y}=1|A=1)$ y el DP_gap. ¿Cumple regla del 80%?
2. TPR y FPR por grupo: armar confusion_matrix separada por grupo. Calcular equal_opportunity_gap = $|TPR_0 - TPR_1|$ y equalized_odds_gap = $\max(|TPR_diff|, |FPR_diff|)$.
3. Calibration curves por grupo: binning de scores en 10 bins. Para cada bin y cada grupo, graficar $\text{mean}(y_true)$ vs $\text{mean}(y_score)$. ¿Las curvas coinciden?
4. Romper calibración: ajustar threshold por grupo (t_0, t_1) tal que se cumpla DP exacta. Recalcular calibración — debe degradarse. (Demostración numérica del teorema.)
5. Post-processing Hardt: buscar (t_0, t_1) que minimicen equalized_odds_gap y reportar el costo en accuracy global. Tabla: baseline vs DP-fixed vs EO-fixed.

Homework verificable

Notebook con:

1. Cargar Adult Census (UCI). Atributo protegido: sex. Target: income > 50K.
2. Entrenar baseline LogisticRegression + reportar accuracy, AUC.
3. Calcular las tres métricas: DP_gap, equal_opportunity_gap, equalized_odds_gap y calibration_gap ($\max |calibration(A=0) - calibration(A=1)|$ sobre bins).
4. Implementar post-processing con threshold por grupo que minimice EO_gap sujeto a $\text{accuracy_drop} \leq 3\text{pp}$.
5. Tabla final: 3 modelos (baseline, DP-mitigated, EO-mitigated) × 5 métricas (accuracy, AUC, DP_gap, EO_gap, calibration_gap). Discutir cuál elegirías y por qué — no hay respuesta universal.

Criterio de aceptación: las 4 métricas implementadas a mano (no llamando a fairlearn), el EO_gap mitigado < 0.05, y un párrafo justificando la elección de métrica en función del dominio de aplicación (crédito vs admisión universitaria vs medicina).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
DP_gap = 0 pero el modelo es claramente in	DP ignora el ground truth — si las base ra
Calibration "perfecta" pero TPR diferente	Es el resultado natural cuando las base ra
Quitar el atributo protegido del input "so	Fairness through unawareness — no funciona
Threshold único 0.5 para todos los grupos	Asume que los scores significan lo mismo e

Usar accuracy global para comparar fairness	Accuracy global puede subir y empeorar al
MetricFrame de fairlearn devuelve nan	Algún grupo tiene 0 positivos predichos o

Preguntas frecuentes

¿Cuál de las tres métricas uso?

Depende del dominio. Crédito o contratación: equal opportunity (no negarle empleo a quien sí calificaría). Justicia penal / riesgo de reincidencia: calibration + FPR equal (el debate ProPublica vs COMPAS giró exactamente sobre cuál priorizar). Publicidad: DP suele alcanzar. Pero la elección es política, no técnica.

¿Demographic parity no es siempre lo que queremos?

No. Si la base rate real difiere entre grupos (ej. distribución de ingresos), DP puede forzar al modelo a aceptar candidatos peores de un grupo y rechazar mejores de otro. Equal opportunity suele ser más defendible.

¿El teorema de imposibilidad significa que fairness es imposible?

No — significa que no se pueden satisfacer las tres simultáneamente cuando las base rates difieren. Hay que elegir cuál priorizar, y documentarlo. El paper de Chouldechova (2017) es lectura obligada.

¿fairlearn o aif360?

fairlearn para empezar (API más limpia, integrada con sklearn). aif360 cuando se necesite catálogo amplio de mitigadores (pre/in/post-processing) y métricas exóticas. Ninguna sustituye entender las definiciones.

¿Y la fairness individual?

Otra familia (Dwork et al. 2012): "individuos similares deben recibir predicciones similares". Más difícil de operacionalizar (requiere métrica de similaridad justificable) y queda fuera del alcance de esta clase. Ver Clase 225-227 para privacy y Clase 228 para causal fairness.

Referencias

- Hardt, M., Price, E., Srebro, N. Equality of Opportunity in Supervised Learning, NeurIPS 2016. <https://arxiv.org/abs/1610.02413>
- Chouldechova, A. Fair prediction with disparate impact: A study of bias in recidivism prediction instruments, Big Data 2017. <https://arxiv.org/abs/1610.07524>
- Kleinberg, J., Mullainathan, S., Raghavan, M. Inherent Trade-Offs in the Fair Determination of Risk Scores, ITCS 2017. <https://arxiv.org/abs/1609.05807>
- Barocas, S., Hardt, M., Narayanan, A. Fairness and Machine Learning (fairmlbook.org), cap. 3 — Classification.
- fairlearn documentation — Microsoft, MIT license.
- AIF360 documentation — IBM, Apache 2.0.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 225 — Clase 225 — Privacidad diferencial: intro

Parte: 7 — Ética, Fairness y Privacidad · Fuente: Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014) caps. 2-3 + Dwork, McSherry, Nissim, Smith (TCC, 2006) *Calibrating Noise to Sensitivity*. Duración estimada: 75 min.

Objetivo

Entender privacidad diferencial (DP) como la única definición formal de privacidad con garantías matemáticas — no "anonimización" heurística que se rompe con un join. Implementar el mecanismo de Laplace desde cero, observar el trade-off privacy-utility vía el presupuesto ϵ (epsilon), y mirar conceptualmente DP-SGD (Abadi et al. 2016): cómo se entrena un modelo sin que un atacante pueda inferir si tu registro estuvo en el training set.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Enunciar la definición de (ϵ, δ) -DP: $P[M(D) \in S] \leq e^{\epsilon} \cdot P[M(D') \in S] + \delta$ para datasets vecinos D, D' .
- Calcular la sensibilidad Δf de funciones típicas (conteos, sumas acotadas, medias) y elegir ruido Laplace o Gaussiano calibrado.
- Implementar `laplace_mechanism(value, sensitivity, epsilon)` y verificar que ϵ chico $\rightarrow$ más ruido $\rightarrow$ menos utilidad.
- Aplicar composición básica: k consultas con ϵ cada una gastan $k \cdot \epsilon$ del presupuesto total.
- Reconocer la idea de DP-SGD: per-sample gradient clipping + ruido gaussiano $\rightarrow$ entrenamiento DP (Opacus, TF-Privacy).

Temas

#	Tema	Por qué importa
1	Anonimización falla (Netflix Prize, AOL se	k-anonymity / pseudonimización son rotas p
2	Definición (ϵ, δ) -DP y datasets vecinos	El ϵ es la garantía; sin él, "privacidad"
3	Sensibilidad Δf	Calibra cuánto ruido hace falta. Conteo: Δ
4	Mecanismos Laplace y Gaussiano	Laplace para ϵ -DP puro; Gaussiano para $(\epsilon,$
5	Composición y post-processing	Cada query gasta presupuesto; cualquier f
6	DP-SGD (Abadi 2016)	Clip per-sample + ruido gaussiano. Es el e

Definiciones y características

- (ϵ, δ) -Differential Privacy (Dwork 2006): mecanismo aleatorizado M es (ϵ, δ) -DP si para todo par de datasets vecinos D, D' (que difieren en 1 registro) y todo conjunto de salidas S : $P[M(D) \in S] \leq e^{\epsilon} \cdot P[M(D') \in S] + \delta$. Si $\delta = 0$ se llama ϵ -DP puro.
- Privacy budget ϵ : chico = más privacidad, menos utilidad. Valores típicos en la práctica: $\epsilon=0.1$ (fuerte), $\epsilon=1$ (estándar de la US Census 2020), $\epsilon=10$ (débil — más marketing que garantía).
- δ : probabilidad de fallar la garantía. Regla: $\delta \ll 1/n$ (con n = tamaño del dataset).
- Sensibilidad Δf (L1): $\Delta f = \max_{\{D, D' \text{ vecinos}\}} |f(D) - f(D')|$. Conteo de registros: $\Delta f=1$. Suma de valores en $[0, B]$: $\Delta f=B$. Media sobre n fijo y valores en $[0, B]$: $\Delta f = B/n$.
- Mecanismo de Laplace: $M(D) = f(D) + \text{Lap}(0, \Delta f/\epsilon)$. Cumple ϵ -DP puro.
- Mecanismo Gaussiano: $M(D) = f(D) + N(0, \sigma^2)$ con $\sigma \geq \sqrt{2 \ln(1.25/\delta)} \cdot \Delta f / \epsilon$. Cumple (ϵ, δ) -DP.
- Composición básica: k mecanismos ϵ_i -DP componen a $(\sum \epsilon_i)$ -DP. Composición avanzada da $\sqrt{2k \ln(1/\delta)} \cdot \epsilon$ — mejor escala.
- Post-processing: si M es (ϵ, δ) -DP, entonces $g(M(D))$ también — no podés "des-privatizar" mirando la salida.
- DP-SGD (Abadi et al. 2016): en cada paso (1) calcular gradiente per-sample, (2) clipearlo a norma C , (3) promediar el batch, (4) sumar ruido gaussiano $N(0, \sigma^2 C^2)$. Tracking del ϵ vía moments accountant / RDP.

Dataset / recursos

- Dataset: sintético — un dataframe de salarios $n=10_000$ con valores en $[0, 200_000]$. Suficiente para Laplace, mean privado, histograma y DP-SGD demo. Sin descarga externa.
- Librerías: numpy, pandas, scikit-learn. En producción real: opacus (PyTorch), tensorflow-privacy, diffprivlib (IBM).

Ejercicios

1. Laplace básico: implementar `laplace_mechanism(value, sensitivity, epsilon)` y verificar empíricamente sobre 10_000 corridas que la varianza es $2 \cdot (\Delta f / \epsilon)^2$.
2. Conteo privado: contar empleados con salario $> 100k$ con $\epsilon \in \{0.1, 1.0, 10.0\}$. Reportar error medio absoluto y discutir el trade-off.
3. Mean privado con clipping: clip salarios a $[0, B]$, sumar con Laplace ($\Delta f=B/n$, $\epsilon=1$), dividir por n . Mostrar bias vs varianza al variar B .
4. Histograma privado: 10 bins de salario, ruido Laplace independiente por bin (sensibilidad = 1 por bin). Comparar con histograma no privado.
5. Composición: hacer 10 conteos con $\epsilon=0.1$ cada uno $\rightarrow$ presupuesto total $\epsilon=1.0$. Mostrar acumulación empírica del ruido.

Homework verificable

Notebook con:

1. Cargar Adult / Census Income (UCI, ~32K filas).
2. Publicar un dashboard DP con 5 estadísticas (count, mean age, mean hours-per-week, count por género, count por education) bajo presupuesto total $\epsilon=1.0$. Repartir el presupuesto entre queries y justificar.
3. Entrenar un LogisticRegression clásico para predecir $\text{income} > 50k$, reportar accuracy.
4. Re-entrenar con DP-SGD manual: clip per-sample gradient norm a $C=1.0$, sumar $N(0, \sigma^2)$ con $\sigma=1.0$. Reportar accuracy y comparar.
5. Discutir: ¿cuánta utilidad perdés? ¿el modelo DP es publicable sin riesgo de membership inference?

Criterio de aceptación: el dashboard cumple $\epsilon=1.0$ total (verificable sumando los ϵ_i), el modelo DP-SGD entrena sin error y la pérdida de accuracy vs no-DP es < 10 pp.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Privatizo la salida pero el atacante reco"	Olvidaste clippear la entrada — un outlier
Calculo Δf de una media como B (no B/n)	Confundís sensibilidad de suma vs media. F
Hago 100 queries con $\epsilon=1$ y digo "es $\epsilon=1$ -DP"	Sin tracking, gastaste $\epsilon=100$ por composici
$\epsilon=10$ o $\epsilon=20$ "porque así da mejor utility"	$\epsilon \geq 10$ da garantía prácticamente nula (e^{10})
DP-SGD sin clippear per-sample	Sin clipping, la sensibilidad del gradient
Reutilizar el dataset privado para "valida"	El proceso de validación también gasta bud

Preguntas frecuentes

¿Qué ϵ es "seguro"?

No hay un número universal. La US Census 2020 usó $\epsilon \approx 19.6$ (TopDown algorithm, criticado por flojo). Apple iOS reporta ϵ por feature (típicamente 2-8 por día). Recomendación práctica: empezar en $\epsilon=1$, justificar cualquier valor mayor. $\epsilon \geq 10$ es difícil de defender ante un comité de ética.

¿DP me protege de TODO ataque?

Te protege contra membership inference y reconstruction bajo el modelo de atacante con conocimiento auxiliar arbitrario. NO te protege contra: ataques al modelo no-DP entrenado paralelamente, side-channels (timing), o si el atacante tiene el dato original (no es encriptación).

¿Local DP vs Central DP?

Central DP: confiás en el curador (el servidor agrega ruido). Más utilidad. Local DP: cada usuario agrega ruido antes de mandar (Apple, RAPPOR de Google). Menos utilidad, pero no confiás en nadie. La elección depende del modelo de amenaza.

¿Vale la pena en deep learning?

Sí, con caveats. DP-SGD penaliza accuracy (5-15 pp típicos), y necesita batches grandes para que el ruido se promedie. Opacus / TF-Privacy automatizan todo. Es obligatorio si vas a publicar el modelo o usar datos médicos/financieros bajo regulación.

¿Y federated learning?

Federated learning (Clase 226) por sí solo NO es DP — el server ve gradientes que filtran información. Se combina con secure aggregation + DP para garantías reales (lo que hace Google Gboard).

Referencias

- Dwork, McSherry, Nissim, Smith. Calibrating Noise to Sensitivity in Private Data Analysis (TCC 2006) — el paper original que define DP.
- Dwork, C., Roth, A. The Algorithmic Foundations of Differential Privacy (Foundations and Trends, 2014) — el libro de referencia.
- Abadi et al. Deep Learning with Differential Privacy (CCS 2016) — DP-SGD + moments accountant.
- Opacus — DP-SGD en PyTorch (Meta).
- TensorFlow Privacy — DP-SGD en TF/Keras (Google).
- IBM diffprivlib — mecanismos básicos sklearn-compatible.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 226 — Clase 226 — Federated learning: intro

Parte: 7 — Ética, Fairness y Privacidad · Fuente: McMahan et al. Communication-Efficient Learning of Deep Networks from Decentralized Data (AISTATS 2017) + Kairouz et al. Advances and Open Problems in Federated Learning (FnTML 2021). Duración estimada: 75 min.

Objetivo

Entrenar un modelo central sin centralizar los datos. Cada cliente (móvil, hospital, banco) entrena local sobre su data, sube solo pesos o gradientes al servidor, que agrega vía FedAvg. Implementar FedAvg manual sobre regresión logística, ver cómo degrada con datos non-IID, y demostrar el ataque básico de gradient leakage (los gradientes filtran data).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar el setup FL: server + K clientes, rondas, partial participation, comunicación de pesos en vez de data.
- Implementar el algoritmo FedAvg (McMahan 2017): $w_{t+1} = \sum_k (n_k / n) * w_k^t$.
- Distinguir cross-device (millones de móviles, intermitentes) vs cross-silo (decenas de hospitales, estables).
- Reconocer el efecto de datos non-IID: FedAvg degrada cuando cada cliente ve una distribución distinta.
- Identificar los riesgos: model poisoning, gradient leakage (Zhu 2019), y las defensas (secure aggregation, DP-FedAvg, Krum).

Temas

#	Tema	Por qué importa
1	Setup FL: server + clientes + rondas	La data nunca sale del cliente; solo viaja
2	FedAvg: muestreo, local epochs, agregación	Es el baseline; todo lo demás se compara c
3	Cross-device vs cross-silo	Define el régimen: K=10 ⁶ intermitentes vs
4	Heterogeneidad: non-IID + system + partial	El paper asume IID; la realidad no lo es.
5	Ataques: model poisoning, gradient leakage	Compartir gradientes ≠ proteger data (Zhu
6	Defensas: secure aggregation, DP, Krum/Mex	Lo que se usa en producción real (Google G

Definiciones y características

- Federated learning (FL): paradigma de entrenamiento donde K clientes con data local entrenan un modelo conjunto coordinados por un servidor, sin compartir data cruda.
- FedAvg: el algoritmo base (McMahan 2017). Cada ronda: server envía pesos w_t ; un subset de clientes corre E épocas de SGD local; server promedia: $w_{t+1} = \sum (n_k / n) * w_k$.
- Cross-device FL: K = millones de teléfonos, clientes intermitentes (sin conexión, batería baja), participación parcial obligatoria. Ejemplo: Gboard de Google.
- Cross-silo FL: K = decenas de organizaciones (hospitales, bancos), clientes estables y siempre online. Ejemplo: consorcios médicos contra cáncer.
- Non-IID data: la distribución $P_k(x, y)$ cambia por cliente. Caso típico: cada hospital ve una mezcla distinta de patologías. FedAvg degrada y oscila.
- Gradient leakage: dado un gradiente compartido (especialmente con batch chico), un atacante puede reconstruir el input original vía optimización inversa (Zhu, Liu, Han, Deep Leakage from Gradients, NeurIPS 2019).
- Secure aggregation (Bonawitz 2017): protocolo criptográfico donde el server solo ve la suma de pesos de N clientes, nunca los individuales — anula el gradient leakage contra el server.
- DP-FedAvg: agregar ruido gaussiano calibrado a los pesos antes de subirlos → garantías formales de differential privacy (Clase 225) sobre la presencia de un cliente.

Dataset / recursos

- Dataset: sintético tabular (clasificación binaria, 2000 muestras × 10 features), split en K=10 clientes. Self-contained, sin descargas.
- Librerías: numpy, scikit-learn. Implementamos FedAvg manualmente — sin flower, PySyft ni TensorFlow Federated, para que el algoritmo quede claro.

Ejercicios

1. Particionado IID: generar 2000 muestras, split aleatorio uniforme en K=10 clientes. Verificar que cada cliente tiene ~200 muestras con clases balanceadas.
2. Local training: implementar `local_train(X, y, w, epochs=5, lr=0.05)` — regresión logística con SGD

manual. Devuelve nuevos pesos w_k .

3. FedAvg loop: 20 rondas. En cada ronda, samplear 5 de 10 clientes; entrenar local; agregar vía `fedavg(weights, sizes)`. Trackear loss global.
4. Centralizado vs federado: entrenar la misma logística sobre todo el data junto. Verificar que FL converge a una accuracy comparable (gap < 0.03 en setting IID).
5. Non-IID: re-partir asignando a cada cliente solo 1-2 clases (cliente 0 ve mayoritariamente clase 0, etc.). Correr FedAvg. Mostrar que la accuracy global degrada y oscila más.

Homework verificable

Notebook con:

1. Implementar FedAvg + variante FedProx (agrega regularización $\mu/2 \cdot \|w_k - w_t\|^2$ al loss local — Li et al. 2020) sobre el mismo dataset non-IID.
2. Comparar curvas de loss FedAvg vs FedProx vs central (3 curvas).
3. Implementar DP-FedAvg: ruido gaussiano $\sigma=0.01$ sobre los pesos agregados. Medir pérdida de accuracy.
4. Reproducir gradient leakage simple sobre 1 muestra: dado el gradiente de regresión lineal con `batch=1`, recuperar x vía minimización de $\|w L(w; \hat{x}, y) - g\|^2$.
5. Discutir (≤ 200 palabras): cuándo FL paga el costo de comunicación vs entrenamiento central + acuerdo de data sharing.

Criterio de aceptación: FedProx supera a FedAvg en setting non-IID (loss final menor); DP-FedAvg con $\sigma=0.01$ pierde <0.05 de accuracy; reconstrucción de x con MSE < 0.01 respecto al original.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
FedAvg diverge en non-IID	Demasiadas épocas locales → client drift.
Loss global oscila ronda a ronda	Sampleo de pocos clientes por ronda. Fix:
Cliente con $n_k=0$ rompe la agregación	División por cero en pesos. Fix: filtrar c
Promedio simple en vez de weighted	Clientes con poca data pesan igual que los
"FL = privado por default"	Falso. Los pesos filtran data (Zhu 2019).
Pesos cargados como listas python en el se	Costo de comunicación se dispara. Fix: env

Preguntas frecuentes

¿FL es differential privacy?

No, son ortogonales. FL no centraliza data, pero los pesos compartidos pueden filtrar muestras individuales. DP da garantías formales (Clase 225). Producción real usa FL + secure aggregation + DP combinados.

¿Cuándo elegir FL en vez de entrenar centralizado?

Cuando la data no puede moverse: regulación (HIPAA, GDPR), data en edge (teclados, wearables), competidores que quieren modelo conjunto sin compartir clientes. Si podés centralizar, centralizá — es 10-100× más barato en comunicación y converge mejor.

¿Cuánto cuesta la comunicación?

Es el cuello de botella real. Cada ronda envía $O(|w|)$ bytes por cliente — para una red de 10M params en float32 son 40 MB por cliente, por ronda. Por eso existen técnicas de compresión, quantization y top-k sparsification.

¿FedAvg está obsoleto?

Sigue siendo el baseline. Variantes modernas: FedProx (regularización local, 2020), SCAFFOLD (control variates para non-IID, 2020), FedOpt (Adam/Yogi del lado server, 2021). Todas se comparan contra FedAvg.

¿Qué framework usar en producción?

Flower es agnóstico al ML framework y el más adoptado en investigación reciente. TensorFlow Federated si ya estás en TF. PySyft si querés combinar con SMPC/HE. Para móvil: TFF Lite y la stack interna de Google (no abierta).

Referencias

- McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B. Communication-Efficient Learning of Deep Networks from Decentralized Data (AISTATS 2017) — el paper original de FedAvg: <<https://arxiv.org/abs/1602.05629>>.
- Kairouz, P. et al. Advances and Open Problems in Federated Learning (FnTML 2021) — survey canónico, 50+ autores: <<https://arxiv.org/abs/1912.04977>>.
- Zhu, L., Liu, Z., Han, S. Deep Leakage from Gradients (NeurIPS 2019) — el ataque de reconstrucción desde gradientes: <<https://arxiv.org/abs/1906.08935>>.
- Bonawitz, K. et al. Practical Secure Aggregation for Privacy-Preserving Machine Learning (CCS 2017) — el protocolo de Google.
- Flower framework — librería FL agnóstica al framework ML.
- Li, T., Sahu, A., Talwalkar, A., Smith, V. Federated Learning: Challenges, Methods, and Future Directions (IEEE Signal Processing, 2020).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Clase 227 — Clase 227 — GDPR y AI Act (EU)

Parte: 7 — Ética, Fairness y Privacidad · Fuente: Reglamento UE 2016/679 (GDPR) + Reglamento UE 2024/1689 (AI Act). Duración estimada: 75 min.

Objetivo

Entender qué exige la regulación europea a un sistema de ML que toca datos personales o decisiones automatizadas. GDPR (en vigor desde 25-may-2018) regula el dato: bases legales, derechos del titular, DPIA. AI Act (Reglamento UE 2024/1689, escalonado 2024-2027) regula el sistema de IA por nivel de riesgo: prohibido / alto / limitado / mínimo. Aterrizamos ambas normas en un mini-toolkit programático que un equipo de datos puede ejecutar antes de poner un modelo en producción.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Identificar la base legal del Art. 6 GDPR aplicable a un tratamiento (consentimiento, contrato, interés legítimo, etc.) y distinguir las categorías especiales del Art. 9 (salud, biometría, raza).
- Implementar los derechos del titular más comunes: acceso, rectificación, supresión (Art. 17 — derecho al olvido) y portabilidad.
- Reconocer cuándo el Art. 22 GDPR (decisiones automatizadas con efectos significativos) exige

supervisión humana y combinarlo con el Art. 14 del AI Act.

- Clasificar un caso de uso de IA en el nivel de riesgo del AI Act (prohibido, alto — Anexo III, limitado, mínimo) y listar las obligaciones que aplican.
- Generar una model card mínima y un checklist DPIA programático como artefactos de compliance.

Temas

#	Tema	Por qué importa
1	Bases legales (Art. 6) y categorías especi	Sin base legal válida, el tratamiento es i
2	Derechos del titular (acceso, supresión, p	El "derecho al olvido" obliga a poder borr
3	DPIA (Art. 35) — Data Protection Impact As	Obligatoria para alto riesgo (perfilado ma
4	AI Act: pirámide de riesgo	Prohibido → alto → limitado → mínimo. Defi
5	Sistemas de alto riesgo (Anexo III)	CV-screening, crédito, biometría, educació
6	GPAI y modelos fundacionales	Transparencia + resumen de datos de entren

Definiciones y características

- Dato personal (GDPR Art. 4.1): toda información sobre una persona física identificada o identificable — incluye IP, cookie ID, hashes débiles.
- Categorías especiales (Art. 9): origen racial, opiniones políticas, religión, datos genéticos, biométricos para identificar, salud, orientación sexual. Requieren base reforzada (consentimiento explícito, interés público en salud, etc.).
- Base legal Art. 6: una de seis: (a) consentimiento, (b) contrato, (c) obligación legal, (d) interés vital, (e) interés público, (f) interés legítimo.
- DPIA (Art. 35): análisis previo de riesgos para los derechos del titular; obligatorio en perfilado sistemático a gran escala, categorías especiales, vigilancia pública.
- Decisión automatizada significativa (Art. 22): el titular tiene derecho a no ser sujeto a una decisión basada solo en tratamiento automatizado con efectos jurídicos o significativos — se levanta con consentimiento explícito, ejecución contractual o ley, siempre con derecho a intervención humana.
- Sistema de IA de alto riesgo (AI Act Anexo III): biometría, infraestructura crítica, educación, empleo, servicios esenciales (crédito, seguros), aplicación de la ley, migración, justicia. Obligaciones: gestión de riesgos, gobernanza de datos, documentación técnica, registros (logs), transparencia al usuario, supervisión humana (Art. 14), robustez, conformidad CE.
- GPAI (General Purpose AI): modelos fundacionales reutilizables. Transparencia + resumen público de datos de entrenamiento; si superan 10^{25} FLOPs entran en "systemic risk" con red-teaming y reporte de incidentes.
- Multas: GDPR hasta 20 M€ o 4% revenue global; AI Act hasta 35 M€ o 7% (prácticas prohibidas).

Dataset / recursos

- Dataset sintético de decisiones de crédito (1000 filas) con campos personales (email, dni, age, salary, score, is_minority) generado en el notebook con `np.random.default_rng(42)` — sin datos reales.
- Librerías: numpy, pandas, scikit-learn, re (regex para PII).

Ejercicios

1. Clasificador de riesgo AI Act: `is_high_risk_use_case("CV screening")` → "alto"; "juego móvil de match-3" → "mínimo". Cubrir los 4 niveles con lookup table basada en Anexo III.
2. DPIA checklist: dado `{"sensitive_categories": True, "automated_decisions": True, "scale": "large"}`, listar las obligaciones GDPR aplicables (DPIA, DPO, consentimiento reforzado, etc.).
3. Right to be forgotten: implementar `right_to_be_forgotten(df, user_id)` que elimine al usuario y devuelva

un registro de auditoría con timestamp + columnas afectadas.

4. Data minimization audit: detectar columnas que parecen email (`r"[\w\.-]+@[\w\.-]+"`) o DNI (`r"\d{8}[A-Z]"`); sugerir hash o remoción.
5. Compliance report: pipeline de scoring de crédito que ejecuta los 7 chequeos del notebook (clasificación de riesgo, DPIA, model card, supervisión humana, minimización, auditoría de PII, registro de borrado) e imprime un reporte único.

Homework verificable

Notebook con:

1. Tomar un caso de uso real propio o público (ej.: recomendador de empleo, scoring de seguros) y clasificarlo en el AI Act con la función del ejercicio 1.
2. Producir una model card completa (intended use, training data summary, performance global y por grupo sensible, limitaciones, fecha, owner).
3. Ejecutar el DPIA checklist y listar las obligaciones GDPR aplicables al caso.
4. Implementar el right to be forgotten sobre un dataset de >10K filas y verificar que tras la deleción el usuario no aparece en ninguna columna (incluido `model.predict`).
5. Generar un compliance report Markdown con todas las secciones (riesgo AI Act, DPIA, model card, PII audit, human-in-the-loop).

Criterio de aceptación: el reporte ejecuta sin errores, identifica correctamente el nivel de riesgo del caso, lista al menos 5 obligaciones GDPR justificadas, y demuestra el borrado de un usuario con auditoría.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Tengo consentimiento, ya puedo hacer todo"	Consentimiento debe ser libre, específico,
Borrar al usuario de la tabla users pero s	El derecho al olvido aplica a todo el ecos
"Es solo un prototipo, GDPR no aplica"	GDPR aplica desde el primer dato personal
Clasificar un CV-screener como "riesgo lim	Anexo III lo lista explícitamente como alt
"Anonimicé los datos" pero quedan IPs y us	Pseudoanonimización ≠ anonimización. Si se
Modelo en producción sin human override pa	Art. 22 GDPR + Art. 14 AI Act exigen super

Preguntas frecuentes

Si mi empresa está fuera de la UE, ¿GDPR/AI Act aplican?

Sí, si tratás datos de personas en la UE (GDPR Art. 3, alcance extraterritorial) o si ofrecés un sistema de IA cuyo output se usa en la UE (AI Act Art. 2). Es el mismo principio del CCPA / LGPD.

¿GDPR me obliga a explicar mi modelo?

Hay debate. Wachter, Mittelstadt & Floridi (2017) argumentan que el Art. 22 + Recitales 71 garantizan información significativa sobre la lógica, no una explicación post-hoc tipo SHAP. En la práctica: documentación de la lógica + derecho a intervención humana + posibilidad de contestar la decisión.

¿Qué hago si entreno un LLM con datos scrapeados de internet?

Riesgo legal real (caso Clearview AI multado por la AEPD y otras DPAs). El AI Act prohíbe el scraping indiscriminado de imágenes faciales (Art. 5). Para texto, depende de base legal e intereses; minimizá, documentá fuentes, ofrecé opt-out.

¿Cuándo debo registrar un DPO (Data Protection Officer)?

Obligatorio si: autoridad pública, tratamiento a gran escala de categorías especiales, o monitoreo sistemático a gran escala (Art. 37). En la práctica, casi cualquier producto de ML con datos personales a escala lo requiere.

¿Cuándo entran en vigor las obligaciones del AI Act?

Escalonado: prohibiciones desde feb-2025, GPAI desde ago-2025, alto riesgo desde ago-2026/2027 según anexo. Sanciones empiezan a aplicarse con cada hito.

Referencias

- Reglamento UE 2016/679 (GDPR) — texto consolidado en EUR-Lex.
- Reglamento UE 2024/1689 (AI Act) — texto consolidado en EUR-Lex.
- European Data Protection Board — guidelines y decisiones.
- Council of Europe Framework Convention on AI (2024) — primer tratado internacional vinculante sobre IA.
- Wachter, S., Mittelstadt, B., Floridi, L. Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation (IDPL, 2017). <<https://academic.oup.com/idpl/article/7/2/76/3860948>>.
- Mitchell, M. et al. Model Cards for Model Reporting (FAT* 2019) — base de la model card que implementamos.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 228 — Clase 228 — Reproducibilidad: seeds, lock files, versionado de datasets

Parte: 7 — Ética, Fairness y Privacidad · Fuente: Pineau et al., Improving Reproducibility in ML Research (JMLR 2021) + Gebru et al., Datasheets for Datasets (CACM 2021) + Mitchell et al., Model Cards for Model Reporting (FAT 2019). Duración estimada: 75 min.*

Objetivo

Cerrar la Parte 7 con el problema que atraviesa todo lo anterior: si un experimento no es reproducible, no es auditable, no es comparable y no es ciencia. Aprender a controlar las tres fuentes de no-determinismo (código, datos, ambiente) con seeds, lock files, hashes de datasets, model cards y manifiestos de pipeline. Entender por qué Hutson (Science, 2018) habló de "crisis de reproducibilidad" en ML y qué piden hoy NeurIPS/JMLR como mínimo.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Sembrar correctamente random, numpy, sklearn (y comentar torch) con una función `seed_everything()` y `PYTHONHASHSEED`.
- Distinguir `requirements.txt` (top-level) de un lock file (`uv.lock`, `poetry.lock`, `conda-lock`) que pinea toda la transitive tree.
- Calcular un hash estable de un DataFrame (`sha256_of_df`) que sobreviva a reorden de columnas e índice y sirva como `dataset_id`.
- Producir un manifest JSON con `{data_hash, code_hash, seed, package_versions}` y validar reproducibilidad antes de re-entrenar.

- Redactar una model card mínima (intended use, training data hash, metrics, limitations) según Mitchell et al. 2019.

Temas

#	Tema	Por qué importa
1	Fuentes de no-determinismo	GPU atomicAdd, dict/set ordering, num_work
2	Seeds en stack Python	random.seed, np.random.default_rng, torch.
3	Lock files vs requirements	uv.lock/poetry.lock pinean toda la transit
4	Ambiente reproducible	Docker + base image pineada por digest (py
5	Versionado de datasets	DVC, Git LFS, lakeFS, S3 versioning, hash
6	Documentación: datasheets + model cards	Geburu 2018 (dataset) y Mitchell 2019 (mode

Definiciones y características

- Determinismo: misma entrada + mismo código + mismo ambiente → mismo output bit-a-bit. En ML rara vez se logra al 100% sobre GPU; se aspira a reproducibilidad estadística.
- Seed: entero que inicializa el estado de un PRNG. Cambiar el seed cambia la trayectoria — reportar un solo seed sin promediar varios es una mala práctica (Pineau 2021).
- PYTHONHASHSEED: variable de entorno que controla el hash de strings/objetos. Desde Python 3.3, por default es aleatorio por proceso — afecta orden de iteración de set/dict con keys hashables custom.
- Float associativity: $(a + b) + c \neq a + (b + c)$ en float32/64 por redondeo. Sumar en otro orden (BLAS multi-threaded, GPU reductions) da resultados que difieren en 1 ULP — suficiente para divergir tras muchas iteraciones.
- Lock file: archivo que pinea la versión exacta de cada dependencia transitiva con hash del wheel. requirements.txt con pandas==2.2.0 no pinea numpy, pytz, etc. — uv.lock sí.
- DVC (Data Version Control): git para datos. Guarda un .dvc pointer en git (hash MD5 + tamaño) y el blob real en storage remoto (S3/GCS). dvc pull re-hidrata.
- Datasheet for Datasets (Geburu et al. 2018): documento que acompaña al dataset con motivación, composición, recolección, uses, distribución. Equivalente al spec sheet de un componente electrónico.
- Model Card (Mitchell et al. 2019): documento que acompaña a un modelo con intended use, factores, métricas desagregadas, training/eval data, ethical considerations, caveats.

Dataset / recursos

- Dataset sintético generado in-notebook (no externo) — la clase es sobre el proceso, no sobre el dato.
- Librerías: stdlib (hashlib, json, importlib.metadata, os, random), numpy, pandas, scikit-learn.
- Opcional para homework: DVC, uv.

Ejercicios

1. Seed everything: implementar seed_everything(seed=42) que cubra random, numpy y PYTHONHASHSEED. Verificar que np.random.rand(5) da el mismo vector en dos corridas consecutivas.
2. Hash de DataFrame: escribir sha256_of_df(df) que ordene columnas alfabéticamente, resetee el índice y serialice a CSV bytes antes de hashear. Mostrar que dos df con columnas en distinto orden dan el mismo hash.
3. Manifest de experimento: dict con {data_hash, code_hash, seed, sklearn_version, numpy_version, pandas_version, python_version} serializado a experiment.json.
4. Validación de reproducibilidad: dado un manifest guardado, re-leer el dataset, recomputar su hash, comparar — abortar con RuntimeError si difiere.

5. Model card mínima: función `build_model_card(model, X, y, intended_use, limitations, date_trained)` que devuelve dict con campos de Mitchell et al. y lo dumpea a JSON.

Homework verificable

Notebook (o script) que:

1. Construya un pipeline `load → train → evaluate` con `seed_everything(42)` al inicio.
2. Calcule `dataset_hash` con `sha256_of_df` y `code_hash` con `sha256` sobre el `.py` (o el source string vía `inspect.getsource`).
3. Guarde `manifest.json` con `data_hash`, `code_hash`, `seed`, `package_versions` y `model_card.json` con métricas + `limitations`.
4. Re-corra el mismo pipeline en otra ejecución; verifique que `accuracy` reportada coincide bit-a-bit y que ambos hashes coinciden.
5. Cree `requirements.lock` con `uv pip compile requirements.in -o requirements.lock` (o `pip freeze > requirements.lock` como fallback) y committee ambos al repo.

Criterio de aceptación: en dos máquinas distintas (o dos venvs limpios) con `uv pip sync requirements.lock`, el pipeline produce el mismo `accuracy` y el mismo `dataset_hash`. El `manifest.json` está commiteado.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Resultados cambian entre corridas a pesar	Hay otra fuente: <code>random</code> <code>stdlib</code> sin <code>seed</code> ear
Hash del DataFrame cambia y los datos "son"	Estás hasheando <code>df.to_csv()</code> directo — incl
<code>requirements.txt</code> con <code>pandas==2.2.0</code> rompe e	No pinea <code>numpy</code> (transitive). Fix: usar <code>uv</code> .
Torch da resultados distintos en GPU con s	<code>atomicAdd</code> en GPU no es determinista. Fix p
Iteración sobre set da orden distinto cada	<code>PYTHONHASHSEED</code> aleatorio. Fix: setearla an
<code>sum(arr) ≠ np.sum(arr)</code> con <code>float32</code>	Float associativity + orden de reducción.

Preguntas frecuentes

¿Es realmente posible reproducir bit-a-bit un entrenamiento en GPU?

En general no sin esfuerzo. Hay ops no deterministas (`atomicAdd`, `cuDNN` convolutions con `autotuner`). Con `torch.use_deterministic_algorithms(True) + CUBLAS_WORKSPACE_CONFIG + num_workers=0 + cudnn.deterministic=True` te acercás, a costa de 1.5-3× más slow. En CPU es mucho más fácil.

¿`requirements.txt` no alcanza si pineo todo con `==`?

No del todo. Aunque pinees `pandas==2.2.0`, `pip` puede resolver `numpy` a una versión distinta entre máquinas si el resolver tiene libertad. Un lock file congela el resultado del resolver con hashes de wheels. `uv.lock`, `poetry.lock`, `Pipfile.lock`, `conda-lock` son equivalentes funcionales.

¿DVC, Git LFS o S3 versioning?

- DVC: si tu workflow es git-céntrico y querés `dvc repro` (pipelines con cache por inputs). Storage backend a elección.
- Git LFS: si los archivos son binarios chicos (<2GB) y querés transparencia total con git. No tiene pipelines.
- S3 versioning: si ya vivís en AWS y solo necesitás "ver versiones anteriores del objeto". Sin metadata de pipeline.
- lakeFS / Quilt / Pachyderm: para escala enterprise con branching real sobre data lakes.

¿Hashear el .csv original o el DataFrame cargado?

El DataFrame normalizado (columnas ordenadas, sin índice). El .csv puede tener BOM, line endings distintos (`\r\n` vs `\n`), encoding distinto y romper el hash sin que el contenido cambie. Hashear post-parsing es semánticamente correcto.

¿Una model card es un trámite burocrático?

No si la usás como gate de release. Auditorías regulatorias (EU AI Act 2024, NIST AI RMF) ya exigen documentación equivalente. Una model card bien hecha es además la entrada del próximo experimento: "el baseline al que tengo que ganarle es éste, entrenado con estos datos, evaluado así".

Referencias

- Pineau, J. et al. Improving Reproducibility in Machine Learning Research (JMLR 2021) — el NeurIPS Reproducibility Checklist.
- Gebru, T. et al. Datasheets for Datasets (CACM 2021).
- Mitchell, M. et al. Model Cards for Model Reporting (FAT* 2019).
- Hutson, M. Artificial intelligence faces reproducibility crisis (Science, 2018).
- PyTorch Reproducibility notes — qué controla cada flag.
- DVC docs · uv (lock files modernos).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Parte 8 — Parte 8 — Capstones — Proyectos Integradores

4 clases en esta parte.

Clase 229 — Clase 229 — Capstone 1: problema tabular end-to-end (EDA, modelo, API, dashboard)

Parte: 8 — Capstones · Fuente: integrador de Partes 0-7 + Huyen, C. Designing Machine Learning Systems (O'Reilly, 2022) caps. 4-7 + Géron, A. Hands-On ML, 3ª ed. caps. 2-3. Duración estimada: 180 min.

Objetivo

Integrar en un único proyecto entregable todo lo aprendido en Partes 0-7: cargar un dataset tabular real, hacer EDA, construir un pipeline ColumnTransformer reproducible, entrenar y tunear un modelo gradient-boosting con MLflow tracking, serializar el modelo, exponerlo vía FastAPI con Pydantic v2, construir un dashboard Streamlit con SHAP, y dejar todo bajo CI con GitHub Actions. La entrega debe poder reproducirse desde un clon limpio con `uv sync` y un `docker compose up`.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diseñar un pipeline ML end-to-end siguiendo el flujo de Huyen (Designing ML Systems): data → features → modelo → tracking → serving → monitoring.

- Tunear un gradient-boosting con Optuna (≥ 50 trials) y loguear cada run en MLflow con parámetros, métricas y artefactos.
- Exponer el modelo en FastAPI con schemas Pydantic v2 validados y latencia P95 < 200 ms.
- Construir un dashboard Streamlit con explicaciones SHAP por predicción + drift monitor (Evidently).
- Publicar una Model Card (Mitchell et al. 2019) con uso intencionado, métricas por subgrupo y limitaciones conocidas.

Fases del capstone

#	Fase	Duración	Entregable
1	EDA	25 min	notebooks/01_eda.ipynb + reporte ydata-pro
2	Feature engineering	25 min	src/features.py con ColumnTransformer repr
3	Modelo + tuning	40 min	src/train.py con Optuna 50 trials + MLflow
4	Tracking + Model Card	20 min	mlruns/ + MODEL_CARD.md
5	API FastAPI	35 min	src/api/main.py con /predict, /health, Ope
6	Dashboard + CI	35 min	app.py Streamlit con SHAP + .github/workfl

Definiciones y características

- Stack 2026 recomendado: uv (lock + venv) + Polars o pandas 2.x + scikit-learn 1.5+ + xgboost/lightgbm + optuna 3.x + mlflow 2.x + fastapi 0.115+ + pydantic 2.x + streamlit 1.x + shap + evidently 0.4+ + docker compose.
- Métricas clasificación: ROC-AUC (ranking), F1 al threshold elegido, log-loss (calibración), Brier score, PR-AUC si hay desbalance $>10:1$.
- Métricas regresión: RMSE, MAE, R^2 , MAPE (cuidado con $y \approx 0$).
- Threshold selection: el modelo devuelve probabilidad; el threshold se elige por F1, Youden's J, o coste-beneficio. No usar 0.5 por default.
- MLflow run: una ejecución de entrenamiento con params, metrics, tags y artifacts (modelo + plots). Optuna integra vía MLflowCallback.
- FastAPI schema: BaseModel Pydantic v2 con Field(..., ge=0, le=100) para validar rangos; OpenAPI auto-generado en /docs.
- Streamlit dashboard: app reactiva en Python puro; cachear cargas pesadas con @st.cache_resource.
- Model Card (Mitchell 2019): documento corto con uso intencionado, datos de entrenamiento, métricas por subgrupo, consideraciones éticas, limitaciones.
- CI smoke test: GitHub Actions corre pytest + levanta la API en background + hace curl /health → debe responder 200.

Dataset / recursos

- Dataset sugerido (elegí uno): UCI Adult (clasificación binaria, 48K filas, mix num+cat), Telco Customer Churn (Kaggle, 7K filas, churn binario), o Ames House Prices (Kaggle, 1.5K filas, regresión).
- Librerías: pandas o polars, scikit-learn, xgboost/lightgbm, optuna, mlflow, fastapi, uvicorn, pydantic, streamlit, shap, evidently, joblib, pytest, httpx.
- Infra local: Docker + compose.yml con 3 servicios (mlflow, api, streamlit).

Ejercicios

1. EDA: cargar dataset, generar ydata-profiling HTML, identificar missing, outliers, balance de clases. Documentar 5 hallazgos en notebooks/01_eda.ipynb.
2. Pipeline FE: armar ColumnTransformer con OneHotEncoder(handle_unknown='ignore') para cat, StandardScaler para num, SimpleImputer para missing. Persistir el preprocessor.

3. Modelo + Optuna: entrenar baseline (LogisticRegression) y challenger (XGBClassifier/LGBMClassifier). Tunear con Optuna 50 trials maximizando ROC-AUC sobre validación. Loguear cada trial en MLflow.
4. API: en `src/api/main.py`, definir `PredictRequest(BaseModel)` con todas las features tipadas y `PredictResponse(BaseModel)` con probability y prediction. Cargar modelo en lifespan. Endpoint `/predict + /health`.
5. Dashboard: `app.py` Streamlit con (a) form input → llama a la API y muestra predicción, (b) plot SHAP waterfall de la última predicción, (c) Evidently report comparando producción vs training data.

Homework verificable

Entregar un repo público en GitHub con:

1. Estructura `src/`, `notebooks/`, `tests/`, `MODEL_CARD.md`, `README.md`, `pyproject.toml` lockeado con uv.
2. MLflow tracking con ≥3 runs registradas (baseline + 2 challengers) y artefactos (modelo, plots de ROC, calibration curve).
3. FastAPI con `/predict` y `/health`, schemas Pydantic v2 validados, OpenAPI en `/docs`, latencia P95 < 200 ms medida con locust o httpx.
4. Dashboard Streamlit con SHAP plot por predicción y screenshot en el README.
5. Model Card completa (uso intencionado, métricas por subgrupo si aplica, limitaciones).
6. CI GitHub Actions corriendo pytest + smoke test contra `/health` con la API levantada en el runner. Badge verde en el README.

Criterio de aceptación: clonar el repo desde cero → uv sync → docker compose up → API responde a `/predict` con un request válido → dashboard Streamlit accesible en `localhost:8501` → CI verde en GitHub.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Modelo tunea AUC=0.99 en validación, 0.65	Data leakage — features computadas con inf
ROC-AUC perfecto pero el modelo es inútil	Target leakage — alguna feature es proxy d
422 Unprocessable Entity al llamar <code>/predic</code>	FastAPI sin schema Pydantic o tipos mal de
Latencia <code>/predict</code> > 1 segundo	Cargás el modelo en cada request. Fix: car
Streamlit re-entrena modelo en cada intera	No cacheaste. Fix: <code>@st.cache_resource</code> para
MLflow run sin artefactos, solo métricas	Olvidaste <code>mlflow.sklearn.log_model(model,</code>

Preguntas frecuentes

¿XGBoost o LightGBM?

Cualquiera. LightGBM suele ser 2-5× más rápido en entrenamiento sobre datasets >100K filas; XGBoost tiene mejor soporte en producción legacy. Para el capstone: elegí uno y justificalo en el README.

¿Necesito GPU?

No. Datasets tabulares <1M filas entrenan en CPU en minutos con XGBoost/LGBM. Reservá GPU para deep learning (Parte 7).

¿FastAPI o Flask?

FastAPI. Pydantic v2 da validación automática + OpenAPI gratis + async nativo. Flask sigue siendo válido pero pedís el doble de boilerplate.

¿Cómo pruebo la API en CI sin levantar Docker?

pytest + httpx.AsyncClient(app=app) — testea la app FastAPI in-process, sin red. Smoke test real con Docker corre en otro job del workflow.

¿La Model Card es opcional?

No. Es entregable obligatorio. 1-2 páginas Markdown con: propósito, datos, métricas por subgrupo (género/edad/región si aplica), limitaciones conocidas, contacto. Mitchell et al. 2019 tiene el template.

Referencias

- Huyen, C. Designing Machine Learning Systems (O'Reilly, 2022) — caps. 4 (Training Data), 5 (Feature Engineering), 7 (Model Deployment).
- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, 3ª ed. (O'Reilly, 2022) — caps. 2-3, el end-to-end project canónico.
- MLflow Tracking — runs, params, metrics, artifacts.
- FastAPI tutorial — Pydantic v2 + OpenAPI auto.
- Streamlit docs — cache_resource, cache_data, layout.
- Mitchell, M. et al. Model Cards for Model Reporting (FAT* 2019) — el template oficial.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 230 — Clase 230 — Capstone 2: NLP o series de tiempo end-to-end

Parte: 8 — Capstones · Fuente: Hyndman & Athanasopoulos, Forecasting: Principles and Practice (3ª ed.) + Hugging Face NLP course. Duración estimada: 180 min.

Objetivo

Entregar un segundo capstone end-to-end eligiendo una de dos ramas: (A) NLP — clasificación de texto / NER / RAG con transformers y endpoint FastAPI, o (B) series de tiempo — forecasting con baselines + modelos modernos, backtesting honesto e intervalos de predicción. En ambas ramas: tracking con MLflow, contenedorización con Docker, reproducibilidad con uv lock + seeds, Model Card, y discusión de drift (texto o forecast).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Elegir entre NLP o series de tiempo según interés/dominio y justificar la decisión en el README del proyecto.
- Construir un pipeline reproducible con split honesto (estratificado en NLP, temporal sin shuffle en series).
- Reportar métricas del dominio: F1/accuracy + slice analysis (NLP) o sMAPE/MASE + pinball loss (series).
- Servir el modelo en un endpoint FastAPI dockerizado, con MLflow tracking del entrenamiento.
- Detectar drift (Evidently text drift en NLP; KS sobre residuos en series) y documentarlo en la Model Card.

Fases del capstone

#	Fase	Entregable
---	------	------------

1	Definición + dataset + Model Card v0	README.md del proyecto con problema, métrica
2	EDA específico del dominio	NLP: distribución de longitudes, balance d
3	Split honesto + baselines	NLP: TF-IDF + Logistic / clase mayoritaria
4	Modelo moderno + MLflow tracking	NLP: fine-tune DistilBERT o sentence-trans
5	Backtesting + intervalos	NLP: slice analysis (longitud, idioma, cla
6	Empaquetado + endpoint + drift	Docker + FastAPI (/predict o /forecast) +

Definiciones y características

- sMAPE (symmetric MAPE): $\text{mean}(2 \cdot |y - \hat{y}| / (|y| + |\hat{y}|))$. Acotada en $[0, 2]$, no estalla cuando $y=0$ (a diferencia de MAPE).
- MASE (Mean Absolute Scaled Error): error escalado por el error del naive estacional in-sample. $\text{MASE} < 1$ mejor que naive; comparable entre series de distinta escala.
- Pinball loss (quantile loss): $\max(\tau \cdot (y - \hat{y}), (\tau - 1) \cdot (y - \hat{y}))$. Loss correcta para predicción de cuantiles (intervalos P10/P90).
- Backtesting expanding window: $\text{train}[0:T] \rightarrow \text{predict}[T:T+h]$, luego $\text{train}[0:T+h] \rightarrow \text{predict}[\dots]$, etc. No re-mezcla; respeta el orden temporal.
- Slice analysis (NLP): medir métricas por subconjunto (longitud del texto, idioma, clase) — un F1 global de 0.92 puede esconder $F1=0.40$ en textos largos.
- Fine-tuning ligero: entrenar solo la cabeza (head) de un transformer pre-entrenado o aplicar LoRA (adapters de bajo rango) $\rightarrow 10\text{--}100\times$ menos parámetros entrenables.
- RAG (Retrieval-Augmented Generation): embeddings $\rightarrow$ vector index (FAISS) $\rightarrow$ recuperar top-k $\rightarrow$ LLM responde con contexto. Reduce alucinaciones, no requiere fine-tuning.
- Drift de forecast: residuos $y - \hat{y}$ cuya distribución cambia en el tiempo — señal de que el modelo necesita re-entrenamiento.

Dataset / recursos

- Rama A — NLP: IMDB reviews (50K, binario), AG News (120K, 4 clases), o reseñas de Yelp en español. Para RAG: Wikipedia dump pequeño o docs internos.
- Rama B — Series: M5 (Walmart, jerárquica), electricity load (UCI), o clima diario (NOAA). Mínimo 2 años con estacionalidad clara.
- Stack común: uv (Parte 7), Docker, MLflow, FastAPI, Evidently. NLP extra: transformers, datasets, sentence-transformers, faiss-cpu. Series extra: statsmodels, statsforecast, darts o neuralprophet.

Ejercicios

1. Definición: escribir el problema en 5 líneas (qué se predice, para qué, métrica primaria, baseline aceptable, costo de error). Elegir rama A o B.
2. EDA del dominio: rama A $\rightarrow$ distribuciones de longitud y balance de clases por idioma; rama B $\rightarrow$ STL + ACF/PACF + test de estacionariedad ADF.
3. Baselines: rama A $\rightarrow$ TF-IDF + LogisticRegression; rama B $\rightarrow$ naive + seasonal naive + ETS. Loggear todo a MLflow.
4. Modelo moderno: rama A $\rightarrow$ fine-tune DistilBERT 2 epochs con transformers.Trainer; rama B $\rightarrow$ SARIMA o NeuralProphet con backtesting expanding window 5 folds.
5. Endpoint + drift: FastAPI con /predict (NLP) o /forecast?horizon=14 (series), dockerizado. Reporte de drift entre primera y segunda mitad del test (Evidently o KS test manual).

Homework verificable

Repositorio con:

1. README.md del proyecto con problema, rama elegida (A o B), métrica primaria y resultado.
2. Notebook de EDA + entrenamiento con seeds fijas (42) y MLflow tracking visible (screenshot o mlruns/ versionado).
3. Dockerfile + compose.yml que levanten FastAPI + MLflow UI con un solo docker compose up.
4. Endpoint funcional: `curl -X POST localhost:8000/predict (NLP) o curl localhost:8000/forecast?horizon=14 (series)` devuelve JSON válido.
5. Model Card (1 página) con métricas globales y por slice/horizonte, datos de entrenamiento, sesgos conocidos, y plan de monitoreo de drift.

Criterio de aceptación: el modelo moderno supera al mejor baseline en la métrica primaria (NLP: $\geq +3$ puntos de F1; series: $MASE < 1.0$ y $sMAPE$ menor que seasonal naive), el endpoint responde en <500 ms, y la Model Card identifica al menos un slice/horizonte donde el modelo es débil.

Errores comunes

Síntoma / mens	Causa y cómo						
$sMAPE = 200\%$	$y=0$ con $\hat{y}>0$. Fix $y-\hat{y}$	/	(	y	+	$\hat{y}$	$+\epsilon)$, o cambiar a MASE.
Modelo de forec	Hiciste train_test						
transformers no	Falta caché o no						
F1 global alto pe	No hiciste slice a						
Intervalos P10/P	Modelo subestini						
OOM al fine-tune	Batch size dema						

Preguntas frecuentes

¿NLP o series? ¿Cuál es más fácil?

Series suele ser más rápido de prototipar (statsforecast entrena SARIMA sobre miles de series en minutos, sin GPU). NLP con transformers requiere GPU para fine-tuning serio, o aceptar fine-tuning lento en CPU con DistilBERT y `max_length=128`.

¿Puedo usar GPT-4/Claude vía API en vez de fine-tunear?

Sí — es válido como baseline en RAG o clasificación zero-shot, pero el capstone pide al menos un modelo propio entrenado y versionado en MLflow. La llamada a API puede ser el comparativo.

¿Por qué MASE además de sMAPE?

Porque MASE es comparable entre series (escala-invariante vs el naive estacional). Si reportás solo sMAPE, no podés saber si 12% es bueno o malo sin contexto. $MASE < 1$ mejor que naive, en cualquier serie.

¿FastAPI o BentoML?

FastAPI por default (ya cubierto en Parte 4). BentoML si necesitás batch inference automática, model registry integrado, o despliegue a SageMaker/Vertex con menos código.

¿La Model Card es realmente necesaria?

Sí — Mitchell et al. (Google, 2019) la propusieron por una razón: sin ella, nadie sabe qué slices son débiles ni cuándo el modelo se rompe. Es lo primero que un evaluador externo (o auditor) pide.

Referencias

- Hyndman, R., Athanasopoulos, G. Forecasting: Principles and Practice (3ª ed., 2021) — <https://otexts.com/fpp3/>

- Hugging Face NLP course — <https://huggingface.co/learn/nlp-course>
- Hvitfeldt, E., Silge, J. Supervised ML for Text Analysis (2021) — <https://smltar.com/> (R, conceptos aplican)
- Statsforecast docs — <https://nixtlaverse.nixtla.io/statsforecast/index.html>
- Darts (forecasting deep + clásico) — <https://unit8co.github.io/darts/>
- Mitchell, M. et al. Model Cards for Model Reporting (FAT* 2019).

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 231 — Clase 231 — Capstone 3: visión por computadora con transfer learning

Parte: 8 — Capstones · Fuente: Géron, Hands-On ML 3ª ed. cap. 14-15 + timm + PyTorch Lightning docs. Duración estimada: 180 min.

Objetivo

Construir un clasificador de imágenes de calidad producción usando transfer learning sobre un backbone moderno (ConvNeXt-tiny / EfficientNetV2-S / ViT-Base/16). Entrenar en dos fases (feature extraction → fine-tuning progresivo), aplicar augmentation moderna (RandAugment, MixUp, CutMix), evaluar con métricas per-clase + slice analysis, y servir vía ONNX + FastAPI con endpoint /predict que recibe imagen en base64. Cerrar el capstone con un fairness check si aplica.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diseñar un pipeline de transfer learning completo: backbone preentrenado en ImageNet → head custom → fine-tuning progresivo con LR diferencial por grupo.
- Aplicar augmentation moderna con Albumentations (RandAugment, MixUp, CutMix, RandomErasing) y verificar invariancia de label.
- Entrenar con PyTorch Lightning + AMP (mixed precision) + torch.compile + grad accumulation, con seed_everything y deterministic algorithms.
- Reportar accuracy + per-class F1 + confusion matrix + slice analysis (Clase 169) y un fairness check (Clase 224) si el dataset tiene atributos sensibles.
- Exportar el modelo a ONNX o TorchScript y servirlo en un endpoint FastAPI /predict que reciba imagen base64.

Fases del capstone

#	Fase	Por qué importa
1	Dataset + EDA	Class imbalance, resolución variable, leak
2	Augmentation	RandAugment + MixUp suben 2-4 puntos de ac
3	Backbone preentrenado	ConvNeXt-tiny / EffNetV2-S / ViT-B/16 vía
4	Fine-tuning progresivo	Feature extraction (head only) → unfreeze
5	Evaluación	Accuracy global engaña con imbalance; per-
6	Serving	Export a ONNX / TorchScript + FastAPI /pre

Definiciones y características

- Transfer learning: reutilizar pesos de un modelo entrenado en una tarea grande (ImageNet, 1.2M imágenes, 1000 clases) como punto de partida para tu tarea chica. Dos modos: feature extraction (congelar backbone, entrenar solo head) y fine-tuning (descongelar progresivamente).
- Backbone moderno (2026): ConvNeXt-tiny (28M params, CNN moderna), EfficientNetV2-S (21M, balance speed/accuracy), ViT-Base/16 (86M, attention puro). Todos disponibles en timm con pesos ImageNet-21k.
- LR diferencial por grupo: el backbone preentrenado necesita LR $\sim 100\times$ menor que el head random. `torch.optim.AdamW({'params': backbone, 'lr': 1e-5}, {'params': head, 'lr': 1e-3})`.
- RandAugment: política automática que aplica $N=2$ transformaciones random con magnitud $M=9$ (rotación, color jitter, equalize, shear...). Plug-and-play en `torchvision.transforms.v2`.
- MixUp / CutMix: combinan 2 imágenes (MixUp = blend pixel-wise $\alpha \approx 0.2$; CutMix = pega un parche de B sobre A). Labels se mezclan en la misma proporción. Regularizador potente para datasets chicos.
- AMP (Automatic Mixed Precision): usar float16 para forward/backward, float32 para pesos. $2\times$ speedup y $2\times$ memoria libre en GPUs $\geq$ Volta. `torch.amp.autocast()` + GradScaler.
- `torch.compile`: en torch 2.x, JIT-compila el grafo del modelo. $+20-40\%$ throughput sin tocar código. `model = torch.compile(model, mode="reduce-overhead")`.
- Slice analysis: accuracy/F1 desagregado por sub-grupo (tamaño del objeto, iluminación, demografía). Revela disparidades que la métrica global esconde (Clase 169 + 224).

Dataset / recursos

- Opciones de dataset (elegí una):
- Cats vs Dogs (Kaggle, ~ 800 MB, 25K imágenes, binario) — el clásico para empezar.
- Food-101 subset (10 clases de 101, ~ 500 MB) — multiclase realista, fondos heterogéneos.
- Plant Disease (PlantVillage, 38 clases, ~ 1.5 GB) — class imbalance natural, alto impacto agro.
- Casting Defect (Kaggle industrial, binario, ~ 100 MB) — defect detection, dataset chico ($\sim 7K$) ideal para mostrar el valor del transfer learning.
- Stack: torch 2.x · torchvision · timm · pytorch-lightning · albumentations · onnx · onnxruntime · fastapi · uvicorn · wandb o mlflow.

Ejercicios

1. Baseline tonto: entrenar una CNN de 2 capas conv from scratch (sin transfer). Reportar accuracy de validation. Esperá $<60\%$ en multiclase — establece el piso.
2. Feature extraction: cargar `timm.create_model('convnext_tiny', pretrained=True, num_classes=N)`. Congelar backbone (for `p in model.parameters(): p.requires_grad = False`), entrenar solo head 5 epochs. Esperá $+20-30$ puntos sobre el baseline.
3. Fine-tuning progresivo: unfreeze último bloque $\rightarrow$ 5 epochs con LR $1e-4 \rightarrow$ unfreeze full $\rightarrow$ 10 epochs con LR diferencial ($1e-5$ backbone, $1e-3$ head). Loggear con W&B/MLflow.
4. Augmentation ablation: comparar (a) sin aug, (b) flip + crop, (c) RandAugment, (d) RandAugment + MixUp + CutMix. Reportar curva `val_acc`.
5. Serving end-to-end: exportar a ONNX (`torch.onnx.export`), validar con onnxruntime que la inferencia da la misma probabilidad $\pm 1e-4$, levantar FastAPI con endpoint POST `/predict` que reciba `{"image_b64": "..."} y devuelva {"class": "...", "prob": 0.94}`. Probar con curl.

Homework verificable

Notebook + repo con:

1. Dataset elegido + EDA (distribución de clases, resolución, ejemplos por clase).
2. Pipeline Lightning con 3 fases de entrenamiento (feature extraction $\rightarrow$ unfreeze parcial $\rightarrow$ unfreeze full).

3. Augmentation con Albumentations + verificación de label invariance.
4. Reporte de métricas: accuracy global, per-class F1, confusion matrix, slice analysis sobre al menos 1 dimensión.
5. Si el dataset tiene atributos sensibles (rostros, demografía): fairness check con disparate impact + equal opportunity (Clase 224).
6. Export a ONNX + script serve.py con FastAPI /predict funcionando localmente.
7. README del repo con resultados, decisiones de diseño y limitaciones.

Criterio de aceptación: accuracy de test > 90% en binario o > 75% en multiclase (10+ clases), el endpoint /predict responde en <500 ms en CPU, y el reporte identifica al menos 1 slice donde el modelo subperforma.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Accuracy de train sube a 99%, val se queda	Overfitting clásico. Fix: subir augmentati
CUDA out of memory al hacer fine-tune full	Batch demasiado grande para AMP off. Fix:
Val accuracy es altísima pero el modelo en	Leakage train/test (mismas imágenes duplic
RuntimeError: shape mismatch al exportar a	El head custom espera shape que el dummy_i
Predicciones ONNX difieren de PyTorch en 5	Esperable (precisión float). Fix: validar
Una clase tiene F1=0.0 y nadie lo notó	Class imbalance (esa clase es 2% del datas

Preguntas frecuentes

¿ConvNeXt, EfficientNetV2 o ViT?

Para datasets chicos (<10K imágenes), ConvNeXt-tiny o EfficientNetV2-S ganan: las CNN tienen inductive bias (locality, translation equivariance) que ViT no tiene y por eso ViT necesita 10× más datos para igualar. Para datasets grandes (>100K) o si tenés pretraining en ImageNet-21k, ViT-Base/16 suele ser top. Probá los 3 vía timm y comparalos — son 3 líneas de código cada uno.

¿Feature extraction o fine-tuning?

Feature extraction primero (más rápido, menos riesgo de catastrophic forgetting). Si la accuracy se estanca, hacer fine-tuning progresivo: descongelar el último bloque, después dos, después full, siempre con LR diferencial. Nunca descongelés todo en epoch 1 — el gradiente random del head corrompe los pesos preentrenados.

¿Por qué Lightning y no PyTorch puro?

Lightning te da gratis: AMP, multi-GPU, gradient accumulation, checkpointing, EarlyStopping, logging a W&B/MLflow, y seed_everything. En un capstone querés invertir el tiempo en el modelo, no en el training loop.

¿ONNX o TorchScript para serving?

ONNX si querés portabilidad (servir desde C++, Java, Node, navegador con onnxruntime-web). TorchScript si te quedás en Python/C++ con torch instalado y querés el path más simple. Para FastAPI en producción, ONNX + onnxruntime suele ser 2-3× más rápido que torch eager.

¿Fairness check siempre?

Solo si el dataset tiene atributos sensibles (rostros con edad/género/etnia, datos médicos con demografía). Si clasificás defectos industriales o platos de comida, no aplica. Cuando aplica, es obligatorio — releé Clase 224 antes de publicar el modelo.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow 3ª ed. (O'Reilly, 2022), cap. 14 (CNN) y 15 (transfer learning).
- timm — PyTorch Image Models (Ross Wightman) — 1000+ backbones preentrenados con API única.
- PyTorch Lightning docs — el training loop estándar de facto.
- Albumentations docs — augmentation rápida y composable.
- He, K. et al. Deep Residual Learning for Image Recognition (CVPR 2016). <<https://arxiv.org/abs/1512.03385>>
- Dosovitskiy, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ICLR 2021, ViT). <<https://arxiv.org/abs/2010.11929>>

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Clase 232 — Clase 232 — Portafolio público en GitHub Pages y presentación

Parte: 8 — Capstones · Fuentes: Chip Huyen, How to build a data science portfolio + Eugene Yan, What I learned from looking at 200 ML portfolios + Mitchell et al., Model Cards for Model Reporting (FAT 2019). Duración estimada: 150 min.*

Objetivo

Empaquetar los tres capstones (229, 230, 231) en un portafolio público que un reclutador entienda en 30 segundos: sitio en GitHub Pages, demos hosted, blog técnico, deck de presentación y CV. La regla es calidad > cantidad: 3 proyectos terminados con métricas honestas valen más que 10 a medias.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Estructurar un sitio de portafolio con MkDocs Material o Quarto desplegado en GitHub Pages.
- Escribir un README de proyecto que comunica problema, approach, métricas y limitaciones sin marketing vacío.
- Publicar demos hosted (Streamlit Community Cloud, HuggingFace Spaces) y un blog técnico por capstone.
- Armar un deck de 10-15 slides que cuenta cada proyecto en formato problema → datos → enfoque → resultado → trade-offs.
- Detectar y evitar los antipatterns clásicos (10 proyectos a medias, README sin números, demo rota).

Componentes del portafolio

#	Componente	Para qué sirve
1	README pulido por proyecto	Es la primera impresión. Badge de CI, hero
2	Demos hosted (Streamlit/HF Spaces)	El reclutador clickea y prueba en 10 seg.
3	Blog técnico (1 post por capstone)	Demuestra que sabés comunicar además de co
4	Presentación (deck 10-15 slides)	Para entrevistas técnicas y meetups. Reusa
5	Video demo (2-3 min)	Loom o screencast. Único formato consumibl
6	CV técnico (1 página)	Verbo de impacto + número medible. Linkea

Definiciones y características

- Portafolio: sitio público (no PDF) que centraliza proyectos, blog, CV y links de contacto. Vive en `username.github.io` o subdominio propio.
- GitHub Pages: hosting estático gratis de GitHub. Build con Jekyll por defecto, pero podés deployar cualquier sitio estático (MkDocs, Quarto, Docusaurus, Hugo).
- MkDocs Material: generador estático en Python; búsqueda built-in, dark mode, navegación lateral. Lo más rápido de levantar.
- Quarto: generador científico (Posit/RStudio); ejecuta notebooks `.qmd/.ipynb` y renderiza con bibliografía BibTeX y matemáticas LaTeX. Ideal si tu blog tiene mucho código + plots.
- Model Card (Mitchell et al., 2019): ficha estándar de un modelo — datos de entrenamiento, métricas por subgrupo, casos de uso previstos y NO previstos, riesgos éticos.
- Hero image / GIF: imagen o screencast al inicio del README que muestra el output en 1 vistazo. Sin hero, nadie scrollea.
- Quick start de 30 segundos: bloque bash con 3-5 líneas para correr el proyecto local. Si no entra en 30 seg, perdiste al reviewer.
- Reproducibilidad (Parte 7): lock file + Dockerfile + CI verde + dataset documentado. Es el diferenciador entre "proyecto de tutorial" y "proyecto serio".

Recursos

- Herramientas: MkDocs Material, Quarto, Streamlit Community Cloud, HuggingFace Spaces, Render, Vercel.
- Templates: `mkdocs-material starter`, Quarto blog template.
- Inspiración: portfolios de Eugene Yan, Chip Huyen, Vicki Boykis, Hamel Husain — todos en `eugeneyan.com`, `huyenchip.com`, etc.

Ejercicios

1. MkDocs Material setup: `pip install mkdocs-material`, `mkdocs new portfolio`, editar `mkdocs.yml` con tema material y deployar a gh-pages con `mkdocs gh-deploy`. Verificar que el sitio carga en `https://<user>.github.io/portfolio`.
2. README de capstone: tomar el capstone 229 (NLP) y reescribir el README con hero image, badges (CI, license, Python version), quick start de 30 seg, sección "decisiones técnicas" (3 bullets) y "limitaciones" (3 bullets).
3. Demo hosted: subir el capstone 230 (CV) a HuggingFace Spaces con Gradio. URL pública compartible.
4. Blog post técnico: escribir 1 post de 800-1500 palabras sobre el capstone 231 (tabular) siguiendo el outline problema → datos → approach → resultado con un plot → trade-offs → próximos pasos.
5. Deck: armar 10-15 slides (Google Slides, Marp o reveal.js) presentando los 3 capstones con la regla de "1 idea por slide".

Homework verificable

Sitio en GitHub Pages con:

1. Index con foto, bio de 3 líneas, links a 3 proyectos, link a blog, link a CV PDF, link a LinkedIn/GitHub.
2. Una página por proyecto (3 capstones) con README + Model Card + link a demo + link a blog post + link a repo.
3. Blog con al menos 1 post técnico publicado.
4. CV PDF de 1 página linkeado desde el index.
5. Demos: al menos 1 de los 3 proyectos con demo hosted funcionando (Streamlit Cloud o HF Spaces).

Criterio de aceptación: un reclutador no técnico abre el sitio, en 30 segundos sabe (a) tu nombre, (b) qué hacés, (c) ve un proyecto con métricas, (d) puede clickear una demo que funciona. Si falla algún paso → el portafolio no aprueba.

Errores comunes

Síntoma	Causa y cómo arreglar
README dice "implementé un modelo de ML pa	Cero métricas, cero credibilidad. Fix: pon
10 proyectos en el portafolio, todos a 30%	Cantidad sin profundidad. Fix: borrar 7, d
Demo rota (link a Streamlit que duerme y n	Streamlit free duerme tras 7 días sin uso,
Hero image inexistente — README es solo te	Sin imagen no scrollean. Fix: GIF de 5 seg
requirements.txt con pandas sin versión	Imposible reproducir. Fix: lock file (uv.l
CV de 3 páginas con responsabilidades gené	Nadie lee la pág 2. Fix: 1 página, verbos

Preguntas frecuentes

¿MkDocs Material o Quarto?

MkDocs Material si el sitio es documentación + landing. Quarto si el sitio tiene mucho notebook ejecutable y math (papers, posts técnicos largos con BibTeX). Ambos deployan a GitHub Pages igual.

¿Necesito dominio propio?

No para empezar. username.github.io funciona perfecto. Comprá dominio (tunombre.dev, ~12 USD/año) cuando el portafolio esté en versión estable y quieras una URL más memorable.

¿Y si no tengo experiencia laboral todavía?

El portafolio reemplaza la falta de experiencia laboral. 3 capstones con métricas, demo y blog post pesan más en una entrevista junior que "tomé 5 cursos en Coursera".

¿Subo el código del homework de las clases?

No al portafolio principal. Mantenelos en un repo aparte (python-data-science-program-homework). El portafolio es solo para proyectos pulidos.

¿Cuánto tiempo dedicar al portafolio antes de aplicar a trabajos?

Regla práctica: 1 semana de pulido full-time tras terminar el capstone 231. Pasada esa semana, aplicás aunque no sea perfecto — el portafolio se itera con feedback real de entrevistas.

Referencias

- Chip Huyen, How to build a data science portfolio — huyenchip.com.
- Eugene Yan, What I learned from looking at 200 ML portfolios — eugeneyan.com.
- MkDocs Material — documentación oficial.
- Quarto — sitio oficial, especialmente Websites y Blogs.
- HuggingFace Spaces — hosting gratis para demos.
- Mitchell, M. et al. Model Cards for Model Reporting (FAT* 2019) — arxiv:1810.03993.

Cierre del programa

Llegaste a la clase 232 de 232. Empezaste con print("hola mundo") y terminás con un portafolio público, tres capstones con métricas honestas, demos hosted y un blog que explica cómo pensás. El programa termina acá; tu trabajo como practicante de data science recién arranca. Lo que sigue no es otro curso — es elegir

una vertical (research, applied science, MLE, product DS), buscar a la gente que está haciendo el trabajo que querés hacer, y empezar a contribuir: issues en open source, posts en tu blog, charlas en meetups, un proyecto nuevo cada trimestre. La ventaja competitiva no es saber más algoritmos: es enviar cosas terminadas, medibles y reproducibles. Eso ya lo sabés hacer.

Gracias por llegar hasta acá. Ahora andá a romperla.

[Volver al índice general](#) · [Índice de Parte 8](#)

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Fin del programa

Bundle unificado generado desde 232 clases en 9 partes.